深度学习的数学基础

叶筱倞

2026 年 3 月 25 日[1]

[1]此书稿为 Mathematical Foundations of Deep Learning（https://arxiv.org/abs/2603.18387）的简体中文翻译版。此书正式版收于 “Chapman & Hall/CRC Mathematics and Artificial Intelligence Series” 系列，由 Taylor & Francis 公司出版。此书稿不可作商业用途。

```
@book{ye2026mathematical,
  title = {Mathematical Foundations of Deep Learning},
  author = {Ye, Xiaojing},
  year = {2026},
  publisher = {Chapman and Hall/CRC}
}
```

目 录

序 **vii**

第一章 深度神经网络 **1**

1.1 函数逼近 1

1.2 浅度和深度神经网络 7

1.3 普适逼近定理 10

1.4 普适逼近定理的证明 12

1.5 网络结构设计 24

1.5.1 激活函数示例 24

1.5.2 网络模块示例 28

1.5.3 具有特殊性质的网络 34

1.5.4 网络训练准则 38

1.6 备注与参考文献 42

1.6.1 关于网络结构设计的备注 42

1.6.2 普适逼近理论的文献 42

第二章 网络训练 **45**

2.1 优化中的基本概念 46

2.2 最优性条件 49

2.2.1 最优性条件的一般原理 51

2.2.2 必要最优性条件 52

2.2.3 充分最优性条件 59

2.3 自动微分 62

2.3.1 一阶导数的计算 64

2.3.2 二阶导数的计算 68

2.4 确定性优化算法 71

2.4.1 无约束优化算法 73
2.4.2 约束优化算法 84
2.5 随机优化算法 98
2.5.1 随机优化的基础 98
2.5.2 实用随机优化算法 110
2.6 备注与参考文献 116
2.6.1 关于网络训练优化的备注 116
2.6.2 数值优化的参考文献 118

第三章 深度最优控制 119
3.1 最优控制理论 119
3.1.1 Euler–Lagrange 方程 120
3.1.2 Hamiltonian 动力学 123
3.1.3 Pontryagin 极大值原理 127
3.1.4 Hamilton–Jacobi–Bellman 方程 146
3.2 最优概率密度控制理论 149
3.2.1 概率控制的极大值原理 152
3.2.2 概率控制的 HJB 方程 158
3.2.3 初值扰动的变分 161
3.3 Neural ODE 方法 164
3.4 备注与参考文献 172
3.4.1 在 PDE 解算子学习中的应用 172
3.4.2 最优控制的参考文献 178

第四章 深度强化学习 181
4.1 强化学习基础 181
4.1.1 强化学习中的基本概念 181
4.1.2 Markov 决策过程 182
4.1.3 Bellman 算子与 Bellman 方程 184
4.1.4 策略最优性与改进理论 193
4.2 强化学习算法 198
4.2.1 基于模型的强化学习算法 198
4.2.2 无模型强化学习算法 204
4.3 备注与参考文献 222
4.3.1 强化学习与最优控制 222

4.3.2 强化学习的文献 223

第五章 生成模型 225

5.1 变分自编码器 226
5.2 生成对抗网络 233
5.3 扩散模型 237
5.4 概率密度控制 250
5.5 流匹配 253
5.6 备注与参考文献 259
5.6.1 生成模型中的噪声采样 259
5.6.2 条件生成模型 263
5.6.3 生成模型的参考文献 266

附录 A 补充材料 269

A.1 Monte Carlo 积分 269
A.2 Banach 空间与不动点理论 271
A.3 信息论基础 275
A.4 随机微分方程基础 284

序

Over the past fifteen years, deep learning has rapidly evolved into one of the central pillars of modern science and engineering, and it is now widely regarded as the most promising approach toward general artificial intelligence. Despite its remarkable empirical success, theoretical understanding has lagged far behind practical advances for much of this period. This book is based on a collection of literature and notes I took in the past years. It is written in response to a growing need, particularly from people interested in learning and understanding deep learning from mathematical perspective: A relatively coherent, principled, and mathematically grounded account of deep learning that focuses on the underlying structures, assumptions, and theoretical guarantees of deep learning models.

At its core, deep learning is a mathematical enterprise. Neural networks are function approximators; training corresponds to solving large-scale nonconvex optimization problems; generalization is fundamentally a question of probability and statistics; and expressivity, stability, and robustness are governed by ideas from approximation theory, functional analysis, and dynamical systems. The goal of this book is to provide some mathematical foundations needed to understand why deep neural networks work, how to train them effectively, and how to leverage deep learning to solve problems arising from machine learning, scientific computing, automatic control, etc.

I hope this book can be beneficial to people in several fields. For readers with a background in mathematics, it provides an entry point into deep learning that respects rigor and abstraction while remaining closely connected to modern practice. For readers trained primarily in machine learning or engineering, it offers a deeper theoretical perspective, unifying familiar concepts under precise mathematical formulations. Throughout the book, I tend to emphasize clarity of assumptions, careful definitions, and explicit connections between theory and algorithms.

The materials in this book are organized around several foundational themes. This book begins by viewing neural networks as parametric function classes and studying their expressive power through the lens of approximation theory. Then the discussion focuses on optimization theory and numerical algorithms, which underpin the training of deep neural networks. Subsequent chapters explore the integration of deep learning with optimal control, rein-

forcement learning, and generative modeling, highlighting the mathematical principles that govern their design and behavior. Wherever possible, connections are drawn between classical results and contemporary developments.

This book is definitely not exhaustive nor intended to be a comprehensive survey. Instead, it should be considered as an introduction to deep learning through a mathematics-oriented approach. It emphasizes ideas, concepts, and results that are likely to remain relevant as the field continues to evolve extremely fast. Proofs are provided for central results, and examples and remarks are included to build intuition and to illustrate how abstract theory informs practical modeling choices.

The intended audience includes graduate students (and advanced undergraduates) and researchers in mathematics, statistics, computer science, electrical engineering, and related fields. A solid background in the standard sequence of calculus courses, linear algebra, and probability is required, while familiarity with advanced calculus and real analysis will be very helpful.

Deep learning continues to reshape both applied sciences and theoretical inquiry. By presenting from a mathematical perspective, I hope this book will help readers develop a deeper understanding of this subject, critically assess emerging ideas, and contribute to the ongoing effort to place deep learning on firm theoretical ground.

I wish to thank Yunmei Chen, Hongyuan Zha, and Haomin Zhou for many years of fruitful collaboration and insightful discussions in the field of mathematics and deep learning, and also acknowledges my excellent students and collaborators, including Nathan Gaby, Shu Liu, Hao Wu, Yaohua Zang, and many others, for the valuable discussions and joint work these years. I am grateful to National Science Foundation for the support of my research.

Xiaojing Ye
Atlanta, Georgia, USA

常用记号

$A := B$	将 A 定义为 B（或 $B =: A$）
s.t.	满足
$\mathbb{N}$	自然数集合 $\{1, 2, \dots\}$
$\mathbb{R}$	实数集合
$\mathbb{R}^n$	n 维实欧几里得空间
$[N]$	对任意 $N \in \mathbb{N}$，表示 $\{1, 2, \dots, N\}$
I_d	$d \times d$ 单位矩阵
$A^\top$	（复）矩阵 A 的（共轭）转置
$\mathrm{tr}(A)$	方阵 A 的迹
$A \succeq B$	$A - B$ 是半正定的（若为 $\succ$，则表示正定）
$\lvert \cdot \rvert$	绝对值，或向量与矩阵的 2-范数
$\lVert \cdot \rVert_*$	由下标 $*$ 指定的函数范数
$x \odot y$	$x, y \in \mathbb{R}^n$ 的逐分量乘积
$\langle x, y \rangle$	$\mathbb{R}^n$ 或 Hilbert 空间中 x 与 y 的内积
$x'(t)$ or $\dot{x}(t)$	函数 $x(t)$ 关于 t 的导数
∇_θ	关于 θ 的梯度（简记 $\nabla = \nabla_x$）
$\nabla\cdot$	关于 x 的散度
∇^2	关于 x 的 Hessian 矩阵
Δ	关于 x 的 Laplace 算子
C^k	k 次连续可微函数的集合
$\arg\min$ ($\arg\max$)	极小值点（极大值点）的集合
$X \sim p$	X 是服从概率分布 p 的随机变量
$x \sim p$	x 是从概率分布 p 中抽取的样本
i.i.d.	独立同分布
$\mathcal{N}(\mu, \Sigma)$	均值为 μ、方差为 Σ 的高斯分布
$\mathcal{N}(\cdot; \mu, \Sigma)$	$\mathcal{N}(\mu; \Sigma)$ 的概率密度函数
$\mathbb{E}[f(X)]$	$f(X)$ 的期望
$\mathbb{E}_{X \sim p(\cdot \mid Y)}[f(X, Y)]$	给定 Y 时 $f(X, Y)$ 的条件期望
$\mathrm{Var}[f(X)]$	$f(X)$ 的方差

第一章　深度神经网络

深度神经网络（deep neural network）是现代深度学习（deep learning）系统的核心构建模块。从数学角度来看，深度神经网络是一种有高度表达能力的函数逼近器（function approximator）。人们受到生物神经系统分层组织的启发而构造了深度神经网络，并发现它们确实可以有效地通过层次化来处理和表示数据。

本章首先介绍深度神经网络的动机和基本概念。随后我们将重点讨论神经网络理论中最基本的结果之一，即普适逼近定理（Universal Approximation Theorem），并在逼近定义于欧几里得空间紧子集上的一类连续函数时，给出带有定量的误差估计的严格证明。

之后，我们会回顾常用的激活函数和构成现代深度神经网络架构的核心模块。这一部分是为缺乏丰富实践经验的读者提供一些深度网络构建的直观理解。更重要的是，我们将通过一些示例，展示如何构造满足特定函数性质的神经网络。这类构造技巧通常能够显著提升网络训练效率和实际性能。最后，我们简要讨论一下训练网络用的目标函数，并给出一些启发性示例，说明我们有时只基于物理定律也能构建它们，不必依赖外部提供的数据。

1.1　函数逼近

函数逼近器在统计学和数据科学中被广泛使用。我们首先回顾统计学中用于线性回归和逻辑分类的两个经典函数逼近器示例。

例 1.1.1 (线性回归，linear regression)**.** 假设有 d 个变量 $x_1, \ldots, x_d \in \mathbb{R}$，称为**特征**（feature），它们组合形成一个**特征向量** $x = (x_1, \ldots, x_d)^\top \in \mathbb{R}^d$。此外，有一个观测值 $y \in \mathbb{R}$，已知它是这些变量的一个仿射函数（affine function）再加上一些噪声：

$$y = \sum_{i=1}^{d} w_i x_i + b + e, \tag{1.1}$$

其中 $w=(w_1,\ldots,w_d)\in\mathbb{R}^d$ 和 b 分别称为**权重**（weight）和**偏置**（bias），它们是未知的。在 (1.1) 中，我们假设噪声 e 服从均值为 0、标准差为未知量 s 的高斯分布 $\mathcal{N}(0,s^2)$。也就是说，e 的概率分布为

$$\mathcal{N}(e;0,s^2):=\frac{1}{\sqrt{2\pi s^2}}e^{-\frac{|e|^2}{2s^2}}.$$

问题是：如何利用样本 (x,y) 以及 (1.1) 中给出的关系来估计 (w,b)?

显然，我们无法从单个样本 (x,y) 中估计 (w,b)。实际中，我们需要一个包含多个（例如 M 个）样本的数据集 $\mathcal{D}$：

$$\mathcal{D}=\Big\{(x^{(j)},y^{(j)})\in\mathbb{R}^d\times\mathbb{R}:\ j\in[M]\Big\},$$

其中每个样本 $(x^{(j)},y^{(j)})$ 都满足 (1.1) 中的相同仿射关系，并且噪声 $e^{(j)}\sim\mathcal{N}(0,s^2)$ 相互独立同分布（i.i.d.），$j\in[M]:=\{1,2,\ldots,M\}$。

为简化记号，我们用 θ 表示 w 和 b 拼接而成的 $(d+1)$ 维列向量：

$$\theta:=\begin{pmatrix}w\\b\end{pmatrix}\in\mathbb{R}^{d+1}.$$

那么，对于数据集 $\mathcal{D}$，如果以下关系

$$y^{(j)}-\theta^\top\begin{pmatrix}x^{(j)}\\1\end{pmatrix}=y^{(j)}-(w^\top x^{(j)}+b)=e^{(j)}\sim\mathcal{N}(0,s^2)$$

对所有 $j\in[M]$ 成立，我们会得到 θ 的**似然函数**（likelihood function）L：

$$\begin{aligned}L(\theta)&:=\prod_{j=1}^M\mathcal{N}(e^j;0,s^2)\\&=\prod_{j=1}^M\mathcal{N}(y^{(j)}-w^\top x^{(j)}-b;0,s^2)\\&=(2\pi s^2)^{-\frac{M}{2}}e^{-\frac{1}{2s^2}\sum_{j=1}^M|y^{(j)}-w^\top x^{(j)}-b|^2}.\end{aligned}$$

统计学中常用的极大似然估计（maximum likelihood estimate）方法，即认为 $L(\theta)$ 取得最大值的点就是最优的 θ^* 。由于负对数函数是严格递减的，θ^* 同时

也是**负对数似然函数**（negative log-likelihood function）ℓ 的最小化解：

$$\begin{aligned}\ell(\theta) &:= -\log L(\theta)\\ &= \frac{1}{2s^2}\sum_{j=1}^{M}|y^{(j)} - w^\top x^{(j)} - b|^2 + \frac{M}{2}\log(2\pi s^2).\end{aligned}$$

因此我们得到**极大似然估计** θ^*：

$$\theta^* = \begin{pmatrix} w^* \\ b^* \end{pmatrix} = \arg\min_{(w,b)} \frac{1}{2s^2}\sum_{j=1}^{M}|y^{(j)} - w^\top x^{(j)} - b|^2. \tag{1.2}$$

注意，当初未知的那个标准差 s 不影响 (1.2) 的解，因此我们可以忽略它。现在记

$$X = \begin{pmatrix} x^{(1)\top} & 1 \\ \vdots & \vdots \\ x^{(M)\top} & 1 \end{pmatrix} \in \mathbb{R}^{M\times(d+1)} \quad 和 \quad Y = \begin{pmatrix} y^{(1)} \\ \vdots \\ y^{(M)} \end{pmatrix} \in \mathbb{R}^M,$$

则可以将最小化问题 (1.2) 写为简洁形式：

$$\min_{\theta\in\mathbb{R}^{d+1}} \frac{1}{2}|Y - X\theta|^2. \tag{1.3}$$

把 (1.3) 中的目标函数对 θ 求梯度，我们得到

$$X^\top X\theta = X^\top Y. \tag{1.4}$$

我们知道 (1.3) 的极小值点必定满足 (1.4) 。

注意，当 M 足够大且 $x^{(j)}$ 足够多样时（这在问题设置合理、且数据采集得当时，通常是成立的），矩阵 X 具有满列秩 $d+1$。（更精确地说，$x^{(1)},\ldots,x^{(M)}$ 张成 $\mathbb{R}^{d+1}$ 中一个不包含 $1 = (1,\ldots,1)^\top$ 的 d 维线性子空间，这样的 X 具有满列秩。）因此，$X^\top X$ 可逆，且 (1.4) 只有唯一解：

$$\theta^* = \begin{pmatrix} w^* \\ b^* \end{pmatrix} = (X^\top X)^{-1}X^\top Y,$$

这个解也是 (1.2) （或等价地 (1.3)）的唯一极小值点（因为 (1.3) 显然是关于 θ 的凸函数）。这就是线性回归里，通过利用数据集 $\mathcal{D}$ 在 (1.1) ，来 “学习” 最优的 w 和 b 的方式。

例 1.1.2 (逻辑分类，Logistic Classification)**.** 假设对于每一个 d 维特征向量 $x = (x_1, \ldots, x_d)^\top \in \mathbb{R}^d$，都有一个取值为二元的分类标识 $y \in \{0, 1\}$，称为**标签**（label）。当特征向量 x 属于“真”类时，标签 $y = 1$；当 x 属于“假”类时，标签 $y = 0$。

在标准的逻辑分类（也称逻辑回归，logistic regression）中，若预测特征向量 x 属于“真”类，即其对应的标签 y 是 1 的概率 $\Pr(y = 1)$ 被设置为：

$$\Pr(y = 1) = \sigma(w^\top x + b), \tag{1.5}$$

其中

$$\sigma(z) := \frac{1}{1 + e^{-z}}, \qquad \forall\, z \in \mathbb{R} \tag{1.6}$$

称为**逻辑函数**（logistic function），更常称为 **Sigmoid 函数**。注意 σ 的值域为 $(0, 1)$，因此可用于表示概率。我们知道

$$\Pr(y = 0) = 1 - \Pr(y = 1) = 1 - \sigma(w^\top x + b).$$

换言之，y 可以看作服从参数为 $\sigma(w^\top x + b) \in (0, 1)$ 的伯努利（Bernoulli）分布的样本。

我们知道，若随机变量 Z 取值为 1 或 0，且服从 Bernoulli(p) 分布（$p \in (0, 1)$），则 $\Pr(Z = z) = p^z (1 - p)^{1-z}$，这是该分布的概率质量函数的一种统一表达形式。

为简化记号，我们记

$$\theta := \begin{pmatrix} w \\ b \end{pmatrix} \in \mathbb{R}^{d+1}.$$

现在我们同样需要一个包含 M 个独立同分布样本的数据集 $\mathcal{D}$

$$\mathcal{D} = \Big\{ (x^{(j)}, y^{(j)}) \in \mathbb{R}^d \times \{0, 1\} : \ j \in [M] \Big\}$$

其中每个样本 $(x^{(j)}, y^{(j)})$ 都满足 (1.5)。于是 θ 的似然函数 L 为：

$$L(\theta) = \prod_{j=1}^{M} \big(\sigma(w^\top x^{(j)} + b)\big)^{y^{(j)}} \big(1 - \sigma(w^\top x^{(j)} + b)\big)^{1 - y^{(j)}},$$

其负对数似然函数 ℓ 为:

$$\begin{aligned}\ell(\theta) &= -\log L(\theta) \\ &= -\sum_{j=1}^{M}\Big(y^{(j)}\log\sigma(w^\top x^{(j)}+b)+(1-y^{(j)})\log(1-\sigma(w^\top x^{(j)}+b))\Big).\end{aligned} \tag{1.7}$$

为了求解 (1.7) 中 ℓ 的极小值，我们首先推导一些简单结果。注意 σ 的导数为

$$\sigma'(z)=\frac{e^{-z}}{(1+e^{-z})^2}=\sigma(z)(1-\sigma(z)).$$

我们定义函数 $h:\mathbb{R}^d\times\mathbb{R}^{d+1}\to\mathbb{R}$ 为

$$h(x,\theta):=\sigma(w^\top x+b).$$

则有

$$\begin{aligned}\nabla_\theta h(x,\theta) &= \sigma'(w^\top x+b)\begin{pmatrix}x\\1\end{pmatrix}^\top \\ &= \sigma(w^\top x+b)\big(1-\sigma(w^\top x+b)\big)\begin{pmatrix}x\\1\end{pmatrix}^\top \in\mathbb{R}^{1\times(d+1)},\end{aligned}$$

从而

$$\begin{aligned}\nabla_\theta\log h(x,\theta) &= (1-\sigma(w^\top x+b))\begin{pmatrix}x\\1\end{pmatrix}^\top, \\ \nabla_\theta\log(1-h(x,\theta)) &= -\sigma(w^\top x+b)\begin{pmatrix}x\\1\end{pmatrix}^\top.\end{aligned}$$

它们的 Hessian 矩阵分别为

$$\begin{aligned}\nabla_\theta^2\log h(x,\theta) &= -\sigma(w^\top x+b)(1-\sigma(w^\top x+b))\begin{pmatrix}x\\1\end{pmatrix}\begin{pmatrix}x\\1\end{pmatrix}^\top, \\ \nabla_\theta^2\log(1-h(x,\theta)) &= -\sigma(w^\top x+b)(1-\sigma(w^\top x+b))\begin{pmatrix}x\\1\end{pmatrix}\begin{pmatrix}x\\1\end{pmatrix}^\top,\end{aligned}$$

对于任意 x 和 θ，它们是相同的负半定矩阵。

基于上述结果，由 (1.7) 得到

$$\nabla_\theta \ell(\theta) = -\sum_{j=1}^{M} \left(y^{(j)}(1-\sigma(w^\top x^{(j)}+b)) - (1-y^{(j)})\sigma(w^\top x^{(j)}+b) \right) \begin{pmatrix} x^{(j)} \\ 1 \end{pmatrix}^\top$$

在实际实现中，由于 $y^{(j)}$ 仅取 1 或 0，上式可以进一步简化。此外，有

$$\nabla_\theta^2 \ell(\theta) = \sum_{j=1}^{M} \sigma(w^\top x^{(j)}+b)(1-\sigma(w^\top x^{(j)}+b)) \begin{pmatrix} x^{(j)} \\ 1 \end{pmatrix} \begin{pmatrix} x^{(j)} \\ 1 \end{pmatrix}^\top \succeq 0$$

这表明 ℓ 关于 θ 是凸的。

因此，满足

$$\nabla_\theta \ell(\theta) = (\partial_w \ell(\theta), \partial_b \ell(\theta)) = 0 \tag{1.8}$$

的任意临界点（critical point）θ 都是 ℓ 的全局最小点，从而也是基于数据集 $\mathcal{D}$ 的极大似然估计。此外，若

$$X = \begin{pmatrix} x^{(1)\top} & 1 \\ \vdots & \vdots \\ x^{(M)\top} & 1 \end{pmatrix} \in \mathbb{R}^{M\times(d+1)}$$

具有满列秩 $d+1$，则对任意非零 $z \in \mathbb{R}^{d+1}$，有

$$z^\top \nabla_\theta^2 \ell(\theta) z = \sum_{j=1}^{M} \sigma(w^\top x^{(j)}+b)(1-\sigma(w^\top x^{(j)}+b)) \, |(x^{(j)\top},\ 1)z|^2 > 0,$$

从而 $\nabla_\theta^2 \ell(\theta) \succ 0$，因此 (1.8) 仅有一个临界点，即 ℓ 的唯一最小点。

例 1.1.1 and 1.1.2 引出了关于给定数据集 $\mathcal{D}$ 的函数逼近的若干问题：

1. 如果我们拥有数据集 $\mathcal{D}$，但没有像 (1.1) 和 (1.5) 中那样用于描述 x 与 y 关系的具体函数 f，该怎么办?

2. 如何找到一个模型（参数化函数），使其在适当参数（例如 w 和 b）下可以（近似地）表示未知函数 f?

3. 如何在模型表示 f 的精度与效率之间取得良好平衡？即模型能够在预设函数距离度量下逼近某一类函数中的任意函数，同时模型规模（参数数量）尽可能小?

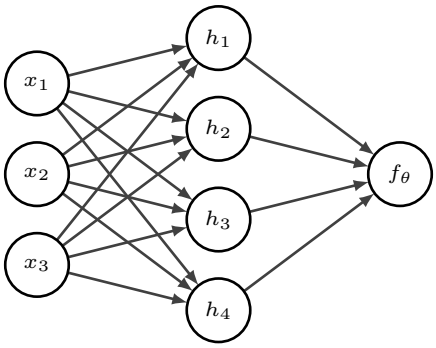


图 1.1: 一个浅层神经网络 f_θ，其输入层宽度（维数）为 $d = 3$，隐藏层宽度为 $d_1 = 4$，输出层宽度为 1。

4. 由于在实际中我们并不知道真实的 f，应如何设定评价标准，以判断模型是否具有正确的参数来逼近目标函数?

5. 在实际中如何求解模型的参数?

6. 为了获得模型的正确参数，我们需要多大的数据集 $\mathcal{D}$（即样本数量）?

在本书中，这些问题将始终围绕我们，我们也会尝试逐步给出答案。

1.2 浅度和深度神经网络

我们考虑如下模型，它结合了线性回归和逻辑分类示例（例 1.1.1 和例 1.1.2）中的思想：

$$f_\theta(x) := \sum_{j=1}^{d_1} a_j \sigma(w_j^\top x + b_j), \tag{1.9}$$

其中 $w_j \in \mathbb{R}^d$，$a_j, b_j \in \mathbb{R}$，$d_1 \in \mathbb{N}$，并且 θ 是将所有这些 w_j、a_j 和 b_j（$j = 1, \ldots, d_1$）拼接而成的向量。为方便起见，在本书的其余部分中，我们通常称 θ 为 “一个参数”，尽管它实际上是 (1.9) 中模型 f_θ 的所有参数的组合。

例 1.2.1 (浅度神经网络)**.** 我们用图 1.1 说明 (1.9) 中的模型 f_θ 在 $d = 3$ 且 $d_1 = 4$ 的情形。

图 1.1 所示的图对应一个模型 f_θ，我们称之为**浅度神经网络**（或浅层神经网络），其中 x_1, x_2, x_3 是**输入层**（input layer）中的神经元，h_1, h_2, h_3, h_4 是**隐藏层**（hidden layer）中的神经元，而 f_θ 是**输出层**（output layer）中的神经

元。这些神经元的取值关系可以表示为

$$h_j(x) = \sigma(w_j^\top x + b_j) \in \mathbb{R}, \quad \forall\, j \in [d_1],$$

从而有

$$f_\theta(x) = \sum_{j=1}^{d_1} a_j h_j(x).$$

这里 σ 称为**激活函数**（activation function），它可以是 (1.6) 中的逻辑函数，也可以是任意一个非仿射函数；不然的话，f_θ 将退化为关于 x 的仿射函数。我们将在第 1.5.1 节介绍现代深度学习中若干常用的激活函数。

奠基性工作 [30] 表明，形如 (1.9) 的浅层神经网络 在定义于 $\mathbb{R}^d$ 的紧子集 Ω 上的连续函数集合中是稠密的。过去几十年里，人们针对更一般的情形发展了大量理论结果，例如使用更深的网络、其他激活函数以及不同的目标函数集合进行逼近。我们将在第 1.6.2 节给出更多相关参考文献。

尽管像 (1.9) 这样的浅层网络具有可证明的函数逼近能力，但人们发现，它们的复合形式——通常称为**深度神经网络**——在函数逼近方面更为强大，而且所需参数数量远少于理论估计所暗示的规模。深度神经网络的一个简单例子是多个浅层网络的直接复合，称为**多层感知机**（multilayer perceptron，简称 MLP），如下一个例子所示。

例 1.2.2 (多层感知机)**.** 多层感知机（MLP）是多个形如 (1.9) 的浅层网络的复合。具体地，一个 MLP 可以有 L 个隐藏层，其中第 l 个隐藏层可以有 d_l 个神经元 $h_j^{(l)}$，这里 $j = 1, \ldots, d_l$ 且 $l = 1, \ldots, L$。输出层则可以根据需要包含一个或多个神经元。

对于一个具有两个隐藏层的 MLP，第一隐藏层中的神经元可以表示为

$$h_j^{(1)}(x) = \sigma^{(1)}(w_j^{(1)\top} x + b_j^{(1)}), \quad w_j^{(1)} \in \mathbb{R}^d,\ b_j^{(1)} \in \mathbb{R},\ j = 1, \ldots, d_1.$$

第二隐藏层中的神经元可以表示为

$$h_j^{(2)}(x) = \sigma^{(2)}(w_j^{(2)\top} h_j(x) + b_j^{(2)}), \quad w_j^{(2)} \in \mathbb{R}^{d_1},\ b_j^{(2)} \in \mathbb{R},\ j = 1, \ldots, d_2.$$

这里 $\sigma^{(1)}$ 和 $\sigma^{(2)}$ 分别是第一隐藏层和第二隐藏层中的激活函数。输出层神经

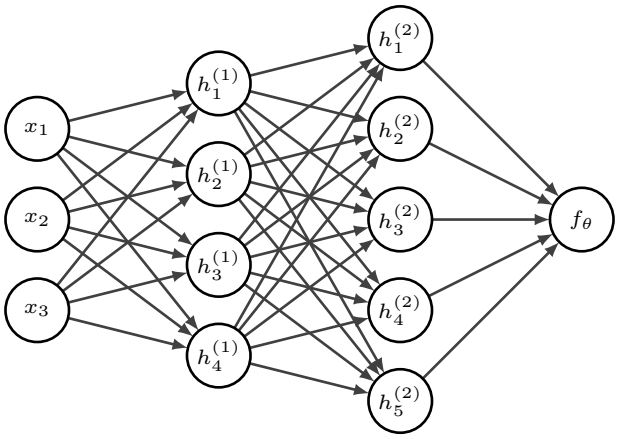


图 1.2: 图中展示了一个深度神经网络 f_θ 的例子。这种网络也称为多层感知机（MLP），其输入层宽度为 $d = 3$，两个隐藏层宽度分别为 $d_1 = 4$ 和 $d_2 = 5$，输出层宽度为 1。

元由下式给出

$$f_\theta(x) = \sum_{j=1}^{d_2} a_j h_j^{(2)}(x) + b, \quad a_j,\ b \in \mathbb{R},\ j = 1, \ldots, d_2,$$

在这个例子中，它是关于第二层也是最后一层隐藏层的一个仿射函数。

输入层宽度为 $d = 3$、两个隐藏层宽度分别为 $d_1 = 4$ 和 $d_2 = 5$、输出层宽度为 1 的 MLP 如图 1.2 所示。参数 θ 是将该 MLP 中所有权重和偏置拼接而成的向量，我们可以计算出 θ 的维数为 $(d+1)d_1 + (d_1+1)d_2 + (d_2+1)$。

一个 MLP 可以有很多隐藏层。隐藏层越多，网络就越深。训练一个模型 f_θ，无论它是 MLP 还是任意其他结构的深度神经网络，都是指寻找参数 θ，使得 f_θ 能够逼近所需的未知函数。这通常统称为**深度学习**。深度学习是一类新的机器学习技术，并且由于其在广泛真实世界应用中的惊人经验成功，它被认为是实现通用人工智能（artificial intelligence，简称 AI）最有前景的方法。

在结束本节之前，我们对深度神经网络作几点说明，并提出一个问题。首先，我们注意到激活函数在构建深度神经网络时起着重要作用。特别地，激活函数不应当是仿射的——否则神经网络不过是仿射函数的复合，最终仍只能得到仿射函数，而其逼近能力非常有限。除了前面见过的逻辑（sigmoid）函数 (1.6) 之外，另一个流行、简单而有效的激活函数是

$$\sigma(x) = \max(0, x), \tag{1.10}$$

称为修正线性单元（rectified linear unit，简称 ReLU）函数。注意，典型的激活函数，如 (1.6) 和 (1.10)，都是一元函数，即从 $\mathbb{R}$ 到 $\mathbb{R}$ 的映射。当它们作用于向量时，是对输入向量的各个分量分别作用，从而得到一个与输入向量维数相同的输出向量。我们将在第 1.5.1 节展示另外几种常用的激活函数。

在本书的其余部分中，我们将深度神经网络用作逼近复杂目标函数的有前景的参数化模型。特别地，我们将具有未指定网络参数 θ 的 f_θ 称为一种**网络架构**（network architecture）。在一个网络架构中，神经元（节点）、它们之间的连接（有向边）以及神经元处的激活函数都是固定的，而参数（如权重和偏置）的取值尚未指定。

与此同时，关于神经网络我们有一个重要问题：**为什么神经网络能够保证具有逼近未知且复杂函数 f 的能力?**

为了以足够严格的数学方式回答这个问题，我们首先需要指定一个函数集合 F，使得函数 f 属于其中（这样我们就可以将搜索范围缩小到 F 中），并指定一个范数 $\|\cdot\|$ 作为衡量两个函数之间距离的度量。然后，对于任意 $\epsilon > 0$，我们需要构造——或者至少证明存在——一个网络架构 f_θ，使得对于任意 $f \in F$，都存在一个参数 θ 满足 $\|f_\theta - f\| \le \epsilon$。接下来的两节中，我们将给出更多细节，并回答这个问题。

1.3 普适逼近定理

上一节提出的问题是很有挑战性的。为了回答这个问题，我们需要对函数 f 及其定义区域作出一些基本假设。在文献中，通常假设 f 是连续的，并且其定义域 Ω 是 $\mathbb{R}^d$ 的一个有界子集。因此，不失一般性，我们假设定义域为 $\Omega = [0,1]^d$，即 $\mathbb{R}^d$ 中的闭单位立方体，并且 f 属于一大类函数；除连续性之外，这类函数还可能具有某些光滑性性质（例如可微性）。

在本节和下一节中，我们遵循 [158]，陈述一个关于深度 ReLU 网络（即所有激活函数都取为 ReLU 的神经网络）逼近性质的定理，并给出其证明。该定理给出了一个具有代表性的定量误差估计，而其证明只需要微积分和基础的数学分析技巧。

现在，我们任意给定 $M > 0$、$k \in \mathbb{N}$，并定义

$$F := \{f \in W^{k,\infty}([0,1]^d) : \|f\|_{W^{k,\infty}([0,1]^d)} \le M\}. \tag{1.11}$$

（我们将在本节稍后对这个索伯列夫（Sobolev）空间 $W^{k,\infty}$ 及其范数作更详细的说明。）那么我们希望说明，对于任意 $\epsilon > 0$，都存在一个深度 ReLU 网络架

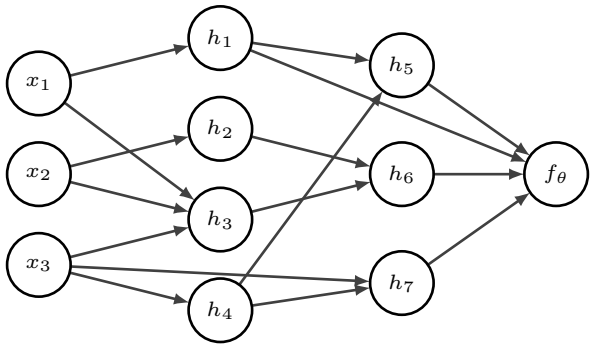


图 1.3: 一个 ReLU 前馈网络 f_θ，具有 3 个输入神经元 x_1、x_2 和 x_3。其余所有神经元都是计算单元。该网络中权重和偏置的总数为 24，即计算单元的数量（本例中为 8，即 $h_1, \ldots, h_7, f_\theta$）与边的数量（本例中为 16）之和。

构 f_θ，其规模（我们将在下文中定义“规模”）依赖于 d、M、k 和 ϵ，使得对任意 $f \in F$，存在某个依赖于 f 的参数 θ 满足

$$\|f_\theta - f\|_{L^\infty([0,1]^d)} \leq \epsilon.$$

这一结果正式称为神经网络的**普适逼近定理**（Universal Approximation Theorem）。（该定理的正式表述及其证明将在下一节给出。）

前馈网络及其规模 由于将要陈述的逼近定理给出了关于误差 ϵ 的网络规模定量估计，我们需要澄清网络规模的确切含义。为此，我们首先指出，该定理的结论适用于**前馈神经网络**（Feed-forward network）。前馈网络可以看作是一类带有跳跃连接的深度网络。图 1.3 展示了一个前馈网络的例子。

在图 1.3 中，网络有 3 个输入神经元 x_1、x_2 和 x_3。其余 8 个神经元称为**计算单元**，因为它们的取值需要根据其父神经元的值计算得到。例如，神经元 h_1 处的取值为

$$h_1(x) = \sigma(w^{(x_1,h_1)}x_1 + w^{(x_3,h_1)}x_3 + b^{h_1}),$$

其中 $w^{(i,j)} \in \mathbb{R}$ 是网络中有向边 (i,j) 的权重，$b^j \in \mathbb{R}$ 是神经元 j 处的偏置。在本节和下一节中，为了方便起见，我们也将偏置称为权重。因此，为了计算 $h_1(x)$，我们需要 3 个权重 $w^{(x_1,h_1)}$、$w^{(x_3,h_1)}$ 和 b^{h_1}。容易看出，总权重数等于边的数量与计算单元数量之和。对于图 1.3 中的这个示例网络，其总权重数为 24。

Lebesgue 空间、Sobolev 空间及其范数 对于可测集合 $\Omega \subset \mathbb{R}^d$ 和可测函数 $f:\Omega \to \mathbb{R}$，f 在 Ω 上的**本性上确界**（essential supremum），记为 $\|f\|_{L^\infty(\Omega)}$，其定义为

$$\|f\|_{L^\infty(\Omega)} = \inf\left\{M \in \mathbb{R}: \begin{array}{l} \exists \Omega_0 \subset \Omega \quad \text{with} \quad \mu(\Omega_0) = 0, \\ \text{s.t. } |f(x)| \le M,\ \forall x \in \Omega \setminus \Omega_0 \end{array}\right\}. \tag{1.12}$$

也就是说，Ω 上存在一个测度为零的子集 Ω_0 ，在这个子集之外，即 $\Omega \setminus \Omega_0$ 上，$\|f\|_{L^\infty(\Omega)}$ 是 $|f|$ 的最小的上界。具有有限 L^∞ 范数的函数集合称为 L^∞ **Lebesgue 空间**。如果 Ω 是紧的且 f 在 Ω 上连续，则 $\|f\|_{L^\infty(\Omega)} = \max_{x\in\Omega}|f(x)|$。在第 1.4 节中我们会频繁使用这一点；为简化记号，在第 1.4 节中所涉及的任何紧定义域上，我们都用 $\|\cdot\|_\infty$ 代表 $\|\cdot\|_{L^\infty}$。

我们进一步假设 $f:\Omega \to \mathbb{R}$ 存在所有 k 阶（弱）偏导数，即对每个满足 $|\mathbf{k}|_1 := \sum_{i=1}^d k_i \le k$ 的多重指标 $\mathbf{k} = (k_1, \ldots, k_d) \in \{0, 1, \ldots, k\}^d$，$D^{\mathbf{k}} f$ 都存在，并定义范数

$$\|f\|_{W^{k,\infty}(\Omega)} = \max_{\mathbf{k}:|\mathbf{k}|_1 \le k} \|D^{\mathbf{k}} f\|_{L^\infty(\Omega)}. \tag{1.13}$$

（在本节和下一节中，我们将用黑体字母表示向量，以便更清晰地呈现数学公式。对于本书其余部分，我们仍使用普通字母表示向量，因为那时不会产生混淆。）具有有限 $W^{k,\infty}$ 范数的函数集合称为 $W^{k,\infty}$ **Sobolev 空间**。我们还知道，$f \in W^{k,\infty}(\Omega)$ 当且仅当对所有满足 $|\mathbf{k}|_1 \le k-1$ 的 $\mathbf{k}$，$D^{\mathbf{k}} f$ 在 Ω 中存在，并且对所有满足 $|\mathbf{k}|_1 = k-1$ 的 $\mathbf{k}$，$D^{\mathbf{k}} f$ 在 Ω 中是 Lipschitz 连续的。

由于 $\|\cdot\|_{L^\infty(\Omega)}$ 和 $\|\cdot\|_{W^{k,\infty}(\Omega)}$ 都是绝对齐次的，即对任意 $\alpha \in \mathbb{R}$ 和 $f \in L^\infty(\Omega)$，有 $\|\alpha f\|_{L^\infty(\Omega)} = |\alpha|\,\|\alpha f\|_{L^\infty(\Omega)}$（$\|\cdot\|_{W^{k,\infty}(\Omega)}$ 也同样如此），因此只需证明本节开头关于 (1.11) 中 F 的定义在 $M=1$ 时的结论即可。我们将在下一节中，把刚才这一结论表述为普适逼近定理的一个版本，并给出其证明。

1.4 普适逼近定理的证明

在本节中，我们证明深度 ReLU 前馈网络的普适逼近定理（在本节中我们简称其为 ReLU 网络）。该定理的正式表述将在本节后面给出。由于该定理还给出了网络深度、权重数和计算单元数的上界，而这些上界依赖于逼近误差容许量 $\epsilon > 0$，因此我们将在证明中跟踪这些数量的（阶的）变化。由于权重数总是大于计算单元数，我们通常只跟踪前者。

现在我们开始证明。我们先从下面这个关于用 ReLU 网络在区间 $[0,1]$ 上

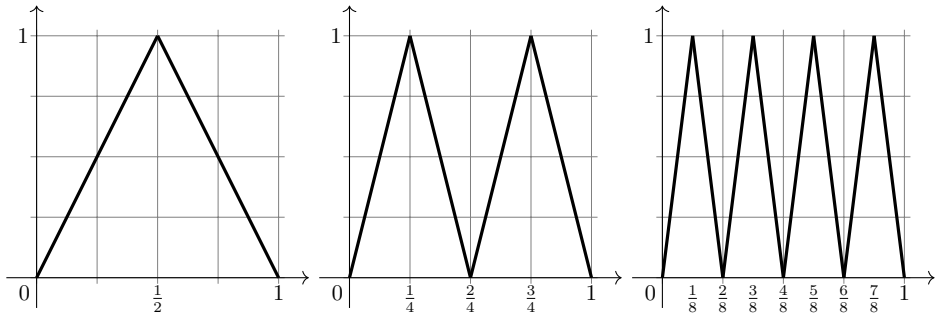


图 1.4: 从左到右：在 (1.14) 中定义的 g_1、g_2 和 g_3。

逼近 x^2 的简单命题开始。

命题 1.4.1. 对于任意 $\epsilon > 0$，定义在 $[0,1]$ 上的函数 $f(x) = x^2$ 都可以被一个具有 $O(\log(1/\epsilon))$ 个权重的 *ReLU* 网络以误差 $\epsilon > 0$ 逼近。

证明. 我们考虑一个基本函数 $g : [0,1] \to [0,1]$，定义为

$$g(x) = \begin{cases} 2x, & \text{if } 0 \le x \le \frac{1}{2}, \\ 2(1-x), & \text{if } \frac{1}{2} \le x \le 1, \end{cases}$$

以及 g 的 s 次复合：

$$g_s(x) := \underbrace{g \circ g \circ \cdots \circ g}_{s \text{ in total}}(x)$$

其中 $s \in \mathbb{N}$。容易验证

$$g_s(x) = \begin{cases} 2^s\left(\frac{2k}{2^s} - x\right), & \text{if } x \in \left[\frac{2k-1}{2^s}, \frac{2k}{2^s}\right],\ k = 1, \ldots, 2^{s-1}, \\ 2^s\left(x - \frac{2k}{2^s}\right), & \text{if } x \in \left[\frac{2k}{2^s}, \frac{2k+1}{2^s}\right],\ k = 0, 1, \ldots, 2^{s-1} - 1. \end{cases} \tag{1.14}$$

$g_1 (= g)$、g_2 和 g_3 的图像如图 1.4 所示。

现在，对任意 $m = 0, 1, \cdots$，我们将 $[0,1]$ 划分为 2^m 个等长区间，并定义一个分段线性函数 f_m 来逼近 $f(x) = x^2$。具体地，我们要求 f_m 在所有网格点 $\frac{k}{2^m}$（其中 $k = 0, \ldots, 2^m$）处与 f 一致：

$$f_m\left(\frac{k}{2^m}\right) = f\left(\frac{k}{2^m}\right) = \left(\frac{k}{2^m}\right)^2.$$

容易验证 $f_0(x)=x$，并且对所有 $x\in[0,1]$ 和 $m\in\mathbb{N}$，下式都成立：

$$f_{m-1}(x)-f_m(x)=\frac{g_m(x)}{2^{2m}}.$$

从 1 到 m 求和，我们可得

$$f_m(x)=f_0(x)-\sum_{s=1}^{m}\frac{g_s(x)}{2^{2s}}=x-\sum_{s=1}^{m}\frac{g_s(x)}{2^{2s}}. \tag{1.15}$$

这对所有 $m=1,2,\ldots$ 都成立。

此外，我们注意到 g 可以写成一个小型 ReLU 网络：

$$g(x)=2\sigma(x)-4\sigma\Big(x-\frac{1}{2}\Big)+2\sigma(x-1).$$

也就是说，如果我们定义

$$h_1^{(1)}(x)=\sigma(x),\quad h_2^{(1)}(x)=\sigma\Big(x-\frac{1}{2}\Big),\quad h_3^{(1)}(x)=\sigma(x-1),$$

其中 σ 是 ReLU 函数，那么有

$$g(x)=2h_1^{(1)}(x)-4h_2^{(1)}(x)+2h_3^{(1)}(x).$$

进一步地，我们定义

$$\begin{aligned}
h_1^{(2)}&=\sigma(g(x))=\sigma\big(2h_1^{(1)}(x)-4h_2^{(1)}(x)+2h_3^{(1)}(x)\big),\\
h_2^{(2)}&=\sigma\Big(g(x)-\frac{1}{2}\Big)=\sigma\Big(2h_1^{(1)}(x)-4h_2^{(1)}(x)+2h_3^{(1)}(x)-\frac{1}{2}\Big),\\
h_3^{(2)}&=\sigma(g(x)-1)=\sigma\big(2h_1^{(1)}(x)-4h_2^{(1)}(x)+2h_3^{(1)}(x)-1\big).
\end{aligned}$$

于是有

$$g_2(x)=2h_1^{(1)}(x)-4h_2^{(2)}(x)+2h_3^{(2)}(x).$$

按照这一模式，我们可以构造一个 ReLU 网络来表示任意的 g_m 和 f_m（$m\geq 1$）。ReLU 网络 f_3 的图像如图 1.5 所示。

沿着图 1.5 所示的 ReLU 网络并增大 m，我们可以看到 ReLU 网络 f_m 有 $3m+1$ 个计算单元 $\{h_1^{(s)},h_2^{(s)},h_3^{(s)},f_m:\forall\, s\in[m]\}$，以及 $12m-5$ 条边（对每个 $s\in[m-1]$，x、$h_1^{(s)}$、$h_2^{(s)}$ 和 $h_3^{(s)}$ 各有 4 条出边，而 $h_1^{(m)}$、$h_2^{(m)}$ 和 $h_3^{(m)}$ 各有 1 条出边）。因此，f_m 中权重的总数为 $15m-4=O(m)$。

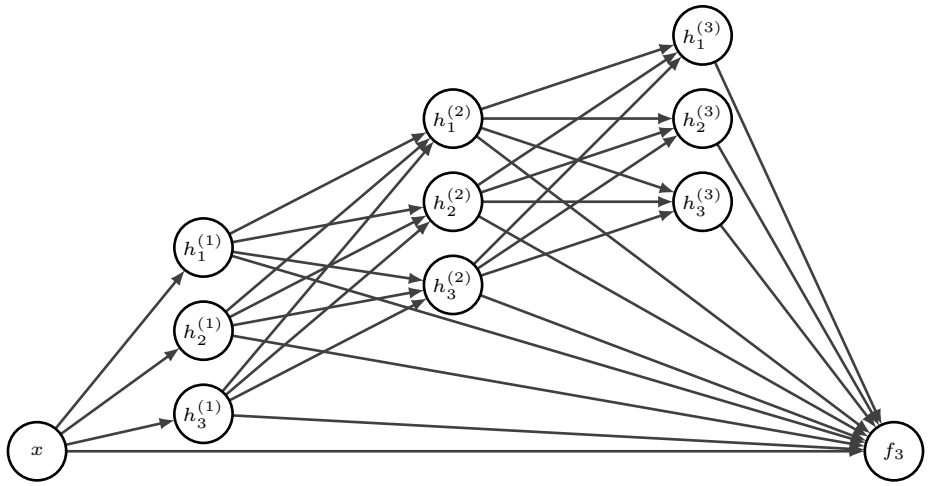


图 1.5: 在 (1.15) 中定义的 ReLU 网络 f_m 在 $m=3$ 时的情形。

同样也很容易求出 f_m 的逼近误差:

$$\begin{aligned}\|f-f_m\|_\infty &= \max_{x\in[0,1]}|f(x)-f_m(x)|\\ &= f_m(x_m^*)-f(x_m^*)\\ &= \frac{1}{2^{2m+2}},\end{aligned}$$

其中 $x_m^*=1-\frac{1}{2^{m+1}}$。因此，为了得到 $\|f-f_m\|_\infty\le\epsilon$，只需令 $\frac{1}{2^{2m+2}}\le\epsilon$。换言之，$f_m$ 的深度和权重数都是 $O(\log(1/\epsilon))$。 □

注意，对任意 $a,b\in\mathbb{R}$，有

$$ab=\frac{1}{2}\Big((a+b)^2-a^2-b^2\Big). \tag{1.16}$$

由于我们可以像命题 1.4.1 那样用 ReLU 网络逼近平方函数 x^2，因此可以基于恒等式 (1.16) 用 ReLU 网络逼近两个实数的乘积。在下一个命题中，我们将说明如何实现这种逼近。

命题 1.4.2. *对于任意 $M\ge1$ 和 $\epsilon\in(0,1)$，存在一个 ReLU 网络 $p_{\epsilon,M}:[-M,M]\times[-M,M]\to\mathbb{R}$，使得*

(i) 对任意 $a,b\in[-M,M]$，有 $|p_{\epsilon,M}(a,b)-ab|\le\epsilon$;

(ii) 若 $a=0$ 或 $b=0$，则 $p_{\epsilon,M}(a,b)=0$；并且

(iii) $p_{\epsilon,M}$ 的深度和权重数都是 $O(\log(1/\epsilon)+\log M)$。

证明. 设 $\delta>0$（我们稍后确定其值），设 $q_\delta:[0,1]\to[0,1]$ 是命题 1.4.1 中以误差 δ 逼近平方函数 $f(x)=x^2$ 的 ReLU 网络。因此 $q_\delta(0)=0$，并且下式

$$|q_\delta(x)-x^2|\le\delta$$

对所有 $x\in[0,1]$ 成立。定义

$$p_{\epsilon,M}(a,b)=\frac{M^2}{2}\Big[q_\delta\Big(\frac{|a+b|}{2M}\Big)-q_\delta\Big(\frac{|a|}{2M}\Big)-q_\delta\Big(\frac{|b|}{2M}\Big)\Big].$$

显然，当 $a=0$ 或 $b=0$ 时，$p_{\epsilon,M}(a,b)=0$。

令 $\delta=\frac{8\epsilon}{3M^2}$，则有

$$\begin{aligned}|p_{\epsilon,M}(a,b)-ab|&\le\frac{M^2}{8}\Big[\Big|q_\delta\Big(\frac{|a+b|}{2M}\Big)-\Big(\frac{|a+b|}{2M}\Big)^2\Big|+\Big|q_\delta\Big(\frac{|a|}{2M}\Big)-\Big(\frac{|a|}{2M}\Big)^2\Big|\\&\qquad+\Big|q_\delta\Big(\frac{|b|}{2M}\Big)-\Big(\frac{|b|}{2M}\Big)^2\Big|\Big]\\&\le\frac{M^2}{8}\cdot(3\delta)\\&=\epsilon.\end{aligned}$$

由于 q_δ 的深度和权重数为 $O(\log(1/\delta))=O(\log(1/\epsilon)+\log M)$，我们知道 $p_{\epsilon,M}$ 也满足同样的估计。 □

我们将 $W^{k,\infty}([0,1]^d)$ 中的单位球记为

$$B_{k,d}=\{f\in W^{k,\infty}([0,1]^d):\|f\|_{W^{k,\infty}([0,1]^d)}\le1\}. \tag{1.17}$$

考虑 $[0,1]^d$ 上如下均匀网格：我们将每个坐标轴上的区间 $[0,1]$ 划分为 N 个等长区间，则网格点集合为

$$\Big\{\frac{\mathbf{m}}{N}\in[0,1]^d\ :\ \mathbf{m}=(m_1,\dots,m_d)\in N_d\Big\},$$

其中

$$N_d:=\{0,1,\dots,N\}^d.$$

这样，我们在 $[0,1]^d$ 中总共有 $(N+1)^d$ 个网格点。

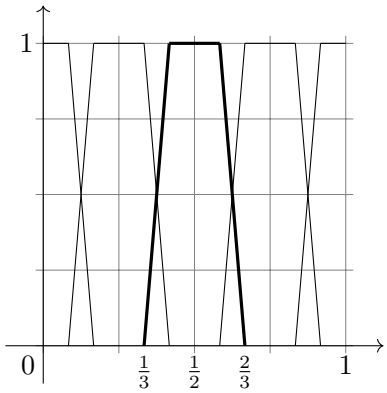


图 1.6: 从左到右：在 $d=1$ 且 $N=4$ 的情形下的 ϕ_0、ϕ_1、ϕ_2（加粗）、ϕ_3 和 ϕ_4。注意对所有 $x\in[0,1]$，都有 $\sum_m \phi_m(x)=1$。

现在定义一个函数 $\psi:\mathbb{R}\to[0,1]$ 为

$$\psi(x)=\begin{cases}1, & \text{if} \quad |x|\le 1,\\ 2-|x|, & \text{if} \quad 1\le|x|\le 2,\\ 0, & \text{if} \quad |x|\ge 2.\end{cases}$$

然后，对任意 $\mathbf{m}=(m_1,\dots,m_d)\in N_d$，定义

$$\phi_{\mathbf{m}}(\mathbf{x})=\prod_{i=1}^{d}\psi\Big(3N\big(x_i-\frac{m_i}{N}\big)\Big) \tag{1.18}$$

其中 $\mathbf{x}=(x_1,\dots,x_d)\in[0,1]^d$。由于 $\|\psi\|_\infty=1$，我们也知道 $\|\phi_{\mathbf{m}}\|_{L^\infty([0,1]^d)}\le 1$。在 $d=1$ 且 $N=4$ 的情形下，$\phi_{\mathbf{m}}$ 的一个例子如图 1.6 所示。

在下面的命题中，我们将说明 $\{\phi_{\mathbf{m}}:\mathbf{m}\in N_d\}$ 在 $[0,1]^d$ 上构成一个单位分解（partition of unity）。

命题 1.4.3. *对于任意 $N\in\mathbb{N}$，按 (1.18) 定义 $\phi_{\mathbf{m}}$。则*

1. *对任意 $\mathbf{m}\in N_d$ 和 $\mathbf{x}\in[0,1]^d$，有 $0\le\phi_{\mathbf{m}}(\mathbf{x})\le 1$。此外，$\|\phi_{\mathbf{m}}\|_{L^\infty([0,1]^d)}=1$。*

2. *$\phi_{\mathbf{m}}$ 的支撑集（support）满足*

$$\begin{aligned}\operatorname{supp}(\phi_{\mathbf{m}})&=\Big\{\mathbf{x}\in[0,1]^d:\Big|x_i-\frac{m_i}{N}\Big|\le\frac{2}{3N},\ \forall\, i\in[d]\Big\}\\ &\subset\Big\{\mathbf{x}\in[0,1]^d:\Big|x_i-\frac{m_i}{N}\Big|\le\frac{1}{N},\ \forall\, i\in[d]\Big\}\end{aligned}$$

并且任意 $\mathbf{x} \in [0,1]^d$ 至多属于 2^d 个 $\phi_{\mathbf{m}}$ 的支撑。

3. 对任意 $\mathbf{x} \in [0,1]^d$，有

$$\sum_{\mathbf{m} \in N_d} \phi_{\mathbf{m}}(\mathbf{x}) = 1.$$

证明. 因为 $\|\psi\|_\infty = 1$ 且 $\phi_{\mathbf{m}}(\frac{\mathbf{m}}{N}) = 1$，所以第一个结论成立。

对任意 $\mathbf{x} \in [0,1]^d$，存在 $\mathbf{m} \in N_d$，使得对任意 $i \in [d]$，以下两种情况中有且只有一种发生：

$$\frac{3m_i - 2}{3N} < x_i < \frac{3m_i - 1}{3N}, \tag{1.19a}$$

$$\frac{3m_i - 1}{3N} \le x_i \le \frac{3m_i}{3N}. \tag{1.19b}$$

这些蕴含了第二个结论成立。

当 (1.19a) 发生时，有

$$\psi\Big(3N\big(x_i - \frac{m_i}{N}\big)\Big) = 2 - \Big|3N\big(x_i - \frac{m_i}{N}\big)\Big| = 2 + 3N\big(x_i - \frac{m_i}{N}\big).$$

当 (1.19b) 发生时，有

$$\psi\Big(3N\big(x_i - \frac{m_i}{N}\big)\Big) = 1.$$

不失一般性，设 $d' \in \{0, 1, \ldots, d\}$ ，使得当 $i \le d'$ 时 (1.19a) 发生，而当 $d' + 1 \le i \le d$ 时 (1.19b) 发生。

如果 $d' = 0$，则对所有 $i \in [d]$，都有 $\psi(3N(x_i - \frac{m_i}{N})) = 1$。因此

$$\phi_{\mathbf{m}}(\mathbf{x}) = \prod_{i=1}^{d} \psi\Big(3N\big(x_i - \frac{m_i}{N}\big)\Big) = 1.$$

如果 $d' = 1$，则

$$\begin{aligned}
\sum_{\mathbf{m} \in N_d} \phi_{\mathbf{m}} &= \phi_{\mathbf{m}}(\mathbf{x}) + \phi_{\mathbf{m}-\mathbf{e}_1}(\mathbf{x}) \\
&= \psi\Big(3N\big(x_1 - \frac{m_1}{N}\big)\Big) + \psi\Big(3N\big(x_1 - \frac{m_1 - 1}{N}\big)\Big) \\
&= (2 + 3Nx_1 - 3m_1) + (2 - 3Nx_1 + 3(m_1 - 1)) \\
&= 1,
\end{aligned}$$

其中

$$\mathbf{e}_i := (0,\dots,0,\underbrace{1}_{i\text{th}},0,\dots,0)\in\{0,1\}^d.$$

事实上，我们可以验证，对任意 $\mathbf{m}\in\{1,\dots,N\}^d$ 和 $i\in[d]$，下式

$$\psi\Big(3N\big(x_i-\frac{m_i}{N}\big)\Big)+\psi\Big(3N\big(x_i-\frac{m_i-1}{N}\big)\Big)=1 \tag{1.20}$$

对任意 $\mathbf{x}\in[0,1]^d$ 成立。

如果 $d'=2$，则

$$\begin{aligned}\sum_{\mathbf{m}\in N_d}\phi_{\mathbf{m}} &= \phi_{\mathbf{m}}(\mathbf{x})+\phi_{\mathbf{m}-\mathbf{e}_1}(\mathbf{x})+\phi_{\mathbf{m}-\mathbf{e}_1-\mathbf{e}_2}(\mathbf{x})+\phi_{\mathbf{m}-\mathbf{e}_2}(\mathbf{x})\\ &= \psi\Big(3N\big(x_1-\frac{m_1}{N}\big)\Big)\psi\Big(3N\big(x_2-\frac{m_2}{N}\big)\Big)\\ &\quad+\psi\Big(3N\big(x_1-\frac{m_1-1}{N}\big)\Big)\psi\Big(3N\big(x_2-\frac{m_2}{N}\big)\Big)\\ &\quad+\psi\Big(3N\big(x_1-\frac{m_1-1}{N}\big)\Big)\psi\Big(3N\big(x_2-\frac{m_2-1}{N}\big)\Big)\\ &\quad+\psi\Big(3N\big(x_1-\frac{m_1}{N}\big)\Big)\psi\Big(3N\big(x_2-\frac{m_2-1}{N}\big)\Big)\\ &= 1\end{aligned}$$

这是利用 (1.20) 得到的。

按照这一思路并利用 (1.20)，可知对任意 $d'\in\{0,1,\dots,d\}$，都有

$$\sum_{\mathbf{m}\in N_d}\phi_{\mathbf{m}}(\mathbf{x})=1.$$

证明完成。 □

如果 $f\in W^{k,\infty}([0,1]^d)$，则我们知道对任意满足 $|\mathbf{k}|_1=\sum_{i=1}^d k_i<k-1$ 的 $\mathbf{k}=(k_1,\dots,k_d)\in\{0,\dots,k\}^d$，$D^{\mathbf{k}}f$ 在 Ω 中存在，并且 $\|D^{\mathbf{k}}f\|_{L^\infty([0,1]^d)}\le\|f\|_{W^{k,\infty}([0,1]^d)}$。对任意 $\mathbf{m}\in N_d$，由于 $\frac{\mathbf{m}}{N}\in[0,1]^d$，我们记 f 在 $\frac{\mathbf{m}}{N}$ 处的 $k-1$ 次 Taylor 多项式为

$$P_{\mathbf{m}}(\mathbf{x})=\sum_{\mathbf{k}:|\mathbf{k}|_1<k}\frac{1}{\mathbf{k}!}D^{\mathbf{k}}f\Big(\frac{\mathbf{m}}{N}\Big)\Big(\mathbf{x}-\frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}, \tag{1.21}$$

其中 $\mathbf{k}! = k_1! \cdots k_d!$，$(\mathbf{x} - \frac{\mathbf{m}}{N})^{\mathbf{k}} = \prod_{i=1}^{d}(x_i - \frac{m_i}{N})^{k_i}$，并且

$$D^{\mathbf{k}} f = \frac{\partial^k f}{\partial x_1^{k_1} \dots \partial x_d^{k_d}}.$$

接下来，我们说明 $f \in W^{k,\infty}([0,1]^d)$ 可以由对所有 $\mathbf{m} \in N_d$ 的 $\phi_{\mathbf{m}}(\mathbf{x})P_{\mathbf{m}}(\mathbf{x})$ 之和来逼近。

引理 1.4.4. *对于任意 $f \in B_{k,d}$ 和 $\epsilon > 0$，存在 $N = N(\epsilon, k, d) \in \mathbb{N}$ 以及形如*

$$f_N(\mathbf{x}) = \sum_{\mathbf{m} \in N_d} \phi_{\mathbf{m}}(\mathbf{x}) P_{\mathbf{m}}(\mathbf{x}), \tag{1.22}$$

的函数 f_N，其中 $P_{\mathbf{m}}(\mathbf{x})$ 按 (1.21) 定义，使得

$$\|f_N - f\|_{L^\infty([0,1]^d)} \le \frac{\epsilon}{2}. \tag{1.23}$$

证明. 注意到，对任意 $\mathbf{x} \in [0,1]^d$，有

$$\begin{aligned}
|f(\mathbf{x}) - f_N(\mathbf{x})| &= \Big| \sum_{\mathbf{m} \in N_d} \phi_{\mathbf{m}}(\mathbf{x})(f(\mathbf{x}) - P_{\mathbf{m}}(\mathbf{x})) \Big| && \text{(1.24a)}\\
&\le \sum_{\mathbf{m}: |x_i - \frac{m_i}{N}| < \frac{1}{N}, \forall i \in [d]} |f(\mathbf{x}) - P_{\mathbf{m}}(\mathbf{x})| && \text{(1.24b)}\\
&\le 2^d \max_{\mathbf{m}: |x_i - \frac{m_i}{N}| < \frac{1}{N}, \forall i \in [d]} |f(\mathbf{x}) - P_{\mathbf{m}}(\mathbf{x})| && \text{(1.24c)}\\
&\le 2^d \frac{1}{\mathbf{k}!} \sum_{\mathbf{k}: |\mathbf{k}|_1 = k} \|D^{\mathbf{k}} f\|_{L^\infty([0,1]^d)} \Big| \Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}} \Big| && \text{(1.24d)}\\
&\le 2^d \cdot 1 \cdot d^k \cdot 1 \cdot \Big(\frac{1}{N}\Big)^k, && \text{(1.24e)}
\end{aligned}$$

其中 (1.24a) 利用了 $\{\phi_{\mathbf{m}} : \mathbf{m} \in N_d\}$ 是单位分解；(1.24b) 利用了 $\|\phi_{\mathbf{m}}\|_{L^\infty([0,1]^d)} \le 1$；(1.24c) 利用了 (1.24b) 中求和项至多有 2^d 项；(1.24d) 利用了 (1.21) 中 $P_{\mathbf{m}}(\mathbf{x})$ 的 Taylor 多项式逼近性质；而 (1.24e) 则是因为 $\mathbf{k}! \ge 1$，(1.24d) 中求和项不超过 d^k 项，$\|D^{\mathbf{k}} f\|_{L^\infty([0,1]^d)} \le \|f\|_{W^{k,\infty}([0,1]^d)} \le 1$，并且对所有 $i \in [d]$，都有 $|x_i - \frac{m_i}{N}| \le 1$。

取

$$N = \Big\lceil \Big(\frac{1}{2^d d^k} \cdot \frac{\epsilon}{2}\Big)^{-\frac{1}{k}} \Big\rceil = \lceil 2^{-(d+1)/k} d^{-1} \epsilon^{-1/k} \rceil, \tag{1.25}$$

它依赖于 ϵ、k 和 d，但不依赖于具体的 f，于是我们得到所需的估计 (1.23)。 □

现在我们已经准备好证明深度 ReLU 神经网络的普适逼近定理。

定理 1.4.5 (普适逼近定理)**.** *对于任意 $d, k \in \mathbb{N}$ 和 $\epsilon \in (0,1)$，存在一个 ReLU 网络架构 h，它只依赖于 d、k 和 ϵ，使得*

1. *架构 h 能够通过适当选择参数来表示 $B_{k,d}$ 中的任意函数。*
2. *架构 h 的深度为 $O(\log(1/\epsilon))$，并且权重数至多为 $c\epsilon^{-d/k}\log(1/\epsilon)$，其中 c 依赖于 d 和 k，但不依赖于 ϵ。*

证明. 对任意 $\epsilon \in (0,1)$ 和 $f \in B_{k,d}$，令 N 和 f_N 如引理 1.4.4 中所给。我们将 Taylor 多项式 $P_{\mathbf{m}}(\mathbf{x})$ 改写为

$$P_{\mathbf{m}}(\mathbf{x}) = \sum_{\mathbf{k}:|\mathbf{k}|_1<k} a^f_{\mathbf{m},\mathbf{k}}\Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}$$

其中系数 $a^f_{\mathbf{m},\mathbf{k}} \in \mathbb{R}$ 定义为

$$a^f_{\mathbf{m},\mathbf{k}} = \frac{1}{\mathbf{k}!} D^{\mathbf{k}} f\Big(\frac{\mathbf{m}}{N}\Big).$$

由于 $f \in B_{k,d}$，我们知道对所有 $\mathbf{m}$ 和 $\mathbf{k}$，都有 $|a^f_{\mathbf{m},\mathbf{k}}| \le 1$。

基于 (1.22)，我们可以将 f_N 写为

$$f_N(\mathbf{x}) = \sum_{\mathbf{m}\in N_d} \sum_{\mathbf{k}:|\mathbf{k}|_1<k} a^f_{\mathbf{m},\mathbf{k}} \phi_{\mathbf{m}}(\mathbf{x})\Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}$$

并注意到，只有 $a^f_{\mathbf{m},\mathbf{k}}$ 依赖于给定的 f，而 $\phi_{\mathbf{m}}$ 和 $(\mathbf{x} - \frac{\mathbf{m}}{N})^{\mathbf{k}}$ 都不依赖于 f。

由 (1.4) 可见，f_N 的求和中至多有 $(N+1)^d d^k$ 项：N_d 有 $(N+1)^d$ 个元素，而 $\{\mathbf{k} : |\mathbf{k}|_1 < k\}$ 的大小上界为 d^k。我们还注意到，(1.4) 中每一项的函数部分都具有如下形式：

$$\phi_{\mathbf{m}}(\mathbf{x})\Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}} \tag{1.26}$$

它是至多 $d+k-1$ 个分段线性函数的乘积：

$$\underbrace{\psi\Big(3N\big(x_1 - \frac{m_1}{N}\big)\Big),\ \ldots,\ \psi\Big(3N\big(x_d - \frac{m_d}{N}\big)\Big)}_{d\ \text{分段线性函数}}, \underbrace{\Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}}_{<k\ \text{线性函数}}. \tag{1.27}$$

为简化记号，我们将 (1.27) 中前 j 项的乘积记为 $Q^j_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，将后 j 项的乘积记为 $Q^{-j}_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，其中 $j = 1, \ldots, d+k-1$。注意，对所有 j，都有 $|Q^j_{\mathbf{m},\mathbf{k}}(\mathbf{x})| \le 1$ 和 $|Q^{-j}_{\mathbf{m},\mathbf{k}}(\mathbf{x})| \le 1$。

接下来，我们用 ReLU 网络逼近 (1.26)。令 $M = d + k$，并令 $p_{\delta,M} : [-M, M] \times [-M, M] \to \mathbb{R}$ 为命题 1.4.2 中的 ReLU 网络，其中 $\delta > 0$ 稍后确定。定义

$$h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) = p_{\delta,M}\Big\{\psi\Big(3N\big(x_1 - \frac{m_1}{N}\big)\Big), p_{\delta,M}\Big[\psi\Big(3N\big(x_2 - \frac{m_2}{N}\big)\Big), \dots\Big]\Big\}. \quad (1.28)$$

也就是说，为了计算 $h_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，我们先将 $p_{\delta,M}$ 作用于 (1.27) 中最后两个因子，然后再将 $p_{\delta,M}$ 作用于所得结果和倒数第三个因子，如此继续，直到作用到 (1.27) 中的第一个因子结束。因此，为了得到 $h_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，$p_{\delta,M}$ 至多重复应用 $d + k - 2$ 次。为简化记号，我们将对 (1.27) 中最后 j 个因子应用 $p_{\delta,M}$ 所得的结果记为 $P^{-j}_{\mathbf{m},\mathbf{k}}(\mathbf{x})$，其中 $j = 2, 3, \dots, d+k-1$。例如，可以看到 $h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) = P^{-(d+|\mathbf{k}|_1-1)}_{\mathbf{m},\mathbf{k}}(\mathbf{x})$。

注意，对任意 $i \in [d]$，$\psi(3N(x_i - \frac{m_i}{N}))$ 都可以由一个仅含一层隐藏层且具有 $O(1)$ 个权重的 ReLU 网络实现。根据命题 1.4.2，ReLU 网络 $p_{\delta,M}$ 的深度为 $O(\log(1/\delta))$，权重数为 $O(\log(1/\delta) + \log M)$。因此，$h_{\mathbf{m},\mathbf{k}}$ 可以由一个深度为 $O(\log(1/\delta))$、权重数为 $c(d,k)(\log(1/\delta) + \log M)$ 的 ReLU 网络实现，其中 $c(d,k)$ 是依赖于 d 和 k、但不依赖于 ϵ 和 M 的常数。

现在我们估计 $h_{\mathbf{m},\mathbf{k}}$ 的逼近误差。注意，对所有 $i \in [d]$ 和 $\mathbf{x} \in [0,1]^d$，都有 $|\psi(3N(x_i - \frac{m_i}{N}))| \le 1$ 和 $|x_i - \frac{m_i}{N}| \le 1$。由命题 1.4.2，对任意满足 $|a| \le 1$ 和 $|b| \le M$ 的 $a, b \in \mathbb{R}$，有

$$|p_{\delta,M}(a,b)| - |b| \le |p_{\delta,M}(a,b) - ab| \le \delta.$$

将其应用于 (1.28)，我们知道所有 $p_{\delta,M}$ 的输入幅值都被 M 所控制，因为 $p_{\delta,M}$ 至多只应用了 $d + k - 2\,(< M)$ 次。

因此，我们有

$$\begin{aligned}
&\Big|h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - \phi_{\mathbf{m}}(\mathbf{x})\Big(\mathbf{x} - \frac{\mathbf{m}}{N}\Big)^{\mathbf{k}}\Big| \\
&\qquad = \Big|P^{-(d+|\mathbf{k}|_1-1)}_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - Q^{d+|\mathbf{k}|_1-1}_{\mathbf{m},\mathbf{k}}(\mathbf{x})\Big| \\
&\qquad \le \Big|P^{-(d+|\mathbf{k}|_1-1)}_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - Q^{1}_{\mathbf{m},\mathbf{k}}(\mathbf{x})P^{-(d+|\mathbf{k}|_1-2)}_{\mathbf{m},\mathbf{k}}(\mathbf{x})\Big| \\
&\qquad\quad + \Big|Q^{1}_{\mathbf{m},\mathbf{k}}(\mathbf{x})P^{-(d+|\mathbf{k}|_1-2)}_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - Q^{2}_{\mathbf{m},\mathbf{k}}(\mathbf{x})P^{-(d+|\mathbf{k}|_1-3)}_{\mathbf{m},\mathbf{k}}(\mathbf{x})\Big| \\
&\qquad\quad + \dots \\
&\qquad\quad + \Big|Q^{d+|\mathbf{k}|_1-3}_{\mathbf{m},\mathbf{k}}(\mathbf{x})P^{-2}_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - Q^{d+|\mathbf{k}|_1-3}_{\mathbf{m},\mathbf{k}}(\mathbf{x})Q^{-2}_{\mathbf{m},\mathbf{k}}(\mathbf{x})\Big| \\
&\qquad \le (d+k)\delta,
\end{aligned}$$

其中最后一个不等式是因为其上方求和中的项数不超过 $d+k-2$，且每一项都被 δ 所控制。此外，若 $\mathbf{x} \notin \operatorname{supp}(\phi_{\mathbf{m}}) := \operatorname{Closure}(\{\mathbf{x} : \phi_{\mathbf{m}}(\mathbf{x}) \geq 0\})$，即 $\mathbf{x}$ 不属于 $\phi_{\mathbf{m}}$ 的支撑集，则有 $h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) = \phi_{\mathbf{m}}(\mathbf{x})(\mathbf{x} - \frac{\mathbf{m}}{N})^{\mathbf{k}} = 0$。

现在定义

$$h(\mathbf{x}) = \sum_{\mathbf{m} \in N_d} \sum_{\mathbf{k}:|\mathbf{k}|_1 < k} a^f_{\mathbf{m},\mathbf{k}} h_{\mathbf{m},\mathbf{k}}(\mathbf{x}). \tag{1.29}$$

注意，系数 $a^f_{\mathbf{m},\mathbf{k}}$ 依赖于 f，而函数 $h_{\mathbf{m},\mathbf{k}}$ 不依赖于 f。于是对任意 $\mathbf{x} \in [0,1]^d$，有

$$\begin{aligned}
|h(\mathbf{x}) - f_N(\mathbf{x})| &= \Big| \sum_{\mathbf{m} \in N_d} \sum_{\mathbf{k}:|\mathbf{k}|_1 < k} a^f_{\mathbf{m},\mathbf{k}} \Big(h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - \phi_{\mathbf{m}}(\mathbf{x}) \Big(\mathbf{x} - \frac{\mathbf{m}}{N} \Big)^{\mathbf{k}} \Big) \Big| \\
&= \Big| \sum_{\mathbf{x} \in \operatorname{supp}(\phi_{\mathbf{m}})} \sum_{\mathbf{k}:|\mathbf{k}|_1 < k} a^f_{\mathbf{m},\mathbf{k}} \Big(h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - \phi_{\mathbf{m}}(\mathbf{x}) \Big(\mathbf{x} - \frac{\mathbf{m}}{N} \Big)^{\mathbf{k}} \Big) \Big| \\
&\leq 2^d \max_{\mathbf{x} \in \operatorname{supp}(\phi_{\mathbf{m}})} \sum_{\mathbf{k}:|\mathbf{k}|_1 < k} \Big| h_{\mathbf{m},\mathbf{k}}(\mathbf{x}) - \phi_{\mathbf{m}}(\mathbf{x}) \Big(\mathbf{x} - \frac{\mathbf{m}}{N} \Big)^{\mathbf{k}} \Big| \\
&\leq 2^d d^k (d+k) \delta,
\end{aligned}$$

其中第一个不等式是因为 $\mathbf{x}$ 至多属于 2^d 个 $\phi_{\mathbf{m}}$ 的支撑集合且 $|a^f_{\mathbf{m},\mathbf{k}}| \leq 1$，最后一个不等式是因为 $\{\mathbf{k} : |\mathbf{k}|_1 < k\}$ 的大小上界为 d^k。取

$$\delta = \frac{\epsilon}{2^{d+1} d^k (d+k)}, \tag{1.30}$$

则对所有 $\mathbf{x} \in [0,1]^d$，都有 $|h(\mathbf{x}) - f_N(\mathbf{x})| \leq \frac{\epsilon}{2}$。因此

$$\|h - f\|_{L^\infty([0,1]^d)} \leq \|h - f_N\|_{L^\infty([0,1]^d)} + \|f_N - f\|_{L^\infty([0,1]^d)} \leq \frac{\epsilon}{2} + \frac{\epsilon}{2} = \epsilon.$$

最后，注意到 (1.29) 中的 h 可以通过至多 $(N+1)^d d^k$ 个 ReLU 网络的线性组合来实现，即

$$\big\{ h_{\mathbf{m},\mathbf{k}} : \mathbf{m} \in N_d,\ |\mathbf{k}|_1 \leq k \big\}.$$

这些 ReLU 网络中的每一个都具有 $O(\log(1/\delta))$ 的深度和 $O(\log(1/\delta) + \log(d+k))$ 个权重。因此，h 的深度仍然是 $O(\log(1/\delta))$，而 h 的总权重数为

$$c(d,k) d^k (N+1)^d (\log(1/\delta) + \log(d+k)).$$

结合 (1.25) 中的 $N = \lceil 2^{-(d+1)/k} d^{-1} \epsilon^{-1/k} \rceil$ 以及 (1.30) 中的 δ，即可得到定理

中关于 h 的深度和权重数的上界。 □

定理 1.4.5 给出了如下网络规模估计：为了在 L^∞ 范数意义下对任意 $f \in F$ 实现 ϵ-逼近，需要网络规模为 $O(\epsilon^{-d/k}\log(1/\epsilon))$。该上界中关于 ϵ 的主导项是 $\epsilon^{-d/k}$，而 $\log(1/\epsilon)$ 是次要项。常数项还依赖于维数 d，但不依赖于 ϵ。这一估计在现有关于 Sobolev 空间上深度神经网络逼近理论中看起来是典型的（某些方法可能会有小幅改进，但主导项 $\epsilon^{-d/k}$ 仍然保留在估计中）。在本章末尾，我们将给出关于这一主题的更多相关参考文献。

1.5 网络结构设计

在本节中，我们将浏览若干重要的网络设计概念，以获得对深度网络在实际中如何使用的基本认识。我们首先介绍一些当前常用的激活函数示例。随后，我们讨论若干重要的网络结构，这些结构常被用作构建大型网络架构的基本模块。

此外，我们还将介绍一些有用的思想与技巧，用于设计具有特定性质的神经网络。这些方法能够使训练过程更加简单且稳定，更重要的是，在实际应用中往往能够显著提升性能与效率。我们还将展示如何利用物理信息（而非外部给定数据）来设计目标函数的思路，这一方向已成为现代科学计算中的一个重要发展趋势。

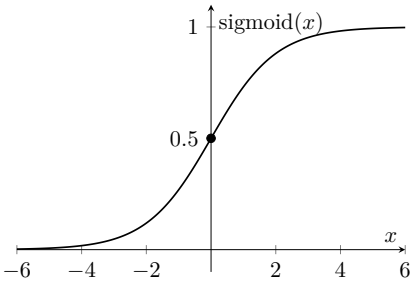
1.5.1 激活函数示例

下面，我们列出若干非线性激活函数示例，它们在现代深度网络结构中很常见。需要注意的是，激活函数通常指非仿射（因此也是非线性）函数，否则神经网络仅仅是仿射函数的复合，结果仍为仿射函数，无法逼近复杂函数。

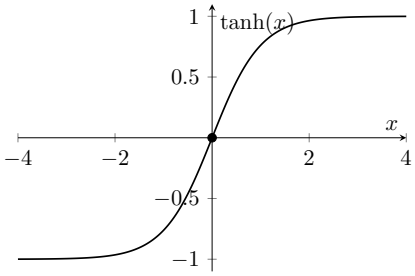
在本小节中，我们仅讨论单变量激活函数，并将在本书后续部分介绍多变量激活函数。我们前面提过，当我们说将单变量函数应用于向量时，指的是将该激活函数分别作用于向量的每一个分量，其结果仍为同维度的向量。本小节中介绍的所有激活函数均显示在图 1.7 中。接下来我们将给出这些激活函数的数学表达形式，并简要描述它们的性质。

Sigmoid 标准 sigmoid 函数定义为

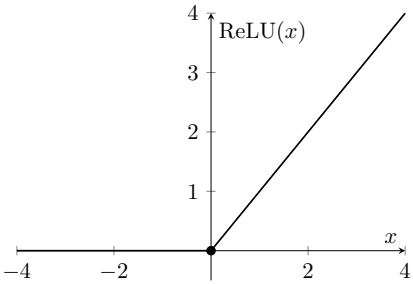
$$\mathrm{sigmoid}(x) = \frac{1}{1+e^{-x}}, \qquad \forall\, x \in \mathbb{R} \tag{1.31}$$



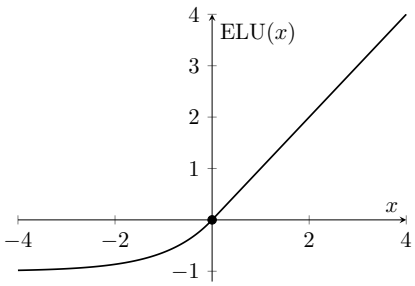
(a) Sigmoid



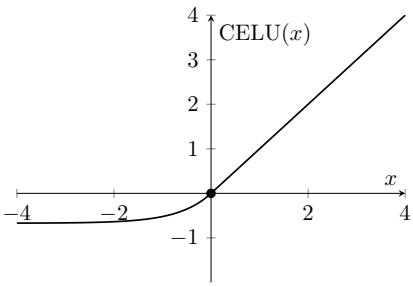
(b) tanh



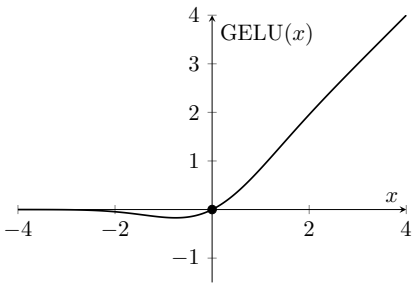
(c) ReLU



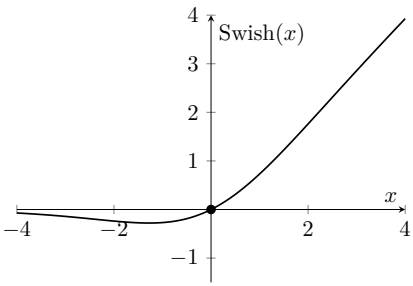
(d) ELU



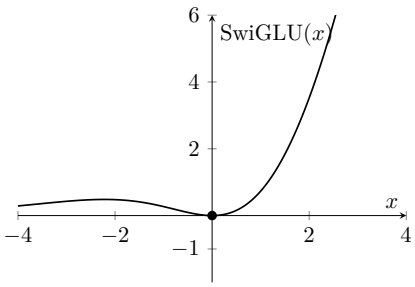
(e) CELU



(f) GELU



(g) Swish



(h) SwiGLU

图 1.7: 激活函数示例。

并如图 1.7a 所示。在文献中，sigmoid 函数通常指当 $x \to \infty$ 时趋于 1、当 $x \to -\infty$ 时趋于 0 的函数，这类函数也可以是非单调的。然而，在深度学习领域中，通常默认 (1.31) 为 sigmoid 函数。当输出已知位于开区间 $(0,1)$ 时，sigmoid 函数可作为神经网络的最后一层。然而，当 x 远离 0 时，其导数会迅速减小，这种现象称为“梯度消失”，往往会导致训练效率降低。

双曲正切函数（tanh） tanh 函数 定义为

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \qquad \forall\, x \in \mathbb{R} \tag{1.32}$$

并如图 1.7b 所示。tanh 函数 (1.32) 可视为 sigmoid 函数的一种变体，其输出范围为 $(-1,1)$。tanh 同样存在梯度消失问题，通常用于循环神经网络中的门控机制（本节后续将看到相关示例）。

修正线性单元（ReLU） ReLU 函数 定义为

$$\mathrm{ReLU}(x) = \max(0, x), \qquad \forall\, x \in \mathbb{R} \tag{1.33}$$

并如图 1.7c 所示。ReLU 函数 (1.33) 常用于分段回归中，用于描述在某一拐点前后具有不同斜率的关系。在统计建模中，当预测变量的影响发生突变时也会出现该函数。在深度学习中，ReLU 是一种非常常用的激活函数。当 $x \geq 0$ 时，它不会产生梯度消失问题。然而在 $x = 0$ 处不可导。深度 ReLU 神经网络本质上是连续的分片的仿射函数，其 Lipschitz 常数可以根据网络权重进行估计。

指数线性单元（ELU） ELU 激活函数 定义为

$$\mathrm{ELU}(x) = \begin{cases} x & \text{if } x \geq 0, \\ e^x - 1 & \text{otherwise}, \end{cases} \tag{1.34}$$

并如图 1.7d 所示。ELU 函数 (1.34) 是 ReLU 函数 (1.33) 的平滑版本。在 $x < 0$ 时，它在一定程度上缓解了梯度消失问题，并有时可以加速训练过程。该函数在深度网络中较为常见，训练稳定性通常优于 ReLU，但计算代价更高。

连续可导指数线性单元（CELU） CELU 激活函数 定义为

$$\mathrm{CELU}(x) = \begin{cases} x & \text{if } x \geq 0, \\ \alpha(e^{x/\alpha} - 1) & \text{otherwise}, \end{cases} \tag{1.35}$$

并在图 1.7e 中给出（取 $\alpha = \frac{2}{3}$）。CELU 函数 (1.35) 是 ELU 的直接推广，引入了可调超参数 $\alpha > 0$，可作为 ReLU 与 ELU 的替代方案。

高斯误差线性单元（GELU） GELU 激活函数 定义为

$$\mathrm{GELU}(x) = \frac{x}{2}\Big(1 + \mathrm{erf}\Big(\frac{x}{2}\Big)\Big), \quad \forall\, x \in \mathbb{R}, \tag{1.36}$$

其中 erf 为高斯误差函数，定义为

$$\mathrm{erf}\Big(\frac{x}{2}\Big) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt.$$

GELU 激活函数 (1.36) 如图 1.7f 所示。它是光滑且单调的，在现代 transformer 架构中广泛使用，尤其是在自然语言处理与大语言模型。相比 ReLU，它通常表现更优，在许多实际应用中逐渐流行起来。

Swish 带有超参数 $\beta > 0$ 的 swish 函数 定义为

$$\mathrm{swish}_\beta(x) = x \cdot \mathrm{sigmoid}(\beta x) = \frac{x}{1 + e^{-\beta x}}, \quad \forall\, x \in \mathbb{R}, \tag{1.37}$$

当 $\beta = 1$ 时退化为 sigmoid linear unit（SiLU）。当 $\beta = 1$ 时的 SiLU 函数如图 1.7g 所示。swish 是一种光滑的非单调激活函数，可以取负值，通常优于 ReLU，并有助于改善梯度传播。

Swish 门控线性单元（SwiGLU） SwiGLU 函数 定义为

$$\mathrm{SwiGLU}(x) = (vx + c) \cdot \mathrm{swish}(wx + b), \quad \forall\, x \in \mathbb{R} \tag{1.38}$$

其中参数为 v, w, b, c，并如图 1.7h 所示。该函数可推广至向量输入情形，此时 V, W 为矩阵，b, c 为向量，乘法变为分量乘积 $\odot$。即对于任意 d 维向量 x, y，$x \odot y := (x_1 y_1, \ldots, x_d y_d) \in \mathbb{R}^d$。SwiGLU 是现代大语言模型中最常用的门控激活函数之一。其门控机制可动态控制信息流，提高非线性表达能力。Swish 函

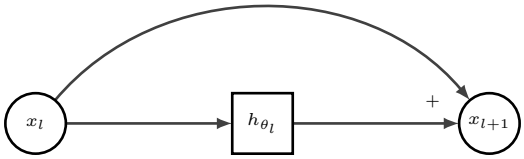


图 1.8: 残差网络（ResNet）第 l 层中的一个基本模块：$x_{l+1} = x_l + h_{\theta_l}(x_l)$，其中 $h_{\theta_l} : \mathbb{R}^d \to \mathbb{R}^d$ 是一个具有参数 θ_l 的任意（浅层或小型多层）网络，且 d 是 x_l 的维度。

数光滑，上无界而下有界。在相同参数规模下，SwiGLU 通常比 ReLU、GELU 等具有更强的建模能力。

1.5.2 网络模块示例

我们讨论若干基础神经网络模块的结构，这些模块常用于构建大型深度网络架构。

残差网络（ResNet）

尽管多层感知机（MLP）展示了通过堆叠浅层网络构建深层网络的思想，但其在实验中的表现往往不稳定。特别是，当层数增加时，训练误差与测试误差反而可能上升，这与“更深网络具有更强表达能力”的直觉相矛盾。这一现象通常被认为是深层 MLP 难以训练导致的。

残差网络（residual network，ResNet）在广泛问题中能够有效克服上述问题。在标准的 MLP 中，第 l 层实现为 $x_{l+1} = h_{\theta_l}(x_l)$。而在 ResNet 中，第 l 层定义为

$$x_{l+1} = x_l + h_{\theta_l}(x_l). \tag{1.39}$$

上式含有从 x_l 到 x_{l+1} 的跳跃连接（skip connection），所以我们仅须学习残差项 h_{θ_l} 。图 1.8 展示了这个 ResNet 模块的结构。

注意，(1.39) 要求 h_{θ_l} 保持其输入的维度，这可以通过合理设计这些隐藏层来实现。实验中，如 (1.39) 所示的 ResNet 模块通常可以堆叠到数千层，同时此类网络中的梯度消失问题在很大程度上得到了缓解。

如我们所见，ResNet 模块 (1.39) 学习的是对恒等映射的修正，而不是全新的变换。如果在某一深度处不需要额外层，则该残差模块 h_{θ_l} 可以简单地去逼近 0 ，使 (1.39) 输出近似等于输入。这一性质使得 ResNet 对冗余层更加鲁棒（robust），并提供了一种隐式正则化形式。此外，只要残差模块 h_{θ_l} 的

Lipschitz 常数保持足够小，使得整个 ResNet 构成一个可逆映射，恒等跳跃连接可以确保信息在前向与反向传播过程中不发生失真。这样，我们通常可以稳定地训练非常深的 ResNet 。

值得注意的是，ResNet 模块可以从多个不同的角度进行解释。其中一个重要观点将其与动力系统联系起来，因为 (1.39) 类似于求解常微分方程（ordinary differential equation，ODE）时前向 Euler 离散化的一步，时间步长为 1。这一视角启发了更先进的网络结构，例如具有连续时间的 Neural ODE 方法。我们将在第 3.3 节中详细讨论 Neural ODE 方法。另一个观点（如前所述）强调，(1.39) 中的残差路径在右端梯度中引入了恒等矩阵，这在适当控制 h_{θ_l} 的 Lipschitz 性质时，有助于降低梯度消失问题的风险。

ResNet 模块的简单设计使得训练非常深的神经网络成为可能，并已成为现代深度学习架构中的基础元素。这些模块如今已广泛应用于各类深度神经网络的构建，覆盖极其广泛的应用领域。

卷积神经网络（CNN）

标准的多层感知机（MLP）由于具有大量可训练权重，还存在过参数化的问题。例如，如果 MLP 的第 l 层和第 $(l+1)$ 层分别有 d_l 和 d_{l+1} 个计算单元，则权重数量至少为 $d_l d_{l+1}$。这往往在训练过程中导致许多不良后果，并且对数据量的需求过高。此外，MLP 的层结构忽略了输入相邻分量之间的相关性，从而降低了学习信息的效率。

卷积神经网络（convolutional neural network，CNN）模块是一类专门设计用于处理输入分量之间具有空间关联的数据（如文本、图像和视频）的有效网络结构。尤其是在视觉识别任务中（包括图像分类、目标检测、语义分割和视频分析），CNN 模块占据主导地位。其成功源于一种结构特性，能够利用局部空间相关性、参数共享以及分层特征学习。

在图 1.9 中，我们展示了一个简单 CNN 卷积核（中间图）作用于图像（左图）后得到的输出结果（右图）。这一思想可以很容易推广到构建更复杂的 CNN 模块。例如，CNN 模块的输入可以是具有红、绿、蓝三个通道的彩色图像，或是一个多通道特征张量（多维矩阵）。CNN 模块本身也可以具有 $C > 1$ 个通道，例如其卷积核可以表示为 $\{w_{ij}^c \in \mathbb{R} : i, j \in [3], c \in [C]\}$。卷积核的尺寸也可以更大，如 5、7 或更大。输出特征张量的尺寸由输入、卷积核大小以及卷积的应用方式共同决定。

在使用 CNN 模块时还有许多其他技巧。例如，如果希望将卷积核应用于输入的边界，可以在输入外围填充人工值（如 0），并从图 1.9 中所示的以 x_{11}

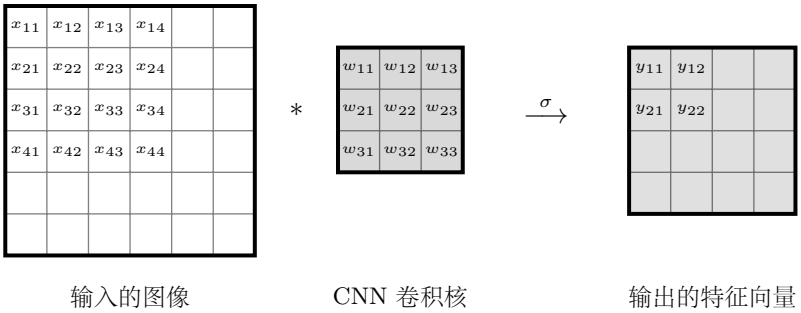


图 1.9: 一个基本卷积神经网络（CNN）模块作用于图像并生成输出的示意图。最左侧图像为一个分辨率为 6×6 的二维灰度输入图像，其像素强度为 $\{x_{ij} \in \mathbb{R} : i, j \in [6]\}$。中间为一个大小为 3×3 的卷积核，其核值为可训练参数 $\{w_{ij} \in \mathbb{R} : i, j \in [3]\}$。最右侧为大小为 4×4 的特征张量，其特征值为 $\{y_{ij} \in \mathbb{R} : i, j \in [4]\}$，表示图像与卷积核卷积后的输出。在该示例中，$3 \times 3$ 卷积核首先作用于以 x_{22} 为中心的 3×3 区域，并得到输出中的第一个特征值 $y_{11} = \sum_{i=1}^{3} \sum_{j=1}^{3} \sigma(w_{ij} x_{ij})$，其中 σ 为激活函数。在应用激活函数 σ 之前还可以加入偏置项。随后相同的卷积核作用于以 x_{23} 为中心的区域，并得到 y_{12}，依此类推。该卷积操作通常简写为 $y = \sigma(w * x)$。当多个此类模块堆叠时，前一 CNN 模块的输出特征张量作为下一模块的输入。

为中心的区域开始应用卷积核。如果希望压缩特征张量的尺寸，可以在卷积时使用更大的步长（图 1.9 中步长为 1，因此除边界外尺寸不变）。除了图 1.9 中提到的加偏置和激活函数外，还有其他常用操作，如批归一化（对多个输出求平均）和最大池化。这些示例在网络上可以轻松找到，此处不再赘述。大多数此类操作已集成于现代深度学习软件包中，可直接在编程中使用。

CNN 模块的关键在于，小型局部卷积核可以作用于整个大规模输入。通过卷积操作，卷积核在输入上滑动，并与局部区域计算内积，从而生成对特定模式（如边缘、角点或纹理）具有强响应的特征张量。这种局部连接大幅减少了参数数量，相比 MLP 中的全连接层，提高了统计效率与泛化能力，尤其在图像等高维输入中表现突出。

由于 CNN 模块中的卷积核在每个空间位置都会重复使用，因此具有平移不变性：输入的平移会导致特征张量相应平移。这一性质与自然图像高度契合，因为图像中的物体通常与其位置无关。因此，CNN 能够学习在空间位置上具有良好泛化能力的稳健特征，同时保持模型规模可控。

通过堆叠多个卷积模块，CNN 自然形成分层的数据表示。浅层通常捕捉边缘和简单纹理等低级特征，而深层则编码物体部件和语义结构等更抽象的概念。这种分层结构在一定程度上类似于哺乳动物大脑皮层中的视觉处理路径，也是 CNN 在感知任务中表现优异的重要原因。

从数学角度看，CNN 可以被视为具有强归纳能力的结构化函数逼近器。卷积操作引入局部性与平稳性假设，而网络深度则实现组合式表示。从数值角度看，由于卷积结构适合现代电脑硬件（尤其是 GPU），CNN 可以高效训练。

CNN 由于其高效性、强归纳偏置以及在数据有限或计算资源受限场景下的优异表现，具有重要意义。因此，CNN 仍在计算机视觉、信号处理和科学计算等领域中发挥核心作用。

图神经网络（GNN）

在大量现代实际应用中，数据来源于由图表示的不规则域，例如社交网络、引文网络、分子图以及物理交互系统。与图像或序列不同，图缺乏固定网格结构，其节点的邻居数量也可能不同。图神经网络（graph neural network，GNN）通过从节点、边以及整体拓扑结构中学习特征来应对这一挑战。

给定一个（有向）图，记为 $G = (V, E)$，其中 $V = \{1, \ldots, n\}$ 是节点集合，$E = \{e_{ij} = (i, j) \in V \times V\}$ 是（有向）边集合。图结构由一个加权邻接矩阵 $A = [A_{ij}]_{i,j=1}^{n} \in \mathbb{R}^{n \times n}$ 表示，其中 $A_{ij} > 0$ 表示存在一条从节点 i 指向节点 j 的边，即 $e_{ij} \in E$。在无权情形下，若 $e_{ij} \in E$，则只需取 $A_{ij} = 1$，否则取 0。每个节点 i 都对应一个行特征向量 $x_i \in \mathbb{R}^{1 \times d}$，将所有节点特征堆叠起来可得到特征矩阵 $X \in \mathbb{R}^{n \times d}$。GNN 的目标是学习同时受 X 和 G 的拓扑结构所影响的节点级、边级或图级表示。

大多数现代 GNN 都可以统一到如下的消息传递神经网络框架之下。在第 l 层，每个节点 i 都维护一个 d_l 维的隐藏表示 $h_i^l \in \mathbb{R}^{d_l}$。更新过程由两步组成：消息聚合和隐藏状态更新。第 l 层中的消息聚合步骤定义为

$$m_i^l = \sum_{j: e_{ji} \in E} \phi_l(h_i^l, h_j^l),$$

其中求和遍历节点 i 的所有邻居节点，而 $\phi_l(\cdot, \cdot)$ 是一个网络模块，用于学习如何从相连的两个节点 i 和 j 的隐藏表示 h_i^l 和 h_j^l 中传递消息（信息）。第 l 层中的隐藏表示更新步骤定义为

$$h_i^{l+1} = \psi_l(h_i^l, m_i^l),$$

其中 ψ_l 也可以由神经网络来实现，例如 MLP 或门控循环单元（GRU）。这一迭代过程使信息能够在图上进行传播，因此在经过 L 层之后，h_i^L 编码了距离节点 i 最多 L 跳范围内的节点信息。

图卷积网络（graph convolutoinal network，GCN）是 GNN 的一种广泛使用的实例。给定图的一个适当加权的邻接矩阵 $A \in \mathbb{R}^{n\times n}$，定义 $\tilde{A} := A + I_n$，以及相应的对角加权度矩阵 $\tilde{D} := \mathrm{diag}([\tilde{D}_{11}, \ldots, \tilde{D}_{nn}])$，其中 $\tilde{D}_{ii} = \sum_{j=1}^{n} \tilde{A}_{ij}$，而 I_n 是 $n \times n$ 单位矩阵。则第 l 个 GCN 层可设为：

$$H_{l+1} := \sigma\Big(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H_l W_l\Big),$$

其中 $H_0 := X \in \mathbb{R}^{n\times d}$ 是前面给出的输入节点特征矩阵，而 $H_l := [h_1^l, \ldots, h_n^l]^\top \in \mathbb{R}^{n\times d_l}$ 是第 l 个隐藏层中的节点特征矩阵。对称归一化 $\tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ 保证了数值稳定性，并平衡了不同度数节点的贡献。这里 $W_l \in \mathbb{R}^{d_l\times d_{l+1}}$ 是一个可训练的权重矩阵，σ 是诸如 ReLU 的非线性激活函数。这一形式可以解释为图上的一种 Laplacian 平滑。

GNN 已经成功应用于社交网络分析、分子性质预测、推荐系统和知识图谱。尽管其效果显著，但仍然存在一些挑战，包括过平滑、大规模图上的可扩展性问题，以及表达能力的理论极限。当前的研究正通过改进网络结构、采样策略以及与图论和谱方法建立联系来应对这些问题。

尽管表现有效，GNN 仍面临一些挑战，例如随着深度增加，节点表示会变得难以区分的过平滑问题，以及在超大规模图上的可扩展性问题。这些局限推动了大量扩展方法的发展，包括基于注意力的聚合、残差连接、基于采样的方法以及归一化技术。尽管如此，图卷积网络仍为从关系数据中学习提供了一个具有原则性且强有力的框架，并构成了许多现代图表示学习方法的基础。

循环神经网络（RNN）

在现实世界应用中，生成和观测到的数据通常表现出很强的序列性和时间相关性，例如文本、语音、音乐、视频等。循环神经网络（recurrent neural network，RNN）是一类专门用于建模此类数据的神经网络结构，其方式是维护一个随时间演化的内部状态。不同于标准的前馈神经网络，RNN 显式地引入了时间步之间的依赖关系，因此适用于时间序列预测、自然语言处理以及动力系统建模等任务。

设 $\{x_t\}_{t=1}^{T}$ 为一个输入序列，例如一个由 T 个单词、标点符号和符号按顺序构成的句子，其中 $x_t \in \mathbb{R}^d$ 是序列中第 t 个元素的数值表示。（在语言模型中，单词、标点和符号通常都表示为向量并嵌入到欧几里得空间中。）一个基本的 RNN 通常维护一个隐藏状态 $h_t \in \mathbb{R}^m$，称为**历史**（history），用于总结过去的信息。状态更新通过一个循环模块 R 来完成，它是当前状态 x_t 与历史 h_{t-1} 的函数。该 R 输出更新后的历史 h_t，并与 x_{t+1} 一起作为下一时刻 t

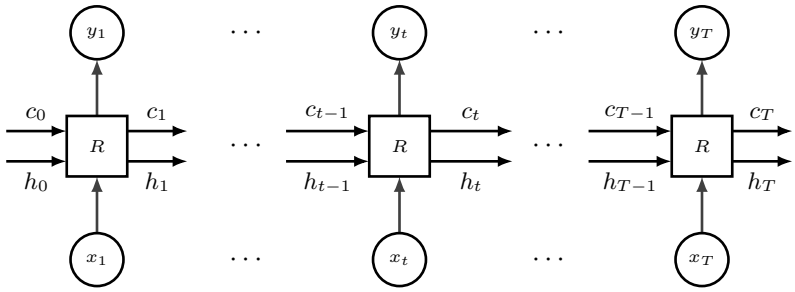


图 1.10: 标准循环神经网络（RNN）的结构。RNN 的核心组成部分是循环模块 R，它是一个被参数化为网络模块的函数。在时刻 t，R 接收当前输入数据 x_t（来自长度为 T 的时序数据 $(x_1, \ldots, x_T)$，例如由 T 个单词、标点和符号组成的句子），以及在前一时刻 $t-1$ 由 R 生成的单元状态 c_{t-1} 和历史 h_{t-1}，并计算下一个单元状态 c_t 和历史 h_t。它还可能生成一个输出 y_t，该输出可与外部给定的观测值进行比较，以训练其自身参数。所有这些输入和输出都表示为向量。这里所示的 RNN 模块 R 被重复使用，因此其结构和参数值在每个时刻 t 都相同。初始的 c_0 和 h_0 在实现中可以是具有平凡取值的占位符，但在训练中不是必须的。一个以单元状态为输入但没有输出的循环模块 R 的示例，即长短期记忆网络（LSTM），见 (1.40)。

时 R 的输入。循环模块 R 也可以设计成具有其他输入和输出，例如单元状态 c_t 和观测量 y_t。图 1.10 展示了一个循环模块 R 具有单元状态作为输入但没有观测输出的 RNN 示例。

从动力系统的角度看，RNN 模块 R 定义了一个非线性状态空间模型，它从过去的历史和状态中提取信息，以生成下一状态所需的新历史。循环结构使信息能够随时间持续存在，从而使网络能够捕捉输入序列中的时间相关性和长程依赖。然而，对 R 的重复迭代可能会在 RNN 训练中导致梯度消失或梯度爆炸。为缓解这些问题，诸如长短期记忆网络（LSTM）和 GRU 等门控结构引入了乘法门来调节信息流。这些机制改善了梯度传播，并使得对长程依赖的建模成为可能。尽管基于注意力的模型已经兴起，RNN 仍然是理解序列建模与动力系统学习的一个基础框架。

我们以 LSTM 为例来说明 RNN 的结构。在 LSTM 中，模块 $R := R_{F,I,O}$ 是一个由三个门组成的模块，分别称为遗忘门（forget gate）F、输入门（input gate）I 和输出门（output gate）O。这三个门中的每一个都是一个小型神经网络：F 是一个具有可训练权重矩阵 W_F 和偏置向量 b_F 的网络，I 具有权重矩阵 (W_I, W_C) 和偏置 (b_F, b_C)，而 O 具有权重矩阵 W_O 和偏置 b_O。这些权重和偏置的维度都被适当设定，以使得下述计算能够成立。

总体上，在以上各门定义之后，一个 LSTM 模块 R 通常被设定为如下输

入输出函数：

$$(c_t, h_t) = R_{F,I,O}(x_t, c_{t-1}, h_{t-1}) \tag{1.40}$$

对每个时刻 $t = 1, \ldots, T$，其具体计算实现为

$$\begin{aligned}
f_t &= \mathrm{sigmoid}(W_F[h_{t-1}; x_t] + b_F), \\
c_t &= f_t \odot c_{t-1} + \mathrm{sigmoid}(W_I[h_{t-1}; x_t] + b_I) \odot \tanh(W_C[h_{t-1}; x_t] + b_C), \\
h_t &= \mathrm{sigmoid}(W_O[h_{t-1}; x_t] + b_O) \odot \tanh(c_t),
\end{aligned}$$

其中激活函数 sigmoid 和 tanh 都是按分量作用的，$\odot$ 表示我们前面解释过的两个向量的分量乘积，而 $[h_{t-1}; x_t]$ 表示将 h_{t-1} 和 x_t 拼接得到的列向量。如果 R 还具有输出 y_t，那么它将被用于与某些观测数据或给定的真实值 $\hat{y}_t$ 进行比较，并出现在训练目标函数中。

1.5.3 具有特殊性质的网络

正如我们所见，神经网络结构设计具有很大的灵活性。这在实际中尤其有用，因为我们可以巧妙地将所需的函数性质直接构造进网络结构中，从而使这些性质自动得到满足。

将所需性质构造进网络的一个主要优点在于，这通常能够有效消除（或至少减少）学习最优网络参数这一优化问题中的约束。这往往会带来更快、更稳定的训练计算过程，并在实际中提升网络性能。下面我们将针对不同的指定性质介绍若干这样的设计技巧示例。

值域受约束的函数

如果我们知道待逼近的函数 $f : \mathbb{R}^d \to \mathbb{R}^n$ 的输出值位于某个给定范围内，那么我们可以将这一约束显式地加到用于参数化该函数的神经网络的最后一步操作中。

例如，如果对所有 x 都有 $f(x) \in (0,1)^n$，那么我们可以将 f 参数化为复合函数 $\mathrm{sigmoid} \circ \phi$，其中 $\phi : \mathbb{R}^d \to \mathbb{R}^n$ 是任意神经网络，而 sigmoid 函数 (1.31) 是按分量作用的。类似地，如果 $f(x) \in (-1,1)^n$，则可以使用 tanh 函数 (1.32)。如果 f 的值域是 $\mathbb{R}^n$ 中的一个长方体 $(a_1, b_1) \times \cdots \times (a_n, b_n)$，其中 $a = (a_1, \ldots, a_n)$ 和 $b = (b_1, \ldots, b_n)$ 是已知的 n 维向量，并满足对所有 $i \in [n]$

都有 $a_i < b_i$，则我们可以将 f 参数化为

$$f(x) = a + (b - a) \odot \mathrm{sigmoid}(\phi(x)),$$

或

$$f(x) = \frac{1}{2}\Big((a + b) + (b - a) \odot (\tanh(\phi(x))\Big).$$

如果 $f(x) \geq 0$，我们可以将 f 参数化为 $\max(0, \phi)$、$\max(0, \phi)^2$，或者直接取 $\frac{1}{2}|\phi|^2$。注意，只要网络 ϕ 是连续可微函数（例如 ϕ 仅使用光滑激活函数），那么后两者也是连续可微的。这些思想还可以推广到更复杂的情形。

如果 $f(x) \in \mathbb{R}^n$ 需要表示关于 n 个取值的一个概率分布，也就是说，$f(x)$ 是一个 n 桶直方图（n-bin histogram ），那么 f 的值域就是定义为

$$\Delta^n := \Big\{(y_1, \ldots, y_n) \in \mathbb{R}^n : 0 \leq y_i \leq 1,\ \sum_{i=1}^{n} y_i = 1\Big\} \subset \mathbb{R}^n$$

的概率单纯形（probability simplex）。（这个概率单纯形也记作 Δ^{n-1}，以强调它是 $\mathbb{R}^n$ 中的一个 $(n-1)$ 维子流形。）在这种情况下，我们可以将 f 参数化为复合函数 $\mathrm{softmax} \circ \phi$，其中 $\phi : \mathbb{R}^d \to \mathbb{R}^n$ 是任意网络，而 softmax 函数定义为

$$\mathrm{softmax}(w) := \frac{1}{\sum_{i=1}^{n} e^{w_i}}\Big(e^{w_1}, \ldots, e^{w_n}\Big) \in \mathbb{R}^n$$

其中 $w \in \mathbb{R}^n$。于是对所有 $x \in \mathbb{R}^d$，都有 $f(x) \in \Delta^n$。

在一些实际问题中，函数 f 还要求逐点被某个给定函数 g 所界定。例如，已知 $g : \mathbb{R}^d \to \mathbb{R}$，并且待求的 f 满足对每个 $x \in \mathbb{R}^d$ 都有 $f(x) \geq g(x)$。在这种情况下，我们可以将 f 参数化为

$$f(x) = g(x) + \frac{1}{2}|\phi(x)|^2,$$

其中 $\phi : \mathbb{R}^d \to \mathbb{R}^n$ 是任意神经网络。

在一些应用中，还会对 f 在某些指定点上的取值提出额外要求。例如，在某些最优控制问题中，我们希望寻找控制系统的 Lyapunov 函数 f。这类函数在某些指定点（称为平衡点）处已知应为正定。作为一个典型例子，考虑标准情形：平衡点设为原点 $0 \in \mathbb{R}^d$，并且 Lyapunov 函数 f 必须满足对每个 $x \in \mathbb{R}^d$ 都有 $f(x) \geq 0$，且当且仅当 $x = 0$ 时有 $f(x) = 0$。在这种情况下，我们可以将 f 参数化为

$$f(x) = g(x) + \frac{1}{2}|\phi(x) - \phi(0)|^2,$$

其中 $\phi:\mathbb{R}^d\to\mathbb{R}^n$ 是任意神经网络，而函数 g 是用户定义的 f 的非负下界。g 的例子包括 $g(x)=\frac{\alpha}{2}|x|^2$ 和 $g(x)=\alpha\log(1+|x|^2)$，其中 $\alpha>0$ 是用户定义的权重，用于决定下界函数 g 的尺度。

偶函数、奇函数与周期函数

如果已知 $f:\mathbb{R}^d\to\mathbb{R}^n$ 关于任意输入 $x=(x_1,\ldots,x_d)\in\mathbb{R}^d$ 的第 i 个分量 x_i 是偶函数，那么我们可以将 f 参数化为

$$f(x)=\phi\Big(x_1,\ldots,x_{i-1},\frac{\psi(x_i)+\psi(-x_i)}{2},x_{i+1},\ldots,x_d\Big),\tag{1.41}$$

其中 $\phi:\mathbb{R}^d\to\mathbb{R}^n$ 和 $\psi:\mathbb{R}\to\mathbb{R}$ 都是任意神经网络。容易验证，对每个 $x\in\mathbb{R}^d$，都有 $f(x_1,\ldots,x_i,\ldots,x_d)=f(x_1,\ldots,-x_i,\ldots,x_d)$。如果 f 关于多个分量都是偶函数，也可以采取同样的方法。如果 f 关于 x_i 是奇函数，那么只需要将 (1.41) 中的 $\frac{\psi(x_i)+\psi(-x_i)}{2}$ 替换为 $\frac{\psi(x_i)-\psi(-x_i)}{2}$ 即可。这一思想可以逐个作用于 x 的不同分量，使得 f 关于某些分量为偶函数，而关于另一些分量为奇函数。

如果已知 $f:\mathbb{R}^d\to\mathbb{R}^n$ 关于任意 $x=(x_1,\ldots,x_d)\in\mathbb{R}^d$ 的第 i 个分量 x_i 是 T-周期的，那么我们可以将 f 参数化为

$$f(x)=\phi\Big(x_1,\ldots,x_{i-1},\cos\Big(\frac{2\pi x_i}{T}\Big),\sin\Big(\frac{2\pi x_i}{T}\Big),x_{i+1},\ldots,x_d\Big),\tag{1.42}$$

其中 $\phi:\mathbb{R}^{d+1}\to\mathbb{R}^n$ 和 $\psi:\mathbb{R}\to\mathbb{R}$ 是任意神经网络。这一方法也可以很容易地推广到 f 关于多个分量都是周期函数的情形，并且不同分量的周期还可以不同。

具有指定边界值、初值和/或终值的函数

在某些实际应用中，例如求解偏微分方程（partial differential equation，PDE）[37]，解通常需要满足给定的边界条件、初始条件和/或终值条件。在这些情况下，我们可以将这些条件直接嵌入到用于参数化解的深度神经网络中。

例如，如果某个 PDE 的解 $u:\mathbb{R}^d\to\mathbb{R}$ 的定义域是 $\bar{\Omega}$，其中 $\bar{\Omega}$ 是开单位盒 $\Omega:=(0,1)^d\subset\mathbb{R}^d$ 的闭包，并且已知该解在边界 $\partial\Omega$ 上等于某个给定函数 g，那么我们可以将 u 参数化为

$$u(x)=g(x)+\phi(x)\prod_{i=1}^{d}x_i(1-x_i)\tag{1.43}$$

对每个 $x=(x_1,\ldots,x_d)\in\Omega$ 都成立，其中 $\phi:\mathbb{R}^d\to\mathbb{R}$ 是任意神经网络。我们可以事先在 Ω 的内部任意指定 $g(x)$ 的取值。容易验证，(1.43) 在 Ω 内部没有任何约束，并且在 $\partial\Omega$ 上始终等于 g。

虽然 (1.43) 展示了如何参数化具有边界条件的函数这一思想，但我们需要注意 $\prod_{i=1}^d x_i(1-x_i)$ 的选取必须谨慎。我们以 $d=1$ 且 $g=0$ 为例。若 $u:\mathbb{R}\to\mathbb{R}$，并且当 $x\to 0$ 时有 $u(x)=O(\sqrt{x})$，则可注意到，当 $x\to 0$ 时，有

$$\frac{u(x)}{x(1-x)}=O\Big(\frac{1}{\sqrt{x}(1-x)}\Big)\to\infty$$

因此，像 (1.43) 那样将 $u(x)$ 参数化为 $u(x)=\phi(x)\cdot x(1-x)$ 并不合适，因为这会要求 $x\to 0$ 时，有 $\phi(x)\to\infty$，然而这在典型神经网络结构下是不可能的，因为一般情况下，神经网络 ϕ 在 $\bar{\Omega}$ 上会处处连续。回到多元情形，我们应将 (1.43) 改写为

$$u(x)=g(x)+\phi(x)\cdot\sqrt{x_1}(1-x_1)\prod_{i=2}^d x_i(1-x_i) \tag{1.44}$$

或类似形式。正如我们所见，(1.44) 的选择要求我们预先估计当 x 趋于边界 $\partial\Omega$ 时 $u(x)$ 的阶。总之，我们应当对 (1.43) 作适当修改，使得 ϕ 只需逼近一个在 $\bar{\Omega}$ 上的一致有界函数——这正是神经网络能够做到的，因为其结构通常只允许其逼近连续函数。

如果时间演化 PDE 的解 $u:\bar{\Omega}\times[0,\infty)\to\mathbb{R}$ 中，Ω 是 $\mathbb{R}^d$ 的一个开子集，并且已知 u 在时刻 $t=0$ 于 $\bar{\Omega}$ 上具有初值 f_0，那么我们可以将 u 参数化为

$$u(x,t)=f_0(x)+t\phi(x,t)$$

其中 $\phi:\bar{\Omega}\times[0,\infty)\to\mathbb{R}$ 是任意神经网络，用以表示关于 x 和 t 的任意函数。如果已知的是 u 的终值，也可以类似地处理。

如果解 $u:\bar{\Omega}\times[0,T]\to\mathbb{R}$ 对某个有限的 $T>0$ 定义，并且其在 $\bar{\Omega}$ 上的初值 f_0 和终值 f_T 都已知，那么我们可以将 u 参数化为

$$u(x,t)=(T-t)f_0(x)+tf_T(x)+t(T-t)\phi(x,t),$$

这样便能自动满足初值和终值条件。也就是说，对所有 $x\in\bar{\Omega}$，都有 $u(x,0)=f_0(x)$ 且 $u(x,T)=f_T(x)$。与此同时，当 $t\in(0,T)$ 时，$u(x,t)$ 没有约束，因为 ϕ 是任意神经网络且 $t(T-t)>0$。根据当 t 趋于 0 或 T 时 $u(x,t)$ 的变化

速率，我们可能还需要像 (1.44) 中那样对上式中的 t 的指数作出修改。

1.5.4 网络训练准则

在问题已经表述清楚且网络结构已经确定之后，我们需要指定一个准则，以便求取最优的网络参数。这一准则通常被设定为一个标量值函数，称为**目标函数**（objective function），它将任意网络参数 θ（即网络中所有参数的集合）映射为一个实数。它也常被称为**损失函数**（loss function），因为在大多数情况下这一函数是需要被最小化的。此后我们称其为目标函数，因为在某些应用中我们也可能寻找这一函数的极大值点或鞍点。

求解给定目标函数的最优解（例如给定损失函数的极小值点）的方法称为优化，这也是第 二 章的主要内容。在深度学习中，这一优化过程也称为网络训练（network training），即寻找网络的良好参数，使其能够很好地解决目标问题。在本小节中，我们将讨论一些为给定问题设定合适目标函数的规则与思路。

在监督学习问题中，训练数据同时包含变量 x 和观测值或标签 y，如例 1.1.1 和例 1.1.2 所示。在这些问题中，目标函数通常基于 y 与预测结果之间的统计关系来构造，预测结果通常记为 $f_\theta(x)$，其中 f_θ 是带参数 θ 的神经网络，用于估计描述该关系的函数。例如，在关于回归的例 1.1.1 中，f_θ 被人为设定为一个仿射函数 $f_\theta(x) = w^\top x + b$，其中 $\theta = (w, b)$，对应的观测值 y 服从分布 $\mathcal{N}(f_\theta(x), s^2)$，即 y 等于 $f_\theta(x)$ 加上一些具有未知方差 s^2 的独立高斯噪声。于是我们可以基于高斯分布的极大似然估计来构造关于 θ 的目标函数。在关于分类的例 1.1.2 中，f_θ 被人为设定为 $f_\theta(x) = \mathrm{sigmoid}(w^\top x + b)$，其中 $\theta = (w, b)$，对应的标签 y 被假设服从概率分布 $\mathrm{Bernoulli}(f_\theta(x))$。于是目标函数也相应地按照极大似然估计来构造。

在无监督学习和强化学习问题中，我们通常没有标签数据 y。在这些情况下，我们需要根据问题本身的表述来设定训练目标函数，而这往往高度依赖具体问题。在第 四 章和第 五 章中，我们将看到许多为相关方法设定目标函数的不同途径。

在本小节结尾，我们给出一个近年来科学计算研究中目标函数设计的例子。其目标是通过将偏微分方程（PDE）的解参数化为深度神经网络来求解 PDE。其中的挑战在于，所考虑的 PDE 往往定义在高维空间上，例如 $\mathbb{R}^d$ 的某个紧子集，其中 $d \geq 5$。尽管数值分析学科已经发展出大量理论与方法，用于数值求解 PDE [4,143,144]，例如有限差分法、有限元法和谱方法，但在高维情形下，使用这些传统数值 PDE 方法，在计算上经常是非常困难的。困难

原因在于，这些方法通常需要对问题定义域（例如 $[0,1]^d$）进行精细的空间离散化，并在所有离散点上近似解的值。然而，离散点点数量会随着维数 d 以指数速度增长，这就是所谓的“维数灾难”（curse of dimensionality）问题，因此传统数值 PDE 方法在高维情况下，计算量太大，现代的电脑无法计算。

使用深度神经网络来逼近 PDE 的解已被证明是在缓解维数问题方面非常有效的方法。其关键思想是根据给定 PDE 来设计用于训练这些深度神经网络的目标函数，而这些 PDE 往往由问题中的物理规律决定（PDE 通常来源于物理定律）。因此，这类方法中的神经网络通常被称为**物理信息神经网络**（physics-informed neural network）。注意，在这些方法中避免了直接的空间离散化，而目标函数通常通过 Monte Carlo 积分（见附录 A.1）来近似，比较有效地解决了高维问题中的积分计算问题。下面的例子展示了这些方法是如何工作的。

例 1.5.1 (求解高维 PDE)**.** 我们考虑一个经典的 PDE，即 Poisson 方程。设 $\Omega=(0,1)^d$ 是 $\mathbb{R}^d$ 中的开单位盒，且给定 $f:\Omega\to\mathbb{R}$。则 Poisson 方程写为

$$-\Delta u(x)=f(x),\quad \forall\, x\in\Omega, \tag{1.45}$$

其中 Δ 是 Laplace 算子，即对每个 $x\in\Omega$，$\Delta u(x)=\sum_{i=1}^d \frac{\partial^2 u}{\partial x_i^2}(x)$。Poisson 方程 (1.45) 还要求 u 满足某种边界条件，否则可能存在无穷多个可能解。例如，在 Poisson 方程的 Dirichlet 边值问题中，边界函数 $g:\partial\Omega\to\mathbb{R}$ 是给定的，并要求解 u 满足对所有 $x\in\partial\Omega$ 都有 $u(x)=g(x)$。现在，我们的方法是将 $u:\bar{\Omega}\to\mathbb{R}$ 参数化为一个深度神经网络，记为 u_θ，其网络参数为 $\theta\in\mathbb{R}^m$（其中 m 是该网络中的参数个数），并设计一个适当的目标函数来训练 θ，使得 u_θ 满足 Poisson 方程 (1.45)，并且在 $\partial\Omega$ 上等于 g。

我们已经见过一种简单技巧，可以像 (1.43) 那样将 u 参数化为自动满足边界条件的深度神经网络。设 u_θ 就是以这种方式构造的，并且在 $\partial\Omega$ 上等于 g。那么剩下的问题就是构造一个关于 θ 的合适目标函数，使得 θ 达到最优时，意味着对所有 $x\in\Omega$ 都有 $-\Delta u_\theta(x)=f(x)$。

构造目标函数的一种简单方式，是基于 (1.45) 两边的误差。具体而言，我们通过求解如下关于 θ 的极小化问题来寻找 u 的最优参数 θ：

$$\min_{\theta\in\mathbb{R}^m}\int_\Omega |\Delta u_\theta(x)+f(x)|^2\,dx. \tag{1.46}$$

由此可见，如果 θ 在 (1.46) 中使目标函数值达到 0，那么 u_θ 就是满足 Poisson 方程 (1.45) 的一个解。

由于 (1.46) 中积分的精确计算不可得，并且 $\Omega \subset \mathbb{R}^d$ 的维数较高（例如 $d \geq 5$），我们使用 Monte Carlo 积分（见附录 A.1）来近似它。更精确地说，我们注意到任意（可积）函数 $h : \Omega \to \mathbb{R}$ 的积分可以按如下方式近似：

$$\int_\Omega h(x)\,dx \approx \frac{1}{N}\sum_{i=1}^{N} h(x^{(i)}), \tag{1.47}$$

其中 $\{x^{(i)} \in \Omega : i \in [N]\}$ 是从 Ω 上均匀分布中抽取的 N 个独立同分布样本。容易检验，上式右端是左端的无偏估计。关于 Monte Carlo 积分的更多细节可见附录 A.1。

根据 (1.47)，我们可以用定义在 $\mathbb{R}^m \to \mathbb{R}$ 上的函数 ℓ 来近似 (1.46) 中的目标函数：

$$\ell(\theta) := \frac{1}{N}\sum_{i=1}^{N} |\Delta u_\theta(x^{(i)}) + f(x^{(i)})|^2, \tag{1.48}$$

其中 $x^{(i)}$ 与前面一样，是从 Ω 上均匀分布中抽取的独立同分布样本。在实际中，(1.48) 中的所有项都可以计算。特别地，Laplace 算子是散度算子与梯度算子的复合，可以通过自动微分来计算，其中自动微分方法的细节将在第 2.3 节中给出。随后，我们可以应用某种数值优化算法（详见第 二 章）来求解 (1.48) 中 ℓ 的极小值点 θ，于是 u_θ 就成为 Poisson 方程 (1.45) 的近似解。这种利用深度神经网络求解高维 PDE 的方法在 [114] 中被提出，并命名为物理信息神经网络。自提出以来，它引起了广泛关注，并催生了大量后续工作、变体与应用。

在给定 Poisson 方程 (1.45) 的情况下，我们也可以设计其他目标函数来训练网络参数 θ。例如，已知 Poisson 方程 (1.45) 的解 u 也满足如下能量极小化问题：

$$\min_u \int_\Omega \Big(\frac{1}{2}|\nabla u(x)|^2 - f(x)u(x)\Big)\,dx \tag{1.49}$$

前提是 $u = g$ 在 $\partial\Omega$ 上成立。（这一点可由 PDE 基础理论中的变分法验证。其思想与第 3.1.1 节中类似。这里省略细节。）因此，如果我们像 (1.43) 那样将 $u : \Omega \to \mathbb{R}$ 参数化为满足边界值条件的神经网络，那么我们可以用其 Monte Carlo 积分来近似 (1.49) 中的积分：

$$\ell(\theta) := \frac{1}{N}\sum_{i=1}^{N} \Big(\frac{1}{2}|\nabla u_\theta(x^{(i)})|^2 - f(x^{(i)})u(x^{(i)})\Big). \tag{1.50}$$

最小化 (1.50) 中的 ℓ 同样会给出某个 θ，使得 u_θ 成为 Poisson 方程 (1.45) 的

近似解。这种将能量泛函用作训练损失函数的方法在 [34] 中被提出，并命名为 deep Ritz 方法。

与 (1.46) 需要 Laplace 算子中的二阶偏导数相比，(1.50) 的优点在于它只涉及采样点处 u 的一阶梯度。注意，当我们应用基于梯度的优化算法（例如第二 章给出的那些算法）来求解 (1.48) 或 (1.50) 中 ℓ 的极小值点时，在优化过程的每一次迭代中，还需要关于 θ 的另一重梯度。

然而需要注意的是，只有少数 PDE 已知具有像 (1.49) 这样的能量形式。另一种途径是，PDE 的理论与应用往往依赖于其弱形式（weak formulation）。事实上，现实应用中的许多 PDE 只具有弱形式意义下的解（这些解称为弱解，weak solution）。在这种情况下，我们并不期望找到像 (1.45) 中那样二次可微的强解（也称经典解），因为它们并不存在。（不过，如果强解确实存在，那么它也是弱解。因此，当我们考虑 PDE 的弱形式时，并不会遗漏任何强解。）

Poisson 方程 (1.45) 的弱形式可以如下导出。同样地，我们先假设 u 满足边界条件 $u = g$ 于 $\partial\Omega$ 上。将任意测试函数 $\psi \in C_0^\infty(\Omega)$ 乘到 (1.45) 两边，其中 $C_0^\infty(\Omega)$ 表示在 Ω 中光滑并且在边界 $\partial\Omega$ 上取零值的函数集合，然后对两边积分，得到

$$-\int_\Omega \Delta u(x)\psi(x)\,dx = \int_\Omega f(x)u(x)\,dx. \tag{1.51}$$

对左侧做分部积分并整理 (1.51)，便得到 Poisson 方程 (1.45) 的弱形式：

$$\int_\Omega \Big(\nabla u(x)\cdot\nabla\psi(x) - f(x)\psi(x)\Big)\,dx = 0, \qquad \forall\,\psi \in C_0^\infty(\Omega). \tag{1.52}$$

基于 (1.52)，我们可以将 ψ 参数化为一个任意的边界值为零的神经网络 ψ_η，其参数为 η，并尝试通过求解如下鞍点问题来寻找最优参数 θ 和 η：

$$\min_\theta \max_\eta \frac{1}{2N}\Big|\sum_{i=1}^N \Big(\nabla u_\theta(x^{(i)})\cdot\nabla\psi(x^{(i)}) - f(x^{(i)})\psi(x^{(i)})\Big)\Big|^2, \tag{1.53}$$

其中 $\{x^{(i)} \in \Omega : i \in [N]\}$ 仍然是从 Ω 上均匀分布中抽取的 N 个独立同分布样本。一旦我们找到最优的 θ，相应的 u_θ 就是 (1.52) 的弱解。在实际中，还存在许多数值技巧，可以更高效、更稳定地处理优化问题 (1.52)。

(1.53) 的优点在于目标函数中只出现梯度。更重要的是，它使我们能够在弱解存在时求得 Poisson 方程 (1.45) 的弱解。这种利用给定 PDE 的弱形式并将测试函数参数化为对抗网络的方法在 [159] 中被提出，并命名为弱对抗网络（weak adversarial network）。

1.6 备注与参考文献

1.6.1 关于网络结构设计的备注

我们观察到，关于深度神经网络逼近能力的理论研究与其实际设计和部署之间存在显著差距。从理论角度看，研究通常关注神经网络能够逼近哪些函数类、如何在严格的数学准则下评估这种逼近，以及为了达到给定精度需要多大的网络。相比之下，实践研究主要强调高效网络结构的设计，通常在启发式方法的指导下，用于解决特定任务。这包括图像处理中被广泛采用的 U-Net [122] 以及自然语言处理中使用的 transformer [150]，它们都是高效且强大的网络结构，但本书并未讨论。然而，关于这些结构逼近误差的基础理论问题却很少被深入研究。

这种差距是有原因的。深度学习在实践上的显著成功鼓励研究者去处理广泛的实际问题，并发展新的方法，以将神经网络的适用性扩展到先前被认为难以处理的任务上。这些进展在很大程度上得益于海量数据集的可获得性以及与几十年前相比计算能力的巨大提升。值得注意的是，许多现代深度网络在表示能力方面缺乏正式的理论保证，而且它们的规模通常远小于逼近理论所给出的理论界——通常是上界。一个体现这种张力的代表性例子是：理论界仍在持续努力研究深度网络是否能够在一般情形下克服维数灾难，而实践者则基本绕开这一问题，转而实现那些具有高度表达能力并在众多应用领域中表现出强大性能的网络结构。目前尚不清楚理论与实践之间的这一鸿沟最终是否会被弥合；然而，这种差距的存在并未显著阻碍任一方向上的进展。

归根结底，更现实的看法或许是承认：为了获得实际上的高效性，设计多样且精心构造的网络架构是必要的。在自然界中，动物物种的多样性使其能够在特定能力上实现专门化——例如卓越的力量、高速运动、飞行能力或高级视觉感知，而这些能力都依赖于适应相应功能的神经系统。类比而言，我们有理由期待，人工神经网络同样应通过面向任务的专门设计原则来构建，以实现最佳性能。

1.6.2 普适逼近理论的文献

普适逼近定理是神经网络理论中的一个基础性结果，它表明在适当条件下，神经网络能够以任意精度逼近一大类函数。奠基性工作 [30, 69] 证明了前馈神经网络可以一致逼近定义在紧域上的任意连续函数。这一结果随后在 [35, 67, 68, 70] 中被推广到更广泛的激活函数类别与逼近情形。在 [83] 中，这一观点被进一步澄清：其证明了一个前馈网络是普适逼近器当且仅当其激活函

数不是多项式。特别地，关于使用 sigmoid 激活函数 [8]、周期函数 [97,98] 以及一般激活函数 [70] 的神经网络逼近能力都已建立。

关于网络规模和误差的显式定量逼近理论已被广泛研究。在 [96] 中，这类结果对光滑函数和解析函数也成立。例如，对于任意定义在 $W^{k,p}(\Omega)$ 中的函数（其中 $p \in [1,\infty]$ 且 $\Omega \subset \mathbb{R}^d$ 为紧集），若希望在 L^p 范数下以误差 $\epsilon > 0$ 进行逼近，则文中表明一个规模为 $O(\epsilon^{-d/k})$、具有非多项式光滑激活函数的浅层网络就足够了 [96]。在 [109] 中，对浅层 ReLU 网络也建立了类似的估计。第 1.4 节中给出的证明遵循 [158]，该文针对 $W^{k,\infty}(\Omega)$ 中的函数，利用深度 ReLU 网络给出了这样的估计。上述估计仍然遭受维数灾难，因为所估计的网络规模会随着维数 d 指数级增长。已有结果表明，在稀疏网格设定下，这一问题可以得到缓解 [100]。如果待逼近的函数已知具有组合结构，那么这一问题还可以被显著减轻 [110]。这一领域仍在持续发展，并不断出现更优的估计结果。

总而言之，普适逼近定理保证了神经网络的表达能力，而近些年里新的结果则阐明了深度、激活函数以及逼近效率的关系。这些结果共同构成了深度神经网络在多种应用领域中能够成功的重要理论基础。

第二章　网络训练

在一个深度学习项目中，一旦网络结构和用于训练的目标函数已经确定，下一项主要任务就是确定最优的网络参数，例如通过最小化目标函数（在这种情况下也称为损失函数）来实现。这一过程通常称为**网络训练**（network training）。用于完成这一任务的数学技术属于**优化**（optimization）这一广义范畴，而优化是应用数学与计算数学中的一个核心主题。

数值优化是应用数学的一个重要领域，其重点在于发展具有可证明收敛性质的算法，以求解参数化的目标函数的极大值点或极小值点。本章介绍优化的基本理论基础，例如最优性条件，以及自动微分；后者在现代深度学习实现中高效计算导数方面起着关键作用。此外，我们还将介绍若干经典的无约束优化算法，例如梯度下降法（包括线搜索策略）、Newton 方法和拟 Newton 方法，以及约束优化方法，例如 Lagrange 算法。本部分的讨论仅限于确定性优化（deterministic optimization）的情形，即目标函数及其关于优化变量的梯度都可以被直接计算。

我们还将介绍随机优化（stochastic optimization）的基本概念，它是了训练深度神经网络时占主导地位的一类方法。特别地，我们将讨论几种广泛使用的随机优化算法，包括随机梯度方法、基于动量的变体以及自适应优化方案。

记号说明　在本章中，我们遵循优化中的记号，将目标函数记为 $f:\mathbb{R}^n\to\mathbb{R}$，将待优化变量记为 $x\in\mathbb{R}^n$，并假设约束集 Ω 是 $\mathbb{R}^n$ 中的闭集。

我们通常把一个优化问题写成最小化问题。在这种情况下，目标函数 f 也称为损失函数，而最优解称为极小值点。在设计优化问题时，我们通常需要保证 $f^*:=f(x^*)=\min_{x\in\Omega}f(x)$ 是一个有限数（否则可能无解）。我们将在第 2.1 节中给出更多细节。

我们经常使用优化方法来寻找神经网络的最优参数，这一过程称为**网络训练**。因此，本章中的目标函数 f 和变量 x 在网络训练中通常分别对应于损失函数 ℓ 和网络参数 θ。

2.1 优化中的基本概念

正如我们前面提到的，一个典型的优化问题写成如下形式：

$$\min_{x\in\Omega} \ f(x), \tag{2.1}$$

其中 $\Omega \subset \mathbb{R}^n$ 称为**约束集**（或**可行集**），$x \in \Omega$ 称为**变量**，而 $f : \Omega \to \mathbb{R}$ 称为**目标函数**（当它需要被最小化时，也称为**损失函数**或**代价函数**）。(2.1) 的目标是在 Ω 中找到 f 的一个极小值点 x^*，即对所有 $x \in \Omega$ 都有 $f(x^*) \le f(x)$。由于最大化目标函数 f 等价于最小化 $-f$，因此在本章其余部分中，我们一般都将优化问题表述为最小化问题。

优化问题有若干种典型的分类方式，下面列出几种。

连续优化与离散优化 离散优化通常指 (2.1) 中的约束集 Ω 是有限集或可数无穷集的情形（例如 $\Omega = \mathbb{N}$，即自然数集）。相比之下，在连续优化问题中，约束集 Ω 是 $\mathbb{R}^n$ 中一个（或若干个）连通区域的并集。在这类区域中，通常除了边界点外，Ω 中的所有点都是内点。这里，x 是 Ω 的一个内点是指存在一个包含于 Ω 中的、以 x 为中心的小开邻域。等价地，x 是一个内点，是指存在 $r > 0$ 使得：

$$B_r(x) := \{y \in \mathbb{R}^n : |y - x| < r\} \subset \Omega. \tag{2.2}$$

集合 $B_r(x)$ 称为以 x 为中心、半径为 r 的**开球**。Ω 的内点集合记为 $\mathrm{int}(\Omega)$。

注意，适当定义的优化问题中的约束集几乎总是闭的，否则 Ω 可能不包含 f 的极小值点。例如，$f(x) = x$ 在 $\Omega = (0, 1) \subset \mathbb{R}$ 中没有极小值点。

通常，与离散优化相比，连续优化问题更容易求解，因为我们可以利用 f 在 x 邻域（例如 $B_r(x)$）中的正则性性质（如连续性、可微性和凸性）来获取有关邻域中其它点的信息。在本书中，我们只考虑连续优化，因为深度学习中的几乎所有网络训练问题都可以表述为或近似为连续优化。

无约束优化与有约束优化 无约束优化指的是 $\Omega = \mathbb{R}^n$ 的情形。在有约束优化中，Ω 可以是 $\mathbb{R}^n$ 的一个显式给定子集，例如 $[-1, 1]^n$，或者更一般地由一组**不等式**和/或**等式**约束描述为

$$\Omega = \{x \in \mathbb{R}^n : g(x) \le 0, \ h(x) = 0\}. \tag{2.3}$$

在 (2.3) 中，我们看到约束集 Ω 由

$$\text{不等式约束:} \qquad g(x) \le 0, \tag{2.4a}$$

$$\text{等式约束:} \qquad h(x) = 0, \tag{2.4b}$$

所决定，其中函数 $g : \mathbb{R}^n \to \mathbb{R}^k$ 和 $h : \mathbb{R}^n \to \mathbb{R}^m$ 定义为

$$g(x) = \begin{pmatrix} g_1(x) \\ \vdots \\ g_k(x) \end{pmatrix} \in \mathbb{R}^k \qquad \text{和} \qquad h(x) = \begin{pmatrix} h_1(x) \\ \vdots \\ h_m(x) \end{pmatrix} \in \mathbb{R}^m.$$

注意，$z \le 0$（$z = 0$）表示向量 z 的每一个分量都非正（都等于 0）。还要注意，等式约束 $h(x) = 0$ 可以化为两个不等式约束 $h(x) \le 0$ 和 $-h(x) \le 0$ 的组合。不过，我们很少这样做，因为一般来说等式约束比不等式约束更容易处理。

显然，一般而言无约束优化问题比有约束优化问题更容易处理。虽然本章我们会研究无约束和有约束优化问题的性质，但在网络训练中，我们通常更倾向于将问题表述为或近似为无约束优化。这可以通过多种方式实现，并且依赖具体情形；我们将在本章后面详细讨论。

凸优化与非凸优化 如果对任意 $x, y \in \Omega$，都有

$$(1-t)x + ty \in \Omega, \qquad \forall\, t \in [0,1], \tag{2.5}$$

则称 Ω 是 $\mathbb{R}^n$ 中的一个**凸集**（convex set）。换言之，如果对 Ω 中的任意两点 x, y，连接 x 和 y 的整条线段都包含在 Ω 中，则 Ω 是一个凸集。

设 Ω 是 $\mathbb{R}^n$ 的一个凸子集，且 $f : \Omega \to \mathbb{R}$ 是定义在 Ω 上的函数。若

$$f((1-t)x + ty) \le (1-t)f(x) + tf(y), \qquad \forall\, t \in [0,1].$$

则称 f 是一个**凸函数**。换言之，当且仅当对任意两点 $x, y \in \Omega$，以 $(x, f(x))$ 和 $(y, f(y))$ 为端点的割线段在限制到线段 $\{(1-t)x + ty : t \in [0,1]\}$ 上时，总位于函数 f 的图像之上或与之重合，我们就称函数 f 是凸的。图 2.1 给出了一个凸函数（左）和一个非凸函数（右）的示例。若 $-f$ 是凸函数，则称 f 为**凹函数**。

当 Ω 是凸集且 f 是凸函数时，问题 (2.1) 称为一个凸优化（convex optimization）问题。若 Ω 由 (2.4) 那样的不等式约束 $g(x) \le 0$ 和等式约束 $h(x) = 0$

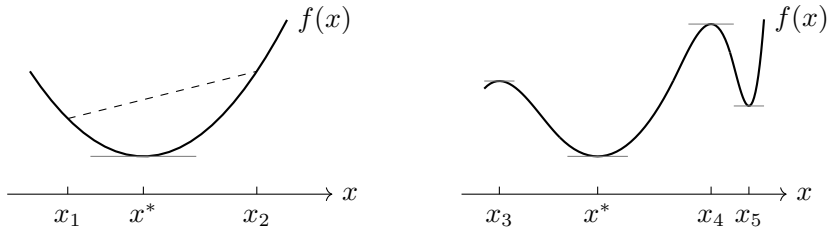


图 2.1: （左）一个凸函数 f。任意临界点，例如 x^*，如果在该点处 f 的切线是水平的（即若 f 在 x^* 处可微，则 $\nabla f(x^*) = 0$），那么它必定是 f 的全局极小值点。对任意 x_1 和 x_2，连接 $(x_1, f(x_1))$ 和 $(x_2, f(x_2))$ 的线段（虚线）在 x_1 与 x_2 之间始终位于 $f(x)$ 之上或与之重合；（右）一个非凸函数 f。临界点可以是局部极小值点，如 x^* 和 x_5，也可以是局部极大值点，如 x_3 和 x_4，或者是鞍点（此图未画出）。在图中所示区间内，x^* 是全局极小值点。

共同确定，则通常要求所有的 g_i 都是凸函数，而所有的 h_j 都是仿射函数，其中 $i \in [k]$ 且 $j \in [m]$。这样的要求是 Ω 为凸集的一个充分但非必要条件。之所以有这一惯例，是因为当 g_i 为凸函数时，其下水平集 $\{x \in \mathbb{R}^n : g_i(x) \le 0\}$ 必为凸集，而凸集的交 $\{x : g(x) \le 0\} = \cap_{i=1}^k \{x : g_i(x) \le 0\}$ 仍然是凸集。出于同样的原因，h_j 和 $-h_j$ 都需要是凸的（因为 $h(x) = 0$ 等价于 $h(x) \le 0$ 和 $-h(x) \le 0$），这意味着对每个 j，h_j 必须是仿射函数。

凸函数具有许多重要且有用的性质。因此，围绕凸优化已经发展出了大量理论与算法结果。然而，在深度学习应用中，优化问题（尤其是网络训练）几乎总是非凸的。因此，在本书中我们将主要考虑非凸优化。

凸性与非凸性会显著影响我们对于获得优化问题“最好”或“最优”解的期待。为说明这一点，我们先定义局部最优与全局最优。如果存在 $r > 0$ 使得

$$f(x) \ge f(x^*), \qquad \forall\, x \in \Omega \cap B_r(x^*). \tag{2.6}$$

则称 x^* 是问题 (2.1) 的一个**局部极小值点**（local minimizer）。此外，如果 (2.6) 中的等号仅当 $x = x^*$ 时成立，则称 x^* 是一个**严格**（strict）局部极小值点。如果

$$f(x) \ge f(x^*), \qquad \forall\, x \in \Omega. \tag{2.7}$$

则称 x^* 是问题 (2.1) 的一个**全局极小值点**（global minimizer）。如果 (2.7) 中的等号仅当 $x = x^*$ 时成立，则称 x^* 是 (2.1) 的严格全局极小值点。类似地，我们也可以定义 f 在 Ω 中的（严格）局部和全局极大值点。图 2.1 也给出了一些局部/全局极小值点和极大值点的示例。显然，一个全局极小值点必然是

局部极小值点，但反之不一定成立。

我们指出，凸优化的一个主要性质是：任意局部极小值点都是全局极小值点，正如下述命题所示。

命题 2.1.1. 设 $\Omega \subset \mathbb{R}^n$ 是一个凸集，且 $f : \Omega \to \mathbb{R}$ 是一个凸函数。如果 $x^* \in \Omega$ 是 f 的一个局部极小值点，那么 x^* 必然是 f 在 Ω 中的一个全局极小值点。

证明. 由于 x^* 是 f 在 Ω 中的一个局部极小值点，根据定义可知存在 $r > 0$ 使得 (2.6) 成立。

现在我们使用反证法。假设存在另一点 $x \in \Omega$ 使得 $f(x) < f(x^*)$。取任意 $\alpha \in (0, \min\{r, |x - x^*|\})$，并设 $t = \alpha/|x - x^*|$ 以及 $y = (1-t)x^* + tx$。那么 $t \in (0,1)$，$y \in \Omega$（因为 Ω 是凸集且 $x, x^* \in \Omega$），并且

$$|y - x^*| = t|x - x^*| = \alpha < r,$$

这意味着 $y \in B_r(x^*)$。因此，我们得到

$$f(y) = f((1-t)x^* + tx) \le (1-t)f(x^*) + tf(x) < f(x^*),$$

其中第一个不等式来自 f 的凸性，第二个不等式来自 $f(x) < f(x^*)$ 且 $t \in (0,1)$。这与 (2.6) 矛盾。 □

2.2 最优性条件

为了为讨论优化问题解的最优性条件（optimality conditions）作准备，我们先固定一些关于梯度、Hessian 矩阵以及链式法则（chain rule）的书写格式，这将在本节其余部分中使表述更加方便。

梯度与链式法则的格式 设 $f : \mathbb{R}^n \to \mathbb{R}^m$ 可微。若 $m = 1$，则我们将 f 在 x 处的**梯度**（gradient）记为一个行向量

$$\nabla f(x) = \big(\partial_1 f(x), \ldots, \partial_n f(x)\big) \in \mathbb{R}^{1\times n}, \tag{2.8}$$

其中 $\partial_i f(x) := \frac{\partial f}{\partial x_i}(x)$。若 $m > 1$ 且我们将 $f(x) = (f_1(x), \ldots, f_m(x))^\top \in \mathbb{R}^m$ 记为列向量，则 f 在 x 处的 **Jacobian 矩阵**是一个 $m \times n$ 矩阵

$$\nabla f(x) = \begin{pmatrix} \nabla f_1(x) \\ \vdots \\ \nabla f_m(x) \end{pmatrix} = \begin{pmatrix} \partial_1 f_1(x) & \partial_2 f_1(x) & \cdots & \partial_n f_1(x) \\ \partial_1 f_2(x) & \partial_2 f_2(x) & \cdots & \partial_n f_2(x) \\ \vdots & \vdots & \ddots & \vdots \\ \partial_1 f_m(x) & \partial_2 f_m(x) & \cdots & \partial_n f_m(x) \end{pmatrix} \in \mathbb{R}^{m \times n}. \tag{2.9}$$

注意，当 $m = 1$ 时，(2.9) 中 Jacobian 的大小与 (2.8) 中梯度的大小是一致的。保持这一格式将有助于我们在下面对函数复合应用链式法则时跟踪相关矩阵的维度。

进一步地，设 $g : \mathbb{R}^m \to \mathbb{R}^k$ 是另一个可微函数。则 $g \circ f : \mathbb{R}^m \to \mathbb{R}^k$ 也是可微的。若我们对 $g \circ f$ 应用链式法则，应得到一个 $k \times n$ 的 Jacobian 矩阵，它按照**从左到右**的顺序书写，即先作用在外层函数 g 的梯度，再作用在内层函数 f 的梯度：

$$\underbrace{\nabla(g \circ f)(x)}_{k \times n} = \underbrace{\nabla_f g(f(x))}_{k \times m} \underbrace{\nabla f(x)}_{m \times n}, \tag{2.10}$$

其中每个矩阵的大小都标在相应的大括号下方。类似地，若还有 $h : \mathbb{R}^k \to \mathbb{R}^l$，则 $h \circ g \circ f : \mathbb{R}^n \to \mathbb{R}^l$，并按照上面的从左到右规则得到

$$\underbrace{\nabla(h \circ g \circ f)(x)}_{l \times n} = \underbrace{\nabla_g h(g(f(x)))}_{l \times k} \underbrace{\nabla_f g(f(x))}_{k \times m} \underbrace{\nabla f(x)}_{m \times n}.$$

同样的规则也适用于三个以上函数的复合。

若 $f : \mathbb{R}^n \to \mathbb{R}$ 二次可微，则我们将 f 在 x 处的 **Hessian 矩阵**记为

$$\nabla^2 f(x) = \begin{pmatrix} \partial_{11}^2 f(x) & \partial_{12}^2 f(x) & \cdots & \partial_{1n}^2 f(x) \\ \partial_{21}^2 f_2(x) & \partial_{22}^2 f(x) & \cdots & \partial_{2n}^2 f(x) \\ \vdots & \vdots & \ddots & \vdots \\ \partial_{n1}^2 f_m(x) & \partial_{n2}^2 f(x) & \cdots & \partial_{nn}^2 f(x) \end{pmatrix} \in \mathbb{R}^{n \times n}, \tag{2.11}$$

其中 $\partial_{ij}^2 f(x) := \frac{\partial^2 f}{\partial x_i \partial x_j}(x)$。我们知道，若 f 在 x 处二次连续可微，则 $\nabla^2 f(x)$ 是对称矩阵。

我们用 “tr” 表示一个方阵的**迹**（trace）。若 $f : \mathbb{R}^n \to \mathbb{R}^n$ 是一个可微向量

场，则 f 在 x 处的**散度**（divergence）定义为

$$\nabla \cdot f(x) := \mathrm{tr}(\nabla f(x)) = \sum_{i=1}^{n} \partial_i f_i(x), \tag{2.12}$$

它是一个标量。若 $f : \mathbb{R}^n \to \mathbb{R}$ 是一个二次可微的标量值函数，则 f 在 x 处的 **Laplacian** 定义为

$$\Delta f(x) := \mathrm{tr}(\nabla^2 f(x)) = \sum_{i=1}^{n} \partial_{ii}^2 f(x). \tag{2.13}$$

2.2.1 最优性条件的一般原理

给定 $\mathbb{R}^n$ 的一个子集 Ω，若非零向量 $v \in \mathbb{R}^n$ 满足存在 $r > 0$ 使得对所有 $\alpha \in [0, r)$ 都有 $x + \alpha v \in \Omega$，则称 v 是点 $x \in \Omega$ 处的一个**可行方向**（feasible direction）。设 $f : \Omega \to \mathbb{R}$ 可微。我们将 f 在 x 处沿方向 v 的**方向导数**（directional derivative）记为

$$D_v f(x) := \langle \nabla f(x), v \rangle = \frac{d}{d\alpha} f(x + \alpha v)\Big|_{\alpha=0} = \lim_{\alpha \to 0^+} \frac{f(x + \alpha v) - f(x)}{\alpha}, \tag{2.14}$$

它是把 f 限制到一条射线 $\{x + \alpha v : \alpha \geq 0\}$ 上时，在 x 点的导数。

一阶必要条件 若 $x \in \Omega$ 是 f 的一个局部极小值点，则 f 在 x 处沿任意可行方向 v 的方向导数必然是非负的。这称为可微函数 f 下优化问题 (2.1) 的局部极小值点的**一阶必要条件**（first-order necessary condition）。

注意，若 x 是 Ω 的一个内点，则对某个 $r > 0$ 有 $B_r(x) \subset \Omega$。因此，我们知道任意非零向量 $v \in \mathbb{R}^n$ 都是 $x \in \Omega$ 处的可行方向，因为对任意 $\alpha \in [0, r/|v|)$ 都有 $x + \alpha v \in B_r(x) \subset \Omega$。这就意味着对任意 $v \in \mathbb{R}^n$ 都有 $D_v f(x) = \langle \nabla f(x), v \rangle = 0$，从而 $\nabla f(x) = 0$。因此，一个内点局部极小值点必定是 f 的一个临界点。因此，在优化问题中我们常常寻找 f 的临界点，这是因为 f 的所有即是内点又是局部极小值的点，都一定是临界点。这一事实在无约束情形 $\Omega = \mathbb{R}^n$ 下尤其有用，因为此时每个点都是内点。

二阶必要条件 设 $f : \mathbb{R}^n \to \mathbb{R}$ 二次连续可微。若 $x \in \Omega$ 是 f 的一个局部极小值点，且 v 是满足 $D_v f(x) = \langle \nabla f(x), v \rangle = 0$ 的一个可行方向，则由 Taylor

定理，作为 α 的函数，$f(x+\alpha v)$ 满足

$$0 \le f(x+\alpha v) - f(x) = \frac{\alpha^2}{2} v^\top \nabla^2 f(x+\xi\alpha v) v \tag{2.15}$$

其中对任意足够小的 $\alpha > 0$，某个 $\xi = \xi(\alpha, x, v) \in [0,1]$ 成立。由于 $\nabla^2 f$ 连续，我们知道当 $\alpha \to 0^+$ 时，(2.15) 蕴含

$$v^\top \nabla^2 f(x) v \ge 0. \tag{2.16}$$

这个 (2.16) 称为优化问题 (2.1) 的**二阶必要条件**（second-order necessary condition）。

注意，若 x 是 Ω 的一个内点且是 f 的局部极小值点，则我们知道 $\nabla f(x) = 0$，并且任意方向 v 都是可行方向。因此，由 (2.16) 可推出 $\nabla^2 f(x) \succeq 0$，即 $\nabla^2 f(x)$ 是半正定矩阵。因此，当 x 是 (2.1) 中 Ω 的一个内点时，$\nabla^2 f(x) \succeq 0$ 是局部极小值点 x 的二阶必要条件。若进一步有 $\nabla^2 f(x) \succ 0$，则 x 是 f 的一个严格局部极小值点，而我们称 $\nabla^2 f(x) \succ 0$ 为 f 的局部极小值点 x 的二阶充分条件。

备注. 我们指出，一阶与二阶必要条件对于一般的非凸函数而言**并非**充分条件。例如，对定义为 $f(x) = x^3$ 的函数 $f: \mathbb{R} \to \mathbb{R}$，有 $f'(0) = f''(0) = 0$，这意味着 f 在 $x = 0$ 处满足这两个条件。然而，$x = 0$ 是 f 的一个鞍点，而不是局部极小值点或极大值点。

上述一阶与二阶必要条件给出了局部极小值点的简明刻画，特别是在这些点是 Ω 的内点时尤为有信息量。然而，在实际中，当局部极小值点 x 位于约束优化问题中 Ω 的边界上时，我们无法直接验证这些条件。这是因为很难确定 x 处所有可行方向的集合，并逐一检查这些必要条件。因此，我们需要对约束优化而言更为实用的必要条件，下面我们就来说明。

2.2.2 必要最优性条件

我们首先考虑只有等式约束时优化问题的一阶和二阶必要条件：

$$\min_{x \in \mathbb{R}^n} \quad f(x), \tag{2.17a}$$

$$\text{s.t.} \quad h(x) = 0. \tag{2.17b}$$

我们要求约束函数 $h:\mathbb{R}^n\to\mathbb{R}^m$ 满足 $m\le n$；否则约束要么是过度冗余的（有些约束是冗余的，可以删去），要么是过定的（约束过多导致约束集 Ω 为空），此时我们可以修改 h，并将问题 (2.17) 转化为一个满足 $m\le n$ 的等价问题。在 $m\le n$ 的前提下，我们知道 Jacobian 矩阵 $\nabla h(x)$ 的秩在任意 x 处都不会超过 m。现在我们引入正规点的定义。

定义 2.2.1 (正规点)**.** 若 h 在 x 处的 Jacobian 矩阵具有满行秩 m，则称 x 是 (2.17b) 的一个**正规点**。

定义 2.2.2 (切空间)**.** 记 (2.17b) 中的约束集为 $\Omega=\{x\in\mathbb{R}^n:h(x)=0\}$。则 Ω 在 $x\in\Omega$ 处的**切空间**定义为

$$T(x)=\{v\in\mathbb{R}^n:\nabla h(x)v=0\}\subset\mathbb{R}^n. \tag{2.18}$$

也就是说，$T(x)$ 是矩阵 $\nabla h(x)$ 的行空间的正交补。

引理 2.2.3. *设 $x\in\Omega=\{x\in\mathbb{R}^n:h(x)=0\}$ 是一个正规点。则 $v\in T(x)$ 当且仅当存在一条曲线 $c:(-\delta,\delta)\to\Omega$，使得 $c(0)=x$ 且 $c'(0)=v$。*

证明. 我们先证明必要性。设 $c(t)$ 是一条满足上述条件的曲线。则对所有 $t\in(-\delta,\delta)$，都有 $c(t)\in\Omega$，即 $h(c(t))=0$。对 t 求导，得到

$$0=\frac{d}{dt}h(c(t))\Big|_{t=0}=\nabla h(c(0))c'(0)=\nabla h(x)v.$$

因此，由 (2.18) 可知 $v\in T(x)$。

接下来证明充分性。设 $x\in\Omega$ 是一个正规点，且 $v\in T(x)$。定义 $\bar h:\mathbb{R}\times\mathbb{R}^m\to\mathbb{R}^m$ 为

$$\bar h(t,w)=h(x+tv+\nabla h(x)^\top w).$$

则有 $\bar h(0,0)=h(x)=0$，并且

$$\partial_w\bar h(t,w)|_{(t,w)=(0,0)}=\nabla h(x)\nabla h(x)^\top\succ 0,$$

这是因为 $\nabla h(x)\in\mathbb{R}^{m\times n}$ 具有满行秩 m 且 $m\le n$。由隐函数定理可知，存在 $t=0$ 的一个邻域 $(-\delta,\delta)$，其中某个 $\delta>0$，以及一个函数 $u:(-\delta,\delta)\to\mathbb{R}^m$，满足 $(0,u(0))=(0,0)$ 且

$$\bar h(t,u(t))=h(x+tv+\nabla h(x)^\top u(t))=0 \tag{2.19}$$

对所有 $t \in (-\delta, \delta)$ 成立。注意到

$$0 = \frac{d}{dt} h(x + tv + \nabla h(x)^\top u(t))\Big|_{t=0} = \nabla h(x)(v + \nabla h(x)^\top u'(0)).$$

由于 $\nabla h(x)v = 0$ 且 $\nabla h(x)\nabla h(x)^\top \succ 0$，可知 $u'(0) = 0$。

定义曲线 $c(t) = x + tv + \nabla h(x)^\top u(t)$，则由 (2.19) 可知，对所有 $t \in (-\delta, \delta)$ 都有 $h(c(t)) = 0$，即 $c(t) \in \Omega$。此外，$c(0) = x + \nabla h(x)^\top u(0) = x$，且 $c'(0) = v + \nabla h(x)^\top u'(0) = v$。证明完成。 □

定理 2.2.4 (Lagrange)**.** *设 $f, h \in C^1$。若 x 是 (2.17) 的一个局部极小值点，且 $x \in \Omega = \{x \in \mathbb{R}^n : h(x) = 0\}$ 是一个正规点，则存在 $\lambda \in \mathbb{R}^m$ 使得*

$$\nabla f(x) + \lambda \nabla h(x) = 0.$$

证明. 由引理 2.2.3，我们知道对任意 $v \in T(x)$，存在某个 $\delta > 0$ 和一条曲线 $c : (-\delta, \delta) \to \Omega$，使得 $c(0) = x$ 且 $c'(0) = v$。

由于 x 是 f 的一个局部极小值点，我们有

$$0 = \frac{d}{dt} f(c(t))\Big|_{t=0} = \nabla f(c(0))c'(0) = \nabla f(x)v.$$

由于 $v \in T(x)$ 是任意的，我们知道 $\nabla f(x)$ 属于 $\nabla h(x)$ 的行空间。这就意味着存在（一个行向量）$\lambda \in \mathbb{R}^m$，使得 $\nabla f(x) + \lambda \nabla h(x) = 0$。 □

备注. 注意，在定理 2.2.4 中要求 $x \in \Omega$ 是一个正规点是必要的：考虑问题 (2.17)，其中 $f, h : \mathbb{R} \to \mathbb{R}$ 定义为

$$f(x) = x \quad \text{和} \quad h(x) = \begin{cases} \frac{1}{2}x^2, & \text{if} \quad x < 0, \\ 0, & \text{if} \quad 0 \le x \le 1, \\ \frac{1}{2}(x-1)^2, & \text{if} \quad x > 1. \end{cases}$$

那么很容易说明，$x = 0$ 是约束 $h(x) = 0$ 下 f 的极小值点。此外，$f, h \in C^1$，$f'(0) = 1$ 且 $h'(0) = 0$（因此 $x = 0$ 不是正规点），而定理 2.2.4 的结论不成立。

为了便于记忆定理 2.2.4 的结果，我们定义 (x, λ) 的 **Lagrange 方程**为

$$L(x, \lambda) = f(x) + \lambda h(x).$$

于是定理 2.2.4 表明，若 $x \in \Omega$ 是一个正规点，并且是等式约束优化问题 (2.17)

的一个局部极小值点，其中 $f \in C^1(\mathbb{R}^n;\mathbb{R})$ 且 $h \in C^1(\mathbb{R}^n;\mathbb{R}^n)$，则存在一个称为 **Lagrange 乘子**的 $\lambda \in \mathbb{R}^m$，使得

$$\nabla f(x) + \lambda \nabla h(x) = 0, \tag{2.20a}$$

$$h(x) = 0. \tag{2.20b}$$

我们可以将 (2.20) 记成 $(\partial_x L(x,\lambda), \partial_\lambda L(x,\lambda)) = (0,0)$。

若 $f, h \in C^2$，则我们可以在定理 2.2.4 的基础上进一步建立 (2.17) 的**二阶必要条件**。

定理 2.2.5. *设 $f \in C^1(\mathbb{R}^n;\mathbb{R})$ 且 $h \in C^1(\mathbb{R}^n;\mathbb{R}^m)$，其中 $m \le n$。若 $x \in \Omega = \{x \in \mathbb{R}^n : h(x) = 0\}$ 是一个正规点，且是 (2.17) 的一个局部极小值点，则存在一个 Lagrange 乘子 $\lambda \in \mathbb{R}^m$（行向量），使得*

1. *$\nabla f(x) + \lambda \nabla h(x) = 0$；并且*

2. *对任意 $v \in T(x)$，都有*

$$v^\top \Big(\nabla^2 f(x) + \sum_{i=1}^m \lambda_i \nabla^2 h(x) \Big) v \ge 0. \tag{2.21}$$

证明. 第一条结论就是定理 2.2.4。对于第二条结论，我们沿用定理 2.2.4 的方法，得到曲线 $c : (-\delta,\delta) \to \Omega$，满足 $c(0) = x$ 且 $c'(0) = v$。现在定义 $\phi : (-\delta,\delta) \to \mathbb{R}$ 为 $\phi(t) := f(c(t))$。由于 x 是局部极小值点且对所有 $t \in (-\delta,\delta)$ 都有 $c(t) \in \Omega$，我们知道

$$0 \le \phi''(0) = v^\top \nabla^2 f(x) v + \nabla f(x) c''(0). \tag{2.22}$$

进一步地，对每个 $i \in [m]$，定义 $\psi_i : (-\delta,\delta) \to \mathbb{R}$ 为 $\psi_i(x) := h_i(c(t))$。于是，由于对所有 $t \in (-\delta,\delta)$ 都有 $h_i(c(t)) = 0$，便有

$$0 = \psi_i''(0) = v^\top \nabla^2 h_i(x) v + \nabla h_i(x)^\top c''(0) \tag{2.23}$$

对每个 $i \in [m]$ 都成立。将 (2.22) 与按 λ_i 加权后的 (2.23) 对所有 $i \in [m]$ 求和，并注意到 $\nabla f(x) + \lambda \nabla h(x) = 0$，即可得到 (2.21)。 □

现在我们考虑同时含有等式约束与不等式约束的优化问题的一阶和二阶

必要条件：

$$\min_{x\in\mathbb{R}^n} \quad f(x), \tag{2.24a}$$

$$\text{s.t.} \quad g(x) \le 0, \tag{2.24b}$$

$$h(x) = 0, \tag{2.24c}$$

其中 $g = (g_1, \ldots, g_k) : \mathbb{R}^n \to \mathbb{R}^k$，$h = (h_1, \ldots, h_m) : \mathbb{R}^n \to \mathbb{R}^m$。对任意满足 $g(x) \le 0$ 且 $h(x) = 0$ 的点 x，我们将 $[k]$ 划分为**活跃集**（active set）

$$A(x) := \{j \in [k] : g_j(x) = 0\} \tag{2.25}$$

和**非活跃集**（inactive set）

$$I(x) := [k] \setminus A(x) = \{j \in [k] : g_j(x) < 0\}. \tag{2.26}$$

记约束集为

$$\Omega = \{x \in \mathbb{R}^n : g(x) \le 0,\ h(x) = 0\}.$$

若 $x \in \Omega$ 满足向量

$$\nabla h_1(x),\ \ldots,\ \nabla h_m(x), \quad \text{和} \quad \nabla g_j(x),\ \forall\, j \in A(x) \tag{2.27}$$

线性无关（注意这些是 $\mathbb{R}^n$ 中的 $m + |A(x)|$ 个行向量），则称 $x \in \Omega$ 是一个**正规点**。下面的结果描述了 (2.24) 的一阶必要条件，即著名的 **Karush–Kuhn–Tucker**（KKT）定理。

定理 2.2.6 (Karush–Kuhn–Tucker)**.** 对于约束优化问题 (2.24)，设 $f \in C^1(\mathbb{R}^n;\mathbb{R})$，$g \in C^1(\mathbb{R}^n;\mathbb{R}^k)$，$h \in C^1(\mathbb{R}^n;\mathbb{R}^m)$。若 $x \in \Omega$ 是一个正规点，且是 f 的一个局部极小值点，则存在 $\lambda \in \mathbb{R}^m$ 和 $\mu \in \mathbb{R}^k$（λ 和 μ 都是行向量），使得

$$\nabla f(x) + \lambda \nabla h(x) + \mu \nabla g(x) = 0, \tag{2.28a}$$

$$h(x) = 0, \tag{2.28b}$$

$$g(x) \le 0, \tag{2.28c}$$

$$\mu \ge 0, \tag{2.28d}$$

$$\mu_j g_j(x) = 0, \quad \forall\, j \in [k]. \tag{2.28e}$$

证明. 注意，由于 $x \in \Omega$，所以 (2.28b) 和 (2.28c) 成立。我们令对所有 $j \in I(x)$ 都有 $\mu_j = 0$。于是，由于对所有 $j \in A(x)$ 都有 $g_j(x) = 0$，可知 (2.28e) 成立。

由于各个 g_j 都是 C^1 函数，且对所有 $j \in I(x)$ 都有 $g_j(x) < 0$，因此存在 $r > 0$，使得对所有 $y \in B_r(x)$ 和 $j \in I(x)$ 都有 $g_j(y) < 0$。记

$$\Omega' := \{y \in \Omega \cap B_r(x) : h(y) = 0,\ g_j(y) = 0,\ \forall\, j \in A(x)\}.$$

由于 $x \in \Omega$ 是一个正规点且是 f 的局部极小值点，我们知道 x 在 Ω' 中仍是一个正规点与局部极小值点（因为 $g, h \in C^1$）。注意，Ω' 只包含等式约束。因此，我们应用定理 2.2.4，得到存在 $\lambda \in \mathbb{R}^m$ 以及对每个 $j \in A(x)$ 存在 $\mu_j \in \mathbb{R}$，使得 (2.28a) 成立（这里我们注意到，对所有 $l \in I(x)$，有 $\mu_l = 0$）。

现在只需证明对所有 $j \in A(x)$ 都有 $\mu_j \geq 0$。我们采用反证法：若不然，则存在某个 $j \in A(x)$ 使得 $\mu_j < 0$。定义

$$\hat{\Omega} := \{x \in \mathbb{R}^n : h(x) = 0,\ g_{j'}(x) = 0,\ \forall\, j' \in A(x) \setminus \{j\}\} \tag{2.29}$$

以及

$$\hat{T}(x) := \{v \in \mathbb{R}^n : \nabla h(x)v = 0,\ \nabla g_{j'}(x)v = 0,\ \forall\, j' \in A(x) \setminus \{j\}\}. \tag{2.30}$$

注意，$\hat{T}(x)$ 是 $\hat{\Omega}$ 在 x 处的切空间，同时也是下式中张成的线性子空间

$$\operatorname{span}\{\nabla h_i(x),\ \nabla g_{j'}(x)\ :\ i \in [m],\ j' \in A(x) \setminus \{j\}\} \tag{2.31}$$

的正交补。由于 x 是正规点，我们知道 $\nabla g_j(x)$ 与张成空间 (2.31) 中的那些向量线性无关，因此 $\nabla g_j(x) \in \hat{T}(x)$。于是存在某个 $v \in \hat{T}(x)$（例如可取 $v = -\nabla g_j(x)^\top$），使得 $\nabla g_j(x)v < 0$。

我们将 v 乘到 (2.28a) 两边，得到

$$0 = \nabla f(x)v + \lambda \nabla h(x)v + \mu \nabla g(x)v = \nabla f(x)v + \mu_j \nabla g_j(x)v,$$

其中第二个等号来自 (2.30) 中 $v \in \hat{T}(x)$ 的定义，以及对所有 $l \in I(x)$ 都有 $\mu_l = 0$。由于 $\mu_j < 0$ 且 $\nabla g_j(x)v < 0$，可得 $\nabla f(x)v < 0$。

由于 x 是 $\hat{\Omega}$ 中的正规点，且 $\hat{T}(x)$ 是 $\hat{\Omega}$ 在 x 处的切空间，由定理 2.2.4 可知，存在 $\delta > 0$ 和一条曲线 $c : (-\delta, \delta) \to \hat{\Omega}$，使得 $c(0) = x$ 且 $c'(0) = v$。此

外，

$$\frac{d}{dt}f(c(t))\Big|_{t=0} = \nabla f(c(0))c'(0) = \nabla f(x)v < 0$$

且

$$\frac{d}{dt}g_j(c(t))\Big|_{t=0} = \nabla g_j(c(0))c'(0) = \nabla g_j(x)v < 0.$$

因此，存在足够小的 $\delta' \in (0,\delta)$，使得对所有 $t \in (0,\delta')$，都有 $c(t) \in B_r(x)$、$f(c(t)) < f(c(0)) = f(x)$、$g_j(c(t)) < g_j(c(0)) = g_j(x) = 0$。同时，对每个 $j' \in A(x) \setminus \{j\}$，都有 $g_{j'}(c(t)) = 0$（因为 $c(t) \in \hat{\Omega}$）；对每个 $l \in I(x)$，都有 $g_l(c(t)) < 0$（因为 $c(t) \in B_r(x)$），且这些都对所有 $t \in (0,\delta')$ 成立。因此，$\{c(t) : t \in (0,\delta')\} \subset \Omega$，这说明曲线 c 上这些点都是可行点，并且取得了更小的 f 值。这与 x 是 f 在 $\hat{\Omega}$ 中的局部极小值点矛盾。因此，对所有 $j \in A(x)$ 都有 $\mu_j \geq 0$，证明完成。 □

为了便于记忆定理 2.2.6 的结果，我们也可以定义 (x,λ,μ) 的 Lagrangian 方程为

$$L(x,\lambda,\mu) = f(x) + \lambda h(x) + \mu g(x).$$

于是，定理 2.2.6 表明：若 $x \in \Omega$ 是一个正规点，且是约束优化问题 (2.24) 的一个局部极小值点，其中 $f \in C^1(\mathbb{R}^n;\mathbb{R})$，$g \in C^1(\mathbb{R}^n;\mathbb{R}^k)$，$h \in C^1(\mathbb{R}^n;\mathbb{R}^m)$，则存在一个 Lagrange 乘子 $\lambda \in \mathbb{R}^m$ 和一个必须非负的 **KKT 乘子** $\mu \in \mathbb{R}^k$，使得 $\partial_x L(x,\lambda,\mu) = 0$ 且 $\mu g(x) = 0$（称为**互补松弛**条件，complementary slackness condition）。注意，在 (2.28c) 与 (2.28d) 成立的前提下，$\mu g(x) = 0$ 与 (2.28e) 是等价的。由 (2.28) 中这些等式与不等式组成的系统称为 **KKT 条件**。满足 KKT 条件的点 x 称为 **KKT 点**。

若 $f,g,h \in C^2$，则还可以建立 (2.24) 的二阶必要条件，如下所示。

定理 2.2.7. *设 $f \in C^2(\mathbb{R}^n;\mathbb{R})$，$g \in C^2(\mathbb{R}^n;\mathbb{R}^k)$，$h \in C^2(\mathbb{R}^n;\mathbb{R}^m)$，且 $k+m \leq n$。若 $x \in \Omega = \{x \in \mathbb{R}^n : g(x) \leq 0,\ h(x) = 0\}$ 是一个正规点，且是 (2.24) 的一个局部极小值点，则存在一个 Lagrange 乘子 $\lambda \in \mathbb{R}^m$ 和一个 KKT 乘子 $\mu \in \mathbb{R}^k$，使得*

1. *KKT 条件 (2.28) 成立；并且*

2. *对任意 $v \in T(x)$，都有*

$$v^\top\Big(\nabla^2 f(x) + \sum_{i=1}^m \lambda_i \nabla^2 h(x) + \sum_{j=1}^k \mu_j \nabla^2 g_j(x)\Big)v \geq 0, \tag{2.32}$$

其中

$$T(x) = \{v \in \mathbb{R}^n : \nabla h(x)v = 0,\ \nabla g_j(x)v = 0,\ \forall\, j \in A(x)\}.$$

证明. 第一条结论就是定理 2.2.4。对于第二条结论，我们沿用定理 2.2.5 的证明方法，得到曲线 $c : (-\delta, \delta) \to \Omega' = \{x \in \mathbb{R}^n : h(x) = 0,\ g_j(x) = 0,\ \forall\, j \in A(x)\}$，使得 $c(0) = x$ 且 $c'(0) = v$。其余证明与定理 2.2.5 的证明非常类似，因此此处省略。 □

2.2.3 充分最优性条件

在某些情况下，我们还需要判断一个给定点是否为局部极小点。这就需要局部极小点的充分条件。在本小节中，我们利用目标函数的二阶导数来推导这样的条件。需要注意的是，一般而言，仅依靠一阶导数无法给出充分条件。

引理 2.2.8. 设 P 和 Q 为两个对称矩阵，其中 Q 是半正定的，而 P 在 Q 的零空间上是正定的，即对于任意满足 $x^\top Qx = 0$ 的非零向量 x，有 $x^\top Px > 0$，则存在 $\bar{\gamma} > 0$，使得对于任意 $\gamma \in (0, \bar{\gamma})$，都有 $P + \frac{1}{\gamma}Q \succ 0$。

证明. 我们采用反证法。如果结论不成立，则对于任意 $k \in \mathbb{N}$，取 $\gamma_k = 1/k$，则存在 x_k 满足 $|x_k| = 1$ 且

$$x_k^\top \Big(P + \frac{1}{\gamma_k}Q\Big)x_k = x_k^\top (P + kQ)x_k \le 0 \tag{2.33}$$

由于 $\{x_k\}$ 有界，存在其子序列 $\{x_{k_j}\}$ 以及向量 $\bar{x}$，使得当 $j \to \infty$ 时 $x_{k_j} \to \bar{x}$。

注意到 $|\bar{x}| = 1$，即 $\bar{x} \ne 0$。对 $k = k_j$ 并令 $j \to \infty$，对 (2.33) 取极限，有

$$\bar{x}^\top P\bar{x} + \lim_{j\to\infty} k_j x_{k_j}^\top Q x_{k_j} \le 0.$$

由于 Q 是半正定的，故对所有 j 有 $x_{k_j}^\top Q x_{k_j} \ge 0$。这意味着 $\bar{x}^\top P\bar{x} \le 0$。

注意到 $\bar{x}^\top P\bar{x}$ 是一个固定数，而当 $j \to \infty$ 时 $k_j \to \infty$。因此必须有

$$\bar{x}^\top Q\bar{x} = \lim_{j\to\infty} x_{k_j}^\top Q x_{k_j} = 0,$$

从而由于 P 在 Q 的零空间上正定，有 $\bar{x}^\top P\bar{x} > 0$。这与上面的 $\bar{x}^\top P\bar{x} \le 0$ 矛盾。 □

现在我们可以给出局部极小值点的充分条件。我们首先考虑带等式约束的优化问题 (2.17)。

命题 2.2.9 (二阶充分条件)**.** 设 $f, h \in C^2$, $x^* \in \mathbb{R}^n$ 且 $\lambda^* \in \mathbb{R}^m$ 满足 *Lagrange* 条件，并且

$$y^\top \Big(\nabla^2 f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla^2 h_i(x^*)\Big) y > 0 \tag{2.34}$$

对所有满足 $\nabla h(x^*)y = 0$ 的非零向量 $y \in \mathbb{R}^n$ 成立，则 x^* 是在约束 $h(x) = 0$ 下 f 的一个严格局部极小点。事实上，存在 $\alpha > 0$ 和 $r > 0$，使得

$$f(x) \geq f(x^*) + \frac{\alpha}{2}|x - x^*|^2$$

对所有满足 $h(x) = 0$ 的 $x \in B_r(x^*)$ 成立。

证明. 取 $\gamma > 0$。定义增广 Lagrange 函数 l_A 为

$$l_A(x, \lambda; \gamma) := f(x) + \lambda h(x) + \frac{1}{2\gamma}|h(x)|^2, \tag{2.35}$$

注意到 l_A 同时也是如下问题的 Lagrange 函数

$$\min_{x \in \mathbb{R}^n} \quad f(x) + \frac{1}{2\gamma}|h(x)|^2, \tag{2.36a}$$

$$\text{s.t.} \quad h(x) = 0. \tag{2.36b}$$

可以看出 (2.36) 与 (2.17) 具有相同的局部极小点。此外，有

$$\nabla_x l_A(x, \lambda; \gamma) = \nabla f(x) + \lambda \nabla h(x) + \frac{1}{\gamma} h(x)^\top \nabla h(x)$$

以及

$$\nabla_{xx}^2 l_A(x, \lambda; \gamma) = \nabla^2 f(x) + \sum_{i=1}^m \Big(\lambda_i + \frac{h_i(x)}{\gamma}\Big) \nabla^2 h_i(x) + \frac{1}{\gamma} \nabla h(x)^\top \nabla h(x).$$

由于 (x^*, λ^*) 满足 Lagrange 条件，即

$$\begin{aligned} \nabla f(x^*) + \lambda^* \nabla h(x^*) &= 0, \\ h(x^*) &= 0, \end{aligned}$$

因此有

$$\nabla_x l_A(x^*, \lambda^*; \gamma) = \nabla f(x^*) + \lambda^* \nabla h(x^*) + \frac{1}{\gamma} h(x^*)^\top \nabla h(x^*) = 0$$

以及

$$\nabla_{xx}^2 l_A(x^*, \lambda^*; \gamma) = \nabla^2 f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla^2 h_i(x^*) + \frac{1}{\gamma} \nabla h(x^*)^\top \nabla h(x^*).$$

注意，对于任何属于 $\nabla h(x^*)^\top \nabla h(x^*)$ 的零空间的向量 y（等价地，y 满足 $\nabla h(x^*)y = 0$），不等式 (2.34) 成立。因此，由引理 2.2.8，存在 $\bar{\gamma} > 0$，使得对于所有 $\gamma \in (0, \bar{\gamma})$，有 $\nabla_{xx}^2 l_A(x^*, \lambda^*; \gamma) \succ 0$。

现在考虑固定 $\gamma \in (0, \bar{\gamma})$ 和 λ^* 时的 $l_A(\cdot, \lambda^*; \gamma)$。此时 $l_A(\cdot, \lambda^*; \gamma) : \mathbb{R}^n \to \mathbb{R}$ 是关于 x 的函数。根据前面的推导，我们有 $\nabla_x l_A(\cdot, \lambda^*; \gamma) = 0$ 且 $\nabla_{xx}^2 l_A(\cdot, \lambda^*; \gamma) \succ 0$。因此，$x^*$ 是 $l_A(\cdot, \lambda^*; \gamma)$ 的严格局部极小点。此外，

$$\begin{aligned} l_A(x, \lambda^*; \gamma) &= l_A(x^*, \lambda^*; \gamma) + \nabla_x l_A(x^*, \lambda^*; \gamma) \\ &\quad + \frac{1}{2}(x - x^*)^\top \nabla_{xx}^2 l_A(x^*, \lambda^*; \gamma)(x - x^*) + o(|x - x^*|^2) \\ &= f(x^*) + \frac{1}{2}(x - x^*)^\top \nabla_{xx}^2 l_A(x^*, \lambda^*; \gamma)(x - x^*) + o(|x - x^*|^2). \end{aligned}$$

由于 $l_A(x^*, \lambda^*; \gamma) \in C^2$，可知存在 $\alpha > 0$ 和 $r > 0$，使得

$$l_A(x, \lambda^*; \gamma) \geq f(x^*) + \frac{\alpha}{2}|x - x^*|^2$$

对所有 $x \in B_r(x^*)$ 成立。若进一步满足 $h(x) = 0$，则有 $l_A(x, \lambda^*; \gamma) = f(x)$，上述不等式即化为 $f(x) \geq f(x^*) + \frac{\alpha}{2}|x - x^*|^2$，对所有满足 $h(x) = 0$ 的 $x \in B_r(x^*)$ 成立。 □

如上所示，二阶充分条件（命题 2.2.9）刻画了严格局部极小值点 x^*。因此，我们可以很容易地将其推广到同时具有等式与不等式约束的优化问题 (2.24)：只需忽略在 x^* 处不活跃的不等式约束（因为在 x^* 处为严格不等式），并注意 x^* 在其余约束上满足等式条件。因此，可以直接将这些活跃的不等式约束（此时均转化为等式约束）纳入，从而得到与命题 2.2.9 相同的结论。这里省略具体细节。

2.3 自动微分

在大多数优化算法中，我们都需要计算目标函数的一阶和二阶导数。一个简单的方法是使用有限差分。具体地说，设 $f:\mathbb{R}^n \to \mathbb{R}$，则我们可以将偏导数 $\frac{\partial f(x)}{\partial x_i}$ 近似为

$$\frac{\partial f(x)}{\partial x_i} \approx \frac{f(x+\epsilon e_i)-f(x)}{\epsilon}, \tag{2.37}$$

其中 $e_i \in \mathbb{R}^n$ 的第 i 个分量为 1，其余分量为 0，而 $\epsilon > 0$ 是用户选取的步长。然而，当 ϵ 较大时，近似 (2.37) 往往不够准确；而当 ϵ 较小时，又会严重受到浮点误差的影响。因此，这种近似并不适用于诸如网络训练这类大规模优化问题。

相比之下，反向传播（back-propagation）是一种更为优雅的方法，用于高效准确地计算函数的一阶和二阶导数（也可延伸计算更高阶，但实践中不常见）。反向传播则是自动微分（automatic differentiation）的一个特例，我们将在本节中详细讨论自动微分。

自动微分是一种基于函数表示和链式法则来计算导数解析值的方法。自动微分的基础在于：任何函数的计算都可以表示为一系列初等运算的复合。所谓初等运算，是指以下这类只接受一个或两个标量作为变量的函数：

- 单变量运算：sin、cos、tan、exp、log 等。
- 双变量运算：+、−、×、/ 等。

对于任意函数 f，我们都可以构造一个**计算图**，利用这些初等运算来表示 f，其中节点表示这些运算的标量输入与输出，边则表示节点之间由所执行运算所决定的关系。

下面的例子说明了如何将一个函数分解为作用于其参数上的这些初等运算，并构造相应的计算图。

例 2.3.1. 考虑如下函数 $f:\mathbb{R}^3 \to \mathbb{R}$：

$$f(x) = \frac{e^{2x_2}\cos(x_2x_3)}{x_1+x_2} \tag{2.38}$$

其中 $x=(x_1,x_2,x_3)\in\mathbb{R}^3$。除了参数节点 x_1、x_2、x_3 外，f 的计算图还引入

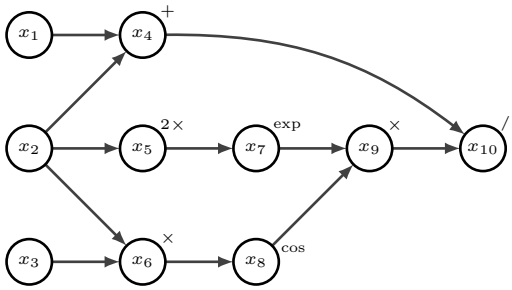


图 2.2: 函数 $f:\mathbb{R}^3\to\mathbb{R}$（定义见 (2.38)）的计算图。$f$ 的自变量由节点 x_1、x_2 和 x_3 表示。其余节点表示中间标量变量。特别地，$x_{10}=f(x_1,x_2,x_3)$。每个节点所执行的初等运算显示在相应节点的右上角。

了新的节点 $x_4,\ldots,x_{10}$，用于表示中间变量：

$$
\begin{aligned}
x_4 &= x_1+x_2,\\
x_5 &= 2\times x_2,\\
x_6 &= x_2\times x_3,\\
x_7 &= e^{x_5}=e^{2x_2},\\
x_8 &= \cos(x_6)=\cos(x_2x_3),\\
x_9 &= x_7\times x_8=e^{2x_2}\cos(x_2x_3),\\
x_{10} &= x_9/x_4=e^{2x_2}\cos(x_2x_3)/(x_1+x_2).
\end{aligned}
\tag{2.39}
$$

这里稍微滥用记号，我们之后用 x_i 同时表示节点及该节点处的数值。函数 (2.38) 的计算图如图 2.2 所示。之后在本节中，我们将从 x_i 到 x_j 的有向边记作 (x_i,x_j)。在这种情况下，我们称 x_i 为 x_j 的**父节点**，而称 x_j 为 x_i 的**子节点**。

可以看到，(2.39) 中的每个节点都是通过对其一个或两个父节点施加某种初等运算而得到的。因此，每个节点都有一条或两条入边。在图 2.2 中，每个节点所执行的运算都标注在对应节点的右上角。注意，图中的运算和连接顺序必须与函数 f 的定义正确对应。我们还注意到，计算图中的所有节点实际上都是 (x_1,x_2,x_3) 的函数。

在计算图中，一旦 x_i 的所有父节点的值已知，x_i 的值就可以计算出来。如果输入变量 $x=(x_1,\ldots,x_n)$ 的值已知，我们就可以算出计算图中所有节点的值。这一按顺序进行计算的过程称为**前向扫描**。

下面我们将说明自动微分如何在第 2.3.1 节中计算一阶导数（如梯度和

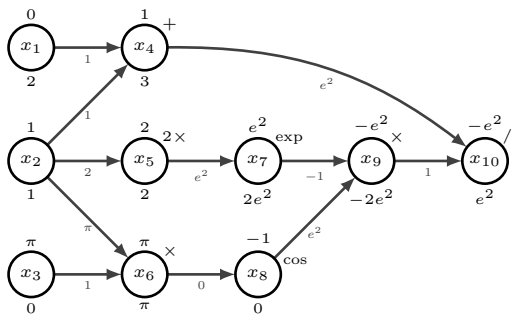


图 2.3: 使用自动微分前向模式计算函数 (2.38) 在 $x = (0, 1, \pi)$ 和 $v = (2, 1, 0)$ 下的 $f(x)$ 与 $D_v f(x)$。x_j 和 $D_v x_j$ 的值分别显示在对应节点 x_j 的上方和下方。节点 x_j 处所执行的初等运算显示在其右上角。有向边 (x_i, x_j) 下方显示的是 $\frac{\partial x_j}{\partial x_i}$ 的值。在终端节点 x_{10} 处，我们得到 $f(x) = -e^2$ 和 $D_v f(x) = e^2$。

Jacobian 矩阵），以及在第 2.3.2 节中计算二阶导数（如 Hessian 矩阵）。无论哪种情况，自动微分都有两种模式，称为**前向模式**（forward mode）和**反向模式**（reverse mode），我们将在下文中详细讨论。

2.3.1 一阶导数的计算

在本小节中，我们考虑如何使用自动微分计算函数的一阶导数。我们首先介绍较为直接且更容易理解的前向模式，然后介绍更常用、也更适合深度学习任务的反向模式。我们还将讨论这两种情况下的计算复杂度和存储需求。

用前向模式计算梯度与 Jacobian 矩阵

自动微分的前向模式可用于计算方向导数 $D_v f(x)$（当 $f : \mathbb{R}^n \to \mathbb{R}$ 为标量值函数时），或 Jacobian-vector 积 $\nabla f(x) v$（当 $f : \mathbb{R}^n \to \mathbb{R}^m$ 为向量值函数时），其中 x 是定义域中的任意点，$v \in \mathbb{R}^n$ 是任意（列）向量。因此，通过取 $v = e_i$（其中 $i \in [n]$，e_i 是第 i 个分量为 1、其余分量为 0 的 n 维向量），我们就能得到梯度（或 Jacobian 矩阵）∇f。

我们先考虑 $f : \mathbb{R}^n \to \mathbb{R}$ 为标量值函数的情形。除了输入变量 $x = (x_1, \ldots, x_n)$ 外，f 的计算图还会引入中间变量 $x_{n+1}, \ldots, x_N$，它们都是 $x_1, \ldots, x_n$ 的函数。例如，在例 2.3.1 和图 2.2 中，$N = 10$。

现在我们按照图 2.2 所示的计算图，在**前向扫描**中（即图 2.2 中从左到右）逐步执行这些初等运算。除了数值 x_j 之外，我们还通过链式法则计算 $D_v x_j$

的值:

$$D_v x_j = \sum_{x_i \in P_j} \frac{\partial x_j}{\partial x_i} D_v x_i, \tag{2.40}$$

其中 P_j 表示 x_j 的父节点集合。注意，(2.40) 中的求和只包含一项或两项。我们从给输入节点 $x_1, \ldots, x_n$ 赋值开始，并注意到对每个 $i \in [n]$，有 $D_v x_i = \langle \nabla x_i, v \rangle = \langle e_i, v \rangle = v_i$。然后我们继续计算它们的子节点，以及这些子节点处的方向导数。当完成前向扫描时，我们就能在最后一个节点 x_N 处同时得到 $f(x) = x_N$ 和 $D_v f(x) = D_v x_N$。图 2.3 展示了在输入 $x = (x_1, x_2, x_3) = (0, 1, \pi)$ 和 $v = (2, 1, 0)$ 时，函数 (2.38) 的中间变量的取值。在图 2.3 中，x_j 与 $D_v x_j$ 的值分别显示在节点 x_j 的上方和下方。有向边 (x_i, x_j) 下方显示偏导数 $\frac{\partial x_j}{\partial x_i}$ 的值。每个中间变量 x_j 处所执行的初等运算显示在其右上角，以便参考。例如，给定 $(x_7, D_v x_7) = (e^2, 2e^2)$（这两个值分别显示在节点 x_7 的上方与下方）以及 $(x_8, D_v x_8) = (-1, 0)$，我们得到

$$x_9 = x_7 \cdot x_8 = e^2 \cdot (-1) = -e^2$$

以及

$$D_v x_9 = \frac{\partial x_9}{\partial x_7} D_v x_7 + \frac{\partial x_9}{\partial x_8} D_v x_8 = (-1) \cdot (2e^2) + e^2 \cdot 0 = -2e^2.$$

对于上述给定的 x 和 v，最终我们得到 $x_{10} = -e^2$ 且 $D_v x_{10} = e^2$。手工计算也给出同样的结果：$\nabla f(x) = (e^2, -e^2, 0)$，从而验证了 $D_v f(x) = \langle \nabla f(x), v \rangle = e^2$。

如果 $f = (f_1, \ldots, f_m) : \mathbb{R}^n \to \mathbb{R}^m$ 是向量值函数，则其计算图与图 2.2 类似，只不过终端节点有 m 个而不是一个。我们将在这些终端节点处得到 $(D_v f_1(x), \cdots, D_v f_m(x))$ 的值，从而构成所需的 Jacobian-vector 积

$$\nabla f(x) v = \begin{pmatrix} D_v f_1(x) \\ \vdots \\ D_v f_m(x) \end{pmatrix}.$$

此时，计算成本将是标量值函数情形的 m 倍。

前向模式的优点在于，一旦节点 x_j 的所有子节点的计算都已完成，用于存储 $(x_j, D_v x_j)$ 的计算机内存就可以释放。由于计算图通常非常大，这在实践中可能是有益的。它的缺点是，如果需要完整的梯度或 Jacobian 矩阵，那么我们必须依次取 $v = e_i$（$i \in [n]$）。因此，总计算成本将是上述方向导数（或 Jacobian-vector 积）成本的 n 倍，这在 n 非常大时，计算量很高。接下来我

们将说明，自动微分的反向模式可以解决这一问题。

用反向模式计算梯度与 Jacobian 矩阵

我们先直接讨论如何计算标量值函数 $f:\mathbb{R}^n\to\mathbb{R}$ 的梯度 $\nabla f(x)$。（对于任意给定的 v，方向导数 $D_v f(x)=\langle\nabla f(x),v\rangle$ 可在之后通过向量内积轻松得到。）反向模式与前向模式的主要区别在于：反向模式在前向扫描中除了 x_j 的值之外，不会额外计算其他量。相反，一旦前向扫描完成，自动微分的反向模式就从终端节点 x_N 开始，沿有向边的反方向依次计算每个节点上的值 $y_j:=\frac{\partial f(x)}{\partial x_j}$，直到输入变量 $x=(x_1,\ldots,x_n)$ 中所有节点对应的 y_i 都被计算出来。此时，梯度 $\nabla f(x)=(y_1,\ldots,y_n)$ 就得到了。这个过程称为**反向扫描**，下面我们详细解释。

开始反向扫描时，我们注意到 $y_N=\frac{\partial f(x)}{\partial x_N}=1$，因为 $f(x)=x_N$。然后，对于任意节点 x_j，应用链式法则得到

$$y_j=\frac{\partial f(x)}{\partial x_j}=\sum_{x_i\in C_j}\frac{\partial f(x)}{\partial x_i}\frac{\partial x_i}{\partial x_j}=\sum_{x_i\in C_j}y_i\frac{\partial x_i}{\partial x_j},$$

其中 C_j 表示 x_j 的所有子节点集合。注意，C_j 可能包含两个以上的节点，这取决于 f 的定义；这与 (2.40) 中的情况不同。图 2.4 展示了在输入 $x=(x_1,x_2,x_3)=(0,1,\pi)$ 和 $v=(2,1,0)$ 时，函数 (2.38) 的节点值 x_j（黑色，位于节点 x_j 上方）和 y_j（灰色，位于节点 x_j 下方）。在图 2.4 中，x_j 的值是在前向扫描中计算得到的；而每个节点 x_j 上的 y_j 以及每条有向边 (x_j,x_i) 上的 $\frac{\partial x_i}{\partial x_j}$ 的值，则是在反向扫描中计算得到的。每个中间变量 x_j 所执行的初等运算再次显示在其右上角，以便参考。例如，已知 $y_4=e^2$、$y_5=-e^2$ 和 $y_6=0$，则我们得到

$$y_2=y_4\frac{\partial x_4}{\partial x_2}+y_5\frac{\partial x_5}{\partial x_2}+y_6\frac{\partial x_6}{\partial x_2}=e^2\cdot 1+(-e^2)\cdot 2+0\cdot\pi=-e^2.$$

当反向扫描完成后，我们得到 $(y_1,y_2,y_3)=(e^2,-e^2,0)$，与前面在 $x=(0,1,\pi)$ 处给出的梯度 $\nabla f(x)$ 一致。

如果 $f=(f_1,\ldots,f_m):\mathbb{R}^n\to\mathbb{R}^m$ 是向量值函数，而我们希望对某个给定的 $u\in\mathbb{R}^m$ 计算 **vector-Jacobian** 积 $u\nabla f(x)\in\mathbb{R}^n$，其中二者都是行向量[1]，

[1]注意这与对于给定 $v\in\mathbb{R}^n$ 的 Jacobian-vector 积 $\nabla f(x)v$ 不同，v 和 $\nabla f(x)v$ 都是列向量。

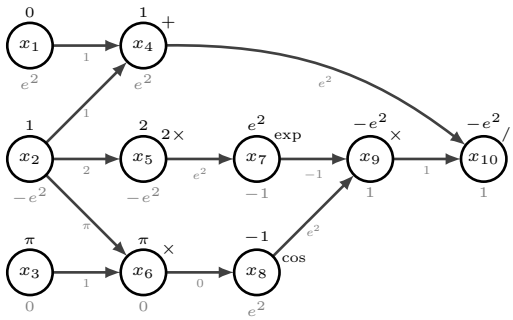


图 2.4: 使用自动微分反向模式计算函数 (2.38) 在 $x = (0, 1, \pi)$ 处的 $\nabla f(x)$。x_j（黑色）和 $y_j = \frac{\partial f(x)}{\partial x_j}$（灰色）的值分别显示在对应节点 x_j 的上方和下方。节点 x_j 所执行的初等运算显示在其右上角。有向边 (x_j, x_i) 下方显示的是 $\frac{\partial x_i}{\partial x_j}$（灰色）的值。黑色数值在前向扫描中计算，灰色数值在反向扫描中计算。最终结果为 $\nabla f(x) = (e^2, e^{-2}, 0)$。

那么我们只需定义

$$\bar{f} := \sum_{i=1}^{m} u_i f_i \ : \ \mathbb{R}^n \to \mathbb{R},$$

这就是一个标量值函数，然后像上面那样使用自动微分的反向模式即可。如果我们希望计算完整的 Jacobian 矩阵 $\nabla f(x)$，那么就需要在反向扫描过程中，对每个节点 x_j 计算一个（列）向量 $y_j = (y_j^1, \ldots, y_j^m) \in \mathbb{R}^m$，其中对 $k = 1, \ldots, m$，$y_j^k = \frac{\partial f_k(x)}{\partial x_j}$。在反向扫描完成之后，我们得到完整的 Jacobian 矩阵

$$\nabla f(x) = \begin{pmatrix} \nabla f_1(x) \\ \vdots \\ \nabla f_m(x) \end{pmatrix} = \Big(y_1, \ldots, y_n \Big) \in \mathbb{R}^{m \times n}.$$

反向模式的优点在于，计算完整梯度的成本只是求值一个标量值函数 $f : \mathbb{R}^n \to \mathbb{R}$ 所需成本的一个较小常数倍（通常约为 4 到 5 倍）。相比之下，前向模式则需要大约 $O(n)$ 倍于函数求值的计算成本。注意，深度学习领域中所谓的反向传播，其实正是自动微分反向模式的一个特例。不过，对于向量值函数 $f : \mathbb{R}^n \to \mathbb{R}^m$，随着 m 的增大，使用前向模式和反向模式计算完整 Jacobian 矩阵 $\nabla f(x)$ 的成本会变得相近。反向模式的一个问题是，前向扫描中计算出的所有值都必须存储下来，而且用于存储 y_j 的内存，直到反向扫描中 x_j 的所有父节点都使用完它之后才能释放。因此，在实际中，自动微分反向模式对内存的需求可能更高。

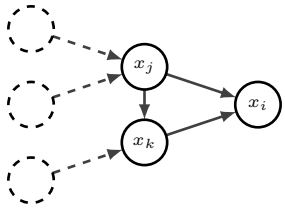


图 2.5: 在前向模式中计算 $D_{uv}f(x)$ 时，与节点 x_i 处量 (2.41) 的计算有关的子图 (取自 f 的计算图)。注意，每个节点最多可有两个父节点。这里展示的子图是最复杂的情形: x_i 有两个父节点 x_j 和 x_k，且 x_j 还是 x_k 的父节点。如果 x_i 只有一个父节点，或者两个父节点 x_j 和 x_k 彼此不是父节点，那么 (2.41) 的计算与这里类似，但更简单。

2.3.2 二阶导数的计算

我们沿用第 2.3.2 节中的相同思路，在本小节中依次考虑如何通过前向模式和反向模式来计算二阶导数。

用前向模式计算 Hessian

设 $f:\mathbb{R}^n \to \mathbb{R}$ 是一个标量值、二次连续可微的函数。此时，f 的 Hessian 是一个 $n\times n$ 对称矩阵 $\nabla^2 f(x) = [\partial_{ij} f(x)]$, 其中对 $i,j \in [n]$, $\partial_{ij} f(x) = \frac{\partial^2 f(x)}{\partial x_i \partial x_j}$。我们首先考虑如何计算

$$D_{uv}x_j := u^\top(\nabla^2 x_j)v \in \mathbb{R}$$

对于计算图中每个节点 x_j 的值，其中给定任意列向量 $u,v \in \mathbb{R}^n$。与前向模式计算梯度类似，我们仍然只在 f 的计算图上执行一次前向扫描。不同之处在于，在前向扫描过程中，我们还需要在每个节点 x_j 处计算若干二阶偏导数。当整个前向扫描完成时，我们就在终端节点 x_N 处得到了所需的值 $u^\top\nabla^2 f(x)v = u^\top(\nabla^2 x_N)v$。

更具体地说，在 f 的计算图中的每个节点 x_i 处，我们需要计算以下量：

$$
\begin{aligned}
&x_i,\ \underbrace{\frac{\partial x_i}{\partial x_j},\ \frac{\partial x_i}{\partial x_k},\ D_u x_i,\ D_v x_i,}_{\text{4 个一阶项}} \\
&\underbrace{\frac{\partial^2 x_i}{\partial x_j^2},\ \frac{\partial^2 x_i}{\partial x_k \partial x_j},\ \frac{\partial^2 x_i}{\partial x_k^2},\ D_u\Big(\frac{\partial x_i}{\partial x_j}\Big),\ D_u\Big(\frac{\partial x_i}{\partial x_k}\Big),\ D_v\Big(\frac{\partial x_i}{\partial x_j}\Big),\ D_v\Big(\frac{\partial x_i}{\partial x_k}\Big),\ D_{uv} x_i}_{\text{8 个二阶项}}.
\end{aligned}
\tag{2.41}
$$

回忆一下，对于每个 $i = n+1, \ldots, N$，节点 x_i 只有一个或两个父节点。不失一般性，我们考虑最复杂的情况：x_i 有两个父节点 x_j 和 x_k，并且 x_j 还是 x_k 的父节点。如果 x_i 只有一个父节点，或者它有两个父节点 x_j 和 x_k 但二者互不为父节点，则推导是类似且更简单的。我们所考虑情形下，f 的计算图对应的子图如图 2.5 所示。为了计算 (2.41) 中的量，我们注意到，这些量在前向扫描过程中已经在父节点 x_j 和 x_k 处被计算并存储了。由于 x_i 是 x_j 和 x_k 的某个给定初等函数（如 (2.39) 中那些形式），我们可以直接计算

$$
x_i,\ \frac{\partial x_i}{\partial x_j},\ \frac{\partial x_i}{\partial x_k},\ \frac{\partial^2 x_i}{\partial x_j^2},\ \frac{\partial^2 x_i}{\partial x_k \partial x_j},\ \frac{\partial^2 x_i}{\partial x_k^2}.
$$

此外，由链式法则可得

$$
\begin{aligned}
D_u x_i &= \frac{\partial x_i}{\partial x_j} D_u x_j + \frac{\partial x_i}{\partial x_k} D_u x_k, \\
D_v x_i &= \frac{\partial x_i}{\partial x_j} D_v x_j + \frac{\partial x_i}{\partial x_k} D_v x_k,
\end{aligned}
$$

因为右端的各项都已知。然后我们有

$$
\begin{aligned}
D_u\Big(\frac{\partial x_i}{\partial x_j}\Big) &= \frac{\partial}{\partial x_j}(D_u x_i) \\
&= \frac{\partial}{\partial x_j}\Big(\frac{\partial x_i}{\partial x_j} D_u x_j + \frac{\partial x_i}{\partial x_k} D_u x_k\Big) \\
&= \frac{\partial^2 x_i}{\partial x_j^2} D_u x_j + \frac{\partial x_i}{\partial x_j}\frac{\partial}{\partial x_j}(D_u x_j) + \frac{\partial^2 x_i}{\partial x_k \partial x_j} D_u x_k + \frac{\partial x_i}{\partial x_j}\frac{\partial}{\partial x_j}(D_u x_k).
\end{aligned}
\tag{2.42}
$$

注意到 $\frac{\partial}{\partial x_j}(D_u x_j) = 0$，而 $\frac{\partial}{\partial x_j}(D_u x_k) = D_u(\frac{\partial x_k}{\partial x_j})$ 在我们计算 x_k 处的量 (2.41) 时已经得到（若 x_j 不是 x_k 的父节点，则该值为 0）。因此，(2.42) 中所有项都是已知的，从而我们可以计算 $D_u(\frac{\partial x_i}{\partial x_j})$。类似地，$D_v(\frac{\partial x_i}{\partial x_j})$、$D_u(\frac{\partial x_i}{\partial x_k})$ 和

$D_v(\frac{\partial x_i}{\partial x_k})$ 也都可以被计算出来。最后，我们有

$$\begin{aligned}
D_{uv}x_i &= D_v(D_u x_i) \\
&= D_v\Big(\frac{\partial x_i}{\partial x_j}D_u x_j + \frac{\partial x_i}{\partial x_k}D_u x_k\Big) \\
&= D_v\Big(\frac{\partial x_i}{\partial x_j}\Big)D_u x_j + \frac{\partial x_i}{\partial x_j}D_{uv}x_j + D_v\Big(\frac{\partial x_i}{\partial x_k}\Big)D_u x_k + \frac{\partial x_i}{\partial x_k}D_{uv}x_k,
\end{aligned}$$

其中右端所有项此时都已知，因此 $D_{uv}x_i$ 也就可以被计算出来。这样，我们就完成了 x_j 处 (2.41) 中所有量的计算，并可以继续到下一个节点。需要指出的是，实际计算成本往往比上述公式表面上看起来更低，因为许多项可能为零；这是由于 x_i 仅仅是由 x_j 和 x_k 通过某个初等运算得到的。我们按此方式完成整个前向扫描，并在终端节点得到所需的量 $D_{uv}x_N = u^\top \nabla^2 f(x)v$。

我们说明一下，上述前向模式也可用于计算曲线 $c(t) := f(x+tv)$ 在 $t=0$ 处的曲率：对任意给定的 $x, v \in \mathbb{R}^n$，有 $c(0) = f(x)$、$c'(0) = D_v f(x)$，以及

$$c''(0) = v^\top \nabla^2 f(x) v = D_{vv} f(x).$$

如果我们希望计算整个 Hessian $\nabla^2 f(x)$，则可以对所有 $i \le j \le n$ 且 $1 \le i \le n$，取 $u = e_i$ 与 $v = e_j$ 来得到 Hessian 的上三角部分。由于当 $f \in C^2$ 时 $\nabla^2 f(x)$ 是对称的，我们便得到完整的 Hessian。这需要对 $\frac{1}{2}n(n+1)$ 组不同的 $(u,v) = (e_i, e_j)$ 进行计算。如果 Hessian 是稀疏的，且非零项的位置已知，那么所需计算的 (u,v) 对数可以显著减少，计算成本也可相应降低。

如果我们只想计算某个给定向量 $v \in \mathbb{R}^n$ 的 Hessian-向量积 $\nabla^2 f(x)v$，那么在 $u^\top \nabla^2 f(x)v$ 中只需让 $u = e_i$ 对所有 $i \in [n]$ 变化即可。此时总计算成本降为 $O(n)$，但当 n 很大时仍然较高。接下来，我们将说明反向模式如何解决这一问题。

用反向模式计算 Hessian

我们直接考虑如何对 $f \in C^2(\mathbb{R}^n; \mathbb{R})$ 和任意向量 $v \in \mathbb{R}^n$ 计算 Hessian-向量积 $\nabla^2 f(x)v$。首先，我们进行前向扫描，在 f 的计算图中每个节点 x_j 处计算 x_j 和 $D_v x_j$ 的值。在终端节点 x_N 处，我们同时得到 $f(x) = x_N$ 和 $D_v f(x) = D_v x_N$ 的值。然后我们进行反向扫描，并在每个节点 x_j 处计算

$z_j := \frac{\partial}{\partial x_j}(D_v f(x))$。注意到

$$z_N = \frac{\partial}{\partial x_N}(D_v f(x)) = D_v\Big(\frac{\partial f(x)}{\partial x_N}\Big) = 0$$

因为 $x_N = f(x)$。然后，由链式法则可知

$$z_j = \frac{\partial}{\partial x_j}(D_v f(x)) = \sum_{x_j \in C_j} \frac{\partial}{\partial x_i}(D_v f(x))\frac{\partial x_i}{\partial x_j} = \sum_{x_i \in C_j} z_i \frac{\partial x_i}{\partial x_j}, \tag{2.43}$$

其中 x_j 的每个子节点 x_i 对应的 z_i 在此时都已被计算出来。当反向扫描结束后，我们就在输入变量 $x = (x_1, \ldots, x_n)$ 对应的每个节点 x_j 处得到

$$z_j = \frac{\partial}{\partial x_j}(D_v f(x)) = \frac{\partial}{\partial x_j}\Big(\sum_{i=1}^n v_i \partial_i f(x)\Big) = \sum_{i=1}^n v_i \partial_{ij}^2 f(x)$$

的值。于是有

$$\nabla^2 f(x) v = \begin{pmatrix} \partial_{11}^2 f(x) & \cdots & \partial_{1n}^2 f(x) \\ \vdots & \ddots & \vdots \\ \partial_{n1}^2 f(x) & \cdots & \partial_{nn}^2 f(x) \end{pmatrix} \begin{pmatrix} v_1 \\ \vdots \\ v_n \end{pmatrix} = \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix} \in \mathbb{R}^n.$$

按照上述反向模式计算 $\nabla^2 f(x) v$ 的成本，只是计算 f 本身成本的一个常数倍（约为 12 倍）。这显著低于使用前向模式完成同一任务时所需的 $O(n)$ 成本。唯一的问题是，反向模式要求存储前向扫描中计算出的所有 x_j，并且在反向扫描中，直到 z_j 被 x_j 的所有父节点使用完之前，存储 z_j 的内存都不能释放。

如果需要完整的 Hessian $\nabla^2 f(x)$，则可以对所有 $i \in [n]$ 取 $v = e_i$，并重复上述过程。总计算成本为 $O(n)$。同样地，如果已知 Hessian 的稀疏结构，则这一成本还可以进一步降低。

2.4 确定性优化算法

在本节中，我们介绍若干具有代表性的确定性优化（deterministic optimization）算法，即目标函数具有解析表达式，不包含随机性或期望，其各阶导数都可以被精确计算（例如通过自动微分）。这一点可以通过对比下面的确定性优化 (2.44) 与随机优化 (2.101) 更加清晰地理解。

确定性优化的一般形式写作

$$\min_{x\in\Omega} f(x), \tag{2.44}$$

其中 Ω 是 $\mathbb{R}^n$ 的一个闭子集，目标函数 $f:\mathbb{R}^n\to\mathbb{R}$ 可能是非凸的。我们假设 f 可微，且其梯度 ∇f 满足 Lipschitz 连续性，具有某个 Lipschitz 常数 $L>0$（在优化领域中通常称为“光滑性”）。需要注意的是，我们通常假设 f 在整个 $\mathbb{R}^n$ 上是 Lipschitz 的，但这一假设往往可以放宽为在某个有界感兴趣区域内 Lipschitz，这一点我们将在后续推导中说明。这一假设较为常见，并且在深度学习中遇到的大多数问题中都成立。

在光滑性假设下，我们有如下重要不等式，它在优化算法的收敛性证明中被频繁使用。

命题 2.4.1. *设 $f:\mathbb{R}^n\to\mathbb{R}$ 连续可微，且 ∇f 是 Lipschitz 连续的，Lipschitz 常数为 $L>0$，则对所有 $x,y\in\mathbb{R}^n$ 有*

$$f(y)\le f(x)+\langle\nabla f(x),y-x\rangle+\frac{L}{2}|y-x|^2. \tag{2.45}$$

证明. 对任意 $x,y\in\mathbb{R}^n$，定义函数 $\phi:\mathbb{R}\to\mathbb{R}$:

$$\phi(t)=f((1-t)x+ty).$$

由于 f 可微，有

$$\phi(1)=\phi(0)+\int_0^1\phi'(t)\,dt,$$

从而得到

$$f(y)=f(x)+\int_0^1\langle\nabla f((1-t)x+ty),y-x\rangle\,dt.$$

两边同时减去 $f(x)+\langle f(x),y-x\rangle$，得到

$$\begin{aligned}
f(y)-f(x)-\langle\nabla f(x),y-x\rangle &= \int_0^1\langle\nabla f((1-t)x+ty)-\nabla f(x),y-x\rangle\,dt\\
&\le \int_0^1|\nabla f((1-t)x+ty)-\nabla f(x)||y-x|\,dt\\
&\le L|y-x|^2\int_0^1 t\,dt\\
&= \frac{L}{2}|y-x|^2,
\end{aligned}$$

其中第二个不等式是由于 ∇f 的 Lipschitz 连续性。 □

2.4.1 无约束优化算法

如果优化问题 (2.44) 是无约束的，即 $\Omega = \mathbb{R}^n$，那么我们的主要任务是根据一阶必要条件找到 f 的临界点。为此，我们考虑如下迭代格式：

$$x_{k+1} = x_k + \alpha_k v_k, \tag{2.46}$$

其中 $k = 0, 1, 2, \ldots$ 称为**迭代计数器**（iteration counter），$\alpha_k > 0$ 为**步长**（step size）（在深度学习中通常称为**学习率**，learning rate），而 v_k 是第 k 次迭代中在 x_k 处的下降方向。这里所谓下降方向是指 v_k 满足 $\langle \nabla f(x_k), v_k \rangle < 0$（我们将在本节后面详细讨论下降方向）。在如 (2.46) 所示的优化算法设计中，关键在于选择合适的 v_k 和 α_k，使得由 (2.46) 生成的序列 $\{x_k\}$ 满足 $|\nabla f(x_k)| \to 0$（当 $k \to \infty$ 时）。

梯度下降法与线搜索

我们首先介绍梯度下降法，并结合一种自动步长选择策略——回溯线搜索（backtracking line search），以说明理想中的优化算法的特性及实际中潜在的问题。

在梯度下降法中，最简单的一步是确定下降方向 v_k。我们暂时假设 v_k 是单位向量，因为此时我们的目标是确定方向。特别地，我们希望找到 v_k 使得方向导数 $D_{v_k} f(x_k)$ 为负且尽可能小，从而在该方向上实现最快下降。由此可得：

$$\begin{aligned} v_k &= \arg\min_{|v|=1} D_v f(x_k) \\ &= \arg\min_{|v|=1} \langle \nabla f(x_k), v \rangle \\ &= -\nabla f(x_k)/|\nabla f(x_k)|. \end{aligned}$$

因此，我们可以直接取 $v_k = -\nabla f(x_k)$，并将尺度调整交由步长 α_k 处理（后文将详细说明）。由于采用负梯度作为下降方向，迭代格式为

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k), \tag{2.47}$$

该迭代方法称为**梯度下降法**（gradient descent method）。

需要指出的是，在梯度下降法 (2.47) 中选择 $v_k = -\nabla f(x_k)$ 只是为了保证 v_k 是下降方向，从而简化讨论。实际上，只要存在常数 $c > 0$ 且对所有 k 有 $\nabla f(x_k)v_k \le -c|\nabla f(x_k)|^2$，则任意这样的 v_k 都可以使用，我们后续推导仍然成立。

在 (2.46) 和 (2.47) 中选择合适的步长 α_k 非常重要，因为它直接影响算法的收敛性与稳定性。较大的步长可以带来更快的进展，但可能导致序列 $\{x_k\}$ 发散或剧烈震荡；较小的步长则会使迭代进展缓慢，导致算法效率低下。

为了在第 k 次迭代中确定合适的步长，一个自然的想法是寻找使 $f(x_k + \alpha v_k)$ 在 $\alpha \ge 0$ 上最小的 α。为此，我们定义线搜索函数 $\phi_k(\alpha) := f(x_k + \alpha v_k)$，并求解一维优化问题 $\alpha_k = \arg\min_{\alpha \ge 0} \phi_k(\alpha)$。采用这种步长选择的梯度下降法称为**最速下降法**（steepest descent method）。然而，即使 ϕ_k 是一维函数，其最小值的求解仍然需要多次计算 f 和 ∇f，因此并不容易实现。

另一种常见方法是使用常数步长 $\alpha > 0$，即 $\alpha_k = \alpha$。事实上可以证明，当 $\alpha \in (0, 2/L)$ 时，由 (2.47) 生成的序列满足 $|\nabla f(x_k)| \to 0$。但在实践中，L 往往未知且难以估计，因此难以确定合适的 α。

为了解决这一问题，人们提出了多种自动选择步长的策略。这里我们介绍一种广泛使用的方法——**回溯线搜索**。其目标是找到满足如下 **Armijo 条件**的步长 α_k（其中 $c \in (0,1)$ 为给定常数）：

$$f(x_k + \alpha_k v_k) \le f(x_k) + c\alpha_k \nabla f(x_k) v_k. \tag{2.48}$$

在梯度下降法中，$v_k = -\nabla f(x_k)$，因此最后一项变为 $-c\alpha_k|v_k|^2$。

一般而言，只要 v_k 是下降方向，就有 $\nabla f(x_k)v_k < 0$，从而 (2.48) 蕴含 $f(x_k + \alpha_k v_k) \le f(x_k)$。因此，该条件也称为**充分下降条件**。从另一角度看，利用 $\phi_k(\alpha) = f(x_k + \alpha v_k)$，Armijo 条件 (2.48) 等价于

$$\phi_k(\alpha) \le \phi_k(0) + c\alpha\phi_k'(0). \tag{2.49}$$

右侧是通过点 $(0, \phi_k(0))$ 的一条直线，其斜率为 $c\phi_k'(0) < 0$。由于 $f(x)$ 有下界 f^*，该直线必然在某个 $\hat{\alpha} > 0$ 处与 $\phi(\alpha)$ 相交，因此 α_k 应取在 0 与 $\hat{\alpha}$ 之间。因此，Armijo 条件既保证目标函数值下降，又避免步长过大。

从 (2.49) 可知，当 α 足够小时条件必然成立，但过小的步长会导致收敛缓慢。因此，还常加入**曲率条件**：

$$\phi_k'(\alpha) \ge \hat{c}\phi_k'(0) \tag{2.50}$$

其中 $\hat{c} \in (c, 1)$。该条件防止 α_k 过小。Armijo 条件与曲率条件的组合称为 **Wolfe 条件**。

接下来介绍回溯线搜索的实现方法。

在回溯线搜索中，我们预先设定步长上界 $\bar{\alpha} > 0$ 和收缩因子 $\rho \in (0, 1)$。在第 k 次迭代中，设 $\alpha_k = \bar{\alpha}\rho^{l_k}$，其中

$$l_k = \arg\min \Big\{ l \in \{0, 1, \dots\} : f(x_k + \bar{\alpha}\rho^l v_k) \le f(x_k) - c\bar{\alpha}\rho^l |v_k|^2 \Big\}. \tag{2.51}$$

具体实现时，从 $\alpha_k = \bar{\alpha}$ 开始，检查 Armijo 条件：

$$f(x_k + \alpha_k v_k) \le f(x_k) - c\alpha_k |v_k|^2. \tag{2.52}$$

若满足，则接受该 α_k；否则令 $\alpha_k \leftarrow \rho\alpha_k$ 并重复上述过程。直到条件满足为止。该算法总结见算法 2.4.1。

Algorithm 2.4.1 带回溯线搜索的梯度下降法

Input: $\bar{\alpha}, \epsilon_{\text{tol}} > 0$, $\rho, c \in (0, 1)$, $x_0 \in \mathbb{R}^n$, $k = 0$

1: **repeat**
2: $\quad v_k \leftarrow -\nabla f(x_k)$
3: $\quad \alpha_k \leftarrow \bar{\alpha}$
4: $\quad$ **while** $f(x_k + \alpha_k v_k) > f(x_k) - c\alpha_k |v_k|^2$ **do**
5: $\quad\quad \alpha_k \leftarrow \rho\alpha_k$
6: $\quad$ **end while**
7: $\quad x_k \leftarrow x_k + \alpha_k v_k$
8: $\quad k \leftarrow k + 1$
9: **until** $|v_k| < \epsilon_{\text{tol}}$

Output: x_k

现在，对于算法 2.4.1 仍然有几个问题尚待回答：(i) 线搜索（第 4–6 行）能否在有限步内终止（如果能，其搜索次数上界是多少）？(ii) 终止判据（第 9 行）能否在有限的 k 下满足（如果能，需要多大的 k）？

我们分别利用下面的引理和定理来回答上述问题 (i) 和 (ii)。

引理 2.4.2. *设 $f : \mathbb{R}^n \to \mathbb{R}$ 属于 C^1，且 ∇f 在子水平集 $\{x \in \mathbb{R}^n : f(x) \le f(x_0)\}$ 上关于某个常数 $L > 0$ 是 Lipschitz 连续的，其中 $x_0 \in \mathbb{R}^n$ 是任意初始点。对任意 $\bar{\alpha} > 0$ 以及 $\rho, c \in (0, 1)$，记*

$$\bar{l} := \left\lceil \frac{\log(2(1-c)\bar{\alpha}^{-1}L^{-1})}{\log \rho} \right\rceil. \tag{2.53}$$

则在算法 2.4.1 的任意一次迭代 k 中，线搜索（第 4–6 行）的次数至多为 $\bar{l}$。即，对所有 k，都有 $l_k \le \bar{l}$，其中 l_k 由 (2.51) 定义。

证明. 我们采用反证法。若 $l_k > \bar{l}$，则有

$$f(x_k) - c\alpha|v_k|^2 < f(x_k + \alpha v_k) \tag{2.54}$$

其中 $\alpha := \bar{\alpha}\rho^{\bar{l}}$。由 (2.53) 中 $\bar{l}$ 的定义可知

$$\bar{l} \ge \frac{\log(2(1-c)\bar{\alpha}^{-1}L^{-1})}{\log\rho},$$

而由于 $\rho \in (0,1)$，故有

$$\alpha = \bar{\alpha}\rho^{\bar{l}} \le \frac{2(1-c)}{L}.$$

另一方面，由 (2.45) 可得

$$\begin{aligned} f(x_k + \alpha v_k) &\le f(x_k) + \langle \nabla f(x_k), \alpha v_k\rangle + \frac{L\alpha^2}{2}|v_k|^2 \\ &= f(x_k) - \alpha\Big(1 - \frac{L\alpha}{2}\Big)|v_k|^2 \\ &\le f(x_k) - c\alpha|v_k|^2, \end{aligned}$$

这与 (2.54) 矛盾。 □

引理 2.4.2 表明:每次迭代所需的线搜索次数有一个一致上界 $\bar{l} = O(\log L)$。注意，线搜索中每次判定准则检查（算法 2.4.1 的第 4 行）都需要在候选点 $x_k + \alpha_k v_k$ 处计算一次 f。

接下来，我们证明：由算法 2.4.1 生成的序列 $\{x_k\}$ 对任意初始点 x_0 都满足当 $k \to \infty$ 时 $|\nabla f(x_k)| \to 0$。

定理 2.4.3 (算法 2.4.1 的收敛性)**.** *对任意 $x_0 \in \mathbb{R}^n$、$\bar{\alpha} > 0$ 以及 $\rho, c \in (0,1)$，由算法 2.4.1 生成的序列 $\{x_k\}$ 满足*

1. *级数 $|\nabla f(x_k)|^2$ 是有界的，即*

$$\sum_{k=0}^{\infty} |\nabla f(x_k)|^2 < \infty\ . \tag{2.55}$$

因此，当 $k \to \infty$ 有 $|\nabla f(x_k)| \to 0$。

2. 对任意 $K > 1$，有

$$\min_{0\le k\le K-1} |\nabla f(x_k)|^2 \le \frac{L(f(x_0)-f^*)}{2c(1-c)\rho K} = O\Big(\frac{1}{K}\Big). \tag{2.56}$$

证明. 由于回溯线搜索得到的步长 α_k 满足 Armijo 条件 (2.48)，因此有

$$c\alpha_k|\nabla f(x_k)|^2 \le f(x_k) - f(x_{k+1})$$

对 $k = 0, 1, \ldots, K-1$ 求和，并注意到 $\alpha_k = \bar{\alpha}\rho^{l_k} \ge \bar{\alpha}\rho^{\bar{l}}$，可得

$$c\bar{\alpha}\rho^{\bar{l}} \sum_{k=0}^{K-1} |\nabla f(x_k)|^2 \le f(x_0) - f(x_K) \le f(x_0) - f^* < \infty \tag{2.57}$$

因为 $f(x_K) \ge f^* > -\infty$。这就证明了 (2.55) 以及第一条结论。

由 (2.53) 中 $\bar{l}$ 的定义可知

$$\bar{l} - 1 \le \frac{\log(2(1-c)\bar{\alpha}^{-1}L^{-1})}{\log\rho}$$

从而

$$\rho^{\bar{l}-1} \ge \frac{2(1-c)}{\bar{\alpha}L}.$$

因此，由 (2.57) 可得

$$\min_{0\le k\le K-1} |\nabla f(x_k)|^2 \le \frac{1}{K}\sum_{k=0}^{K-1} |\nabla f(x_k)|^2 \le \frac{L(f(x_0)-f^*)}{2c(1-c)\rho K}. \tag{2.58}$$

这就证明了第二条结论。 □

定理 2.4.3 使我们确信：由算法 2.4.1 生成的迭代序列 $\{x_k\}$ 以次线性速率 $O(1/k)$ 收敛到某个临界点。对于一般的光滑非凸优化问题，这一结果通常被认为是最好的收敛性结果之一。

与此同时，该算法本身并不要求对问题有太多先验了解：我们无需为步长的选取而苦恼，因为线搜索会自动找到合适的步长。此外，只要超参数（即算法中需要手动设定的参数）取值落在规定范围内，收敛性就能得到保证，而这些范围与具体问题无关。相比之下，回忆一下，如果使用常数步长 $\alpha_k \equiv \alpha \in (0, 2/L)$，则必须知道 ∇f 的 Lipschitz 常数 L，而这通常难以估计，并且会随着不同的 f 而变化。

尽管算法 2.4.1 具有上述令人满意的性质，但我们往往更关心优化算法在实践中的效率。原因在于，从实践角度看，即使牺牲一些超参数选择的自由度或收敛速率的理论保证，只要能够获得显著更快的经验收敛速度（从而减少求解所需的时间、资源和能耗），通常也是值得的。我们知道，当目标函数 f 是病态的（ill-conditioned）时，即使 f 是强凸的（此时条件数 L/μ 很大，其中 μ 是 f 的强凸参数），梯度下降法的收敛也会非常缓慢。因此，人们投入了大量工作来构造更高效的优化算法，尤其是针对大规模优化问题。接下来，我们考虑若干利用目标函数二阶信息来缓解优化问题病态性的优化方法。

Newton 法与拟 Newton 法

梯度下降法易于实现，因为其下降方向 v_k 只需在每个迭代点 x_k 处取为负梯度 $-\nabla f(x_k)$。然而，即使步长选取得像最速下降法那样，梯度下降法的收敛仍然可能非常慢。这主要是由于目标函数 f 的 Hessian 矩阵的病态性所致。现在我们考虑其另一个极端—— **Newton 法**。Newton 法在结构上比大多数下降法更复杂，但从收敛速度的角度来看，它是最快的优化方法之一。

Newton 法的迭代仍具有典型下降法的形式：

$$x_{k+1} = x_k + \alpha_k v_k^N, \tag{2.59}$$

其中关键在于 **Newton 方向**的选择，记为：

$$v_k^N := -(\nabla^2 f(x_k))^{-1} \nabla f(x_k). \tag{2.60}$$

Newton 方向的思想是：更新点 x_{k+1} 应当是原目标函数在 x_k 处二阶近似的极小点。我们考虑如下近似：

$$f(x) \approx \tilde{f}_k(x) := f(x_k) + \nabla f(x_k)(x - x_k) + \frac{1}{2}(x - x_k)^\top \nabla^2 f(x_k)(x - x_k).$$

对上述 $\tilde{f}_k$ 关于 x 求梯度可见，只要 $\nabla^2 f(x_k)$ 可逆，由 (2.59) 给出的 x_{k+1} 的确是 $\tilde{f}_k$ 的临界点。由于 Hessian 提供了目标函数在 x_k 附近的二阶信息，因此预期 (2.60) 中的 Newton 方向 v_k^N 比仅使用一阶信息的梯度下降法更有效。

由此可见，(2.60) 中的 Newton 方向涉及 Hessian $\nabla^2 f(x_k)$ 的逆。因此，若 $\nabla^2 f(x_k)$ 是奇异的，则 Newton 法无法运行。进一步地，若 $\nabla^2 f(x_k)$ 不是正定的，则 (2.60) 中定义的 v_k^N 甚至可能不是下降方向。这些问题通常可以通过步长线搜索、信赖域（trust region）技术、Gauss–Newton 近似等方式加以

修正。由于这里的目的是说明思想，并在后文展示 Newton 法及其重要变体的收敛性，我们暂时直接假设 v_k^N 存在且是下降方向。

我们还注意到，(2.60) 中的 Newton 步 v_k^N 计算代价非常高，因为它需要求 Hessian 矩阵的逆，其计算成本通常是关于 n 的三次方级别。我们稍后会讨论如何解决这一问题，这里暂时假设每一步迭代都能得到 v_k^N。

现在考虑使用 Newton 方向 v_k^N 且步长 α_k 为常数时，Newton 法的局部收敛性质。所谓局部收敛，是指对于满足充分最优性条件的某个局部极小值点 x^*（即 $\nabla f(x^*)=0$ 且 $\nabla^2 f(x^*)\succ 0$），若初始点足够接近 x^*，则 Newton 法收敛到 x^*。我们承认这些条件看起来较强，在实践中也不一定容易满足；但对于非凸问题而言，它们往往是目前已知最好的收敛性保证。Newton 法的这一具体收敛结果如下所示。

定理 2.4.4 (Newton 法的局部二次收敛)**.** *设 $f:\mathbb{R}^n\to\mathbb{R}$ 属于 C^2，且 $\nabla^2 f$ 是 Lipschitz 连续的，即存在 $M>0$ 使得*

$$|\nabla^2 f(\hat{x})-\nabla^2 f(x)|\le M|\hat{x}-x|$$

对任意 $x,\hat{x}\in\mathbb{R}^n$ 成立，其中 $|A|:=\max_{x:|x|\le 1}|Ax|$ 表示矩阵的 2-范数。设 x^* 是一个局部极小点，满足 $\nabla f(x^*)=0$ 且 $\nabla^2 f(x^*)\succ 0$，并设 $\{x_k\}$ 是采用 Newton 方向 v_k^N 且每一步步长均取 $\alpha_k=1$ 的 Newton 法所生成的序列，则有：

1. *若 x_0 足够接近 x^*，则当 $k\to\infty$ 时，$x_k\to x^*$，且收敛速率为二次（具体将在证明中给出）。*

2. *此外，当 $k\to\infty$ 时，$|\nabla f(x_k)|\to 0$，且收敛速率同样为二次。*

证明. 注意到

$$\begin{aligned}x_{k+1}-x^* &= x_k-v_k^N-x^*\\ &= x_k-(\nabla^2 f(x_k))^{-1}\nabla f(x_k)-x^*\\ &= (\nabla^2 f(x_k))^{-1}(\nabla^2 f(x_k)(x_k-x^*)-\nabla f(x_k)).\end{aligned}\tag{2.61}$$

由 Taylor 定理可知

$$\nabla f(x_k)-\nabla f(x^*)=\int_0^1 \nabla^2 f(x_k+t(x^*-x_k))(x_k-x^*)\,dt.\tag{2.62}$$

因此有

$$
\begin{aligned}
&|\nabla^2 f(x_k)(x_k - x^*) - (\nabla f(x_k) - \nabla f(x^*))| \\
&\qquad = \Big| \int_0^1 \Big(\nabla^2 f(x_k) - \nabla^2 f(x_k + t(x_k - x^*))\Big)(x_k - x^*)\, dt \Big| \\
&\qquad \le M|x_k - x^*|^2 \int_0^1 t\, dt \\
&\qquad = \frac{M}{2}|x_k - x^*|^2
\end{aligned} \tag{2.63}
$$

其中第一个等号来自 (2.62)，而不等号则利用了 $\nabla^2 f$ 的 M-Lipschitz 连续性。

由于 $\nabla^2 f(x^*) \succ 0$ 且 $f \in C^2$，可知在 x^* 的某个邻域内，$\nabla^2 f(x)$ 在每一点都可逆。进一步地，存在 $r > 0$，使得

$$
|(\nabla^2 f(x))^{-1}| \le 2|(\nabla^2 f(x^*))^{-1}| \tag{2.64}
$$

对所有 $x \in B_r(x^*)$ 成立。

结合 (2.61)、(2.63) 和 (2.64)，可得

$$
\begin{aligned}
|x_{k+1} - x^*| &\le \frac{M}{2} \cdot 2\,|(\nabla^2 f(x^*))^{-1}|\,|x_k - x^*|^2 \\
&= M\,|(\nabla^2 f(x^*))^{-1}|\,|x_k - x^*|^2.
\end{aligned}
$$

若选取 x_0 满足

$$
|x_0 - x^*| \le \min\Big(r, \frac{1}{2M|(\nabla^2 f(x^*))^{-1}|}\Big),
$$

则可通过归纳法证明存在常数 $C > 0$，使得

$$
|x_{k+1} - x^*| \le C|x_k - x^*|^2 \quad \text{和} \quad |x_k - x^*| < 1/C \tag{2.65}
$$

对所有 k 成立。这表明 $x_k \to x^*$，且 (2.65) 的第一个条件就是所谓的**二次收敛速率**。

此外，由 (2.60) 中 v_k^N 的定义可知

$$
\nabla f(x_k) + \nabla^2 f(x_k) v_k^N = 0
$$

因此

$$
\begin{aligned}
|\nabla f(x_{k+1})| &= |\nabla f(x_{k+1}) - \nabla f(x_k) - \nabla^2 f(x_k) v_k^N| \\
&= \Big| \int_0^1 \nabla^2 f(x_k + t(x_{k+1} - x_k))(x_{k+1} - x_k)\, dt - \nabla^2 f(x_k) v_k^N \Big| \\
&= \Big| \int_0^1 \nabla^2 f(x_k + t v_k^N) v_k^N\, dt - \nabla^2 f(x_k) v_k^N \Big| \\
&\le \int_0^1 |\nabla^2 f(x_k + t v_k^N) - \nabla^2 f(x_k)| |v_k^N|\, dt \\
&\le \frac{M}{2} |v_k^N|^2 \\
&\le \frac{M}{2} |(\nabla^2 f(x_k))^{-1}|^2 |\nabla f(x_k)|^2 \\
&\le 2M |(\nabla^2 f(x^*))^{-1}|^2 |\nabla f(x_k)|^2,
\end{aligned}
$$

这说明 $|\nabla f(x_k)|$ 也以二次速率收敛到 0。 □

现在我们回到之前尚未解决的问题：如何计算 (2.60) 中的 v_k^N。对于大规模问题，即 n 达到百万量级甚至更高时，直接求 $\nabla^2 f(x_k)$ 的逆是不可行的，因为这需要 $O(n^3)$ 的计算成本。与其直接求逆，我们可以通过求解如下线性方程组来得到 v_k^N：

$$\nabla^2 f(x_k) v = -\nabla f(x_k). \tag{2.66}$$

这就是标准线性系统

$$Qx = b \tag{2.67}$$

的形式，其中 Q 是对称正定矩阵（这里假设 (2.66) 中的 $\nabla^2 f(x_k)$ 是正定的；当 x_k 足够接近满足 $\nabla^2 f(x^*) \succ 0$ 的 x^* 时，这确实成立）。

针对如 (2.67) 这样的巨大线性系统，人们已经设计出了大量数值方法。这里我们只介绍**共轭梯度法**（conjugate gradient，CG），这是求解 (2.67) 最经典且高效的方法之一。求解线性系统的标准 CG 方法见算法 2.4.2。需要注意的是，若所有计算都精确进行，则 CG 方法至多在 n 步内收敛；其每一步的主要计算开销是矩阵向量乘法 Qd_j（见 (2.68a)），而这一结果在第 j 次迭代中还会被重复利用来计算其他量。在实践中，通常只需约 $\sqrt{n}$ 步迭代就能非常接近 $Qx = b$ 的真实解，这取决于 Q 的条件数（即 Q 的最大与最小特征值之比）。若条件数过大，也有许多对 CG 的预条件修正方法可以缓解病态性问题 [105]。若 Hessian 是稀疏的，则计算成本还能进一步降低。因此，通过求解 (2.66) 来估计 v_k^N，CG 是一种在实践中可行的方法。

Algorithm 2.4.2 用于求解 $Qx = b$ 的共轭梯度（CG）方法

Input: Initial $x_0 \in \mathbb{R}^n$, $g_0 = Qx_0 - b$, $d_0 = -g_0$.

1: **for** $j = 0, 1, , \ldots, n-1$ **do**

2:

$$\alpha_j = \frac{g_j \cdot d_j}{g_j \cdot (Qd_j)}, \tag{2.68a}$$

$$x_{j+1} = x_j + \alpha_j d_j, \tag{2.68b}$$

$$g_{j+1} = Qx_{j+1} - b = g_j + \alpha_j (Qd_j), \tag{2.68c}$$

$$\beta_j = \frac{g_j \cdot (Qd_j)}{d_j \cdot (Qd_j)}, \tag{2.68d}$$

$$d_{j+1} = -g_{j+1} + \beta_j d_j. \tag{2.68e}$$

3: **end for**

Output: x_n.

总之，虽然 Newton 法在其原始形式下对大规模问题而言计算上不可行，但存在许多变通方法可以得到高效的**非精确 Newton 法**。上面使用 CG 来近似 Newton 步的方法，即所谓的 Newton–CG 方法，就是其中之一。需要指出的是，这类非精确 Newton 法通常并不显式计算或存储 Hessian 矩阵，而只需要若干 Hessian-vector 乘积。因此，尽管它们使用了 f 的二阶导数，仍常被称为 Hessian-free 方法。

另一类 Hessian-free 方法称为**拟 Newton 法**（quasi-Newton method），它们通常被认为是在小规模到中等规模的光滑非凸优化问题中最优先考虑的方法。拟 Newton 法的主要思想如下。假设迭代点 x_{k+1} 足够接近 x_k，则有

$$\begin{aligned}\nabla f(x_k) &= \nabla f(x_{k+1}) + \nabla^2 f(x_{k+1})(x_k - x_{k+1}) + o(|x_k - x_{k+1}|) \\ &\approx \nabla f(x_{k+1}) + \nabla^2 f(x_{k+1})(x_k - x_{k+1}).\end{aligned}$$

将其整理，并假设该近似足够准确，则得到

$$\nabla^2 f(x_{k+1})(x_{k+1} - x_k) = \nabla f(x_{k+1}) - \nabla f(x_k). \tag{2.69}$$

如果能找到一个矩阵 $B_k \in \mathbb{R}^{n \times n}$，在每一步迭代中起到 Hessian $\nabla^2 f(x_k)$ 的替代作用，则 (2.69) 可写为

$$B_{k+1}(\underbrace{x_{k+1} - x_k}_{=:\, s_k}) = \underbrace{\nabla f(x_{k+1}) - \nabla f(x_k)}_{=:\, y_k}. \tag{2.70}$$

方程 (2.70) 称为**割线方程**，满足该条件的优化算法称为**拟 Newton 法**。若每一步都能得到这样的 B_k，则拟 Newton 法的更新格式为

$$x_{k+1} = x_k - \alpha_k B_k^{-1} \nabla f(x_k), \tag{2.71}$$

其中步长 α_k 通常通过线搜索选取，使其满足 Wolfe 条件 (2.48) 和 (2.50)。

在实践中，一个真正可用的拟 Newton 法 不仅需要满足 (2.70)，还要求 B_k 易于更新，并且其逆（或对任意 $b \in \mathbb{R}^n$ 的 $B_k^{-1}b$）易于计算。Broyden–Fletcher–Goldfarb–Shannon（BFGS）方法是满足这些条件的最著名拟 Newton 法之一。BFGS 中 B_k 的更新公式为（注意 y_k 是行向量，s_k 是列向量）：

$$B_{k+1} = B_k + \frac{y_k^\top y_k}{y_k s_k} - \frac{(B_k s_k)(B_k s_k)^\top}{s_k^\top (B_k s_k)}, \tag{2.72}$$

其中所有分母均为标量，而分子均为秩为 1 的矩阵。

在 BFGS 的实现中，我们还需要在每一步更新 $H_k := B_k^{-1}$，以便利用 H_k 轻松计算 (2.71)。为此，我们利用 Sherman–Morrison–Woodbury 公式：若矩阵 $A \in \mathbb{R}^{n\times n}$ 可逆，且两个列向量 $u, v \in \mathbb{R}^n$ 满足 $1 + v^\top A u \neq 0$，则

$$(A + uv^\top)^{-1} = A^{-1} - \frac{(A^{-1}u)(v^\top A^{-1})}{1 + v^\top A^{-1} u}. \tag{2.73}$$

该公式可以通过直接计算验证。

我们重复使用 Sherman–Morrison–Woodbury 公式 (2.73)，可得到 H_k 的更新公式：

$$\begin{aligned}
H_{k+1} &= B_{k+1}^{-1} \\
&= \Big(B_k + \frac{y_k^\top y_k}{y_k s_k} - \frac{(B_k s_k)(B_k s_k)^\top}{s_k^\top (B_k s_k)}\Big)^{-1} \\
&= H_k + \Big(1 + \frac{y_k H_k y_k^\top}{y_k s_k}\Big)\frac{s_k s_k^\top}{y_k s_k} - \frac{H_k(s_k y_k^\top) + (s_k y_k^\top) H_k}{y_k s_k}.
\end{aligned}$$

这就是 BFGS 方法中 H_k 的更新公式。

BFGS 方法（以及所有拟 Newton 法）的一个缺点在于：它们需要存储一个可能是稠密的 $n \times n$ 矩阵 H_k（即使真实 Hessian 是稀疏的），这限制了其在大规模问题中的应用。有一种变体称为 limited-memory BFGS（L-BFGS），它只利用最近若干步（通常 3 到 20 步）迭代中的曲率信息来构造 H_k 的近似。更多细节可参见 [105]。当某个迭代点 x_k 足够接近满足 $\nabla^2 f(x^*) \succ 0$ 的局部

极小点 x^* 时，可以证明 BFGS 方法以超线性速率收敛，即

$$\lim_{k\to\infty}\frac{|x_{k+1}-x^*|}{|x_k-x^*|}=0.$$

相比之下，L-BFGS 方法具有线性收敛速率，即

$$\lim_{k\to\infty}\frac{|x_{k+1}-x^*|}{|x_k-x^*|}=\gamma$$

其中常数 $\gamma\in(0,1)$ [105]。

2.4.2 约束优化算法

在本小节中，我们介绍若干约束优化算法。我们指出，这些算法在网络训练中很少直接使用，因为我们通常会避免将这类大规模优化问题构造成约束形式，因为那样会更具挑战性。不过，这些算法为如何利用无约束优化算法来求解约束问题提供了很有启发性的思路。当我们希望对网络参数施加额外约束与正则化，或者设计自适应网络结构时，它们可能会很有用。

求解约束优化问题的一大类方法，是将原问题替换为一列无约束优化问题。这些无约束问题的解可以收敛到原约束问题的解。我们首先介绍这一类方法中一个常用的算法。

二次罚函数法

我们考虑带有等式约束的优化问题 (2.17)，这里将其重述如下：

$$\min_{x\in\mathbb{R}^n}\quad f(x), \tag{2.74a}$$

$$\text{s.t.}\quad h(x)=0. \tag{2.74b}$$

我们介绍的第一个算法称为**二次罚函数法**（quadratic penalty method）。该方法将约束问题 (2.74) 修改为如下无约束问题

$$\min_{x\in\mathbb{R}^n} Q_\gamma(x), \tag{2.75}$$

其中**二次罚函数** Q_γ 定义为

$$Q_\gamma(x):=f(x)+\frac{1}{2\gamma}|h(x)|^2 \tag{2.76}$$

而 $\gamma > 0$ 称为**罚参数**。该罚参数 γ 用于惩罚约束 $h(x) = 0$ 的违反程度：γ 越小，惩罚越重。因此，设定一列罚参数 $\{\gamma_k\}$ 使得当 $k \to \infty$ 时 $\gamma_k \to 0^+$ 是合理的。同时，对每个 k，我们以某个初值求解对应 $\gamma = \gamma_k$ 的无约束极小化问题 (2.75)，得到一个解 x_k，并期望解序列 $\{x_k\}$ 收敛到原约束问题 (2.74) 的一个解。如果我们明智地选择 γ_k 和初值，那么无约束问题 $\min_x Q_{\gamma_k}$ 可能会很快被求解。

现在我们来描述上述二次罚函数法的收敛性质。

定理 2.4.5. *设 x_k 是 Q_{γ_k} 的全局极小值点，并且 $\gamma_k \to 0^+$。那么，$\{x_k\}$ 的每一个聚点[2] x^* 都是约束问题 (2.74) 的一个全局极小值点。*

证明. 设 $\bar{x}$ 是 (2.74) 的一个全局极小值点，即 $h(\bar{x}) = 0$ 且

$$f(\bar{x}) \le f(x)$$

对所有满足 $h(x) = 0$ 的 x 成立。另一方面，由于 x_k 是 Q_{γ_k} 在 $\mathbb{R}^n$ 上的全局极小值点，因此有 $Q_{\gamma_k}(x_k) \le Q_{\gamma_k}(\bar{x})$，即

$$f(x_k) + \frac{1}{2\gamma_k}|h(x_k)|^2 \le f(\bar{x}) + \frac{1}{2\gamma_k}|h(\bar{x})|^2 = f(\bar{x}), \tag{2.77}$$

或者等价地，

$$|h(x_k)|^2 \le 2\gamma_k(f(\bar{x}) - f(x_k)).$$

由于假设 f 有一个有限下界 f^*，因此有

$$|h(x_k)|^2 \le 2\gamma_k(f(\bar{x}) - f(x_k)) \le 2\gamma_k(f(\bar{x}) - f^*),$$

其中 $f(\bar{x}) - f^*$ 是一个有限常数。

设 x^* 是 $\{x_k\}$ 的一个聚点，则存在 $\{x_k\}$ 的一个子序列 $\{x_{k_j}\}$，使得当 $j \to \infty$ 时 $x_{k_j} \to x^*$。

由 h 的连续性可得

$$|h(x^*)|^2 = \lim_{j\to\infty} |h(x_{k_j})|^2 \le \lim_j 2\gamma_{k_j}(f(\bar{x}) - f^*) = 0,$$

这是因为当 $j \to \infty$ 时，有 $\gamma_{k_j} \to 0$。因此 $h(x^*) = 0$，这说明 x^* 是一个可行点。

[2]如果序列 $\{x_k\}$ 存在一个子序列，例如 $\{x_{k_j}\}$，使得当 $j \to \infty$ 时 $\{x_{k_j}\}$ 收敛到 $\hat{x}$，则称 $\hat{x}$ 是序列 $\{x_k\}$ 的一个聚点（accumulation point）。聚点也称为极限点（limit point）。

Algorithm 2.4.3 用于求解 (2.74) 的二次罚函数法

Input: Initial penalty parameter $\gamma_0 > 0$, tolerance ϵ_0 and guess $x_{0,0} \in \mathbb{R}^n$.

1: **for** $k = 0, 1, 2, \ldots$ **do**
2: Start from $x_{k,0}$, find x_k such that $|\nabla Q_{\gamma_k}(x_k)| \leq \epsilon_k$.
3: **if** x_k is "satisfactory" **then**
4: Terminate.
5: **end if**
6: Choose $\gamma_{k+1} \in (0, \gamma_k)$.
7: Choose $\epsilon_{k+1} \in (0, \epsilon_k)$.
8: Choose an initial guess $x_{k+1,0}$ for the next iteration.
9: **end for**

Output: x_k.

进一步地，由 (2.77) 可知，对所有 k 都有 $f(x_k) \leq f(\bar{x})$。由于 f 是连续的，因此有

$$f(x^*) = \lim_{j\to\infty} f(x_{k_j}) \leq f(\bar{x}).$$

由于 $\bar{x}$ 是全局极小值点，因此必有 $f(x^*) = f(\bar{x})$，并且 x^* 也是一个全局极小值点。 □

虽然定理 2.4.5 给出了一个非常理想的收敛性质，但在实践中很难得到非凸函数 Q_γ 的全局极小值点。

接下来，我们介绍一个更实际的算法。对于任意 k，它不需要 Q_{γ_k} 的全局极小值点，但在一定条件下仍能保证收敛到 (2.74) 的一个 Lagrange 点（即满足 Lagrange 条件，但不一定是全局极小值点）。该算法见算法 2.4.3。

备注. 关于算法 2.4.3，我们有几点说明。首先，在适当的光滑性条件下，算法 2.4.3 第 2 行中的 x_k 在实践中是可以找到的。更具体地说，注意到 $\gamma_k > 0$ 且

$$\nabla Q_{\gamma_k}(x) = \nabla f(x) + \frac{1}{\gamma_k} h(x)^\top \nabla h(x).$$

如果 f 和 $h^\top \nabla h$ 都是 Lipschitz 连续的，那么 Q_{γ_k} 也是 Lipschitz 连续的。因此，存在某种算法，例如带回溯线搜索的梯度下降法（算法 2.4.1），能够从任意初值 $x_{k,0}$ 出发，在有限步内找到 x_k。但是需要注意，$h^\top \nabla h(x)$ 在很多情况下未必是 Lipschitz 连续的，例如 $h(x) = x^2$。不过，如果 h 是仿射函数，则它是 Lipschitz 的。

其次，我们常常令 γ_k 递减、ϵ_k 非增，但最终都需要满足当 $k \to \infty$ 时 $\gamma_k \to 0^+$ 且 $\epsilon_k \to 0^+$。一个常用的方法是用某个预先指定的 $(0, 1)$ 中常数来缩

放 γ_k 和 ϵ_k。

第三，初值的一个典型选择是 $x_{k+1,0} = x_k$。但也可能存在更好的选择。

最后但同样重要的是，我们注意到算法 2.4.3 在 x_k “足够满意” 时终止，这听起来似乎比较主观。在实践中，我们会在 γ_k、$|h(x_k)|$ 和 $|\nabla Q_{\gamma_k}(x_k)|$（这些量都可以计算）都足够小时终止算法。

我们看到，算法 2.4.3 是一种很有意思的方法：由于 $\epsilon_k \to 0$，我们有

$$|\nabla Q_{\gamma_k}(x_k)| = \left|\nabla f(x_k) + \frac{1}{\gamma_k} h(x_k)^\top \nabla h(x_k)\right| \le \epsilon_k.$$

若记 $\lambda_k := h(x_k)^\top/\gamma_k$，那么 $\nabla f(x_k) + \lambda_k \nabla h(x_k)$ 可能会接近于 0。因此，如果 (x_k, λ_k) 有一个聚点 (x^*, λ^*)，那么对于某个子序列 $\{(x_{k_j}, \lambda_{k_j})\}$，我们将有

$$\nabla f(x^*) + \lambda^* \nabla h(x^*) = \lim_{j\to\infty} \nabla f(x_{k_j}) + \lambda_{k_j} \nabla h(x_{k_j}) = 0$$

从而 (x^*, λ^*) 满足第一个 Lagrange 条件。因此，剩下要证明的是 $h(x^*) = 0$，即第二个 Lagrange 条件。下面的定理给出了一个完整结果。

定理 2.4.6. *在算法 2.4.3 中，设当 $k \to \infty$ 时 $\epsilon_k \to 0^+$ 且 $\gamma_k \to 0^+$，那么对于 $\{x_k\}$ 的任意一个聚点 x^*，若 x^* 是正则点，则 x^* 是 (2.74) 的一个 Lagrange 点。此外，对于任意收敛到 x^* 的子序列 $\{x_{k_j}\}$，记 $\lambda_{k_j} := \nabla h(x_{k_j})/\gamma_{k_j}$，则当 $j \to \infty$ 时 λ_{k_j} 也收敛到某个 λ^*，且 (x^*, λ^*) 满足 Lagrange 条件。*

证明. 设 $\{x_{k_j}\}$ 是 $\{x_k\}$ 的任意一个子序列，满足当 $j \to \infty$ 时 $x_{k_j} \to x^*$。回忆一下，对任意固定的 $\gamma > 0$，有 $\nabla Q_\gamma(x) = \nabla f(x) + \frac{1}{\gamma} h(x)^\top \nabla h(x)$。根据算法 2.4.3 第 1 行中 x_k 的性质，有

$$\left|\nabla f(x_{k_j}) + \frac{1}{\gamma_{k_j}} h(x_{k_j})^\top \nabla h(x_{k_j})\right| \le \epsilon_{k_j},$$

这意味着

$$|h(x_{k_j})^\top \nabla h(x_{k_j})| \le \gamma_{k_j}(\epsilon_{k_j} + |\nabla f(x_{k_j})|).$$

由于 f、h 和 ∇h 的连续性，以及当 $j \to \infty$ 时 $x_{k_j} \to x^*$、$\epsilon_{k_j} \to 0$、$\gamma_{k_j} \to 0$，可知

$$|h(x^*)^\top \nabla h(x^*)| = \lim_{j\to\infty} |h(x_{k_j})^\top \nabla h(x_{k_j})| = 0.$$

由于 x^* 是正则点，即 $\nabla h(x^*)$ 具有满行秩，因此有 $h(x^*) = 0$，从而 x^* 是一个可行点。

接下来我们证明 λ_{k_j} 当 $j \to \infty$ 时收敛到某个 λ^*。为此，注意到 ∇h 和行列式 $\det(\cdot)$ 都是连续函数，而 $\nabla h(x^*)$ 具有满行秩。因此，x^* 是集合

$$\{x \in \mathbb{R}^n : \nabla h(x)\text{是满行秩的}\} = \{x \in \mathbb{R}^n : \det(\nabla h(x)\nabla h(x)^\top) > 0\}$$

的一个内点。因此，对所有充分大的 j，我们知道 $\nabla h(x_{k_j})\nabla h(x_{k_j})^\top$ 具有满秩，因此可逆。

令 $\lambda_{k_j} := h(x_{k_j})^\top/\gamma_{k_j}$。则有

$$\lambda_{k_j}\nabla h(x_{k_j}) = \nabla Q_{\gamma_{k_j}}(x_{k_j}) - \nabla f(x_{k_j}). \tag{2.78}$$

由于 $\nabla h(x_{k_j})\nabla h(x_{k_j})^\top$ 可逆，因此有

$$\lambda_{k_j} = \big(\nabla Q_{\gamma_{k_j}}(x_{k_j}) - \nabla f(x_{k_j})\big)\nabla h(x_{k_j})^\top\big(\nabla h(x_{k_j})\nabla h(x_{k_j})\big)^{-1}.$$

令 $j \to \infty$，我们有 $|\nabla Q_{\gamma_{k_j}}(x_{k_j})| \le \epsilon_{k_j} \to 0$，并且

$$\begin{aligned}\lim_{j\to\infty}\lambda_{k_j} &= \lim_{j\to\infty}\Big(\nabla Q_{\gamma_{k_j}}(x_{k_j}) - \nabla f(x_{k_j})\Big)\nabla h(x_{k_j})^\top\Big(\nabla h(x_{k_j})\nabla h(x_{k_j})^\top\Big)^{-1}\\ &= -\nabla f(x^*)\nabla h(x^*)^\top(\nabla h(x^*)\nabla h(x^*)^\top)^{-1}\\ &=: \lambda^*.\end{aligned}$$

这说明 $\{\lambda_{k_j}\}$ 收敛到上述定义的 λ^*。

此外，在 (2.78) 中令 $j \to \infty$，可得

$$\nabla f(x^*) + \lambda^*\nabla h(x^*) = 0.$$

因此 (x^*, λ^*) 满足 Lagrange 条件 (2.20)。 □

备注. 定理 2.4.6 中要求 x^* 为正则点这一条件是必要的，否则我们无法得到 $h(x^*) = 0$，这意味着 x^* 可能不是可行点。不过，我们仍然可以证明 $h(x^*)^\top\nabla h(x^*) = 0$，这意味着 x^* 是 $|h(x)|^2$ 的一个驻点。

我们还需要注意，当 γ_k 趋于 0 时，在极小化 Q_{γ_k} 的过程中会出现病态性问题。注意到

$$\nabla Q_{\gamma_k}(x) = \nabla f(x) + \frac{1}{\gamma_k}h(x)^\top\nabla h(x) = \nabla f(x) + \frac{1}{\gamma_k}\sum_{i=1}^m h_i(x)\nabla h_i(x)$$

以及

$$\nabla^2 Q_{\gamma_k}(x) = \nabla^2 f(x) + \frac{1}{\gamma_k}\sum_{i=1}^{m} h_i(x)\nabla^2 h_i(x) + \frac{1}{\gamma_k}\nabla h(x)^\top h(x).$$

若令 $x = x_{k_j}$ 且 $\gamma_k = \gamma_{k_j}$，并注意到当 $j \to \infty$ 时 $x_{k_j} \to x^*$ 且 $\lambda_{k_j} = h(x_{k_j})^\top/\gamma_{k_j} \to \lambda^*$，则我们预期 $\nabla^2 Q_{\gamma_{k_j}}(x_{k_j})$ 接近于

$$\nabla^2 f(x^*) + \sum_{i=1}^{m} \lambda_i^* \nabla^2 h_i(x^*) + \lim_{j\to\infty} \frac{1}{\gamma_{k_j}} \nabla h(x_{k_j})^\top \nabla h(x_{k_j}). \tag{2.79}$$

前两项之和趋近于 Lagrange 函数 $l(x,\lambda) = f(x) + \lambda h(x)$ 关于 x 在 (x^*, λ^*) 处的 Hessian，即 $\partial_x^2 l(x,\lambda)$。若 x^* 是严格局部极小点，则该 Hessian 的特征值是固定的且为正。然而，(2.79) 中最后一项是棘手的：我们知道 $\nabla h(x_{k_j})^\top \nabla h(x_{k_j})$ 趋近于 $\nabla h(x^*)^\top \nabla h(x^*) \in \mathbb{R}^{n\times n}$，而后者是不可逆的的，因为 $m < n$。在这种情况下，$1/\gamma_{k_j}$ 会把所有正特征值放大，从而当 $\gamma_{k_j} \to 0$ 时，使得 $\nabla^2 Q_{\gamma_{k_j}}$ 在 x^* 处的条件数变得非常大。这意味着，在实践中极小化 $Q_{\gamma_{k_j}}$ 可能非常困难。对此有一种缓解方法；参见 [105, Chapter 17.1]。

Lagrange 方法

我们考虑一种针对带等式约束优化问题的替代方法，它基于 Lagrange 函数：

$$l(x,\lambda) = f(x) + \lambda h(x), \tag{2.80}$$

其中 $f, h \in C^2$。该 **Lagrange 方法**定义为

$$x_{k+1} = x_k - \alpha_k(\nabla f(x_k) + \lambda_k \nabla h(x_k))^\top, \tag{2.81a}$$

$$\lambda_{k+1} = \lambda_k + \beta_k h(x_k)^\top, \tag{2.81b}$$

其中 k 是迭代计数器，$\alpha_k, \beta_k > 0$ 是步长。转置 $^\top$ 是为了使向量形状与我们的约定一致，即 $x \in \mathbb{R}^n$ 是列向量，而 $\lambda \in \mathbb{R}^m$ 是行向量。

在讨论 Lagrange 方法 (2.81) 的性质之前，我们先给出一种直观的方式来记忆该方法。对任意 $x \in \mathbb{R}^n$，注意到

$$\max_{\lambda\in\mathbb{R}^m} l(x,\lambda) = \begin{cases} f(x), & \text{if } h(x) = 0, \\ +\infty, & \text{if } h(x) \neq 0. \end{cases}$$

因此，我们可以将等式约束优化问题 (2.74) 重写为

$$\min_{x\in\mathbb{R}^n}\{f(x):h(x)=0\}=\min_{x\in\mathbb{R}^n}\max_{\lambda\in\mathbb{R}^m} l(x,\lambda).$$

因此，Lagrange 方法 (2.81) 可以理解为：对 x 做梯度下降以极小化 $l(x,\lambda)$，同时对 λ 做梯度上升以极大化它。由于 (2.81) 中只需要梯度和 Jacobian，我们称其为一阶 Lagrange 方法。

现在我们考察 Lagrange 方法 (2.81) 的收敛性质。为简化分析，我们假设步长为常数，即对某些 $\alpha,\beta>0$，有 $\alpha_k=\alpha$ 且 $\beta_k=\beta$。通过对 λ 做适当缩放，不失一般性地可设 $\beta=\alpha$。在这种情况下，Lagrange 方法 (2.81) 可以看作一个形如 $z_{k+1}=F(z_k)$ 的**不动点迭代**，其中 $z_k:=(x_k,\lambda_k)\in\mathbb{R}^{n+m}$ 是由 x 与 λ 拼接而成的列向量，而

$$F(z):=\begin{pmatrix} x-\alpha(\nabla f(x)+\lambda\nabla h(x))^\top \\ \lambda^\top+\alpha h(x)\end{pmatrix} \tag{2.82}$$

对任意 $z=(x,\lambda)\in\mathbb{R}^{n+m}$ 成立。这里所谓 F 的不动点，是指满足 $z=F(z)$ 的点 z。在此定义下，有如下引理，其证明非常简单，因此略去。

引理 2.4.7. *一个点 $z^*=(x^*,\lambda^*)$ 满足 Lagrange 条件*

$$\nabla f(x^*)+\lambda^*\nabla h(x^*)=0, \tag{2.83a}$$

$$h(x^*)=0 \tag{2.83b}$$

当且仅当 z^* 是 F 的一个不动点。

接下来，我们证明：若 (x^*,λ^*) 满足二阶充分条件，即 (x^*,λ^*) 满足 (2.83) 以及

$$\nabla^2 f(x^*)+\sum_{i=1}^m \lambda_i^*\nabla^2 h_i(x^*)\succ 0, \tag{2.84}$$

则只要初值 (x_0,λ_0) 足够接近 (x^*,λ^*) 且 α 足够小，由 Lagrange 方法 (2.81) 生成的序列 $\{(x_k,\lambda_k)\}$ 就会收敛到 (x^*,λ^*)。

定理 2.4.8. *设 (x^*,λ^*) 满足等式约束优化问题 (2.74) 的二阶充分条件，即满足 (2.83) 和 (2.84)，并且 $\alpha,\beta>0$ 足够小，则存在 (x^*,λ^*) 的一个邻域，使得对该邻域中的任意初值 (x_0,λ_0)，由 Lagrange 方法 (2.81) 生成的序列 $\{(x_k,\lambda_k)\}$ 收敛到 (x^*,λ^*)。*

证明. 如前所述，通过适当缩放 λ，我们可假设 $\alpha = \beta$。注意 Lagrange 方法即为 $z_{k+1} = F(z_k)$。由引理 2.4.7，我们只需证明 z_k 收敛到 F 的一个不动点。由于 $z^* = (x^*, \lambda^*)$ 是 F 的不动点，因此有

$$|z_{k+1} - z^*| = |F(z_k) - F(z^*)|.$$

注意到

$$\nabla F(z) = I + \alpha \begin{pmatrix} -H(x,\lambda) & -\nabla h(x)^\top \\ \nabla h(x) & 0 \end{pmatrix} \in \mathbb{R}^{(n+m)\times(n+m)},$$

其中

$$H(x,\lambda) := \nabla^2 f(x) + \sum_{i=1}^{m} \lambda_i \nabla^2 h_i(x).$$

另一方面，由均值定理，存在一个关于 z_k 和 z^* 的矩阵值函数 M（M 依赖于 F），使得

$$F(z_k) - F(z^*) = M(z_k, z^*)(z_k - z^*),$$

其中 $(M(z_k, z^*))_i$，即 $M(z_k, z^*)$ 的第 i 行，是 ∇F_i 在连接 z_k 与 z^* 的线段上的某一点处的取值。两边取范数，得到

$$|F(z_k) - F(z^*)| \le |M(z_k, z^*)||z_k - z^*|. \tag{2.85}$$

现在我们断言：当 $\alpha > 0$ 足够小时，有 $|\nabla F(z^*)| < 1$。为此，注意到 $\nabla F(z^*) = I + \alpha M^*$，其中

$$M^* := \begin{pmatrix} -H(x^*,\lambda^*) & -\nabla h(x^*)^\top \\ \nabla h(x^*) & 0 \end{pmatrix}.$$

因此，只需证明 M^* 的所有特征值都具有负实部，因为这样一来当 $\alpha > 0$ 足够小时，$\nabla F(z^*) = I + \alpha M^*$ 的所有特征值实部都落在 $(0,1)$ 中。

设 ξ 是 M^* 的任意一个特征值，$z = (x, \lambda)$ 是对应特征向量。（注意根据特征向量的定义，$z \neq 0$。）则 $M^* z = \xi z$。左乘 $z^{\top}$[3]，可得 $z^\top M^* z = \xi |z|^2$。因此

$$\mathrm{Re}(z^\top M^* z) = \mathrm{Re}(\xi)|z|^2 \tag{2.86}$$

[3]注意，由于 z 可能是复向量，这里的记号 $^\top$ 表示复共轭转置。

其中 $\mathrm{Re}(\cdot)$ 表示实部。此外，注意到

$$\begin{aligned}\mathrm{Re}(z^\top M^* z) &= -\mathrm{Re}(x^\top H(x^*,\lambda^*)x) - \mathrm{Re}(x^\top \nabla h(x^*)^\top \lambda^*) \\ &\qquad + \mathrm{Re}(\lambda^{*\top} \nabla h(x^*)x) \\ &= -\mathrm{Re}(x^\top H(x^*,\lambda^*)x).\end{aligned} \tag{2.87}$$

由于 (2.84)，有 $H(x^*,\lambda^*) \succ 0$，因此只要 $x \neq 0$，便有 $\mathrm{Re}(x^\top H(x^*,\lambda^*)x) > 0$。

现在可见，若 $x \neq 0$，则比较 (2.86) 与 (2.87)，再注意到 $z \neq 0$，便得 $\mathrm{Re}(\xi) < 0$。

若 $x = 0$，则 $z = (0,\lambda)$。比较 $M^* z = \xi z$ 两边的前 n 个分量，可得 $\nabla h(x^*)^\top \lambda = 0$。由于 x^* 是正则点，$\nabla h(x^*)$ 具有满行秩，因此可知 $\lambda = 0$。但这意味着 $z = 0$，与 $z \neq 0$ 矛盾。因此，M^* 的所有特征值都具有负实部。

至此，我们知道：当 $\alpha > 0$ 足够小时，有

$$|\nabla F(z^*)| = |I + \alpha M^*| < 1.$$

由 ∇F 与范数的连续性，存在 $r > 0$ 和 $\kappa \in (0,1)$，使得对所有 $z \in \bar{B}_r(z^*) := \{z : |z - z^*| \leq r\}$，都有 $M(z,z^*) \leq \kappa$。

若 $z_0 = (x_0,\lambda_0) \in \bar{B}_r(z^*)$，则由 (2.85) 有

$$|z_1 - z^*| \leq |M(z_0,z^*)|\,|z_0 - z^*| \leq \kappa |z_0 - z^*| \leq r,$$

因此也有 $|M(z_1,z^*)| \leq \kappa \in (0,1)$。若 $z_k \in \bar{B}_r(z^*)$，则同理可得

$$|z_{k+1} - z^*| \leq \kappa |z_k - z^*| \leq r$$

且 $|M(z_{k+1},z^*)| \leq \kappa \in (0,1)$。由归纳法可知，$z_k$ 以线性速率收敛到 z^*。 □

增广 Lagrange 方法

在第 2.2.3 节中，我们引入了 Lagrange 函数的一个变体，称为**增广 Lagrange 函数**，用以推导约束优化问题的二阶充分条件。与之对应，也存在一种求解约束优化问题的方法，称为**增广 Lagrange 方法**，下面我们对其进行讨论。

需要指出的是，增广 Lagrange 方法也与二次罚函数法有关。两者的区别在于：由于引入了 Lagrange 乘子，增广 Lagrange 方法能够有效缓解二次罚

函数法中的病态性问题。

为了理解增广 Lagrange 方法的构造动机，我们先回顾等式约束优化问题：

$$\min_{x\in\mathbb{R}^n} \quad f(x), \tag{2.88a}$$

$$\text{s.t.} \quad h(x)=0. \tag{2.88b}$$

我们还回忆一下，在定理 2.4.6 中证明二次罚函数法收敛时，序列 $\{h(x_k)^\top/\gamma_k\}$ 的聚点实际上正是对应于序列 $\{x_k\}$ 的聚点 x^* 的 Lagrange 乘子 λ^*。因此，我们期望当 k 足够大时，有 $h(x_k)\approx\gamma_k\lambda^*$。在二次罚函数法中，我们需要 $\gamma_k\to 0^+$（以便得到 $h(x_k)\to 0$），而这会导致 $\nabla^2 Q_{\gamma_k}$ 的病态性。而在增广 Lagrange 方法中，这一问题可以得到缓解，从而 γ_k 不需要像二次罚函数法那样非常快地趋于 0，同时仍能保证 $h(x_k)$ 很快趋于 0。

为此，我们回忆增广 Lagrange 函数 定义如下，它是原问题 (2.88) 的 Lagrange 函数与二次罚函数的“混合”：

$$l_A(x,\lambda;\gamma):=f(x)+\lambda h(x)+\frac{1}{2\gamma}|h(x)|^2. \tag{2.89}$$

对 l_A 关于 x 求梯度，有

$$\begin{aligned}\nabla_x l_A(x,\lambda;\gamma)&=\nabla f(x)+\lambda\nabla h(x)+\frac{1}{\gamma}h(x)^\top\nabla h(x)\\&=\nabla f(x)+\Big(\lambda+\frac{1}{\gamma}h(x)^\top\Big)\nabla h(x).\end{aligned}$$

假设 k 足够大，并记 x_k 为 $l_A(x,\lambda_k;\gamma_k)$ 的极小点，则我们期望得到

$$\lambda_k+\frac{h(x_k)^\top}{\gamma_k}\approx\lambda^*,$$

或者等价地，

$$h(x_k)^\top\approx\gamma_k(\lambda^*-\lambda_k),$$

当 (x_k,λ_k) 接近 (x^*,λ^*) 时成立。若 $\lambda_k\to\lambda^*$，则 $\lambda^*-\lambda_k\to 0$，因此我们可能不需要像二次罚函数法那样让 $\gamma_k\to 0$ 得那么快。这也启发我们在每次更新 x_k 后，作如下更新

$$\lambda_{k+1}=\lambda_k+\frac{h(x_k)^\top}{\gamma_k} \tag{2.90}$$

而 x_k 则通过

$$x_k = \arg\min_{x\in\mathbb{R}^n} \ l_A(x, \lambda_k; \gamma_k). \tag{2.91}$$

若保持二次罚函数法中的设定，即 $\gamma_k \to 0^+$，并进一步假设 $\{\lambda_k\}$ 有界，则可以证明增广 Lagrange 方法 也能保证收敛，如下所示。

命题 2.4.9. *设 $(\bar{x}, \bar{\lambda})$ 是由 (2.90) 与 (2.91) 生成的序列 $\{(x_k, \lambda_k)\}$ 的任意一个聚点，且 $\gamma_k \to 0$。则 $\bar{x}$ 是 (2.88) 的一个全局极小值点。*

证明. 记 $f^* := \min\{f(x) : h(x) = 0,\ x \in \mathbb{R}^n\}$。则由每个 k 上的 (2.91)，对任意 x 有

$$l_A(x_k, \lambda_k; \gamma_k) \le l_A(x, \lambda_k; \gamma_k).$$

对右侧所有满足 $h(x) = 0$ 的 x 取下确界，得到

$$l_A(x_k, \lambda_k; \gamma_k) \le f^*. \tag{2.92}$$

设子序列 $\{(x_{k_j}, \lambda_{k_j})\}$ 当 $j \to \infty$ 时收敛到 $(\bar{x}, \bar{\lambda})$。则令 $j \to \infty$ 可得

$$f(\bar{x}) + \bar{\lambda} h(\bar{x}) + \lim_{j\to\infty} \frac{1}{2\gamma_{k_j}} |h(x_{k_j})|^2 \le f^*. \tag{2.93}$$

注意到 $f(\bar{x}) + \bar{\lambda} h(\bar{x})$ 是某个常数。由于 $\gamma_k \to 0$ 且 $|h(x_k)| \ge 0$，因此必有

$$|h(\bar{x})|^2 = \lim_{j\to\infty} |h(x_{k_j})|^2 = 0,$$

这说明 $h(\bar{x}) = 0$，从而 $\bar{x}$ 是可行的。此外，由 (2.93) 可知 $f(\bar{x}) \le f^*$。因此 $\bar{x}$ 是 (2.88) 的一个全局极小值点。 □

虽然命题 2.4.9 给出了收敛结果，但我们注意到 (2.91) 在实践中较难实现，因为 $l_A(\cdot, \lambda_k; \gamma_k)$ 可能是非凸的，而我们一般无法得到其全局极小值点 x_k。作为替代，我们可以对所有 k 近似求出 $l_A(\cdot, \lambda_k; \gamma_k)$ 的驻点。我们在算法 2.4.4 中总结了增广 Lagrange 方法。在下一个命题中，我们将证明：若算法 2.4.4 生成的 $\{x_k\}$ 收敛到某个正则点 x^*，则该 x^* 满足问题 (2.88) 的 Lagrange 条件。

命题 2.4.10. *设 $f, h \in C^1$，$\{\lambda_k\}$ 有界，且 $\gamma_k, \epsilon_k \to 0^+$。假设对所有 k，$\{x_k\}$ 满足 $|\nabla_x l_A(x_k, \lambda_k; \gamma_k)| \le \epsilon_k$，并且 $x_k \to x^*$，其中 x^* 是正则点（注意我们并未假设 x^* 是局部极小点）。则有 $h(x^*) = 0$，并且 $\tilde{\lambda}_k := \lambda_k + \frac{h(x_k)^\top}{\gamma_k} \to \lambda^*$，其中 (x^*, λ^*) 满足 Lagrange 条件 (2.83)。*

Algorithm 2.4.4 用于 (2.88) 的增广 Lagrange 方法

Input: Initial $\gamma_0 > 0$, $\epsilon_0 > 0$, $x_{0,0} \in \mathbb{R}^n$, $\lambda_0 \in \mathbb{R}^m$.

for $k = 0, 1, 2, \ldots$ **do**
  Start from $x_{k,0}$, find x_k such that $|\nabla_x l_A(x_k, \lambda_k; \gamma_k)| \le \epsilon_k$.
  if x_k is "satisfactory" **then**
    Terminate.
  end if
  Set $\lambda_{k+1} = \lambda_k + \frac{\nabla h(x_k)^\top}{\gamma_k}$.
  Choose $\gamma_{k+1} \in (0, \gamma_k)$.
  Choose $\epsilon_{k+1} \in (0, \epsilon_k)$.
  Choose an initial guess $x_{k+1,0}$ for the next iteration.
end for

Output: x_k.

证明. 由于 $x_k \to x^*$ 且 x^* 是正则点，即 $\nabla h(x^*)$ 具有满行秩，我们可设对所有 k 都有 $\nabla h(x_k)$ 具有满行秩，从而 $\nabla h(x_k)\nabla h(x_k)^\top$ 可逆。注意

$$\begin{aligned}\nabla_x l_A(x_k, \lambda_k; \gamma_k) &= \nabla f(x_k) + \lambda_k \nabla h(x_k) + \frac{1}{\gamma_k} h(x_k)^\top \nabla h(x_k) \\ &= \nabla f(x_k) + \tilde{\lambda}_k \nabla h(x_k), \end{aligned} \tag{2.94}$$

其中

$$\tilde{\lambda}_k := \lambda_k + \frac{h(x_k)^\top}{\gamma_k}.$$

将 (2.94) 两边右乘 $\nabla h(x_k)^\top (\nabla h(x_k)\nabla h(x_k)^\top)^{-1}$ 并整理，得到

$$\tilde{\lambda}_k = (\nabla_x l_A(x_k, \lambda_k; \gamma_k) - \nabla f(x_k))\nabla h(x_k)^\top (\nabla h(x_k)\nabla h(x_k)^\top)^{-1}.$$

由于 $x_k \to x^*$ 且当 $k \to \infty$ 时 $|\nabla_x l_A(x_k, \lambda_k; \gamma_k)| \to 0$，因此

$$\tilde{\lambda}_k \to \lambda^* := -\nabla f(x^*)\nabla h(x^*)^\top (\nabla h(x^*)\nabla h(x^*)^\top)^{-1}.$$

在 (2.94) 中令 $k \to \infty$，可得

$$\nabla f(x^*) + \lambda^* \nabla h(x^*) = 0.$$

由于 λ_k 有界，并且 $\lambda_k + \frac{h(x_k)^\top}{\gamma_k} = \tilde{\lambda}_k \to \lambda^*$，故序列 $\{\frac{h(x_k)^\top}{\gamma_k}\}$ 有界。由于 $\gamma_k \to 0^+$，因此有 $h(x_k) \to 0$。当 $x_k \to x^*$ 且 h 连续时，可知 $h(x^*) = 0$。 □

我们注意到，命题 2.4.10 仍然要求 $\gamma_k \to 0$。在实践中，通常有以下三种可能情况。

第一种情况是：对于某个 k，我们无法找到满足 $|\nabla_x l_A(x_k, \lambda_k; \gamma_k)| \le \epsilon_k$ 的 x_k。这通常发生在 $l_A(\cdot, \lambda_k; \gamma_k)$ 无下界的时候。

第二种情况是：对每个 k，$\{x_k\}$ 都满足 $|\nabla_x l_A(x_k, \lambda_k; \gamma_k)| \le \epsilon_k$，但 $\{x_k\}$ 不收敛，或者它收敛到一个不可行点 x^*。这通常发生在原始问题（等式约束）没有可行解时。在这种情况下，$\{x_k\}$ 收敛到一个不可行点 x^*，并且该点是 $|h(x)|^2$ 的一个驻点，即 $h(x^*)^\top \nabla h(x^*) = 0$。

第三种情况是：$\{x_k\}$ 存在并收敛到一个点 x^*，该点是局部极小点但不是正则点，即 $\nabla h(x^*)$ 不具有满行秩。在这种情况下，$\nabla h(x^*) \nabla h(x^*)^\top$ 不可逆，因此 $\lambda_k + \frac{h(x_k)}{\gamma_k}$ 可能不收敛。

我们指出，增广 Lagrange 方法有时未必高效。这通常是因为当 $\gamma_k \to 0$ 时，$\nabla^2_{xx} l_A(\cdot, \lambda_k; \gamma_k)$ 的条件数会变得非常大。

在不等式约束优化中的应用 我们可以将增广 Lagrange 方法用于同时带有等式与不等式约束的优化问题。为此，我们可以先把不等式约束转化为等式约束。这个过程可以通过引入**平方松弛变量**来完成，如下所示。

考虑带有等式和不等式约束的一般优化问题

$$\min_{x \in \mathbb{R}^n} \quad f(x), \tag{2.95a}$$

$$\text{s.t.} \quad h(x) = 0, \tag{2.95b}$$

$$g_j(x) \le 0, \quad j = 1, \ldots, m, \tag{2.95c}$$

通过引入平方松弛变量 $y = (y_1, \ldots, y_m)^\top \in \mathbb{R}^m$，我们可以将 (2.95) 改写成等价形式

$$\min_{x \in \mathbb{R}^n, y \in \mathbb{R}^m} \quad f(x), \tag{2.96a}$$

$$\text{s.t.} \quad h(x) = 0, \tag{2.96b}$$

$$g_j(x) + y_j^2 = 0, \quad j = 1, \ldots, m, \tag{2.96c}$$

问题 (2.96) 的增广 Lagrange 函数为

$$l_A(x, y, \lambda, \mu; \gamma) = f(x) + \lambda h(x) + \frac{1}{2\gamma}|h(x)|^2 + \mu \tilde{g}(x, y) + \frac{1}{2\gamma}|\tilde{g}(x, y)|^2, \tag{2.97}$$

其中

$$\tilde{g}(x,y) := \Big(\tilde{g}_1(x,y_1),\ldots,\tilde{g}_m(x,y_m)\Big)^\top \in \mathbb{R}^m$$

且

$$\tilde{g}_j(x,y_j) := g_j(x) + y_j^2, \quad \text{for } j = 1,\ldots,m.$$

当 λ、μ 和 γ 固定时，我们需要求解关于 (x,y) 的无约束子问题：

$$\min_{x,y}\ l_A(x,y,\lambda,\mu;\gamma). \tag{2.98}$$

若在 (2.98) 中固定 x，则 y 存在闭式解：首先，$y=(y_1,\ldots,y_m)$ 的各个分量彼此分离，每个 y_j 都求解

$$\min_{y_j}\ \mu_j(g_j(x)+y_j^2) + \frac{1}{2\gamma}(g_j(x)+y_j^2)^2,$$

这等价于

$$\min_{z_j\geq 0}\ \mu_j(g_j(x)+z_j) + \frac{1}{2\gamma}(g_j(x)+z_j)^2.$$

我们可以对上式中关于 z_j 的项配方，去掉与 z_j 无关的常数项，得到等价优化问题：

$$\min_{z_j\geq 0}\ \frac{1}{2\gamma}z_j^2 + \Big(\frac{1}{\gamma}\nabla g_j(x)^\top + \mu_j\Big)z_j,$$

其极小点可写成闭式形式：

$$z_j^* = \max(0, -(g_j(x)+\gamma\mu_j)).$$

注意，z_j^* 依赖于 x、μ_j 和 γ。记

$$g_j^+(x,\mu_j;\gamma) := g_j(x) + z_j^* = \max(g_j(x), -\gamma\mu_j).$$

将 $y_j^2 = z_j^*$ 代回增广 Lagrange 函数 (2.97)，得到一个不含 y 的简化增广 Lagrange 函数：

$$\begin{aligned} l_A(x,\lambda,\mu;\gamma) := {} & f(x) + \lambda h(x) + \frac{1}{2\gamma}|h(x)|^2 \\ & + \sum_{j=1}^m \Big(\mu_j g_j^+(x,\mu;\gamma) + \frac{1}{2\gamma}(g_j^+(x,\mu;\gamma))^2\Big). \end{aligned}$$

我们还可以通过在求和项中配方，进一步简化 $l_A(x,\lambda,\mu;\gamma)$：

$$\begin{aligned}\mu_j g_j^+(x,\mu;\gamma)+\frac{1}{2\gamma}(g_j^+(x,\mu;\gamma))^2 &= \frac{1}{2\gamma}\Big((g_j^+(x,\mu_j;\gamma)+\gamma\mu_j)^2-\gamma^2\mu_j^2\Big)\\ &= \frac{1}{2\gamma}\Big(\max(g_j(x)+\gamma\mu_j,0)^2-\gamma^2\mu_j^2\Big)\\ &= \frac{\gamma}{2}\Big(\max\Big(\frac{1}{\gamma}g_j(x)+\mu_j,0\Big)^2-\mu_j^2\Big).\end{aligned}$$

其余部分与增广 Lagrange 方法（算法 2.4.4）相同，只不过乘子变成了 (λ,μ)。不过我们注意到，$\max(\cdot,0)^2$ 并不是二次可微的。这在极小化 $l_A(\cdot,\lambda_k,\mu_k;\gamma_k)$ 来求 x_k 时可能会带来问题。

尽管如此，根据上面的分析，我们仍然期望当 $k\to\infty$ 时有

$$\lambda_k+\frac{1}{\gamma_k}h(x_k)^\top\to\lambda^*$$

以及

$$\mu_k+\frac{1}{\gamma_k}g^+(x_k,\mu_k;\gamma_k)=\max\Big(\mu_k+\frac{1}{\gamma_k}g(x_k),0\Big)\to\mu^*$$

其中最大值按分量取，且 (λ^*,μ^*) 是对应于 $h(x^*)=0$ 与 $g(x^*)\le 0$ 的 Lagrange 乘子。注意到 $\mu^*\ge 0$，这与 KKT 条件是一致的。

2.5 随机优化算法

深度学习通常需要大规模的数据样本，因此目标函数通常在一种特定的设置下构建，在这种设置中，随机优化算法相比传统的确定性方法更具适应性。在本节中，我们将介绍随机优化算法，包括其形式、动机、优缺点，以及当前网络训练中广泛使用的若干算法。特别地，我们关注其在求解非凸无约束优化问题中的应用，这类问题是神经网络训练中最常见的。

2.5.1 随机优化的基础

回顾标准的监督学习问题，我们通常给定一个数据集 $\mathcal{D}=\{(x_i,y_i)\in\mathbb{R}^d\times\mathbb{R}^m : i\in[N]\}$，其中包含 N 对样本，且 N 可以非常大，例如 10^6 或更大。我们的目标是找到映射 $u:\mathbb{R}^d\to\mathbb{R}^m$，并将其参数化为具有参数 $\theta\in\mathbb{R}^n$ 的深度神经网络 u_θ，使得对于所有 $i\in[N]$，$u_\theta(x_i)$ 近似等于 y_i。为简便起见，假设我们将关于 θ 的损失函数 $\ell:\mathbb{R}^n\to\mathbb{R}$ 设为 $u_\theta(x_i)$ 与 y_i 之间的平方差的

平均值，并通过求解

$$\min_{\theta\in\mathbb{R}^n}\ \ell(\theta) := \frac{1}{N}\sum_{i=1}^{N} f_i(\theta), \quad \text{其中} \quad f_i(\theta) = \frac{1}{2}|u_\theta(x_i) - y_i|^2. \tag{2.99}$$

由于 $\nabla_\theta f_i(\theta)$ 和 $\nabla_\theta^2 f_i(\theta)$ 可以通过自动微分（如第 2.3 节所述）计算，我们可以得到 $\ell(\theta)$ 的梯度和 Hessian。因此，从理论上讲，我们可以应用第 2.4.1 节中介绍的任意无约束优化算法，例如梯度下降法、Newton 法以及拟 Newton 方法，来寻找 ℓ 的（局部）极小值。然而，事实证明，对于形式如 (2.99) 的损失函数，采用确定性优化算法在实际中往往不可行，或者至少效率较低。下面我们对此进行分析。

为了与本章中使用的符号保持一致，我们仍用 x 表示待优化变量（而不是 θ），并用 F_N 表示损失函数以强调其为 N 项的平均值，将优化问题 (2.99) 写为

$$\min_{x\in\mathbb{R}^n}\ F_N(x) := \frac{1}{N}\sum_{i=1}^{N} f_i(x). \tag{2.100}$$

我们指出，F_N 也称为**有限和**目标函数，这是一类在深度学习问题中常见的目标函数。它是（无约束）随机优化问题一般形式的一个特例，后者通常写为

$$\min_{x\in\mathbb{R}^n}\ F(x) := \mathbb{E}_\xi[f(x;\xi)], \tag{2.101}$$

其中 ξ 是一个随机变量，其样本（实现）可以是从 $\mathcal{D}$ 中独立抽取的单个或一部分样本，而 $\mathcal{D}$ 包含 N 个样本，这些样本来自某个（可能未知的）数据分布。

例如，如果我们有一个数据集 $\mathcal{D} = \{z_i : i \in [N]\}$，称为一个**批次**（batch），其中每个样本 z_i 对应一个关于 x 的函数 f_i，并且 ξ 被设为从 $\mathcal{D}$ 中按均匀分布抽取的一个样本，则 (2.101) 可简化为 (2.100)。也可以将 (2.100) 视为通过 Monte Carlo 积分（见附录 A.1）对 (2.101) 的近似，其中使用的数据集为 $\mathcal{D}$。损失函数 F 和 F_N 分别称为**期望损失**（expected loss）和**经验损失**（empirical loss）。在本节的其余部分，我们主要关注最小化 (2.100) 中的 F_N，并在需要时讨论其与最小化 (2.101) 中 F 的联系。

如果将 (2.100) 视为一个确定性优化问题来求解，则标准梯度下降法在第 k 次迭代中的更新为

$$x_{k+1} = x_k - \frac{\alpha_k}{N}\sum_{i=1}^{N}\nabla f_i(x_k), \tag{2.102}$$

其中 $\alpha_k > 0$ 为步长。我们称 (2.102) 为**批量梯度**（batch gradient method）（或**全梯度**）方法。这实际上是将梯度下降法直接应用于 (2.100)。

为了理解随机优化，我们考虑求解 (2.100) 的基本随机更新形式，在第 k 次迭代中为

$$x_{k+1} = x_k - \alpha_k \nabla f_{i_k}(x), \tag{2.103}$$

其中 i_k 是从 $\{1, \ldots, N\}$ 中随机抽取的，并且对所有 $j < k$，i_k 与 i_j 在统计上相互独立。我们称 (2.103) 为**基本随机梯度**（basic stochastic gradient）方法。需要注意的是，(2.103) 中的 x_k 不仅依赖于 F_N、初始值 x_0 和步长 α_k，还依赖于随机序列 $\{i_j : j < k\}$。实际上，$\{x_k : k \geq 1\}$ 可以看作一个随机过程。

通过比较批量梯度方法 (2.102) 与基本随机梯度方法 (2.103)，我们可能会觉得前者这种确定性优化方法更具吸引力。这是可以理解的，因为存在许多成熟的确定性优化算法，并且我们通常期望通过这些算法获得良好的收敛速度和解的精度。此外，(2.100) 中 F_N 的结构还允许我们利用并行计算技术，从而显著降低每次迭代的计算时间。

然而，从多个方面来看，随机梯度方法 (2.103) 在深度学习应用中已被证明更加高效 [20]。例如，考虑一个网络训练问题，其中给定的数据集 $\mathcal{D}$ 构造不佳：$\mathcal{D}$ 由某个较小数据集 $\mathcal{D}_{\text{sub}}$ 的十个完全相同的副本组成。（尽管我们通常会避免这种情况，但在实际中采集的数据往往具有相似性和重复性。这里仅用于提供一个直观动机，说明批量梯度方法可能不如基本随机梯度方法高效。）在这种情况下，批量梯度方法仅使用 $\mathcal{D}_{\text{sub}}$ 也会产生相同的迭代序列，但由于实际使用的是 $\mathcal{D}$，其计算成本却增加了 10 倍。相比之下，基本随机梯度方法的计算成本和收敛行为保持不变。这表明，与 (2.102) 中使用全部样本相比，基本随机梯度方法 (2.103) 可能更加高效。

实际上，已有大量文献对基本随机梯度方法与先进的确定性优化方法进行了比较。例如，一些研究结果表明，适当调节常数步长的基本随机梯度方法在性能上显著优于 limited-memory BFGS（L-BFGS）方法——这是 BFGS 方法（第 2.4.1 节）的一种高效且广泛使用的变体。更具体地，在一个使用逻辑损失函数和 RCV1 数据集的二分类问题中，基本随机梯度方法在前几个 epoch 内显著快于 L-BFGS 并达到较好的结果，而 L-BFGS 则在较后期才追赶并超过基本随机梯度方法 [20]。

这一结果看起来令人惊讶且反直觉。然而，[13] 中的如下示例对此现象提供了一个有力的解释。

例 2.5.1. 设 N 为奇整数，并令 $M = \frac{N-1}{2}$。我们将区间 $[-1, 1]$ 划分为 $2M$

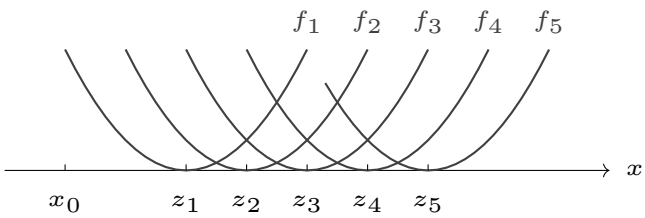


图 2.6: 当 $N=5$ 且 $M=\frac{N-1}{2}=2$ 时，(2.104) 中的 z_i 与 (2.105) 中的 f_i 的示意图。这里 $z_1,\ldots,z_N$ 是将区间 $[-1,1]$ 等分为 $2M=4$ 段后的网格点，其中 $z_1=-1$ 且 $z_N=1$。函数 $f_i(x)=(x-z_i)^2$ 是图中所示的二次函数，而其平均函数 $F_N(x):=\frac{1}{N}\sum_{i=1}^N f_i(x)=x^2+\frac{1}{N}\sum_{i=1}^N z_i^2$（图中未画出）在 $x=0$ 处有唯一极小值（在此图中也对应点 z_3）。我们可以考察在初值为 x_0 时，应用批量梯度下降法 (2.102) 和基本随机梯度法 (2.103) 的收敛行为。基本随机梯度下降法在初期以比批量梯度下降法更低的计算代价收敛到极小点 $x=0$，但当 x_k 接近 $x=0$ 时，它可能会 “迷失方向”，因为每次迭代它只使用一个随机选取的 f_i 的梯度。

个等长小段，每段长度为 $\frac{1}{M}=\frac{2}{N-1}$。记该划分的 $N=2M+1$ 个网格点为

$$z_i=-1+\frac{2(i-1)}{N-1},\quad i=1,\ldots,N. \tag{2.104}$$

注意到 $z_1=-1$ 且 $z_N=1$。现在定义

$$f_i(x):=(x-z_i)^2 \tag{2.105}$$

对所有 $i=1,\ldots,N$，并定义

$$F_N(x):=\frac{1}{N}\sum_{i=1}^N f_i(x)=\frac{1}{N}\sum_{i=1}^N (x-z_i)^2. \tag{2.106}$$

图 2.6 给出了当 $N=5$ 且 $M=2$ 时 z_i 和 f_i 的示意图。

注意到 $f_i(x)=(x-z_i)^2$ 是一个在 z_i 处取得最小值的二次函数。与此同时，平均函数 F_N 也是一个简单的二次函数：

$$F_N(x)=x^2+\underbrace{\frac{1}{N}\sum_{i=1}^N z_i^2}_{\text{常数}}, \tag{2.107}$$

其在 $x=0$ 处取得唯一的最小值。

如果我们从一个距离极小点 0 很远的初值 x_0 出发（见图 2.6），对 (2.106) 应用批量梯度下降法 (2.102) 或者更先进的确定性优化算法，如 L-BFGS，那么它们实际上是在最小化 (2.107)，并能够生成一个稳定收敛到极小点 $x=0$ 的迭代序列 $\{x_k\}$。相比之下，如果我们对同样的初值 x_0 使用基本随机梯度法 (2.103)，则可以预期生成的迭代序列 $\{x_k\}$ 在前几个 epoch 中仍然会快速收敛到 0，但每次迭代的计算代价要低得多：批量梯度下降法需要计算所有 f_i 的梯度，而基本随机梯度下降法在每次迭代中只需计算一个随机选取的 f_i 的梯度，因此其代价仅为前者的 $1/N$。

该例子还表明，如果使用常数步长，那么基本随机梯度法可能只能保证收敛到极小点附近的某个邻域。换言之，为了确保收敛到极小点，可能必须采用递减步长策略。

从理论角度看，我们还可以对最小化 F_N 时，批量梯度下降法和基本随机梯度法所需总计算成本的上界进行估计。注意到 F_N 是一个强凸且光滑的函数，批量梯度下降法实际上就是标准梯度下降法，并且已知具有线性收敛速度。更具体地，可以证明批量梯度法以线性速度收敛：

$$F_N(x_k) - F_N^* \le O(\gamma^k),$$

其中 $F_N^* > \infty$ 是 F_N 在 $\mathbb{R}$ 上的（可能未知的）最小值，而 $\gamma \in (0,1)$ 依赖于比值 $L/\mu \ge 1$。这里 L 是 ∇F_N 的 Lipschitz 常数，而 μ 是 F_N 的强凸参数，即

$$F_N(\hat{x}) \ge F_N(x) + \nabla F_N(x)(\hat{x}-x) + \frac{\mu}{2}|\hat{x}-x|^2$$

对所有 $x, \hat{x} \in \mathbb{R}^n$ 都成立。我们知道，当 L/μ 更大时，F_N 的病态性更严重，而 γ 也更接近于 1。（在本例中，由 (2.107) 定义的 F_N 满足 $L=\mu=1$。不过我们指出，即使在实际应用中 F_N 是强凸的，比值 L/μ 仍然可能非常大。）

由于上述线性收敛速度，可以很容易推出：为了得到 F_N 的一个 ϵ-最优解 x_k，即

$$F_N(x_k) - F_N^* \le \epsilon$$

对任意用户选定的误差容忍度 $\epsilon > 0$，所需迭代次数 k 的量级为 $O(\log(1/\epsilon))$。由于批量梯度下降法每次迭代的计算代价为 N（因为它需要计算 $f_1, \ldots, f_N$ 的梯度以得到 F_N 的梯度），因此获得一个 ϵ-最优解所需的总计算代价与 $N\log(1/\epsilon)$ 成正比。

另一方面，我们也可以考察基本随机梯度下降法 (2.103) 在最小化 (2.106) 所定义的 F_N 时的计算代价。由于 F_N 是强凸的，可以证明基本随机梯度法以

次线性（sublinear）速度收敛：

$$\mathbb{E}[F_N(x_k)] - F_N^* = O(1/k),$$

其中 x_k 是由基本随机梯度下降法生成的第 k 个迭代点。上述收敛速度仅为次线性的，因此从**渐近**（asymptotically）意义上看，它比线性收敛慢得多。然而，上述次线性收敛速度并不依赖于样本数 N。因此，为达到一个 ϵ-最优解所需的迭代次数为 $O(1/\epsilon)$。就每次迭代的计算代价而言，它为 1，因为基本随机梯度下降法每次只需计算 $f_1, \ldots, f_N$ 中一个随机选取函数的梯度。总而言之，其总计算代价与 $1/\epsilon$ 成正比，并且同样不依赖于 N。

现在我们看到，批量梯度下降法 (2.102) 和基本随机梯度下降法 (2.103) 所需的计算代价分别与 $N\log(1/\epsilon)$ 和 $1/\epsilon$ 成正比。当 N 固定时，随着 $\epsilon \to 0$，我们知道从渐近意义上看，$N\log(1/\epsilon)$ 远小于 $1/\epsilon$。然而，实际情况是，$\epsilon > 0$ 往往会事先被设定为一个足够小的数，而样本数 N 在收敛速度中起着重要作用，特别是在优化过程的早期阶段。这就解释了为什么基本随机梯度下降法在前几个 epoch 中的收敛速度甚至可能快于 L-BFGS。

我们还要指出，随机优化算法（例如基本随机梯度下降法 (2.103)）的另一个优点在于：即使是在最小化如 (2.101) 所示的期望损失函数 F 时，它们仍然适用。相比之下，确定性优化算法（例如批量梯度算法 2.102）则变得不可行，因为在实践中我们无法精确计算 F 的梯度。这些优点正是随机优化算法如今在深度学习研究与应用中变得如此普遍的原因。

接下来，我们考虑随机梯度方法的收敛性质和计算复杂度。比较批量梯度方法 (2.102) 与基本随机梯度方法 (2.103)，可以看出它们每次迭代的计算代价分别是 N 和 1，因为 (2.102) 计算了 N 个梯度，而 (2.103) 只计算了一个。按照深度学习中的惯例，当一个方法在期望意义下或等价意义下用完给定数据集 $\mathcal{D}$ 中的全部 N 个样本时，我们称它完成了一个 epoch。因此，批量梯度方法 (2.102) 每次迭代完成一个 epoch，而基本随机梯度方法 (2.103) 则每经过 N 次迭代完成一个 epoch。我们还将看到所谓的小批量随机梯度方法，它在第 k 次迭代中使用 $\mathcal{D}$ 中的 N_k 个样本（称为一个 mini-batch），其中 N_k 介于 1 和 N 之间（为简便起见，假设 N_k 不随 k 改变），因此它每经过 N/N_k 次迭代完成一个 epoch。

随机梯度方法的收敛性

现在我们来研究随机梯度方法的收敛行为。正如前面指出的那样，随机梯度方法（例如 (2.103)）既可以用于寻找经验损失 (2.100) 的极小值，也可以用于寻找期望损失 (2.101) 的极小值。对于后者，我们用 p_ξ 表示 ξ 的概率分布。因此，在本节的其余部分中，我们用 $F:\mathbb{R}^n \to \mathbb{R}$ 表示损失函数，它可以是上述两种情形中的任意一种。随机梯度（stochastic gradient，SG）方法的一般形式见算法 2.5.1。下面我们将更详细地说明该算法中的各个步骤。

Algorithm 2.5.1 用于求解 (2.100) 或 (2.101) 的随机梯度方法

Input: 初始猜测 $x_1 \in \mathbb{R}^n$。

1: **for** $k = 1, 2, \ldots$ **do**
2: 生成 ξ_k 的一个实现值。
3: 计算 $g(x_k, \xi_k)$。
4: 选择步长 $\alpha_k > 0$。
5: 令 $x_{k+1} \leftarrow x_k - \alpha_k g(x_k, \xi_k)$。
6: **end for**

我们首先解释算法 2.5.1 中第 2 行和第 3 行是如何实现的。具体来说，我们考虑 SG 方法在期望损失情形和经验损失情形下的三种实现方式：基本 SG、小批量 SG，以及带有二阶信息的小批量 SG（为简洁起见，后者下文称为二阶小批量 SG）。我们指出，在这三种情形中，$\{\xi_k : k \geq 1\}$ 都是一列联合独立的随机变量。一旦确定了 ξ_k，向量 $g(x_k, \xi_k)$ 就根据所指定的 SG 方法定义为：

$$g(x_k, \xi_k) = \begin{cases} \nabla_x f(x_k; \xi_k) & \text{(基本 SG)} \\ \dfrac{1}{N_k} \displaystyle\sum_{i=1}^{N_k} \nabla_x f(x_k; \xi_{k,i}) & \text{(小批量 SG)} \\ H_k \dfrac{1}{N_k} \displaystyle\sum_{i=1}^{N_k} \nabla_x f(x_k; \xi_{k,i}) & \text{(二阶小批量 SG)} \end{cases} \tag{2.108}$$

在基本 SG 情形下，我们已经在有限和情形 (2.100) 中给出了更新公式 (2.103)。在这种情况下，ξ_k 指的是从 $\{1, \ldots, N\}$ 中按均匀分布抽取的一个样本索引。如果抽取到的索引为 $i_k \in \{1, \ldots, N\}$，则在算法 2.5.1 的第 2 行中有 $\xi_k = i_k$，而在第 3 行中有 $g(x_k, \xi_k) = \nabla f_{i_k}(x_k)$。如果将基本 SG 应用于 (2.101) 中定义的期望损失 F，那么 ξ_k 是从 p_ξ 中抽取的一个样本，并且我们令 $g(x_k, \xi_k) = \nabla_x f(x_k, \xi_k)$。

在小批量 SG 情形下，对于经验损失，ξ_k 指的是从 $\{1, \ldots, N\}$ 中均匀抽取的 N_k 个索引（其中 N_k 称为小批量大小，它由用户预先选定），而 $g(x_k, \xi_k)$

是在 ξ_k 所选索引处对 f_i 的梯度取平均。其显式形式见 (2.108)。类似地，在期望损失情形中，ξ_k 是从 p_ξ 中独立抽取的 N_k 个样本组成的集合，而 $g(x_k,\xi_k)$ 则是在 ξ_k 所指定样本上梯度的平均值。

二阶小批量 SG 情形与前一种情况类似，只是额外引入了一个对称正定的缩放矩阵 H_k。这个矩阵是基于不同的二阶信息引入策略计算出来的。选择 H_k 的一个一般性原则是：在对 ξ_k 取期望的意义下，$-g(x_k,\xi_k)$ 应当是一个下降方向。关于 H_k 的构造，我们将在后面给出更多细节。

现在我们列出建立算法 2.5.1 中随机梯度下降方法收敛性所需的一些假设。

假设 2.5.2. 设损失函数 $F:\mathbb{R}^n\to\mathbb{R}$ 连续可微，并且满足以下假设。

1. $F^* := \inf_{x\in\mathbb{R}^n} F(x)$ 是有限的。

2. 梯度 ∇F 是 L-Lipschitz 连续的，即

$$|\nabla F(\hat{x})-\nabla F(x)|\le L|\hat{x}-x|,\quad \forall\, x,\hat{x}\in\mathbb{R}^n.$$

3. 存在标量 μ 和 μ_G，满足 $0<\mu\le\mu_G$，使得对所有 k 都有

$$\nabla F(x_k)\cdot\mathbb{E}_{\xi_k}[g(x_k,\xi_k)]\ge\mu|\nabla F(x_k)|^2, \tag{2.109}$$

$$|\mathbb{E}_{\xi_k}[g(x_k,\xi_k)]|\le\mu_G|\nabla F(x_k)|. \tag{2.110}$$

4. 存在标量 $M,M_G\ge 0$，使得对所有 k 都有

$$\mathbb{E}_{\xi_k}[|g(x_k,\xi_k)|^2]\le M+M_G|\nabla F(x_k)|^2. \tag{2.111}$$

假设 2.5.2 中前两个条件较为常见，并且在实际应用中通常都成立。条件 (2.109) 要求在对 ξ_k 取期望的意义下，$g(x_k,\xi_k)$ 与 $\nabla F(x_k)$ 之间的夹角应小于 $\pi/2$。这是要求 $-g(x_k,\xi_k)$ 在期望意义下成为下降方向的一个典型条件。条件 (2.110) 实质上要求 $g(x_k,\xi_k)$ 的期望（即一阶矩，first moment）对每个 k 都是 $O(|\nabla F(x_k)|)$ 量级。结合 (2.109) 可知，在对 ξ_k 取期望时，$|g(x_k,\xi_k)|$ 应与 $|\nabla F(x_k)|$ 成比例。条件 (2.111) 要求 $g(x_k,\xi_k)$ 的二阶矩（second moment）对每个 k 都能被某个常数 $M>0$ 与 $O(|\nabla F(x_k)|^2)$ 之和所控制。

注意，如果 $g(x_k,\xi_k)$ 是 $F(x_k)$ 的无偏估计，即

$$\mathbb{E}_{\xi_k}[g(x_k,\xi_k)]=\nabla F(x_k),$$

那么条件 (2.109) 和 (2.110) 都以 $\mu = \mu_G = 1$ 成立。在许多 SG 方法的标准设定中，例如基本 SG 方法，可以证明 $g(x_k, \xi_k)$ 确实是 $\nabla F(x_k)$ 的无偏估计。

在假设 2.5.2 下，我们可以得到第 k 次迭代中 $\mathbb{E}_{\xi_k}[F(x_{k+1})]$ 与 $F(x_k)$ 之差的一个简单上界，如下引理所示。

引理 2.5.3. *设假设 2.5.2 中的条件成立。令 x_k 为从任意初始点 x_0 出发，由算法 2.5.1 中的随机梯度方法生成的迭代序列。则有*

$$\mathbb{E}_{\xi_k}[F(x_{k+1})] - F(x_k) \le -\Big(\mu - \frac{1}{2}\alpha_k L M_G\Big)\alpha_k |\nabla F(x_k)|^2 + \frac{1}{2}\alpha_k^2 L M. \quad (2.112)$$

证明. 由于 ∇F 是 L-Lipschitz 连续的，我们有

$$\begin{aligned} F(x_{k+1}) - F(x_k) &\le \nabla F(x_k) \cdot (x_{k+1} - x_k) + \frac{L}{2}|x_{k+1} - x_k|^2 \\ &= -\alpha_k \nabla F(x_k) \cdot g(x_k, \xi_k) + \frac{\alpha_k^2 L}{2}|g(x_k, \xi_k)|^2. \end{aligned}$$

对两边关于 $\xi_k \sim p_\xi$ 取期望，并注意到 x_k 与 ξ_k 相互独立（因为 ξ_k 是在 x_k 计算出来之后独立生成的），可得

$$\mathbb{E}_{\xi_k}[F(x_{k+1})] - F(x_k) \le -\alpha_k \nabla F(x_k) \cdot \mathbb{E}_{\xi_k}[g(x_k, \xi_k)] + \frac{\alpha_k^2 L}{2}\mathbb{E}_{\xi_k}[|g(x_k, \xi_k)|^2].$$

将条件 (2.109) 和 (2.111) 分别应用于上式右端的第一项和第二项，得到

$$\begin{aligned} \mathbb{E}_{\xi_k}[F(x_{k+1})] - F(x_k) &\le -\alpha_k \mu |\nabla F(x_k)|^2 + \frac{\alpha_k^2 L}{2}\Big(M + M_G[|\nabla F(x_k)|^2]\Big) \\ &= -\Big(\mu - \frac{1}{2}\alpha_k L M_G\Big)\alpha_k |\nabla F(x_k)|^2 + \frac{1}{2}\alpha_k^2 L M, \end{aligned}$$

从而完成证明。 □

现在我们已准备好证明算法 2.5.1 中随机梯度方法的收敛性。我们在两种步长 α_k 的设定下考察该方法的收敛性。第一种设定是在所有迭代中使用常数步长，第二种设定是使用预先给定的自动递减步长。

定理 2.5.4 (具有常数步长的 SG)**.** *设假设 2.5.2 中的条件成立。令 x_k 为由算法 2.5.1 中的随机梯度方法从任意初值 x_0 出发生成的迭代序列，并且对所有 k 取常数步长 $\alpha_k = \alpha$，其中*

$$0 < \alpha \le \frac{\mu}{L M_G}. \quad (2.113)$$

则有

$$\mathbb{E}\Big[\sum_{k=1}^{K}|\nabla F(x_k)]|^2\Big] \le \frac{K\alpha LM}{\mu} + \frac{2(F(x_1)-F^*)}{\mu\alpha}. \tag{2.114}$$

因此，

$$\mathbb{E}\Big[\frac{1}{K}\sum_{k=1}^{K}|\nabla F(x_k)]|^2\Big] \le \frac{\alpha LM}{\mu} + \frac{2(F(x_1)-F^*)}{\mu\alpha K}. \tag{2.115}$$

当 $K\to\infty$ 时，上式收敛到正标量 $\alpha LM/\mu$。

证明. 对 (2.112) 关于 $\xi_1,\ldots,\xi_k$ 取期望（注意对每个 k，x_k 由 $\xi_1,\ldots,\xi_{k-1}$ 决定），并利用 (2.113) 中对 α 的约束，可得

$$\begin{aligned}\mathbb{E}[F(x_{k+1})]-\mathbb{E}[F(x_k)] &\le -\Big(\mu-\frac{1}{2}\alpha LM_G\Big)\alpha|\nabla F(x_k)|^2+\frac{1}{2}\alpha^2LM\\ &\le -\frac{1}{2}\mu\alpha|\nabla F(x_k)|^2+\frac{1}{2}\alpha^2LM.\end{aligned}$$

对两边从 $k=1,2,\ldots,K$ 求和，得到

$$F^*-F(x_1)\le\mathbb{E}[F(x_{k+1})]-F(x_1)\le-\frac{1}{2}\mu\alpha\sum_{k=1}^{K}|\nabla F(x_k)|^2+\frac{1}{2}\alpha^2LM,$$

其中第一个不等式来自于对所有 k 都有 $F(x_{k+1})\ge F^*$。对该不等式整理，即得 (2.114)。再将 (2.114) 两边除以 K，即可得到 (2.115)。 □

我们可以看到，(2.115) 中得到的上界并不好，因为除非 $M=0$，否则当 $K\to\infty$ 时它并不会趋于 0。然而，由 (2.111) 可知，$M=0$ 意味着二阶矩 $\mathbb{E}_{\xi_k}[|g(x_k,\xi_k)|^2]$ 必须以与 $|\nabla F(x_k)|^2$ 一样快的速度衰减。即使 $g(x_k,\xi_k)$ 是 $\nabla F(x_k)$ 的无偏估计，这仍然很难做到，因为这实际上要求 $g(x_k,\xi_k)$ 中的噪声（方差）趋于 0，而这在现实应用中很少能够实现。

尽管如此，我们仍然可以从估计式 (2.115) 中得到一些启发。其收敛到一个依赖于 M 和步长 α 的正数，这解释了我们在上面的例 2.5.1 中看到的现象：基本 SG 方法在初期可以很高效，但最终可能只是在极小点 $x=0$ 的某个邻域中徘徊，而不是收敛到 0 本身。通过使用更小的步长 α，这个邻域的大小可以减小，但这显然会拖慢整体收敛速度，因此未必值得。

接下来，我们考虑算法 2.5.1 中 SG 方法在递减步长下的收敛性。我们采

用经典文献 [119] 中刻画的步长规则：

$$\sum_{k=1}^{\infty}\alpha_k=\infty \qquad 和 \qquad \sum_{k=1}^{\infty}\alpha_k^2<\infty. \tag{2.116}$$

于是我们得到如下结果。

定理 2.5.5 (具有递减步长的 SG)**.** *设假设 2.5.2 中的条件成立。令 x_k 为由算法 2.5.1 中的随机梯度方法从任意初值 x_0 出发生成的迭代序列，其步长 α_k 满足规则 (2.116)。则有*

$$\lim_{K\to\infty}\mathbb{E}\Big[\sum_{k=1}^{K}\alpha_k|\nabla F(x_k)]|^2\Big]<\infty, \tag{2.117}$$

从而

$$\lim_{K\to\infty}\mathbb{E}\Big[\frac{1}{A_K}\sum_{k=1}^{K}\alpha_k|\nabla F(x_k)]|^2\Big]=0, \tag{2.118}$$

其中 $A_K:=\sum_{k=1}^{K}\alpha_k$。

证明. 注意到 (2.116) 中的条件 $\sum_{k=1}^{\infty}\alpha_k^2<\infty$ 蕴含 $\alpha_k\to 0$ 当 $k\to\infty$。因此，不妨设对所有 k 都有 $\alpha_k\le\frac{\mu}{LM_G}$。于是，对 (2.112) 关于 $\xi_1,\ldots,\xi_k$ 取期望，得到

$$\begin{aligned}\mathbb{E}[F(x_{k+1})]-\mathbb{E}[F(x_k)]&\le-\Big(\mu-\frac{1}{2}\alpha_k LM_G\Big)\alpha_k|\nabla F(x_k)|^2+\frac{1}{2}\alpha_k^2 LM\\&\le-\frac{1}{2}\mu\alpha_k|\nabla F(x_k)|^2+\frac{1}{2}\alpha_k^2 LM.\end{aligned}$$

对两边从 $k=1,2,\ldots,K$ 求和，得到

$$F^*-F(x_1)\le\mathbb{E}[F(x_{k+1})]-F(x_1)\le-\frac{1}{2}\mu\sum_{k=1}^{K}\alpha_k|\nabla F(x_k)|^2+\frac{1}{2}LM\sum_{k=1}^{K}\alpha_k^2.$$

整理该不等式，可得

$$\sum_{k=1}^{K}\alpha_k|\nabla F(x_k)|^2\le\frac{2(F(x_1)-F^*)}{\mu}+\frac{LM}{\mu}\sum_{k=1}^{K}\alpha_k^2. \tag{2.119}$$

再次利用 (2.116) 中 $\sum_{k=1}^{\infty}\alpha_k^2<\infty$，我们知道右端随着 $K\to\infty$ 单调增加且有有限上界，这就推出了 (2.117)。

Algorithm 2.5.2 子采样无 Hessian 不精确 Newton 法

Input: 初始猜测 $x_1 \in \mathbb{R}^n$，常数 $\rho, \gamma, \eta \in (0,1)$，$K_{\mathrm{CG}} \in \mathbb{N}$。

1: **for** $k = 1, 2, \ldots$ **do**

2: 生成 ξ_k 和 ξ_k^H 的实现值。

3: 计算 $g(x_k, \xi_k)$。

4: 通过无 Hessian 的 CG 求解

$$H(x_k; \xi_k^H)s = -g(x_k; \xi_k) \tag{2.120}$$

来计算 s_k；如果达到最大迭代次数 K_{CG}，或者满足

$$|H(x_k; \xi_k^H)s + g(x_k, \xi_k)| \le \rho |g(x_k; \xi_k)| \tag{2.121}$$

则终止 CG。

5: 通过回溯线搜索并使用衰减率 γ 选择步长 $\alpha_k > 0$，使得

$$f(x_{k+1}; \xi_k) \le f(x_k; \xi_k) + \eta\, \alpha_k\, g(x_k; \xi_k) \cdot s_k \tag{2.122}$$

其中 $x_{k+1} = x_k + \alpha_k s_k$。

6: **end for**

进一步地，将 (2.119) 两边除以 A_K，并注意到 (2.116) 中条件 $A_K = \sum_{k=1}^K \alpha_k \to \infty$ 当 $K \to \infty$，即可得到 (2.118)。 □

基于定理 2.5.5，我们很容易作出如下判断：

$$\liminf_{k\to\infty} \mathbb{E}[|\nabla F(x_k)|^2] = 0.$$

换言之，序列 $\{\mathbb{E}[|\nabla F(x_k)|^2] : k \ge 1\}$ 必然存在一个收敛到 0 的子序列。

二阶随机优化方法

正如 [20] 所指出的，二阶信息（也称曲率信息）对于消除 SG 方法中的某些关键问题（例如病态性）并显著提升其性能是非常有帮助的。这受到二阶方法在求解广泛的确定性优化问题中取得巨大成功的启发，例如拟 Newton 方法这一类方法。我们再次强调，使用二阶信息并不意味着必须计算或存储 F 的 Hessian 矩阵；对于深度学习问题而言，当维数 n 达到数百万甚至数十亿时，这通常是不可行的，或者至少在计算上代价极其高昂。

我们用不精确 Newton 法的一个变体来说明二阶信息在随机优化算法中的使用。具体地，我们考虑子采样的 Hessian-free inexact Newton 法作为一个例子。该方法总结于算法 2.5.2。

为了实现该方法，我们注意到，除了如 (2.108) 中用于计算 $g(x_k, \xi_k)$ 的大小为 N_k 的 ξ_k 之外，还需要再采样一个大小为 N_k^H 的实现 ξ_k^H，从而得到一个随机 Hessian 估计

$$H(x_k; \xi_k^H) := \frac{1}{N_k^H} \sum_{i=1}^{N_k^H} \nabla^2 f(x_k; \xi_{k,i}^H). \tag{2.123}$$

研究发现，该方法对估计 $H(x_k; \xi_k^H)$ 中的噪声似乎不像对 $g(x_k; \xi_k)$ 中的噪声那样敏感。因此，通常将 N_k^H 取得比 N_k 更小，以降低计算代价。注意，这里并不会显式地计算和存储 $H(x_k; \xi_k^H)$。相反，该方法使用共轭梯度（CG）算法（算法 2.4.2）来通过线性系统 (2.120) 求解 $s_k \in \mathbb{R}^n$，而这只需要若干次矩阵-向量乘法（关于如何利用自动微分计算这类乘法的细节，见第 2.3 节）。这正是该方法被称为 Hessian-free 的原因。

虽然在 $H(x_k; \xi_k^H)$ 正定的前提下，CG 算法被证明能够在有限步内精确求解 (2.120)，但实际中通常会提前终止 CG：要么当达到预设的最大迭代次数 K_{CG}，要么当 (2.120) 两边之差足够小，即满足 (2.121) 时。由 CG 算法为 (2.120) 得到的解 s_k 起到下降方向的作用，而随后在固定 ξ_k 的情况下，采用标准的回溯线搜索策略来确定步长 α_k，并进而得到下一个迭代点 x_{k+1}。

2.5.2 实用随机优化算法

尽管上述介绍的标准随机梯度下降方法仍然是最基本的方法，但它们的局限性促使人们发展出自适应优化方法（adaptive optimization methods）。在本小节中，我们介绍若干当前深度学习中广泛使用的自适应优化器，并对其动机与性质作一些说明。然而，在写作本文时，这些方法在仅假设温和正则性条件下求解具有光滑非凸目标函数的随机优化问题时，尚未拥有完整的收敛性保证。对其收敛行为的研究目前仍是一个活跃的研究方向。

尽管如此，这些方法在经验上看起来比标准的基本/小批量 SG 方法更快、更高效（也更稳定）。因此，它们往往是深度学习研究和工业应用中训练网络的首选方法。与此同时，未来极有可能还会不断发明出更高效、更稳定的随机优化算法。

动量方法

为简化记号，我们用 g_k 表示诸如 (2.108) 中的随机下降方向：

$$g_k := g(x_k; \xi_k).$$

加速收敛的两个关键技术之一是使用动量（momentum）。动量的思想是引入过去梯度的加权滑动平均来改进 g_k。这有助于平滑优化轨迹，从而通过减小振荡使算法更快收敛。带有**动量**的基本随机梯度下降方法写为

$$m_{k+1} = \beta_1 m_k + (1-\beta_1) g_k, \tag{2.124a}$$

$$x_{k+1} = x_k - \alpha_k m_{k+1}, \tag{2.124b}$$

其中 $m_k \in \mathbb{R}^n$ 是第 k 次迭代中的动量，α_k 是步长（学习率）。动量 m_1 初始化为 0，权重参数 β_1 在实践中通常取为 0.9。

尽管 (2.124) 能带来速度提升，步长 α_k 仍然依赖于待最小化目标函数梯度的 Lipschitz 常数，并且为了收敛还需要满足递减策略。这给实践者带来了额外的调参负担，以便为这些步长找到最优表现。下面将介绍的**自适应方法**具有抵消 Lipschitz 常数影响的特征，并且常常可以在固定步长下正常工作。这些自适应方法实际上是在使用对角缩放矩阵 H_k，参见 (2.108)，以融入目标函数的二阶信息。

自适应梯度（AdaGrad）

Adaptive gradient（AdaGrad）是最早为大规模和稀疏优化问题提出的自适应方法之一 [33]。它在每次迭代中引入一个向量形式的原始二阶矩估计 $v_k \in \mathbb{R}^n$。然后利用 v_k 的各个分量分别对 g_k 中对应的分量进行缩放。AdaGrad 的形式为

$$v_{k+1} = v_k + g_k \odot g_k, \tag{2.125a}$$

$$x_{k+1} = x_k - \alpha_k \frac{g_k}{\sqrt{v_{k+1}} + \epsilon}, \tag{2.125b}$$

其中 $\odot$ 表示两个向量的分量乘积，结果仍为同维向量，参数 $\epsilon > 0$ 通常取为 10^{-8}，以保证分母远离 0。(2.125b) 中的平方根和除法也都是逐分量进行的。因此，在 (2.125b) 中，g_k 的各个分量是分别被缩放的，而该比值对于目标函数乘以任意正的常数缩放是不变的（这种缩放会改变目标函数的 Lipschitz 常

数)。因此，步长 α_k 不受这种缩放影响，从而在迭代过程中可以保持固定。

AdaGrad 在具有稀疏特征的情形中尤其有效，例如自然语言处理，因为不常出现的参数会获得相对更大的更新。然而，它的主要缺点是分母单调增加，这使得 (2.125b) 中的比值迅速趋于 0。这往往会导致过早收敛以及训练过程停滞。

均方根传播（RMSProp）

Root mean square propagation（RMSProp）[145] 被提出作为对 AdaGrad 的一种改进，以缓解其学习率衰减过快的问题。RMSProp 的形式为

$$v_{k+1} = \beta_2 v_k + (1-\beta_2) g_k \odot g_k, \tag{2.126a}$$

$$x_{k+1} = x_k - \alpha_k \frac{g_k}{\sqrt{v_{k+1}} + \epsilon}, \tag{2.126b}$$

其中 β_2 通常取为 0.99，且 v_1 初始化为 0。由此可见，(2.126a) 中的加权求和缓解了平方梯度不断累积的问题，因此 (2.126b) 中的分母向量不会像 AdaGrad 中那样快速增大。

由于最近的梯度有较高权重，RMSProp 在许多应用中表现良好，并曾广泛用于训练循环神经网络。然而，RMSProp 似乎对超参数选择较为敏感。

自适应矩估计（Adam）

Adaptive Momentum Estimate（Adam）优化器 [76] 结合了动量与 RMSprop 技术，从而提供更平衡、更高效的优化过程。Adam 的关键方程如下：Adam 通过同时维护梯度的一阶矩和二阶矩的指数滑动平均，融合了动量方法与 RMSProp 的思想。具体地，在第 k 次迭代中，Adam 按如下方式更新一阶矩估计 m_k 和原始二阶矩估计 v_k：

$$m_{k+1} = \beta_1 m_k + (1-\beta_1) g_k, \tag{2.127a}$$

$$v_{k+1} = \beta_2 v_k + (1-\beta_2) g_k \odot g_k, \tag{2.127b}$$

其初值为 $m_1 = v_1 = 0$。由于这样的初值会使 m_k 和 v_k 向 0 偏置，Adam 试图通过它们的修正版本来校正这种偏差：

$$\hat{m}_{k+1} = \frac{m_{k+1}}{1-\beta_1^k}, \tag{2.128a}$$

$$\hat{v}_{k+1} = \frac{v_{k+1}}{1-\beta_2^k}, \tag{2.128b}$$

其中，只要 $\beta_1, \beta_2 \in (0,1)$，(2.128) 中两个修正分母都会在 $k \to \infty$ 时趋于 1。然后 Adam 按如下方式更新 x_k：

$$x_{k+1} = x_k - \alpha_k \frac{\hat{m}_{k+1}}{\sqrt{\hat{v}_{k+1}} + \epsilon}. \tag{2.129}$$

超参数通常仍与前面类似：$\beta_1 = 0.9$，$\beta_2 = 0.999$，$\epsilon = 10^{-8}$。步长 α_k 可以固定为常数，并且通常取 10^{-3}，这一般效果较好，但有时也可能小到 10^{-5}。

Adam 因其在广泛的网络结构和数据集上都具有快速收敛性和鲁棒性而被广泛采用。然而，后续研究表明，在某些情形下 Adam 的泛化性能可能不如带动量的 SG，并且在某些理论设定下甚至可能不收敛 [116]。

带解耦权重衰减的 Adam（AdamW）

控制 x 的取值并稳定优化过程的一种典型方法，是直接在目标函数中加入 Tikhonov 正则化。也就是说，可以将某个优化算法，例如 Adam，应用于最小化 $f(x) + \frac{\lambda}{2}|x|^2$，而不是 $f(x)$。其中 λ 是用于权衡项 $|x|^2$ 的超参数。正如我们所见，Tikhonov 正则化倾向于减小 x 各个分量的幅值。然而，这种修改会改变目标函数、它的一阶矩和二阶矩，最重要的是，它还会改变最优解本身。

Adam with Decoupled Weight Decay（AdamW） [90] 则提供了一种替代方案：在更新 x 时直接添加一个衰减项。更具体地说，AdamW 进行与 Adam 中相同的计算 (2.127)–(2.128)，但仅将 (2.129) 改为

$$x_{k+1} = x_k - \alpha_k \lambda x_k - \alpha_k \frac{\hat{m}_{k+1}}{\sqrt{\hat{v}_{k+1}} + \epsilon}, \tag{2.130}$$

这与 (2.129) 相比多了一个额外项 $\alpha_k \lambda x_k$。这里 λ 被称为衰减权重，在实际中通常取为 0.01。

在多种经验测试中，相较于 Adam，AdamW 似乎能够改善泛化能力和稳定性，特别是在诸如 transformer 这样的大规模模型中。因此，AdamW 已成

为许多现代深度学习框架中的默认优化器。

由 Newton-Schulz 正交化的动量（Muon）

Momentum Orthogonalized by Newton-Schulz（Muon） [73] 是一种利用深度神经网络中权重参数奇异值的优化方法。回忆一下，深度神经网络中的大多数构件都具有形如 $Wx+b$ 的仿射操作，其中 W 是权重矩阵。为便于表述，我们称 W 为一个二维（2D）权重矩阵，并在下文中假设其大小为 $d_{\text{out}} \times d_{\text{in}}$。

注意，在我们前面讨论的所有优化算法中，W 都被重新展平为一个向量，并与 b 一起拼接成总参数向量 $\theta \in \mathbb{R}^n$ 的一部分（本章中我们使用 $x \in \mathbb{R}^n$，但在网络训练中它指的就是参数 $\theta \in \mathbb{R}^n$）。因此，在网络训练的优化过程中，W 的矩阵结构被忽略了。

然而，权重矩阵在仿射变换中扮演着重要角色，并且对网络的 Lipschitz 常数有很强的影响；该网络本质上是一个从输入映射到对应输出的映射。因此，Muon 提出通过正交化来正则化关于权重矩阵的梯度。为此，Muon 对矩阵动量 $\mu M_k + g_k$ 施加若干次 Newton–Schulz (NS) 迭代，以将所有奇异值归一化为 1。Muon 的形式为

$$M_{k+1} = \mu M_k + g_k, \tag{2.131a}$$

$$O_{k+1} = \mathrm{NS}(M_{k+1}; (a,b,c), \epsilon, K_{\mathrm{NS}}), \tag{2.131b}$$

$$x_{k+1} = x_k - \alpha\lambda x_k - \alpha O_k, \tag{2.131c}$$

其中权重参数 μ 通常取为 0.95，λ 取为 0.1，步长 α 取为 0.001。

现在我们来解释 (2.131) 中的各项。首先注意，Muon 中的 g_k 指的是目标函数关于神经网络中每一个单独权重矩阵的（随机）梯度。Muon 也可以用于更新四维卷积参数，只需将四个维度中的三个恰当地展平，使其变成类似权重矩阵那样的二维矩阵。至于其他形式为标量或向量的参数，其梯度仍使用先前介绍的算法来更新，例如 Adam 和 AdamW。因此，下文中我们假设 g_k 是一个二维矩阵，且其大小与对应权重矩阵 $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 相同。动量 M_k 也是一个同样大小的矩阵。

Muon 方案中的关键步骤是 (2.131b)。对于任意一个大小为 $m \times n$ 的矩阵 M 作为输入（不失一般性，下文中假设 $m \geq n$），该步骤返回一个与其同样大小的矩阵 O，作为 $UV^\top$ 的近似，其中 $M = U\Sigma V^\top$ 是 M 的**奇异值分解**（singular value decomposition，SVD）。我们知道，SVD 可以将任意实矩阵或复矩阵 M 分解为 $U\Sigma V^\top$，其中 $U \in \mathbb{R}^{m\times n}$ 的 n 个列向量在 $\mathbb{R}^m$ 中正交归一

（即，U 的列向量是 $\mathbb{R}^m$ 的某组标准正交基的一个子集），V 是一个 $n \times n$ 的正交矩阵，而 Σ 是一个 $n \times n$ 的对角矩阵，其非负对角元称为 M 的**奇异值**。因此，U 和 V 的列向量分别称为 M 的左、右**奇异向量**。关于 SVD 的细节可参见 [49,148]。因此，步骤 (2.131b) 的作用是从 $M = U\Sigma V^\top$ 中去掉 Σ，并近似得到 M 的正交版本 $O = UV^\top$。也就是说，O 是如下问题的解

$$\min_{O \in \mathbb{R}^{m \times n}} \frac{1}{2} \|O - M\|_F^2, \tag{2.132}$$

其中 $\|A\|_F := (\sum_{i=1}^m \sum_{j=1}^n |A_{ij}|^2)^{1/2}$ 是 $m \times n$ 矩阵 $A = [A_{ij}]_{i,j}$ 的 Frobenius 范数。这实际上等价于将 A 重新看作一个 mn 维向量并施加标准的向量 2-范数。此外，Frobenius 范数是由内积 $\langle A, B \rangle_F := \mathrm{tr}(A^\top B)$（对任意 $A, B \in \mathbb{R}^{m \times n}$）诱导出的范数。由于 Frobenius 范数具有这些性质，很容易看出 (2.132) 的极小化解是 $UV^\top$。这就是 Muon 中所谓的正交化。

然而，SVD 的计算代价较高，因此在实践中往往希望避免用它来求解 (2.132)。NS 迭代是一种能够快速逼近 (2.132) 解的方法，因此 Muon 在 (2.131b) 中采用了它。

NS 迭代的思想如下。我们定义关于 σ 的一个五次多项式 $\phi : \mathbb{R} \to \mathbb{R}$（这里的 σ 指矩阵的一个正奇异值）：

$$\phi(\sigma) := a\sigma + b\sigma^3 + c\sigma^5. \tag{2.133}$$

我们也将这一定义扩展到矩阵 M 上，并把 ϕ 作用于矩阵：

$$\begin{aligned}
\phi(M) &:= aM + b(MM^\top)M + c(MM^\top)^2 M \\
&= \Big(aI_m + b(MM^\top) + c(MM^\top)^2\Big)M \\
&= \Big(aI_m + b(U\Sigma^2 U^\top) + c(U\Sigma^4 U^\top)\Big)U\Sigma V^\top \\
&= U\Big(a\Sigma + b\Sigma^3 + c\Sigma^5\Big)V^\top,
\end{aligned} \tag{2.134}$$

其中我们重复利用了 SVD 分解 $M = U\Sigma V^\top$ 及 $U^\top U = I_n$ 来得到上述等式。我们看到，(2.134) 中的 $\phi(M)$ 实际上是将 (2.133) 分别作用于 M 的每一个奇异值 σ。

因此，为了让 NS 迭代快速将所有奇异值归一化为 1，我们只需找到 (2.133) 中的系数 (a, b, c)，使得对任意正数 σ，经过少数几次 ϕ 的迭代后都能映射到

1。换句话说，我们希望找到 (2.133) 中的 (a,b,c)，使得

$$\phi^{(j)}(\sigma) \approx 1, \quad \forall \sigma > 0$$

并尽可能让 j 小，其中 $\phi^{(j)}$ 表示 ϕ 复合 j 次。如果能够找到这样的 (a,b,c)，那么对于任意 $M = U\Sigma V^\top$，我们就可以用 $\phi^{(j)}(M)$ 来近似 $O = UV^\top$。

在实践中，简单的取值 $(a,b,c) = (2,-1.5,0.5)$ 看起来已经足够有效。经过一些细致优化后，取值 $(a,b,c) = (3.4445,-4.775,2.0315)$ 似乎效果更好，因此目前已成为 PyTorch 等流行深度学习编程包中的默认实现设置 [107]。在 Muon 方法中，(2.131b) 中的 NS 迭代通常在达到最大迭代次数 K_{NS}（通常 5 次就足够）时终止，或者当 (a,b,c) 的相对变化小于 ϵ（通常取为 10^{-7}）时终止。

我们还可以将 Nesterov 动量的思想融入 Muon 方法 (2.131) 中。这通常通过在 (2.131b) 中用 $g_k + \mu B_{k+1}$ 代替 B_{k+1} 来实现，其中 μ 的默认取值为 0.95。

在实际实现中，对步长也有若干修改方式。特别地，在 [73] 中，步长 α 被乘以 $(\max(1, d_{\mathrm{out}}/d_{\mathrm{in}}))^{1/2}$；而在 [88] 中，则乘以 $0.2(\max(d_{\mathrm{out}}, d_{\mathrm{in}}))^{1/2}$。这些做法在大语言模型训练中经验上都表现得较为高效。

2.6 备注与参考文献

2.6.1 关于网络训练优化的备注

本章涵盖了优化理论中的大多数基本概念，以及若干优化算法和技术。重点放在非凸光滑优化问题上，这类问题在现代深度学习中极为常见。事实上，在过去几十年中，人们已经发展出大量关于确定性优化方法的结果，包括系统的收敛性分析和成熟的软件包。然而，在求解现代大规模优化问题时，它们似乎表现乏力 [20]。相比之下，深度学习领域的研究人员正在不断发明新的随机优化算法，以应对此类问题带来的挑战。尽管这些算法通常尚未具备完整而严格的收敛性保证，但它们惊人的效率使其在当今大型深度网络训练中占据主导地位。

除了 [20] 中给出的评论之外，我们还从实践角度对这一现象补充几点说明。众所周知，训练大规模神经网络对资源的需求极其巨大。它需要大量图形处理单元（graphics processing unit，GPU）以及巨大的电力消耗。一些大型网络，例如为现代大语言模型设计的网络，每个都拥有数十亿到数千亿个参数，

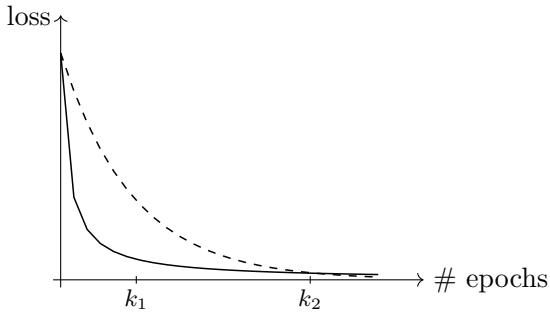


图 2.7: 两种优化算法下损失函数值衰减的示意性比较。第一种算法（实线）以次线性速率 $O(1/k)$ 收敛，其中 k 为 epoch 数；第二种算法（虚线）以线性速率 $O(\gamma^k)$ 收敛，其中某个 $\gamma \in (0,1)$。第一种算法具有较快的初始收敛，并且可能在 k_1 个 epoch 后就达到令人满意的解，而第二种算法具有更快的渐近收敛，并将在 k_2 个 epoch 后超过第一种算法。

其训练通常需要在 GPU 集群系统上运行数月才能完成。考虑到当前人工智能的迅猛发展、新的深度学习技术层出不穷，以及不同实体之间的激烈竞争，人们更倾向于选择那些能够在较短时间内以略微降低解质量为代价获得满意结果的、更快更高效的算法，这并不令人意外。相比之下，传统优化领域的方法更侧重于精心设计的算法结构、严格的收敛性分析、上下复杂度界下的紧致复杂度估计，以及灵活的超参数设置。

图 2.7 给出了一个示意图来解释这种情况。在图 2.7 中，横轴表示 epoch 数，也即实际计算成本；纵轴表示训练数据集上的损失函数值。实线表示一个具有次线性收敛率 $O(1/k)$ 的优化算法的表现，而虚线表示另一个具有线性收敛率 $O(\gamma^k)$ 的算法，其中某个 $\gamma \in (0,1)$。从理论上看，我们自然更倾向于第二种算法，因为它的线性收敛优于次线性收敛率，并且在渐近意义下会更快地收敛到最小损失函数值。

然而，面对图 2.7 中的两条衰减曲线，实践者可能会更偏好具有次线性收敛率（实线）的第一种算法，因为它在初始阶段收敛更快。如果该算法在第 k_1 个 epoch 时给出的结果已经足够令人满意，那么这种选择尤其合理。尽管具有线性收敛率（虚线）的算法会在 k_2 个 epoch 之后超过第一种算法，但解质量的提升可能不足以证明额外大量计算投入的合理性。例如，在推荐系统或生成式预训练变换器（generative pre-trained transformer，GPT）中，训练数据集上的评估指标提升几个百分点，人类可能根本察觉不到，但在实践中训练时间却可能是 4 个月（k_2）与 1 个月（k_1）。在这种情形下，大多数实践者都会选择使用（至少先尝试使用）具有次线性收敛率的算法。

由于深度神经网络规模巨大且结构复杂，许多因素都可能强烈影响深度学习模型的性能。在这些因素中，方法学与网络结构的选择往往最为关键。例如，一个经过中等程度训练的卷积神经网络，其图像分类精度可能远高于一个经过精心训练的有相同参数量的全连接网络。此外，数据样本的质量、初始化方式的选择以及预处理过程，都会对网络训练的经验性能产生显著影响。因此，在设计和训练深度学习模型时，实践者往往会优先关注这些方面，而通常直接选择一个在实践中以高效和快速收敛著称的常用优化器（如 Adam 或 AdamW），而不会过多在意其理论收敛性质。

2.6.2 数值优化的参考文献

关于优化理论及其应用的系统性介绍可参见 [29]。对于凸分析和优化算法中的基础理论结果，我们参考 [11, 13, 16, 19, 21, 102, 120]。一般连续优化理论与算法可见经典著作 [14, 105]。

第 2.1 节和第 2.2 节中关于基础优化理论与最优性条件的讨论主要遵循 [29]。第 2.3 节关于自动微分的内容遵循 [105]，并在全文中补充了一些额外细节与解释，特别是关于二阶导数计算的部分。第 2.4 节关于确定性优化算法的内容主要遵循 [14, 29, 105]。第 2.5.1 节给出的随机优化基础内容遵循 [20]。我们在第 2.5.2 节中对实用随机优化算法给出概述，其内容主要基于互联网上的讨论和简短教程，因为这些方法都是近几年才出现的。

第三章　深度最优控制

最优控制（optimal control）为设计一系列决策提供了严格的数学框架，这些决策在物理与操作约束条件下，将动力系统引导至期望目标，例如最大化回报（或等价地，最小化代价）。它在广泛的现实应用中发挥着核心作用。

在本章中，我们首先系统研究最优控制的基本概念与核心结果。特别地，我们将介绍最优控制的基础理论，包括 Pontryagin 极大值原理和 Hamilton–Jacobi–Bellman 方程，并给出它们的证明。随后，我们将这些经典结果推广到概率密度的最优控制，也称为平均场控制，其动机在于现代深度学习系统常常涉及对大量智能体的控制。因此，在这些情形下，许多感兴趣的问题都可以自然地表述为关于概率分布的控制问题。

尽管最优控制具有重要性并被广泛应用，但经典方法在高维情形或复杂非线性动力学存在时会遭遇严重的可扩展性限制。为应对这些挑战，我们引入一种基于深度学习的方法，称为 neural ordinary differential equation（Neural ODE）方法，将其作为求解广泛一类最优控制问题的有力工具。该方法既可以处理标准最优控制问题，也可以处理概率控制问题。最后，我们将以学习演化偏微分方程解算子这一任务为代表性例子，展示在解决新兴科学计算问题方面时，将深度学习与最优控制相结合产生的巨大潜力。

3.1　最优控制理论

在本节中，我们将对最优控制理论中的基础结果作简明而系统的回顾。我们首先介绍变分法及其在推导 Euler–Lagrange 方程中的作用。随后，我们引入 Hamiltonian 动力学，它为一般控制系统提供了统一且富有洞见的视角。最后，我们将介绍 Pontryagin 极大值原理与 Hamilton–Jacobi–Bellman 方程，并给出严格证明。

3.1.1 Euler–Lagrange 方程

最优控制是一个复杂的优化问题，其性质与第 二 章中讨论的标准优化非常不同。在最优控制问题中，被优化的对象是一个函数，称为**控制**（control），它通常表示为从时间区间 $[0,T]$（也可以扩展到 $[0,\infty)$ 或 $\mathbb{R}$，但我们将大部分讨论限制在有限时间窗口上，因为这已经具有足够的一般性）映射到某个多维欧几里得空间 $\mathbb{R}^m$ 的子集 $\mathcal{U}$ 的映射。这个子集 $\mathcal{U}$ 称为**控制的容许集**（admissible set of controls）。

尽管最优控制与标准优化有着相似的目标，即寻求使某个目标函数最小化或最大化的解，但其理论与求解方法与第 二 章中给出的标准优化方法非常不同（但仍然相关）。由于被优化的对象是一个随时间变化的控制函数 $u:[0,T]\to\mathcal{U}\subset\mathbb{R}^m$，我们需要新的理论、分析方法和计算工具来求解最优控制问题。

最优控制的基本方法称为**变分法**（calculus of variations），这种方法也称为**变分方法**。我们通过一个经典例子来介绍变分法和 Lagrangian 函数，并进一步导出 Euler–Lagrange 方程。

设给定一个光滑函数 $L:\mathbb{R}^d\times\mathbb{R}^d\to\mathbb{R}$。我们希望找到一条曲线 $x^*:[0,T]\to\mathbb{R}^d$，使得泛函

$$I[x]:=\int_0^T L(x(t),\dot{x}(t))\,dt, \tag{3.1}$$

在约束 $x(0)=x_0$ 和 $x(T)=x_1$ 下取最小值，其中 $x_0,x_1\in\mathbb{R}^d$ 为给定值。在 (3.1) 中，$\dot{x}$ 是 x 关于时间的导数（目前我们可以将 $\dot{x}$ 看作控制）。我们通常将 $x(t)$ 解释为位置（更一般地，我们称之为状态），将 $\dot{x}(t)$ 解释为时刻 t 的速度。与优化问题类似，我们的第一步是寻找 x^* 的最优性条件。也就是说，假设 x^* 是 (3.1) 中定义的 I 的一个极小化解，我们希望知道 x^* 必须满足哪些条件。这些关于最优控制的条件，对应于第 二 章中约束优化的 Lagrange 条件与 KKT 条件。

在文献中，(3.1) 中的被积函数 L 被称为 **Lagrangian 函数**。注意，这应与第 二 章中等式约束优化问题的 Lagrangian 函数区分开来，因为它们的定义和用途不同。我们通常将 L 写成关于 $x,v\in\mathbb{R}^d$ 的函数（其中 v 替代 $\dot{x}$，并被解释为速度）。在本节余下部分中，我们还使用如下记号表示偏导数：

$$\partial_x L:=(\partial_{x_1}L,\ldots,\partial_{x_d}L)\,,$$
$$\partial_v L:=(\partial_{v_1}L,\ldots,\partial_{v_d}L)\,,$$

其中

$$\partial_{x_i} L = \frac{\partial L}{\partial x_i} \quad 和 \quad \partial_{v_i} L = \frac{\partial L}{\partial v_i}$$

对每个 $i = 1, \ldots, d$ 成立。下面的定理表明，如果 x^* 是 (3.1) 的一个最优解，那么它必须满足所谓的 **Euler–Lagrange 方程**。该定理的证明使用了变分法的思想，具体见下文。

定理 3.1.1 (Euler–Lagrange equation)**.** *假设 x 是带边界约束的 (3.1) 的一个最优解，则 x 满足 Euler–Lagrange 方程:*

$$\frac{d}{dt}\Big(\partial_v L(x(t), \dot{x}(t))\Big) = \partial_x L(x(t), \dot{x}(t)). \tag{3.2}$$

证明. 回忆一下，无约束光滑优化问题的解位于目标函数梯度为零的临界点之中。我们希望采用类似的方法，尝试找到那些使 (3.1) 中定义的泛函 I 的“梯度”为零的曲线。然而，由于这里的“梯度”是关于曲线而不是向量的，我们不能直接这样求梯度。因此，我们先把问题转化为一个可以应用标准微积分的问题。

为此，我们任取一条光滑曲线 $y : [0, T] \to \mathbb{R}^d$，满足 $y(0) = y(T) = 0$。我们通常称 y 为 x 的一个扰动。现在我们看到，对任意 $\epsilon \in \mathbb{R}$，曲线 $x + \epsilon y : [0, T] \to \mathbb{R}^d$ 仍然满足边界约束，因为在 $t = 0$ 和 $t = T$ 时分别有 $x(0) + \epsilon y(0) = x_0$ 和 $x(T) + \epsilon y(T) = x_1$。定义 $\phi : \mathbb{R} \to \mathbb{R}$ 为

$$\phi(\epsilon) := I[x + \epsilon y] = \int_0^T L(x(t) + \epsilon y(t), \dot{x}(t) + \epsilon \dot{y}(t))\, dt. \tag{3.3}$$

由于 x 是 (3.1) 的一个极小化解，我们有

$$\phi(0) = I[x] \le I[x + \epsilon y] = \phi(\epsilon) \tag{3.4}$$

对任意 $\epsilon \in \mathbb{R}$ 成立。

由于 $L : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ 是一个光滑函数，我们知道 ϕ 是光滑的，并由 (3.4) 得到

$$\phi'(0) = 0.$$

现在我们需要求出 ϕ' 的表达式。为此，我们回忆 (3.3) 并得到

$$\phi'(\epsilon) = \int_0^T \Big[\partial_x L\Big(x(t) + \epsilon y(t), \dot{x}(t) + \epsilon \dot{y}(t)\Big) \cdot y(t) \\ + \partial_v L\Big(x(t) + \epsilon y(t), \dot{x}(t) + \epsilon \dot{y}(t)\Big) \cdot \dot{y}(t)\Big] dt.$$

于是当 $\epsilon = 0$ 时，我们得到

$$0 = \phi'(0) = \int_0^T \Big[\partial_x L(x(t), \dot{x}(t)) \cdot y(t) + \partial_v L(x(t), \dot{x}(t)) \cdot \dot{y}(t)\Big] dt. \tag{3.5}$$

现在我们来研究，在对所有满足 $y(0) = y(T) = 0$ 的 $y : [0, T] \to \mathbb{R}^d$ 都成立的前提下，(3.5) 蕴含什么。设我们从 $1, \ldots, d$ 中任选一个 i，并令

$$y(t) = (0, \ldots, 0, \underbrace{z(t)}_{i\text{th}}, 0, \ldots, 0) \in \mathbb{R}^d \tag{3.6}$$

对所有 $t \in [0, T]$ 成立，其中 $z : [0, T] \to \mathbb{R}$ 是一条满足 $z(0) = z(T) = 0$ 的光滑函数。那么，(3.6) 中定义的 y 在时刻 0 和 T 也满足约束。将 (3.6) 中定义的这个 y 代入 (3.5)，得到

$$0 = \int_0^T \Big(\partial_{x_i} L(x(t), \dot{x}(t)) z(t) + \partial_{v_i} L(x(t), \dot{x}(t)) \dot{z}(t)\Big) dt.$$

对上式右侧积分中的第二项做分部积分，得到

$$0 = \int_0^T \Big(\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt} \partial_{v_i} L(x(t), \dot{x}(t))\Big) z(t)\, dt. \tag{3.7}$$

这对所有满足 $z(0) = z(T) = 0$ 的光滑曲线 z 都成立。

由 (3.7)，我们断言

$$\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt} \partial_{v_i} L(x(t), \dot{x}(t)) = 0 \tag{3.8}$$

对所有 $t \in [0, T]$ 都成立。为了证明 (3.8)，我们可以采用反证法：如果在某个 $t \in [0, T]$ 处有 $\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt} \partial_{v_i} L(x(t), \dot{x}(t)) > 0$，那么由于 L 的光滑性，存在一个子区间 $(t - \delta, t + \delta)$（若 $t = 0$ 或 $t = T$，则取该区间的一半），使得在该区间内有 $\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt} \partial_{v_i} L(x(t), \dot{x}(t)) > 0$。取 $z > 0$ 于区间

$(t-\delta, t+\delta)$ 上，而在其他地方取 0，则有

$$
\begin{aligned}
0 &< \int_{t-\delta}^{t+\delta} \Big(\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt}\partial_{v_i} L(x(t), \dot{x}(t))\Big) z(t)\, dt \\
&= \int_0^T \Big(\partial_{x_i} L(x(t), \dot{x}(t)) - \frac{d}{dt}\partial_{v_i} L(x(t), \dot{x}(t))\Big) z(t)\, dt
\end{aligned}
$$

这与 (3.7) 矛盾。

由于 (3.8) 对任意 $i = 1, \ldots, d$ 都成立，将 (3.8) 重新整理即可得到 Euler–Lagrange 方程 (3.2)。 □

注意，Euler–Lagrange 方程 (3.2) 是关于 x 的一个由 d 个二阶常微分方程（second-order ODEs）构成的非线性系统。这些 ODE 的形式如 (3.8) 所示：

$$
\frac{d}{dt}\partial_{v_i} L(x(t), \dot{x}(t)) = \partial_{x_i} L(x(t), \dot{x}(t))
$$

其中 $i = 1, \ldots, d$。

我们还指出，对 L 和 x 的光滑性要求可以进一步放宽（例如，所涉及的梯度/导数除了在 $[0, T]$ 上某个测度为零的集合外处处存在）。在这种情况下，可以借助测度论给出严格证明，而结论保持不变。

3.1.2 Hamiltonian 动力学

Euler–Lagrange 方程 (3.2) 给出了 (3.1) 极小化解的一个必要条件，其中 x 的约束设置在区间 $[0, T]$ 的端点处。速度 $\dot{x}$ 可以被视为 x 的控制。

接下来，我们介绍另一个重要概念，称为 Hamiltonian 动力学，它与 Euler–Lagrange 方程 (3.2) 密切相关，但在一般最优控制问题中更为常见。与 (3.2) 是一个由 d 个二阶 ODE 构成的系统不同，Hamiltonian 动力学构成的是一个由 $2d$ 个一阶 ODE 构成的系统。

为此，设给定 $x : [0, T] \to \mathbb{R}^d$，我们定义另一条曲线 $p : [0, T] \to \mathbb{R}^d$，称为**广义动量**，如下：

$$
p(t) := \partial_v L(x(t), \dot{x}(t)) \tag{3.9}
$$

对所有 $t \in [0, T]$ 成立。我们的目标是将 (3.2) 重写为关于 (x, p) 的一个由 $2d$ 个一阶 ODE 构成的系统。

注意，将 Euler–Lagrange 方程转换为 Hamiltonian 动力学通常要求

$$
p = \partial_v L(x, v) \tag{3.10}
$$

能够在给定 (x,p) 时唯一确定 v。这可以通过例如假设在每个 x 和 v 处，$\partial^2_{vv}L(x,v)$ 非奇异来实现，从而可以应用隐函数定理说明这一要求成立。在本节余下部分中，我们假设 (3.10) 的这一要求成立，并将其解记为 $v(x,p)$。也就是说，$p=\partial_v L(x,v(x,p))$。

定义 3.1.2 (Hamiltonian)**.** 设 $x,p\in\mathbb{R}^d$，且 $v(x,p)$ 是 (3.10) 的解，则定义 $H:\mathbb{R}^d\times\mathbb{R}^d\to\mathbb{R}$ 为

$$H(x,p)=p\cdot v(x,p)-L(x,v(x,p)). \tag{3.11}$$

我们称 H 为 **Hamiltonian**函数。

与上面的 Lagrangian 函数 L 类似，我们将 H 的偏导数记为

$$\begin{aligned}\partial_x H &:= (\partial_{x_1}H,\ldots,\partial_{x_d}H)\,,\\ \partial_p H &:= (\partial_{p_1}H,\ldots,\partial_{p_d}H)\,,\end{aligned}$$

其中

$$\partial_{x_i}H=\frac{\partial H}{\partial x_i}\quad 和\quad \partial_{p_i}H=\frac{\partial H}{\partial p_i}$$

对每个 $i=1,\ldots,d$ 成立。现在我们准备说明，Euler–Lagrange 方程 (3.2) 可以利用 H 转化为 Hamiltonian 动力学。

定理 3.1.3 (Hamiltonian dynamics)**.** *设 x 是 Euler–Lagrange 方程 (3.2) 的解，并按 (3.9) 定义 p。则 $(x,p):[0,T]\to\mathbb{R}^d\times\mathbb{R}^d$ 这一对函数满足 Hamiltonian 动力学:*

$$\begin{cases}\dot{x}(t)=\partial_p H(x(t),p(t)),\\ \dot{p}(t)=-\partial_x H(x(t),p(t)).\end{cases} \tag{3.12}$$

此外，$H(x(t),p(t))$ 是 t 的常函数。

证明. 由 (3.11) 中 Hamiltonian H 的定义，其中 $v=v(x,p)$ 满足 $p=\partial_v L(x,v)$，我们有

$$\begin{aligned}\partial_x H(x,p)&=p\cdot\partial_x v(x,p)-\partial_x L(x,v(x,p))-\partial_v L(x,v(x,p))\cdot\partial_x v(x,p)\\ &=-\partial_x L(x,v(x,p))\end{aligned} \tag{3.13}$$

因为 $p = \partial_v L(x, v(x,p))$。因此，我们有

$$\begin{aligned}\dot{p}(t) &= \frac{d}{dt}\Big(\partial_v L(x(t), \dot{x}(t))\Big) \\ &= \partial_x L(x(t), \dot{x}(t)) \\ &= \partial_x L(x(t), v(x(t), p(t))) \\ &= -\partial_x H(x(t), p(t)),\end{aligned}$$

其中第一步是对 (3.9) 两边关于 t 求导所得，第二步由 Euler–Lagrange 方程 (3.2) 得到，第三步是因为 $p(t) = \partial_v L(x(t), \dot{x}(t))$ 蕴含 $\dot{x}(t) = v(x(t), p(t))$，最后一步来自 (3.13)。

此外，由 (3.11) 中 H 的定义可知

$$\begin{aligned}\partial_p H(x,p) &= v(x,p) + p \cdot \partial_p v(x,p) - \partial_v L(x, v(x,p)) \cdot \partial_p v(x,p) \\ &= v(x,p)\end{aligned}$$

因为 $p = \partial_v L(x, v(x,p))$。因此，对每个 $t \in [0,T]$，有

$$\partial_p H(x(t), p(t)) = v(x(t), p(t))$$

成立。同样，由于 $p(t) = \partial_v L(x(t), \dot{x}(t))$ 蕴含 $\dot{x}(t) = v(x(t), p(t))$，于是得到

$$\dot{x}(t) = \partial_p H(x(t), p(t)).$$

至此，我们已经说明 $(x(t), p(t))$ 满足 ODE 系统 (3.12)。

为了说明 $H(x(t), p(t))$ 是常数，我们对其关于 t 求导：

$$\begin{aligned}\frac{d}{dt} H(x(t), p(t)) &= \partial_x H(x(t), p(t)) \cdot \dot{x}(t) + \partial_p H(x(t), p(t)) \cdot \dot{p}(t) \\ &= -\dot{p}(t) \cdot \dot{x}(t) + \dot{x}(t) \cdot \dot{p}(t) \\ &= 0,\end{aligned}$$

其中第二步由 (3.12) 得到。证明完成。 □

例 3.1.4. 我们考虑经典力学中的一个例子，其 Lagrangian 函数为

$$L(x,v) = \frac{1}{2} m |v|^2 - V(x), \tag{3.14}$$

其中 x 是物体的位置，m 是物体的质量，v 是物体的速度。函数 $V:\mathbb{R}^d\to\mathbb{R}$ 是物体的势能。因此，(3.14) 中的 L 是动能与势能之差。

我们还可以看到

$$\partial_x L(x,v) = -\nabla V(x) \quad 和 \quad \partial_v L(x,v) = mv.$$

将它们代入 Euler–Lagrange 方程 (3.2)，得到

$$m\ddot{x}(t) = -\nabla V(x(t)).$$

因此，函数 (3.14) 的 Euler–Lagrange 方程就是描述 Newton 第二运动定律的二阶 ODE。进一步地，广义动量恰好就是经典力学中的动量：

$$p = \partial_v L(x,v) = mv,$$

这意味着

$$v = v(x,p) = \frac{p}{m}.$$

将 x 和 $v(x,p)=\frac{p}{m}$ 代入 Hamiltonian H (3.11)，可得

$$\begin{aligned} H(x,p) &= p\cdot\frac{p}{m} - L\Big(x,\frac{p}{m}\Big) \\ &= \frac{|p|^2}{m} - \frac{1}{2}m\frac{|p|^2}{m^2} + V(x) \\ &= \frac{|p|^2}{2m} + V(x), \end{aligned}$$

它就是动能与势能之和。如果再将 $p(t)=mv(t)$ 代回上式中的 H，则得到

$$H(x(t),p(t)) = \frac{1}{2}m|v(t)|^2 + V(x(t)).$$

由定理 3.1.3 可知，$H(x(t),p(t))$ 关于 t 保持为常数。这意味着动能与势能之和随时间不变，也就是说，总能量守恒。

我们还注意到，Hamiltonian 动力学为

$$\begin{cases} \dot{x}(t) = \partial_p H(x(t),p(t)) = \frac{p(t)}{m}, \\ \dot{p}(t) = -\partial_x H(x(t),p(t)) = -\nabla V(x(t)). \end{cases}$$

如果 $V(x) := mgx$，其中 x 是高度，g 是重力加速度（地球表面上约为

$9.81\mathrm{m/sec}^2$），那么有 $\dot{p}(t) = -mg$。

3.1.3 Pontryagin 极大值原理

上述讨论揭示了 Lagrangian 力学与 Hamiltonian 力学之间的一些基本联系。同时，Hamiltonian 动力学在控制理论中展现出重要意义：最优控制的必要条件可以通过 Hamiltonian 动力学来刻画。这正是本小节的目标。

首先，我们考虑一个关于商品交易的最优控制问题作为示例。假设我们是一名小麦交易商，在时刻 t 拥有现金 $x_1(t)$ 和库存中的 $x_2(t)$ 单位小麦。在初始时刻 $t=0$，我们有 $x_1(0)=x_{1,0}$ 和 $x_2(0)=x_{2,0}$，其中 $x_{1,0}, x_{2,0} \geq 0$ 为给定值。设 $c>0$ 为每单位小麦每单位时间的存储成本。为简便起见，假设我们已知任意时刻 t 每单位小麦的价格 $q(t)$。现在我们需要找到最优的 $u(t)$，它表示在时刻 t 买入或卖出小麦的速率（若 $u(t)>0$ 表示买入，若 $u(t)<0$ 表示卖出），使得在给定终止时刻 T 的总持有价值最大化。

为了将这个商品交易问题数学化，我们注意到 $x(t)=(x_1(t), x_2(t)) \in \mathbb{R}^2$ 的动力学由下式给出

$$\begin{cases} \dot{x}_1(t) = -cx_2(t) - q(t)u(t), \\ \dot{x}_2(t) = u(t), \end{cases} \tag{3.15}$$

其初值为 $x_1(0)=x_{1,0}$ 和 $x_2(0)=x_{2,0}$。之所以有 (3.15)，是因为金钱支出的变化率 $\dot{x}_1(t)$ 等于库存小麦 $x_2(t)$ 的存储成本与以速率 $u(t)$ 买入小麦的支出（或卖出小麦的收入）；而库存小麦的变化率 $\dot{x}_2(t)$ 就等于 $u(t)$。时刻 T 的总持有价值由下式给出

$$I[u] := x_1(T) + q(T)x_2(T), \tag{3.16}$$

它是时刻 T 的手头现金 $x_1(T)$ 与库存小麦价值 $q(T)x_2(T)$ 之和。因此，该最优控制问题就是寻找速率函数 $u:[0,T]\to\mathbb{R}$，使得在初值 $x(0)=(x_1(0),x_2(0))=(x_{1,0},x_{2,0})$ 以及约束 (3.15) 下，由 (3.16) 定义的总持有价值 I 最大化。这里为简便起见，我们允许 $u(t)$ 取任意实数。在实际中，我们通常要求对每个 t 都有 $u(t)\in\mathcal{U}$，其中 $\mathcal{U}$ 是 $\mathbb{R}$ 的一个子集（若 $u(t)$ 是一个 m 维向量，则 $\mathcal{U}$ 是 $\mathbb{R}^m$ 的一个子集）。在下文中，我们用 $\mathcal{U}_T := \{u:[0,T]\to\mathcal{U}\subset\mathbb{R}^m\}$ 表示整个时间区间 $[0,T]$ 上控制 u 的容许集。

上面的商品交易例子代表了一类典型的一般最优控制问题的构造形式。更

准确地说，这些最优控制问题写成

$$\max_{u\in\mathcal{U}_T} \quad I[u] := \int_0^T r(x(t),u(t))\,dt + g(x(T)), \tag{3.17a}$$

$$\text{s.t.} \quad \dot{x}(t) = f(x(t),u(t)), \tag{3.17b}$$

$$x(0) = x_0, \tag{3.17c}$$

其中待优化的变量是控制轨迹 $u \in \mathcal{U}_T$。

将上面的商品交易例子与 (3.17) 相比较，我们看到动力学 (3.15) 对应于 ODE 约束 (3.17b)，初值 $x(0) = (x_{1,0}, x_{2,0})$ 对应于 (3.17c)，而 (3.16) 中的总持有价值 $x_1(T) + q(T)x_2(T)$ 就对应于 (3.17a) 中的 $g(x(T))$，它被称为**终端奖励**，terminal reward（或称终端收益）。在 (3.17a) 中，我们还有一个在商品交易例子 (3.16) 中未出现的附加项 $\int_0^T r(x(t),u(t))\,dt$。这是所谓**运行奖励**（running reward）$r : \mathbb{R}^d \times \mathbb{R}^m \to \mathbb{R}$ 的积分，正如我们稍后将说明的那样，它可以很容易地并入终端奖励 g 中。

注意，我们没有在 (3.17b) 中显式地把时间变量 t 写入 ODE 的定义函数 f 中，也没有写成 $f(t,x(t),u(t))$ 的形式。这是因为时间变量 t 可以通过引入一个导数恒为 1 的附加分量并入 $x(t)$ 中。更准确地说，我们可以定义

$$\bar{x}(t) := (t, x(t)),$$
$$\bar{f}(\bar{x}(t),u(t)) := (1, f(t,x(t),u(t))),$$

于是有

$$\dot{x}(t) = f(t,x(t),u(t)) \quad \Longleftrightarrow \quad \dot{\bar{x}}(t) = \bar{f}(\bar{x}(t),u(t))$$

而右侧仍然具有 (3.17b) 的形式。

最优控制问题 (3.17) 常被称为**固定终止时刻且终点自由**问题，因为终止时刻 T 是固定的，而终点 $x(T)$ 不是固定的。最优控制还有另一种变体，称为固定终点且终止时刻自由问题，它与 (3.17) 具有类似的最优性理论。本节中我们也会讨论这种变体。

接下来，我们给出关于 (3.17) 解的最优性条件中最重要的结果之一。这一结果称为 **Pontryagin 极大值原理**。我们先给出 control Hamiltonian 函数的定义，该函数将在 Pontryagin 极大值原理及其证明中使用。

定义 3.1.5 (Control Hamiltonian)**.** 设 f 和 r 分别是 (3.17) 中的 ODE 定义函数和运行奖励函数。则 **control Hamiltonian**函数 $H : \mathbb{R}^d \times \mathbb{R}^d \times \mathcal{U} \to \mathbb{R}$

定义为

$$H(x,p,u) := p \cdot f(x,u) + r(x,u) \tag{3.18}$$

对任意 $(x,p,u) \in \mathbb{R}^d \times \mathbb{R}^d \times \mathcal{U}$ 成立。

与 (3.11) 中的 Hamiltonian 函数比较可知，(3.18) 中的 control Hamiltonian 函数多了控制变量 $u \in \mathcal{U} \subset \mathbb{R}^m$，(3.11) 中的控制速度 v 被 (3.17b) 中由 u 控制的 ODE 中的函数 f （我们称 f 为此 ODE 的**定义函数**）所替代，而负的 Lagrangian，$-L$（其中 (3.1) 中对 L 的积分是要最小化的）则被 (3.17a) 中要最大化的运行奖励函数 r 所替代。

现在我们给出两个在 Pontryagin 极大值原理及其证明中至关重要的引理。

引理 3.1.6. 设 $x : [0,T] \to \mathbb{R}^d$ 是如下初值问题的解

$$\begin{cases} \dot{x}(t) = f(x(t), u(t)), \quad \forall\, t \in [0,T], \\ x(0) = x_0 \end{cases} \tag{3.19}$$

其中 $f : \mathbb{R}^d \times \mathbb{R}^m \to \mathbb{R}^d$ 为给定定义函数, $u : [0,T] \to \mathbb{R}^m$ 为控制函数, $x_0 \in \mathbb{R}^d$ 为给定初值。假设对每个 t，$f(\cdot, u(t))$ 关于 x 是 L_f-*Lipschitz* 连续且为 C^1。令 $y_0 \in \mathbb{R}^d$ 任意固定。对任意 $\epsilon > 0$，设 x^ϵ 是初值问题

$$\begin{cases} \dot{x}^\epsilon(t) = f(x^\epsilon(t), u(t)), \\ x^\epsilon(0) = x_0 + \epsilon y_0 + o(\epsilon). \end{cases} \tag{3.20}$$

的解。则有

$$x^\epsilon(t) = x(t) + \epsilon y(t) + o(\epsilon) \tag{3.21}$$

在 $[0,T]$ 上关于 t 一致成立，即

$$\lim_{\epsilon \to 0} \sup_{t \in [0,T]} \frac{|x^\epsilon(t) - x(t) - \epsilon y(t)|}{\epsilon} = 0,$$

其中 y 是初值问题

$$\begin{cases} \dot{y}(t) = \partial_x f(x(t), u(t)) y(t), \\ y(0) = y_0 \end{cases} \tag{3.22}$$

的解。

证明. 我们定义

$$\delta(t) := \frac{1}{2}|x^\epsilon(t) - x(t)|^2. \tag{3.23}$$

则有

$$\begin{aligned}\dot{\delta}(t) &= (x^\epsilon(t) - x(t)) \cdot (\dot{x}^\epsilon(t) - \dot{x}(t)) \\ &= (x^\epsilon(t) - x(t)) \cdot (f(x^\epsilon(t), u(t)) - f(x(t), u(t))) \\ &= (x^\epsilon(t) - x(t)) \cdot \partial_x f(\xi^\epsilon(t), u(t))(x^\epsilon(t) - x(t)) \\ &\leq 2L_f \delta(t),\end{aligned}$$

其中由于 Taylor 定理，$\xi^\epsilon(t)$ 是位于 $x(t)$ 与 $x^\epsilon(t)$ 连线段上的某一点。由 Grönwall 不等式，对所有 $t \in [0, T]$ 有

$$\delta(t) = \delta(0)e^{2L_f t} \leq \delta(0)e^{2L_f T}.$$

因此，当 ϵ 足够小时，

$$|x^\epsilon(t) - x(t)| = \sqrt{2\delta(t)} \leq \sqrt{2\delta(0)}e^{L_f T} = |\epsilon y_0 + o(\epsilon)|e^{L_f T} \leq C\epsilon|y_0|e^{L_f T}$$

对所有 $t \in [0, T]$ 成立，其中 C 是某个与 ϵ 和 t 无关的常数。因此，当 $t \to 0$ 时，$\|x^\epsilon - x\|_{L^\infty([0,T])} \to 0$，即 $x^\epsilon(t) - x(t) = O(\epsilon)$，并且当 $\epsilon \to 0$ 时在 $[0, T]$ 上一致趋于 0。

对每个 $t \in [0, T]$ 定义

$$y(t) := \lim_{\epsilon \to 0} \frac{x^\epsilon(t) - x(t)}{\epsilon}.$$

则显然有

$$y(0) = \lim_{\epsilon \to 0} \frac{x^\epsilon(0) - x(0)}{\epsilon} = \lim_{\epsilon \to 0} \left(y_0 + \frac{o(\epsilon)}{\epsilon}\right) = y_0,$$

并且

$$
\begin{aligned}
\dot{y}(t) &= \lim_{\epsilon\to 0}\frac{\dot{x}^\epsilon(t)-\dot{x}(t)}{\epsilon}\\
&= \lim_{\epsilon\to 0}\frac{f(x^\epsilon(t),u(t))-f(x(t),u(t))}{\epsilon}\\
&= \lim_{\epsilon\to 0}\frac{1}{\epsilon}\Big(\partial_x f(x(t),u(t))(x^\epsilon(t)-x(t))\\
&\qquad\qquad +\frac{1}{2}(x^\epsilon(t)-x(t))\partial^2_{xx}f(\eta^\epsilon(t),u(t))(x^\epsilon(t)-x(t))\Big)\\
&= \lim_{\epsilon\to 0}\Big(\partial_x f(x(t),u(t))\frac{x^\epsilon(t)-x(t)}{\epsilon}\\
&\qquad\qquad +\frac{\epsilon}{2}\frac{x^\epsilon(t)-x(t)}{\epsilon}\partial^2_{xx}f(\eta^\epsilon(t),u(t))\frac{x^\epsilon(t)-x(t)}{\epsilon}\Big)\\
&= \partial_x f(x(t),u(t))y(t),
\end{aligned}
$$

其中 $\eta^\epsilon(t)$ 是位于 $x(t)$ 与 $x^\epsilon(t)$ 连线段上的某一点。注意，第一步要求 $\partial_\epsilon\partial_t x^\epsilon(t)=\partial_t\partial_\epsilon x^\epsilon(t)$。这成立的一个充分条件是 Clairaut 定理：若二阶偏导都存在且连续，则偏导次序可以交换。当 f 足够光滑时，例如 f 为 C^1，这一点成立。

总结而言，我们得到 $x^\epsilon(t)=x(t)+\epsilon y(t)+o(\epsilon)$ 在 $[0,T]$ 上一致成立，其中 y 是初值问题 (3.22) 的解。 □

为了严格证明 Pontryagin 极大值原理，我们需要覆盖控制轨迹的所有可能变化。理论上说，最优控制问题并不要求控制 u 关于 t 连续。因此，我们考虑下面这种所谓的简单变分，它允许对控制函数 u 进行不连续的改变。

定义 3.1.7 (简单变分，simple variation)**.** 对任意 $\tau\in(0,T]$ 和充分小的 ϵ，使得 $0<\tau-\epsilon<\tau\le T$，定义 $u(t)$ 的**简单变分**（也称为**针状变分**或 **needle-like variation**）为

$$
u^\epsilon(t):=\begin{cases} w, & \text{if } \tau-\epsilon<t<\tau,\\ u(t), & \text{otherwise.}\end{cases}
\tag{3.24}
$$

对于简单变分 u^ϵ，令 x^ϵ（注意这里的 x^ϵ 不再是上面 (3.20) 中定义的那个）为如下初值问题的解

$$
\begin{cases} \dot{x}^\epsilon(t)=f(x^\epsilon(t),u^\epsilon(t)), \quad \forall\, t\in[0,T],\\ x^\epsilon(0)=x_0.\end{cases}
\tag{3.25}
$$

定量刻画原轨道 x 与该 x^ϵ 之间的差异是很重要的。为此，我们从现在起

要求 $f \in C^1(\mathbb{R}^d \times \mathbb{R}^m; \mathbb{R}^d)$。然后我们说明，下面定义的函数 y 能够捕捉它们差异的一阶项。

定义 3.1.8 (扰动函数，perturbation)**.** 在 $t \in [0, \tau)$ 时，令 $y(t) = 0$ 成立。而在 $[\tau, T]$ 上，$y(t)$ 是初值问题

$$\begin{cases} \dot{y}(t) = \partial_x f(x(t), u(t)) y(t), \\ y(\tau) = f(x(t), w) - f(x(t), u(t)) \end{cases} \tag{3.26}$$

的解。我们称此 y 为**扰动函数**。

引理 3.1.9. *假设 x^ϵ 是带有简单变分 u^ϵ 的问题 (3.25) 的解，则当 $\epsilon \to 0$ 时，*

$$x^\epsilon(t) = x(t) + \epsilon y(t) + o(\epsilon) \tag{3.27}$$

在 $[0, T]$ 上关于 t 一致成立，其中 $y : [0, T] \to \mathbb{R}^d$ 是定义 3.1.8 中的扰动函数。

证明. 令 $t \in (0, \tau)$ 任意固定，则对任意 $\epsilon \in (0, \tau - t)$，有 $0 < t < \tau - \epsilon$，从而 $x^\epsilon(t) = x(t)$。

对于 $t \in [\tau, T]$，有

$$\begin{aligned} x^\epsilon(t) - x(t) &= \int_{\tau-\epsilon}^{t} \Big(f(x(s), w) - f(x(s), u(s)) \Big) ds \\ &= \Big(f(x(\tau), w) - f(x(\tau), u(\tau)) \Big) \epsilon + o(\epsilon) \end{aligned}$$

当 $\epsilon \to 0$ 时成立，这是由于 f 关于 u 的连续性。由于在区间 $[\tau, T]$ 上，$x(t)$ 与 $x^\epsilon(t)$ 满足以同一个 u 构成的定义函数 f 的 ODE，只是在时刻 τ 的初值不同：

$$x^\epsilon(\tau) = x(\tau) + \Big(f(x(\tau), w) - f(x(\tau), u(\tau)) \Big) \epsilon + o(\epsilon).$$

由引理 3.1.6 可知，当 $\epsilon \to 0$ 时，对所有 t 都有 $x^\epsilon(t) = x(t) + \epsilon y(t) + o(\epsilon)$，其中 y 是定义 3.1.8 中的扰动函数。 □

在证明 Pontryagin 极大值原理之前，我们还要引入控制系统中状态（state）$x : [0, T] \to \mathbb{R}^d$ 的协态（co-state）$p : [0, T] \to \mathbb{R}^d$ 的概念。在下面的讨论中，我们将每个时刻的 $p(t) \in \mathbb{R}^d$ 看作一个行向量（注意它与 $x(t)$ 具有相同维数），并且它的作用类似于约束（由初值问题 (3.19) 构成）对应的 Lagrange 乘子。协态也称为伴随函数（adjoint function），它满足所谓的伴随方程（adjoint equation），如下所示。

定义 3.1.10 (协态与伴随方程)**.** 设 x 是在控制 u 下初值问题 (3.19) 的解。若 $p:[0,T]\to\mathbb{R}^d$ 满足如下终值问题

$$\begin{cases}\dot{p}(t)=-p(t)\partial_x f(x(t),u(t))-\partial_x r(x(t),u(t)), & \forall\, t\in[0,T],\\ p(T)=\nabla g(x(T)).\end{cases}\tag{3.28}$$

则称 p 为**协态**。(3.28) 中的 ODE 称为**伴随方程**。

无运行奖励时的 Pontryagin 极大值原理

Pontryagin 极大值原理给出了最优控制的一个必要条件。因此，要建立该原理，必须假设给定控制是最优的（正如在优化中推导必要最优性条件时，我们要假设局部极小点在其邻域内取得最小目标函数值一样）。现在我们的目标函数 $I[u]$ 由 (3.17a) 给出。我们先从 I 仅含终端奖励 $g(x(T))$ 的情形开始。

引理 3.1.11. *终端奖励 $I[u]=g(x(T))$（假设 (3.17a) 中没有运行奖励）的变分由下式给出*

$$\frac{d}{d\epsilon}I[u^\epsilon]\Big|_{\epsilon=0}=p(\tau)\Big(f(x(\tau),w)-f(x(\tau),u(\tau))\Big).\tag{3.29}$$

证明. 根据引理 3.1.9，我们有

$$I[u^\epsilon]=g(x^\epsilon(T))=g(x(T)+\epsilon y(T)+o(\epsilon)),$$

其中 y 是定义 3.1.8 中的扰动函数。因此，

$$\frac{d}{d\epsilon}I[u^\epsilon]\Big|_{\epsilon=0}=\nabla g(x(T))y(T).\tag{3.30}$$

另一方面，在没有 r 的情况下，由伴随方程 (3.28) 可得

$$\begin{aligned}\frac{d}{dt}(p(t)y(t))&=\dot{p}(t)y(t)+p(t)\dot{y}(t)\\&=-p(t)\partial_x f(x(t),u(t))y(t)+p(t)\partial_x f(x(t),u(t))y(t)\\&=0,\end{aligned}$$

这说明 $p(t)y(t)$ 关于 t 为常数。因此有

$$\nabla g(x(T))y(T)=p(T)y(T)=p(\tau)y(\tau),\tag{3.31}$$

其中第一个等号来自 (3.28) 中 p 的终端条件。注意到 $y(\tau)$ 的值由 (3.26) 给出，将 (3.30) 与 (3.31) 结合即可得到 (3.29)。 □

现在我们已准备好给出并证明 Pontryagin 极大值原理（Pontryagin Maximum Principle）。同样地，我们仍考虑目标函数中没有运行奖励的最优控制问题 (3.17)。

定理 3.1.12 (Pontryagin 极大值原理)**.** *设 $u(\cdot)$ 是 (3.17) 的最优解，x 是满足 (3.17b) 与 (3.17c) 的对应轨迹。则存在一个函数 $p:[0,T]\to\mathbb{R}^n$，使得 (x,p) 满足 control Hamiltonian 动力学：*

$$\begin{cases} \dot{x}(t) = \partial_p H\left(x(t), p(t), u(t)\right), \\ \dot{p}(t) = -\partial_x H\left(x(t), p(t), u(t)\right), \end{cases} \tag{3.32}$$

其中 x 的初始条件与 p 的终端条件分别为

$$x(0) = x_0 \quad \text{和} \quad p(T) = \nabla g(x(T)).$$

此外，对每个 t，下面这一恒等式成立，它被称为 ***Pontryagin* 极大值原理***：*

$$H(x(t), p(t), u(t)) \;=\; \max_{w\in\mathcal{U}} H(x(t), p(t), w). \tag{3.33}$$

证明. 当没有运行奖励 r 时，control Hamiltonian (3.18) 为

$$H(x,p,u) = p\cdot f(x,u).$$

因此，(3.32) 就是 (3.17b) 中给出的 x 的 ODE 与 (3.28) 中给出的 p 的伴随方程的组合。x 的初始条件和 p 的终端条件分别由 (3.17c) 与 (3.28) 给出。

令 $t\in(0,T)$ 固定，且 $w\in\mathcal{U}$ 任意，则由于 u 是最优的，$\epsilon\mapsto I[u^\epsilon]$ 在 $\epsilon=0$ 处取得极大值。因此

$$0 \geq \frac{d}{d\epsilon} I[u^\epsilon]\Big|_{\epsilon=0} = p(\tau)\Big(f(x(t),w) - f(x(t),u(t))\Big). \tag{3.34}$$

因此，我们得到

$$\begin{aligned} H(x(t),p(t),w) &= p(t)f(x(t),w) \\ &\le p(t)f(x(t),u(t)) \\ &= H(x(t),p(t),u(t)). \end{aligned}$$

又由于 $u(t)\in\mathcal{U}$，于是得到极大值原理 (3.33)。 □

注意，带控制的 Hamiltonian 动力学 (3.32) 与无控制时的 (3.12) 具有相同的形式。为了记住这一动力学，只需记住它是关于状态与协态二元组 (x,p) 的一阶 ODE 非线性系统，其定义函数为 $(\partial_p H, -\partial_x H)$。这种动力学结构称为**辛形式**（symplectic form）。

带运行奖励的 Pontryagin 极大值原理

现在考虑目标函数 $I[u]$ 包含运行奖励的一般情形：

$$I[u] := \int_0^T r(x(t),u(t))\,dt + g(x(T)). \tag{3.35}$$

此时，control Hamiltonian 保持其原始形式 (3.18)：

$$H(x,p,a) = p\cdot f(x,a) + r(x,u).$$

为了推导这种情形下的 Pontryagin 极大值原理，我们只需引入一个辅助变量 $x_{d+1}:[0,T]\to\mathbb{R}$ 来记录运行奖励，这样目标函数就可以写成不含运行奖励的形式，从而化归到上面已讨论过的情形。

为此，定义 x_{d+1} 满足

$$\begin{cases} \dot{x}_{d+1}(t) = r(x(t),u(t)), \quad \forall\, t\in[0,T], \\ x_{d+1}(0) = 0, \end{cases} \tag{3.36}$$

其中 x 满足原始初值问题：

$$\begin{cases} \dot{x}(t) = f(x(t),u(t)), \quad \forall\, t\in[0,T], \\ x(0) = x_0. \end{cases} \tag{3.37}$$

将 x 与 x_{d+1} 连接成一个单一的扩展状态变量 $\bar{x}$ ，可得

$$\bar{x} = \begin{pmatrix} x \\ x_{d+1} \end{pmatrix} = \begin{pmatrix} x_1 \\ \vdots \\ x_d \\ x_{d+1} \end{pmatrix} \in \mathbb{R}^{d+1}. \tag{3.38}$$

接着，我们将扩展状态变量 $\bar{x}$ 与原控制 u 的定义函数 $\bar{f} : \mathbb{R}^{d+1} \times \mathcal{U} \to \mathbb{R}^{d+1}$ 记为

$$\bar{f}(\bar{x}, u) := \begin{pmatrix} f(x,u) \\ r(x,u) \end{pmatrix} = \begin{pmatrix} f_1(x,u) \\ \vdots \\ f_d(x,u) \\ r(x,u) \end{pmatrix} \in \mathbb{R}^{d+1} \tag{3.39}$$

并将终端奖励函数 $\bar{g} : \mathbb{R}^{d+1} \to \mathbb{R}$ 记为

$$\bar{g}(\bar{x}) := g(x) + x_{d+1}. \tag{3.40}$$

注意到 (3.39) 中的 $\bar{f}(\bar{x}, u)$ 不依赖于 x_{d+1}，并且有

$$\partial_{\bar{x}} \bar{f}(\bar{x}, u) = \begin{pmatrix} \partial_x f(x,u), & \partial_{x_{d+1}} f(x,u) \\ \partial_x r(x,u), & \partial_{x_{d+1}} r(x,u) \end{pmatrix} = \begin{pmatrix} \partial_x f(x,u), & 0 \\ \partial_x r(x,u), & 0 \end{pmatrix} \in \mathbb{R}^{(d+1)\times(d+1)}.$$

因此，我们的控制问题通过 (3.40) 中的终端奖励函数 $\bar{g}$ 化为一个等价的无运行奖励问题：

$$\min_{u \in \mathcal{U}_T} \ \bar{I}[u] := \bar{g}(\bar{x}(T)) = g(x(T)) + x_{d+1}(T) \tag{3.41}$$

并满足由 (3.36) 与 (3.37) 合并而成的 ODE 与初值约束：

$$\begin{cases} \dot{\bar{x}}(t) = \bar{f}(\bar{x}(t), u(t)), \quad \forall\, t \in [0, T], \\ \bar{x}(0) = \bar{x}_0. \end{cases} \tag{3.42}$$

将定理 3.1.12 应用于带有奖励 $\bar{I}$ 的控制问题 (3.41)，我们得到关于 control Hamiltonian

$$\bar{H}(\bar{x}, \bar{p}, u) = \bar{p} \cdot \bar{f}(\bar{x}, u)$$

的 Pontryagin 极大值原理。于是存在一个协态函数 $\bar{p}:[0,T]\to\mathbb{R}^{d+1}$，满足伴随方程

$$\begin{aligned}\Big(\dot{p}(t),\ \dot{p}_{d+1}(t)\Big) &= \dot{\bar{p}}(t)\\ &= -\partial_{\bar{x}}\bar{H}\Big(\bar{x}(t),\bar{p}(t),u(t)\Big)\\ &= -\bar{p}(t)\cdot\partial_{\bar{x}}\bar{f}\Big(\bar{x}(t),\bar{p}(t),u(t)\Big)\\ &= -\Big(p(t),\ p_{d+1}(t)\Big)\cdot\begin{pmatrix}\partial_x f(x(t),u(t)), & 0\\ \partial_x r(x(t),u(t)), & 0\end{pmatrix}\\ &= \Big(-p(t)\cdot\partial_x f(x(t),u(t)) - p_{d+1}\partial_x r(x(t),u(t)),\ 0\Big)\end{aligned} \tag{3.43}$$

以及终值

$$\bar{p}(T) = (p(T),\ p_{d+1}(T)) = (\nabla g(x(T)),\ 1). \tag{3.44}$$

由 (3.43) 与 (3.44) 可见，$p_{d+1}:[0,T]\to\mathbb{R}$ 满足终值问题

$$\begin{cases}\dot{p}_{d+1}(t)=0, & \forall\, t\in[0,T],\\ p_{d+1}(T)=1,\end{cases}$$

从而可得 $p_{d+1}(t)=1$ 对所有 $t\in[0,T]$ 都成立。

由 $p_{d+1}(t)=1$，我们有

$$\bar{H}(\bar{x},\bar{p},u) = \bar{p}\cdot\bar{f}(\bar{x},u) = p\cdot f(x,u)+r(x,u) = H(x,p,u), \tag{3.45}$$

这就是带运行奖励的 control Hamiltonian。同时，由 (3.43) 与 (3.44) 可知，伴随变量 p 满足

$$\begin{cases}\dot{p}(t) = -p(t)\cdot\partial_x f(x(t),u(t)) - \partial_x r(x(t),u(t)), & \forall\, t\in[0,T],\\ p(T)=\nabla g(x(T)),\end{cases}$$

其中上式中关于 p 的 ODE 正是采用 (3.45) 中定义的 control Hamiltonian H 后的 $\dot{p}(t) = -\partial_x H(x(t),p(t),u(t))$。总之，定理 3.1.12 中的 Pontryagin 极大值原理对于带运行奖励的最优控制问题 (3.17) 同样成立。

固定终点问题的 Pontryagin 极大值原理

现在我们考虑固定终点（固定到达状态）且终止时刻自由的最优控制问题。这类问题的一般形式为

$$\max_{u\in\mathcal{U}_T} \quad I[u] := \int_0^T r(x(t),u(t))\,dt, \tag{3.46a}$$

$$\text{s.t.} \quad \dot{x}(t) = f(x(t),u(t)), \tag{3.46b}$$

$$x(0) = x_0, \tag{3.46c}$$

$$x(T) = x_1, \tag{3.46d}$$

其中 $x_0, x_1 \in \mathbb{R}^d$ 是给定的初值和终值，而 T 是首次到达 x_1 的时间，并依赖于控制 u。

我们说明如下几点：(i) 对 u 的唯一要求是在任意时刻 t 都有 $u(t) \in \mathcal{U}$，但终端时刻 T 可以变化；(ii) (3.46a) 中的目标函数不再含有终端奖励；(iii) 通过适当设置运行奖励（例如在 $\mathbb{R}^d \times \mathcal{U}$ 上一致取 $r \le -\delta$，其中 $\delta > 0$ 为给定常数），可以假设首次到达时间 T 是有限的，使得延迟到达会减少最终累积的运行奖励。

为了推导固定终点最优控制问题 (3.46) 的 Pontryagin 极大值原理，我们需要引入如下更复杂的控制变分。

定义 3.1.13 (多重变分，multiple variations of a control)**.** 对任意 $K \in \mathbb{N}$，选取时间点 $0 < \tau_1 < \tau_2 < \cdots < \tau_K$、正数 $\lambda_1, \cdots, \lambda_K > 0$，以及控制变量 $w_1, w_2, \ldots, w_K \in \mathcal{U}$。对于足够小的 $\epsilon > 0$，使得区间 $[\tau_k - \lambda_k\epsilon, \tau_k]$（$k \in [K] := \{1, \ldots, K\}$）彼此不重叠，我们定义控制 $u : [0,T] \to \mathcal{U}$ 的**多重变分**为

$$u^\epsilon(t) := \begin{cases} w_k, & \text{if } \tau_k - \lambda_k\epsilon \le t < \tau_k \text{ for any } k \in [K], \\ u(t), & \text{otherwise.} \end{cases} \tag{3.47}$$

对于 (3.47) 中定义的多重变分 u^ϵ，令 x^ϵ 为如下初值问题的解

$$\begin{cases} \dot{x}^\epsilon(t) = f(x^\epsilon(t), u^\epsilon(t)) \quad \text{for any } t \ge 0, \\ x^\epsilon(0) = x_0. \end{cases} \tag{3.48}$$

为了简化后续表述，我们记矩阵值函数 $\Phi(t,s)$ 为如下问题的解

$$\begin{cases} \frac{d}{dt}\Phi(t,s) = \partial_x f(x(t),u(t)) \quad \text{for any } t \geq s \geq 0, \\ \Phi(s,s) = I_d. \end{cases} \tag{3.49}$$

我们还记

$$y_{\tau_k} = f(x(\tau_k), w_k) - f(x(\tau_k), u(\tau_k))$$

其中 $k = 1, \ldots, K$，这是根据 (3.47) 中定义的多重变分 u^ϵ 给出的。于是，沿着引理 3.1.9 的思路，很容易得到下面的引理。这里省略其证明。

引理 3.1.14. 设 x^ϵ 是 (3.48) 的解，而 x 是原控制 u 的解。则对任意有限的 $T > 0$，当 $\epsilon \to 0$ 时，有

$$\max_{0 \leq t \leq T} \frac{|x^\epsilon(t) - x(t) - \epsilon y(t)|}{\epsilon} \to 0$$

其中 y 是如下的**扰动函数**:

$$y(t) = \begin{cases} 0, & \text{if } 0 \leq t < \tau_k, \\ \sum_{k=1}^{k'} \lambda_k \Phi(t,\tau_k) y_{\tau_k}, & \text{if } \tau_k \leq t < \tau_{k+1},\ k' = 1, \ldots, K-1, \\ \sum_{k=1}^{K} \lambda_k \Phi(t,\tau_k) y_{\tau_k}, & \text{if } \tau_k \leq t. \end{cases}$$

定义 3.1.15 (Cone of variations)**.** 我们将时刻 t 处的**变分锥**定义为

$$C(t) := \left\{ \sum_{k=1}^{K} \lambda_k \Phi(t,\tau_k) y_{\tau_k} \ : \ \begin{array}{l} K \in \mathbb{N}, \text{ and } \lambda_k > 0, w_k \in \mathcal{U}, \\ 0 < \tau_1 \leq \cdots \leq \tau_K < t,\ \forall k \in [K] \end{array} \right\}. \tag{3.50}$$

注意，$C(t)$ 是 $\mathbb{R}^d$ 中的一个凸锥，因为对所有 k 都有 $\lambda_k \geq 0$，并且它由对控制 u 施加某种多重变分后在状态 $x(t)$ 上产生的所有到 ϵ 一阶（即 $O(\epsilon)$ 量级）的变化构成。

$C(t)$ 的性质对于建立固定终点问题 (3.46) 的 Pontryagin 极大值原理非常重要。为了研究 $C(t)$ 的性质，我们需要下面这个拓扑引理。

引理 3.1.16 (Zeros of a vector field)**.** 设 S 是 $\mathbb{R}^n$ 中一个紧（闭且有界）且凸的子集，q 是 S 的一个内点。若 $\phi : S \to \mathbb{R}^n$ 是一个连续向量场，并满足如下严格不等式:

$$|\phi(x) - x| < |q - x|, \quad \forall x \in \partial S. \tag{3.51}$$

则存在 $x^* \in S$，使得 $\phi(x^*) = q$。

证明. 我们先考虑一个简单情形，即 $S = \bar{B}_1(0) := \{x \in \mathbb{R}^n : |x| \le 1\}$，它是以 0 为中心、半径为 1 的闭单位球，而 $q = 0 \in \mathbb{R}^n$ 是 $\bar{B}_1(0)$ 的一个内点。此时，(3.51) 化为

$$|\phi(x) - x| < |x|, \quad \forall\, x \in \partial\bar{B}_1(0).$$

将上式两边平方，得到

$$|\phi(x)|^2 - 2\phi(x) \cdot x < 0,$$

这意味着当 $|x| = 1$ 时，对所有 x 都有 $\phi(x) \cdot x > \frac{1}{2}|\phi(x)|^2 \ge 0$。

由于 $\bar{B}_1(0)$ 与 $\partial\bar{B}_1(0)$ 都是 $\mathbb{R}^n$ 中的紧集，且 ϕ 连续，因此存在 $M, m > 0$ 使得

$$\phi(x) \cdot x \ge m \quad \text{和} \quad |\phi(x)| \le M$$

对所有 $x \in \bar{B}_1(0)$（包括其边界）成立。取 $\alpha \in (0, 2m/M)$ 并定义 $\psi(x) := x - \alpha\phi(x)$，则有

$$\begin{aligned} |\psi(x)|^2 = |x - \alpha\phi(x)|^2 &= |x|^2 - 2\alpha\phi(x) \cdot x + \alpha^2|\phi(x)|^2 \\ &\le |x|^2 - 2m\alpha + \alpha^2 M = |x|^2 - \alpha(2m - \alpha M) < |x|^2 \le 1 \end{aligned}$$

对所有 $x \in \bar{B}_1(0)$ 成立。由 Brouwer 不动点定理，存在 $x^* \in \bar{B}_1(0)$ 使得 $\psi(x^*) = x^*$。即 $x^* - \alpha\phi(x^*) = x^*$，或等价地 $\phi(x^*) = 0 = q$。

在一般情形下，我们总可以先通过平移使 S 的内点 q 移到 0，再以 $q = 0$ 为中心对 S 作径向伸缩，将 S 变换为 $\bar{B}_1(0)$。这两步操作的组合是一个同胚（即从 $\mathbb{R}^n$ 到 $\mathbb{R}^n$ 的连续映射且其逆映射也连续），从而就把一般情形化归为上面的简单情形 $S = \bar{B}_1(0)$。 □

现在考虑给定固定终点 $x_1 \in \mathbb{R}^d$ 的最优控制问题。仍令 $T > 0$ 为满足初值问题

$$\begin{cases} \dot{x}(t) = f(x(t), u(t)), & \forall\, t \ge 0 \\ x(0) = x_0 \end{cases}$$

的轨迹 x 首次到达终点 x_1 的时刻。则总奖励泛函为

$$I[u] = \int_0^T r(x(t), u(t))\, dt,$$

其中 $r:\mathbb{R}^d\times\mathcal{U}\to\mathbb{R}$ 是给定的运行奖励函数。

我们再次考虑满足如下初值问题的辅助函数 $x_{d+1}:[0,T]\to\mathbb{R}$

$$\begin{cases}\dot{x}_{d+1}(t)=r(x(t),u(t)), & \forall\, t\geq 0,\\ x_{d+1}(0)=0.\end{cases}$$

记

$$\bar{x}:=\begin{pmatrix}x\\ x_{d+1}\end{pmatrix}\in\mathbb{R}^{d+1},\quad \bar{x}_0:=\begin{pmatrix}x_0\\ 0\end{pmatrix}\in\mathbb{R}^{d+1},$$

并记

$$\bar{f}(\bar{x},u):=\begin{pmatrix}f(x,u)\\ r(x,u)\end{pmatrix}\in\mathbb{R}^{d+1},$$

以及终端奖励泛函 $\bar{g}(\bar{x}):=x_{d+1}$。那么，具有固定终点 $x_1\in\mathbb{R}^d$ 的最优控制问题 (3.46)（回忆一下，$x_0,x_1\in\mathbb{R}^d$ 分别是 $x:[0,T]\to\mathbb{R}^d$ 的给定起点和终点，而这里的 x_1 不是 $x(t)$ 的第一个分量；在本证明中我们不讨论 $x(t)$ 的各个单独分量）可以写成

$$\max_u \quad I[u]:=\bar{g}(\bar{x}(T))=x_{d+1}(T), \tag{3.52a}$$

$$\text{s.t.}\quad \begin{cases}\dot{\bar{x}}(t)=\bar{f}(\bar{x}(t),u(t)), & \forall\, t\in[0,T],\\ \bar{x}(0)=\bar{x}_0,\\ x(T)=x_1.\end{cases} \tag{3.52b}$$

其中 u 的容许集为 $\{u:[0,\infty)\to\mathcal{U}\}$。

为了为最优控制问题 (3.52) 建立 Pontryagin 极大值原理，我们令 u 表示最优控制，x 表示对应的最优轨迹。我们的第一个目标是构造 Pontryagin 极大值原理中的协态 p。为此，我们将使用上面定义的记号 $\bar{x}$、$\bar{f}$ 和 $\bar{g}$。具体地，我们按 (3.50) 定义此时的变分锥为

$$C:=C(T)=\left\{\sum_{k=1}^{K}\lambda_k\bar{\Phi}(T,\tau_k)\bar{y}_{\tau_k}\ :\ \begin{array}{l}K\in\mathbb{N},\ \text{and}\ \lambda_k\geq 0, w_k\in\mathcal{U},\\ 0<\tau_1\leq\cdots\leq\tau_K<T,\ \forall\, k\in[K]\end{array}\right\}. \tag{3.53}$$

其中

$$\bar{y}_{\tau_k}=\bar{f}(\bar{x}(\tau_k),w_k)-\bar{f}(\bar{x}(\tau_k),u(\tau_k)),$$

而 $\bar{\Phi}(t,s)$ 满足

$$\begin{cases} \frac{d}{dt}\bar{\Phi}(t,s) = \partial_{\bar{x}}\bar{f}(\bar{x}(t),u(t)), & \text{for any } t \geq s \geq 0 \\ \bar{\Phi}(s,s) = I_{d+1}. \end{cases}$$

下面的引理指出了 C 的一个关键性质。

引理 3.1.17 (变分锥的几何性质)**.** 单位向量 $e_{d+1} = (0,\ldots,0,1) \in \mathbb{R}^{d+1}$ 不是 (3.53) 中 C 的内点。

证明. 我们采用反证法。假设 e_{d+1} 是 C 的内点,则存在线性无关向量 $z_1,\ldots,z_{d+1} \in \mathbb{R}^{d+1}$ 以及正数 $\lambda_1,\ldots,\lambda_{d+1} > 0$，使得

$$e_{d+1} = \sum_{k=1}^{d+1} \lambda_k z_k,$$

其中

$$z_k = \bar{\Phi}(T,\tau_k)\bar{y}_{\tau_k}$$

对某些时间点 $0 < \tau_1 < \cdots < \tau_{d+1} < T$ 成立（由于 e_{d+1} 是 C 的内点，我们总可以安全地假设这些时间点互不相同，因为我们可以让 $\tau_1,\ldots,\tau_{d+1}$ 两两不同）。

现在令 $\eta > 0$ 并定义

$$S := \Big\{ \sum_{k=1}^{d+1} \lambda_k z_k : 0 \leq \lambda_k \leq \eta,\ \forall\, k \in [d+1] \Big\}.$$

由于 $z_1,\ldots,z_{d+1}$ 线性无关，我们知道 S 有内点。此外，S 是一个紧的凸多面体，其所有边长都与 η 成比例，并且有一个位于 $0 \in \mathbb{R}^{d+1}$ 的顶点。

由于 e_{d+1} 是 C 的内点，因此存在足够小的 $\mu > 0$ 使得 $q := \mu e_{d+1}$ 是 S 的内点。此外，存在 $\delta := \delta(\eta,\mu) > 0$，使得对所有 $\bar{y} \in \partial S$ 有 $|\bar{y} - q| > \delta$。

对任意小的 $\epsilon > 0$，定义 $\phi^\epsilon : S \to \mathbb{R}^{d+1}$ 为

$$\phi^\epsilon(z) := \frac{\bar{x}^\epsilon(T) - \bar{x}(T)}{\epsilon} \tag{3.54}$$

其中 z 是 S 中的任意点，且 $\bar{x}$ 是初值问题

$$\begin{cases} \dot{\bar{x}}(t) = \bar{f}(\bar{x}(t), u(t)), \quad \forall\, t \in [0, T], \\ \bar{x}(0) = \bar{x}_0, \end{cases}$$

的解，而 $\bar{x}^\epsilon$ 是初值问题

$$\begin{cases} \dot{\bar{x}}^\epsilon(t) = \bar{f}(\bar{x}^\epsilon(t), u^\epsilon(t)), \quad \forall\, t \in [0, T], \\ \bar{x}^\epsilon(0) = \bar{x}_0, \end{cases}$$

的解，且 u^ϵ 是控制 u 的多重变分，定义为

$$u^\epsilon(t) = \begin{cases} w_k, & \text{if } \tau_k - \lambda_k \epsilon \le t < \tau_k,\ k = 1, \ldots, K, \\ u(t), & \text{otherwise.} \end{cases}$$

由引理 3.1.14 可知，对任意有限 T，当 $\epsilon \to 0$ 时，

$$\max_{0 \le t \le T} \frac{|\bar{x}^\epsilon(t) - \bar{x}(t) - \epsilon \bar{y}(t)|}{\epsilon} \to 0$$

其中 $\bar{y}$ 是 $\bar{x}^\epsilon$ 与 $\bar{x}$ 之间的一阶扰动，它的定义与引理 3.1.14 中相同，只不过其中的 $\Phi(t, \tau_k)$ 与 y_{τ_k} 要替换成这里在 (3.54) 中决定 z 的对应对象。因此，

$$|\phi^\epsilon(z) - \bar{y}(T)| = \left| \frac{\bar{x}^\epsilon(T) - \bar{x}(T)}{\epsilon} - \bar{y}(T) \right| = \left| \frac{\bar{x}^\epsilon(T) - \bar{x}(T) - \epsilon \bar{y}(T)}{\epsilon} \right| \le |O(\epsilon)|.$$

我们可以取 ϵ 足够小，使得 $|O(\epsilon)| < \delta = \delta(\eta, \mu)$，从而有

$$|\phi^\epsilon(z) - \bar{y}(T)| \le |O(\epsilon)| < \delta = \delta(\eta, \mu) \le |\bar{y} - q|, \quad \forall\, \bar{y} \in \partial S.$$

由引理 3.1.16，存在 $z^* \in S$，使得 $\phi^\epsilon(z^*) = q = \mu e_{d+1}$。令 u^ϵ 是与该 z^* 对应的控制 u 的多重变分，x^ϵ 是沿着 u^ϵ 的轨迹。则 $\phi^\epsilon(z^*) = \frac{1}{\epsilon}(\bar{x}^\epsilon(T) - \bar{x}(T)) = \mu e_{d+1}$，即 $x^\epsilon(T) = x(T) = x_1$ 且 $x^\epsilon_{d+1}(T) - x_{d+1}(T) = \mu\epsilon > 0$。这意味着在时刻 T，$x^\epsilon(T) = x_1$，同时获得了更大的奖励：

$$\int_0^T r(x^\epsilon(t), u^\epsilon(t))\, dt = x^\epsilon_{d+1}(T) > x_{d+1}(T) = \int_0^T r(x(t), u(t))\, dt,$$

这与 u 和 x 的最优性矛盾。因此，e_{d+1} 不是 C 的内点。 □

引理 3.1.17 表明，e_{d+1} 要么位于凸锥 C 的边界上，要么在 C 之外。若 $e_{d+1} \in \partial C$，则称这是所谓的**异常问题**（abnormal problem），此时 Pontryagin 极大值原理 没有作用。若 $e_{d+1} \notin C$，则下面给出的原理成立。

定理 3.1.18 (固定重点问题的 Pontryagin 极大值原理)**.** *假设最优控制问题 (3.46) 不是异常的，且 u 是该问题的一个最优解，x 是与 u 对应的轨迹。则存在函数 $p : [0,T] \to \mathbb{R}^d$ 满足伴随方程，并且如定理 3.1.12 中所述的极大化原理对三元组 (x,p,u) 成立。*

证明. 当 (3.46) 不是异常问题时，有 $e_{d+1} \notin C$。注意 C 是闭的且凸的。由支撑超平面定理，存在 $v = (v_1, \ldots, v_{d+1}) \in \mathbb{R}^{d+1}$，使得 $v_{d+1} > 0$ 且对所有 $z \in C$ 都有 $v \cdot z \leq 0$。不失一般性，我们通过将 v 替换为 v/v_{d+1}，可以假设 $v_{d+1} = 1$。

设 $\bar{p} = (p, p_{d+1}) : [0,T] \to \mathbb{R}^{d+1}$ 满足终值问题

$$\begin{cases} \dot{\bar{p}}(t) = -\bar{p}(t)\partial_{\bar{x}}\bar{f}(\bar{x}(t), u(t)), \quad \forall\, t \in [0,T], \\ \bar{p}(T) = v. \end{cases} \tag{3.55}$$

因此，对所有 $t \in [0,T]$，有 $\dot{p}_{d+1}(t) = 0$，且 $p_{d+1}(T) = v_{d+1} = 1$，从而推出 $p_{d+1}(t) = 1$。于是，根据 (3.55)，p 满足的伴随方程

$$\begin{aligned} \dot{p}(t) &= -\bar{p}(t)\partial_x \bar{f}(\bar{x}(t), u(t)) \\ &= -p(t)\partial_x f(x(t), u(t)) - p_{d+1}(t)\partial_x r(x(t), u(t)) \\ &= -p(t)\partial_x f(x(t), u(t)) - \partial_x r(x(t), u(t)) \end{aligned}$$

在整个区间 $[0,T]$ 上成立。

对任意 $\tau \in [0,T)$ 和任意 $w \in \mathcal{U}$，定义

$$\bar{y}_\tau := \bar{f}(\bar{x}(\tau), w) - \bar{f}(\bar{x}(\tau), u(\tau))\bar{y}(t)$$

并定义 $\bar{y}(T) := \bar{\Phi}(T,\tau)\bar{y}_\tau$，它是如下初值问题的解

$$\begin{cases} \dot{\bar{y}}(t) = \partial_{\bar{x}}\bar{f}(\bar{x}(t), u(t)), \quad \forall\, t \in [\tau, T], \\ \bar{y}(\tau) = \bar{y}_\tau. \end{cases}$$

此外，由 (3.53) 中 C 的定义可知，$\bar{y}(T) \in C$。

现在注意到，由本证明开头给出的 C 的几何性质以及终端条件 $\bar{p}(T) = v$，

有

$$0 \geq v \cdot \bar{y}(T) = \bar{p}(T) \cdot \bar{y}(T). \tag{3.56}$$

此外，还有

$$\begin{aligned}\frac{d}{dt}(\bar{p}(t) \cdot \bar{y}(t)) &= \dot{\bar{p}}(t) \cdot \bar{y}(t) + \bar{p}(t) \cdot \dot{\bar{y}}(t) \\ &= -\bar{p}(t)\partial_{\bar{x}}\bar{f}(\bar{x}(t), u(t)) \cdot \bar{y}(t) + \bar{p}(t) \cdot \partial_{\bar{x}}\bar{f}(\bar{x}(t), u(t))\bar{y}(t) \\ &= 0,\end{aligned}$$

因此 $\bar{p}(t) \cdot \bar{y}(t)$ 关于 t 为常数，从而有

$$\bar{p}(T) \cdot \bar{y}(T) = \bar{p}(\tau) \cdot \bar{y}(\tau). \tag{3.57}$$

结合 (3.56) 与 (3.57)，可得 $0 \geq \bar{p}(\tau)\bar{y}(\tau) = \bar{p}(\tau)\bar{y}_\tau$，即

$$0 \geq \bar{p}(\tau) \cdot (\bar{f}(\bar{x}(\tau), w) - \bar{f}(\bar{x}(\tau), u(\tau)).$$

因此，

$$\begin{aligned}\bar{H}(\bar{x}(\tau), \bar{p}(\tau), w) &= \bar{p}(\tau) \cdot \bar{f}(\bar{x}(\tau), w) \\ &\leq \bar{p}(\tau) \cdot \bar{f}(\bar{x}(\tau), u(\tau)) \\ &= \bar{H}(\bar{x}(\tau), \bar{p}(\tau), u(\tau)).\end{aligned}$$

由于 $w \in \mathcal{U}$ 与 $\tau \in [0, T)$ 都是任意的，并且 $u(\tau) \in \mathcal{U}$，于是得到

$$\bar{H}(\bar{x}(\tau), \bar{p}(\tau), u(\tau)) = \max_{w \in \mathcal{U}} \bar{H}(\bar{x}(\tau), \bar{p}(\tau), w) \tag{3.58}$$

对每个 τ 都成立。注意，由 $\bar{H}$ 的定义，有

$$\bar{H}(\bar{x}, \bar{p}, u) = \bar{p} \cdot \bar{f}(\bar{x}, u) = p \cdot f(x, u) + r(x, u), \tag{3.59}$$

因为 $p_{d+1} = v_{d+1} = 1$。将 (3.59) 代入 (3.58)，即可得到固定终点最优控制问题 (3.52)，或等价地 (3.46) 的极大值原理。 □

3.1.4 Hamilton–Jacobi–Bellman 方程

在本节中，我们考虑一种不同的求解最优控制问题的方法。该方法基于动态规划（dynamic programming，DP）的思想，即将给定问题嵌入到一类更大的问题集合中，并一次性求解整个问题族。然后，该整体问题族的解可以直接推出原始问题的解。

回顾具有固定时间 T 且终点自由的最优控制问题，其形式为

$$\max_{u\in\mathcal{U}_T} \quad I[u] := \int_0^T r(z(s),u(s))\,ds + g(z(T)), \tag{3.60a}$$

$$\text{s.t.} \quad \begin{cases} \dot{z}(s) = f(z(s),u(s)), \quad \forall\, s\in[0,T], \\ z(0) = x, \end{cases} \tag{3.60b}$$

其中初始值 $x\in\mathbb{R}^d$ 已知。我们可以将该问题 (3.60) 嵌入到一个更一般的问题族中，其中初始值 $x\in\mathbb{R}^d$ 和起始时间 $t\in[0,T]$ 可以任意给定：

$$\max_{u} \quad I_{x,t}[u] := \int_t^T r(z(s),u(s))\,ds + g(z(T)), \tag{3.61a}$$

$$\text{s.t.} \quad \begin{cases} \dot{z}(s) = f(z(s),u(s)), \quad \forall\, s\in[t,T], \\ z(t) = x, \end{cases} \tag{3.61b}$$

其中 u 只需满足对每个 $s\in[t,T]$ 有 $u(s)\in\mathcal{U}$。于是，原始最优控制问题 (3.60) 就是 (3.61) 的一个特例，对应于给定的 x 且 $t=0$。

我们定义**值函数**（value function）$v:\mathbb{R}^d\times[0,T]\to\mathbb{R}$ 为

$$v(x,t) := \sup_u I_{x,t}[u]. \tag{3.62}$$

显然，对于所有 $x\in\mathbb{R}^d$，都有 $v(x,T)=g(x)$。

现在的问题是，是否可以通过构造关于 v 的问题并求解它来得到 $v(x,t)$，然后再利用得到的值函数 v 来确定最优控制 u。这正是动态规划的典型策略。

定理 3.1.19 (Hamilton–Jacobi–Bellman equation)**.** *假设 v 关于 (x,t) 是 C^1 的，则 v 满足如下非线性偏微分方程，称为* ***Hamilton–Jacobi–Bellman*** *(HJB) 方程：*

$$\partial_t v(x,t) + \max_{w\in\mathcal{U}} \Big\{ \nabla v(x,t)\cdot f(x,w) + r(x,w) \Big\} = 0 \tag{3.63}$$

并满足终端条件 $v(x,T)=g(x)$，对所有 $x\in\mathbb{R}^d$ 和 $t\in[0,T]$ 成立。

证明. 我们首先证明 (3.63) 在取上确界且不等式（小于等于 0）的形式下成立，然后在最优控制 u 存在的情况下说明其原始形式成立。

为此，我们沿用之前的记号，定义

$$\mathcal{U}_T := \{u:[0,T]\to\mathcal{U} \text{ measurable}\} \tag{3.64}$$

对于任意 $x\in\mathbb{R}^d$、$t\in[0,T)$、满足 $t+\epsilon\le T$ 的 $\epsilon>0$，以及任意 $w\in\mathcal{U}$，考虑在区间 $[t,t+\epsilon]$ 上的初值问题：

$$\begin{cases}\dot{z}(s)=f(z(s),w), \quad \forall\, s\in[t,t+\epsilon],\\ z(t)=x.\end{cases} \tag{3.65}$$

则在区间 $[t,t+\epsilon]$ 上获得的收益为 $\int_t^{t+\epsilon} r(z(s),w)ds$。记 $x(t+s)=z(t+s)$，则在区间 $[t+\epsilon,T]$ 上能够获得的最大收益为 $v(x(t+\epsilon),t+\epsilon)$，这是由值函数 v 的定义得到的。

另一方面，由于 $v(x,t)$ 是在 (x,t) 处的最大收益，其最优性意味着

$$v(x,t)=v(z(t),t)\ge\int_t^{t+\epsilon} r(z(s),w)\,ds+v(x(t+\epsilon),t+\epsilon) \tag{3.66}$$

对 (3.66) 进行整理并除以 ϵ，得到

$$\frac{v(x(t+\epsilon),t+\epsilon)-v(x,t)}{\epsilon}+\frac{1}{\epsilon}\int_t^{t+\epsilon} r(z(s),w)\,ds\le 0.$$

令 $\epsilon\to 0$，则有 $x(t+s)=z(t+s)\to z(t)=x$，因此

$$\partial_t v(x,t)+\nabla v(x,t)\cdot\dot{z}(t)+r(x,w)\le 0.$$

由于 $\dot{z}(t)=f(z(t),w)$，我们得到

$$\partial_t v(x,t)+\nabla v(x,t)\cdot f(x,w)+r(x,w)\le 0. \tag{3.67}$$

由于 (3.67) 对任意 $w\in\mathcal{U}$ 都成立，我们得到

$$\partial_t v(x,t)+\sup_{w\in\mathcal{U}}\Big\{\nabla v(x,t)\cdot f(x,w)+r(x,w)\Big\}\le 0. \tag{3.68}$$

接下来，我们说明该上确界实际上可以取到最大值，并且 (3.68) 中的不等式可以取等号。为此，假设从时间 t 开始始终使用最优控制 u，则有

$$v(x,t) = \int_t^{t+\epsilon} r(x(s),u(s))\,ds + v(x(t+\epsilon),t+\epsilon)$$

对上述等式整理并除以 ϵ，得到

$$\frac{v(x(t+\epsilon),t+\epsilon) - v(x,t)}{\epsilon} + \frac{1}{\epsilon}\int_t^{t+\epsilon} r(x(s),u(s))\,ds = 0.$$

令 $\epsilon \to 0$，得到

$$\partial_t v(x,t) + \nabla v(x,t)\cdot f(x(t),u(t)) + r(x(t),u(t)) = 0. \tag{3.69}$$

由于对每个 t 都有 $u(t) \in \mathcal{U}$，我们知道 (3.68) 中的上确界实际上可以取到最大值，并且等式成立，从而得到 HJB 方程 (3.63)。 □

现在我们来看如何利用 HJB 方程来求解最优控制。假设我们能够求解 HJB 方程并得到其解 v，则定义 $\pi : \mathbb{R}^d \times [0,T] \to \mathcal{U}$ 为

$$\pi(x,t) := \arg\max_{w\in\mathcal{U}}\{\nabla v(x,t)\cdot f(x,w) + r(x,w)\}. \tag{3.70}$$

此时有

$$\partial_t v(x,t) + \nabla v(x,t)\cdot f(x,\pi(x,t)) + r(x,\pi(x,t)) = 0. \tag{3.71}$$

对于任意 $x \in \mathbb{R}^d$ 和 $t \in [0,T)$，我们可以求解如下初值问题（假设 $\pi(\cdot,t)$ 足够光滑）得到 x^*：

$$\begin{cases} \dot{x}^*(s) = f(x^*(s),\pi(x^*(s),s)), \quad \forall\, s \in [t,T], \\ x^*(t) = x. \end{cases} \tag{3.72}$$

则可以断言，问题 (3.61) 的最优控制 u^* 满足

$$u^*(s) := \pi(x^*(s),s)$$

对所有 $s \in [t, T]$ 成立。为了说明该 u^* 确实是最优的，我们注意到

$$
\begin{aligned}
I_{x,t}[u^*] &= \int_t^T r(x^*(s), u^*(s))\, ds + g(x^*(T)) \\
&= -\int_t^T \Big(\partial_s v(x^*(s), s) + \partial_x v(x^*(s), s) \cdot f(x^*(s), u^*(s))\Big) ds + g(x^*(T)) \\
&= -\int_t^T \Big(\partial_s v(x^*(s), s) + \partial_x v(x^*(s), s) \cdot \dot{x}^*(s)\Big) ds + g(x^*(T)) \\
&= -\int_t^T \frac{d}{ds}\Big(v(x^*(s), s)\Big) ds + g(x^*(T)) \\
&= v(x^*(t), t) - v(x^*(T), T) + g(x^*(T)) \\
&= v(x, t) \\
&= \sup_u I_{x,t}[u],
\end{aligned}
$$

其中第二个等号来自 (3.71)，第三个来自 (3.72) 中的 ODE，第六个等号来自终端条件 $v(x, T) = g(x)$。这说明 u^* 是 (3.61) 的一个最优控制。

至此，我们可以看到，通过求解 HJB 方程 (3.63) 可以作为求解最优控制问题的一种替代方法，其步骤如上所述。

然而需要指出的是，HJB 方程 (3.63) 是关于 v 的一个非线性一阶偏微分方程，并且由于其中包含最大化运算，还需要在每个时间 t 上对 $\mathcal{U}$ 求解一个优化问题，使其更加复杂。在某些特殊情况下，最大值可以以闭式形式给出，从而可以消去 (3.63) 中的最大化运算。然而，即便没有该最大化运算，由于其高维性（空间维度为 d，对应变量 x），该偏微分方程在实际中仍然可能难以甚至无法求解。尽管如此，HJB 方程（从动态规划角度）与最优控制之间的紧密联系，为研究和求解控制问题提供了重要的理论分析工具。

3.2 最优概率密度控制理论

深度学习问题通常涉及海量的数据点或智能体。例如，在一个大型多智能体控制系统中，每个智能体的状态都可以表示为一个向量，其分量描述其位置、朝向、速度和加速度。在许多情况下，这些智能体可以被视为服从某些特定概率分布的样本。通常，这类学习问题被表述为使这些样本随时间演化，这等价于它们所服从的概率分布的演化，从而使某个目标函数在给定时间区间 $[0, T]$ 上被最小化或最大化。因此，这类问题可以表述为概率密度的最优控制。在本节中，我们将把第 3.1 节中经典最优控制的理论结果推广到最优概率控制的情

形。

设 Ω 是 $\mathbb{R}^d$ 的一个开子集，其中 d 是状态向量的维数（例如上文提到的某个智能体的位置、朝向和加速度的拼接后的总维度）。在本节的其余部分中，我们通常把一个样本称为一个智能体。设有 N 个智能体，其状态为 $\{x_i \in \Omega : i = 1, \ldots, N\}$，并且它们服从某个初始分布 p 于 Ω 上，即 $x_i \sim p$。在固定时间区间 $[0, T]$ 上，一个随时间演化的控制向量场 $u : \Omega \times [0, T] \to \mathbb{R}^d$ 的作用下，智能体状态按照

$$\dot{x}_i(t) = u(x_i(t), t), \quad i = 1, \ldots, N. \tag{3.73}$$

进行变化。每个智能体在 $[0, T]$ 上积累某些运行奖励，并在时刻 T 获得一个终端奖励。不同于经典最优控制，这些奖励**不仅依赖于该特定智能体本身，还依赖于所有智能体的整体行为**。下面我们给出若干运行奖励与终端奖励函数的例子。

我们用 $\rho(\cdot, t)$ 表示在每个时刻 t 时状态 $x_i(t)$ 的时间边缘分布，即 $x_i(t) \sim \rho(\cdot, t)$。则 (3.73) 对应于连续方程

$$\partial_t \rho(x, t) + \nabla \cdot \big(\rho(x, t) u(x, t)\big) = 0 \tag{3.74}$$

定义在 $\Omega \times [0, T]$ 上。为简化记号，我们将 $\rho(\cdot, t)$ 和 $u(\cdot, t)$ 分别记为 ρ_t 和 u_t，并且在后文中常常省略积分中的 (x)，除非为了说明清楚而必须写出。

我们定义 Ω 上概率密度函数的空间：

$$P = \Big\{ p : \Omega \to [0, \infty) : \int_\Omega p(x)\, dx = 1 \Big\}. \tag{3.75}$$

此外，对于给定的终端时间 $T > 0$，我们定义随时间演化的概率密度函数空间为：

$$P_T = \big\{ \rho : \Omega \times [0, T] \to \mathbb{R} : \rho(\cdot, t) \in P,\ \forall\, t \in [0, T] \big\},$$

我们假设在任意固定时刻的可容许控制向量场集合为

$$U = \big\{ w \in C(\bar{\Omega}; \mathbb{R}^d) : \mathrm{Lip}(w) \le M_U,\ (w \cdot n)|_{\partial\Omega} = 0 \big\}, \tag{3.76}$$

其中某个常数 $M_U > 0$ 已给定；也就是说，w 在 $\bar{\Omega}$ 上是 M_U-Lipschitz 连续的，并满足边界条件 $w(x) \cdot n(x) = 0$，这里 $n(x) \in \mathbb{R}^d$ 表示 $\partial\Omega$ 在点 x 处的外法向量。w 的这个边界条件保证总概率密度保持为 1。于是我们将随时间演化

的可容许控制向量场集合定义为

$$U_T = \big\{u : \bar{\Omega} \times [0,T] \to \mathbb{R}^d : u(\cdot,t) \in U,\ \forall\, t \in [0,T]\big\}. \tag{3.77}$$

注意，(3.77) 并不要求 $u(x,\cdot)$ 关于 t 是连续函数，因此允许 u 瞬时改变方向，从而使下面得到的结果适用于更广泛的情形。

现在，我们可以将最优概率控制问题写成如下的一般形式：

$$\max_{u\in U_T} \quad I[u] := \int_0^T R(\rho_t, u_t)\, dt + G(\rho_T), \tag{3.78a}$$

$$\text{s.t.} \quad \partial_t \rho_t + \nabla\cdot(\rho_t u_t) = 0, \quad \forall\,(t,x) \in [0,T]\times\Omega, \tag{3.78b}$$

$$\rho_0 = p, \quad \forall\, x \in \Omega, \tag{3.78c}$$

其中 I 是总奖励泛函，U_T 是可容许控制向量场集合，而 p 是给定的初始概率分布，即 $x_i(0) \sim p$。在 (3.78) 中，$R : P \times U \to \mathbb{R}$ 和 $G : P \to \mathbb{R}$ 分别是运行奖励泛函（running reward functional）和终端奖励泛函（terminal reward functional）。我们假设 $I^* := \max_{u\in U_T} I[u]$ 是有限的。

对于概率分布和控制，我们作出以下若干假设，以确保后续推导可以严格进行。

假设 3.2.1. 我们作如下假设：

(i) 对所有 $x \in \Omega$，$\rho(x,\cdot)$ 在 $[0,T]$ 上关于 t 可微。

(ii) 对所有 $t \in [0,T]$，$\rho(\cdot,t)u(\cdot,t)$ 在 Ω 上可微。

(iii) 对所有 $x \in \Omega$，$\rho(x,\cdot)u(x,\cdot)$ 在 $(0,T)$ 上处处有 Lebesgue 点。

(iv) 对所有 $w \in U$，$R(\cdot,w)$ 在 P 上关于 L^2 意义是 Fréchet 可微的。

(v) $G(\cdot)$ 在 P 上关于 L^2 意义是 Fréchet 可微的。

假设 3.2.1 的 (i) 和 (ii) 可以放宽为弱可微，只要 (3.74) 成立即可。假设 3.2.1 的 (iii) 保证了对所有 $t \in (0,T)$，有 $\frac{d}{dt}\int_0^t \rho(x,s)u(x,s)\,ds = \rho(x,t)u(x,t)$。这一点会在下面引理 3.2.7 的证明中的某一步被使用（我们会在证明中明确指出）。假设 3.2.1 的 (iv) 和 (v) 使得在任意 $p \in P$ 处，R 和 G 都有泛函导数（functional derivative），而这对于后续建立 control Hamiltonian 动力系统是必要的。注意，在实际应用中，运行奖励泛函和终端奖励泛函通常都满足假设 3.2.1 的 (iv) 和 (v)。

3.2.1 概率控制的极大值原理

现在，在本小节中，我们为具有任意初始值 $p \in P$ 的最优概率控制问题 (3.78) 建立一个极大值原理（maximum principle，MP）。作为第 3.1 节中经典最优控制 Pontryagin 极大值原理的对应，这里建立的 MP 是关于最优解 u 的一个逐时刻必要条件，它定义在无限维空间 P_T 与 U_T 上。为了发展这一 MP，我们首先引入伴随偏微分方程（PDE）和控制 Hamilton 泛函的定义。

定义 3.2.2 (Adjoint PDE)**.** 设 $u \in U_T$ 是一个控制向量场，$\rho \in P_T$ 是满足 (3.78b) 和初值条件 (3.78c) 的相应演化概率函数。定义如下演化 PDE:

$$\partial_t \phi_t + u_t \cdot \nabla \phi_t = -\frac{\delta}{\delta \rho_t} R(\rho_t, u_t) \tag{3.79}$$

定义在 $\Omega \times [0,T]$ 上。我们称 (3.79) 为与 (ρ, u) 相关联的**伴随 PDE**。若 $\phi : \Omega \times [0,T] \to \mathbb{R}$ 满足 (3.79) 且终端条件为 $\phi_T = \frac{\delta}{\delta \rho_T} G(\rho_T)$，则称 ϕ 为 (ρ, u) 的**伴随函数**。

在全文中，我们假设在控制问题 (3.78) 中，对任意 $w \in U$，$\frac{\delta}{\delta \rho} R(\cdot, w)$、$\frac{\delta}{\delta \rho} G(\cdot)$ 以及 u 都足够光滑，从而使得下述假设成立。

假设 3.2.3. 对于任意初始值 $p \in P$，与 (3.78) 对应的伴随函数 $\phi : \Omega \times [0,T] \to \mathbb{R}$ 存在。

注意，伴随函数 ϕ 具有空间变量 x 和时间变量 t，而伴随方程是一个演化 PDE。相比之下，在经典最优控制中，伴随函数仅是 t 的函数，而伴随方程是一个 ODE。

定义 3.2.4 (Hamiltonian functional)**.** 设 $p \in P$，$w \in U$，并令 $f : \Omega \to \mathbb{R}$ 为一个可微函数，我们定义 **Hamiltonian 泛函**为

$$H(p, f, w) = \langle p,\ w \cdot \nabla f \rangle + R(p, w). \tag{3.80}$$

其中 R 是 (3.78) 中的运行奖励泛函。

现在我们可以验证，由任意 $u \in U_T$ 所诱导的 (ρ, ϕ) 满足辛形式的控制 Hamilton 动力系统，如下命题所述。

命题 3.2.5 (Control Hamiltonian dynamics)**.** 设 $u \in U_T$ 是一个控制向量场，$\rho \in P_T$ 是相应的演化概率，ϕ 是 (ρ, u) 的伴随函数。则 (ρ, ϕ) 满足控制

Hamilton 动力系统:

$$\begin{cases} \partial_t \rho_t = \dfrac{\delta}{\delta \phi_t} H(\rho_t, \phi_t, u_t) = -\nabla \cdot (\rho_t u_t), \\ \partial_t \phi_t = -\dfrac{\delta}{\delta \rho_t} H(\rho_t, \phi_t, u_t) = -u_t \cdot \nabla \phi_t - \dfrac{\delta}{\delta \rho_t} R(\rho_t, u_t) \end{cases} \tag{3.81}$$

对所有 t 都成立。

利用 (3.74)、(3.79) 和 (3.80)，动力系统 (3.81) 很容易验证，因此我们省略命题 3.2.5 的证明。

在下文中，我们记 u^* 为 (3.78) 的一个最优控制向量场，ρ^* 为其对应的概率分布。然后我们如下定义 u^* 的一个针状变分 u^ϵ：对于任意时刻 $\tau \in (0,T)$、$\epsilon \in (0,\tau)$，以及 $w \in U$，定义

$$u_t^\epsilon := \begin{cases} w, & \tau - \epsilon < t \le \tau, \\ u_t^*, & \text{otherwise}. \end{cases} \tag{3.82}$$

我们记由控制向量场 $u^\epsilon \in U_T$ 和初值 $\rho_0^\epsilon = p$ 所确定的演化概率为 $\rho_t^\epsilon \in P_T$。接下来，我们引入扰动函数，它描述了 ρ_t^ϵ 相对于 ρ_t 的一阶近似。

定义 3.2.6 (Perturbation function)**.** 对任意 $\tau \in (0,T)$，我们如下定义函数 $\sigma : [0,T] \times \Omega \to \mathbb{R}$：若 $t \in [0,\tau)$，则

$$\sigma_t = 0\,; \tag{3.83}$$

若 $t \in [\tau, T]$，则 σ 定义为如下初值问题的解：

$$\begin{cases} \partial_t \sigma_t = -\nabla \cdot \big(u_t^* \sigma_t\big), & \forall\, t > \tau, \\ \sigma_\tau = -\nabla \cdot \big(\rho_\tau^* (w - u_\tau^*)\big). \end{cases} \tag{3.84}$$

注意，对于固定的 x，$\sigma_t(x)$ 在 $t = \tau$ 处未必可微，并且 σ_t 依赖于 τ 和 w，但**不**依赖于 ϵ。在下面的引理中，我们证明 ρ_t^ϵ 相对于 ρ_t 的一阶近似正是 σ_t。

引理 3.2.7. *设 $\tau \in (0,T)$，$\epsilon \in (0,\tau)$，$w \in U$，并且 u_t^ϵ 由 (3.82) 定义。令 σ_t 为定义 3.2.6 中的扰动函数。则对每个 $t \in [0,T]$，都有*

$$\lim_{\epsilon \to 0} \frac{1}{\epsilon} \|\rho_t^\epsilon - \rho_t^* - \epsilon \sigma_t\|_2 = 0. \tag{3.85}$$

证明. 我们分三种情形来证明 (3.85)：$t \in [0,\tau)$、$t = \tau$ 和 $t \in (\tau, T]$。

对于任意 $t \in [0,\tau)$，取 $\epsilon \in (0,\tau-t)$，则有 $t<\tau-\epsilon$。由 (3.83)，我们知道 $\sigma_t=0$。由 (3.82)，对所有 $s<\tau-\epsilon$，有 $\rho_s^\epsilon=\rho_s^*$。因此 $\rho_t^\epsilon=\rho_t^*$。

对于 $t=\tau$,我们只需证明:由 (3.84) 中初值条件定义的 σ_τ 等于 $\lim_{\epsilon\to 0}\frac{\rho_\tau^\epsilon-\rho_\tau^*}{\epsilon}$，并且这一结论在 Ω 上几乎处处成立。为此，任取 $\psi\in H_0^1(\Omega)$，则有

$$
\begin{aligned}
\int_\Omega \Big(\lim_{\epsilon\to 0}\frac{\rho_\tau^\epsilon-\rho_\tau^*}{\epsilon}\Big)\psi\,dx &= \lim_{\epsilon\to 0}\frac{1}{\epsilon}\int_\Omega(\rho_\tau^\epsilon-\rho_\tau^*)\psi\,dx\\
&= \lim_{\epsilon\to 0}\frac{1}{\epsilon}\int_\Omega\Big(\int_{\tau-\epsilon}^{\tau}(\partial_s\rho_s^\epsilon-\partial_s\rho_s^*)\,ds\Big)\psi\,dx\\
&= \lim_{\epsilon\to 0}\frac{1}{\epsilon}\int_{\tau-\epsilon}^{\tau}\int_\Omega(w\rho_s^\epsilon-u_s^*\rho_s^*)\cdot\nabla\psi\,dxds \qquad (3.86)\\
&= \int_\Omega\Big(\lim_{\epsilon\to 0}\frac{1}{\epsilon}\int_{\tau-\epsilon}^{\tau}(w-u_s^*)\rho_s^*ds\Big)\cdot\nabla\psi\,dx\\
&= \int_\Omega(w-u_\tau^*)\rho_\tau^*\cdot\nabla\psi\,dx\\
&= -\int_\Omega\nabla\cdot\big((w-u_\tau^*)\rho_\tau^*\big)\psi\,dx\\
&= \int_\Omega\sigma_\tau\psi\,dx
\end{aligned}
$$

其中，(3.86) 的第一个等号使用了 Lebesgue 控制收敛定理；第二个等号来自 $\rho_{\tau-\epsilon}^\epsilon=\rho_{\tau-\epsilon}^*$ 以及对时间积分；第三个等号来自 ρ^ϵ 和 ρ^* 的连续方程、对 Ω 的分部积分，以及 $w,u_\tau^*\in U$（从而 $(w\cdot n)|_{\partial\Omega}=0$ 且 $(u_\tau^*\cdot n)|_{\partial\Omega}=0$）；第四个等号由 Lebesgue 控制收敛定理得到；第五个等号由假设 3.2.1 的 (iii) 得到；第六个等号来自分部积分以及 $w,u_\tau^*\in U$；最后一个等号来自 (3.84) 中的初值条件。

对于 $t>\tau$，定义 $\delta(t):=\frac{1}{2}\|\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t\|_2^2$，则有

$$
\begin{aligned}
\dot\delta(t) &= \int_\Omega(\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t)\;\partial_t(\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t)\,dx\\
&= -\int_\Omega(\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t)\;\nabla\cdot\big(u_t^*(\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t)\big)\,dx\\
&= \int_\Omega u_t^*\cdot\nabla\Big(\frac{1}{2}|\rho_t^\epsilon-\rho_t^*-\epsilon\sigma_t|^2\Big)\,dx \qquad (3.87)\\
&\le dM_U\delta(t),
\end{aligned}
$$

其中第二个等号来自 ρ_t^ϵ、ρ_t^* 和 σ_t 在 $(\tau,T]$ 上满足的连续方程；第三个等号来自分部积分以及 $(u_t^*\cdot n)|_{\partial\Omega}=0$；最后一个不等式来自 $\mathrm{Lip}(u_t)\le M_U$，从而

$\|\nabla \cdot u_t^*\|_\infty \le dM_U$。由 Grönwall 不等式，我们有对所有 $t \in (\tau, T]$，

$$\delta(t) \le \delta(\tau) e^{dM_U(T-\tau)} \le \delta(\tau) e^{dM_U T}.$$

由于我们已经证明了 (3.85) 在 $t = \tau$ 时成立，即 $\delta(\tau) = o(\epsilon^2)$，从而得到 $\delta(t) = o(\epsilon^2)$，这就推出了对所有 $t \in (\tau, T]$，(3.85) 成立。 □

在下一个引理中，我们推导 I 的一阶变分，这对于利用 u^* 的最优性来证明极大值原理是至关重要的。

引理 3.2.8. *令 u^ϵ 如 (3.82) 所定义，并令 I 为 (3.78) 中对应于控制向量场 u^ϵ 的泛函，则*

$$\frac{d}{d\epsilon} I[u^\epsilon]\Big|_{\epsilon=0} = R(\rho_s^*, w) - R(\rho_s^*, u_s^*) + \left\langle \frac{\delta}{\delta \rho_T^*} G(\rho_T^*), \sigma_T \right\rangle + \int_0^T \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt$$

其中 σ_t 由定义 3.2.6 给出。

证明. 首先，我们有

$$I[u^\epsilon] = \int_0^{\tau-\epsilon} R(\rho_t^\epsilon, u_t^*)\, dt + \int_{\tau-\epsilon}^{\tau} R(\rho_t^\epsilon, w)\, dt + \int_\tau^T R(\rho_t^\epsilon, u_t^*)\, dt + G(\rho_T^\epsilon).$$

对 ϵ 求导，利用引理 3.2.7，并令 $\epsilon = 0$，可得

$$\begin{aligned}
\frac{d}{d\epsilon} I[u^\epsilon]\Big|_{\epsilon=0} &= \int_0^\tau \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt - R(\rho_\tau^*, u_\tau^*) + R(\rho_\tau^*, w) \\
&\quad + \int_\tau^T \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt + \left\langle \frac{\delta}{\delta \rho_T^*} G(\rho_T^*), \sigma_T \right\rangle \\
&= R(\rho_\tau^*, w) - R(\rho_\tau^*, u_\tau^*) + \int_0^T \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt + \left\langle \frac{\delta}{\delta \rho_T^*} G(\rho_T^*), \sigma_T \right\rangle,
\end{aligned}$$

从而完成证明。 □

现在我们可以建立极大值原理，它给出了任意最优控制向量场 u^* 的每个时刻满足的必要条件。

定理 3.2.9 (极大值原理)**.** *设 u^* 是 (3.78) 的一个最优解，其对应密度为 ρ^*，并设 ϕ^* 是 (ρ^*, u^*) 的伴随函数。则 (ρ^*, ϕ^*, u^*) 满足控制 Hamilton 动力系统 (3.81)，其中 $\rho_0^* = p$ 且 $\phi_T^* = \frac{\delta}{\delta \rho_T} G(\rho_T)$，并且对所有 $\tau \in [0, T]$，下式成立：*

$$H(\rho_\tau^*, \phi_\tau^*, u_\tau^*) \;=\; \max_{w \in U} H(\rho_\tau^*, \phi_\tau^*, w). \tag{3.88}$$

证明. 取任意 $\tau \in [0,T]$ 与 $w \in U$，并用该 τ 和 w 按照 (3.82) 定义 u^ϵ。注意到 u^* 是 (3.78) 的最优解，并应用引理 3.2.8，可得

$$0 \geq \frac{d}{d\epsilon} I[u^\epsilon]\big|_{\epsilon=0} = R(\rho_\tau^*, w) - R(\rho_\tau^*, u_\tau^*) + \left\langle \frac{\delta}{\delta \rho_T^*} G(\rho_T^*), \sigma_T \right\rangle + \int_0^T \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt. \tag{3.89}$$

另一方面，我们有

$$\langle \phi_T^*, \sigma_T \rangle = \langle \phi_\tau^*, \sigma_\tau \rangle + \int_\tau^T \Big(\langle \partial_t \phi_t^*, \sigma_t \rangle + \langle \phi_t^*, \partial_t \sigma_t \rangle \Big) dt. \tag{3.90}$$

将 (3.79) 中关于 (ρ^*, u^*) 的伴随 PDE 以及 σ 在 (3.84) 中满足的条件代入 (3.90)，并进行分部积分，得到

$$\begin{aligned} \langle \phi_T^*, \sigma_T \rangle &= \langle \phi_\tau^*, \sigma_\tau \rangle - \int_\tau^T \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt \\ &= \langle \phi_\tau^*, \sigma_\tau \rangle - \int_0^T \left\langle \frac{\delta}{\delta \rho_t^*} R(\rho_t^*, u_t^*), \sigma_t \right\rangle dt, \end{aligned} \tag{3.91}$$

其中第二个等号是因为对所有 $t \in [0,\tau)$，有 $\sigma_t = 0$。

结合 (3.89) 和 (3.91)，以及终端条件 $\phi_T^* = \frac{\delta}{\delta \rho_T^*} G(\rho_T^*)$，我们得到

$$0 \geq R(\rho_\tau^*, w) - R(\rho_\tau^*, u_\tau^*) + \langle \phi_\tau^*, \sigma_\tau \rangle. \tag{3.92}$$

另一方面，有

$$\begin{aligned} \langle \phi_\tau^*, \sigma_\tau \rangle &= \big\langle \phi_\tau^*, -\nabla \cdot \big(w - u_\tau^*) \rho_\tau^* \big) \big\rangle \\ &= \langle u_\tau^* \cdot \nabla \phi_\tau^*, \rho_\tau^* \rangle - \langle w \cdot \nabla \phi_\tau^*, \rho_\tau^* \rangle, \end{aligned} \tag{3.93}$$

其中第一个等号由 (3.84) 中初值条件给出的 σ_τ 得到，最后一个等号来自分部积分。结合 (3.92) 和 (3.93)，我们有

$$\begin{aligned} 0 &\geq \big(R(\rho_\tau^*, w) + \langle u_\tau^* \cdot \nabla \phi_\tau^*, \rho_\tau^* \rangle \big) - \big(R(\rho_\tau^*, u_\tau^*) + \langle w \cdot \nabla \phi_\tau^*, \rho_\tau^* \rangle \big) \\ &= H(\rho_\tau^*, \phi_\tau^*, w) - H(\rho_\tau^*, \phi_\tau^*, u_\tau^*). \end{aligned}$$

由于这对每个 $\tau \in [0,T]$ 以及任意 $w, u_\tau^* \in U$ 都成立，因此断言 (3.88) 得证。 □

例 3.2.10. 设 $T>0$ 任意固定，且 $\Omega=\mathbb{R}^d$。考虑控制问题 (3.78)，其中运行奖励为

$$R(\rho_t,u_t)=-\frac{1}{2}\int_\Omega |u_t(x)|^2\rho_t(x)\,dx \tag{3.94}$$

且终端奖励为

$$G(\rho_T)=-\frac{1}{2}\int_\Omega |x|^2\rho_T(x)\,dx. \tag{3.95}$$

我们可以取任意正的 $p\in P$ 作为 (3.78) 中的初始 ρ_0，也就是说，对所有 $x\in\Omega$，都有 $p(x)>0$。

我们也可以把终端奖励 (3.95) 取为 $G(\rho_T)=-\frac{1}{2}\int_\Omega |x-x^*|^2\rho_T(x)\,dx$，其中 x^* 任意；下面的计算是类似的。这里为简便起见，我们取 $x^*=0$。

由定义 3.2.4，Hamiltonian 泛函为

$$H(\rho_t,\phi_t,u_t)=\langle\rho_t,u_t\cdot\nabla\phi_t\rangle-\frac{1}{2}\int_\Omega |u_t|^2\rho_t\,dx.$$

由 (3.81)，我们得到 (ρ,ϕ) 的控制 Hamilton 动力系统为

$$\begin{cases}\partial_t\rho_t=-\nabla\cdot(\rho_t u_t),\\ \partial_t\phi_t=-u_t\cdot\nabla\phi_t+\frac{1}{2}|u_t|^2,\end{cases} \tag{3.96}$$

其中 ρ 的初值为

$$\rho_0=p \tag{3.97}$$

而 ϕ 的终值为

$$\phi_T=\frac{\delta}{\delta\rho_T}G(\rho_T)=-\frac{1}{2}|x|^2. \tag{3.98}$$

由极大值原理 (3.88)，有

$$\begin{aligned}H(\rho_t,\phi_t,u_t)&=\max_{w\in U}H(\rho_t,\phi_t,w)\\ &=\max_{w\in U}\Big\{\langle\rho_t,w\cdot\nabla\phi_t\rangle-\frac{1}{2}\int_\Omega |w|^2\rho_t\,dx\Big\}.\end{aligned} \tag{3.99}$$

对上面的极大化表达式做配方可得：极大化子 u_t 满足 $\rho_t(u_t-\nabla\phi_t)=0$。若进一步假设 $\rho_t>0$，则有

$$u_t=\nabla\phi_t. \tag{3.100}$$

将 (3.100) 代入控制 Hamilton 动力系统 (3.96)，并结合 (3.97) 与 (3.98)，可

知最优 ρ 满足初值问题

$$\begin{cases} \partial_t \rho_t + \nabla \cdot (\rho_t \nabla \phi_t) = 0, \quad \forall\, t \in [0, T], \\ \rho_0 = p, \end{cases} \tag{3.101}$$

而其对应的伴随函数 ϕ 满足终值问题

$$\begin{cases} \partial_t \phi_t + \frac{1}{2}|\nabla \phi_t|^2 = 0, \quad \forall\, t \in [0, T], \\ \phi_T = -\frac{1}{2}|x|^2. \end{cases} \tag{3.102}$$

我们可以验证，(3.102) 的解为

$$\phi_t(x) = \frac{|x|^2}{2(t - T - 1)}. \tag{3.103}$$

这说明最优控制向量场为

$$u_t(x) = \frac{x}{t - T - 1} \tag{3.104}$$

对所有 x 和 t 都成立。下一小节中，我们将继续这个例子，考察其值泛函与 HJB 方程。

3.2.2 概率控制的 HJB 方程

在本小节中，我们推导与最优概率控制问题 (3.78) 相关的 Hamilton–Jacobi–Bellman（HJB）方程。为此，我们需要定义价值泛函，它对应的是经典最优控制中价值函数。注意，这是一个定义在乘积空间 $P \times [0, T]$ 上的随时间演化的泛函。该价值泛函定义如下。

定义 3.2.11 (价值泛函)**.** 对任意 $p \in P$ 和 $t \in [0, T]$，定义随时间演化的泛函 V 为

$$V(p, t) = \sup_{u \in U_T} \left\{ \int_t^T R(\rho_s, u_s)\, ds + G(\rho_T) \right\} \tag{3.105}$$

其中 ρ_s 满足初值问题：

$$\begin{cases} \partial_s \rho_s + \nabla \cdot (u_s \rho_s) = 0, \quad \forall\, s \in [t, T], \\ \rho_t = p. \end{cases} \tag{3.106}$$

我们称 H 为 (3.78) 的**价值泛函**（value functional）。

注意，对于任意 t，$V(\cdot,t):P\to\mathbb{R}$ 是定义在 Ω 上概率密度函数的无限维空间上的泛函。相比之下，经典最优控制中的价值函数是定义在某个有限维欧几里得空间子集上的函数。

现在我们可以建立价值泛函 V 的 HJB 方程如下。

定理 3.2.12 (概率控制的 HJB 方程)**.** *若对所有 t，$V(\cdot,t)$ 在 P 上是连续 Fréchet 可微的，且对所有 $p\in P$，$V(p,\cdot)$ 在 $[0,T]$ 上属于 C^1，则 $V:[0,T]\times P\to\mathbb{R}$ 满足*

$$\partial_t V(p,t)+\max_{w\in U}\Big\{\Big\langle w\cdot\nabla\frac{\delta}{\delta p}V(p,t),p\Big\rangle+R(p,w)\Big\}=0, \tag{3.107}$$

并满足 Ω 上的终值条件 $V(\cdot,T)=G(\cdot)$。

证明. 由 (3.105) 和 (3.106) 可知，对任意 $p\in P$，有 $V(p,T)=G(p)$。现在取 $p\in P$，$t\in[0,T)$。对于任意 $\epsilon>0$，考虑区间 $[t,t+\epsilon]\subset[0,T)$ 上的常值控制 $w\in U$，其对应的演化 ρ_s 满足

$$\begin{cases}\partial_s\rho_s(x)+\nabla\cdot(w\rho_s)=0, & \forall\, s\in(t,t+\epsilon),\\ \rho_t=p.\end{cases}$$

由价值泛函的定义 (3.105)，有

$$V(p,t)\geq\int_t^{t+\epsilon}R(\rho_s,w)\,ds+V(\rho_{t+\epsilon},t+\epsilon).$$

将上式整理并除以 ϵ，得到

$$0\geq\frac{1}{\epsilon}\int_t^{t+\epsilon}R(\rho_s,w)\,ds+\frac{V(\rho_{t+\epsilon},t+\epsilon)-V(p,t)}{\epsilon}.$$

令 $\epsilon\to0$ 且注意 $\rho_t=p$，得到

$$0\geq R(p,w)+\partial_t V(p,t)+\Big\langle\frac{\delta}{\delta p}V(p,t),\partial_t\rho_t\Big\rangle.$$

注意到 $\partial_t\rho_t=-\nabla\cdot(w\rho_t)=-\nabla\cdot(wp)$，并利用分部积分，得到

$$0\geq R(p,w)+\partial_t V(p,t)+\Big\langle w\cdot\nabla\frac{\delta}{\delta p}V(p,t),p\Big\rangle.$$

由于 w 是任意选取的，可得

$$0 \geq \partial_t V(p,t) + \max_{w\in U}\Big\{\Big\langle w\cdot\nabla\frac{\delta}{\delta p}V(p,t), p\Big\rangle + R(p,w)\Big\}. \tag{3.108}$$

下面说明 (3.108) 中的不等式实际上为等式。为此，取任意 $(t,p)\in[0,T)\times P$，设 $u^*:[t,T]\times\Omega\to\mathbb{R}$ 与 $\rho^*:[t,T]\times\Omega\to\mathbb{R}$ 为在 (3.105) 中实现 $V(p,t)$ 的最优解。则对任意 $\epsilon>0$，由最优性及 $\rho_t^*=p$，有

$$V(\rho_t^*,t) = \int_t^{t+\epsilon} R(\rho_s^*,u_s^*)\,ds + V(\rho_{t+\epsilon}^*, t+\epsilon).$$

与之前相同，除以 ϵ 并令 $\epsilon\to 0$，再利用分部积分，得到

$$0 = \partial_t V(\rho_t^*,t) + \Big\langle u_t^*\cdot\nabla\frac{\delta}{\delta p}V(\rho_t^*,t), \rho_t^*\Big\rangle + R(\rho_t^*,u_t^*). \tag{3.109}$$

结合 (3.108) 与 (3.109)，并利用 $\rho_t^*=p$ 以及 $u_t^*\in U$，可知最大值可以达到，从而 (3.108) 中的不等式为等式，证明完成。 □

例 3.2.13. 我们继续考虑例 3.2.10，其运行奖励为 (3.94)，终端奖励为 (3.95)，且初值为任意 $\rho_0=p>0$。

由 (3.107) 可知，HJB 方程为

$$\partial_t V(p,t) + \max_{w\in U}\Big\{\Big\langle p, w\cdot\nabla\frac{\delta}{\delta p}V(p,t)\Big\rangle - \frac{1}{2}\int_\Omega |w|^2 p\,dx\Big\} = 0. \tag{3.110}$$

受 (3.99) 中配方技巧和最优解 (3.104) 的启发，我们尝试令 $w=\frac{x}{t-T-1}$ 作为 (3.110) 中的极大值点。通过做配方可得 $\nabla\frac{\delta}{\delta p}V(p,t)=w$，从而推出

$$V(p,t) = \frac{1}{2(t-T-1)}\int_\Omega |x|^2 p(x)\,dx. \tag{3.111}$$

可以验证，(3.111) 中的 $V(p,t)$ 确实满足 HJB 方程 (3.110)，且在所有时间点 t，都在 $w=\frac{x}{t-T-1}$ 取得最大值。此外，其终值为

$$V(p,T) = -\frac{1}{2}\int_\Omega |x|^2 p(x)\,dx = G(p). \tag{3.112}$$

注意到 (3.111) 还表明最优控制为

$$u_t(x) = \nabla\frac{\delta}{\delta\rho_t}V(\rho_t,t) = \frac{x}{t-T-1}, \tag{3.113}$$

这与 (3.104) 一致。

3.2.3 初值扰动的变分

在本小节中，我们考虑 $G(\rho_T)$ 关于初值 p 的 Fréchet 导数，其中 $\rho : \Omega \times [0,T] \to \mathbb{R}$ 满足连续方程 (3.74)，其初值为 $\rho_0 = p$。该结果揭示了伴随函数 ϕ 的一个重要性质。

假设智能体的初始状态 $\{x_i \in \Omega : i = 1, \ldots, N\}$ 发生扰动，使得初始概率 p 变为

$$p^\epsilon := p + \epsilon h + o(\epsilon) \in P, \tag{3.114}$$

其中某些函数 $h, o(\epsilon) : \Omega \to \mathbb{R}$ 满足

$$\int_\Omega h(x)\,dx = 0, \quad \int_\Omega o(\epsilon)\,dx = 0. \tag{3.115}$$

注意，(3.115) 是 $p^\epsilon \in P$ 的必要条件。令 ρ^ϵ 满足如下初值问题：

$$\begin{cases} \partial_t \rho_t^\epsilon + \nabla \cdot (\rho_t^\epsilon u) = 0, \quad \forall\, t \in [0,T], \\ \rho_0^\epsilon = p^\epsilon. \end{cases} \tag{3.116}$$

我们关注 (3.116) 中的 ρ_t^ϵ 与 (3.78b)–(3.78c) 中的 ρ_t 之间的差异，它们使用相同的控制向量场 $u \in U_T$，但初值分别为 p^ϵ 与 p。

引理 3.2.14. *设 ρ_t^ϵ 和 ρ_t 分别以 p^ϵ 和 p 为初值满足 (3.116)。则*

$$\lim_{\epsilon \to 0} \sup_{0 \le t \le T} \frac{1}{\epsilon} \|\rho_t^\epsilon - \rho_t - \epsilon s_t\|_2 = 0, \tag{3.117}$$

其中 s_t 满足初值问题：

$$\begin{cases} \partial_t s_t + \nabla \cdot (s_t u_t) = 0, \quad \forall\, t \in [0,T], \\ s_0 = h. \end{cases} \tag{3.118}$$

证明. 定义 $\delta(t) := \frac{1}{2}\|\rho_t^\epsilon - \rho_t - \epsilon s_t\|_2^2$。仿照 (3.87) 并将其中的 σ_t 替换为 s_t，可得对所有 $t \in [0,T]$ 都有 $\delta(t) \le \delta(0) e^{dM_U T}$。由于 $\delta(0) = \frac{1}{2}\|p^\epsilon - p\|_2^2 = o(\epsilon^2)$，因此有

$$\lim_{\epsilon \to 0} \sup_{0 \le t \le T} \frac{(2\delta(t))^{1/2}}{\epsilon} \le \lim_{\epsilon \to 0} \frac{(2\delta(0))^{1/2}}{\epsilon} e^{dM_U T/2} = 0,$$

这就证明了 (3.117)。 □

命题 3.2.15. 对任意 $T>0$、$u\in U_T$、初值 $p\in P$，设 ρ_t 是如下初值问题的解

$$\begin{cases}\partial_t\rho_t+\nabla\cdot(\rho_t u_t)=0, \quad \forall t\in[0,T],\\ \rho_0=p.\end{cases} \tag{3.119}$$

则对于任意在 P 上 *Fréchet* 可微的泛函 $G:P\to\mathbb{R}$，有

$$\frac{\delta}{\delta p}G(\rho_T)=\phi_0, \tag{3.120}$$

其中 $p=\rho_0$ 是 ρ 的初值，而 ϕ 满足终值问题：

$$\begin{cases}\partial_t\phi_t+u_t\cdot\nabla\phi_t=0, \quad \forall t\in[0,T],\\ \phi_T=\frac{\delta}{\delta\rho_T}G(\rho_T).\end{cases} \tag{3.121}$$

证明. 对任意满足 (3.115) 的 h，令 p^ϵ 为 (3.114) 中定义的初值，并令 ρ^ϵ 满足初值问题 (3.116)。由引理 3.2.14 可知

$$G(\rho_T^\epsilon)=G\big(\rho_T+\epsilon s_T+o(\epsilon)\big), \tag{3.122}$$

其中 s_t 是 (3.118) 的解。因此

$$\frac{d}{d\epsilon}G(\rho_T^\epsilon)\Big|_{\epsilon=0}=\Big\langle\frac{\delta}{\delta\rho_T}G(\rho_T),s_T\Big\rangle=\langle\phi_T,s_T\rangle, \tag{3.123}$$

其中第二个等号是由 (3.121) 中 ϕ_t 的终值条件得到的。

另一方面，注意到

$$\begin{aligned}\frac{d}{dt}\langle\phi_t,s_t\rangle&=\langle\partial_t\phi_t,s_t\rangle+\langle\phi_t,\partial_t s_t\rangle\\&=-\langle u_t\cdot\nabla\phi_t,s_t\rangle-\langle\phi_t,\nabla\cdot(s_t u_t)\rangle\\&=0,\end{aligned}$$

其中第二个等号使用了 (3.121) 中 ϕ 的 PDE 以及 (3.118) 中 s 的 PDE；第三个等号则来源于分部积分以及边界上 $(u_t\cdot n)|_{\partial\Omega}=0$。因而，$\langle\phi_t,s_t\rangle$ 关于 t 为常数，从而

$$\langle\phi_T,s_T\rangle=\langle\phi_0,s_0\rangle=\langle\phi_0,h\rangle. \tag{3.124}$$

结合 (3.123) 与 (3.124)，并注意到 $\frac{\delta}{\delta p}G(\rho_T)=\frac{d}{d\epsilon}G(\rho_T^\epsilon)|_{\epsilon=0}$，可得

$$\left\langle \frac{\delta}{\delta p}G(\rho_T), h\right\rangle = \langle \phi_0, h\rangle. \tag{3.125}$$

由于 h 是任意的，所以 (3.120) 成立。 □

例 3.2.16. 令 $T>0$ 和 $p\in P$ 任意给定，并取 $\Omega=\mathbb{R}^d$。定义 $u\in U_T$ 为

$$u_t(x)=\frac{x}{2(t-T-1)} \tag{3.126}$$

对所有 $x\in\Omega$ 与 $t\in[0,T]$ 成立。令 ρ 为带有向量场 (3.126) 中 u 且初值为 p 的 (3.119) 的解。定义泛函 $G:P\to\mathbb{R}$ 为

$$G(q)=-\frac{1}{2}\int_\Omega |x|^2 q(x)\,dx, \quad \forall\, q\in P. \tag{3.127}$$

注意，对所有 q 都有 $\frac{\delta}{\delta q}G(q)=-\frac{1}{2}|x|^2$。

我们可以验证

$$\phi_t(x)=\frac{|x|^2}{2(t-T-1)} \tag{3.128}$$

是终值问题 (3.121) 的解，其中 u 由 (3.126) 给出，终值为 $\phi_T=\frac{\delta}{\delta\rho_T}G(\rho_T)=-\frac{1}{2}|x|^2$。

现在，由命题 3.2.15，我们断言

$$\frac{\delta}{\delta p}G(\rho_T)=\phi_0=-\frac{|x|^2}{2(T+1)}. \tag{3.129}$$

为了说明断言 (3.129)，先注意到

$$\rho_t(x)=\left(\frac{T+1}{T+1-t}\right)^{d/2} p\left(\left(\frac{T+1}{T+1-t}\right)^{1/2} x\right)$$

是带有 (3.126) 中向量场 u 且初值为给定 p 的初值问题 (3.119) 的解。因此

$$\rho_T(x)=(T+1)^{d/2}p\big(\sqrt{T+1}x\big).$$

于是我们得到

$$
\begin{aligned}
G(\rho_T) &= -\frac{1}{2}\int_\Omega |x|^2 \rho_T(x)\,dx \\
&= -\frac{1}{2}\int_\Omega |x|^2 (T+1)^{d/2} p\big(\sqrt{T+1}x\big)\,dx \\
&= -\frac{1}{2(T+1)}\int_\Omega |y|^2 p(y)\,dy,
\end{aligned}
$$

其中最后一个等号来自变量替换 $y=\sqrt{T+1}x$。因此 $\frac{\delta}{\delta p}G(\rho_T) = -\frac{|x|^2}{2(T+1)}$，从而证明了 (3.129)。

3.3 Neural ODE 方法

尽管我们在上文已经看到了许多关于最优控制的重要理论结果，但在实际中如何用数值方法求解一般控制问题仍然是一个问题。在本节中，我们介绍一种称为 **Neural Ordinary Differential Equation** 方法的途径来解决这一问题。下文中我们简称其为 Neural ODE 方法。

我们关注第 3.1 节中给出的如下具有固定终止时间 T 和自由终点的一般最优控制问题：

$$
\max_{u\in\mathcal{U}_T} \quad I[u] := g(x(T)), \tag{3.130a}
$$

$$
\text{s.t.} \quad \begin{cases} \dot{x}(t) = f(t, x(t), u(t)), \quad \forall\, t\in[0,T], \\ x(0) = x_0 \end{cases} \tag{3.130b}
$$

其中，终端奖励函数 $g:\mathbb{R}^d\to\mathbb{R}$、将控制作为自变量之一的定义函数 $f:[0,T]\times\mathbb{R}^d\times\mathcal{U}\to\mathbb{R}^d$，以及某个初值 $x_0\in\mathbb{R}^d$ 都是给定的。

我们之所以专注 (3.130)，是因为它涵盖了现实应用中大多数最优控制问题。这也部分出于一个实际问题：我们将使用深度神经网络来逼近控制 $u(t)$，而这种逼近有时难以精确满足某些约束（例如在固定终点最优控制问题中精确到达某个指定状态）。

我们还说明一下：为了便于后续表述，在 (3.130) 中时间变量 t 又重新作为显式输入出现在 f 中。与此同时，(3.130) 中没有运行奖励 r，因为我们可以像 (3.36) 那样简单地添加一个辅助变量来将运行奖励纳入考虑。

Neural ODE 方法的思想是将控制函数 u 参数化为一个深度神经网络 $u_\theta:[0,T]\to\mathcal{U}\subset\mathbb{R}^m$，其中 $\theta\in\mathbb{R}^n$ 表示由其 n 个网络参数组成的向量。

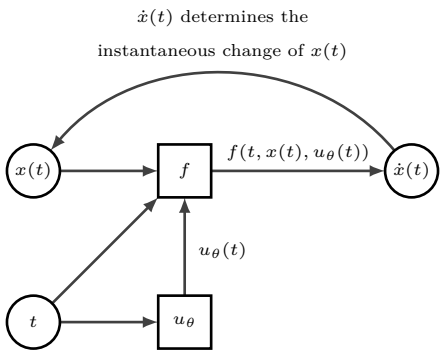


图 3.1: 由 Neural ODE 方法求解的控制系统。控制 ODE 为 $\dot{x}(t) = f(t, x(t), u(t))$，其中 $f : [0, T] \times \mathbb{R}^d \times \mathcal{U} \to \mathbb{R}^d$ 由控制问题决定，而控制 $u : [0, T] \to \mathcal{U}$ 被参数化为参数为 θ 的深度神经网络 u_θ。在原始的 Neural ODE 方法 [27] 中，函数 $f : [0, T] \times \mathbb{R}^d \to \mathbb{R}^d$ 被参数化为参数为 θ 的深度神经网络 f_θ，而 ODE 为 $\dot{x}(t) = f_\theta(t, x(t))$。

现在，最优控制问题 (3.130) 就化为

$$\max_{\theta \in \mathbb{R}^n} \quad J(\theta) := I[u_\theta] = g(x(T)), \tag{3.131a}$$

$$\text{s.t.} \quad \begin{cases} \dot{x}(t) = f(t, x(t), u_\theta(t)), \quad \forall\, t \in [0, T], \\ x(0) = x_0. \end{cases} \tag{3.131b}$$

注意，此时待优化的变量不再是时间相关的控制函数 $u : [0, T] \to \mathcal{U}$。相反，它变成了待优化的向量 $\theta \in \mathbb{R}^n$，就像标准优化中那样。一旦找到最优的 θ 值，神经网络 $u_\theta : [0, T] \to \mathcal{U}$ 就是控制问题 (3.130) 所期望的（近似）最优解 u。以控制函数 u_θ 为基础的控制系统如图 3.1 所示。

鉴于待优化的变量是向量 θ，自然可以采用我们在第 2.4 节中介绍的一些基于梯度的优化算法。使用此类算法时的关键要素，是目标函数 J 关于 θ 的梯度。因此，我们的主要目标是求出 (3.131) 中定义的函数 J 的 $\nabla_\theta J(\theta)$。如果我们能够在任意 θ 处计算 $\nabla_\theta J(\theta)$，那么就可以将其提供给任意基于梯度的优化算法，以寻找 $J(\theta)$ 的（局部）极大值点，它们对应于 u_θ 的（次）最优网络参数 θ。Neural ODE 方法正提供了一种计算 $\nabla_\theta J(\theta)$ 的途径。

为了说明 Neural ODE 方法如何工作，我们先引入两个简单的辅助函数。第一个 $\tau : [0, T] \to [0, T]$ 是恒等函数：

$$\tau(t) = t$$

对所有 $t \in [0,T]$ 都成立，因此它满足初值问题

$$\begin{cases} \dot{\tau}(t) = 1, \quad \forall\, t \in [0,T], \\ \tau(0) = 0. \end{cases} \tag{3.132}$$

第二个 $\sigma : [0,T] \to \mathbb{R}^n$ 是关于时间的常函数：

$$\sigma(t) = \theta$$

对所有 $t \in [0,T]$ 都成立，因此它满足初值问题

$$\begin{cases} \dot{\sigma}(t) = 0, \quad \forall\, t \in [0,T], \\ \sigma(0) = \theta. \end{cases} \tag{3.133}$$

进一步地，我们将增广函数 $z : [0,T] \to \mathbb{R}^{d+1+n}$ 记为

$$z(t) := \begin{pmatrix} x(t) \\ \tau(t) \\ \sigma(t) \end{pmatrix} \in \mathbb{R}^{d+1+n}, \quad \text{其初值为} \quad z(0) := \begin{pmatrix} x_0 \\ 0 \\ \theta \end{pmatrix} \in \mathbb{R}^{d+1+n}, \tag{3.134}$$

将 z 的增广 ODE 定义函数 h 记为

$$h(z(t)) := \begin{pmatrix} f(\tau(t), x(t), u_{\sigma(t)}(\tau(t)) \\ 1 \\ 0 \end{pmatrix} \in \mathbb{R}^{d+1+n}, \tag{3.135}$$

并将增广终端奖励函数 $\bar{g} : \mathbb{R}^{d+1+n} \to \mathbb{R}$ 记为

$$\bar{g}(z) := g(x)$$

对任意 $z = (x, \tau, \sigma) \in \mathbb{R}^{d+1+n}$ 成立。注意，我们知道 u_θ（由我们选择的神经网络结构给出）与 f（控制问题 (3.130) 的给定 ODE 定义函数）的表达式。此外，我们看到 ODE $\dot{z}(t) = h(z(t))$ 是自治的（h 不显式依赖于 t）。

现在我们可以将最优控制问题 (3.131) 改写成如下等价形式：

$$\max_{\theta\in\mathbb{R}^n} \quad J(\theta) := \bar{g}(z(T)), \tag{3.136a}$$

$$\text{s.t.} \quad \begin{cases} \dot{z}(t) = h(z(t)), \quad \forall\, t \in [0,T], \\ z(0) = z_0. \end{cases} \tag{3.136b}$$

对任意 $y_0 \in \mathbb{R}^{d+1+n}$ 和 $\epsilon > 0$，考虑 $z^\epsilon : [0,T] \to \mathbb{R}^{d+1+n}$ 满足初值问题

$$\begin{cases} \dot{z}^\epsilon(t) = h(z^\epsilon(t)), \quad \forall\, t \in [0,T], \\ z^\epsilon(0) = z_0 + \epsilon y_0. \end{cases}$$

由引理 3.1.6，当 $\epsilon \to 0$ 时，

$$z^\epsilon(t) = z(t) + \epsilon y(t) + o(\epsilon)$$

在 $[0,T]$ 上一致成立。也就是说，当 $\epsilon \to 0$ 时，

$$\sup_{0\le t\le T} \frac{|z^\epsilon(t) - z(t) - \epsilon y(t)|}{\epsilon} \to 0,$$

其中 $y : [0,T] \to \mathbb{R}^{d+1+n}$ 是如下初值问题的解

$$\begin{cases} \dot{y}(t) = \nabla_z h(z(t))y(t), \quad \forall\, t \in [0,T], \\ y(0) = y_0. \end{cases}$$

因此有

$$\frac{d}{d\epsilon}\bar{g}(z^\epsilon(T))\Big|_{\epsilon=0} = \nabla_z \bar{g}(z(T)) \cdot y(T).$$

此外，我们知道与状态 z 对应的协态 $p : [0,T] \to \mathbb{R}^{1\times(d+1+n)}$（$p(t)$ 是行向量）满足终值问题：

$$\begin{cases} \dot{p}(t) = -p(t)\nabla_z h(z(t)), \quad \forall\, t \in [0,T], \\ p(T) = \nabla_z \bar{g}(z(T)) \end{cases}$$

并且像之前一样，对所有 t 有

$$\begin{aligned}\frac{d}{dt}\Big(p(t)\cdot y(t)\Big) &= \dot{p}(t)\cdot y(t) + p(t)\cdot \dot{y}(t)\\ &= -p(t)\nabla_z h(z(t))\cdot y(t) + p(t)\cdot \nabla_z h(z(t))y(t)\\ &= 0.\end{aligned}$$

这意味着 $p(t)\cdot y(t)$ 关于 t 为常数。于是有

$$\frac{d}{d\epsilon}\bar{g}(z^\epsilon(T))\Big|_{\epsilon=0} = \nabla_z \bar{g}(z(T))\cdot y(T) = p(T)\cdot y(T) = p(0)\cdot y_0.$$

我们注意到 $p(t)$ 有三个分量块

$$p(t) = (p_x(t), p_\tau(t), p_\sigma(t)) \in \mathbb{R}^{1\times(d+1+n)}$$

并记

$$z := \begin{pmatrix} x \\ \tau \\ \sigma \end{pmatrix} \in \mathbb{R}^{d+1+n} \quad 和 \quad h(z) := \begin{pmatrix} f(\tau, x, u_\sigma(\tau)) \\ 1 \\ 0 \end{pmatrix} \in \mathbb{R}^{d+1+n}.$$

于是有

$$\partial_z h(z) = \begin{pmatrix} \partial_x f & \partial_\tau f + \partial_u f \partial_\tau u & \partial_u f \partial_\sigma u \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix} \in \mathbb{R}^{(d+1+n)\times(d+1+n)}.$$

注意，由于对所有 $t \in [0,T]$ 都有 $\tau(t) = t$ 和 $\sigma(t) = \theta$，因此 $\partial_\tau f = \partial_t f$、$\partial_\tau u = \partial_t u$，以及 $\partial_\sigma u = \partial_\theta u$。

因此，协态 $p = (p_x, p_\tau, p_\sigma) : [0,T] \to \mathbb{R}^{1\times(d+1+n)}$ 满足以如下终值问题：

$$\dot{p}(t) = -p(t)\nabla_z h(z(t)),$$

也即

$$\begin{cases} \dot{p}_x(t) = -p_x(t)\partial_x f(t, x(t), u_\theta(t)) \in \mathbb{R}^{1\times d} \\ \dot{p}_\tau(t) = -p_x(t)\Big(\partial_t f(t, x(t), u_\theta(t)) + \partial_u f(t, x(t), u_\theta(t))\partial_t u_\theta(t)\Big) \in \mathbb{R}^{1\times 1} \\ \dot{p}_\sigma(t) = -p_x(t)\Big(\partial_u f(t, x(t), u_\theta(t))\partial_\theta u_\theta(t)\Big) \in \mathbb{R}^{1\times n} \end{cases} \tag{3.137}$$

其终值为

$$p(T) = \bar{g}(z(T)) = g(x(T)),$$

也即

$$\begin{cases} p_x(T) = \nabla g(x(T)) \in \mathbb{R}^{1\times d}, \\ p_\tau(T) = 0 \in \mathbb{R}^{1\times 1}, \\ p_\sigma(T) = 0 \in \mathbb{R}^{1\times n}. \end{cases} \tag{3.138}$$

假设我们任意选取 $\eta \in \mathbb{R}^n$，并令 $y_0 = (0, 0, \eta^\top)^\top \in \mathbb{R}^{d+1+n}$，则上述推导给出

$$\nabla_\theta J(\theta) \cdot \eta = \frac{d}{d\epsilon} J(\theta + \epsilon\eta)\Big|_{\epsilon=0} = \frac{d}{d\epsilon}\bar{g}(z^\epsilon(T))\Big|_{\epsilon=0} = p(0) \cdot y_0 = p_\sigma(0) \cdot \eta.$$

由于 η 是任意的，因此我们知道

$$\nabla_\theta J(\theta) = p_\sigma(0). \tag{3.139}$$

现在我们看到，为了计算任意 θ 下的 $\nabla_\theta J(\theta)$，只需先求解初值问题 (3.131b) 得到 $x(T)$，再求解终值问题 (3.137)–(3.138) 得到 $p_\sigma(0)$。这样便得到 (3.139)。我们在算法 3.3.1 中总结了计算 $\nabla_\theta J(\theta)$ 的过程。注意，初值问题和终值问题都可以用成熟的数值 ODE 求解器来求解，例如中点（midpoint）方法和 Runge–Kutta 方法 [24]。由于 $\nabla_\theta J(\theta)$ 可以被计算出来，我们便可以采用任意基于梯度的优化算法来寻找 $J(\theta)$ 的一个（局部）极大值点。

关于 Neural ODE 方法的若干说明

关于使用 Neural ODE 方法求解最优控制问题，我们有如下几点说明。

首先，Neural ODE 方法将控制 $u : [0, T] \to \mathcal{U}$ 参数化为神经网络 u_θ。由于在实际中我们几乎总是将神经网络设计为时间上的连续函数，因此所得控制解在时间上也是连续的。问题在于，如果最优控制本身在时间上不是连续的，

Algorithm 3.3.1 在 Neural ODE 方法中计算 (3.131) 的 $\nabla_\theta J(\theta)$

Input: 当前的 $\theta \in \mathbb{R}^n$。

1: 通过如下初值问题求解 $x(T)$：

$$\begin{cases} \dot{x}(t) = f(t, x(t), u_\theta(t)), \quad \forall\, t \in [0, T], \\ x(0) = x_0. \end{cases}$$

2: 通过如下终值问题求解 $p(0) = (p_x(0), p_\tau(0), p_\sigma(0))$：

$$\begin{cases} \dot{p}(t) = -p(t)\nabla_z h(z(t)), \quad \forall\, t \in [0, T], \\ p(T) = (\nabla g(x(T)), 0, 0), \end{cases}$$

其中伴随方程的显式形式由 (3.137) 给出。

Output: $\nabla_\theta J(\theta) = p_\sigma(0)$.

那么我们无法精确找到它；但如果网络结构具有足够强的表达能力，仍有可能得到一个很好的逼近。与此同时，用深度网络来参数化控制的一个优点是，我们可以适当设计网络，使其既具有强表示能力，又能保证其输出始终落在 $\mathcal{U}$ 中（例如 $\mathcal{U}$ 是 $\mathbb{R}^m$ 中的有界集合），从而在学习控制时避免显式施加约束。关于神经网络设计的一些例子见第 1.5.3 节。

第二，Neural ODE 方法最初由 [27] 提出，作为残差网络（ResNet）结构的连续时间版本，它可以看作是对由深度网络参数化定义函数 f 的 ODE 使用 Euler 格式所得的结果。更具体地说，在 [27] 中，ODE 具有一种简单而灵活的形式，其中 f 直接被参数化为一个参数为 $\theta \in \mathbb{R}^n$ 的深度神经网络 $f_\theta : [0, T] \times \mathbb{R}^d \to \mathbb{R}^d$：

$$\begin{cases} \dot{x}(t) = f_\theta(t, x(t)), \quad \forall\, t \in [0, T], \\ x(0) = x_0. \end{cases} \tag{3.140}$$

与之对应的协态 p 的伴随方程也可以像上面那样推导出来，其形式比 (3.137) 更简单：

$$\begin{cases} \dot{p}_x(t) = -p_x(t)\partial_x f_\theta(t, x(t)) \in \mathbb{R}^{1\times d}, \\ \dot{p}_\tau(t) = -p_x(t)\partial_t f_\theta(t, x(t)) \in \mathbb{R}^{1\times 1}, \\ \dot{p}_\sigma(t) = -p_x(t)\partial_\theta f_\theta(t, x(t)) \in \mathbb{R}^{1\times n}, \end{cases} \tag{3.141}$$

其终值仍为 $p(t) = (\nabla g(x(T)), 0, 0) \in \mathbb{R}^{1\times(d+1+n)}$。于是我们可以将终端奖励 $g(x(T))$ 作为目标函数 $J(\theta)$，并且可以像算法 3.3.1 中那样计算 $\nabla_\theta J(\theta)$，只不

过此时需将其中的初值问题和终值问题分别替换为 (3.140) 和 (3.141)。

第三，Neural ODE 方法也可以被视为一种一般性的深度学习方法。例如，假设我们有一个标准监督学习问题，其给定数据集为 $\mathcal{D} = \{(x_{i,0}, x_i^*) \in \mathbb{R}^d \times \mathbb{R}^d : i \in [M]\}$，我们的目标是寻找映射 $F : \mathbb{R}^d \to \mathbb{R}^d$，使得对所有 $i \in [M]$ 都有 $F(x_{i,0}) \approx x_i^*$。与其直接将 F 参数化为一个深度神经网络（这可能较难训练），我们可以利用 Neural ODE 方法，将 ODE 的定义函数参数化为神经网络 f_θ，并把 F 设为从初始值 $x_i(0) := x_{i,0}$ 到其终值 $x_i(T)$ 的映射，即对所有 $i \in [M]\}$ 定义 $F(x_{i,0}) := x_i(T)$，再训练 θ 使得对所有 $i \in [M]$ 都有 $x_i(T) \approx x_i^*$。此时，网络训练问题可以表述为

$$\max_{\theta \in \mathbb{R}^n} \quad J(\theta) := -\frac{1}{2M} \sum_{i=1}^{M} |x_i(T) - x_i^*|^2, \tag{3.142a}$$

$$\text{s.t.} \quad \begin{cases} \dot{x}_i(t) = f_\theta(t, x_i(t)), & \forall\, t \in [0, T], \\ x_i(0) = x_{i,0}, & i = 1, \ldots, M. \end{cases} \tag{3.142b}$$

只要问题能够被表述为带有 ODE 约束的优化问题，我们都可以使用 Neural ODE 方法来求解。

第四，我们也可以利用 Neural ODE 方法来求解许多概率控制问题。其思想是，在任意时刻 t，用足够多的运动粒子（如智能体）$\{x_i(t) \in \mathbb{R}^d : i \in [M]\}$ 来表示定义在 $\mathbb{R}^d$ 上的概率密度函数 $\rho(\cdot, t)$。粒子数 M 可以人为设定，也可以由问题中可用的数据决定。当我们将向量场参数化为深度神经网络 $f_\theta : [0, T] \times \mathbb{R}^d \to \mathbb{R}^d$ 时，这些粒子将按照 ODE (3.142b) 运动。相应地，$\rho(\cdot, t)$ 的演化由如下连续方程给出：

$$\partial_t \rho(x, t) = -\nabla \cdot (\rho(x, t) f_\theta(t, x)). \tag{3.143}$$

因此，我们可以把粒子作为 ρ 的表示，并学习最优的 f_θ，使得 $\rho(\cdot, t)$ 在给定目标函数下达到最大化或最小化。关于这种方法的一个具体例子，见第 五 章中第 5.4 节。

第五，当我们将 Neural ODE 方法用于概率控制时，有时需要估计 $\rho(x, t)$ 的取值。例如，正如我们将在第 5.4 节中展示的那样，我们需要估计负熵

$$\int \rho(x, t) \log \rho(x, t) dx \approx \frac{1}{M} \log \rho(x_i(t), t).$$

其中右端的近似来自 Monte Carlo 积分（关于这种积分近似方法的细节，见

第 A.1 节），并且 $x_i(t) \sim \rho(\cdot, t)$。因此，我们需要利用这些运动粒子来估计 $\log \rho(x,t)$。为此，我们可以采用如下 ODE：

$$\frac{d}{dt} \log \rho(x(t), t) = -\nabla \cdot f_\theta(t, x(t)), \tag{3.144}$$

其中左侧的 d/dt 是关于 t 的全导数，也就是说，对 $\rho(x(t),t)$ 中的 $x(t)$ 与 t 都要对 t 求导，而 ∇ 与 $\nabla\cdot$ 分别表示关于 x 的梯度与散度。为了证明 (3.144)，我们注意到

$$\begin{aligned}
\frac{d}{dt}\rho(x(t),t) &= \partial_t \rho(x(t),t) + \nabla \rho(x(t),t) \cdot \dot{x}(t) \\
&= \partial_t \rho(x(t),t) + \nabla \rho(x(t),t) \cdot f_\theta(t, x(t)) \\
&= -\nabla \cdot (\rho(t,x) f_\theta(t,x)) + \nabla \rho(x(t),t) \cdot f_\theta(t, x(t)) \\
&= -\rho(x(t),t) \nabla \cdot f_\theta(t, x(t)),
\end{aligned}$$

其中第二个等号使用了 ODE $\dot{x}(t) = f_\theta(t, x(t))$，第三个等号使用了连续方程 (3.143)。假设 ρ 处处为正，将两边同时除以 $\rho(x(t),t)$，便得到 (3.144)。

注意，(3.143) 只提供了一种计算 $\log \rho(x(t),t)$ 变化量的方法。在实际中，我们未必知道初始分布 $\rho(\cdot, 0)$（或者某个给定 $T > 0$ 时刻的终端分布 $\rho(\cdot, T)$）。不过，这通常不会影响 θ 的学习，因为初始或终端分布往往是预先给定的，并且不依赖于 θ。因此，在建模过程中，我们通常可以将它从目标函数中消去，而不会影响网络训练过程中所求得的解 θ。

3.4 备注与参考文献

3.4.1 在 PDE 解算子学习中的应用

在本小节中，我们介绍深度最优控制在学习演化偏微分方程（evolution PDE）解算子中的一个应用。我们首先概述演化 PDE 以及解算子的定义。然后说明如何利用深度最优控制来学习解算子。在下面的讨论中，我们聚焦于关键组成部分和核心思想，并简要提及向一般情形推广的方法。

演化 PDE 是指描述系统如何随时间变化的一类 PDE，用于刻画诸如热传导、波动、流体等动态过程。设 Ω 是 $\mathbb{R}^d$ 的一个开子集，F 是定义某个演化 PDE 的微分算子，则该演化 PDE 可写为

$$\partial_t v(x,t) = F[v](x,t), \quad \forall\, (x,t) \in \Omega \times [0,T], \tag{3.145}$$

其中 $T > 0$ 是某个给定的终止时刻。

下面给出几个出现在广泛现实问题中的演化 PDE 示例（其中 $\gamma > 0$ 是某个给定常数）：

- 热方程：
$$\partial_t v(x,t) = F[v](x,t) := \Delta v(x,t). \tag{3.146}$$

- Hamilton–Jacobi 方程：
$$\partial_t v(x,t) = F[v](x,t) := \Delta v(x,t) + \frac{\gamma}{2}|\nabla v(x,t)|^2. \tag{3.147}$$

- Allen–Cahn 方程：
$$\partial_t v(x,t) = F[v](x,t) := \Delta v(x,t) - \gamma v(x,t)(v(x,t)^2 - 1). \tag{3.148}$$

在上述例子中，∇ 和 Δ 分别照常表示 u 关于 x 的梯度和 Laplace 算子，而 $v : \bar{\Omega} \times [0,T] \to \mathbb{R}$ 若满足相应 PDE，并满足某些初始条件和边界条件（我们稍后会详细说明），则称其为该方程的一个解。微分算子 F 是指将 v 映射为由 v 及其到某一阶的导数所确定的函数。上述例子中的微分算子 F 由于包含 Laplace 算子，因此都是二阶的。

我们说明一下，微分算子 F 可以是非线性的，例如 Hamilton–Jacobi 方程 (3.147) 和 Allen–Cahn 方程 (3.148) 中的情形。非线性往往会给求解这些 PDE 带来巨大挑战，因为这类 PDE 的解可能会随着时间变化而变得刚性很强且不稳定。

数值求解 PDE 的另一个主要挑战是维数。如果维数 d 很大（例如 $d \geq 5$），那么经典的数值 PDE 求解器，如有限差分法、有限元法和谱方法，这些在数值 PDE 文献中已经被深入研究的方法 [9]，往往会由于所谓的维数灾难而在计算上变得不可行。这是因为这些方法通常需要对区域 $\Omega \subset \mathbb{R}^d$（或某个变换域）做空间离散化，而离散化后未知变量的数量会随着 d 指数级增长，从而使得大维情形下的离散问题变得无法计算。

我们还注意到，在上面的例子 (3.146)、(3.147) 和 (3.148) 中，这些微分算子是**自治的**，因为它们不显式依赖于 t。相反，它们通过 $v(x,t)$ 隐式依赖于 t。不过，我们可以很容易地将稍后要介绍的方法推广到处理非自治的 F 所对应的 PDE，只需使用增广变量 $\bar{x} := (t,x)$ 并将 PDE 转化为自治形式即可。这种做法我们在本章见过，所以我们在此省略细节。

此外，还值得指出的是，我们将要介绍的方法也可以处理更高阶的演化

PDE。例如，对于具有二阶时间导数的演化 PDE

$$\partial_{tt}^2 v(x,t) = F[v](x,t), \tag{3.149}$$

我们可以将 (3.149) 转化为一个仅含一阶时间导数的等价 PDE：

$$\begin{cases} \partial_t v(x,t) = w(x,t), \\ \partial_t w(x,t) = F[v](x,t), \end{cases}$$

于是需要寻找满足上述两个 PDE 组成的系统的解 $(w,v):\bar{\Omega}\times[0,T]\to\mathbb{R}^2$。

为简化起见，下面我们假设 F 是自治的且 $\Omega=\mathbb{R}^d$。在这种情形下，我们可以不考虑边界条件。回忆一下，演化 PDE 往往需要某种初始条件来保证解的唯一性。例如，在 (3.146) 和 (3.147) 中，只要 v 是该演化 PDE 的一个解，那么对于任意常数 $C\in\mathbb{R}$，$v+C$ 也满足同一个 PDE。

初始条件由某个给定的初值函数 $g:\bar{\Omega}\to\mathbb{R}$ 决定。我们用 $v^g:\bar{\Omega}\times[0,T]\to\mathbb{R}$ 表示如下初值问题的解

$$\begin{cases} \partial_t v(x,t) = F[v](x,t), \quad \forall\,(x,t)\in\Omega\times[0,T], \\ v(x,0) = g(x), \quad x\in\bar{\Omega}. \end{cases} \tag{3.150}$$

然后，我们称由

$$S_F(g) := v^g. \tag{3.151}$$

定义的映射 $S_F : C^1(\bar{\Omega};\mathbb{R}) \to C^{2,1}(\bar{\Omega}\times[0,T];\mathbb{R})$ 为与 F 相关联的**解算子**（solution operator），它对应于问题 (3.150)，其中值域 $C^{2,1}(\bar{\Omega}\times[0,T];\mathbb{R})$ 是指定义在 $\bar{\Omega}\times[0,T]$ 上、在内部点关于 x 二次连续可导且关于 t 一次连续可导的实值函数集合。换言之，与 F 相关联的解算子 S_F，就是将任意初值 g 映射到其在 (3.150) 中对应解 v^g 的映射。我们的目标是发展一种方法，用于求得任意给定 F 的 S_F（或其近似）。

我们强调一下，学习解算子 S_F 的问题与给定 g 下求解 (3.150) 的问题在本质上是不同的。学好解算子 S_F 后，我们可以用很小的计算量得到任意 g 对应的 PDE 解 v^g。相比之下，求解 (3.150) 意味着在给定初值 g 的情况下采用某种方法（例如例 1.5.1 中的 PINN 方法），而当换成另一个初值 g 时就必须从头再来。如果我们需要许多不同初值所对应的解，这样的计算代价是很高的。当维数 d 很高时，拥有解算子 S_F 的好处就更加明显，因为这时即使只对单个 g 求解 (3.150) 也已经代价高昂。

现在我们说明，深度最优控制如何为任意给定 F 的解算子 S_F 提供一种逼近方法。为此，注意到如果我们将函数 $v:\mathbb{R}^d\to\mathbb{R}$ 参数化为一个参数为 $\theta\in\mathbb{R}^n$ 的深度神经网络 u_θ，那么我们实际上就将一个 n 维向量 θ 作为函数 v 的代表，而 v 本身是某个无限维函数空间中的点。如果 $v:\mathbb{R}^d\times[0,T]\to\mathbb{R}^d$ 随时间演化（如演化 PDE 的那些随时间 t 变化的解），那么我们就可以用参数轨迹 $\theta:[0,T]\to\mathbb{R}^n$ 来近似表示 v。也就是说，通过适当地引导（控制）$\theta(t)$，我们用 $u_{\theta(t)}(\cdot)$ 作为任意 $t\in[0,T]$ 时刻上 $v(\cdot,t)$ 的一个近似。

在这里，我们想暂停一下并提出如下问题：一个参数随时间演化的深度神经网络 $u_{\theta(t)}:\mathbb{R}^d\to\mathbb{R}$，是否与一个参数固定的深度神经网络 $\hat{u}_\eta:\mathbb{R}^d\times[0,T]\to\mathbb{R}$ 具有同样的表达能力？这里说 $u_{\theta(t)}$ 与 $\hat{u}_\eta$ 具有同样的表达能力，是指对任意 t 都有 $u_{\theta(t)}(\cdot)=\hat{u}_\eta(\cdot,t)$。如果答案是肯定的，那么我们就知道 $u_{\theta(t)}$ 是一个与 $\hat{u}_\eta$ 具有相同表示能力的函数逼近器，而后者根据万能逼近定理（定理 1.4.5）已知能够表示一大类函数（例如欧氏空间某个紧子集上的 $W^{k,\infty}$ 函数）。下面这个引理对此问题给出了肯定回答。

引理 3.4.1. *对于任意可微的前馈神经网络结构 $\hat{u}_\eta:\mathbb{R}^d\times[0,T]\to\mathbb{R}$，其定义为 $(x,t)\mapsto\hat{u}_\eta(x,t)$，参数为 η，都存在一个前馈神经网络 $u_{\theta(t)}:\mathbb{R}^d\to\mathbb{R}$，其定义为 $x\mapsto u_{\theta(t)}(x)$，并以 $\theta(t)$ 作为其在时刻 t 的参数，其中 $\theta(t)$ 关于时间可微，使得对所有 $x\in\mathbb{R}^d$ 和 $t\in\mathbb{R}$，都有 $u_{\theta(t)}(x)=\hat{u}_\eta(x,t)$。*

证明. 我们将前馈网络 $\hat{u}_\eta$ 记为

$$\hat{u}_\eta(x,t)=\hat{w}_L^\top\hat{h}_{L-1}(x,t)+\hat{b}_L$$

其中

$$\hat{h}_l(x,t)=\sigma(\hat{W}_l\hat{h}_{l-1}(x,t)+\hat{b}_l),\quad l=1,\ \ldots,\ L-1,$$

这里 σ 是可微激活函数，

$$\hat{h}_0(x,t)=\sigma(\hat{W}_0x+\hat{w}_0t+\hat{b}_0),$$

参数集合为

$$\eta=(\hat{w}_L,\ \hat{b}_L,\ \hat{W}_{L-1},\ \ldots,\ \hat{W}_0,\ \hat{w}_0,\ \hat{b}_0).$$

注意，η 的所有分量都是与 x 和 t 无关的常数。

类似地，我们将网络 $u_{\theta(t)}$ 记为

$$u_{\theta(t)}(x)=w_l(t)^\top h_{l-1}(x,t)+b_l(t)$$

其中

$$h_l(x,t)=\sigma(W_l(t)h_{l-1}(x,t)+b_l(t)),\quad l=1,\ \ldots,\ L-1,$$

其中 $h_0(x,t)=\sigma(W_0(t)x+b_0(t))$，且

$$\theta(t)=(w_L(t),\ b_L(t),\ W_{L-1}(t),\ \ldots,\ W_0(t),\ b_0(t)).$$

注意，$\theta(t)$ 是随时间演化的。因此，我们可以令对所有 t 都有 $b_0(t)=\hat{w}_0t+\hat{b}_0$，并令 $\theta(t)$ 的所有其他分量都是常数，且分别与 η 中对应分量相同，即

$$w_L(t)\equiv\hat{w}_L,\quad b_L(t)\equiv\hat{b}_L,\quad \ldots,\quad b_1(t)\equiv\hat{b}_1,\quad W_0(t)\equiv\hat{W}_0$$

对所有 t 成立。按此设置，我们不难验证，对所有 x 和 t，都有 $u_{\theta(t)}(x)=\hat{u}_\eta(x,t)$，并且 $\theta(t)$ 是可微的。 □

备注. 虽然我们在引理 3.4.1 中是针对前馈神经网络陈述的，但很容易看出该结果对一般神经网络同样成立。要看出这一点，只需注意，对于任意神经元 $\hat{y}(x,t)$，我们都可以像引理 3.4.1 中那样将其改写为 $y_{\eta(t)}(x)$。对于任意神经元 $z(t,x,\hat{y}(x,t))$，我们可以将其写成

$$\hat{z}(t,x,\hat{y}(x,t))=\hat{z}(t,x,y_{\eta(t)}(x))=z_{(\eta(t),\xi(t))}(x)$$

其中 $z_{\eta,\xi}$ 是某个以 x 为输入、参数 (η,ξ) 随 t 演化的神经网络。重复应用这一结果即可证明该结论。

现在我们看到，使用一个结构固定但参数随时间演化的深度神经网络 $u_{\theta(t)}$ 来逼近演化 PDE 的解 $v(\cdot,t)$ 是合理的。下一个问题是如何控制 $\theta(t)$，使得 $u_{\theta(t)}$ 能够表现得像初值问题 (3.150) 的解。为了满足 (3.150) 中的初始条件，我们可以直接设定

$$\begin{aligned}\theta(0)&=\arg\min_{\theta_0}\|u_{\theta_0}-g\|^2_{L^2(\Omega)}\\&=\arg\min_{\theta_0}\int|u_{\theta_0}(x)-g(x)|^2\,dx.\end{aligned}\tag{3.152}$$

注意，在实际中，(3.152) 中的积分通常通过 Monte Carlo 积分来逼近，特别是在高维情形下；而这个最小化问题通常较容易求解，因为我们只是在用一个神经网络去逼近已知函数 g。

为了满足 (3.150) 中的 PDE，我们用其近似 $u_{\theta(t)}(x)$ 替代解 $v(x,t)$，得到

$$\partial_t u_{\theta(t)}(x) = F[u_{\theta(t)}](x), \quad \forall\,(x,t) \in \Omega \times [0,T]. \tag{3.153}$$

注意，由于我们假设 F 是自治的，因此在 (3.153) 右侧的括号中省略了 t。另一方面，由链式法则可得

$$\partial_t u_{\theta(t)}(x) = \nabla_\theta u_{\theta(t)}(x) \cdot \dot{\theta}(t). \tag{3.154}$$

将 (3.153) 和 (3.154) 结合起来，我们看到，为了使 $u_{\theta(t)}$ 成为演化 PDE (3.150) 的解，参数 θ 应满足

$$\nabla_\theta u_{\theta(t)}(x) \cdot \dot{\theta}(t) = F[u_{\theta(t)}](x) \tag{3.155}$$

对所有 t 都成立。因此，应当以某种方式控制 $\theta(t)$，使得 (3.155) 两侧之差在每个 t 时刻都尽量小。

由于我们的目标是找到可作用于任意初始值 g 的解算子 S_F，我们需要一个控制向量场，它能够引导 $\dot{\theta}(t)$，使得从由 g 决定的任意初始 $\theta(0)$ 出发时，(3.155) 两边之间的差都能够被最小化。为此，我们收集一个由 M 个初始函数组成的集合 $\{g_i : \bar{\Omega} \to \mathbb{R} : i \in [M]\}$（理想情况下，它们应尽可能与实际应用中可能输入给 S_F 的那些初值相似）。然后我们像 (3.152) 那样为它们求得相应的初始参数 $\theta_{i,0}$：

$$\theta_i(0) = \underset{\theta_{i,0} \in \mathbb{R}^n}{\arg\min} \int |u_{\theta_{i,0}}(x) - g_i(x)|^2 \, dx. \tag{3.156}$$

我们注意到上面的问题可以对 i 并行求解。我们用 $\mathcal{D}_{\text{init}} = \{\theta_i(0) \in \mathbb{R}^n : i \in [M]\}$ 表示这个初值集合。接着，我们尝试寻找一个控制向量场 $h : \mathbb{R}^n \to \mathbb{R}^n$，它也被参数化为一个参数为 η 的深度神经网络 h_η，使得从 $\mathcal{D}_{\text{init}}$ 中任一初始值 $\theta_i(0)$ 出发时，(3.155) 总是（近似）成立。然后，我们通过求解如下最优控

制问题来训练这个控制向量场 h_η:

$$\max_{\eta} \quad J(\eta) := -\frac{1}{M}\sum_{i=1}^{M} r_i(T), \tag{3.157a}$$

$$\text{s.t.} \quad \begin{cases} \dot{\theta}_i(t) = h_\eta(\theta_i(t)), \quad \forall\, t \in [0,T],\ i \in [M], \\ \dot{r}_i(t) = \|\nabla_\theta u_{\theta_i(t)} \cdot h(\theta_i(t)) - F[u_{\theta_i(t)}]\|^2_{L^2(\Omega)}, \\ \theta_i(0) \in \mathcal{D}_{\text{init}}, \quad r_i(0) = 0, \quad i \in [M]. \end{cases} \tag{3.157b}$$

在 (3.157b) 中，$r_i(T)$ 表示由 $\theta_i(t)$ 在区间 $[0,T]$ 上生成的 (3.155) 两侧总平方误差。我们可以像第 3.3 节 Neural ODE 方法中那样构造 (3.157) 的伴随方程并计算 $\nabla_\eta J(\eta)$，再采用任意基于梯度的优化算法来训练代表控制向量场的神经网络 h_η。

一旦网络 h_η 被训练好，解算子 S_F 的实现方式如下。对于任意新的初始条件 $g : \bar{\Omega} \to \mathbb{R}$，我们首先根据 (3.152) 计算 $\theta(0)$。这一步简单直接，因为它只是将神经网络拟合到定义在 Ω 上的已知函数 g。然后，从该 $\theta(0)$ 出发，求解如下 ODE 得到 $\theta(t)$:

$$\dot{\theta}(t) = h_\eta(\theta(t)), \quad \forall\, t \in [0,T],$$

这个 ODE 可以用任意数值 ODE 求解器快速而准确地求解。接着，参数取为 $\theta(t)$ 的网络 $u_{\theta(t)}(\cdot)$ 就是演化 PDE (3.150) 在初值 g 下的解 $v^g(\cdot,t)$ 的一个近似。在实现 S_F 的这两步过程中，求 $\theta(0)$ 的计算成本占主导，但这仍远低于从头开始求解问题 (3.150) 的代价。

在实际中，有若干技巧可以提高控制向量场 h_η 的精度并加快其训练。例如，我们可以更谨慎地选择初值集合 $\{g_i : \bar{\Omega} \to \mathbb{R} : i \in [M]\}$ 来提升数据质量，使它们更贴近训练后的解算子 S_F 将会应用到的那些初值。我们还可以针对训练初值集合 $\mathcal{D}_{\text{init}}$ 中的每个初值分别求解对应的 $\theta_i(t)$，并生成多个时间点 $t_j \in (0,T]$ 上的样本点 $\theta_i(t_j)$。这些样本点可以构造成一个正则项，并加到 (3.157b) 中 $r_i(t)$ 的 ODE 上。更多细节可参见 [45,47]。

3.4.2 最优控制的参考文献

最优控制是应用数学与工程中的一门核心学科，并且在过去几十年中得到了深入研究。文献中有大量优秀著作涵盖了最优控制的理论与计算 [7,39,42,65,92,112]。注意，[39,92,112] 的原始版本都早在参考文献表和现代文献数据

库中所显示年份的几十年前就已出版。

从历史上看，最优控制源自变分法，而变分法可以追溯到数个世纪以前；不过，随着 20 世纪中期航空航天技术和计算技术在 1960 年代和 1970 年代的发展，它逐渐成熟为一个独立领域。早期工作将积分型代价泛函的极小化与控制函数的变分联系起来，形成了推广经典 Euler–Lagrange 方程的最优性必要条件 [126]。本章没有涉及随机最优控制，其中 ODE 被随机微分方程（SDE）替代，而奖励函数则以期望形式出现。其主要思想与确定性最优控制相似，但需要认真考虑扩散效应及相关分析。感兴趣的读者可参见 [39, 42] 了解更多细节。

经典文献中的一个奠基性结果是 1950 年代提出的 Pontryagin 极大值原理，它以 Hamilton 系统和伴随（协态）演化的形式给出了最优控制的必要条件。该原理的重要性在于，它将关于控制函数的无限维优化约化为逐点条件，从而简化了许多问题的分析与实现。与此同时，Bellman 动态规划原理提出了一种以价值函数和递归最优性为核心的替代框架，它构成了连续与离散最优控制以及后来的强化学习的基础（我们将在下一章介绍更多细节）。这两个基础性结果的核心思想和主要结论已在第 3.1 节中介绍，其中大部分内容遵循 [39]，并对证明细节做了一些扩展。这些结果向最优概率密度控制的推广见第 3.2 节，其内容遵循 [46]。作为残差网络连续时间变体的 Neural ODE 方法由 [27] 提出，事实证明它是求解标准最优控制以及许多其他一般控制和学习问题的有力工具，正如我们在第 3.3 节所解释的那样。我们还在第 3.4.1 节中参考 [45, 47] 介绍了其在学习 PDE 解算子中的应用，并将在第 5.4 节中进一步介绍其在生成建模中的概率密度控制方法。

第四章　深度强化学习

强化学习（reinforcement learning）研究的是序贯决策问题，其中智能体根据观测到的状态采取行动并与环境进行交互。其目标是设计一个最优策略，使得在时间过程中从环境中获得的期望累计奖励最大化。

本章重点讨论强化学习的两个核心方面。第一部分关注其数学基础。我们介绍强化学习系统的基本组成，并说明它们如何在 Markov 决策过程的框架下进行形式化。我们给出一系列核心理论结果，尤其是作为强化学习支柱的 Bellman 算子和 Bellman 方程，并提供严格的数学证明。

第二部分讨论强化学习算法，包括基于模型和无模型两类方法。基于模型的方法深度依赖于 Bellman 理论，我们介绍两种经典算法：策略迭代和价值迭代。鉴于现实世界强化学习问题的复杂性，我们进一步讨论一系列在实践上表现优异的无模型算法。在介绍过程中，我们说明如何结合深度学习来处理大规模和高维强化学习问题。

4.1　强化学习基础

4.1.1　强化学习中的基本概念

在强化学习中，**智能体**（agent）通常指能够在指定时间点改变状态、执行动作并从环境中接收反馈的主体。例如，智能体可以是机器人、汽车、无人机等。智能体的**状态**（state）通常指其位置、速度、方向等信息的组合。智能体执行的**动作**（action）可以是加速、刹车、机械臂旋转等。**环境**（environment）可以是围绕智能体的物理世界（包括人类），它根据智能体的状态和动作向其提供奖励，同时也决定智能体的状态转移。**奖励**（reward）通常表示为一个数值，智能体据此学习如何执行最优行为，以在时间过程中获得最大的总奖励。在强化学习中，我们通常采用离散时间设定，因为奖励和状态测量通常是在离散时间点进行的。

在本章中，我们用 x 表示智能体的状态，用 u 表示动作[1]，用 r 表示奖励。它们被表示为数或向量，以便进行数值计算。我们用 $\mathcal{X}$ 和 $\mathcal{U}$ 分别表示**状态空间**和**动作空间**。需要注意的是，$\mathcal{X}$ 和 $\mathcal{U}$ 可以是有限集合、可数集合，或者欧氏空间的子集。我们还给出如下平稳策略的定义，它描述了智能体在任意状态 x 下应采取的动作。

定义 4.1.1 (平稳策略，stationary policy)**.** 如果对于每个 $x \in \mathcal{X}$，$\pi(\cdot|x)$ 是定义在动作空间 $\mathcal{U}$ 上的与时间无关的概率分布，则称 π 为 $\mathcal{X}$ 上的一个**平稳策略**（或简称**策略**），即对于任意 $u \in \mathcal{U}$，有 $\pi(u|x) \geq 0$，并且

$$\int_{\mathcal{U}} \pi(u|x)\, du = 1.$$

如果 $\mathcal{U}$ 是离散集合，例如有限集合或可数集合，则积分应替换为对 $\mathcal{U}$ 的求和。我们用 Π 表示定义在 $\mathcal{X}$ 上的所有平稳策略的集合。

在实际应用中，也存在非平稳策略，即 $\pi_t(\cdot|x)$ 依赖于时间 t。然而，在大多数强化学习研究和应用中，通常采用平稳策略的设定，因此本章仅考虑平稳策略。

4.1.2 Markov 决策过程

当给定一个平稳策略 π 时，智能体可以在时刻 $t = 0$ 从状态空间 $\mathcal{X}$ 上某个初始状态分布 p_0 中抽取初始状态 X_0，并按照策略 π 执行动作 U_0。随后，环境根据 X_0 和 U_0 提供奖励 R_1，并将智能体转移到新状态 X_1。这一过程会随时间持续进行，最终形成如下轨迹：

$$X_0,\ U_0,\ R_1,\ X_1,\ U_1,\ R_2,\ \cdots,\ X_t,\ U_t,\ R_{t+1},\ X_{t+1},\ \cdots \tag{4.1}$$

注意，在 (4.1) 中我们使用大写字母，是因为在强化学习问题中，这些量一般都是随机的（除非 p_0 和 π 是确定性的，即它们只映射到特定值而不具有任何随机性）。

在强化学习的设定中，轨迹 (4.1) 被假设或构造成一个 **Markov 过程**。更准确地说，对于任意时刻 t，随机变量 $U_t, R_{t+1}, X_{t+1}, \cdots$ 仅依赖于 X_t 的取值，而不依赖于过去的随机变量 $X_0, \cdots, X_{t-1}, U_{t-1}, R_t$ 的取值。在这种情况下，(4.1) 被称为 **Markov 决策过程**（Markov decision process）。

[1]在强化学习文献中，状态和动作通常分别记为 s 和 a。这里我们使用 x 和 u 以体现它们与第 三 章中最优控制的状态和控制之间的对应关系。

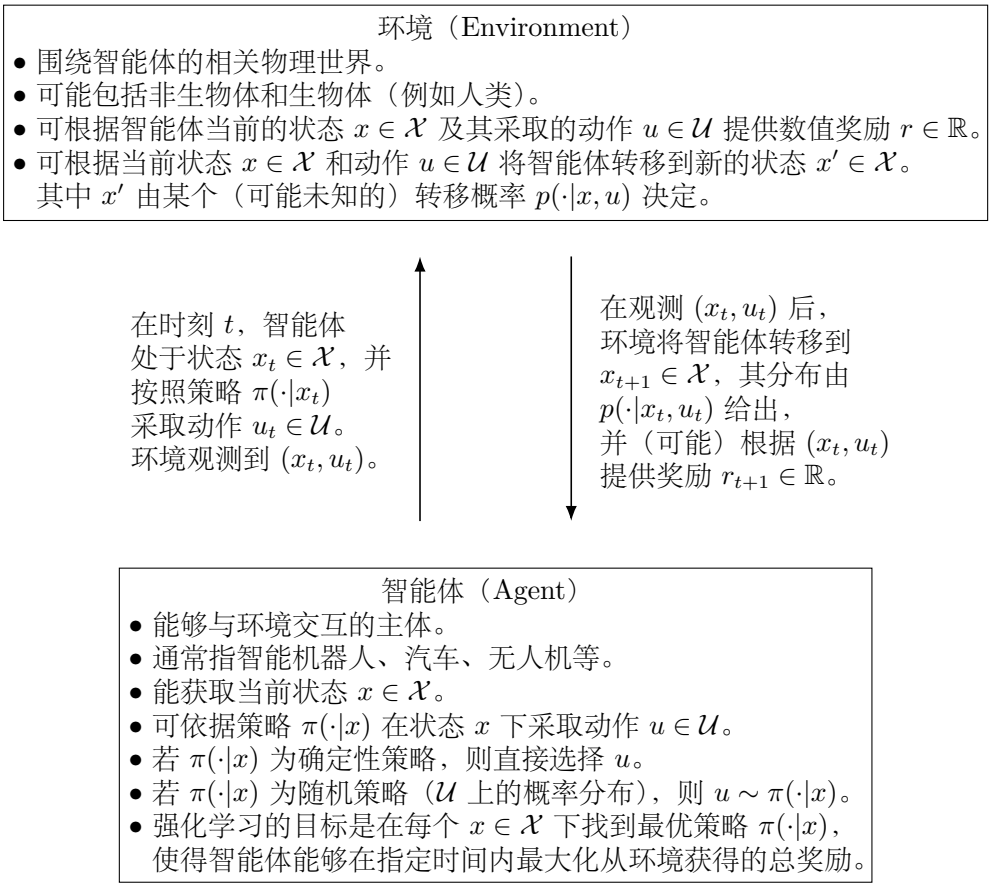


图 4.1: 典型的强化学习（RL）系统的结构。该系统由智能体和环境组成，其功能分别在对应方框中说明。智能体通过按照策略 π 采取动作与环境交互。在任意时刻 t，环境根据智能体当前状态 x_t 和其采取的动作 u_t，提供奖励 r_{t+1} 并将其转移到新状态 x_{t+1}。强化学习的目标是寻找一个最优策略 π，使智能体在时间过程中获得最大累计奖励。

从概率的角度来看，Markov 决策过程 (4.1) 由如下策略分布和状态转移概率分布所决定。对于任意 $x \in \mathcal{X}$，有

$$U_t \sim \pi(\cdot|X_t = x), \tag{4.2}$$

其中 $\pi \in \Pi$ 是一个平稳策略。对于任意 $x \in \mathcal{X}$ 和 $u \in \mathcal{U}$，下一状态 X_{t+1} 服从转移概率

$$X_{t+1} \sim p(\cdot|X_t = x, U_t = u), \tag{4.3}$$

注意，这个转移概率 $p(\cdot|x,u)$ 完全由环境决定，不受任何策略影响。当给定一个策略 π 时，在本章的推导中我们还会频繁使用如下状态到状态的转移概率

$$p^\pi(x'|x) := \int_{\mathcal{U}} \pi(u|x)p(x'|u,x)\,du \tag{4.4}$$

其中 $x' \in \mathcal{X}$ 是任意状态。

值得注意的是，如果给定一个策略 π，并且 (4.4) 中的状态到状态转移概率 p^π 已知，那么 Markov 决策过程 (4.1) 就退化为一个 **Markov 奖励过程**（Markov reward process）：

$$X_0,\ R_1,\ X_1,\ R_2,\ \cdots,\ X_t,\ R_{t+1},\ X_{t+1},\ \cdots \tag{4.5}$$

因为动作将由策略 π 自动给出，而状态转移由 p^π 决定。Markov 奖励过程 (4.5) 可以被观测到，但在这一过程中策略 π 不能改变。

在某些强化学习问题中，奖励 R_{t+1} 也是一个随机变量，它服从某个条件概率分布 $p_R(\cdot|X_t = x, U_t = u)$（同样，$p_R$ 由环境决定，并且不受任何策略影响）。然而，在大多数情况下，求解强化学习问题时只需要用到 R_{t+1} 的期望，因此为简化起见，我们直接设 $R_{t+1} = r(X_t, U_t)$，其中 $r(x,u): \mathcal{X} \times \mathcal{U} \to \mathbb{R}$ 是确定性的（并由环境给定），并且在给定 x 和 u 时可以被计算。

4.1.3 Bellman 算子与 Bellman 方程

有了上述所有概念之后，我们已经准备好给出强化学习问题的数学表述。为此，我们首先引入一个重要术语，称为状态价值函数（它可以看作最优控制中价值函数的对应物）。在强化学习中，状态价值函数将每个状态 $x \in \mathcal{X}$ 映射为从 $X_0 = x$ 出发的轨迹 (4.1) 过程中所获得折扣奖励总和的期望，如下定义所示。

定义 4.1.2 (状态价值函数，state value function)**.** 设 $\pi \in \Pi$ 是定义在状态空间 $\mathcal{X}$ 和动作空间 $\mathcal{U}$ 上的一个平稳策略，$\gamma \in (0,1)$ 是一个**折扣因子**（discount factor）。则**状态价值函数**（简称**价值函数**），记作 v^π，是一个从 $\mathcal{X}$ 到 $\mathbb{R}$ 的映射，定义为

$$v^\pi(x) = \mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \Big| X_0 = x\Big], \tag{4.6}$$

其中 $\{R_{t+1} : t \geq 0\}$ 是从 $X_0 = x$ 出发、并遵循策略分布 (4.2) 与状态转移概率 (4.3) 的 Markov 决策过程轨迹 (4.1) 中所获得的奖励。

关于定义 4.1.2 中引入的折扣因子 γ，我们有几点说明。这个折扣因子通常取值于 $(0,1)$，因为我们通常无法完全确定未来的情况。我们也不确定像 (4.1) 这样的过程会持续多久，而折扣因子 $\gamma \in (0,1)$ 能够在较弱条件下保证总奖励始终有界（例如，对所有 t，R_t 都小于某个常数上界 $M_R > 0$）。在实际应用中，我们使用折扣因子，是因为人类和动物的行为似乎更偏好即时奖励。在金融中，也经常考虑折扣因子，因为即时奖励可能比延迟奖励产生更多的利息和收益。最后但同样重要的是，折扣因子 $\gamma \in (0,1)$ 使我们更容易为强化学习构建强有力的数学理论和数值算法。

当然，我们也可以考虑 $\gamma = 1$ 的 Markov 决策过程下的强化学习。通常这是指存在一个终止状态 $\bar{x} \in \mathcal{X}$，使得智能体几乎必然（在某种概率意义下）在有限时间内到达 $\bar{x}$，并且之后一直停留在 $\bar{x}$，即对所有 $u \in \mathcal{U}$，都有 $p(\bar{x}|\bar{x}, u) = 1$。尽管如此，在本章中我们主要考虑带有折扣因子 $\gamma \in (0,1)$ 的这种常见情形。

现在我们注意到状态价值函数 v^π 在 (4.6) 中定义后具有一个重要性质：对于每个 $x \in \mathcal{X}$，有

$$\begin{aligned}
v^\pi(x) &= \mathbb{E}[R_1|X_0 = x] + \gamma\, \mathbb{E}\Big[\sum_{t=1}^{\infty} \gamma^{t-1} R_{t+1} \Big| X_0 = x\Big] \\
&= \int \pi(u_0|x) r(x, u_0)\, du_0 \\
&\quad + \gamma \iint \pi(u_0|x) p(x_1|x, u_0) \mathbb{E}\Big[\sum_{t=1}^{\infty} \gamma^{t-1} R_{t+1} \Big| X_1 = x_1\Big] dx_1 du_0 \qquad (4.7) \\
&= \int \pi(u_0|x) \Big[r(x, u_0) + \gamma \int p(x_1|x, u_0) v^\pi(x_1)\, dx_1\Big] du_0 \\
&= \mathbb{E}_{U_0 \sim \pi(\cdot|x)} \Big[r(x, U_0) + \gamma \mathbb{E}_{X_1 \sim p(\cdot|x, U_0)}[v^\pi(X_1)]\Big],
\end{aligned}$$

其中我们使用了策略分布和状态转移概率分布 (4.2) 与 (4.3)，以及

$$v^\pi(x_1) = \mathbb{E}\Big[\sum_{t=1}^{\infty} \gamma^{t-1} R_{t+1} \Big| X_1 = x_1\Big]$$

这基于定义 4.1.2（就好像 Markov 决策过程 (4.1) 是从 x_1 开始的一样）。方程 (4.7) 被称为 v^π 的 **Bellman 方程**。

到这里，我们已经看到任意策略 $\pi \in \Pi$ 都可以确定一个如 (4.1) 所示的 Markov 决策过程，以及定义 4.1.2 中的状态价值函数 $v^\pi : \mathcal{X} \to \mathbb{R}$。我们还注意到 (4.7) 给出的状态价值函数具有如下性质：v^π 等于对其自身施加某种运算后的结果，正如 (4.7) 右端所示。这表明该运算在强化学习中的重要性。为了更清楚地描述这一运算，我们先考虑 $\mathcal{X}$ 上所有一致有界函数所构成的集合：

$$B(\mathcal{X}) := \Big\{v : \mathcal{X} \to \mathbb{R} : \sup_{x\in\mathcal{X}} |v(x)| < \infty\Big\}. \tag{4.8}$$

容易看出，$B(\mathcal{X})$ 是一个实线性空间，即对于任意 $v_1, v_2 \in B(\mathcal{X})$ 和 $a_1, a_2 \in \mathbb{R}$，都有 $a_1 v_1 + a_2 v_2 \in B(\mathcal{X})$。此外，我们在 $B(\mathcal{X})$ 上定义 ∞-范数：对任意 $v \in B(\mathcal{X})$，$\|v\|_\infty := \sup_{x\in\mathcal{X}} |v(x)|$。于是很容易验证，带有该 ∞-范数的 $B(\mathcal{X})$ 是一个 Banach 空间（见附录 A.2）。现在我们可以正式给出 (4.7) 右端运算的定义了。

定义 4.1.3 (与给定策略对应的 Bellman 算子)**.** 设 $\pi \in \Pi$ 是 $\mathcal{X}$ 上的一个平稳策略。则**与 π 相关的 Bellman 算子** T^π 是从 $B(\mathcal{X})$ 到 $B(\mathcal{X})$ 的一个映射，记为 $T^\pi : v \mapsto T^\pi v$，其中对每个 $x \in \mathcal{X}$ 有

$$T^\pi v(x) := \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma \mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X')]\Big]. \tag{4.9}$$

注意，$T^\pi v(x)$ 应始终理解为 $(T^\pi v)(x)$。现在，借助 (4.9) 中 Bellman 算子 T^π 的定义，我们可以将 v^π 的 Bellman 方程 (4.7) 简洁地写成：

$$v^\pi = T^\pi v^\pi.$$

这表明 v^π 是 T^π 的一个不动点。关于 Banach 空间中的算子及其不动点，已有大量丰富的结果。特别地，如果一个算子对某个 $\gamma \in (0,1)$ 是 γ-压缩的，那么由不动点理论可知它具有许多重要性质（见附录 A.2）。接下来我们将证明，T^π 的确是 γ-压缩的。

命题 4.1.4 (T^π 是 γ-压缩的)**.** 对任意 $\pi \in \Pi$ 和 $\gamma \in (0,1)$，与 π 对应的

Bellman 算子 T^π 是一个 γ-压缩映射。也就是说，对任意 $v, \hat{v} \in B(\mathcal{X})$，有

$$\|T^\pi v - T^\pi \hat{v}\|_\infty \le \gamma \|v - \hat{v}\|_\infty. \tag{4.10}$$

证明. 对任意 $x \in \mathcal{X}$，有

$$\begin{aligned}
|T^\pi v(x) - T^\pi \hat{v}(x)| &= \Big| \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma \mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X')]\Big] \\
&\qquad - \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma \mathbb{E}_{X'\sim p(\cdot|x,U)}[\hat{v}(X')]\Big]\Big| \\
&= \Big|\gamma\, \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[\mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X') - \hat{v}(X')]\Big]\Big| \\
&= \gamma \Big| \iint \pi(u|x) p(x'|x,u)(v(x') - \hat{v}(x'))\, dx' du \Big| \\
&= \gamma \Big| \int p^\pi(x'|x)(v(x') - \hat{v}(x'))\, dx' \Big| \\
&\le \gamma \int p^\pi(x'|x) |v(x') - \hat{v}(x')|\, dx' \\
&\le \gamma \|v - \hat{v}\|_\infty \int p^\pi(x'|x)\, dx' \\
&= \gamma \|v - \hat{v}\|_\infty,
\end{aligned}$$

其中,第二个等号用到了期望的线性性;第四个等号中用了 $p^\pi(x'|x) = \int \pi(u|x)p(x'|x,u)du$;最后一个等号则使用了 $\int p^\pi(x'|x)\, dx' = 1$。由于 x 是任意的，所以命题 (4.10) 得证。 □

接下来，我们介绍定义在 $\mathcal{X}$ 上的另一个 Bellman 算子，它在推导强化学习中策略最优性条件时非常有用。注意，这个 Bellman 算子不需要事先给定任何策略。

定义 4.1.5 (Bellman 算子)**.** **$\mathcal{X}$ 上的 Bellman 算子** T^* 是一个从 $B(\mathcal{X})$ 到 $B(\mathcal{X})$ 的映射，记为 $T^* : v \mapsto T^* v$，其中对每个 $x \in \mathcal{X}$ 定义:

$$T^* v(x) := \sup_{\pi\in\Pi} \Big\{ \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma \mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X')]\Big]\Big\}. \tag{4.11}$$

我们说明一下: T^π 是 $B(\mathcal{X})$ 上的一个线性算子，而由于 (4.11) 中存在上确界，T^* 一般是非线性算子。现在我们希望证明，T^* 与 T^π 一样，也是 γ-压缩的。然而，由于其非线性，我们在证明前需要一个小引理。

引理 4.1.6. 设 A 是任意集合，$f, g : A \to \mathbb{R}$ 是定义在 A 上的两个实值函数。

则

$$\left|\sup_{a\in A} f(a) - \sup_{a\in A} g(a)\right| \le \sup_{a\in A} |f(a) - g(a)|. \tag{4.12}$$

证明. 首先注意到，对任意两个实值函数 $f_1, f_2 : A \to \mathbb{R}$，有

$$\sup_{a\in A}(f_1(a) + f_2(a)) \le \sup_{a\in A} f_1(a) + \sup_{a\in A} f_2(a). \tag{4.13}$$

这个不等式 (4.13) 很容易验证：对任意 $a \in A$，都有

$$f_1(a) + f_2(a) \le \sup_{a\in A} f_1(a) + \sup_{a\in A} f_2(a),$$

然后对左端关于 a 在 A 上取上确界，就得到 (4.13)。

现在证明 (4.12)。我们有

$$\begin{aligned}\sup_{a\in A} f(a) &= \sup_{a\in A}\Big((f(a) - g(a)) + g(a)\Big)\\ &\le \sup_{a\in A}(f(a) - g(a)) + \sup_{a\in A} g(a)\\ &\le \sup_{a\in A}|f(a) - g(a)| + \sup_{a\in A} g(a),\end{aligned}$$

其中第一个不等式使用了 (4.13)，因此有

$$\sup_{a\in A} f(a) - \sup_{a\in A} g(a) \le \sup_{a\in A}|f(a) - g(a)|.$$

类似地，也有

$$\sup_{a\in A} g(a) - \sup_{a\in A} f(a) \le \sup_{a\in A}|f(a) - g(a)|.$$

将前面这两个不等式结合起来，就得到 (4.12)。 □

现在我们可以证明 (4.11) 中定义的 T^* 也是一个 γ-压缩映射。

命题 4.1.7 (T^* 是 γ-压缩的)**.** *对任意 $\gamma \in (0,1)$，Bellman 算子 T^* 是一个 γ-压缩映射。也就是说，对任意 $v, \hat{v} \in B(\mathcal{X})$，有*

$$\|T^*v - T^*\hat{v}\|_\infty \le \gamma\|v - \hat{v}\|_\infty. \tag{4.14}$$

证明. 对任意 $x \in \mathcal{X}$，有

$$
\begin{aligned}
|T^* v(x) - T^* \hat{v}(x)| &= \Big| \sup_{\pi \in \Pi} \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[r(x,U) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,U)}[v(X')] \Big] \\
&\qquad - \sup_{\pi \in \Pi} \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[r(x,U) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,U)}[\hat{v}(X')] \Big] \Big| \\
&\le \sup_{\pi \in \Pi} \Big| \gamma \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[\mathbb{E}_{X' \sim p(\cdot|x,U)}[v(X') - \hat{v}(X')] \Big] \Big| \\
&= \gamma \sup_{\pi \in \Pi} \Big| \int p^\pi(x'|x)(v(x') - \hat{v}(x'))\, dx' \Big| \\
&\le \gamma \sup_{\pi \in \Pi} \int p^\pi(x'|x) |v(x') - \hat{v}(x')|\, dx' \\
&\le \gamma \|v - \hat{v}\|_\infty \sup_{\pi \in \Pi} \int p^\pi(x'|x)\, dx' \\
&= \gamma \|v - \hat{v}\|_\infty
\end{aligned}
$$

其中，第一个不等式使用了引理 4.1.6。由于 x 是任意的，因此命题 (4.14) 得证。 □

由于 T^π 和 T^* 都是 γ-压缩映射，Banach 不动点定理（定理 A.2.7）意味着它们各自都有唯一不动点，并且对应的不动点迭代从任意初始值出发都收敛到该不动点。下面的定理总结了这些结论。

定理 4.1.8 (Bellman 算子具有唯一不动点)**.** *关于折扣因子 $\gamma \in (0,1)$ 下定义在 $\mathcal{X}$ 上的 Bellman 算子，如下两条结论成立：*

- *对任意定义在 $\mathcal{X}$ 上的平稳策略 $\pi \in \Pi$，与 π 相关的 Bellman 算子 T^π 有唯一不动点 v^π。进一步地，对任意 $v \in B(\mathcal{X})$，设 $\{v_k : k = 0, 1, \dots\}$ 是由不动点迭代 $v_{k+1} = T^\pi v_k$、初值 $v_0 = v$ 所生成的序列，则当 $k \to \infty$ 时，有*

$$
\|(T^\pi)^k v - v^\pi\|_\infty = \|v_k - v^\pi\|_\infty \to 0. \tag{4.15}
$$

  且收敛速度是 γ-线性的。

- *Bellman 算子 T^* 有唯一不动点 v^*。进一步地，对任意 $v \in B(\mathcal{X})$，设 $\{v_k : k = 0, 1, \dots\}$ 是由不动点迭代 $v_{k+1} = T^\pi v_k$、初值 $v_0 = v$ 所生成的序列，则当 $k \to \infty$ 时，有*

$$
\|(T^*)^k v - v^*\|_\infty = \|v_k - v^*\|_\infty \to 0. \tag{4.16}
$$

且收敛速度是 γ-线性的。

证明. 由于 T^π 和 T^* 都是在 $B(\mathcal{X})$ 上的 γ-压缩映射，因此唯一不动点的存在性以及不动点迭代的收敛性都可由定理 A.2.7 直接推出。 □

定义 4.1.9 ($B(\mathcal{X})$ 中的偏序)**.** 设 $v, \hat{v} \in B(\mathcal{X})$。若

$$v \leq \hat{v} \tag{4.17}$$

对每个 $x \in \mathcal{X}$ 都有 $v(x) \leq \hat{v}(x)$，则称 $v \leq \hat{v}$。若同时有 $v \leq \hat{v}$ 且 $\hat{v} \leq v$，则称 $v = \hat{v}$。

现在我们有关于 Bellman 算子的三个重要单调性性质。这些结果在后文中会被频繁使用。

定理 4.1.10. 设 $v, \hat{v} \in B(\mathcal{X})$（它们不一定是 Π 中某个策略对应的价值函数），则以下结论成立：

(i) 设 $\pi \in \Pi$ 是任意一个平稳策略，T^π 是与之相关的 *Bellman* 算子，则

$$T^\pi v \leq T^* v. \tag{4.18}$$

(ii) 如果 $v \leq \hat{v}$，那么对任意平稳策略 $\pi \in \Pi$，与 π 相关的 *Bellman* 算子 T^π 满足

$$T^\pi v \leq T^\pi \hat{v}. \tag{4.19}$$

(iii) 如果 $v \leq \hat{v}$，那么 *Bellman* 算子 T^* 满足

$$T^* v \leq T^* \hat{v}. \tag{4.20}$$

证明. 任取 $x \in \mathcal{X}$。对 (i)，注意到

$$\begin{aligned}
T^\pi v(x) &= \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[r(x, U) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,U)} [v(X')] \Big] \\
&\leq \sup_{\pi \in \Pi} \mathbb{E}_{U \sim \pi(\cdot|x)} \Big[r(x, U) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,U)} [v(X')] \Big] \\
&= T^* v(x).
\end{aligned}$$

对 (ii)，有

$$\begin{aligned}T^\pi v(x) &= \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X')]\Big]\\ &\le \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\hat{v}(X')]\Big]\\ &= T^\pi\hat{v}(x)\end{aligned}$$

其中不等式使用了对每个 $x\in\mathcal{X}$ 都有 $v(x)\le\hat{v}(x)$。

类似地，对 (iii)，有

$$\begin{aligned}T^* v(x) &= \sup_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[v(X')]\Big]\\ &\le \sup_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\hat{v}(X')]\Big]\\ &= T^*\hat{v}(x),\end{aligned}$$

这里同样使用了对每个 $x\in\mathcal{X}$ 都有 $v(x)\le\hat{v}(x)$。

由于上面所取的 $x\in\mathcal{X}$ 是任意的，因此这三个结论都成立。 □

由于强化学习的目标是寻找最优策略（我们尚未给出其数学定义；它将在后面的定义 4.1.14 中给出），因此去考虑某种意义下“贪婪”的策略是很自然的。下面的定义给出了相对于 $B(\mathcal{X})$ 中一个函数的贪婪策略的具体含义。

定义 4.1.11 (贪婪策略，greedy policy)**.** 对任意 $v\in B(\mathcal{X})$，若 π 满足

$$T^\pi v = T^* v, \tag{4.21}$$

其中 T^π 是与 π 相关的 Bellman 算子，则称 π **相对于** v **是贪婪的**。注意，对某个 $v\in B(\mathcal{X})$，可能存在多个相对于它的贪婪策略。

一个重要问题是：对任意 $v\in B(\mathcal{X})$，是否总存在一个相对于 v 贪婪的平稳策略 π？由于平稳策略的 Markov 性质，这个问题的答案为“是”的充要条件是：对每个 $x\in\mathcal{X}$，都存在 $u(x)\in\mathcal{U}$（不要求唯一）满足

$$\begin{aligned}&r(x,u(x))+\gamma\mathbb{E}_{X'\sim p(\cdot|x,u(x))}[\hat{v}(X')]\\ &\qquad = \sup_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U)+\gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\hat{v}(X')]\Big].\end{aligned} \tag{4.22}$$

在这种情况下，我们可以构造一个策略 π，使得它在每个 x 处都映射到 $u(x)$（如果有多个 $u(x)$ 满足 (4.22)，我们任选其一即可）。那么根据定义 4.1.11，该

π 就是相对于 v 的贪婪策略。

我们可以看到，对每个 $x \in \mathcal{X}$，存在满足 (4.22) 的 $u(x)$，当且仅当上确界能够在动作空间 $\mathcal{U}$ 中取到。很容易看出，在以下两种情况下这会发生：(i) $\mathcal{U}$ 是有限集；或者 (ii) $\mathcal{U} \subset \mathbb{R}^m$ 是紧集，并且函数

$$r(x,\cdot) + \gamma \int p(x'|x,\cdot)v(x')\,dx' \ : \ \mathcal{U} \to \mathbb{R} \tag{4.23}$$

是连续的（因为定义在 $\mathbb{R}^m$ 中紧域上的连续函数一定能在该域上取到最大值）。条件 (i) 和 (ii) 都是充分条件，但并非必要条件。在实际设定中，对任意给定的 $v \in B(\mathcal{X})$，我们通常都能得到一个相对于它贪婪的策略。

定理 4.1.12. *设 v^* 是 T^* 的不动点。如果 π 相对于 v^* 是贪婪的，那么 $v^\pi = v^*$。*

证明. 由于 π 相对于 v^* 是贪婪的，根据定义 4.1.11，有

$$T^\pi v^* = T^* v^*. \tag{4.24}$$

另一方面，因为 v^* 是 T^* 的不动点，所以有 $T^* v^* = v^*$。因此，我们得到 $T^\pi v^* = v^*$，这意味着 v^* 同时也是 T^π 的一个不动点。由于 v^π 是 T^π 的唯一不动点，所以必有 $v^\pi = v^*$。 □

我们已经见过这样一个关于状态 x 和动作 u 的函数：在给定某个状态价值函数 v 的情况下，它评估期望总奖励，例如 (4.22) 中所示的函数。这个函数在强化学习的算法发展中也非常有用。下面我们给出它的正式定义。

定义 4.1.13 (Q 函数)**.** 对任意 $v \in B(\mathcal{X})$，我们定义与 v 对应的**质量函数**（quality function），简称 **Q 函数**（Q-function）（也称为**状态–动作价值函数**），为

$$q(x,u) := r(x,u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,u)}[v(X')] \tag{4.25}$$

其中 $x \in \mathcal{X}$，$u \in \mathcal{U}$。注意，$q : \mathcal{X} \times \mathcal{U} \to \mathbb{R}$。

特别地，我们将与 v^π（T^π 的唯一不动点）对应的 Q 函数记为

$$q^\pi(x,u) := r(x,u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,u)}[v^\pi(X')], \tag{4.26}$$

将与 v^*（T^* 的唯一不动点）对应的 Q 函数记为

$$q^*(x,u) := r(x,u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,u)}[v^*(X')]. \tag{4.27}$$

我们还注意到，反过来，v^π 和 v^* 也都可以分别由它们对应的 Q 函数 q^π 和 q^* 表示。也就是说，

$$v^\pi(x) = \mathbb{E}_{U\sim\pi(\cdot|x)}[q^\pi(x,U)]$$

以及

$$v^*(x) = \sup_{\pi\in\Pi} \mathbb{E}_{U\sim\pi(\cdot|x)}[q^*(x,U)]. \tag{4.28}$$

如果对每个 $x \in \mathcal{X}$，(4.28) 中的上确界都能够在 Π 中取到（例如，当 $\mathcal{U}$ 是有限集时），那么 (4.28) 就可化为

$$v^*(x) = \max_{u\in\mathcal{U}} q^*(x,u).$$

4.1.4 策略最优性与改进理论

现在我们已经准备好正式定义强化学习中最优策略的含义。

定义 4.1.14 (最优策略，optimal policy)**.** 如果 $\pi^* \in \Pi$ 满足

$$\mathbb{E}_{U\sim\pi^*(\cdot|x)}[q^*(x,U)] = v^*(x) \tag{4.29}$$

对每个 $x \in \mathcal{X}$ 都成立，其中 v^* 是 (4.11) 中定义的 T^* 的唯一不动点，而 q^* 定义于 (4.27)，则称 π^* 为一个**最优策略**。

显然，如果 π^* 相对于 v^* 是贪婪的，那么 π^* 就是一个最优策略，因为 (4.29) 意味着 $T^{\pi^*}v^* = T^*v^* = v^*$。我们将在下面的一个定理中将这一点表述为一个正式结论。

我们还定义如下一个特殊函数 $\bar{v} \in B(\mathcal{X})$：对每个 $x \in \mathcal{X}$，

$$\bar{v}(x) := \sup_{\pi\in\Pi} v^\pi(x). \tag{4.30}$$

注意，我们并不知道 $\bar{v}$ 是否是某个策略 $\pi \in \Pi$ 的价值函数。不过，由定义 (4.30) 可知，对任意 $\pi \in \Pi$ 都有

$$v^\pi \le \bar{v}. \tag{4.31}$$

我们还可以定义与 $\bar{v}$ 对应的 Q 函数为

$$\bar{q}(x,u) := r(x,u) + \gamma\mathbb{E}_{X'\sim p(\cdot|x,u)}[\bar{v}(X')]. \tag{4.32}$$

由 (4.30) 可见，

$$\bar{q}(x,u) = \sup_{\pi\in\Pi} q^{\pi}(x,u). \tag{4.33}$$

此外，关于 $\bar{v}$ 及其与 v^* 的关系，我们有如下重要引理。

引理 4.1.15. *如果存在一个平稳策略 $\bar{\pi}\in\Pi$ 相对于 $\bar{v}$ 是贪婪的，那么*

$$\bar{v} = v^{\bar{\pi}} \le v^*, \tag{4.34}$$

其中 $v^{\bar{\pi}}$ 是与 $\bar{\pi}$ 相关的 $T^{\bar{\pi}}$ 的唯一不动点，而 v^* 是 T^* 的唯一不动点。

证明. 对任意 $\hat{\pi}\in\Pi$，记其对应的 Bellman 算子为 $T^{\hat{\pi}}$，其唯一不动点为 $v^{\hat{\pi}}$，满足 $T^{\hat{\pi}}v^{\hat{\pi}} = v^{\hat{\pi}}$。

现在对任意 $x\in\mathcal{X}$，有

$$\begin{aligned}
v^{\hat{\pi}}(x) &= T^{\hat{\pi}}v^{\hat{\pi}}(x)\\
&= \mathbb{E}_{U\sim\hat{\pi}(\cdot|x)}\Big[r(x,U) + \gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[v^{\hat{\pi}}(X')]\Big]\\
&\le \mathbb{E}_{U\sim\hat{\pi}(\cdot|x)}\Big[r(x,U) + \gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\bar{v}(X')]\Big]\\
&\le \sup_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma\mathbb{E}_{X'\sim p(\cdot|x,U)}[\bar{v}(X')]\Big]\\
&= T^*\bar{v}(x)\\
&= T^{\bar{\pi}}\bar{v}(x),
\end{aligned}$$

其中第二个等号由定义 4.1.3 对策略 $\hat{\pi}$ 得到；第一个不等号由 (4.31) 得到；倒数第二个等号由 T^* 的定义得到；最后一个等号成立是因为 $\bar{\pi}$ 相对于 $\bar{v}$ 是贪婪的，因此 $T^{\bar{\pi}}\bar{v} = T^*\bar{v}$。

由于上面所取的 $\hat{\pi}\in\Pi$ 和 $x\in\mathcal{X}$ 都是任意的，所以由 (4.31) 中 $\bar{v}(x) = \sup_{\pi\in\Pi} v^{\pi}(x)$ 的定义可知，

$$\bar{v} \le T^{\bar{\pi}}\bar{v}, \tag{4.35}$$

对 (4.35) 两边同时施加 $T^{\bar{\pi}}$，并利用定理 4.1.10(ii)，得到

$$T^{\bar{\pi}}\bar{v} \le (T^{\bar{\pi}})^2\bar{v}. \tag{4.36}$$

结合 (4.35) 和 (4.36)，可得 $\bar{v}\le(T^{\bar{\pi}})^2\bar{v}$。继续对 (4.36) 两边施加 $T^{\bar{\pi}}$，并结合 (4.35)，就得到

$$\bar{v} \le T^{\bar{\pi}}\bar{v} \le \cdots \le (T^{\bar{\pi}})^k\bar{v}$$

对任意 k 都成立。令 $k \to \infty$，并回忆 $(T^{\bar{\pi}})^k \bar{v} \to v^{\bar{\pi}}$ 在 ∞-范数下收敛，即得

$$\bar{v} \leq v^{\bar{\pi}}. \tag{4.37}$$

由于 $\bar{\pi} \in \Pi$，由 (4.31) 知道 $v^{\bar{\pi}} \leq \bar{v}$。因此，$v^{\bar{\pi}} = \bar{v}$。

现在证明 $v^{\bar{\pi}} \leq v^*$。注意到

$$v^{\bar{\pi}} = T^{\bar{\pi}} v^{\bar{\pi}} \leq T^* v^{\bar{\pi}}, \tag{4.38}$$

其中不等号由定理 4.1.10(i) 得到。对上式两边施加 T^*，得到

$$T^* v^{\bar{\pi}} \leq (T^*)^2 v^{\bar{\pi}}. \tag{4.39}$$

继续对 (4.39) 两边施加 T^*，并利用定理 4.1.10(iii)，得到

$$v^{\bar{\pi}} \leq T^* v^{\bar{\pi}} \leq \cdots \leq (T^*)^k v^{\bar{\pi}}$$

对任意 k 都成立，其中第一个不等号由 (4.38) 得到。令 $k \to \infty$，并回忆 $(T^*)^k \bar{v} \to v^*$ 在 ∞-范数下收敛，即得

$$v^{\bar{\pi}} \leq v^*. \tag{4.40}$$

综上，我们得到 $\bar{v} = v^{\bar{\pi}} \leq v^*$。 □

定理 4.1.16 (策略改进定理，Policy Improvement Theorem)**.** *以下结论成立：*

(i) *对任意 $\pi_0 \in \Pi$，设 π 相对于 v^{π_0} 是贪婪的（其中 v^{π_0} 是与 π_0 相关的 Bellman 算子 T^{π_0} 的唯一不动点），即 $T^\pi v^{\pi_0} = T^* v^{\pi_0}$，则*

$$v^{\pi_0} \leq v^{\pi}. \tag{4.41}$$

*我们称 π 是**对 π_0 的改进**。*

(ii) *如果另外对某个 $x \in \mathcal{X}$ 有 $v^{\pi_0}(x) < T^* v^{\pi_0}(x)$，那么 $v^{\pi_0}(x) < v^{\pi}(x)$。在这种情况下，我们说 π **在 x 处严格改进了** π_0。*

(iii) *如果 $T^* v^{\pi^*} = v^{\pi^*}$，那么 π^* 是一个最优策略。*

证明. 对 (i)，我们有

$$v^{\pi_0} = T^{\pi_0} v^{\pi_0} \le T^* v^{\pi_0} = T^{\pi} v^{\pi_0},$$

其中不等号由定理 4.1.10(i) 得到，最后一个等号成立是因为 π 相对于 v^{π_0} 是贪婪的。继续对两边施加 T^{π}，并利用定理 4.1.10(ii)，可得

$$v^{\pi_0} \le T^{\pi} v^{\pi_0} \le \cdots \le (T^{\pi})^k v^{\pi_0}$$

对所有 k 都成立。令 $k \to \infty$，并注意到 $(T^{\pi})^k v^{\pi_0} \to v^{\pi}$ 在 ∞-范数下收敛，即得 $v^{\pi_0} \le v^{\pi}$。

对 (ii)，我们沿用 (i) 中相同的推导，得到

$$v^{\pi_0}(x) < T^* v^{\pi_0}(x) \le (T^{\pi})^k v^{\pi_0}(x)$$

对该 x 的所有 k 都成立。令 $k \to \infty$，可得 $v^{\pi_0}(x) < T^* v^{\pi_0}(x) \le v^{\pi}(x)$。

对 (iii)，由于 $T^* v^{\pi^*} = v^{\pi^*}$，可知 $v^{\pi^*} = v^*$，即 T^* 的唯一不动点。注意到 v^{π^*} 也是 T^{π^*} 的不动点，因此有 $T^{\pi^*} v^* = v^*$，即

$$\mathbb{E}_{U \sim \pi^*(\cdot|x)}[q^*(x, U)] = v^*(x)$$

对每个 $x \in \mathcal{X}$ 都成立。由定义 4.1.14，可知 π^* 是一个最优策略。 □

现在我们对最优策略的形式有了更清晰的认识。下面的定理将策略的最优性条件、它与 $\bar{v}$ 的关系，以及贪婪性质联系在一起。

定理 4.1.17. 以下命题彼此等价:

(i) π^* 是一个最优策略，即 $\mathbb{E}_{U \sim \pi^*(\cdot|x)}[q^*(x, U)] = v^*(x)$。

(ii) $v^* = v^{\pi^*} = \bar{v}$。

(iii) π^* 相对于 v^* 是贪婪的。

证明. 如果 (i) 成立，则有 $T^{\pi^*} v^* = v^*$，这意味着 v^* 是 T^{π^*} 的唯一不动点。因此 $v^* = v^{\pi^*}$。再由 (4.31) 知道 $v^{\pi^*} \le \bar{v}$。结合引理 4.1.15 中给出的 $\bar{v} \le v^*$，便得到 (ii)。

如果 (ii) 成立，那么有

$$T^{\pi^*} v^* = T^{\pi^*} v^{\pi^*} = v^{\pi^*} = v^* = T^* v^*,$$

其中第一个和第三个等号由 (ii) 得到，第二个等号成立是因为 π^* 是 T^{π^*} 的不动点，最后一个等号成立是因为 v^* 是 T^* 的不动点。

如果 (iii) 成立，那么

$$T^{\pi^*}v^* = T^*v^* = v^*,$$

其中第一个等号成立是因为 π^* 相对于 v^* 是贪婪的。根据定义 4.1.14，这说明 π^* 是一个最优策略。 □

推论 4.1.18. 如果 π^* 是一个最优策略，那么 π^* 相对于 $\bar{v}$ 是贪婪的。

证明. 如果 π^* 是一个最优策略，由定理 4.1.17 可知 $v^{\pi^*} = \bar{v} = v^*$。因此，

$$T^{\pi^*}\bar{v} = T^{\pi^*}v^{\pi^*} = v^{\pi^*} = v^* = T^*v^* = T^*\bar{v},$$

这表明 π^* 相对于 $\bar{v}$ 是贪婪的。 □

总结 现在我们对本节所建立的理论结果作一个简要总结。在具有无限时间（$T = \infty$）和折扣因子 $\gamma \in (0,1)$ 的标准设定下，我们知道每一个平稳策略 $\pi \in \Pi$ 都会产生一个 Bellman 算子 $T^\pi : B(\mathcal{X}) \to B(\mathcal{X})$，它具有唯一不动点 v^π。这里 $v^\pi : \mathcal{X} \to \mathbb{R}$ 是策略 π 的状态价值函数，意味着 $v^\pi(x)$ 是当智能体处于状态 $x \in \mathcal{X}$ 时未来总奖励的期望。Q 函数（状态–动作价值函数）$q^\pi(x,u)$ 则表示当智能体位于状态 $x \in \mathcal{X}$ 并采取特定动作 $u \in \mathcal{U}$ 时未来总奖励的期望。

我们还有一个概念上“逐点最优”的价值函数 $\bar{v} : \mathcal{X} \to \mathbb{R}$，它在每个 $x \in \mathcal{X}$ 处都定义为所有平稳策略 Π 上价值函数的上确界。然而，$\bar{v}$ 本身并不能说明它是否对应于 Π 中的某个策略，因此也就不知道它是否对应于某个 Bellman 算子。

与此同时，我们还有一个概念上“最优”的 Bellman 算子 $T^* : B(\mathcal{X}) \to B(\mathcal{X})$。它有唯一不动点 $v^* \in B(\mathcal{X})$，因此也对应一个 Q 函数 q^*，但并不总能保证 v^* 是 Π 中某个策略的状态价值函数。然而，如果对每个 $x \in \mathcal{X}$，函数

$$r(x,\cdot) + \gamma \int p(x'|x,\cdot)v^*(x')\,dx' \quad : \quad \mathcal{U} \to \mathbb{R}$$

的最大值能够在 $\mathcal{U}$ 中取得，那么 v^* 就存在一个贪婪策略 $\pi^* \in \Pi$。（我们前面提到过，如果 $\mathcal{U}$ 是有限集，或者 $\mathcal{U}$ 是紧集且上面定义的实值函数在 $\mathcal{U}$ 上连续，那么最大值就可以在 $\mathcal{U}$ 中取得。）更重要的是，π^* 的 Bellman 算子正是

T^*，其唯一不动点就是 v^*，而 $\bar{v}$ 也正是 v^*。这意味着根据定义，π^* 是一个最优策略，而 v^* 就是最优状态价值函数。

4.2 强化学习算法

强化学习算法可以按照许多不同方式进行分类。这是因为强化学习算法所应用的领域和问题设定非常多样。在本节中，我们按照该领域中最典型的一种分类方式，将强化学习算法分为两类：基于模型（model-based）的方法和无模型（model-free）的方法。一般而言，基于模型的方法考虑这样的问题：我们拥有能够（近似）准确描述由环境决定的奖励和状态转移概率的模型；而在无模型的方法中，我们并不具备这些信息，因此需要利用智能体与环境交互所生成的数据来求解问题。我们将在下面两个小节中分别详细介绍这两类算法。

4.2.1 基于模型的强化学习算法

基于模型的强化学习算法是指应用于如下问题的算法：我们已知以下信息

- 奖励函数 $r(x,u)$，或者定义在 $\mathbb{R}$ 上的条件奖励概率分布 $p_R(\cdot|x,u)$，从而我们能够计算奖励的期望值

$$\mathbb{E}_{R\sim p_R(\cdot|x,u)}[R] = \int r p_R(r|x,u)\,dr,$$

  对每个状态–动作对 $(x,u)\in\mathcal{X}\times\mathcal{U}$ 都成立。为了便于讨论，在下面讨论基于模型算法时，我们假设函数 r 是已知的。

- 对每个状态–动作对 $(x,u)\in\mathcal{X}\times\mathcal{U}$，定义在 $\mathcal{X}$ 上的状态转移概率

$$p(\,\cdot\,|\,x,u).$$

  它表示在当前状态为 x 且执行动作 u 时，下一个状态的概率分布。

注意，上述信息是由问题设定中的环境决定的。具体来说，我们准确知道：在当前状态为 x 且执行动作 u 时，可以获得多少奖励，以及下一个状态是什么（或者下一个状态的概率分布是什么）。这就是我们称这类强化学习问题为**基于模型**的原因。

当我们处理一个基于模型的强化学习问题时，通常可以直接求出该问题中的最优策略，而不需要让智能体与环境交互并收集必要的数据来估计这些量。

在许多这类情形中，我们可以使用动态规划或类似思想来寻找最优策略。下面我们将详细介绍一些常用的基于模型的强化学习算法。这些算法高度依赖于我们在第 4.1 节中讨论的 Markov 决策过程和 Bellman 算子的性质。

策略迭代

策略迭代（policy iteration）是一种迭代方案，可以为基于模型的强化学习生成一列持续改进的策略。在一定条件下（稍后将具体说明），这列策略可以被证明收敛到最优策略。策略迭代算法背后的理论基础是策略改进定理（定理 4.1.16）。

策略迭代算法的典型过程如下。我们任选一个初始猜测 $v_0 \in B(\mathcal{X})$（它不一定是某个价值函数），并找到一个相对于 v_0 是贪婪的策略 π_0。现在，在第 k 次迭代中（其中任意 $k \geq 0$），第一步是求当前策略 π_k 的价值函数 v^{π_k}，这一步称为**策略评估**（policy evaluation），因为我们试图求得当前策略 π_k 所对应的价值函数 v^{π_k}；然后在第二步中，这一步称为**策略改进**（policy improvement），我们寻找一个相对于 v^{π_k} 是贪婪的策略 π_{k+1}，并且可以证明 π_{k+1} 是对 π_k 的改进。随后将迭代计数器 k 加 1，并继续下一次迭代。我们在算法 4.2.1 中总结了策略迭代算法。

Algorithm 4.2.1 基于模型强化学习的策略迭代

Input: $v_0 \in B(\mathcal{X})$ 且 $\pi_0 \in \Pi$ 相对于 v_0 贪婪，$k = 0$。

1: **repeat**
2: （策略评估）求 v^{π_k}，使其满足 $T^{\pi_k} v^{\pi_k} = v^{\pi_k}$。
3: （策略改进）求 $\pi_{k+1} \in \Pi$，使其相对于 v^{π_k} 贪婪。
4: $k \leftarrow k + 1$
5: **until** 收敛。

Output: π_k。

当我们执行策略迭代算法（算法 4.2.1）时，会得到一列策略

$$\pi_0,\ \pi_1,\ \ldots,\ \pi_k,\ \ldots. \tag{4.42}$$

根据贪婪策略的定义（定义 4.1.11），我们知道策略改进步骤会产生 $T^{\pi_{k+1}} v^{\pi_k} = T^* v^{\pi_k}$。此外，由策略改进定理（定理 4.1.16(i)），有

$$v^{\pi_k} \leq v^{\pi_{k+1}}$$

对每个 $k \geq 0$ 都成立。

如果状态空间 $\mathcal{X}$ 和动作空间 $\mathcal{U}$ 都是有限集合，那么我们知道贪婪策略的个数也是有限的，因此迭代序列 (4.42) 必将在有限步后停止，即对某个 k 有 $\pi_k = \pi_{k+1}$。此时，我们有

$$v^{\pi_k} = T^{\pi_k} v^{\pi_k} = T^{\pi_{k+1}} v^{\pi_k} = T^* v^{\pi_k}$$

其中第一个等号是因为 v^{π_k} 是 T^{π_k} 的不动点，第二个等号由 $\pi_k = \pi_{k+1}$ 得到。因此，由于 v^* 是 T^* 的唯一极小化子，我们有 $v^{\pi_k} = v^*$。再由定理 4.1.17，可知 π_k 是一个最优策略。

现在我们需要考虑策略迭代算法（算法 4.2.1）的实际操作。更具体地说，我们需要找到求解策略评估（第 2 行）和策略改进（第 3 行）两个子问题的方法。根据问题规模的不同，我们需要采用不同的方法来解决这些子问题，下面我们将分别说明。

小规模问题 所谓小规模问题，是指状态空间 $\mathcal{X}$ 和动作空间 $\mathcal{U}$ 都是较小的有限集合（例如元素个数在 10^3 左右）。在这种情况下，对任意策略 π，我们都可以计算状态奖励

$$r^\pi(x) := \mathbb{E}_{U\sim\pi(\cdot|x)}[r(x,U)] \tag{4.43}$$

以及状态到状态的转移概率

$$p^\pi(x'|x) := \sum_{u\in\mathcal{U}} \pi(u|x) p(x'|x,u) \tag{4.44}$$

对所有 $x, x' \in \mathcal{X}$ 都成立。记 $n = |\mathcal{X}|$ 为 $\mathcal{X}$ 中状态的数量。则定义在 $\mathcal{X}$ 上的 v^π 与 r^π 都可以表示为 n 维列向量，而转移概率则可以组成一个 $n \times n$ 矩阵 $P^\pi = [p^\pi(x'|x)]_{x,x'\in\mathcal{X}}$。因此，$v^\pi$ 的 Bellman 方程

$$v^\pi(x) = T^\pi v^\pi(x) = \mathbb{E}_{U\sim\pi(\cdot|x)}\Big[r(x,U) + \gamma \mathbb{E}_{X'\in p(\cdot|x,U)}[v^\pi(X')]\Big]$$

对所有 $x \in \mathcal{X}$ 可以写成如下线性系统

$$v^\pi = r^\pi + \gamma P^\pi v^\pi, \tag{4.45}$$

其中 I_n 是 $n \times n$ 单位矩阵。

由 Gershgorin 圆盘定理可知，P^π 的所有特征值都位于 $\mathbb{R}$ 中如下集合内：

$$\bigcup_{x\in\mathcal{X}} D\Big(p^\pi(x|x), \sum_{x'\neq x} p^\pi(x'|x)\Big) \tag{4.46}$$

其中 $D(c,r) := [c-r, c+r]$ 是 $\mathbb{R}$ 上的闭区间。由于对所有 $x, x' \in \mathcal{X}$ 都有 $\sum_{x'\in\mathcal{X}} p^\pi(x'|x) = 1$ 且 $p^\pi(x'|x) \geq 0$，我们知道集合 (4.46) 包含在 $[-1,1]$ 中。因此，P^π 所有特征值的最大模不超过 1。于是只要 $\gamma < 1$，就有 $I_n - \gamma P^\pi$ 可逆，从而 (4.45) 有唯一解

$$v^\pi = (I_n - \gamma P^\pi)^{-1} r^\pi. \tag{4.47}$$

在实际中，解 (4.47) 可以使用已有的数值线性方程组求解器计算。一般情况下，其计算代价为 $O(n^3)$（如果 P^π 具有特殊结构，则代价可能更低）。这就是每次迭代中策略评估（算法 4.2.1 第 2 行）的求解过程。

对于策略改进（算法 4.2.1 第 3 行），我们可以直接把 π_{k+1} 设为如下确定性贪婪策略：对任意 $x \in \mathcal{X}$，令

$$\pi_{k+1}(u|x) = \begin{cases} 1, & \text{如果 } u = \arg\max_{u'\in\mathcal{U}} q^{\pi_k}(x,u'), \\ 0, & \text{否则}, \end{cases} \tag{4.48}$$

其中 q^{π_k} 是与 π_k 对应的 Q 函数，定义见 (4.26)。如果 $q^{\pi_k}(x,\cdot)$ 在 $\mathcal{U}$ 中有多个极大值点，则在 (4.48) 中任选其一即可。

中等规模问题 在这种情况下，我们通常有状态空间大小 $|\mathcal{X}| \approx 10^6$ 或类似规模。此时，直接求解 (4.45) 一般代价很高。一个可行方法是：对给定策略 π，通过固定点迭代 $v_{j+1} = T^\pi v_j$（$j \geq 0$）并从某个初值 v_0 出发，来求解策略评估子问题（算法 4.2.1 第 2 行）。由于 T^π 是 γ-压缩映射，因此根据定理 4.1.8，

$$\|v_j - v^\pi\|_\infty \leq \gamma^j \|v_0 - v^\pi\|_\infty$$

对所有 j 都成立。因此，对任意 π，我们都可以以 γ 的线性速率逼近 v^π。在实际中，我们可以在 $\|v_{j+1} - v_j\|_\infty < \varepsilon_{\text{tol}}$ 时终止固定点迭代，其中 $\varepsilon_{\text{tol}} > 0$ 是某个预先给定的容差（用户定义的超参数），并认为策略评估（算法 4.2.1 第 2 行）已被近似求解，然后转入策略改进（算法 4.2.1 第 3 行）。

为了求解策略改进子问题，我们可以像 (4.48) 那样，把 π_{k+1} 设为确定性贪婪策略。当 $\mathcal{U}$ 是一个规模较小或中等的有限集合时，这在计算上是可行的。

大规模问题 所谓大规模问题，是指那些 $\mathcal{X}$ 含有 10^{10} 个状态，甚至远远更多；或者 $\mathcal{X}$ 是 $\mathbb{R}^d$ 中的连续区域，且 $d \geq 5$，使得哪怕进行适度的空间离散化也会得到超过 10^{10} 个状态（这也是维数灾难问题）。在实际中，这类问题的规模往往更大，以至于在现代计算机内存中存储价值函数或策略都是不可行的。因此，我们通常用函数逼近器来表示价值函数和策略。在这些情形下，深度神经网络被证明是非常有效的函数逼近器。

更具体地说，在策略评估步骤（算法 4.2.1 第 2 行）中，我们有当前估计的策略 π_k。为了求出其价值函数 $v^{\pi_k}: \mathcal{X} \to \mathbb{R}$，我们把它参数化为一个深度神经网络，并通过求解以下最小化问题来训练它：

$$v^{\pi_k} = \arg\min_{v} \frac{1}{2} \int |v(x) - T^{\pi_k} v(x)|^2 \, dx \tag{4.49}$$

其中 T^{π_k} 是与策略 π_k 对应的 Bellman 算子，定义见 (4.9)。最小化问题 (4.49) 是关于 v 的网络参数进行的。由于 T^{π_k} 是 γ-压缩映射，它有唯一不动点，因此问题 (4.49) 有唯一极小化子，而这个极小化子正是所需的 v^{π_k}。

策略改进步骤（算法 4.2.1 第 3 行）要求寻找一个相对于 v^{π_k} 贪婪的策略 π_{k+1}，如 (4.48) 所示。这归结为：对每个状态 $x \in \mathcal{X}$，求解一个关于贪婪动作 $u \in \mathcal{U}$ 的极大化问题。这一问题的难点取决于这类极大化问题是否容易求解，而这会随着具体应用问题而变化。因此，用来求解该策略改进子问题的方法往往需要具体问题具体分析。

尽管如此，仍有一些应用中状态空间 $\mathcal{X}$ 极其巨大，而动作空间却很小。例如，在围棋中，状态空间 $\mathcal{X}$ 包含所有可能的棋盘局面，且 $|\mathcal{X}| \approx 10^{170}$，而 $|\mathcal{U}|$ 仅有几百个（在很多状态 x 下，$\mathcal{U}$ 中合法动作的数量通常更少）。此外，从 (x, u) 到下一个状态 x' 的转移是确定性的：一旦在状态 x（当前棋盘局面）下执行动作 u（在棋盘上落子），下一个状态 x' 就完全确定。在这种情况下，(4.48) 中的极大化问题对于任意给定的 x 都可以通过计算机进行简单搜索而容易求解。

然而，即便 $|\mathcal{U}|$ 很小，策略改进子问题仍然具有挑战性：状态空间 $\mathcal{X}$ 依旧过于庞大，不可能对每个 $x \in \mathcal{X}$ 都求出一个动作 u。在实践中，我们可以将策略 π（即在给定任意状态 $x \in \mathcal{X}$ 时，定义在 $\mathcal{U}$ 上的条件概率）参数化为一个深度神经网络（称为策略网络），并与参数化价值函数的深度神经网络（称为价值网络）一起训练。例如，给定当前价值函数 v^{π}，我们可以从 $\mathcal{X}$ 中采样 M 个状态 $\{x_i : i \in [M]\}$，并对每个采样状态 x_i 求解

$$u_i = \arg\max_{u \in \mathcal{U}} \Big\{ r(x_i, u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x_i, u)} [v^{\pi}(X')] \Big\}.$$

Algorithm 4.2.2 基于模型强化学习的价值迭代

Input: $v_0 \in B(\mathcal{X})$, $k = 0$。
1: **repeat**
2: （价值改进）设 $v_{k+1} = T^* v_k$。
3: $k \leftarrow k + 1$
4: **until** 收敛。
5: 找到一个相对于 v_k 贪婪的策略 π。
Output: π。

然后利用这些样本对 $\{(x_i, u_i) : i \in [M]\}$ 来训练策略网络。这当然并不总是简单且易于实际实现的。但它展示了一种处理策略改进步骤的通用思路。我们还将在第 4.2.2 节中介绍若干实际算法，以进一步应对这一问题。

价值迭代

价值迭代是另一种用于求解基于模型强化学习问题的迭代方案。价值迭代的思想是生成一列价值函数，使其直接逼近最优价值函数 v^*。当找到价值函数 v^* 后，我们就可以直接得到最优策略，即相对于 v^* 贪婪的那个策略。相比之下，策略迭代则需要在迭代过程中生成一列持续改进的策略。

由于 T^* 是 γ-压缩映射，我们知道 v^* 是其唯一不动点，并且可以从任意初始值 v_0 出发，通过固定点迭代 $v_{k+1} = T^* v_k$ 来逼近它。价值迭代算法本质上正是在实现这一固定点迭代方案。我们在算法 4.2.2 中总结了价值迭代算法。

为了实现价值改进步骤（算法 4.2.2 第 2 行），我们对每个 x 和 k 设定

$$v_{k+1}(x) = \max_{u \in \mathcal{U}} \Big\{ r(x, u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x,u)}[v_k(X')] \Big\}. \tag{4.50}$$

对于小规模到中等规模的问题，这通常是可行的。然而，对于大规模问题，我们需要使用函数逼近器（例如深度神经网络）来逼近 v_k。具体而言，对每个给定的 v_k，我们可以从 $\mathcal{X}$ 中采样 M 个状态 $\{x_i : i \in [m]\}$，并通过求解

$$u_i = \arg\max_{u \in \mathcal{U}} \Big\{ r(x_i, u) + \gamma \mathbb{E}_{X' \sim p(\cdot|x_i,u)}[v_k(X')] \Big\}$$

来得到对应的 u_i，对每个 $i \in [M]$ 都如此。然后利用这些样本对 $\{(x_i, u_i) : i \in$

$[M]\}$ 来训练 v_{k+1}：

$$v_{k+1} = \arg\min_{v} \frac{1}{2M} \sum_{i=1}^{M} \Big| v(x_i) - \Big(r(x_i, u_i) + \gamma \mathbb{E}_{X' \sim p(\cdot|x_i,u_i)}[v_k(X')] \Big) \Big|^2,$$

其中最小化是相对于 v 的网络参数进行的。由定理 4.1.8 可知，v_k 在 ∞-范数下以 γ 的线性速率收敛到 v^*。当 v_k 收敛后，我们可以像 (4.48) 中那样，选择一个相对于输出 v_k 贪婪的策略 π。注意，只有在价值迭代算法（算法 4.2.2）的这一时刻，我们才会计算一个策略，而它是对最优策略 π^* 的一个近似。

4.2.2 无模型强化学习算法

在许多现实世界的应用中，由于环境或问题设定的复杂性，我们可能并不具备关于奖励函数 r（或奖励概率 $p_R(\cdot|x,u)$）以及状态转移概率 $p(\cdot|x,u)$ 的精确模型。在这些情况下，我们需要让智能体执行动作并与环境进行交互。智能体可以在这些交互过程中收集数据，利用这些数据我们可以估计奖励和状态转移概率，并找到能够产生最大总回报的最优策略。这类方法称为**无模型**强化学习算法。

在本小节中，我们将介绍一系列无模型强化学习算法。这些方法之间有许多相似之处，但由于问题设定和数据可获得性的多样性，它们之间也存在差异。尽管如此，我们仍应牢记第 4.1 节中建立的基本结果，因为这些结果有助于理解这些算法。

Monte Carlo 方法

原始的 Monte Carlo 方法是一种无模型方法，常用于小规模到中等规模的问题。回忆一下，对于给定策略 $\pi \in \Pi$，其价值函数定义为

$$v^\pi(x) = \mathbb{E}_{\pi,p}\Big[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \Big| X_0 = x \Big],$$

对每个 $x \in \mathcal{X}$ 都成立，其中 R_t 是在时间 t 收到的奖励。在中等规模以内的问题中，$\mathcal{X}$ 是一个大小约为 10^6 或类似规模的有限集合，因此价值函数 $v^\pi : \mathcal{X} \to \mathbb{R}$ 可以作为一个**表格**（即大小为 $|\mathcal{X}|$ 的数组）存储在计算机内存中，并根据观测数据方便地进行更新，具体如下所述。

由于在无模型设定中我们并不知道奖励函数 r 和转移概率 p，因此需要让智能体在环境中执行动作并收集样本数据，以估计这些量。在这种情况下，设

我们让智能体按照当前估计的策略 π 与环境交互，则会观测到一个由该 π 诱导的 Markov 决策过程：

$$X_0,\ U_0,\ R_1,\ \ldots,\ X_t,\ U_t,\ R_{t+1},\ X_{t+1},\ U_{t+1},\ldots \tag{4.51}$$

为了扩展价值函数 v^π 的概念，我们定义**总回报**为

$$G_t^\pi(x) := \sum_{\tau=t}^{\infty} \gamma^{\tau-t} R_{\tau+1}, \tag{4.52}$$

它表示从时间 t 开始且 $X_t = x$，在 t 之后所获得的总奖励，注意这里相当于假设 Markov 决策过程是从时间 t 而不是从 0 开始的。我们还定义其期望总回报为

$$g_t^\pi(x) := \mathbb{E}_{\pi,p}\Big[G_t^\pi(x)\Big|X_t = x\Big]. \tag{4.53}$$

于是，对任意 t，$g_t^\pi(x)$ 都可以在乘以适当缩放因子 γ^t 后用于估计 $v^\pi(x)$。在 Monte Carlo 方法中，我们需要利用 $g_t^\pi(x)$ 的样本，也即 $G_t^\pi(x)$ 的样本，来逼近价值函数 v^π。

更具体地说，我们让智能体按照当前策略 π 行动并在环境中执行动作。然后我们观测到 Markov 奖励过程的一个样本轨迹，称为一个**回合**（episode）。一个回合只是采样得到的 Markov 决策过程 (4.51) 中去掉动作后的片段：

$$x_0,\ r_1,\ x_1,\ r_2,\ \ldots,\ x_t,\ r_{t+1},\ x_{t+1},\ \ldots, x_T,\ r_{T+1}, \tag{4.54}$$

它在实践中会在某个有限时间 T 终止。如果我们在时间 t 观测到 $x_t = x$，那么就把

$$\sum_{\tau=t}^{T} \gamma^{\tau-t} r_{\tau+1} \tag{4.55}$$

视为 $G_t^\pi(x)$ 的一个样本。注意，对于某个 $x \in \mathcal{X}$，在一个回合中，我们可能得到零个、一个或多个类似 (4.55) 的样本，这取决于状态 x 在该回合中出现的次数。在实践中，我们也可以运行多个回合，并对类似 (4.55) 的样本取平均。简化起见，我们仍将这个平均值记为 $G_t^\pi(x)$（这里 t 用来表示总回报是从时间 t 到达状态 x 后开始累积的；但由于 $G_t^\pi(x)$ 对任意 t 都是 $v^\pi(x)$ 的估计，因此 t 的具体数值并不重要）。在这种情况下，Monte Carlo 方法通过如下方式更新 $v^\pi(x)$：

$$v^\pi(x) \leftarrow v^\pi(x) + \alpha(G_t^\pi(x) - v^\pi(x)), \tag{4.56}$$

其中 $\alpha > 0$ 为步长。

Monte Carlo 方法已知在回合长度趋于无穷时可以给出价值函数 v^π 的无偏估计，但该估计的方差通常较大。得到 v^π 的估计之后，我们可以找到一个相对于 v^π 贪婪的策略 π'，然后使用更新后的策略 π' 再次重复 Monte Carlo 方法的过程，以估计新的价值函数 $v^{\pi'}$，如此往复。因此，Monte Carlo 方法会像价值迭代算法（算法 4.2.2）那样生成一列不断改进的策略，只不过它不需要知道奖励函数 r 和转移概率 p。

时序差分方法

时序差分（temporal difference，TD）是另一种与 Monte Carlo 方法密切相关的无模型方法。二者的区别在于：对于当前策略 π，TD 方法中的 v^π 表格会在遍历形如 (4.54) 的采样 Markov 奖励过程中（一个回合内）持续更新；相比之下，Monte Carlo 方法只在每个回合结束时才更新该表格。

更具体地说，设 $x_t = x$，并且智能体获得奖励 r_{t+1} 后转移到状态 x_{t+1}，则我们可以通过如下方式更新 $v^\pi(x)$：

$$v^\pi(x) \leftarrow v^\pi(x) + \alpha\Big(\underbrace{\overbrace{r_{t+1} + \gamma v^\pi(x_{t+1})}^{\text{TD 目标}} - v^\pi(x)}_{\text{TD 误差}}\Big) \tag{4.57}$$

其中 $\alpha > 0$ 是步长。正如我们所见，TD 方法 (4.57) 使用所谓的 **TD 目标** $r_{t+1} + \gamma v^\pi(x_{t+1})$ 来替代 Monte Carlo 方法 (4.56) 中的 $G_t^\pi(x)$。其目标是使 (4.57) 中的 **TD 误差**变为零，从而得到 v^π 的 Bellman 方程 (4.7)。注意，TD 方法使用当前估计的一部分，即 $v^\pi(x_{t+1})$，来估计值 $v^\pi(x)$。这一策略称为**自举**（bootstrapping）。与 Monte Carlo 方法相比，TD 方法得到的 v^π 估计是有偏的，但其方差通常更小。

上面讨论的 TD 方法通常简称为 TD(0) 方法。它可以推广为：对 $n =$

$1, 2, \ldots$，使用 n 步总回报作为 TD 目标。为此，我们记

$$
\begin{aligned}
G_{t,1}^{\pi}(x) &:= R_{t+1} + \gamma v^{\pi}(X_{t+1}), \\
G_{t,2}^{\pi}(x) &:= R_{t+1} + \gamma R_{t+2} + \gamma^2 v^{\pi}(X_{t+2}), \\
&\vdots \\
G_{t,n}^{\pi}(x) &:= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n v^{\pi}(X_{t+n}), \\
&\vdots \\
G_{t,\infty}^{\pi}(x) &:= G_t^{\pi}, \qquad \text{（假设当 } t > T+1 \text{ 时 } R_t = 0\text{）}.
\end{aligned}
$$

然后对任意 $\lambda \in [0,1]$，定义

$$
G_t^{\pi,\lambda}(x) := \sum_{n=1}^{\infty} (1-\lambda)\lambda^{n-1} G_{t,n}^{\pi}(x). \tag{4.58}
$$

注意，$(1-\lambda)\lambda^{n-1}$ 在和式 (4.58) 中起到权重的作用，并且总权重满足 $\sum_{n=1}^{\infty}(1-\lambda)\lambda^{n-1} = 1$。于是，作为 TD(0) 方法推广的 TD($\lambda$) 方法通过如下方式更新 $v^{\pi}(x)$：

$$
v^{\pi}(x) \leftarrow v^{\pi}(x) + \alpha(G_t^{\pi,\lambda}(x) - v^{\pi}(x)). \tag{4.59}
$$

事实上，TD(λ) 方法正是 TD(0) 方法与 Monte Carlo 方法之间的一种插值：当 $\lambda = 0$ 时，有

$$
G_t^{\pi,0}(x) = G_{t,0}^{\pi}(x) = R_{t+1} + \gamma v^{\pi}(X_{t+1})
$$

因此 TD(λ) 恰好就是 TD(0) 方法；当 $\lambda \to 1$ 时，(4.58) 中所有权重都趋近于 1，而 TD(λ) 在渐近意义下变为 Monte Carlo 方法。

对于任意策略 π，由 (4.57) 迭代更新得到的 $v^{\pi}(x)$ 在适当的步长设置和无限量回合数据的条件下，可以证明 (4.57) 收敛到 π 的真实价值函数。这里我们省略证明。

和 Monte Carlo 方法一样，在更新完价值函数 v^{π} 之后，我们还需要找到一个相对于 v^{π} 贪婪的策略 π'，然后使用 π' 重复 TD 更新 (4.57) 的过程，以估计新函数 $v^{\pi'}$，如此继续。因此，TD 方法与 Monte Carlo 方法相似，但在实践中通常被认为比后者更高效地利用数据。

SARSA 算法

SARSA算法也是一种无模型方法，它与 TD 方法有很多相似之处，并适用于小规模到中等规模的问题。不同之处在于，SARSA 算法学习的是质量函数（状态–动作价值函数）q^π，而不是 TD 方法中的状态价值函数 v^π。给定一个策略 π，智能体可以执行动作并生成一个 Markov 决策过程

$$X_0,\ U_0,\ R_1,\ \ldots,\ X_t,\ U_t,\ R_{t+1},\ X_{t+1},\ U_{t+1},\ \ldots \tag{4.60}$$

为了学习 q^π，SARSA 算法同样在计算机内存中保存一个定义在 $\mathcal{X}\times\mathcal{U}$ 上的 q^π 表格。然后通过对 (4.60) 采样一个回合，我们观测到如下 Markov 决策过程样本轨迹

$$x_0,\ u_0,\ r_1,\ \ldots,\ x_t,\ u_t,\ r_{t+1},\ x_{t+1},\ u_{t+1},\ \ldots$$

并通过如下方式更新 $q^\pi(x,u)$：

$$q^\pi(x_t,u_t)\leftarrow q^\pi(x_t,u_t)+\alpha(r_{t+1}+\gamma q^\pi(x_{t+1},u_{t+1})-q^\pi(x_t,u_t)). \tag{4.61}$$

我们可以看出 (4.57) 与 (4.61) 有很强的相似性（在 (4.61) 中我们直接使用 $q^\pi(x_t,u_t)$ 而不是 $q^\pi(x,u)$，但其思想与 (4.57) 中的 $v^\pi(x_t)$ 是相同的）。(4.61) 的目标是在 x_{t+1} 是按照 $p(\cdot|x_t,u_t)$ 采样得到的前提下，使 $r_{t+1}+\gamma q^\pi(x_{t+1},u_{t+1})$ 在期望意义下等于 $q^\pi(x_t,u_t)$，从而得到 (4.26)。由于每次这样的更新都需要从样本轨迹中提取一个五元组 $(x_t,u_t,r_{t+1},x_{t+1},u_{t+1})$，因此 SARSA 算法的名字正是（State, Action, Reward, State, Action）的缩写。

对于任意策略 π，通过 SARSA 更新规则 (4.61) 迭代更新得到的函数 q^π 也可以被证明收敛到 π 的真实 Q 函数。之后，我们可以把策略更新为相对于 π 贪婪的策略，即

$$\pi'(u|x)=\begin{cases}1, & \text{如果}u=\arg\max_{u'\in\mathcal{U}} q^\pi(x,u'),\\ 0, & \text{否则}.\end{cases} \tag{4.62}$$

之所以按这种方式选择上面的贪婪策略，是因为

$$T^{\pi'}v^\pi(x)=T^*v^\pi(x)=\max_{\hat\pi\in\Pi}\mathbb{E}_{U'\sim\hat\pi(\cdot|x)}[q^\pi(x,U')]=\max_{u'\in\mathcal{U}}q^\pi(x,u'),$$

前提是对每个 $x\in\mathcal{X}$，在 $\mathcal{U}$ 中都能取到一个极大值点。得到 π' 后，我们可以按照 π' 生成新的样本轨迹，并在这些轨迹上继续应用 SARSA 更新规则 (4.61)

来获得 $q^{\pi'}$，如此反复。这一过程会生成一列策略 $\pi_1, \pi_2, \ldots, \pi_k, \ldots$，其中对每个 k，π_{k+1} 都相对于 v^{π_k} 是贪婪的。由策略改进定理（定理 4.1.16）可知，π_k 收敛到 π^*。

在实践中，SARSA 算法并不需要很长的回合来更新 q，因为它可以像 (4.61) 那样在一个回合进行过程中持续更新，并可在任意时刻终止。

SARSA 算法的一个主要缺点是，它需要五元组 $(x_t, u_t, r_{t+1}, x_{t+1}, u_{t+1})$ 来实现更新 (4.61)。如果某个状态–动作对 (x_{t+1}, u_{t+1}) 没有被采样到——而这种情况经常会发生，因为当前对 q 的估计可能表明在状态 x_{t+1} 下某个动作 u_{t+1} 不太有价值——那么我们可能永远无法更新 $q(x_{t+1}, u_{t+1})$。

这一缺点是无模型算法中的一个常见问题，其核心在于强化学习中的**利用**（exploitation）与**探索**（exploration）之间的权衡。利用意味着算法总是根据先前得到的状态–动作价值 q 选择当前最优动作；而探索则保留一定概率去尝试其他动作，从而可能发现更好的动作。现实生活中的类比是：利用意味着我们总是去当前最喜欢的餐馆并尝遍菜单上的所有菜，而探索意味着我们偶尔会随机尝试别的餐馆，看看是否有更好的菜品。

在许多无模型强化学习算法中，探索是通过 **ϵ-贪婪策略**

$$\pi(u|x) = \begin{cases} \frac{\epsilon}{|\mathcal{U}|} + 1 - \epsilon, & \text{如果} u = \arg\max_{u' \in \mathcal{U}} q^\pi(x, u'), \\ \frac{\epsilon}{|\mathcal{U}|}, & \text{否则}. \end{cases} \tag{4.63}$$

来实现的，其中某个小参数 $\epsilon \in (0, 1)$，并且 $|\mathcal{U}| < \infty$，例如 $\mathcal{U}$ 是有限集，或者是欧氏空间中的连续紧子集。（当 $\mathcal{U}$ 无界时，我们也可以使用非均匀分布来代替均匀分布。）为了实现这一策略，我们可以先抽取一个 Bernoulli 样本，其有两个可能结果：以概率 $1 - \epsilon$ 得到正面，以概率 ϵ 得到反面；若得到正面，就选择极大值动作 u，若得到反面，就按照均匀分布从 $\mathcal{U}$ 中随机选择一个动作。

类似于 n 步时序差分方法，SARSA 算法也可以使用更长的元组

$$(x_t,\ u_t,\ r_{t+1},\ x_{t+1},\ u_{t+1},\ \ldots,\ r_{t+n},\ x_{t+n},\ u_{t+n})$$

作为 Markov 决策过程样本轨迹中的数据。在这种情况下，我们得到 SARSA 算法的一个变体，称为 **n 步 SARSA** 算法，其更新 q 的方式为

$$\begin{aligned} q^\pi(x_t, u_t) \leftarrow\ & q^\pi(x_t, u_t) \\ & + \alpha\Big(r_{t+1} + \cdots + \gamma^{n-1} r_{t+n} + \gamma^n q^\pi(x_{t+n}, u_{t+n}) - q^\pi(x_t, u_t)\Big). \end{aligned}$$

Q-learning 算法

Q-learning算法是 SARSA 算法的一个变体，它把学习当前策略 π 的 Q 函数 q^π 的目标改为直接学习最优 Q 函数 q^*。这类似于价值迭代算法（算法 4.2.2）直接学习最优状态价值函数 v^*）。回忆 (4.27)，最优状态–动作价值函数 q^* 满足

$$\begin{aligned} q^*(x,u) &= r(x,u) + \mathbb{E}_{X'\sim p(\cdot|x,u)}[v^*(X')] \\ &= r(x,u) + \mathbb{E}_{X'\sim p(\cdot|x,u)}[\max_{u'\in\mathcal{U}} q^*(X',u')]. \end{aligned} \tag{4.64}$$

转移概率 $p(\cdot|x,u)$ 由环境决定，而不是由任何策略决定。因此，如果我们在一个回合中观测到 $(x_t,u_t,r_{t+1},x_{t+1})$，那么可以把 $v^*(x_{t+1})$ 看作 $\mathbb{E}_{X'\sim p(\cdot|x_t,u_t)}[v^*(X')]$ 的一个无偏估计。与 SARSA 算法中的更新规则类似，Q-learning 算法中对 q 的更新为

$$q(x_t,u_t) \leftarrow q(x_t,u_t) + \alpha\Big(r_{t+1} + \gamma \max_{u'\in\mathcal{U}} q(x_{t+1},u') - q(x_t,u_t)\Big). \tag{4.65}$$

在一定条件下，也可以证明以上更新得到的 q 收敛到 q^*。Q-learning 算法的其余部分与 SARSA 算法相同。一旦通过反复迭代 (4.65) 学到了 q^*，我们就可以像 (4.62) 那样找到一个相对于 q^* 贪婪的策略，而它将是一个最优策略。

与 SARSA 算法相比，Q-learning 算法的主要优点在于，它只使用形如 $(x_t,u_t,r_{t+1},x_{t+1})$ 的四元组作为样本。它们不包含 u_{t+1}，因此比 SARSA 算法对数据的需求更低。

总结 到目前为止，我们可以对目前讨论过的无模型方法作一个快速总结，包括 Monte Carlo 方法、时序差分（TD）方法、SARSA 算法以及 Q-learning 算法。更具体地说，给定一个策略 π，Monte Carlo 方法使用完整回合来估计价值函数 v^π；TD(0) 方法使用当前估计的 v^π 来估计其自身（即一种自举策略）；而 TD(λ) 方法则是 TD(0) 方法与 Monte Carlo 方法之间的一种插值。对于 Monte Carlo 和 TD 方法，我们都会得到一个相对于 v^π 贪婪的新策略，并重复这一过程。它们可以看作是策略迭代算法（算法 4.2.1）的无模型版本。

相比之下，SARSA 算法学习的是当前策略 π 的 Q 函数 q^π，然后再把 π 更新为相对于 q^π 贪婪的策略。之后，由 SARSA 算法生成的策略序列预计将收敛到最优策略 π^*。因此，SARSA 算法也可以看作是策略迭代算法的无模型版本，只不过它在每次迭代中学习的是 q^π 而不是 v^π。

Q-learning 算法则通过规则 (4.65) 直接学习最优 Q 函数 q^*。当其收敛时，

我们得到（对）q^* 的近似，并将最优策略设为相对于 q^* 贪婪的那个策略。因此，Q-learning 算法可以视为价值迭代算法（算法 4.2.2）的无模型版本，只不过它逼近的是 q^* 而不是 v^*。

所有这些方法都可用于小规模到中等规模的问题，因为它们需要将 v^π 或 q^π 作为表格存储在计算机内存中。接下来我们考虑它们可用于大规模问题的变体。

深度 Q 网络（DQN）算法

我们考虑 Q-learning 算法的一个变体，称为深度 Q 网络（deep Q-network，DQN）算法，它可以求解具有大状态空间 $\mathcal{X}$ 和动作空间 $\mathcal{U}$ 的无模型强化学习问题。DQN 算法的关键在于使用函数逼近器来逼近最优 Q 函数 q^*。我们使用深度神经网络作为这类逼近器。

回忆一下，q^* 的 Bellman 方程由 (4.64) 给出。因此，很自然地，可以考虑使用如下损失函数来训练 q^*：

$$\frac{1}{2}\Big|q^*(x_t,u_t)-\Big(r_{t+1}+\gamma\max_{u'\in\mathcal{U}}q^*(x_{t+1},u')\Big)\Big|^2 \tag{4.66}$$

其中样本四元组为 $(x_t,u_t,r_{t+1},x_{t+1})$。通过运行许多回合，我们可以获得大量形如 $(x_t,u_t,r_{t+1},x_{t+1})$ 的样本四元组，并对它们取平均，将其作为 Q 网络的损失函数。在每次迭代中，我们可以取一个相对于当前 q^* 估计的 ϵ-贪婪策略 π，然后再次运行回合来更新 q^*。

在实践中，DQN 算法使用一个**经验回放**（experience replay）缓冲区来存储先前获得的 $(x_t,u_t,r_{t+1},x_{t+1})$。注意，一旦 (x_t,u_t) 确定，$r_{t+1}=r(x_t,u_t)$ 或 $r_{t+1}\sim p_R(\cdot|x_t,u_t)$，以及 $x_{t+1}\sim p(\cdot|x_t,u_t)$ 都与策略无关，因此由之前的策略得到的数据可以被存入经验回放缓冲区，并在新策略对应的 q^* 训练中继续使用。因为同一批样本可以被不同策略复用，这样就提高了整体的采样使用效率。

注意，上述 DQN 设定中的一个问题是，q^* 在损失函数 (4.66) 的两处同时出现。这通常会导致训练不稳定，因为一旦当前估计的 q^* 偏向某个特定的状态–动作序列，它在后续训练中会被进一步强化，从而造成更大的估计误差。为缓解这一问题，我们可以设置两个 DQN，分别记为 q 和 $\hat{q}$，称为训练 DQN 和目标 DQN。在更新 q 时，保持 $\hat{q}$ 不变，并求解

$$\min_q\frac{1}{2}\iiint\Big|q(x,u)-\Big(r(x,u)+\gamma\max_{u'\in\mathcal{U}}\hat{q}(x',u')\Big)\Big|^2dx'dudx \tag{4.67}$$

在进行若干次 (4.67) 更新 q 之后，我们令 $\hat{q} \leftarrow q$，然后再重复若干轮 (4.67)。从经验上看，这常常能提升计算稳定性。然而，误差累积问题仍然存在。

双重深度 Q 网络算法

注意，在上面的 DQN 算法中，我们在更新 q 时固定 $\hat{q}$。也就是说，用于更新 q 的目标由如下函数给出：

$$r(x,u) + \gamma \max_{u' \in \mathcal{U}} \hat{q}(x', u').$$

如果我们假设在上式中的极大化问题里选择了一个贪婪动作 $u' \in \mathcal{U}$，那么该目标实际上等价于

$$r(x,u) + \gamma \hat{q}(x', \arg\max_{u' \in \mathcal{U}} \hat{q}(x', u')). \tag{4.68}$$

因此，如果 $\hat{q}$ 对某个 (x,u) 产生了过高估计，那么这种误差仍会继续累积。

在双重深度 Q 网络（Double DQNs）算法中，我们仅仅把 (4.68) 改为如下形式：

$$r(x,u) + \gamma \hat{q}(x', \arg\max_{u' \in \mathcal{U}} q(x', u')),$$

并用它作为训练 q 的目标。当 q 和 $\hat{q}$ 每隔若干次迭代交替更新时，从经验上发现，这种做法可以显著缓解误差累积问题。

策略梯度算法

Monte Carlo 算法、时序差分算法和 SARSA 算法都旨在学习某个给定策略 π 的状态价值函数 v^π 或状态–动作价值函数 q^π。另一方面，Q-learning 算法、DQN 方法及其变体则试图学习 q^*。只有当最优的 v^* 或 q^* 被学到之后，这些方法才将最优策略设为相对于这些价值函数贪婪的那个策略。

策略梯度算法（policy gradient）采取了不同的思路，即直接学习最优策略 π^*。在策略梯度算法中，我们同样将策略参数化为一个深度神经网络。该网络要么是从状态空间 $\mathcal{X}$ 到动作空间 $\mathcal{U}$ 的映射，要么是在每个 $x \in \mathcal{X}$ 上给出定义在 $\mathcal{U}$ 上的概率分布。然而，我们需要一个标量值目标函数来训练这个策略 π^*。具体而言，我们需要计算目标函数相对于策略的梯度（更准确地说，是相对于该策略网络参数的梯度）。

假设当前我们已有一个策略 π，则可以用它生成一个长度为 t 的样本回合

(t 任意):

$$z_t = (x_0,\ u_0,\ x_1,\ u_1,\ \ldots,\ x_t,\ u_t), \tag{4.69}$$

其中 $x_0 \sim p_0$，这里 p_0 是某个初始状态分布（它预先给定，和任何策略都无关）；而对任意时刻 t，有 $u_t \sim \pi(\cdot|x_t)$ 由策略 π 决定，且 $x_{t+1} \sim p(\cdot|x_t, u_t)$ 由环境决定。这里我们假设回合可能在有限时间终止，且折扣因子 $\gamma \in [0,1]$，但下面展示的推导同样适用于无限时间范围且折扣因子 $\gamma \in (0,1)$ 的情形。

为了下面的推导，我们对任意时间 t 定义

$$\rho_t^\pi(z_t) := p_0(x_0)\pi(u_0|x_0)p(x_1|x_0,u_0)\ldots p(x_t|x_{t-1},u_{t-1})\pi(u_t|x_t). \tag{4.70}$$

另外，我们把终止于有限时间 T 的回合 z_T 的总回报定义为

$$R(z_T) := \sum_{t=0}^{T} \gamma^t r_{t+1} = \sum_{t=0}^{T} \gamma^t r(x_t, u_t). \tag{4.71}$$

于是我们将目标函数（即由策略 π 决定的 z_T 的期望总回报）记为

$$J(\pi) := \mathbb{E}_{Z_T \sim \rho_T^\pi}[R(Z_T)] = \int R(z_T)\rho_T^\pi(z_T)\, dz_T. \tag{4.72}$$

现在我们的目标是求出 $\nabla_\pi J(\pi)$，即 J 关于 π 的梯度。注意，这里以及下文中的 ∇_π 仍应理解为对 π 的网络参数所取的梯度。

为了把梯度 $\nabla_\pi J(\pi)$ 写成一个在实践中可实现的目标函数，我们首先注意到

$$J(\pi) = \int R(z_T)\rho_T^\pi(z_T)\, dz_T = \sum_{t=0}^{T} \gamma^t \underbrace{\int r(x_t, u_t)\rho^\pi(z_T)\, dz_T}_{=:\, J_t(\pi)} \tag{4.73}$$

这是由 (4.71) 与 (4.72) 联立得到的。我们看到，$J_t(\pi)$ 可以写为

$$
\begin{aligned}
J_t(\pi) &= \int r(x_t, u_t)\rho_T^\pi(z_T)\, dz_T \\
&= \iint r(x_t, u_t)p_0(x_0)\pi(u_0|x_0)\cdots\pi(u_T|x_T)\,(du_T\cdots dx_{t+1})dz_t \\
&= \int \Big[r(x_t, u_t)p_0(x_0)\cdots\pi(u_t|x_t) \\
&\qquad\qquad \cdot\Big(\int p(x_{t+1}|x_t, u_t)\cdots\pi(u_T|x_T)\, du_T\cdots dx_{t+1}\Big)\Big]dz_t \\
&= \int r(x_t, u_t)\underbrace{p_0(x_0)\cdots\pi(u_t|x_t)}_{=\rho_t^\pi(z_t)}\, dz_t,
\end{aligned}
\tag{4.74}
$$

其中第二个等号来自 (4.69) 中 $\rho_T^\pi(z_T)$ 的定义以及 $dz_T = (du_T dx_T\cdots dx_{t+1})dz_t$；第三个等号来自先对 $(u_T, x_T, \ldots, x_{t+1})$ 积分；最后一个等号则是因为

$$
\int p(x_{t+1}|x_t, u_t)\cdots\pi(u_T|x_T)\, du_T\cdots dx_{t+1} = 1. \tag{4.75}
$$

现在我们先考察 $\nabla_\pi J_t(\pi)$，它由下式给出

$$
\begin{aligned}
\nabla_\pi J_t(\pi) &= \nabla_\pi\Big(\int r(x_t, u_t)\rho_t^\pi(z_t)\, dz_t\Big) \\
&= \int r(x_t, u_t)\rho_t^\pi(z_t)\Big(\nabla_\pi \log\rho_t^\pi(z_t)\Big)\, dz_t \\
&= \int r(x_t, u_t)\rho_T^\pi(z_T)\Big(\nabla_\pi \log\rho_t^\pi(z_t)\Big)\, dz_T
\end{aligned}
\tag{4.76}
$$

其中第二个等号利用了 $\nabla_\pi\rho_t^\pi(z_t) = \rho_t^\pi(z_t)\nabla_\pi\log\rho_t^\pi(z_t)$，而最后一个等号再次利用了 (4.75)。我们进一步考察 $\nabla_\pi\log\rho_t^\pi(z_t)$，并注意到由 (4.70) 有

$$
\begin{aligned}
\nabla_\pi\log\rho_t^\pi(z_t) &= \nabla_\pi\Big(\log p_0(x_0) + \sum_{s=0}^{t}\log\pi(u_s|x_s) + \sum_{s=0}^{t}\log p(x_{s+1}|x_s, u_s)\Big) \\
&= \sum_{s=0}^{t}\nabla_\pi\log\pi(u_s|x_s)
\end{aligned}
\tag{4.77}
$$

其中最后一个等号是因为上式括号中的中间一项才包含 π（第一项只包含初始分布 p_0，最后一项只包含由环境决定的状态转移分布）。因此，结合 (4.76) 与

(4.77)，我们得到 $\nabla_\pi J_t(\pi)$ 的最终形式：

$$\nabla_\pi J_t(\pi) = \int r(x_t, u_t)\Big(\sum_{s=0}^{t} \nabla_\pi \log \pi(u_s|x_s)\Big)\rho_T^\pi(z_T)\, dz_T. \tag{4.78}$$

现在有

$$\begin{aligned}
\nabla_\pi J(\pi) &= \nabla_\pi\Big(\sum_{t=0}^{T} \gamma^t J_t(\pi)\Big) \\
&= \sum_{t=0}^{T} \gamma^t \nabla_\pi J_t(\pi) \\
&= \int \sum_{t=0}^{T}\sum_{s=0}^{t} \gamma^t r(x_t, u_t)\nabla_\pi \log \pi(u_s|x_s)\rho_T^\pi(z_T)\, dz_T \\
&= \int \sum_{t=0}^{T}\Big(\sum_{s=t}^{T} \gamma^s r(x_s, u_s)\Big)\nabla_\pi \log \pi(u_t|x_t)\rho_T^\pi(z_T)\, dz_T \\
&= \mathbb{E}_{Z_T\sim\rho_T^\pi}\Big[\sum_{t=0}^{T}\Big(\sum_{s=t}^{T} \gamma^s R_{s+1}\Big)\nabla_\pi \log \pi(U_t|X_t)\Big],
\end{aligned} \tag{4.79}$$

其中第四个等号来自交换关于 t 与 s 的求和顺序，然后再交换符号 s 与 t；最后一个等号则来自对所有 s 有 $R_{s+1} = r(X_s, U_s)$。

注意，(4.79) 右端在实践中是可以近似的。如果采用 Monte Carlo 方法，那么可以通过让智能体按照当前估计策略 π 在环境中行为来采样一个回合 $z_T \sim \rho_T^\pi$（这里时间 T 取决于实际样本回合 z_T 的长度，而非预先固定）。在这种情况下，奖励 $r_{s+1} = r(x_t, u_t)$ 可以被存储并用于估计 (4.79)。如果我们想利用估计的 Q 函数 $q(x_t, u_t)$（它可像上面的 DQN 和双重 DQN 算法那样，被近似为一个输入来自 $\mathcal{X}\times\mathcal{U}$ 的深度神经网络）来近似奖励 $\sum_{s=t}^{T}\gamma^s r_{s+1}$，那么我们注意到 $\sum_{s=t}^{T}\gamma^s r_{s+1} = \gamma^t q(x_t, u_t)$。

现在我们看到，在 (4.79) 中，唯一尚未确定的项是 $\log\pi$。如果 $\mathcal{X}$ 是一个大的有限集，或者是欧氏空间中的连续子集，我们可以把 $\pi(\cdot|x)$ 参数化为在任意 $x \in \mathcal{X}$ 下定义在 $\mathcal{U}$ 上的一个概率分布。更具体地说，如果 $\mathcal{U} = \{u_1, \ldots, u_m\}$ 是一个包含 m 个动作的有限集合，那么我们可以把 $\pi : \mathcal{X} \to \mathbb{R}^m$ 参数化为一个深度神经网络，并将该网络的最后一层操作设为 softmax 函数：

$$\mathrm{softmax}(w) := \frac{1}{\sum_{i=1}^{m} e^{w_i}}(e^{w_1}, e^{w_2}, \ldots, e^{w_m}) \in \mathbb{R}^m \tag{4.80}$$

其中 $w \in \mathbb{R}^m$。在这种情况下，我们知道对每个 $i \in [m]$ 和每个 $x \in \mathcal{X}$，都有 $\pi(u_i|x) > 0$ 且 $\sum_{i=1}^m \pi(u_i|x) = 1$，而在状态 x 下实际执行的动作 u 应当按照定义在 $\mathcal{U}$ 上的分布 $\pi(\cdot|x)$ 来采样。在这种情况下，对任意 $(x,u) \in \mathcal{X} \times \mathcal{U}$，可以直接求出 $\nabla_\pi \pi(u|x)$——即 $\pi(u|x)$ 关于 π 的网络参数的梯度。

如果 $\mathcal{U}$ 是 $\mathbb{R}^m$ 某个连续子集，其中 $m \in \mathbb{N}$，那么我们可以设置两个深度神经网络，$\mu_\pi : \mathcal{X} \to \mathbb{R}^m$ 和 $\sigma_\pi : \mathcal{X} \to (0,\infty)$（我们也可以很容易将 σ_π 推广为取值于 $(0,\infty)^m$，使得 σ_π 可以作为一个对角协方差矩阵的对角元，或者直接取为一个 $m \times m$ 的对称正定矩阵作为协方差矩阵），并令 $\pi(\cdot\,|x) = \mathcal{N}(\,\cdot\,; \mu_\pi(x), \sigma_\pi(x)^2 I_m)$。这样一来，我们可以获得 $\nabla_\pi \pi(u|x)$ 的表达式以及它对 π 的参数的导数。

一旦我们知道如何计算任意 π 下的 $\nabla_\pi J(\pi)$，便可以使用梯度上升法或类似方法来更新策略 π 的网络参数，并逼近最优策略 π^*。

Actor-Critic 算法

注意到我们上面讨论的算法，要么学习状态（或状态–动作）价值函数（例如 DQN 与双重 DQN 算法），要么学习策略（例如策略梯度算法）。Actor-Critic（AC）算法是一种新的方法，它同时学习状态价值函数 v^π 和 actor π。在 AC 算法中，π 和 v^π 都被参数化为深度神经网络，它们分别称为 actor 网络和 critic 网络。critic 网络的目标是评估由 actor 网络 π 所决定的当前策略的价值函数 v^π，而 actor 网络的目标则是利用 critic 网络给出的反馈来改进策略 π。更具体地，回忆一下

$$\nabla_\pi J(\pi) = \mathbb{E}_{Z_T \sim \rho_T^\pi}\Big[\sum_{t=0}^T \Big(\sum_{s=t}^T \gamma^s R_{s+1}\Big) \nabla_\pi \log \pi(u_t|x_t)\Big]. \tag{4.81}$$

在 AC 算法中，我们并不通过采样完整回合来更新 π。相反，我们使用与 TD 方法中样本四元组 $(x_t, u_t, r_{t+1}, x_{t+1})$ 类似的思想，将 $\sum_{s=t}^T \gamma^s R_{s+1}$ 替换为

$$\gamma^t \sum_{s=t}^T \gamma^{s-t} R_{s+1} = \gamma^t (r_{t+1}) + \gamma v^\pi(x_{t+1}),$$

其中 v^π 为 critic 网络。一旦通过梯度上升算法或类似方法更新了 actor 网络 π，我们就转而通过最小化

$$\frac{1}{2}|r_{t+1} + \gamma v^\pi(x_{t+1}) - v^\pi(x_t)|^2. \tag{4.82}$$

来更新 critic 网络 v^π。在实际实现中，上式中的 $r_{t+1}+\gamma v^\pi(x_{t+1})$ 被视作目标状态价值函数，并且 (4.82) 相对于 v^π 参数的梯度只作用于第二个 v^π。也就是说，在 AC 算法中，经验上将 (4.82) 的近似梯度设为 $-(r_{t+1}+\gamma v^\pi(x_{t+1})-v^\pi(x_t))\cdot\nabla_{v^\pi}v^\pi(x_t)$。

信赖域策略优化（TRPO）算法

尽管 actor-critic（AC）算法建立了一个交替更新策略（actor）及其价值函数（critic）的新框架，但其经验计算往往表现得不够稳定。其中一个主要原因在于：相对于网络参数的梯度通常过于激进，因此需要更好的约束来缓解稳定性问题。

回忆一下，策略梯度算法考虑了如下函数关于当前策略 π 的梯度

$$J(\pi)=\mathbb{E}_{Z_T\sim\rho_T^\pi}[R(Z_T)]$$

其中这里我们还假设采样得到的回合具有无限长度，且总回报定义为 $R(z_T)=\sum_{t=0}^{\infty}\gamma^t r(x_t,u_t)$，以便于下面的讨论。我们还注意到，这个函数可以写为

$$J(\pi)=\mathbb{E}_{X\sim p_0}[v^\pi(X)]=\mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi(\cdot|X)}[q^\pi(X,U)]\Big]. \tag{4.83}$$

正如我们所看到的，策略梯度步骤计算不稳定的主要原因在于：当 v^π 或 q^π 被参数化为深度神经网络时，它们存在估计误差。在这种情况下，使用 π 去生成新的回合可能代价高昂，因为许多高风险采样动作可能发生，从而提高数据生成的实际成本。

为了解决这一问题，信赖域策略优化（trust-region policy optimization，TRPO）算法通过引入一个先前估计的策略 π_k（假设它在样本回合生成过程中表现良好），将 (4.83) 中的目标函数改写为：

$$J(\pi)=\mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\frac{\pi(U|X)}{\pi_k(U|X)}q^\pi(X,U)\Big]\Big]. \tag{4.84}$$

容易验证，新的目标函数 (4.84) 与 (4.83) 中的目标函数是完全等价的。

由于 TRPO 算法要求关于策略 π 网络参数的梯度不能过于激进，因此它提出要约束先前的 $\pi_k(\cdot|x)$ 与新的 $\pi(\cdot|x)$ 在每个 $x\in\mathcal{X}$ 处的差异，并假设未知的 Q 函数 q^π 可以由先前获得的 q^{π_k} 很好地近似。更具体地说，TRPO 算法

提出如下方案来更新策略 π：

$$\max_{\pi} \quad \tilde{J}(\pi) := \mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\frac{\pi(U|X)}{\pi_k(U|X)}q^{\pi_k}(X,U)\Big]\Big], \tag{4.85a}$$

$$\text{s.t.} \quad \mathbb{E}_{X\sim p_0}\Big[\mathrm{KL}(\pi_k(\cdot|X),\, \pi(\cdot|X))\Big] \le \delta, \tag{4.85b}$$

其中 $\delta > 0$ 是某个用户选定的超参数。正如我们所见，这一过程会产生一列策略 $\{\pi_k : k \ge 1\}$ 和 Q 函数 $\{q^{\pi_k} : k \ge 1\}$。其优点在于，相邻策略在 KL 散度 (4.85b) 的意义下必须足够接近，以避免对 π_k 的网络参数产生不稳定的更新。

基于上述思想，我们可以设计许多 TRPO 算法的变体。对于原始 TRPO 算法 (4.85)，(4.85a) 中的目标函数可以近似为 π 的仿射函数，而 (4.85b) 可以近似为关于 π 网络参数的一个二次函数的不等式约束。借助这些近似，求解 (4.85) 的计算代价可以大大降低，但与我们稍后介绍的某些其他变体相比，它仍然略显昂贵。

作为与 TRPO 算法及其变体相关的最后一点说明，我们注意到，进一步稳定数值计算的一种实践上很有效的方法是引入所谓的**基线**（baseline）。其原因在于，尽管像 (4.84) 这样的目标函数看上去是对真实目标函数 (4.83) 的无偏估计，但其样本方差往往较大，并可能导致不理想的收敛问题。基线函数被设计成真实目标函数的另一个无偏估计，并且当它从 (4.84) 中减去时，该估计仍保持无偏。与此同时，这个基线函数的目标是降低样本方差，并改善 TRPO 算法及其变体在经验上的收敛稳定性。

设计基线函数有多种选择。一种典型做法是：对任意 $x \in \mathcal{X}$，将策略网络 $\pi(\cdot|x)$ 设计成一个正态分布，这样 (4.85b) 中的 KL 散度就有具体的表达式，从而更易处理。与此同时，我们将从 (4.85a) 中减去的基线函数设计为

$$\begin{aligned}
\mathbb{E}_{X\sim p_0}[v^{\pi_k}(X)] &= \int p_0(x)v^{\pi_k}(x)\,dx \\
&= \int p_0(x)v^{\pi_k}(x)\Big(\int \pi(u|x)\,du\Big)\,dx \\
&= \iint p_0(x)\pi_k(u|x)\frac{\pi(u|x)}{\pi_k(u|x)}v^{\pi_k}(x)\,dudx \\
&= \mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\frac{\pi(U|X)}{\pi_k(U|X)}v^{\pi_k}(X)\Big]\Big],
\end{aligned}$$

它同样是 $J(\pi)$ 的一个无偏估计（假设 v^{π} 可以由 v^{π_k} 很好地近似）。因此，目标函数 (4.85a) 中的 Q 函数 $q^{\pi_k}(X,U)$ 将被替换为 $q^{\pi_k}(X,U) - v^{\pi_k}(X)$。大量数值实验表明，这样做能够减轻计算不稳定的问题。

近端策略优化（PPO）算法

近端策略优化（proximal policy optimization，PPO）算法是上述 TRPO 算法的后续变体。通过对 TRPO 作出一些适度修改，PPO 算法通常展现出更强的学习性能和更快的数值计算速度。PPO 算法的思想是使用 Lagrange 方法的一个放松版本来处理约束 (4.85b)。这里所谓 “放松版本” 是指，Lagrange 乘子的更新是以启发式方式进行的。

更具体地说，PPO 算法的惩罚版本将 TRPO 算法中的方案 (4.85) 修改为一个无约束问题：

$$\tilde{J}(\pi) := \mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\frac{\pi(U|X)}{\pi_k(U|X)}a^{\pi_k}(X,U) - \lambda\mathrm{KL}\Big(\pi_k(\cdot|X),\pi(\cdot|X)\Big)\Big]\Big],$$

其中 $a^{\pi_k}(x,u) := q^{\pi_k}(x,u) - v^{\pi_k}(x)$ 称为**优势函数**（注意它可以为正也可以为负，这取决于 q^{π_k} 中动作 u 所带来的价值是否高于其期望值 v^{π_k}），它在许多无模型强化学习算法中都很常见，而 λ 起到 Lagrange （KKT）乘子的作用。在 PPO 算法中，λ 的更新方式非常简单：若 KL 散度小于 $2\delta/3$，则将 λ 更新为 $\lambda/2$；若 KL 大于 $3\delta/2$，则更新为 2λ；否则保持不变，其中 $\delta > 0$ 同样是用户选取的超参数。

PPO 算法的另一个简单而有效的版本称为 PPO-Clip。在 PPO-Clip 中，目标函数改为如下形式：

$$\tilde{J}(\pi) := \mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi_k(\cdot|X)}\Big[\min\Big(\xi_k(X,U)a^{\pi_k}(X,U),\, \bar{\xi}_k^{\pi}(X,U)a^{\pi_k}(X,U)\Big)\Big]\Big]$$

其中比值 ξ_k 定义为

$$\xi_k^{\pi}(x,u) := \frac{\pi(U|X)}{\pi_k(U|X)},$$

而其裁剪值 $\bar{\xi}_k^{\pi}(x,u)$ 被限制在区间 $[1-\epsilon, 1+\epsilon]$ 内：

$$\bar{\xi}_k^{\pi}(x,u) := \begin{cases} 1-\epsilon, & \text{如果 } \xi_k^{\pi}(x,u) \le 1-\epsilon, \\ 1+\epsilon, & \text{如果 } \xi_k^{\pi}(x,u) \ge 1+\epsilon, \\ \xi_k^{\pi}(x,u), & \text{否则}, \end{cases}$$

其中 $\epsilon \in (0,1)$ 为用户选定的超参数。注意，之所以这样设置裁剪值，是因为优势函数值 a^{π_k} 可能为正，也可能为负。这一 PPO-Clip 版本比放松 Lagrange 方法更易实现，并且在许多经验测试中显得非常有效。

深度确定性策略梯度（DPG）算法

深度确定性策略梯度（deterministic policy gradient，DPG）算法是一种在学习过程中避免对动作进行采样的方法。通过假设策略是贪婪的，DPG 算法可以在 actor-critic 算法中，直接把策略网络（actor 网络）并入价值网络（critic 网络）。

由于贪婪策略都是确定性的，因此 DPG 算法中的 actor 可以设为一个深度神经网络 $w:\mathcal{X}\to\mathcal{U}$。换言之，确定性策略 π 可被写为如下 one-hot 概率：

$$\pi(u|x):=\begin{cases}1,\ \text{在}w(x)\ \text{处，其中}\ w(x)=\arg\max_{u\in\mathcal{U}}q(x,u),\\ 0,\ \text{在每个}u\in\mathcal{U}\setminus\{w(x)\}\text{处}.\end{cases}$$

如果极大值点不唯一，则 $w(x)$ 可以取其中任意一个。由于 actor 网络 w 的输出始终是 $\mathcal{U}$ 中的一个极大值点，因此 DPG 算法中的 critic 可以设为最优 Q 函数 q^*，它同样被参数化为定义在 $\mathcal{X}\times\mathcal{U}$ 上的深度神经网络。假设 critic 网络 q^* 关于参数 $u\in\mathcal{U}$ 可微，则有

$$\partial_u q^*(x,w(x))\nabla_w w(x),$$

其中 $\nabla_w w(x)$ 应理解为 actor 网络 w 关于其网络参数的梯度。

注意，使用贪婪策略 $w(x)$ 会导致

$$q^*(x,w(x))=\max_{\pi\in\Pi}\mathbb{E}_{U\sim\pi(\cdot|x)}[q^*(x,U)]=v^*(x).$$

因此，我们可以定义一个关于 w 的目标函数 J 来更新 w 的参数：

$$J(w):=\mathbb{E}_{X\sim p_0}[v^*(X)]=\mathbb{E}_{X\sim p_0}[q^*(X,w(X))].$$

我们看到，$J(w)$ 关于 w 的梯度可以表示为

$$\nabla_w J(w)=\nabla_w\mathbb{E}_{X\sim p_0}[q^*(X,w(X))]=\mathbb{E}_{X\sim p_0}[\partial_u q^*(X,w(X))\nabla_w w(X)].$$

DPG 算法中最优 Q 函数 q^* 的更新与 Q-learning 算法相似。更具体地说，我们需要让智能体按照当前估计策略 π（即 w）在环境中行为，从而采样形如 $(x_t,u_t,r_{t+1},x_{t+1})$ 的四元组。然后，对于每个样本四元组，我们希望最小化误差

$$\frac{1}{2}|r_{t+1}+\gamma q^*(x_{t+1},w(x_{t+1}))-q^*(x_t,u_t)|^2.$$

在训练过程中，DPG 算法将上述量关于 q^* 网络参数的梯度近似为

$$-\Big(r_{t+1}+\gamma q^*(x_{t+1},w(x_{t+1}))-q^*(x_t,u_t)\Big)\cdot\nabla_{q^*}q^*(x_t,u_t),$$

其中 $\nabla_{q^*}q^*(x_t,u_t)$ 应理解为 q^* 在给定样本对 (x_t,u_t) 处关于其网络参数的梯度。正如我们所见，项 $r_{t+1}+\gamma q^*(x_{t+1},w(x_{t+1}))$ 被视为目标值，而梯度 ∇_{q^*} 并不作用于该项中的 q^*。在实践中，上述梯度会对所有采样四元组取平均，以更新 q^*。

DPG 算法也存在许多变体。例如，我们可以将 π 参数化为某种典型概率分布，例如当 $\mathcal{U}$ 是某个欧氏空间中的连续子集时，将其设为正态分布。这可以改善学习过程中的探索能力，并有可能发现比只学习贪婪动作更优的动作。

Soft Actor-Critic（SAC）算法

深度确定性策略梯度（DPG）算法有时会表现出问题性的收敛行为，因为关于 q^* 网络参数的梯度往往并不准确。应对这一问题的一种解决思路是给当前估计策略 π 加上一个正则项。为此，回忆一下，π 的目标函数可以定义为

$$J(\pi):=\mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi(\cdot|X)}[q^\pi(X,U)]\Big].$$

我们希望对每个 $X\sim p_0$，向上述目标函数中加入加权熵函数

$$\beta H(\pi(\cdot|X))=-\beta\mathbb{E}_{U\sim\pi(\cdot|X)}[\log\pi(U|X)]$$

其中 $\beta>0$ 是熵项 H 的用户选定权重。于是，对 π 进行最大化时所采用的目标函数 J 变为

$$J(\pi)=\mathbb{E}_{X\sim p_0}\Big[\mathbb{E}_{U\sim\pi(\cdot|X)}[q^\pi(X,U)-\beta\log\pi(U|X)]\Big]. \tag{4.86}$$

在理论上，该目标函数 J 的极大化子具有如下闭式表达：

$$\pi(u|x)=\frac{e^{q^\pi(x,u)/\beta}}{\int e^{q^\pi(x,u)/\beta}du} \tag{4.87}$$

对每个 x 都成立。然而，这一闭式形式只在 $\mathcal{X}$ 和 $\mathcal{U}$ 的规模较小或中等、并且它们上的 $q^\pi(x,u)$ 值可以存储为表格时才行。

对于大规模问题，我们需要将 Q 函数 $q^\pi:\mathcal{X}\times\mathcal{U}\to\mathbb{R}$ 参数化为一个深度神经网络。对于生成回合中的一个样本四元组 $(x_t,u_t,r_{t+1},x_{t+1})$，我们希望最

小化如下误差：

$$\frac{1}{2}\Big|\Big(r_{t+1}+\gamma\mathbb{E}_{U_{t+1}\sim\pi(\cdot|x_{t+1})}q^{\pi}(x_{t+1},U_{t+1})\Big)-q^{\pi}(x_t,u_t)\Big|^2. \tag{4.88}$$

在实践中，我们需要对所有形如 $(x_t,u_t,r_{t+1},x_{t+1})$ 的采样四元组上的误差取平均，并对该平均误差相对于 q^{π} 的网络参数求梯度，以更新它。

在 SAC 算法的实现中，还需要考虑高估 q^{π} 的问题。缓解这一问题的一种办法是使用两个深度神经网络 q_1^{π} 和 q_2^{π}，并将如下值（而不是 (4.88) 中大括号内的那一项）作为目标：

$$r_{t+1}+\gamma\min_{j=1,2}\mathbb{E}_{U_{t+1}\sim\pi(\cdot|x_{t+1})}[q_j^{\pi}(x_{t+1},U_{t+1})]. \tag{4.89}$$

然后，将 (4.88)（其中目标设为 (4.89)）关于 $q^{\pi}(x_t,u_t)$ 的网络参数的梯度同时用于更新 q_1^{π} 和 q_2^{π}。

尽管我们能够更新 Q 函数网络 q^{π} 的参数，但如何计算 (4.86) 中 $J(\pi)$ 的极大值点仍是一个问题。其原因在于，闭式表达 (4.87) 中的归一化常数并不容易计算，因为它可能需要在一个大的动作空间 $\mathcal{U}$ 上进行积分。在这种情况下，我们可以对 π 使用重参数化技巧，例如在 $\mathcal{U}$ 是 $\mathbb{R}^m$ 的连续子集时，我们将 $\pi(\cdot|x)$ 设为正态分布 $\mathcal{N}(\cdot;\mu_{\pi}(x),\sigma_{\pi}^2 I_m)$。当 $\mathcal{U}$ 是一个大的有限动作空间时，我们也可以将 $\pi(\cdot|x)$ 参数化为定义在 $\mathcal{U}$ 上的一个概率分布，方法是在每个 $x\in\mathcal{X}$ 处将 π 表示为一个深度神经网络，并令其最后一个操作为 softmax 函数，这与上面的策略梯度算法中的做法类似。无论哪种情况，只要当前估计的 Q 函数 q^{π} 已知，我们就能够计算 $\nabla_{\pi}J(\pi)$，即 J 关于 π 的网络参数的梯度。

4.3 备注与参考文献

4.3.1 强化学习与最优控制

强化学习和最优控制是两个密切相关的序贯决策框架。这两个框架都研究智能体应如何随时间采取行动，以优化给定的累积和/或终端性能目标，其方法也被扩展到多智能体情形。从历史上看，最优控制起源于控制理论和应用数学，而强化学习则源自人工智能和心理学。尽管二者具有共同的基础，这两个领域在建模假设、求解方法以及应用领域方面仍存在差异。

在标准的最优控制设定中，控制系统通常被建模为由常微分方程（ODE）描述的受控动力系统。最优控制问题主要分为两大类：一类是终止时间 T 固

定但终点 $x(T)$ 自由，另一类是终点 $x(T)$ 固定但终止时间 T 自由，如我们在第 3.1 节中所示。在这两种情况下，ODE 的形式以及运行和/或终端奖励（或等价地，代价）都是已知的，最优控制的最优性条件可以通过 Pontryagin 最大值原理和连续时间下的 Hamilton–Jacobi–Bellman 方程来刻画 [12,23]。

相比之下，强化学习通常将决策过程建模为 Markov 决策过程，正如我们在第 4.1 节中所示。在强化学习问题中，转移动态 $p(x'|x,u)$ 和奖励函数可能是未知的，需要在求解最优控制（在强化学习中称为最优策略）的过程中通过学习获得。对于这类问题，很多不同的方法（通常是无模型算法）都被广泛应用。我们在第 4.2 节中展示了一些具有代表性且影响较大的算法。

强化学习与最优控制的理论是紧密联系的。例如，当系统动力学已知且状态空间和动作空间为连续时，强化学习可归结为随机最优控制。在这种情况下，动态规划为两个领域提供了统一框架，而 Bellman 理论支撑了两个领域中的最优性条件和算法发展 [17]。此外，强化学习中的价值函数与最优控制中的对应概念是直接类比的，而策略梯度方法通过 Pontryagin 最大值原理和轨迹优化与最优控制具有紧密联系。文献中还展示了某些强化学习目标与随机控制表述之间的等价性，包括熵正则化控制和路径积分控制 [74,146]。

尽管存在上述联系，强化学习与最优控制之间仍存在重要差异。例如，如前所述，最优控制通常假设动力学已知，而强化学习强调在最少先验知识下从数据中学习。因此，最优控制问题的解的质量高度依赖于模型的准确性，而强化学习算法通常需要在计算效率与数据（包括提供的和生成的）质量之间进行权衡。此外，探索在强化学习中至关重要，而在标准最优控制中通常并不存在，后者通常通过鲁棒或随机建模来处理不确定性。

尽管如此，随着近年来研究的进展，强化学习与最优控制之间的界限开始变得模糊 [115]。特别是，深度学习已成为解决两个领域中大规模和高维问题的有力工具。使用深度神经网络所带来的共同问题，如稳定性、鲁棒性和泛化能力，仍有待进一步的理论和算法发展来解决。为此，理解这两个领域之间的关系有助于设计更加高效且可靠的决策系统。

4.3.2 强化学习的文献

关于强化学习的系统性介绍可参见 [140]。文献 [142] 对经典强化学习算法进行了系统综述。第 4.1 节在很大程度上遵循了 [142] 中理论部分的思路，并在证明细节上进行了显著扩展，同时通过引理 4.1.15 和定理 4.1.17 给出了新的主要结果，从而阐明了 $\bar{v}$ 与最优 Bellman 算子 T^* 的不动点 v^* 之间的关系。强化学习与最优控制及其联系在 [15] 中有深入研究。深度强化学习在过去十

年中迅速发展，近期综述可参见 [6,44,152]。

经典强化学习算法，如策略迭代、价值迭代和时序差分方法，在 [140,142] 中有详细讨论。Q-learning 方法最早在 [155] 中提出，并在 [156] 中证明其收敛到 Bellman 算子的一个不动点。动态规划与强化学习之间的联系最早在 [155] 中被提出。SARSA 算法由 [124] 提出。Monte Carlo 方法的早期工作见 [134]。最初的策略梯度方法 [141] 和 actor-critic 方法 [80] 在深度学习出现之前就已提出，但它们对现代强化学习算法的发展产生了深远影响。开创深度强化学习的深度 Q-learning 算法在 Atari 游戏基准上实现了人类水平的表现 [99]。几乎在同一时期，深度强化学习的另一项突破是 AlphaGo 算法 [132]，它击败了围棋世界冠军，而围棋曾被认为是计算机难以战胜人类高手的经典棋类游戏。深度强化学习还被用于发现更高效的矩阵乘法方法，从而减少乘法运算次数 [41]。

近年来，大量工作致力于改进深度强化学习算法。对偶 Q 网络结构在 [153] 中提出，其中同一网络的两个分支分别预测状态价值函数和优势函数。标准 Q-learning 由于最大化操作会系统性地高估状态价值。为缓解这一问题，提出了双 Q 网络结构 [149]。在 [43] 中，引入了带噪声的深度 Q 网络，通过加入噪声增强随机性并促进探索，同时网络能够在收敛过程中逐渐降低噪声。分布式深度 Q 网络结构在 [10,31] 中提出，旨在学习奖励分布的更完整信息，而不仅仅是其期望。一种结合多种上述 Q 网络特性的深度网络结构被提出，用于提升 ATARI 基准上的训练速度和最终性能 [61]。优先经验回放方法用于提升训练稳定性和速度，见 [127]。优势函数的扩展也在 [129] 中进行了研究。这些方法都是将深度学习用于价值函数逼近的变体，包括状态价值函数、Q 函数（状态-动作价值函数）以及优势函数。

深度神经网络也被用于学习强化学习问题的最优策略。信赖域策略优化（TRPO）算法在 [128] 中提出。随后提出的近端策略优化（PPO）算法见 [130]。在 [133] 中提出了深度确定性策略梯度（DDPG）算法。一种基于 DDPG 且能够处理连续动作空间的 actor-critic 无模型算法在 [84] 中提出。Soft actor-critic 算法见 [58,59]。该领域正处于快速发展阶段。预计在不久的将来会出现更强大、更稳定、更鲁棒的深度强化学习算法。

第五章　生成模型

本章介绍**生成模型**（generative model），这是过去十年中出现的一类重要的深度学习方法，用于模拟复杂的数据分布以及生成它们的新样本。生成模型具有广泛的现实应用，可用于生成逼真的文本、音乐、图像、视频等内容，这类内容通常称为人工智能生成内容（AI-generated content，AIGC）。

生成建模背后的原型数学问题可以表述如下。设给定数据集 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$，其中包含 M 个从某个未知概率分布 p_{data}（定义在 $\mathbb{R}^d$ 上）独立抽取的样本。生成建模的目标是仅通过 $\mathcal{D}$ 学习如何生成 p_{data} 的新样本。

生成模型是一个很有挑战的问题。首先，我们只有有限数据集 $\mathcal{D}$ 可用，而 p_{data} 的解析形式是未知的。其次，样本的维数 d 可能极其大。例如，每个 x_i 可能表示一幅图像，此时 d 往往达到 10^4 甚至更高。

在条件生成的设定下，这个问题会变得更加复杂，此时目标是针对任意用户指定的条件变量（通常称为**提示词**）y，从未知条件分布 $p_{\text{data}}(\cdot \mid y)$ 中生成样本。其目的是生成在性质上与用户需求或偏好相一致的样本。

本章主要关注无条件生成模型，因为它们的理论基础和方法发展更加成熟。此外，许多现有的条件生成模型都是建立在无条件生成的模型和方法之上的。

近年来，生成建模取得了快速进展，但目前仍不存在一个能够完全解决该问题的统一框架。本章将介绍文献中几种最具影响力的生成建模范式，并详细考察它们的构造与性质。我们首先从经典的变分自编码器框架以及与之密切相关、几乎同时出现的生成对抗网络方法开始。随后讨论更近期的发展，包括扩散模型、概率密度控制以及流匹配方法，这些方法利用（随机）微分方程来生成样本。最后，我们讨论与这些模型相关的实践和理论问题，并简要介绍几种条件生成的方法。

5.1 变分自编码器

变分自编码器（Variational Autoencoder，VAE）是一种网络架构，它可以利用由 p_{data} 的有限个独立样本组成的数据集 $\mathcal{D}$ 来生成 p_{data} 的新样本。VAE 模型由两个函数组成，称为编码器（encoder）和解码器（decoder）。编码器和解码器通常分别参数化为深度神经网络，记为 g_η 和 f_θ，其网络参数分别为 η 和 θ。

编码器 g_η 学习将每个数据样本 $x \in \mathcal{D}$ 映射到一个**潜变量**（latent variable）z，其中 z 位于**潜在空间**（latent space）$\mathbb{R}^m$ 中。编码器的输出 $g_\eta(x)$ 通常不是在给定输入 x 时直接确定 z，而是被设定为某个概率分布的参数。例如一个典型的做法是令 $g_\eta(x)$ 为向量 $\mu_\eta(x) \in \mathbb{R}^m$ 和正标量 $\sigma_\eta(x) > 0$ 所组成的一对（或者也可以是 $\mathbb{R}^m$ 中的正向量，作为对角协方差矩阵的对角元，甚至直接是一个 m-维对称正定协方差矩阵，但考虑到这种推广不复杂，我们这里只讨论 σ_η 为标量值的简单情形）。然后将 $\mu_\eta(x)$ 和 $\sigma_\eta(x)$ 分别设为某个高斯分布的均值和标准差，并从该分布中抽取潜变量 z。更准确地说，z 是从条件概率分布 $q_\eta(\,\cdot\,|\,x) := \mathcal{N}(\,\cdot\,;\,\mu_\eta(x), \sigma_\eta(x)^2 I_m)$ 中抽取的样本。在实践中，这可以很容易地实现为 $z = \mu_\eta(x) + \sigma_\eta(x)\epsilon$，其中 $\epsilon \sim \mathcal{N}(0, I_m)$。

解码器 f_θ 旨在将任意 $z \in \mathbb{R}^m$ 映射到一个新样本 $x' \in \mathbb{R}^d$。同样，解码器的输出 $f_\theta(z)$ 对于每个输入 z 也不是直接确定 x'，而是被设为向量 $\mu_\theta(z) \in \mathbb{R}^d$ 和正标量 $\sigma_\theta(z) > 0$ 所组成的一对。然后将新样本 x' 设为从条件概率分布 $p_\theta(\cdot|z) := \mathcal{N}(\,\cdot\,;\,\mu_\theta(z), \sigma_\theta(z)^2 I_d)$ 中抽取的样本。在实践中，这实现为 $x' = \mu_\theta(z) + \sigma_\theta(z)\epsilon$，其中 $\epsilon \sim \mathcal{N}(0, I_d)$。

对编码器 g_η 的一个附加要求是，其边缘分布（marginal distribution）

$$q_\eta(z) := \int q_\eta(z|x) p_{\text{data}}(x)\, dx$$

应当近似于标准高斯分布 $\mathcal{N}(0, I_m)$，从而在 g_η 和 f_θ 训练完成后，我们可以很容易地从这个边缘分布 q_η 中抽样，并将解码器 f_θ 应用于这些样本上，以获得服从 p_{data} 的新样本。

我们前面介绍了若干概率分布：数据概率分布 p_{data}（其解析形式未知，但我们拥有 M 个来自它的 i.i.d. 样本）定义在 $\mathbb{R}^d$ 上，潜在空间 $\mathbb{R}^m$ 上的条件分布 $q_\eta(\cdot|x)$ 与边缘分布 $q_\eta(\cdot)$，以及定义在 $\mathbb{R}^d$ 上的条件概率 $p_\theta(\cdot|z)$。由此，我们还得到定义在 $\mathbb{R}^d$ 上的边缘分布 $p_{\eta,\theta}(\cdot)$：

$$p_{\eta,\theta}(x) = \int p_\theta(x|z) q_\eta(z)\, dz, \quad \forall\, x \in \mathbb{R}^d, \tag{5.1}$$

原始样本所在的空间 $\mathbb{R}^d$

- 给定数据集 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$，其中包含 M 个样本它们是从 $\mathbb{R}^d$ 上某个未知概率分布 p_{data} 中独立抽取的。

↓ 编码器网络 g_η，网络参数 η

潜在空间 $\mathbb{R}^m$

- 对于每个 $x \in \mathcal{D}$，即 $x \sim p_{\text{data}}$，潜变量 z 是一个样本它服从 $\mathbb{R}^m$ 上的条件概率 $q_\eta(\cdot|x)$。
- 在 VAE 中，具有网络参数 η 的编码器 g_η 通常设为 $g_\eta(x) = (\mu_\eta(x), \sigma_\eta(x))$，使得 $q_\eta(\cdot|x) = \mathcal{N}(\cdot; \mu_\eta(x), \sigma_\eta(x)^2 I_m)$。因而可以实现为 $z = \mu_\eta(x) + \sigma_\eta(x)\epsilon$，其中 $\epsilon \sim \mathcal{N}(0, I_m)$。
- 在 VAE 中，边缘概率 $q_\eta(\cdot) = \int q_\eta(\cdot|x) p_{\text{data}}(x) dx$ 被假设为近似 $\mathcal{N}(\cdot; 0, I_m)$，从而在 f_θ 训练完成后，能够容易地从 $q_\eta(\cdot)$ 生成新样本 z。

↓ 解码器网络 f_θ，网络参数 θ

生成样本所在的空间 $\mathbb{R}^d$

- 对于每个潜变量样本 $z \sim q_\eta(\cdot)$，我们使用 $f_\theta(z)$ 生成一个 x 它服从 $\mathbb{R}^d$ 上的条件概率 $p_\theta(\cdot|z)$。
- 在 VAE 中，具有网络参数 θ 的解码器 f_θ 通常设为 $f_\theta(z) = (\mu_\theta(z), \sigma_\theta(z))$，使得 $p_\theta(\cdot|z) = \mathcal{N}(\cdot; \mu_\theta(z), \sigma_\theta(z)^2 I_d)$。因而可以实现为 $x = \mu_\theta(z) + \sigma_\theta(z)\epsilon$，其中 $\epsilon \sim \mathcal{N}(0, I_d)$。
- 这给出联合概率 $p_{\eta,\theta}(x, z) = p_\theta(x|z) q_\eta(z)$ 以及边缘概率 $p_{\eta,\theta}(x) = \int p_{\eta,\theta}(x, z) dz$。
- 在 VAE 中，对每个 $x \in \mathbb{R}^d$，要求后验概率 $p_{\eta,\theta}(\cdot|x) = p_{\eta,\theta}(x|\cdot) q_\eta(\cdot) / p_{\eta,\theta}(x)$ 和 $q_\eta(\cdot|x)$ 在 KL 意义下接近。
- 在 VAE 中，边缘概率 $p_{\eta,\theta}$ 要求在 KL 意义下接近 p_{data}。注意 $p_{\eta,\theta}$ 是 $x = \mu_\theta(z) + \sigma_\theta(z)\epsilon$ 的边缘概率，其中 $z \sim q_\eta = \mathcal{N}(0, I_m)$ 且 $\epsilon \sim \mathcal{N}(0, I_d)$，因而这一要求强制这样的生成样本 x 服从 p_{data} 分布。

图 5.1: 变分自编码器（VAE）框架中的空间、映射（编码器与解码器）以及概率分布。

其中 $p_{\eta,\theta}(x,z) := p_\theta(x|z)q_\eta(z)$ 是 (x,z) 的联合分布。根据 Bayes 法则，对于任意 $x \in \mathbb{R}^d$，我们还可以得到后验分布 $p_{\eta,\theta}(z|x)$：

$$p_{\eta,\theta}(z|x) = \frac{p_\theta(x|z)q_\eta(z)}{p_{\eta,\theta}(x)}, \quad \forall\, z \in \mathbb{R}^m. \tag{5.2}$$

将上述条件分布设为高斯分布不仅引入了多样性（相比于使用确定性映射），而且使得实际实现更为方便。例如，两个高斯分布之间的 Kullback–Leibler（KL）散度有具体的表达式，因此可以直接写出网络训练中的损失函数。我们稍后将在本节中展示这些细节。

除了上述设定外，我们还需要编码器和解码器所诱导的概率分布之间满足以下两个重要条件：

- 生成样本的边缘概率分布应当等于原始数据的概率分布。即，

$$p_{\eta,\theta}(x) = p_{\text{data}}(x) \tag{5.3}$$

  对于所有 $x \in \mathbb{R}^d$ 都成立。

- 对于任意 $x \in \mathbb{R}^d$，假设 x 是一个新生成样本，那么给定 x 时潜变量 z 的后验分布 $p_{\eta,\theta}(z|x)$，应当与 x 是一个 p_{data} 的真样本时对应的潜变量 z 的条件概率 $q_\eta(z|x)$ 相一致。也就是说，对于任意 $x \in \mathbb{R}^d$ 和 $z \in \mathbb{R}^m$，都有

$$p_{\eta,\theta}(z|x) = q_\eta(z|x). \tag{5.4}$$

在 VAE 框架中，(5.3) 和 (5.4) 都是通过最小化两边之间的 KL 散度来实现的。

数据空间 $\mathbb{R}^d$、潜在空间 $\mathbb{R}^m$、编码器 g_η 和解码器 f_θ 之间的关系，以及上述讨论中涉及到的所有概率分布，已在图 5.1 中进行了总结。

现在我们开始推导构造编码器网络参数 η 与解码器网络参数 θ 的目标函数所需的数学形式，这些目标函数基于前述 KL 散度。更准确地说，为了得到 (5.3)，我们希望最小化

$$\begin{aligned}
\mathrm{KL}(p_{\text{data}}, p_{\eta,\theta}) &= \int p_{\text{data}}(x) \log \frac{p_{\text{data}}(x)}{p_{\eta,\theta}(x)}\, dx \\
&= -H(p_{\text{data}}) - \int p_{\text{data}}(x) \log p_{\eta,\theta}(x)\, dx \\
&= -H(p_{\text{data}}) - \mathbb{E}_{X\sim p_{\text{data}}}[\log p_{\eta,\theta}(X)],
\end{aligned} \tag{5.5}$$

其中 $H(p_{\text{data}})$ 是 p_{data} 的熵，并且与 θ 和 η 无关。

因此，如果我们想对 (θ,η) 最小化 $\mathrm{KL}(p_{\text{data}}, p_{\eta,\theta})$，只需要求解

$$\max_{\theta,\eta}\ \mathbb{E}_{X\sim p_{\text{data}}}[\log p_{\eta,\theta}(X)]. \tag{5.6}$$

数值 $\log p_{\eta,\theta}(x)$ 被称为点 x 处的**证据**（evidence）。(5.6) 的目标是最大化所有服从 p_{data} 的给定样本的证据值的期望。稍后我们将进一步考察证据 $\log p_{\eta,\theta}(x)$，并说明它与 (5.4) 中两个概率分布之间的联系。

为了得到 (5.4)，我们也尝试最小化左右两个概率分布之间的 KL 散度。也就是说，对每个 $x\in\mathbb{R}^d$，我们希望最小化

$$\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x)) = \int q_\eta(z|x)\log\frac{q_\eta(z|x)}{p_{\eta,\theta}(z|x)}\,dz \tag{5.7}$$

相对于 θ 和 η。如果我们希望对所有服从 p_{data} 的样本最小化这个 KL，则需要求解

$$\min_{\theta,\eta}\ \mathbb{E}_{X\sim p_{\text{data}}}[\mathrm{KL}(q_\eta(\cdot|X),\ p_{\eta,\theta}(\cdot|X))]. \tag{5.8}$$

现在我们有两个问题，即 (5.6) 和 (5.8)，需要求解。然而，它们之间关系密切，可以合并为一个问题。

为了看出这一点，我们需要考察证据 $\log p_{\eta,\theta}(x)$ 与 $\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x))$ 之间的关系——这两个函数的期望分别出现在 (5.6) 和 (5.8) 中：对每个 $x\in\mathbb{R}^d$，有

$$\begin{aligned}
&\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x))\\
&\quad= \int q_\eta(z|x)\log\frac{q_\eta(z|x)}{p_{\eta,\theta}(z|x)}\,dz\\
&\quad= \int q_\eta(z|x)\log\Big(\frac{q_\eta(z|x)}{q_\eta(z)}\frac{p_{\eta,\theta}(x)}{p_\theta(x|z)}\Big)\,dz\\
&\quad= \int q_\eta(z|x)\log\frac{q_\eta(z|x)}{q_\eta(z)}\,dz - \int q_\eta(z|x)\log p_\theta(x|z)\,dz + \log p_{\eta,\theta}(x)\\
&\quad= \mathrm{KL}(q_\eta(\cdot|x),\ q_\eta) - \int q_\eta(z|x)\log p_\theta(x|z)\,dz + \log p_{\eta,\theta}(x),
\end{aligned} \tag{5.9}$$

其中第二个等号来自 Bayes 法则

$$p_{\eta,\theta}(z|x) = \frac{p_\theta(x|z)q_\eta(z)}{p_{\eta,\theta}(x)}.$$

将 (5.9) 重新整理可得

$$\log p_{\eta,\theta}(x) = \mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x)) + \underbrace{\int q_\eta(z|x)\log p_\theta(x|z)\,dz - \mathrm{KL}(q_\eta(\cdot|x),\ q_\eta(\cdot))}_{=:\,\mathrm{ELBO}_{\eta,\theta}(x)}. \tag{5.10}$$

因此，证据 $\log p_{\eta,\theta}(x)$ 正好等于 $\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x))$ 与所谓的**证据下界**（evidence lower bound，ELBO）之和，其中 ELBO 定义为

$$\mathrm{ELBO}_{\eta,\theta}(x) := \int q_\eta(z|x)\log p_\theta(x|z)\,dz - \mathrm{KL}(q_\eta(\cdot|x),\ q_\eta(\cdot)). \tag{5.11}$$

我们注意到，对任意 x，都有 $\mathrm{ELBO}_{\eta,\theta}(x) \le \log p_{\eta,\theta}(x)$，因为根据 (5.10)，它们的差就是 $\mathrm{KL}(q_\eta(\cdot|x),\ p_{\eta,\theta}(\cdot|x))$，而两个概率分布之间的 KL 散度总是非负的。

ELBO 最初之所以得名，是因为文献中通常按如下方式推导：对任意 $x \in \mathbb{R}^d$，有

$$\begin{aligned}
\log p_{\eta,\theta}(x) &= \log \int p_{\eta,\theta}(x,z)\,dz \\
&= \log \int \frac{p_{\eta,\theta}(x,z)}{q_\eta(z|x)} q_\eta(z|x)\,dz \\
&\ge \int q_\eta(z|x)\log \frac{p_{\eta,\theta}(x,z)}{q_\eta(z|x)}\,dz \\
&= \int q_\eta(z|x)\log \Big(\frac{q_\eta(z)}{q_\eta(z|x)} p_\theta(x|z)\Big)\,dz \\
&= \int q_\eta(z|x)\log p_\theta(x|z)\,dz - \mathrm{KL}(q_\eta(\cdot|x),\ q_\eta(\cdot)) \\
&= \mathrm{ELBO}_{\eta,\theta}(x),
\end{aligned}$$

其中不等式是基于 Jensen 不等式，因为对数函数是凹函数；第三个等号来自 $p_{\eta,\theta}(x,z) = p_\theta(x|z)q_\eta(z)$。因此，如果我们想最大化证据 $\log p_{\eta,\theta}(x)$，就可以尝试最大化 $\mathrm{ELBO}_{\eta,\theta}(x)$。

现在我们看到，(5.10) 还给出了 ELBO 的另一种解释：最大化 $\mathrm{ELBO}_{\eta,\theta}(x)$ 不仅会增大证据 $\log p_{\eta,\theta}(x)$，还会减小每个 x 处的 $\mathrm{KL}(q_\eta(\cdot|x), p_{\eta,\theta}(\cdot|x))$。更准

确地，对 (5.10) 两边取期望，有

$$\mathbb{E}_{X\sim p_{\text{data}}}[\text{ELBO}_{\eta,\theta}(X)] = \mathbb{E}_{X\sim p_{\text{data}}}[\log p_{\eta,\theta}(X)] \\ - \mathbb{E}_{X\sim p_{\text{data}}}[\text{KL}(q_\eta(\cdot|X),\ p_{\eta,\theta}(\cdot|X))].$$

因此，最大化问题 (5.6) 与最小化问题 (5.8) 可以通过同一种方式同时实现——求解 ELBO 最大化问题：

$$\max_{\theta,\eta}\ \mathbb{E}_{X\sim p_{\text{data}}}[\text{ELBO}_{\eta,\theta}(X)]. \tag{5.12}$$

在本节余下部分，我们将进一步考察 $\text{ELBO}_{\eta,\theta}$ 的结构，并利用给定数据 $\mathcal{D}$ 将 (5.12) 写成一个可实现的优化问题。

回顾 (5.11) 中 ELBO 的定义，我们知道 (5.12) 中的目标函数可以写为

$$\mathbb{E}_{X\sim p_{\text{data}}}[\text{ELBO}_{\eta,\theta}(X)] = \underbrace{\iint p_{\text{data}}(x)q_\eta(z|x)\log p_\theta(x|z)\,dzdx}_{=:\,A(\theta,\eta)} \\ - \underbrace{\int p_{\text{data}}(x)\text{KL}(q_\eta(\cdot|x), q_\eta(\cdot))\,dx}_{=:\,B(\eta)} \tag{5.13}$$

根据编码器 g_η 的设定，有

$$q_\eta(z|x) = \mathcal{N}(z;\mu_\eta(x),\sigma_\eta(x)^2 I_m). \tag{5.14}$$

因此，我们可以从标准高斯 $\mathcal{N}(0, I_m)$ 中抽取 ϵ，并设

$$z_\eta(x,\epsilon) = \mu_\eta(x) + \sigma_\eta(x)\epsilon. \tag{5.15}$$

于是我们知道，$z_\eta(x,\epsilon)$ 是从概率分布 $q_\eta(\cdot|x)$ 中抽取的一个样本。此外，对于任意 $z\in\mathbb{R}^m$，根据解码器 f_θ 的设定，有

$$p_\theta(x|z) = \mathcal{N}(x;\mu_\theta(z),\sigma_\theta(z)^2 I_d). \tag{5.16}$$

因此，

$$\log p_\theta(x|z) = -\frac{|x-\mu_\theta(z)|^2}{2\sigma_\theta(z)^2} - \frac{d}{2}\log\left(2\pi\sigma_\theta(z)^{2d}\right).$$

现在我们可以将 (5.13) 中定义的 $A(\theta,\eta)$ 重写为

$$A(\theta,\eta)=\iint p_{\text{data}}(x)\mathcal{N}(\epsilon;0,I_m)\log p_\theta(x|z_\eta(x,\epsilon))\,d\epsilon dx. \tag{5.17}$$

其中 $z_\eta(x,\epsilon)$ 和 $\log p(x|z)$ 分别由 (5.15) 与 (5.16) 给出。在实际实现中，由于我们只拥有若干来自 p_{data} 的样本，因此可以使用 Monte Carlo 积分来近似 (5.17) 中的 $A(\theta,\eta)$：对于每个样本 $x\in\mathcal{D}$（其中 $\mathcal{D}$ 包含从 p_{data} 中独立抽取的 M 个样本），我们抽取一个 $\epsilon\sim\mathcal{N}(0,I_d)$，并使用

$$\frac{1}{M}\sum_{(x,\epsilon)}\mathcal{N}(\epsilon;0,I_m)\log p_\theta(x|z_\eta(x,\epsilon)),$$

来近似 (5.17) 中的 $A(\theta,\eta)$，其中求和是对这 M 组 (x,ϵ) 样本对进行的。

接下来，我们注意到 (5.13) 中定义的 $B(\eta)$ 的被积函数 $\mathrm{KL}(q_\eta(\cdot|x),q_\eta(\cdot))$ 具有闭式表达，因为 $q_\eta(\cdot|x)$ 与 $q_\eta(\cdot)$ 都是高斯分布。具体而言，有

$$q_\eta(z|x)=\mathcal{N}(z;\mu_\eta(x),\sigma_\eta(x)^2I_m)$$

以及

$$q_\eta(z)=\mathcal{N}(z;0,I_m).$$

因此，我们可以得到它们 KL 散度的表达式（同维高斯分布之间的 KL 散度见 (A.12)）：

$$\mathrm{KL}(q_\eta(\cdot|x),q_\eta(\cdot))=\frac{1}{2}\Big(m\sigma_\eta(x)^2+|\mu_\eta(x)|^2-m-2m\log\sigma_\eta(x)\Big).$$

于是，

$$B(\eta)=\frac{1}{2}\int p_{\text{data}}(x)\Big(m\sigma_\eta(x)^2+|\mu_\eta(x)|^2-m-2m\log\sigma_\eta(x)\Big)\,dx, \tag{5.18}$$

它同样可以通过 Monte Carlo 积分近似为

$$\frac{1}{2}\sum_{x\in\mathcal{D}}\Big(m\sigma_\eta(x)^2+|\mu_\eta(x)|^2-m-2m\log\sigma_\eta(x)\Big).$$

现在我们已经得到了 $A(\theta,\eta)$ 与 $B(\eta)$ 的表达式 (5.17) 和 (5.18)，以及它们的可实现形式，因此可以关于 (θ,η) 最大化 (5.13) 中定义的 ELBO 期望。

在 (θ,η) 优化完成后，我们可以按如下方式从未知概率分布 p_{data} 中生成

新样本。首先模拟 $z \sim \mathcal{N}(0, I_m)$ 和 $\epsilon \sim \mathcal{N}(0, I_d)$，并设 $x = \mu_\theta(z) + \sigma_\theta(z)\epsilon$，则 x 就是一个服从概率分布 p_{data} 的新样本。我们可以独立且同时地生成任意多个这样的 (z, ϵ) 对，并计算对应的 x，这些 x 都是服从概率分布 p_{data} 的新样本。

5.2 生成对抗网络

生成对抗网络，Generative Adversarial Networks（GANs）是一种与变分自编码器框架大约同时发展起来的深度网络架构。GAN 的思想直接而简单。与前面一样，设 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$ 是一个给定数据集，由从 $\mathbb{R}^d$ 上某个未知概率分布 p_{data} 独立抽取的 M 个样本组成。GAN 的思想是构造一个**生成器**（generator）函数 $g_\theta : \mathbb{R}^d \to \mathbb{R}^d$，并将其参数化为带参数 θ 的深度神经网络，使得对于任意 $z \sim \mathcal{N}(0, I_d)$，$g_\theta(z)$ 都是一个服从概率分布 p_{data} 的样本。

现在的问题是，如何确定参数 θ，使得 $g_\theta(z)$ 确实服从未知分布 p_{data}。为此，我们注意到我们既有来自 $\mathcal{D}$ 的真实样本，也有由 $z \sim \mathcal{N}(0, I_d)$ 生成的样本 $g_\theta(z)$。因此，我们可以给这些样本赋予二元标签，即给来自 $\mathcal{D}$ 的每个样本赋标签 1，给每个生成的（伪造的）样本 $g_\theta(z)$ 赋标签 0。更具体地，我们记

$$\mathcal{D}_{\text{real}} := \Big\{(x_i, 1) : x_i \in \mathcal{D},\ i \in [M]\Big\} \tag{5.19}$$

以及

$$\mathcal{D}_{\text{fake}} := \Big\{(g_\theta(z_j), 0) : z_j \sim \mathcal{N}(0, I_d),\ j \in [N]\Big\}, \tag{5.20}$$

其中 1 和 0 表示样本的标签。在 (5.20) 中，我们假设生成样本的数量为任意的 N。

有了 (5.19) 中的两个数据集 $\mathcal{D}_{\text{real}}$ 和 (5.20) 中的 $\mathcal{D}_{\text{fake}}$ 后，GAN 进一步引入了一个分类器（classifier），也称为**判别器**（discriminator），它被参数化为一个带参数 η 的**对抗网络**（adversarial network）$a_\eta : \mathbb{R}^d \to (0, 1)$。这个对抗网络 a_η 的训练目标是区分 (5.20) 中来自 $\mathcal{D}_{\text{fake}}$ 的生成样本与 (5.19) 中来自 $\mathcal{D}_{\text{real}}$ 的真实样本。GAN 的示意图见图 5.2。

a_η 的功能建立在 Bernoulli 概率分布之上。回忆一下，对于任意 $p \in (0, 1)$ 和 $X \sim \text{Bernoulli}(p)$，$X = 1$ 的概率为 p，而 $X = 0$ 的概率为 $1 - p$。因此，X 的概率质量函数可以写为 $\text{Prob}(X = x) = p^x(1 - p)^{1-x}$，其中 $x = 0$ 或 1。基于这一表述，我们可以将 $a_\eta(x) \in (0, 1)$ 解释为样本 x 为真实样本的概率。具体而言，对于任意来自 $\mathcal{D}_{\text{real}}$ 的 x_i，这一概率质量函数为 $a_\eta(x_i)$；而对于任意

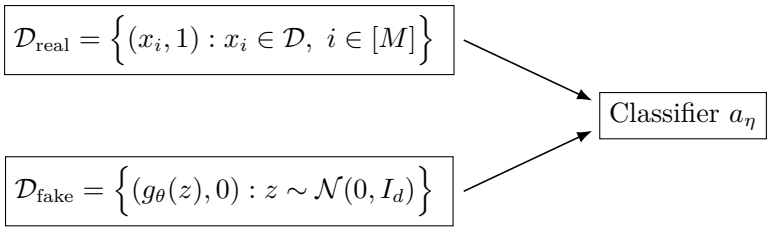


图 5.2: 生成对抗网络（GANs）的示意图。数据集 $\mathcal{D}_{\text{real}}$ 由 $\mathcal{D}$ 中给定的 M 个真实样本及其标签 1 组成。数据集 $\mathcal{D}_{\text{fake}}$ 由生成样本 $g_\theta(z)$ 组成，其标签为 0，其中 $g_\theta : \mathbb{R}^d \to \mathbb{R}^d$ 是参数为 θ 的生成器网络，且 $z \sim \mathcal{N}(0, I_d)$。分类器 $a_\eta : \mathbb{R}^d \to (0, 1)$ 是参数为 η 的对抗网络，它对每个输入样本赋予一个位于 $(0, 1)$ 中的概率值，表示它认为该样本是真实样本的概率。

来自 $\mathcal{D}_{\text{fake}}$ 的 $g_\theta(z_j)$，则为 $a_\eta(g_\theta(z_j))$。由于 $\mathcal{D}_{\text{real}}$ 和 $\mathcal{D}_{\text{fake}}$ 中的所有样本都是独立抽取的，并且它们的标签已知分别为 1 和 0，因此我们可以构造它们的联合概率分布为

$$\Big(\prod_{i=1}^{M} a_\eta(x_i)\Big)^{1/M} \Big(\prod_{j=1}^{N} \big(1 - a_\eta(g_\theta(z_j))\big)\Big)^{1/N}, \tag{5.21}$$

其中我们去掉了所有等于 1 的项（例如 $(1 - a_\eta(x_i))^0 = a_\eta(g_\theta(z_j))^0 = 1$），并依据数据规模 M 和 N 用指数对该量做了归一化。对 (5.21) 两边取对数，得到

$$E(\theta, \eta) := \frac{1}{M} \sum_{i=1}^{M} \log a_\eta(x_i) + \frac{1}{N} \sum_{j=1}^{N} \log(1 - a_\eta(g_\theta(z_j))). \tag{5.22}$$

现在我们来解释 (5.22) 中 $E(\theta, \eta)$ 的含义。如果生成器 g_θ 固定，则分类器 a_η 应被训练为给所有 x_i 赋值 1（或非常接近 1），并给所有 $g_\theta(z_j)$ 赋值 0（或非常接近 0）。这等价于在固定 θ 时，关于 η 最大化 $E(\theta, \eta)$。通过这种方式，分类器 a_η 可以提高其判别能力，从而正确地区分一个样本是真实的还是生成的。

反过来，如果分类器 a_η 固定，则应训练生成器 g_θ 去迷惑分类器，即找到一个参数 θ，使得对所有 x_i 和 z_j 都有 $a_\eta(x_i) = a_\eta(g_\theta(z_j)) = \frac{1}{2}$。这等价于在固定 η 时，关于 θ 最小化 $E(\theta, \eta)$。通过这种方式，生成器 g_θ 可以提高其生成能力，使得对分类器 a_η 来说，$g_\theta(z_j)$ 与真实样本看起来没有区别，因此 a_η 无论接收到什么样本都只能赋予概率 $\frac{1}{2}$，即 a_η 完全无法判断该样本是真实的还是生成的。

基于上述解释，将训练 θ 和 η 的问题设为如下鞍点（极小极大）问题是

很自然的：

$$\min_{\theta} \max_{\eta} \; E(\theta, \eta). \tag{5.23}$$

在实践中，这个鞍点问题 (5.23) 可以通过交替地使用梯度下降更新 θ，然后梯度上升更新 η 来求解。其收敛可能不稳定，但在实际实现中有一些缓解这一问题的方法。

我们还可以进一步分析鞍点问题 (5.23) 的含义。设 θ 固定，且生成器 $g_\theta : \mathbb{R}^d \to \mathbb{R}^d$ 是一个可逆映射。记 p_θ 为当 $z \sim \mathcal{N}(0, I_d)$ 时生成样本 $g_\theta(z)$ 的概率分布。则根据变量替换公式，我们知道

$$p_\theta(x) = \mathcal{N}(g_\theta^{-1}(x); 0, I_d) \det(\nabla g_\theta^{-1}(x)). \tag{5.24}$$

若 $M, N \to \infty$，即我们拥有无限多的真实样本和生成样本，则 (5.22) 中的 $E(\theta, \eta)$ 变为

$$E(\theta, \eta) = \int p_{\text{data}}(x) \log a_\eta(x)\, dx + \int p_\theta(x) \log(1 - a_\eta(x))\, dx. \tag{5.25}$$

对于任意固定的 θ，我们发现 (5.25) 中 $E(\theta, \cdot)$ 的临界点 η^* 应满足

$$\frac{p_{\text{data}}(x)}{a_{\eta^*}(x)} - \frac{p_\theta(x)}{1 - a_{\eta^*}(x)} = 0,$$

这意味着

$$a_{\eta^*}(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_\theta(x)}$$

对每个 x 都成立。将这一最优的 η^* 代回 $E(\theta, \eta)$，可得

$$\begin{aligned} E(\theta, \eta^*) &= \int p_{\text{data}}(x) \log a_{\eta^*}(x)\, dx + \int p_\theta(x) \log(1 - a_{\eta^*}(x))\, dx \\ &= 2\,\mathrm{JS}(p_{\text{data}}, p_\theta) - 2\log 2, \end{aligned}$$

其中 $\mathrm{JS}(p, q) := (\mathrm{KL}(p, r) + \mathrm{KL}(q, r))/2$ 称为两个概率分布 p 和 q 之间的 **Jensen–Shannon**（JS）散度，而 $r := (p + q)/2$ 是这两个分布的点态平均。（关于 JS 散度的更多细节见附录 A.3。）因此，鞍点问题 (5.23) 等价于

$$\min_{\theta} \; \mathrm{JS}(p_{\text{data}}, p_\theta), \tag{5.26}$$

形式 (5.26) 表明，可以使用两种概率分布之间相当广泛的一类差异或散

度，作为学习生成器网络 g_θ 参数 θ 的目标函数。在原始 GAN 工作之后，一个流行的选择是使用 Wasserstein-1 距离，即将 (5.26) 改为

$$\min_\theta \; W_1(p_{\text{data}}, p_\theta), \tag{5.27}$$

其中两个概率分布 p 和 q 之间的 Wasserstein-1 距离定义为

$$W_1(p,q) := \min_{\pi\in\Pi(p,q)} \iint |x-y|\pi(x,y)\,dxdy \tag{5.28}$$

而 $\Pi(x,y)$ 是定义在 $\mathbb{R}^d\times\mathbb{R}^d$ 上、边缘分布分别为 p 和 q 的联合分布集合，即

$$\Pi(p,q) := \left\{\pi:\mathbb{R}^d\times\mathbb{R}^d\to[0,\infty): \begin{array}{l} p(x)=\int\pi(x,y)\,dy, \\ q(y)=\int\pi(x,y)\,dx, \end{array} \forall\, x,y\in\mathbb{R}^d\right\}.$$

Wasserstein 距离具有许多重要的数学性质，包括它是在 $\mathbb{R}^d$ 上概率分布空间中定义良好的一个度量。它是最优传输（optimal transport）领域中的关键概念之一，而该领域在过去几十年中已被广泛研究。关于 Wasserstein 距离的更多细节见附录 A.3。然而，Wasserstein 距离的主要问题在于，由于其带约束的结构和维数因素，它在数值计算中较为复杂。

为了求解最小化问题 (5.27)，我们可以将 Wasserstein-1 距离转化为其对偶形式，而这一形式相对更容易处理。Wasserstein-1 距离 $W_1(p,q)$ 的对偶形式为

$$W_1(p,q) = \max_{f\in\mathrm{Lip}_1(\mathbb{R}^d)} \int f(x)p(x)\,dx - \int f(x)q(x)\,dx, \tag{5.29}$$

其中 $\mathrm{Lip}_1(\mathbb{R}^d)$ 是定义在 $\mathbb{R}^d$ 上的所有 1-Lipschitz 连续函数的集合，即若对任意 $x,y\in\mathbb{R}^d$ 有 $|f(x)-f(y)|\le|x-y|$，则 $f\in\mathrm{Lip}_1(\mathbb{R}^d)$。

利用 (5.29) 中 Wasserstein-1 距离的对偶形式，我们可以将最小化问题 (5.27) 重写为一个等价形式，这就是所谓 Wasserstein GAN 模型中用于学习生成器网络 g_θ 最优参数 θ 的最小化问题：

$$\min_\theta \max_{f\in\mathrm{Lip}_1(\mathbb{R}^d)} \int f(x)p_{\text{data}}(x)\,dx - \int f(x)p_\theta(x)\,dx. \tag{5.30}$$

(5.30) 中的第二个积分可以写成一个更易处理的形式

$$\int f(x)p_\theta(x)\,dx = \int f(g_\theta(z))\mathcal{N}(z;0,I_d)\,dz$$

这是由变量替换公式 (5.24) 得到的。

在实际实现中，(5.30) 中的积分用 Monte Carlo 积分替代，而 $f:\mathbb{R}^d\to\mathbb{R}$ 则被参数化为一个深度神经网络，并对其参数施加适当约束，使得 f 近似满足 $f\in\mathrm{Lip}_1(\mathbb{R}^d)$。从这个意义上说，$f$ 在 Wasserstein GAN 中扮演着与原始 GAN 工作中的对抗网络 a_η 相似的角色。感兴趣的读者可以参考本章末尾列出的相关文献以获取更多细节。

在生成器网络 g_θ 训练完成后，我们可以先从 $\mathcal{N}(0,I_d)$ 中模拟任意多个样本，然后将 g_θ 应用于这些标准高斯样本，从而得到新的样本，而这些新样本服从概率分布 p_{data}。

5.3 扩散模型

扩散模型（Diffusion Models）是一类新的生成模型。其目标仍然是在仅给定数据集 $\mathcal{D}=\{x_i\in\mathbb{R}^d:i\in[M]\}$ 的情况下，学习如何生成服从 $\mathbb{R}^d$ 上某个未知概率分布 p_{data} 的新样本，其中该数据集由从 p_{data} 独立抽取的 M 个样本组成。与之前一样，我们并不知道 p_{data} 的解析形式。

扩散模型采用了一种新颖的样本生成方法：在 $\mathbb{R}^d$ 上、时间区间 $[0,T]$ 内构造一个**随机微分方程**（stochastic differential equation，SDE），其中 $T>0$ 是任意选定的：

$$dZ_t=\mu_t(Z_t)dt+(\Sigma_t(Z_t))^{1/2}dW_t, \tag{5.31}$$

其中

$$\mu:\ \mathbb{R}^d\times[0,T]\to\mathbb{R}^d$$

称为**漂移**（drift），而

$$\Sigma:\ \mathbb{R}^d\times[0,T]\to\mathbb{R}^{d\times d}$$

称为**扩散**（diffusion）系数。μ 和 Σ 都是随时间变化的。这个 SDE 的性质是：从 $\mathbb{R}^d$ 上某个简单的初始化概率 p_{init} 出发（通常取为 $\mathcal{N}(0,I_d)$ 或类似形式），该 SDE (5.31) 可以将任意在时刻 $t=0$ 从 p_{init} 中抽取的初值 Z_0 沿时间正向推进到时刻 $t=T$ 的 Z_T，并使得 Z_T 服从概率分布 p_{data}，即 Z_T 是从 p_{data} 中抽取的样本。

注意，由于目标是生成任意多个服从 p_{data} 的新样本 Z_T，我们需要能够方便地从 p_{init} 中抽取初值 Z_0。我们还需要条件概率分布具有闭式形式，以便于后续训练损失函数的实现，这一点我们稍后会详细说明。基于这些原因，初始化概率 p_{init} 往往选取得较为简单，以便容易从中采样。如上所述，p_{init} 的

一个典型选择是定义在 $\mathbb{R}^d$ 上的标准高斯分布 $\mathcal{N}(0, I_d)$，或者其某种平移均值或缩放方差的简单变体。

为了实现简单和计算高效，Σ 通常被设为单位矩阵 I_d 的一个与状态无关的标量倍数，即 σI_d，其中 $\sigma : [0,T] \to [0,\infty)$ 仅依赖时间 t。这种简化不影响扩散模型的核心思想，并且很容易推广到更一般的情形。

现在扩散模型的目标可以概括如下：找到一个漂移 $\mu : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$ 和一个扩散系数 $\sigma : [0,T] \to [0,\infty)$，使得对于任意满足初值 $Z_0 \sim p_{\text{init}} = \mathcal{N}(0, I_d)$ 且遵循如下 SDE 的轨道 Z_t（为简化记号，以下记 $\mu_t(z) := \mu(z,t) \in \mathbb{R}^d$ 和 $\sigma_t := \sigma(t) \in \mathbb{R}$）：

$$dZ_t = \mu_t(Z_t)dt + \sigma_t dW_t \tag{5.32}$$

在区间 $[0,T]$ 上成立，并且有 $Z_T \sim p_{\text{data}}$。一旦找到了这样的 μ 和 σ，我们就可以任意多次地抽取 $Z_0 \sim p_{\text{init}}$，并数值求解 SDE (5.32)（例如使用 Euler-Maruyama 方法等数值 SDE 求解器），从每个 Z_0 得到一个新的样本 $Z_T \sim p_{\text{data}}$。

由此可见，现在的主要任务是为 (5.32) 找到这样的 μ_t 和 σ_t。为此，扩散模型首先尝试确定另一个 SDE

$$dX_t = f_t(X_t)dt + g_t dW_t \tag{5.33}$$

其中带有人工设计的 $f : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$ 和扩散系数 $g : [0,T] \to [0,\infty)$，使得 (5.33) 恰好执行与 (5.32) 相反的过程：也就是说，对于任意选取为服从 p_{data} 的给定样本之一的初值 X_0，我们都可以数值求解 (5.33) 并得到 $X_T \sim p_{\text{init}}$。从这个意义上说，SDE (5.33) 可以将 p_{data} 转换为 p_{init}。设计这一新的 SDE (5.33) 的目的，是为了模拟其轨道并学习相应的时间演化概率。通过这样做，我们就能学习到所需要的 μ_t 和 σ_t，这一点将在本节稍后说明。

在继续之前，我们指出，一般而言，要为 (5.33) 找到合适的 f 和 g，使得从 $X_0 \sim p_{\text{data}}$ 出发时有 $X_T \sim p_{\text{init}}$，是非常困难的。我们将在本节进一步讨论这一问题。尽管如此，要找到使得当 T 足够大时 X_T 近似服从 p_{init} 的 f 和 g 却并不困难。例如，一个常见的选择是使用 Ornstein–Uhlenbeck（OU）过程，并使其扩散系数与漂移相匹配：

$$dX_t = -aX_t dt + \sqrt{2a}\, dW_t. \tag{5.34}$$

也就是说，

$$f_t(x) = -ax \quad \text{和} \quad g_t \equiv \sqrt{2a} \tag{5.35}$$

对所有 t 和 x 都成立，其中 $a > 0$ 是用户选定的常数。对于该 SDE (5.34)，已知在初值 $X_0 = x_0 \sim p_{\text{data}}$ 条件下，X_t 的条件概率是高斯分布：

$$\mathcal{N}(x_0 e^{-at}, (1 - e^{-2at}) I_d)$$

对任意 $t \geq 0$ 成立。因此，我们有

$$X_T | (X_0 = x_0) \ \sim \ \mathcal{N}(x_0 e^{-aT}, (1 - e^{-2aT}) I_d) \ \approx \ \ p_{\text{init}} := \mathcal{N}(0, I_d)$$

当 T 足够大时，对任意 x_0 都成立，而与 p_{data} 的具体形式无关。目前，我们将接受 OU 过程 (5.34) 作为 (5.33) 的选择，并接受 $X_0 \sim p_{\text{data}}$ 且 $X_T \sim p_{\text{init}} = \mathcal{N}(0, I_d)$ 这一设定。

现在的问题是，(5.35) 中这些简单的 f 和 g 与 (5.32) 中所需的 μ 和 σ 有什么关系，以及如何利用 f 和 g 来构造 μ 和 σ。

为了回答这两个问题，我们注意到，SDE (5.33) 的 Fokker–Planck 方程（关于 Fokker–Planck 方程的细节，见附录 A.4）为

$$\partial_t \rho_t = -\nabla \cdot (f_t \rho_t) + \frac{1}{2} g_t^2 \Delta \rho_t \tag{5.36}$$

其中 $\rho : \mathbb{R}^d \times [0, T] \to [0, \infty)$ 是每个 $t \in [0, T]$ 时刻 X_t 的概率密度，并满足 $\rho_0 = p_{\text{data}}$ 以及 $\rho_T = p_{\text{init}}$。另一方面，我们也有 SDE (5.32) 的 Fokker–Planck 方程：

$$\partial_t \pi_t = -\nabla \cdot (\mu_t \pi_t) + \frac{1}{2} \sigma_t^2 \Delta \pi_t \tag{5.37}$$

它定义在 $\mathbb{R}^d \times [0, T]$ 上，并且求解 (5.32) 的过程 Z_t 满足对所有 t 都有 $Z_t \sim \pi_t$。

通过比较这两个 Fokker–Planck 方程 (5.36) 和 (5.37)，可以证明：如果 $\pi_0 = \rho_T$，并且我们根据 f_t 和 g_t 适当地选择 μ_t 和 σ_t，则对所有 $t \in [0, T]$ 都有 $\pi_t = \rho_{T-t}$。该结果见下述定理。

定理 5.3.1. *设 ρ 和 π 分别满足 Fokker–Planck 方程 (5.36) 和 (5.37)，并且 ρ_t 与 π_t 处处为正，且 $\pi_0 = \rho_T$。假设对每个 $t \in [0, T]$，$f_t : \mathbb{R}^d \to \mathbb{R}^d$ 都是某个常数 $L_f > 0$ 下的 L_f-Lipschitz 连续函数。如果*

$$\mu_t(x) = -f_{T-t}(x) + \frac{1}{2} g_{T-t}^2(x) \frac{\nabla(\rho_{T-t}(x) + \pi_t(x))}{\pi_t(x)} \tag{5.38}$$

并且

$$\sigma_t = g_{T-t} \tag{5.39}$$

对所有 $x \in \mathbb{R}^d$ 和 $t \in [0,T]$ 成立，则

$$\pi_t = \rho_{T-t} \tag{5.40}$$

对所有 $t \in [0,T]$ 在 $\mathbb{R}^d$ 上几乎处处成立。特别地，$\pi_T = \rho_0 = p_{data}$。

证明. 我们将 ρ_{T-t} 和 π_t 之间的平方 L^2 误差的一半记为

$$\delta(t) := \frac{1}{2}\|\rho_{T-t} - \pi_t\|^2_{L^2(\mathbb{R}^d)} = \frac{1}{2}\int_{\mathbb{R}^d} |\rho_{T-t}(x) - \pi_t(x)|^2\, dx$$

其中 $t \in [0,T]$。在本证明余下部分，为简化记号，我们省略 $\mathbb{R}^d$ 和 (x)。现在注意到

$$\begin{aligned}
\delta'(t) &= \int (\rho_{T-t} - \pi_t)(-\partial_t \rho_{T-t} - \partial_t \pi_t)\, dx \\
&= \int (\rho_{T-t} - \pi_t)\Big[\nabla\cdot(f_{T-t}\rho_{T-t}) - \frac{1}{2}g^2_{T-t}\Delta\rho_{T-t} + \nabla\cdot(\mu_t\pi_t) - \frac{1}{2}\sigma_t^2\Delta\pi_t\Big]\, dx \\
&= \int (\rho_{T-t} - \pi_t)\Big[\nabla\cdot\Big(f_{T-t}\rho_{T-t} - \frac{1}{2}g^2_{T-t}\nabla\rho_{T-t} + \mu_t\pi_t - \frac{1}{2}\sigma_t^2\nabla\pi_t\Big)\Big]\, dx \\
&= \int (\rho_{T-t} - \pi_t)[\nabla\cdot(f_{T-t}(\rho_{T-t} - \pi_t))]\, dx \\
&= -\frac{1}{2}\int f_{T-t}\nabla|\rho_{T-t} - \pi_t|^2\, dx \\
&= \frac{1}{2}\int (\nabla\cdot f_{T-t})|\rho_{T-t} - \pi_t|^2\, dx \\
&\le \|\nabla\cdot f_{T-t}\|_{L^\infty}\delta(t) \\
&\le dL_f\delta(t),
\end{aligned}$$

其中第二个等号由 (5.36) 和 (5.37) 得到，第三个等号利用了 $\Delta\rho = \nabla\cdot(\nabla\rho)$（对 π 同理），第四个等号由 (5.38) 和 (5.39) 得到，第五和第六个等号由微积分中的散度定理得到，而最后两个不等式则来自于

$$|\nabla\cdot f_{T-t}| \le \|\nabla\cdot f_{T-t}\|_{L^\infty(\mathbb{R}^d)} \le dL_f$$

这是由于每个时刻 t 上 f_t 的 L_f-Lipschitz 连续性所致。

根据 Grönwall 不等式，我们知道

$$\delta(t) \le \delta(0)e^{dL_f t} \le \delta(0)e^{dL_f T} = 0$$

$$\boxed{X_0 \sim \rho_0 = p_{\text{data}}} \xrightarrow[\partial_t \rho_t = -\nabla \cdot (f_t \rho_t) + \frac{1}{2} g_t^2 \Delta \rho_t]{dX_t = f_t(X_t)dt + g_t dW_t} \boxed{X_T \sim \rho_T \approx p_{\text{init}} = \mathcal{N}(0, I_d)}$$

$$\boxed{Z_T \sim \pi_T = p_{\text{data}}} \xleftarrow[\partial_t \pi_t = -\nabla \cdot (\mu_t \pi_t) + \frac{1}{2} \sigma_t^2 \Delta \pi_t]{dZ_t = \mu_t(Z_t)dt + \sigma_t dW_t} \boxed{Z_0 \sim \pi_0 \approx p_{\text{init}} = \mathcal{N}(0, I_d)}$$

图 5.3: 扩散模型的正向过程（上）与反向过程（下）。在上图中，时间 t 从左到右增加（从 0 到 T）；在下图中，时间 t 从右到左增加。正向过程由 X_t 的 SDE 描述，或者等价地，由 ρ_t 的 Fokker–Planck 方程描述，其中对每个 t 都有 $X_t \sim \rho_t$。反向过程由 Z_t 的 SDE 描述，或者等价地，由 π_t 的 Fokker–Planck 方程描述，其中对每个 t 都有 $Z_t \sim \pi_t$。f_t、g_t、μ_t 和 σ_t 之间的关系由 (5.38) 和 (5.39) 给出，它们推出对每个 $t \in [0, T]$ 都有 $\pi_t = \rho_{T-t}$。注意，(5.38) 可以写成 (5.41)，从而给出 μ_t 关于 f_t、g_t 和 $\nabla \log \rho_t$ 的表达式。

对所有 t 都成立，因为 $\delta(0) = \frac{1}{2}\|\rho_T - \pi_0\|^2_{L^2(\mathbb{R}^d)} = 0$。这说明

$$\rho_{T-t} = \pi_t$$

对所有 $t \in [0, T]$ 在 $\mathbb{R}^d$ 上几乎处处成立，从而完成证明。 □

定理 5.3.1 提示我们将 (5.38) 中的 μ 设为如下形式：由于对所有 t 都有 $\pi_t = \rho_{T-t}$，因此 (5.38) 可化为

$$\mu_t = -f_{T-t} + g_{T-t}^2 \frac{\nabla \rho_{T-t}}{\rho_{T-t}} = -f_{T-t} + g_{T-t}^2 \nabla \log \rho_{T-t}. \tag{5.41}$$

这表明，Fokker–Planck 方程 (5.37) 变为

$$\partial_t \pi_t = -\nabla \cdot \Big(\left(-f_{T-t} + g_{T-t}^2 \nabla \log \rho_{T-t}\right) \pi_t \Big) + \frac{1}{2} g_{T-t}^2 \Delta \pi_t. \tag{5.42}$$

因此，我们知道，与 Fokker–Planck 方程 (5.42) 相对应的 SDE (5.32) 变为

$$dZ_t = (-f_{T-t}(Z_t) + g_{T-t}^2 \nabla \log \rho_{T-t}(Z_t))dt + g_{T-t} dW_t. \tag{5.43}$$

此外，如果我们从初值 $Z_0 \sim \pi_0 = \mathcal{N}(0, I_d)$ 开始，那么通过求解 (5.43)，就会在时间 T 得到一个新的样本 $Z_T \sim p_{\text{data}}$。上述关系如图 5.3 所示。

由于我们已经知道了例如 (5.35) 中给出的 f 和 g，因此在 (5.43) 中唯一尚需确定的项是对每个 t 的函数 $\nabla \log \rho_t$。为此，我们需要一种称为**得分匹配**（score matching）的方法。得分匹配的目的是近似 $\nabla \log \rho : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$，

该函数称为 **Stein 得分函数**（Stein score function）（在本节中我们简称其为得分函数），其中对所有 $t \in [0,T]$，都有 $X_t \sim \rho_t$，X_t 是满足 SDE (5.33) 的过程，而 ρ_t 是相应 Fokker–Planck 方程 (5.36) 的解。

直觉上，近似 $\nabla \log \rho$ 的方法是：将一个映射 $s : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$ 参数化为深度神经网络，通过最小化对所有 x 和 t 的 $|s_t(x) - \nabla \log \rho_t(x)|^2$ 来训练 s，并用 s_t 作为得分函数 $\nabla \log \rho_t$ 的近似。然而，这在实践中不可行，因为我们并不知道 $\nabla \log \rho_t$ 的真实值。

我们将得分匹配作为一种实用的替代方法来近似得分函数 $\nabla \log \rho_t$。为了具体化这一过程，我们使用 OU 过程 (5.34) 作为 SDE (5.33)。正如前面提到的，在给定初值 $X_0 = x_0$ 的条件下，$X_t = x_t$ 的条件概率为

$$\rho_t(x_t|x_0) = \mathcal{N}(x_t; \alpha_t x_0, \beta_t^2 I_d), \tag{5.44}$$

其中

$$\alpha_t = e^{-at} \quad \text{和} \quad \beta_t^2 = 1 - e^{-2at}.$$

因此，由于高斯分布的简单性，我们可以计算**条件**得分函数：

$$\nabla_{x_t} \log \rho_t(x_t|x_0) = -\frac{x_t - \alpha_t x_0}{\beta_t^2}. \tag{5.45}$$

在 (5.45) 中，我们特别说明梯度是相对于 x_t 的，因为 $\rho_t(x_t|x_0)$ 同时是 x_t 和 x_0 的函数。

注意，(5.45) 并不是我们希望网络 s 去近似的边缘得分函数 $\nabla_{x_t} \log \rho_t(x_t)$。然而，我们可以找到一种方法，利用条件得分函数 $\nabla_{x_t} \log \rho_t(x_t|x_0)$ 来表示

$\nabla_{x_t} \log \rho_t(x_t)$。为此，我们注意到

$$
\begin{aligned}
\nabla_{x_t} \log \rho_t(x_t) &= \frac{\nabla_{x_t} \rho_t(x_t)}{\rho_t(x_t)} \\
&= \frac{1}{\rho_t(x_t)} \nabla_{x_t} \Big(\int \rho_t(x_t, x_0)\, dx_0 \Big) \\
&= \frac{1}{\rho_t(x_t)} \nabla_{x_t} \Big(\int \rho_t(x_t|x_0) p_{\text{data}}(x_0)\, dx_0 \Big) \\
&= \frac{1}{\rho_t(x_t)} \int \nabla_{x_t} \rho_t(x_t|x_0) p_{\text{data}}(x_0)\, dx_0 \qquad (5.46) \\
&= \frac{1}{\rho_t(x_t)} \int [\nabla_{x_t} \log \rho_t(x_t|x_0)] \rho_t(x_t|x_0) p_{\text{data}}(x_0)\, dx_0 \\
&= \int [\nabla_{x_t} \log \rho_t(x_t|x_0)] \frac{\rho_t(x_t|x_0) p_{\text{data}}(x_0)}{\rho_t(x_t)}\, dx_0 \\
&= \int [\nabla_{x_t} \log \rho_t(x_t|x_0)] \rho_t(x_0|x_t)\, dx_0 \\
&= \mathbb{E}_{X_0 \sim \rho_t(\cdot|X_t = x_t)} [\nabla_{x_t} \log \rho_t(X_t|X_0)]
\end{aligned}
$$

其中，在第二个等号中 $\rho_t(x_t, x_0)$ 是时刻 t 下 (X_t, X_0) 的联合概率分布；第三个等号使用了 $\rho_t(x_t, x_0) = \rho_t(x_t|x_0)\rho_0(x_0) = \rho_t(x_t|x_0)p_{\text{data}}(x_0)$。在第四个等号中，我们利用 Fubini 定理交换了关于 x_t 的梯度与关于 x_0 的积分；在第五个等号中，由于 $\rho_t(x_t)$ 对被积函数而言是常数，因此将该分母移入积分；第六个等号则使用了 Bayes 公式。

(5.46) 给出的关系展示了得分函数 $\nabla_{x_t} \log \rho_t(x_t)$ 与条件得分函数 $\nabla_{x_t} \log \rho_t(x_t|x_0)$ 之间的有用联系。然而，我们仍然不知道后验概率 $\rho_t(x_0|x_t)$，也不知道如何从中采样，以便计算 (5.46) 末尾的期望。为了将这个期望写成某种可计算的形式，我们需要用到一个概率论中的简单事实，如下引理所示。

引理 5.3.2 (Mean squared error)**.** 设 X 和 Y 分别是取值于 $\mathbb{R}^d$ 和 $\mathbb{R}^m$ 的两个随机变量。令 $w : \mathbb{R}^d \times \mathbb{R}^m \to \mathbb{R}^k$ 为一个函数。则对任意 $y \in \mathbb{R}^m$，定义 $\phi_y : \mathbb{R}^k \to \mathbb{R}$ 为

$$\phi_y(c) := \mathbb{E}_{X \sim p_{X|Y}(\cdot|Y=y)} [|c - w(X, Y)|^2].$$

则 ϕ_y 有唯一极小值点 $c_y \in \mathbb{R}^k$，其表达式为

$$c_y := \mathbb{E}_{X \sim p_{X|Y}(\cdot|Y=y)} [w(X, Y)].$$

证明. 注意到

$$\begin{aligned}\phi_y(c) &= \mathbb{E}_{X\sim p_{X|Y}(\cdot|Y=y)}[|c-w(X,Y)|^2] \\ &= \int |c-w(x,y)|^2 p_{X|Y}(x|y)\,dx.\end{aligned}$$

由于对每个 x 都有 $p_{X|Y}(x|y)\geq 0$，可知 ϕ_y 是关于 c 的凸函数。因此，ϕ_y 的每个临界点都是其全局极小值点。

现在我们对 ϕ_y 关于 c 求梯度，令其等于 0，并求其临界点。为此，有

$$\nabla_c\phi_y(c) = 2\int (c-w(x,y))p_{X|Y}(x|y)\,dx = 0,$$

从而推出唯一解

$$c = \int c\,p_{X|Y}(x,y)\,dx = \int w(x,y)p_{X|Y}(x,y)\,dx = \mathbb{E}_{X\sim p_{X|Y}(\cdot|Y=y)}[w(X,Y)].$$

证毕。 □

引理 5.3.2 蕴含函数 $u: y\mapsto c_y$ 是良定义的，因为 c_y 存在且唯一。其结果总结在如下定理中。

定理 5.3.3. 设引理 *5.3.2* 中的条件成立。定义 $u:\mathbb{R}^m\to\mathbb{R}^k$，使其将任意 $y\in\mathbb{R}^m$ 映射为

$$u(y) = \operatorname*{arg\,min}_{c\in\mathbb{R}^m} \mathbb{E}_{X\sim p_{X|Y}(\cdot|Y=y)}[|c-w(X,Y)|^2].$$

则对每个 $y\in\mathbb{R}^m$，都有

$$u(y) = \mathbb{E}_{X\sim p_{X|Y}(\cdot|Y=y)}[w(X,Y)]$$

成立。

证明. 该结论由引理 5.3.2 立即得到，因此略去证明。 □

定理 5.3.3 使我们能够将感兴趣的条件期望改写为一个最小二乘优化问

题。特别地，我们注意到 (5.46) 实际上等价于

$$\begin{aligned}\nabla_{x_t}\log\rho_t(x_t) &= \mathbb{E}_{X_0\sim\rho_t(\cdot|X_t=x_t)}[\nabla_{x_t}\log\rho_t(X_t|X_0)]\\ &= \arg\min_{c\in\mathbb{R}^d}\mathbb{E}_{X_0\sim\rho_t(\cdot|X_t=x_t)}[|c-\nabla_{x_t}\log\rho_t(X_t|X_0)|^2].\end{aligned}$$

这可以通过应用定理 5.3.3 看出，其中 X、Y 和 $w(X,Y)$ 分别替换为 X_0、X_t 和 $\nabla_{x_t}\log\rho_t(X_t|X_0)$（可将其视为 X_0 与 X_t 的函数）。

到这里，我们已经构造出了一个可以用于训练深度神经网络 s 使得 $s_t=\nabla\log\rho_t$ 的损失函数。更具体地，由于我们希望对所有 x 和 t 都有 $s_t(x)=\nabla\log\rho_t(x)$，我们可以使用通过数值求解 SDE (5.33)（即当前情形下 (5.34) 中的 OU 过程）得到的样本轨道中的 x_t。由此可知，对每个 $t\in[0,T]$，s_t 可以通过最小化下式学习出来：

$$\begin{aligned}&\int\mathbb{E}_{X_0\sim\rho_t(\cdot|X_t=x_t)}[|s_t(x_t)-\nabla_{x_t}\log\rho_t(X_t|X_0)|^2]\,dx_t\\ =&\iint|s_t(x_t)-\nabla_{x_t}\log\rho_t(x_t|x_0)|^2]\rho_t(x_0|x_t)\rho_t(x_t)\,dx_0dx_t\\ =&\iint|s_t(x_t)-\nabla_{x_t}\log\rho_t(x_t|x_0)|^2]\rho_t(x_t|x_0)p_{\text{data}}(x_0)\,dx_tdx_0\end{aligned}\tag{5.47}$$

其中第二个等号使用了

$$\rho_t(x_0|x_t)\rho_t(x_t)=\rho_t(x_t,x_0)=\rho_t(x_t|x_0)\rho_0(x_0)$$

以及 $\rho_0(x_0)=p_{\text{data}}(x_0)$。

由于我们通常希望在整个时间区间 $[0,T]$ 上同时训练 s_t，因此可以人为设定一个定义在 $[0,T]$ 上的概率密度 $\xi(t)$，满足对所有 t 都有 $\xi(t)>0$ 且 $\int_0^T\xi(t)\,dt=1$，并通过求解

$$\min_s\int_0^T\iint\left|s_t(x_t)-\nabla_{x_t}\log\rho_t(x_t|x_0)\right|^2\rho_t(x_t|x_0)p_{\text{data}}(x_0)\,dx_tdx_0dt,\tag{5.48}$$

来训练网络 $s:\mathbb{R}^d\times[0,T]\to\mathbb{R}^d$，其中最小化是相对于 s 的网络参数进行的。

为了便于实际实现，我们还可以进一步简化 (5.48) 中的损失函数。当我们使用 OU 过程 (5.34) 来生成样本轨道 $\{x_t\}$ 和 ρ_t 时，条件概率 $\rho_t(x_t|x_0)$ 由 (5.44) 给出，而条件得分函数 $\nabla_{x_t}\log\rho_t(x_t|x_0)$ 则由 (5.45) 给出。事实上，由于 $\rho_t(x_t|x_0)=\mathcal{N}(x_t;\alpha_tx_0,\beta_t^2I_d)$，我们可以直接从标准高斯分布 $\mathcal{N}(0,I_d)$ 中

独立采样 ϵ，并将 x_t 改写为 x_0 和 ϵ 的函数如下：

$$x_t = \alpha_t x_0 + \beta_t \epsilon. \tag{5.49}$$

在这种情况下，条件概率 $\rho_t(x_t|x_0)$ 变得更加简单：

$$\text{当} \quad X_t = a_t x_0 + \beta_t \epsilon \quad \text{时} \quad X_t \sim \rho_t(\cdot|x_0), \quad \text{其中} \quad \epsilon \sim \mathcal{N}(0, I_d), \tag{5.50}$$

而条件得分函数则化简为

$$\nabla_{x_t} \log \rho_t(x_t|x_0) = -\frac{\epsilon}{\beta_t}. \tag{5.51}$$

为了抵消在 OU 过程中当 $t \to 0$ 时 $\beta_t \to 0$ 的影响，我们可以选取 $\xi(t) \propto \beta_t^2$。在实践中，我们还可以把待训练的网络设为 $\varepsilon : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$，而不是 (5.48) 中使用的 s，使得

$$s_t(x) = -\frac{\varepsilon_t(x)}{\beta_t} \tag{5.52}$$

对所有 x 和 t 都成立。将 (5.49)、(5.50) 和 (5.51) 代入 (5.48)，我们得到一个训练网络 ε 的极其简单的最小化问题：

$$\min_{\varepsilon} \int_0^T \iint |\varepsilon_t(\alpha_t x_0 + \beta_t \epsilon) - \epsilon|^2 \mathcal{N}(\epsilon; 0, I_d) p_{\text{data}}(x_0)\, dx_0 d\epsilon dt. \tag{5.53}$$

在数值实现中，(5.53) 中的损失函数可以通过 Monte Carlo 积分（见附录 A.1）近似为

$$\frac{1}{NM} \sum_t \sum_{(x_0, \epsilon)} |\varepsilon_t(\alpha_t x_0 + \beta_t \epsilon) - \epsilon|^2. \tag{5.54}$$

更具体地说，我们先从定义在 $(0, 1)$ 上的概率密度 ξ 中采样 N 个时间点。然后，对每个采样到的时间点 t，从原始数据集 $\mathcal{D}$（由服从 p_{data} 分布的 M 个样本组成）中选取一个样本 x_0，并独立地抽取一个 $\epsilon \sim \mathcal{N}(0, I_d)$。这些样本被代入 (5.54) 中，这就是我们在实践中用于训练网络 ε 的损失函数。一旦网络 ε 训练完成，我们就可以按 (5.52) 定义 s_t，并知道它是得分函数 $\nabla \log \rho_t$ 的一个近似。

用学习好的得分函数生成新样本 一旦我们得到了近似得分函数 s_t，就可以在 (5.43) 中用它替代真实得分函数 $\nabla \log \rho_t$。注意，对于 OU 过程，我们已知

f 和 g 具有 (5.35) 中的简单形式，即 $f_t = -ax$ 且 $g_t = \sqrt{2a}$。于是用于生成的 SDE (5.43) 为

$$dZ_t = (-aZ_t + 2as_t(Z_t))dt + \sqrt{2a}\, dW_t. \tag{5.55}$$

实际的样本生成可以按照 Euler-Maruyama 格式进行：将区间 $[0,T]$ 划分为长度相等的子区间，步长为 $h > 0$，然后计算

$$Z_{t+h} = Z_t + (-aZ_t + 2as_t(Z_t))h + \sqrt{2ah}\zeta_t \tag{5.56}$$

其中 $t = 0, h, 2h, \ldots, T$，初值 Z_0 与扰动项 ζ_t 都从 $\mathcal{N}(0, I_d)$ 中相互独立地采样。这样，Z_T 就是一个新生成的样本，并且服从概率分布 p_{data}。

(5.56) 有一个明显缺点：在每一个时间点 t，我们都需要从标准高斯分布中采样一个 ζ_t。当时间步很多且我们需要生成大量样本时，这会非常慢。

我们可以通过将 SDE (5.55) 转换为一个 ODE 来克服这一缺点，使得其解在每个时刻 t 都服从相同的概率分布 π_t。为此，我们注意到在 (5.42) 中，$\Delta\pi_t = \nabla \cdot (\pi_t \nabla \log \pi_t)$。此外，由定理 5.3.1 可知，对所有 $t \in [0,T]$ 都有 $\pi_t = \rho_{T-t}$。因此，Fokker–Planck 方程 (5.42) 可以改写为关于 π 的连续方程：

$$\partial_t \pi_t = -\nabla \cdot \Big((-f_{T-t} + \frac{1}{2} g_{T-t}^2 \nabla \log \rho_{T-t})\, \pi_t \Big), \tag{5.57}$$

对应的 ODE 为

$$\dot{z}_t = -f_{T-t}(z_t) + \frac{1}{2} g_{T-t}^2 \nabla \log \rho_{T-t}(z_t). \tag{5.58}$$

其中 $\dot{z}_t$ 表示 z_t 关于时间的导数。在实践中，我们可以用训练得到的得分函数 s_{T-t} 替代 $\nabla \log \rho_{T-t}$，并从任意初值 $z_0 \sim \mathcal{N}(0, I_d)$ 出发求解该 ODE (5.58) 得到 z_T。所生成的 z_T 同样服从所需的分布 $\pi_T = \rho_0 = p_{\text{data}}$。

有许多数值 ODE 求解器，例如中点法和四阶 Runge-Kutta 方法 [24]，它们在求解 ODE (5.58) 时比求解 SDE (5.55) 快得多。相对于训练得到的得分函数 s_t 对真实得分函数 $\nabla \log \rho_t$ 的近似误差而言，这些求解器带来的数值误差通常是非常小的。

可用于生成训练用样本轨迹的其它 SDE 事实上，除了 OU 过程 (5.34) 之外，还有许多其他备选过程可用于设计 (5.33)。例如，下面这类方差保持型（variance preserving，VP）SDE 是 OU 过程的一个小推广，并且在实际实现

中经常使用：

$$dX_t = -\gamma_t dt + \sqrt{2\gamma_t} dW_t, \tag{5.59}$$

其中对每个 t，γ_t 都取值于区间 $[\gamma_{\min}, \gamma_{\max}]$ 内。这里的上下界 $\gamma_{\min}$ 和 $\gamma_{\max}$ 是用户指定的正数，而 $\gamma_{\min}$ 往往被设为一个较小的正数，例如 0.01，以防止 γ_t 过于接近 0 并造成数值计算不稳定。另一方面，$\gamma_{\max}$ 的选择则较为灵活，例如可以取 20。

VP SDE (5.59) 与 OU 过程类似，其解在给定初值 $X_0 = x_0$ 条件下也服从高斯分布。更准确地说，在给定 $X_0 = x_0$ 的条件下，X_t 的条件概率分布为

$$\rho_t(x_t|x_0) = \mathcal{N}(x_t; \alpha_t x_0, \beta_t^2 I_d)$$

其中

$$\alpha_t = e^{-\int_0^t \gamma_\tau d\tau} \tag{5.60}$$

并且

$$\beta_t^2 = 1 - e^{-2\int_0^t \gamma_\tau \, d\tau}. \tag{5.61}$$

对于 (5.59) 中的 γ_t，一个简单选择是

$$\gamma_t = \gamma_{\min} + t(\gamma_{\max} - \gamma_{\min})$$

其中 $t \in [0, 1]$。此时

$$\int_0^t \gamma_\tau \, d\tau = t\gamma_{\min} + \frac{t^2}{2}(\gamma_{\max} - \gamma_{\min}),$$

因此 (5.60) 和 (5.61) 中的 α_t 和 β_t 都有闭式表达式。我们指出，尽管名称如此，VP SDE (5.59) 并不是真的在所有时间都保持方差不变，但它的方差确实会随着 t 变大而趋于某个常数。

关于选择 SDE (5.33) 的讨论 由于样本数据分布 p_{data} 在不同应用中可能各不相同，我们希望找到某种“通用”的漂移 f（以及扩散系数 g），使得正向时间 SDE (5.33) 能够把 $X_0 \sim \rho_0 = p_{\text{data}}$ 移动到 $X_T \sim \rho_T = p_{\text{init}} = \mathcal{N}(0, I_d)$。这里所谓“通用”，是指 f 和 g 被选为使得 SDE (5.33)，或者等价地 Fokker–Planck 方程 (5.36)，能够在有限时间 T 内把**任意给定的** p_{data} 转换为 $\mathcal{N}(0, I_d)$。换言之，f 和 g 的选取不应依赖于 p_{data}。然而，这并不总是容易做到，因为在实践中 p_{data} 是未知且通常相当复杂的概率分布。事实上，我们可以断言，这样

的通用 f 和 g 并不存在，如下命题所示。

命题 5.3.4. *如果存在某些 f 和 g，使得 Fokker–Planck 方程 (5.36) 能够将 $\rho_0 = p_{data}$ 转换到 $\rho_T = \mathcal{N}(0, I_d)$，那么 f 和 g 必须依赖于 p_{data}。*

证明. 我们采用反证法。假设 f 和 g 不依赖于 p_{data}。根据命题 5.3.1 中所描述的结论，存在一个 π，使得对每个 t 都有 $\pi_t = \rho_{T-t}$ 在 $\mathbb{R}^d$ 上几乎处处成立，其中 ρ_t 满足带有此类 f 和 g 的 (5.36)，且端点条件为 $\rho_0 = p_{\text{data}}$ 和 $\rho_T = \mathcal{N}(0, I_d)$。这意味着，$\pi$ 的 Fokker–Planck 方程 (5.42) 可写为

$$\partial_t \pi_t = -\nabla \cdot \Big((-f_{T-t} + g_{T-t}^2 \nabla \log \pi_t) \pi_t \Big) + \frac{1}{2} g_{T-t}^2 \Delta \pi_t.$$

其初值为 $\pi_0 = \rho_T = \mathcal{N}(0, I_d)$。然而，$\pi_0$、$f$ 和 g 中都不包含任何关于 p_{data} 的信息。这与 $\pi_T = \rho_0 = p_{\text{data}}$ 相矛盾。 □

由此可见，在有限时间区间 $[0, T]$ 内将 p_{data} 转换为 p_{init} 本身就是一个不简单的问题，或许并不比将 p_{init} 转换为 p_{data} 简单多少。尽管我们使用了 OU 过程 (5.34) 使得概率分布 ρ_T 近似于 p_{init}，但根据命题 5.3.4，我们知道不可能在有限时间 T 内精确到达 $p_{\text{init}} = \mathcal{N}(0, I_d)$。从理论上讲，我们应当把通过 Fokker–Planck 方程 (5.36) 或 SDE (5.33) 求得的真实 ρ_T 作为样本生成中的初始分布 π_0。然而，这在实践中并不可行，因为对 ρ_T 进行评估或从中采样同样是困难的。

人们自然会思考，是否存在某个合适的漂移 f（以及扩散系数 g），使得 Fokker–Planck 方程 (5.36) 的解 ρ 满足 $\rho_0 = p_{\text{data}}$ 且 $\rho_T = \mathcal{N}(0, I_d)$（或者反过来，这正是最初的目标生成问题）。在对 p_{data} 施加某些温和条件的情况下，这个问题的答案是肯定的。

事实上，这与薛定谔桥问题 Schrödinger bridge problem（SBP）的可行性有关：给定 $\mathbb{R}^d$ 中两个概率分布 p_0 和 p_1，SBP 可以写为

$$\begin{aligned} \min_f \quad & \frac{1}{2} \int_0^T \int |f_t(x)|^2 \rho_t(x)\, dx dt, \\ \text{s.t.} \quad & \partial_t \rho_t = -\nabla \cdot (f_t \rho_t) + \frac{\sigma^2}{2} \Delta \rho_t, \quad 0 \le t \le T, \\ & \rho_0 = p_0, \quad \rho_T = p_1, \end{aligned} \tag{5.62}$$

其中 $\sigma > 0$ 是给定的常数扩散系数。如果令 $p_0 = p_{\text{data}}$ 且 $p_1 = \mathcal{N}(0, I_d)$，那么 SBP 不仅要求从 p_{data} 转移到 $\mathcal{N}(0, I_d)$，从而直接回答了我们之前关于 f（或

μ）可行性的问题，而且还要求在 (5.62) 中的目标函数意义下达到最优性。然而，一般而言，求解 (5.62) 的一个可行解或最优解都是困难的。

尽管存在上面提到的近似问题，扩散模型在实践中仍然表现良好。上述讨论揭示了扩散模型的局限性，并说明了为什么有必要学习得分函数 $\nabla \log \rho_t$，以构造 (5.32) 中的生成漂移 μ_t。

5.4 概率密度控制

在第 5.3 节对扩散模型的推导中，我们有两个与状态无关的标量函数 g_t 和 σ_t。它们分别是 Fokker–Planck 方程 (5.36) 和 (5.42) 中的扩散系数，并且对所有 t 都满足 $\sigma_t = g_{T-t}$，然而，它们在学习和生成过程中并不起关键作用，而只是决定了相应 SDE (5.33) 和 (5.43) 中波动的大小。

根据定理 5.3.1，我们可以直接对所有 t 取 $g_t = 0$ 和 $\sigma_t = 0$，此时 Fokker–Planck 方程 (5.36) 和 (5.42) 分别退化为如下两个连续方程

$$\partial_t \rho_t = -\nabla \cdot (f_t \rho_t) \tag{5.63}$$

和

$$\partial_t \pi_t = -\nabla \cdot (-f_{T-t} \pi_t) \tag{5.64}$$

它们都定义在 $\mathbb{R}^d \times [0, T]$ 上。此外，ρ 和 π 仍然满足

$$\pi_t = \rho_{T-t}$$

对所有 $t \in [0, T]$ 成立。特别地，$\pi_0 = \rho_T$ 且 $\pi_T = \rho_0$。

我们还注意到，在这种情况下，相应的 SDE (5.33) 和 (5.43) 分别退化为如下两个 ODE

$$\dot{x}_t = f_t(x_t) \tag{5.65}$$

和

$$\dot{z}_t = -f_{T-t}(z_t), \tag{5.66}$$

分别成立。

因此，如果我们能够找到一个漂移 $f : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$，使得 (5.63) 的解 ρ_t 满足 $\rho_0 = p_{\text{data}}$ 且 $\rho_T = \mathcal{N}(0, I_d)$，那么就有 $\pi_0 = \mathcal{N}(0, I_d)$ 且 $\pi_T = p_{\text{data}}$。于是，我们只需从标准高斯分布 $\mathcal{N}(0, I_d)$ 中抽取一个样本 z_0，将 ODE (5.66) 求解到时间 T，即可得到 z_T，它是一个服从概率分布 $\pi_T = p_{\text{data}}$ 的新样本。

$$\boxed{x_0 \sim \rho_0 = p_{\text{data}}} \xrightarrow[\partial_t \rho_t = -\nabla \cdot (f_t \rho_t)]{\dot{x}_t = f_t(x_t)} \boxed{x_T \sim \rho_T \approx p_{\text{init}} = \mathcal{N}(0, I_d)}$$

$$\boxed{z_T \sim \pi_T = p_{\text{data}}} \xleftarrow[\partial_t \pi_t = -\nabla \cdot (\mu_t \pi_t)]{\dot{z}_t = \mu_t(z_t)} \boxed{z_0 \sim \pi_0 \approx p_{\text{init}} = \mathcal{N}(0, I_d)}$$

图 5.4: 概率控制的前向过程（上）和反向过程（下）。在上图中，时间 t 从左到右增加（从 0 到 T）；在下图中，时间 t 从右到左增加。前向过程由 x_t 的 ODE 描述，或者等价地，由其对应的 ρ_t 的连续方程描述，并且在每个 t 都有 $x_t \sim \rho_t$。反向过程由 z_t 的 ODE 描述，或者等价地，由其对应的 π_t 的连续方程描述，并且在每个 t 都有 $z_t \sim \pi_t$。有 $\mu_t = -f_{T-t}$，这使得对每个 $t \in [0, T]$ 都有 $\pi_t = \rho_{T-t}$。近似关系 $\rho_T \approx \mathcal{N}(0, I_d)$ 是通过寻找使 $\mathrm{KL}(\rho_T, \mathcal{N}(0, I_d))$ 最小的 f 来实现的，如 (5.67) 所示。

这一关系如图 5.4 所示。

现在我们可以看到，关键在于找到这个漂移 f。根据命题 5.3.4，我们知道若要在有限时间 T 内使 $\rho_T = \mathcal{N}(0, I_d)$，则 f 必须依赖于 p_{data}。因此，人工设计的 f 不在我们的考虑范围之内。相反，我们必须从给定的 p_{data} 样本中学习这个 f。

我们可以将寻找 f 的问题视为一个概率密度控制问题，即学习一个控制向量场 $f : \mathbb{R}^d \times [0, T] \to \mathbb{R}^d$，使其能够将概率密度 ρ_t 随时间从 $\rho_0 = p_{\text{data}}$ 驱动到时刻 T 的 $\rho_T = \mathcal{N}(0, I_d)$。更准确地说，这个概率密度控制问题可以写为

$$\min_f \quad \mathrm{KL}(\rho_T, \mathcal{N}(0, I_d)), \tag{5.67a}$$

$$\text{s.t.} \quad \partial_t \rho_t = -\nabla \cdot (f_t \rho_t), \quad 0 \le t \le T, \tag{5.67b}$$

$$\rho_0 = p_{\text{data}}. \tag{5.67c}$$

一旦找到 (5.67) 的解 f，我们就知道 $\rho_T = \mathcal{N}(0, I_d)$ 近似成立，于是便可在 ODE (5.66) 中使用该 f 来生成样本。

为了求解 (5.67)，我们可以使用第 3.3 节中介绍的 neural ODE 方法。设 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$ 是由从 p_{data} 中独立抽取的 M 个样本组成的数据集，我们可以对所有 i 设定一组初值 $x_0^{(i)} = x_i$，并对每个 $x_0^{(i)}$ 求解 ODE (5.65)。于是我们知道对所有 $t \in [0, T]$，都有 $x_t^{(i)} \sim \rho_t$。

剩下的问题是如何用点集 $\{x_T^{(i)}\}$ 来表示 (5.67a) 中的目标函数。我们注意

到该目标函数可以分解为两部分：

$$\mathrm{KL}(\rho_T, \mathcal{N}(0, I_d)) = \underbrace{\int \rho_T(x) \log \rho_T(x)\, dx}_{=:A} - \underbrace{\int \rho_T(x) \log \mathcal{N}(x; 0, I_d)\, dx}_{=:B}. \tag{5.68}$$

很容易看出，(5.68) 中的项 B 为

$$\begin{aligned} B &= \int \rho_T(x) \log \mathcal{N}(x; 0, I_d)\, dx \\ &= -\int \rho_T(x) \Big(\frac{1}{2}|x|^2 + \frac{d}{2}\log(2\pi) \Big)\, dx \\ &= -\frac{1}{2}\int |x|^2 \rho_T(x)\, dx - \frac{d}{2}\log(2\pi). \end{aligned}$$

然后我们可以使用 Monte Carlo 积分，将 B 近似为

$$B \approx -\frac{1}{2}\sum_{i=1}^{M} |x_T^{(i)}|^2 - \frac{d}{2}\log(2\pi), \tag{5.69}$$

因为 $x_T^{(i)} \sim \rho_T$。

对于 (5.68) 中的项 A，我们注意到它是 ρ_T 的负熵。我们需要找到一种在实践中可计算 A 的方法。为此，我们令 h_t 表示每个时刻 t 上 ρ_t 的负熵，即

$$h_t := \int \rho_t(x) \log \rho_t(x)\, dx. \tag{5.70}$$

注意，对每个 t 都有 $h_t \in \mathbb{R}$。特别地，$A = h_T$，而 h_0 是 p_{data} 的负熵，它是一个未知常数。此外，我们有

$$\begin{aligned} \dot{h}_t &= \frac{d}{dt}\Big(\int \rho_t(x) \log \rho_t(x)\, dx \Big) \\ &= \int (\log \rho_t(x) + 1) \partial_t \rho_t(x)\, dx \\ &= -\int (\log \rho_t(x) + 1) \nabla \cdot (\rho_t(x) f_t(x))\, dx \\ &= \int \nabla(\log \rho_t(x) + 1) \cdot (\rho_t(x) f_t(x))\, dx \\ &= \int \nabla \rho_t(x) \cdot f_t(x)\, dx \\ &= -\int \rho_t(x) \nabla \cdot f_t(x)\, dx, \end{aligned} \tag{5.71}$$

其中第三个等号来自连续方程 (5.63)。在实践中，由于 $x_t^{(i)} \sim \rho_t$，我们可以使用 Monte Carlo 积分近似 (5.71) 右端的积分：

$$\dot{h}_t = -\int \rho_t(x)\nabla \cdot f_t(x)\,dx \approx -\frac{1}{M}\sum_{i=1}^{M} \nabla \cdot f_t(x_t^{(i)}). \tag{5.72}$$

因此，在实际实现中，我们可以加入一个辅助变量 h_t，并将 ODE (5.72) 与 $\{x_t^{(i)} : i \in [M]\}$ 的 ODE (5.65) 一起求解，从而得到 $A = h_T - h_0$。由于 h_0 是 p_{data} 的负熵，它是一个与 f 无关的常数，因此在计算中我们可以直接假设它为零，因为这不会改变 (5.67) 的解。

总之，我们已经将寻找 $f : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$ 的问题 (5.67) 形式化为如下一个在实践中可实现的问题：

$$\min_f \quad h_T + \frac{1}{2}\sum_{i=1}^{M} |x_T^{(i)}|^2, \tag{5.73a}$$

$$\text{s.t.} \quad \dot{x}_t^{(i)} = f_t(x_t^{(i)}), \quad i \in [M], \tag{5.73b}$$

$$\dot{h}_t = -\frac{1}{M}\sum_{i=1}^{M} \nabla \cdot f_t(x_t^{(i)}), \quad 0 \le t \le T, \tag{5.73c}$$

$$x_0^{(i)} = x_i, \quad h_0 = 0, \tag{5.73d}$$

其中 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$ 是从 p_{data} 中独立抽取的 M 个给定样本的集合。我们在 (5.73) 中省略了所有无关常数。与往常一样，这里的最小化是相对于 f 的网络参数进行的。

我们可以使用 neural ODE 方法来求解 (5.73)，在该方法中，(5.73a) 中损失函数关于网络 f 参数的梯度可以通过该方法中的伴随 ODE 部分来计算。

正如之前所提到的，一旦通过求解 (5.73) 训练好了 f，我们就可以从 $\mathcal{N}(0, I_d)$ 中抽取初值 z_0，并使用训练好的 f 求解 ODE (5.66) 到时间 T。这样便得到 z_T，它是一个服从概率分布 p_{data} 的新样本。

5.5 流匹配

流匹配（Flow matching）是一种替代性的生成模型框架，其目标是直接学习将 $\mathcal{N}(0, I_d)$ 传输到 p_{data} 的向量场。和之前一样，我们只给定一个数据集 $\mathcal{D} = \{x_i \in \mathbb{R}^d : i \in [M]\}$，其中包含了从 p_{data} 中独立抽取的 M 个样本。在 flow matching 中，我们的目标是利用这些样本学习一个向量场 $u : \mathbb{R}^d \times [0,T] \to \mathbb{R}^d$，

使得 ODE

$$\dot{x}_t = u_t(x_t) \tag{5.74}$$

在任意初值 $x_0 \sim \mathcal{N}(0, I_d)$ 下的解 x_T 满足 $x_T \sim p_{\text{data}}$。由于在 flow matching 方法中 T 的取值并不重要，因此在本节余下部分我们取 $T = 1$。

我们再次将目标的时变向量场 u 参数化为一个深度神经网络。于是，flow matching 的关键就在于设计一个在实践中可实现的 u 的训练问题。为此，我们采用与第 5.3 节扩散模型中相似的方法，先考虑一些“条件”向量场。

注意到，将 $\mathcal{N}(0, I_d)$ 传输到 p_{data} 是困难的，但将 $\mathcal{N}(0, I_d)$ 传输到 p_{data} 中任意一个特定样本 z 却是容易的。例如，对于任意固定的 $z \sim p_{\text{data}}$（即，z 是一个服从 p_{data} 的给定数据样本），在时间区间 $[0, 1]$ 内，从任意点 $\epsilon \in \mathbb{R}^d$ 移动到 z 的最简单方式，就是沿着从 ϵ 到 z 的直线并以恒定速度运动。也就是说，我们可以令运动轨迹 x_t 满足

$$x_t = (1 - t)\epsilon + tz. \tag{5.75}$$

显然有 $x_0 = \epsilon$ 且 $x_1 = z$。我们还注意到由 (5.75) 可推出两个结论：

$$\dot{x}_t = z - \epsilon \tag{5.76}$$

以及

$$\epsilon = \frac{x_t - tz}{1 - t} \tag{5.77}$$

其中我们约定 (5.77) 右端（以及本节后面类似表达式）在 $t \to 1^-$ 时以极限意义定义，并且我们将确保这不会在后续推导中引起实际的数学问题。

结合 (5.76) 和 (5.77)，可得

$$\dot{x}_t = z - \epsilon = \frac{z - x_t}{1 - t}. \tag{5.78}$$

现在我们可以如下定义所谓的**条件向量场** $v^z : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d$：固定任意 $z \in \mathbb{R}^d$，对任意 $x \in \mathbb{R}^d$ 和 $t \in [0, 1]$，定义

$$v_t^z(x) := \frac{z - x}{1 - t}. \tag{5.79}$$

它的重要性在于：从任意初始值 $\epsilon \in \mathbb{R}^d$ 出发，只要沿着这个条件向量场 v_t^z 演化，就一定会在时刻 $t = 1$ 到达 z。

我们还可以利用这个条件向量场构造 ρ^z 的一个连续方程。更准确地说，

我们对任意 z 取 $\rho_0^z = \mathcal{N}(0, I_d)$（可以将其看作在 $\mathbb{R}^d$ 中服从标准高斯分布的大量粒子），并定义如下连续方程

$$\partial_t \rho_t^z = -\nabla \cdot (v_t^z \rho_t^z). \tag{5.80}$$

由于 v^z 的性质，我们知道当 $t \to 1^-$ 时有 $\rho_t^z \to \delta_z$（δ_z 是以点 z 为中心的 Dirac delta 函数）。尽管如此，我们仍然有对所有 x 成立的 $\rho_t^z(x) \geq 0$，并且 $\int \rho_t^z(x)\,dx = 1$，因为 ρ_t^z 描述的是这些粒子（它们自时间 0 起由 v^z 驱动）在时刻 t 的概率密度。

一个特别重要的事实是，如果初始点 ϵ 服从 $\mathcal{N}(0, I_d)$，那么就有

$$x_t = (1-t)\epsilon + tz \sim \mathcal{N}(\,\cdot\,; tz, (1-t)^2 I_d).$$

再注意到 $x_t \sim \rho_t^z$，于是得到

$$\rho_t^z = \mathcal{N}(tz, (1-t)^2 I_d). \tag{5.81}$$

现在，借助 ρ_t^z，我们可以定义如下关于 t、x 和 z 的函数：

$$\rho_t(x, z) := \rho_t^z(x) p_{\text{data}}(z). \tag{5.82}$$

我们注意到，ρ_t 具有若干使其成为联合概率分布的性质：对所有 $x, z \in \mathbb{R}^d$ 和 $t \in [0, 1]$ 都有 $\rho_t(x, z) \geq 0$，并且

$$\begin{aligned}
\iint \rho_t(x, z)\,dz dx &= \iint \rho_t^z(x) p_{\text{data}}(z)\,dz dx \\
&= \int p_{\text{data}}(z) \Big(\int \rho_t^z(x)\,dx \Big) dz \\
&= \int p_{\text{data}}(z)\,dz \\
&= 1
\end{aligned}$$

其中第三个等号用到了 $\int \rho_t^z(x)\,dx = 1$。这些都说明 $\rho_t(x, z)$ 在每个 t 上都是 (x, z) 的联合概率分布。

在不引起混淆的情况下，我们还将 $\rho_t(x, z)$ 关于 x 的边缘概率记作

$$\rho_t(x) := \int \rho_t(x, z)\,dz. \tag{5.83}$$

同样地，对所有 x 和 t 都有 $\rho_t(x) \geq 0$，且 $\int \rho_t(x)\,dx = \iint \rho_t(x,z)\,dzdx = 1$。此外，我们发现这个边缘概率分布满足

$$\rho_0(x) = \int \rho_0^z(x) p_{\text{data}}(z)\,dz = \int \mathcal{N}(x;0,I_d) p_{\text{data}}(z)\,dz = \mathcal{N}(x;0,I_d),$$

其中第一个等号利用了 (5.82)，而最后两个等号利用了 $\rho_0^z(x) = \mathcal{N}(x;0,I_d)$，它是 x 的函数而与 z 无关，并且

$$\begin{aligned}
\rho_1(x) &= \lim_{t\to 1^-} \rho_t(x) \\
&= \lim_{t\to 1^-} \int \rho_t^z(x) p_{\text{data}}(z)\,dz \\
&= \int \Big(\lim_{t\to 1^-} \rho_t^z(x)\Big) p_{\text{data}}(z)\,dz \\
&= \int \delta_z(x) p_{\text{data}}(z)\,dz \\
&= p_{\text{data}}(x).
\end{aligned} \tag{5.84}$$

（严格来说，(5.84) 中极限与积分的交换一般并不成立；然而，我们通常将 δ_z 看成均值为 z 且方差极小的高斯概率密度函数，并且在假设 p_{data} 为连续函数时，(5.84) 中的推导仍然成立。）这两个边缘分布恰好对应于我们的初始分布 $\mathcal{N}(0,I_d)$ 和目标分布 p_{data}，因此我们希望找到一个向量场 $u:\mathbb{R}^d\times[0,1]\to\mathbb{R}^d$，使得连续方程

$$\partial_t \rho_t = -\nabla\cdot(u_t\rho_t) \tag{5.85}$$

确实能够将 $\rho_0 = \mathcal{N}(0,I_d)$ 传输到 $\rho_1 = p_{\text{data}}$。

为此，我们注意到

$$\begin{aligned}
\partial_t \rho_t(x) &= \frac{\partial}{\partial t}\int \rho_t(x,z)\,dz \\
&= \frac{\partial}{\partial t}\int \rho_t^z(x) p_{\text{data}}(z)\,dz \\
&= \int \Big(\partial_t \rho_t^z(x)\Big) p_{\text{data}}(z)\,dz \\
&= -\int \nabla\cdot(v_t^z(x)\rho_t^z(x)) p_{\text{data}}(z)\,dz \\
&= -\nabla\cdot\Big(\int v_t^z(x)\rho_t^z(x) p_{\text{data}}(z)\,dz\Big)
\end{aligned} \tag{5.86}$$

其中第四个等号利用了 (5.80)，最后一个等号则交换了关于 x 的散度 $\nabla\cdot$ 与关

于 z 的积分次序。注意，最后一个等号中的积分是逐分量作用于被积函数的，因为该被积函数是一个向量。

比较 (5.85) 和 (5.86)，我们可以看出

$$-\nabla\cdot(u_t(x)\rho_t(x)) = -\nabla\cdot\Big(\int v_t^z(x)\rho_t^z(x)p_{\text{data}}(z)\,dz\Big)$$

换句话说，存在一个向量场 $w:\mathbb{R}^d\times[0,1]\to\mathbb{R}^d$，使得对所有 x 和 t 都有 $\nabla_x\cdot(w_t(x)\rho_t(x))=0$，并且

$$u_t(x)\rho_t(x) = \int v_t^z(x)\rho_t^z(x)p_{\text{data}}(z)\,dz + w_t(x)\rho_t(x). \tag{5.87}$$

两边同除以 $\rho_t(x)$，得到

$$\begin{aligned} u_t(x) &= \int v_t^z(x)\frac{\rho_t^z(x)p_{\text{data}}(z)}{\rho_t(x)}\,dz + w_t(x) \\ &= \int v_t^z(x)\frac{\rho_t(x,z)}{\rho_t(x)}\,dz + w_t(x) \\ &= \int v_t^z(x)\rho_t(z|x)\,dz + w_t(x), \end{aligned}$$

其中 $\rho_t(z|x)$ 是在时刻 t 条件于 x 给定时 z 的条件分布（而它们在时刻 t 的联合分布是 $\rho_t(x,z)$）。我们可以安全地忽略 w_t（因为只要满足 $\nabla\cdot(w_t\rho_t)=0$，将连续方程 (5.85) 中的 u_t 替换为 u_t-w_t 并不会改变其解 ρ_t），并取

$$u_t(x) = \int v_t^z(x)\rho_t(z|x)\,dz. \tag{5.88}$$

带有任意给定 $z\in\mathbb{R}^d$ 的条件向量场 v_t^z 与目标向量场 u_t 之间的对比如图 5.5 所示。

由定理 5.3.3 可知，对任意 x 和 t，都有

$$\begin{aligned} u_t(x) &= \int v_t^z(x)\rho_t(z|x)\,dz \\ &= \mathbb{E}_{Z\sim\rho_t(\cdot|x)}[v_t^Z(x)] \\ &= \arg\min_{c\in\mathbb{R}^d}\mathbb{E}_{Z\sim\rho_t(\cdot|x)}[|c-v_t^Z(x)|^2]. \end{aligned}$$

因此，当我们将向量场 $u:\mathbb{R}^d\times[0,1]\to\mathbb{R}^d$ 参数化为一个深度神经网络时，可以通过对 $\rho_t(x)$（它在每个 x 上都是正的）以及 $[0,1]$ 上的均匀分布求总体损

$$\boxed{x_0 \sim \mathcal{N}(0, I_d)} \xrightarrow[\partial_t \rho_t^z = -\nabla \cdot (v_t^z \rho_t^z)]{\dot{x}_t = v_t^z(x_t) := \frac{z - x_t}{1-t}} \boxed{\begin{array}{l} x_1 = z \\ \text{where } z \in \mathbb{R}^d \text{ is} \\ \text{arbitrary and fixed} \\ \text{beforehand} \end{array}}$$

$$\boxed{x_0 \sim \mathcal{N}(0, I_d)} \xrightarrow[\partial_t \rho_t = -\nabla \cdot (u_t \rho_t)]{\dot{x}_t = u_t(x_t)} \boxed{x_1 \sim \rho_1 = p_{\text{data}}}$$

图 5.5: Flow matching 框架中通向任意固定点 $z \in \mathbb{R}^d$ 的条件轨迹（上）以及平均轨迹（下）。在上图和下图中，时间 t 都从 0 增加到 $T = 1$。条件轨迹由带有条件向量场 v_t^z 的 ODE 描述，或者等价地由其对应的 ρ_t^z 的连续方程描述，并且在每个 t 都有 $x_t \sim \rho_t^z$，且无论 x_0 是什么，当 $t \to 1$ 时都有 $x_t \to z$。平均轨迹由带有目标向量场 u_t 的 ODE 描述，或者等价地由其对应的 ρ_t 的连续方程描述，并且满足 $\rho_1 = p_{\text{data}}$。目标向量场 u_t 是利用 v_t^z 与 $z \sim p_{\text{data}}$ 的联合信息，通过 (5.90) 来学习的。

失来训练它：

$$\min_u \int_0^1 \mathbb{E}_{X \sim \rho_t} \mathbb{E}_{Z \sim \rho_t(\cdot|X)} [|u_t(X) - v_t^Z(X)|^2] \, dt,$$

这可以写成更易读的形式

$$\min_u \int_0^1 \iint \rho_t(x) \rho_t(z|x) |u_t(x) - v_t^z(x)|^2 \, dx dz dt. \tag{5.89}$$

注意到

$$\rho_t(x)\rho_t(z|x) = \rho_t(x, z) = \rho_t^z(x) p_{\text{data}}(z) = \mathcal{N}(x; tz, (1-t)^2 I_d) p_{\text{data}}(z),$$

其中第二个等号来自 (5.82)，第三个等号来自 (5.81)。将其代回 (5.89)，我们得到一个等价问题

$$\min_u \int_0^1 \iint p_{\text{data}}(z) \mathcal{N}(x; tz, (1-t)^2 I_d) |u_t(x) - v_t^z(x)|^2 \, dx dz dt. \tag{5.90}$$

(5.90) 中的目标函数还可以进一步简化。为此，我们注意到，为了得到一个样本 $x \sim \mathcal{N}(tz, (1-t)^2 I_d)$，我们可以令

$$x = tz + (1-t)\epsilon \tag{5.91}$$

其中 $\epsilon \sim \mathcal{N}(0, I_d)$。结合 (5.79)，我们知道 (5.90) 中的 $v_t^z(x)$ 变为

$$v_t^z(x) = \frac{z-x}{1-t} = \frac{z-(tz+(1-t)\epsilon)}{1-t} = z-\epsilon. \tag{5.92}$$

因此，我们可以将最小化问题 (5.90) 重写为

$$\min_u \int_0^1 \iint p_{\text{data}}(z)\mathcal{N}(\epsilon;0,I_d)|u_t(tz+(1-t)\epsilon)-(z-\epsilon)|^2\, d\epsilon dz dt. \tag{5.93}$$

我们可以使用 Monte Carlo 积分来近似这个损失函数：

$$\frac{1}{NM}\sum_t \sum_{(z,\epsilon)} |u_t(tz+(1-t)\epsilon)-(z-\epsilon)|^2 \tag{5.94}$$

在实际实现中，我们从 $[0,1]$ 上的均匀分布中抽取 N 个时间点。对每一个时间点 t，我们针对来自 $\mathcal{D}$ 的每个样本 z（$\mathcal{D}$ 由从 p_{data} 中独立抽取的 M 个样本组成），模拟一个 $\epsilon \sim \mathcal{N}(0, I_d)$。然后将这些量全部代入经验损失 (5.94) 中训练 u。训练得到的 u 就是 (5.74) 中所需向量场的一个近似。

5.6 备注与参考文献

5.6.1 生成模型中的噪声采样

在第 5.1、5.3 和 5.5 节中讨论的变分自编码器框架、扩散模型以及 flow matching 方法里，我们最终常常会得到如下形式的损失函数

$$\iint \phi(x,\epsilon)p_{\text{data}}(x)\mathcal{N}(\epsilon;0,I_d)\, d\epsilon dx \tag{5.95}$$

其中 ϕ 是关于 (x,ϵ) 的某个标量值函数。例子包括 (5.17)、(5.53) 和 (5.93)。这里不需要将时间变量 t 纳入考虑：(5.95) 应当在每个 t 上被最小化，而在实际数值实现中我们只是人为地在 $[0,T]$ 上对 t 进行采样。因此，t 的性质与 (x,ϵ) 不同。

我们总是使用 Monte Carlo 积分来近似形如 (5.95) 的损失函数。更具体地说，我们有一组给定样本 $\{x_i \in \mathbb{R}^d : i \in [m]\}$，共 m 个（在本章前面的各节中我们使用的是 M，但这里改用 m，因为我们将考虑对每个给定数据样本抽取 n 个高斯噪声），这些样本被假设为从未知数据分布 p_{data} 中独立抽取而来。与此同时，我们还需要独立地采样 $\epsilon \sim \mathcal{N}(0, I_d)$，即 d 维标准高斯分布。

对 ϵ 有两种采样方式。第一种方式是对每个 x_i 只采样一个 $\epsilon_i \sim \mathcal{N}(0, I_d)$，第二种方式则是对每个 x_i 采样多个彼此独立的 ϵ。在理论上，第一种方式在 Monte Carlo 积分中是正确的，因为它将 (5.95) 中的 $p_{\text{data}}(x)\mathcal{N}(\epsilon; 0, I_d)$ 作为 (x, ϵ) 的联合概率分布。然而，第二种方式也很有吸引力，因为我们可以很容易地从 $\mathcal{N}(0, I_d)$ 中抽取任意多个样本，而不像 x_i 这样的样本那样都是给定的。那么，如果采用第二种方式，会不会有什么问题呢?

为了便于讨论，我们考虑对每个 x_i 采样 n 个独立 ϵ 的情形。更准确地说，假设我们用来构造 (5.95) 的 Monte Carlo 近似的样本为

$$(x_i, \epsilon_{ij}), \quad i \in [m],\ j \in [n], \tag{5.96}$$

其中 x_i 和 ϵ_{ij} 全部彼此独立。于是我们用

$$\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\phi(x_i, \epsilon_{ij}). \tag{5.97}$$

来近似 (5.95)。因此，我们上面提到的两种方式之间的区别，仅仅在于第一种方式中 $n = 1$，而第二种方式中 $n > 1$。

很容易看出，无论 n 的取值为何，(5.97) 的期望都是 (5.95)。换句话说，(5.97) 始终是 (5.95) 的一个无偏估计。

现在我们想考察 (5.97) 的方差，并看看当 $n > 1$ 时它的方差是否会更小。为此，我们先为类似情形推导一个一般性结果。

命题 5.6.1. 设 $m, n \in \mathbb{N}$，并设 $\{A_{ij} : i \in [m],\ j \in [n]\}$ 是 mn 个标量值随机变量，满足

1. 对所有 $i \in [m]$ 和 $j \in [n]$，都有 $\mathbb{E}[A_{ij}] = 0$。
2. 对所有 $i \in [m]$ 和 $j \in [n]$，都有 $\mathrm{Var}[A_{ij}] = v$，其中某个 $v > 0$。
3. 对每个 $i \in [m]$，A_{ij} 与 $\{A_{i'j'} : i' \in [m] \setminus \{i\},\ j \in [n]\}$ 独立。
4. 对每个 $i \in [n]$，对所有 $j, j' \in [n]$ 且 $j \neq j'$，都有 $\mathrm{Cov}[A_{ij}, A_{ij'}] = c$，其中某个 $c > 0$。

则有

$$\mathbb{E}\Big[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\Big] = 0$$

以及

$$\mathrm{Var}\left[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\right]=\frac{v+(n-1)c}{mn}.$$

特别地，当 $n=1$ 时，

$$\mathrm{Var}\left[\frac{1}{m}\sum_{i=1}^{m}A_{i1}\right]=\frac{v}{m}.$$

证明. 我们首先注意到

$$\mathbb{E}\Big[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\Big]=\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\mathbb{E}[A_{ij}]=0.$$

对每个 $i\in[m]$，记

$$B_i:=\sum_{j=1}^{n}A_{ij}.$$

那么我们知道 $B_1,\ldots,B_m$ 彼此独立。此外，有

$$\mathbb{E}[B_i]=\mathbb{E}\Big[\sum_{j=1}^{n}A_{ij}\Big]=\sum_{j=1}^{n}\mathbb{E}[A_{ij}]=0$$

以及

$$\begin{aligned}\mathrm{Var}[B_i]&=\mathrm{Var}\left[\sum_{j=1}^{n}A_{ij}\right]\\&=\sum_{j=1}^{n}\mathrm{Var}[A_{ij}]+2\sum_{1\le j<j'\le n}\mathrm{Cov}[A_{ij},A_{ij'}]\\&=nv+n(n-1)c.\end{aligned}$$

因此，

$$\begin{aligned}\operatorname{Var}\left[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\right] &= \frac{1}{(mn)^2}\operatorname{Var}\left[\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\right] \\ &= \frac{1}{(mn)^2}\operatorname{Var}\left[\sum_{i=1}^{m}B_i\right] \\ &= \frac{1}{(mn)^2}\sum_{i=1}^{m}\operatorname{Var}[B_i] \\ &= \frac{v+(n-1)c}{mn}.\end{aligned}$$

令 $n=1$，便得到 $\operatorname{Var}[\frac{1}{m}\sum_{i=1}^{m}A_{i1}]=\frac{v}{m}$。 □

我们可以通过计算它们的差值来比较 $\frac{1}{m}\sum_{i=1}^{m}A_{i1}$ 和 $\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}$ 的方差：

$$\begin{aligned}\operatorname{Var}\left[\frac{1}{m}\sum_{i=1}^{m}A_{i1}\right]-\operatorname{Var}\left[\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}\right] &= \frac{v}{m}-\frac{v+(n-1)c}{mn} \\ &= \frac{(n-1)(v-c)}{mn}. \qquad (5.98)\end{aligned}$$

因此，当 $v>c$ 时，使用 $n>1$ 可能是有益的，因为无偏估计 $\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}A_{ij}$ 的方差会更小。

为了将命题 5.6.1 的结果应用到我们的问题中，我们为了记号简洁起见，假设 (5.95)，也就是 (5.97) 的期望，为零。这是因为期望的取值并不会影响方差的大小。

现在我们将命题 5.6.1 中的 A_{ij} 取为 $\phi(X_i,E_{ij})$，其中对所有 $i\in[m]$ 和 $j\in[n]$，$X_i\sim p_{\text{data}}$ 且 $E_{ij}\sim\mathcal{N}(0,I_d)$，并且它们彼此独立，那么命题 5.6.1 中的 v 和 c 分别为

$$v=\operatorname{Var}_{(X_i,E_{ij})}[\phi(X_i,E_{ij})]$$

以及

$$c=\operatorname{Cov}_{(X_i,E_{ij},E_{ij'})}[\phi(X_i,E_{ij}),\phi(X_i,E_{ij'})].$$

因此，我们需要检查 v 和 c 哪一个更大。

一般来说，$\phi(x,\epsilon)$ 是一个复杂函数，例如 (5.53) 和 (5.93)。我们可以看到，在这些情形中，ϕ 包含了一些需要训练的深度神经网络。因此，在一般情形下，我们无法断定 v 和 c 哪一个更大。不过，我们注意到，即使 $v-c$ 为

正（并且固定），其带来的收益（即差值 (5.98) 的大小）也未必显著：(i) 增大 n 会使得 $\frac{n-1}{n} \to 1$，从而差值 (5.98) 在较大 n 时趋于饱和，但 (5.97) 的计算代价却会上升；(ii) 样本量 m 越大（这由我们已有的数据决定，无法在 (5.97) 的计算中改变），差值 (5.98) 就越小，这说明如果我们已经有大量样本，那么使用较大的 n 可能并不值得。

尽管如此，我们仍然可以考虑一个简单情形，即 $\phi(x, \epsilon) = x + \epsilon$，其中 x 和 ϵ 都是标量值。此时有 $X_i \sim p_{\text{data}}$ 定义在 $\mathbb{R}$ 上，而 $E_{ij}, E_{ij'} \sim \mathcal{N}(0, 1)$，并且它们彼此独立。在这种情况下，

$$v = \text{Var}_{(X_i, E_{ij})}[X_i + E_{ij}] = \text{Var}[X_i] + 1$$

以及

$$c = \text{Cov}_{(X_i, E_{ij}, E_{ij'})}[X_i + E_{ij}, X_i + E_{ij'}] = \text{Var}[X_i].$$

因此，我们确实在这个简单例子中得到了 $v > c$。此外，我们还注意到 $X_i + E_{ij}$ 与 $X_i + E_{ij'}$ 是相关的，因为它们的协方差为正。

这个简单例子还给出了一个警示：当我们试图通过向给定样本中加入噪声或类似方式来制造更多“训练样本”时，需要格外谨慎，因为这些人为构造出的样本在统计上是相关的（它们的协方差非零）。因此，我们前面讨论的方法中某些理论推导所依赖的假设，可能不再成立。

5.6.2 条件生成模型

本章中讨论的生成模型旨在生成与观测数据相似的新样本，这些观测数据服从某个未知概率分布 p_{data}。在真实场景中，我们经常希望在给定某些输入信息 $y \in \mathbb{R}^k$ 的条件下生成数据（这里为了便于下文讨论，我们将其表示为向量，而在不同方法中它也可以是某种指定形状的张量），例如类别标签、属性、文本描述或其他模态的信息。这就引出了**条件生成**（conditional generation）问题，其目标是从条件分布 $p_{\text{data}}(x|y)$ 中生成样本，从而实现可控、有针对性且可解释的生成。

现有的条件生成框架主要区别在于它们如何表示、学习以及生成服从 $p_{\text{data}}(x|y)$ 的样本，但它们都建立在各自对应的无条件生成原型（比如本章前面介绍的那些方法）之上。它们一个共性是将条件变量 y 注入到生成过程中，或者通过概率建模显式引入，或者通过神经网络结构引入。下面我们简要介绍几种现有的条件生成方法。需要注意的是，在生成模型社区中，我们也常将这一过程称为**引导生成**（guided generation），并将 y 称为**引导信号**而不是

条件，这些术语有时有助于避免与推导中出现的各种条件概率分布相混淆。

Conditional GAN 是一种基于 GAN 的条件生成模型，它在生成器 g_θ 和判别器 a_η 中同时引入引导信号 y。生成器 g_θ 是一个参数为 θ 的神经网络，并学习从一个简单概率分布的样本 z（通常像 GAN 中那样从 $\mathcal{N}(0, I_d)$ 中抽取，定义在 $\mathbb{R}^d$ 上）和一个引导信号 $y \in \mathbb{R}^k$ 到数据空间的映射。因此，生成器 g_θ 可以用来生成服从条件概率 $p_\theta(x|y)$ 的样本。与此同时，判别器被参数化为对抗网络 $a_\eta(\cdot|y)$，它在给定 y 的条件下区分真实样本和生成样本。参数 (θ, η) 的训练通过求解如下鞍点问题来实现：

$$\min_\theta \max_\eta \Big\{ \iint p_{\text{data}}(x, y) \log a_\eta(x|y)\, dxdy + \iint p(y) p_\theta(x|y) \log(1 - a_\eta(x|y))\, dxdy \Big\}. \tag{5.99}$$

在实际计算中，通过对给定数据集中的独立样本对 (x, y) 做 Monte Carlo 积分来近似 (5.99) 中第一个积分，其中 y 是与 x 对应的引导信号。(5.99) 中的概率 $p(y)$ 指的是引导信号的概率分布，在训练过程中通过对所有需要用到的引导信号采样来实现。Conditional GAN 在生成清晰样本方面尤其有效，并被广泛应用于图像生成和迁移。

我们也可以将从条件概率 $p_{\text{data}}(\cdot|y)$ 中采样的思想应用到流匹配（flow matching）框架中。更具体地说，我们可以将 flow matching 方法（第 5.5 节）中的概率 $\rho_t(x)$ 替换为带有引导信号 y 的条件概率 $\rho_t(x|y)$。为此，我们首先注意到

$$\rho_t(x|y) = \int \rho_t(x|y, z) p_{\text{data}}(z|y)\, dz = \int \rho_t^z(x) p_{\text{data}}(z|y)\, dz \tag{5.100}$$

其中 ρ_t^z 是在 (5.81) 中给出的、具有给定目标 z 的概率流，而 $p_{\text{data}}(z|y)$ 在利用 Monte Carlo 积分近似 (5.100) 时，同样会由带有引导变量 y 标记的给定样本 z 来替代。注意到，在 (5.100) 中我们假设 $\rho_t(x|y, z) = \rho_t(x|z) = \rho_t^z(x)$，因为从 x 到任意指定的 $z \sim p_{\text{data}}$ 的路径是由 x 和 z 决定的，而不是由 y 决定的，这是因为关于 y 的信息已经完全包含在 z 之中。与第 5.5 节中的推导类似，我们可以证明，起始时刻 $t = 0$ 和终止时刻 $t = 1$ 的边缘分布满足

$$\rho_0(\cdot|y) = \mathcal{N}(0, I_d) \quad \text{和} \quad \rho_1(\cdot|y) = p_{\text{data}}(\cdot|y),$$

并且带引导的向量场 $u_t(\cdot|\cdot):\mathbb{R}^d\times\mathbb{R}^k\to\mathbb{R}^d$ 为

$$u_t(x|y)=\int v_t^z(x)\rho_t(z|x,y)\,dz=\operatorname*{arg\,min}_{c\in\mathbb{R}^d}\mathbb{E}_{Z\sim\rho_t(\cdot|x,y)}\Big[|c-v_t^Z(x)|^2\Big],$$

这意味着，当 u_t 被参数化为深度神经网络时，它可以通过求解

$$\min_u\int_0^1\mathbb{E}_{(X,Y)\sim p_{\text{data}}}\mathbb{E}_{Z\sim\rho_t(\cdot|X,Y)}\Big[|u_t(X|Y)-v_t^Z(X)|^2\Big]\,dt. \tag{5.101}$$

来训练。注意，这里的最小化仍然是关于 u 的网络参数进行的。文献表明，这种条件生成在每个引导信号 y 在给定数据集中都有大量样本 x 与之对应的应用中最为有效，例如类别引导 [103]。然而，在一般情形下这可能是很困难的，因为许多引导信号 y 是不重复且复杂的，例如图像的标题（captions of figures）[86]。

扩散模型也可以被修改为条件扩散模型。为此，人们发展了 classifier guidance [32, 139] 和 classifier-free guidance [64] 方法。具体而言，由 Bayes 法则可知

$$\rho_t(x|y)=\frac{\rho_t(y|x)\rho_t(x)}{p(y)}$$

其中 $p(y)$ 与上文一样表示引导信号的分布，而 $\rho_t(x)$ 和 $\rho_t(x|y)$ 分别是前面提到的无条件和条件概率分布。对上式两边取对数，再对 x 求梯度，就得到

$$\underbrace{\nabla_x\log\rho_t(x|y)}_{\text{conditional score}}=\underbrace{\nabla_x\log\rho_t(y|x)}_{\text{classifier score}}+\underbrace{\nabla_x\log\rho_t(x)}_{\text{unconditional score}}. \tag{5.102}$$

因此，只要我们得到 classifier score，就可以获得所需的 conditional score，并按照扩散模型中的推导训练网络 $s_t(x|y)$ 来近似它。

基于这一关系，classifier guidance 方法训练一个时变函数 $\phi_t(y|x)$，并将其参数化为深度神经网络，用以近似 classifier score $\nabla_x\rho_t(y|x)$。我们强调，这个函数 ϕ 是与扩散模型中的 unconditional score 分开训练的；更重要的是，它是时变的，即对任意时刻 t 和 $x\sim\rho_t(\cdot)$ 都能逼近这个 $\nabla_x\rho_t(y|x)$。[86] 表明，在扩散模型中使用与 (5.75) 相同的线性插值时，带引导的向量场为

$$u_t(x|y)=u_t(x)+\frac{1-t}{t}\phi_t(y|x) \tag{5.103}$$

同时文中还给出了一个可定制插值系数的公式表。这种方法在类别条件和文本条件两种情形下，都比 (5.101) 更能准确逼近带引导的向量场 [104]。

由于 classifier score 与 unconditional score 是分开学习的，因此通常需要在 (5.103) 右端第二项中的 ϕ_t 前乘上一个标量 $\gamma \in \mathbb{R}$ 来校准 classifier score 的权重。这个标量通常取为 $\gamma > 1$ [32]。

另一种不需要单独训练 classifier score 函数的方法，称为 classifier-free guidance (CFG)，由 [64] 提出。由 (5.102) 可见，classifier score 就是 conditional score 与 unconditional score 的差。因此，CFG 方法只训练一个单独的网络来近似 conditional score $\nabla_x \rho_t(x|y)$，并取一个固定向量 y_{null} 作为占位符，用于输入样本 z 没有引导信号时（即，$z \sim p_{\text{data}}(\cdot)$）。根据这个想法，我们可以得到如下带引导信号的向量场 [86]：

$$u_t(x|y) = (1-\gamma)u_t(x|y_{\text{null}}) + \gamma u_t(x|y). \tag{5.104}$$

训练 u 的损失函数中还包括另一层数据选择：首先根据某个用户选取的 $p \in (0,1)$，通过 Bernoulli(p) 随机抽取 y_{null} 或某个真实引导信号 y；然后如果抽到了 y_{null}，则对 z 做无引导采样，否则就在给定所抽引导信号 y 的条件下对 z 采样。接下来的步骤与扩散模型中的做法相同 [64]。一些后续工作针对 CFG 采样提出了不同的直观解释或理论论证，参见，例如 [22,28,57]。

5.6.3 生成模型的参考文献

早期的生成建模工作主要由经典概率方法主导，例如高斯混合模型、隐 Markov 模型以及其他具有手工设计结构的潜变量模型。尽管这些方法在数学上基础扎实且具有可解释性，但它们在扩展到高维数据（如图像、音频信号和自然语言）时往往表现不佳。将深度神经网络作为灵活的函数逼近器引入后，生成模型发生了重大转变，使其能够更有效地刻画复杂的、多模态的数据分布。

自编码器的概念最早出现在 [62,123] 等早期工作中，其目标是利用潜变量建立低维表示，从而抓住给定高维数据的本质。VAE 最早由 [78] 提出。[79] 给出了关于 VAE 的系统性介绍。VAE 采用编码器–解码器结构，将数据映射到连续潜空间中，并通过生成式解码器重构样本。这一形式使得高效采样、结构化表示学习以及稳定优化成为可能。然而，人们经常观察到 VAE 会生成过于平滑或模糊的样本，这反映了基于似然的目标与感知可信度之间的一种权衡。

针对 VAE，人们提出了大量改进和扩展。例如，条件 VAE 在 [136] 中被提出，并表明输入条件在编码后可以在潜空间中以不同方式表示。在 [137] 中，构造了 ladder VAE，通过数据相关的近似似然项递归地修正生成分布。[56]

提出了一种作用于输出变量的自回归模型。[81] 表明，通过使用对抗损失可以提升生成样本的质量。过去几年中，人们还研究了多种不同形式的后验变分近似，包括 normalizing flows [77, 117]、图模型 [91] 以及面向时序数据的模型 [52, 53]。

生成对抗网络（GANs）最初由 [50] 提出。关于 GAN 及其变体的综述可见 [54,93,154]。原始 GAN 在训练上被认为是不稳定的 [40,72]，自那以后人们提出了大量变体和改进 [5,55,94,95,113,160]。其中，Wasserstein GAN 由 [5] 提出，并进一步发展出了若干变体 [1,157]。

展示扩散模型基本思想的早期工作出现在 [135] 中，该文引入了通过 Markov 链将数据逐步扩散为噪声，并学习一个逆时间过程来恢复数据分布的概念。扩散模型在实践上的成功则由去噪扩散概率模型（denoising diffusion probabilistic models，DDPMs）[63] 显著推动。[103] 通过改进噪声调度和网络结构，对 DDPM 进行了提升，从而显著提高了样本保真度。此外，[75] 分析了扩散模型中的设计选择，并提出了能够提升训练稳定性和效率的原则性参数化方式。这些进展使得扩散模型在许多基准任务上超越了 GAN，尤其是在模式覆盖和训练鲁棒性方面。DDPM 与由 SDE 表示的基于 score 的生成建模之间的联系在 [139] 中得以建立。由 ODE 表示的确定性版本则见于 [138]。除图像生成之外，扩散模型还已成功扩展到条件生成、超分辨率、文生图合成以及科学计算应用等场景。典型例子包括 classifier-guided diffusion [32] 和大规模文本条件模型（如 latent diffusion models [121]），后者通过在学习得到的潜空间中操作来降低计算成本。

与扩散模型相比，flow matching 是一个更加简洁且进化的生成建模框架 [2,85,89]，并且已经在语音 [82]、图像 [36]、视频 [111]、机器人 [18] 以及蛋白质 [71] 等大规模生成任务中得到了应用。[87,131] 表明，Flow Matching 的线性版本可以看作 bridge matching 某种极限情形。flow matching 中使用向量场的思想与 [27,51] 中基于似然的概率密度匹配相关。近期的改进与扩展见于 [60,101,147]。flow matching 框架还被推广到了离散状态情形 [25,48] 以及 Riemann 流形上 [26]。[66] 进一步表明，flow matching 框架适用于一般连续时间 Markov 过程。

总体而言，包括上述所有变体在内的生成模型构成了一类灵活且具有坚实理论基础的生成模型方法，而当前的持续研究则致力于加速采样、提升可扩展性，并将其适用范围拓展到更广泛的数据模态。

附录 A　补充材料

A.1　Monte Carlo 积分

蒙特卡洛（Monte Carlo）积分是一种在高维欧几里得空间子集上积分的近似计算方法。对于此类积分，经典的数值积分方法通常不适用。这是因为这些经典方法需要对积分区域进行空间离散化，这样会带来的维度灾难问题，即离散化得到的格点数随着维度以指数速度增长。相比之下，Monte Carlo 积分不需要显式的空间离散化，而是通过遵循某种概率分布从积分区域中抽取样本点，并在这些样本点上计算被积函数的值。随后，对这些点上的函数值进行适当的平均通常可以得到对目标积分的合理近似。在本节中，我们将给出关于 Monte Carlo 积分方法及其相关数学性质的一些细节。

Monte Carlo 积分的基本思想如下。若 $\Omega \subset \mathbb{R}^d$ 为有界集合且 $h : \Omega \to \mathbb{R}$ 在 Ω 上可积，则其积分可以通过如下方法逼近：

$$\int_\Omega h(x)\,dx \approx \frac{|\Omega|}{N}\sum_{i=1}^{N} h(x^{(i)}), \tag{A.1}$$

其中 $\{x^{(i)} \in \Omega : i \in [N]\}$ 是从 Ω 上的均匀分布独立抽取的 N 个样本，而 $|\Omega|$ 表示 Ω 在 $\mathbb{R}^d$ 中的体积。接下来，我们考虑该方法的一个简单推广（式 (A.1) 是此推广形式的一个特例），并分析这种 Monte Carlo 积分近似的偏差和方差。

这种推广称为 **重要性采样**（importance sampling）。设 Ω 是 $\mathbb{R}^d$ 的一个（可能无界的）子集，p 是定义在 Ω 上的概率密度函数，且对所有 $x \in \Omega$ 有 $p(x) > 0$。设 $h : \Omega \to \mathbb{R}$ 在 Ω 上可积，且 $X^{(1)}, \ldots, X^{(N)}$ 是从分布 p 中抽取的 N 个独立同分布随机变量。记

$$I_h := \int_\Omega h(x)\,dx \quad \text{和} \quad J_h^N := \frac{1}{N}\sum_{i=1}^{N} \frac{h(X^{(i)})}{p(X^{(i)})}. \tag{A.2}$$

则 J_h^N 是对目标积分 J_h 的一个统计估计。注意，J_h^N 是一个样本复杂度为 N 的统计量，而 I_h 是一个待由 J_h^N 近似的常数。

我们可以看到，(A.1) 是 (A.2) 在均匀分布 $p(x) = 1/|\Omega|$（对所有 $x \in \Omega$）下的特例。然而，(A.1) 要求 Ω 是 $\mathbb{R}^d$ 的有界子集，而 (A.2) 则允许处理无界 Ω 的情况。

现在我们来分析 J_h^N 对 I_h 的近似误差。首先注意到

$$\begin{aligned}\mathbb{E}[J_h^N] &= \mathbb{E}\Big[\frac{1}{N}\sum_{i=1}^{N}\frac{h(X^{(i)})}{p(X^{(i)})}\Big] = \frac{1}{N}\sum_{i=1}^{N}\mathbb{E}_{X^{(i)}\sim p}\Big[\frac{h(X^{(i)})}{p(X^{(i)})}\Big] \\ &= \frac{1}{N}\sum_{i=1}^{N}\int_{\Omega}\frac{h(x)}{p(x)}p(x)\,dx = J_h,\end{aligned}$$

这说明 J_h^N 是 I_h 的无偏估计。

此外，对于每个 $X^{(i)}$，有

$$\mathrm{Var}\Big[\frac{h(X^{(i)})}{p(X^{(i)})}\Big] = \mathbb{E}\Big[\Big|\frac{h(X^{(i)})}{p(X^{(i)})}\Big|^2\Big] - \Big|\mathbb{E}\Big[\frac{h(X^{(i)})}{p(X^{(i)})}\Big]\Big|^2 = \int_{\Omega}\frac{h(x)^2}{p(x)}\,dx - I_h^2.$$

因此，

$$\begin{aligned}\mathrm{Var}[J_h^N] &= \mathrm{Var}\Big[\frac{1}{N}\sum_{i=1}^{N}\frac{h(X^{(i)})}{p(X^{(i)})}\Big] = \frac{1}{N}\,\mathrm{Var}\Big[\frac{h(X^{(i)})}{p(X^{(i)})}\Big] \\ &= \frac{1}{N}\Big(\int_{\Omega}\frac{h(x)^2}{p(x)}\,dx - I_h^2\Big) = O\Big(\frac{1}{N}\Big),\end{aligned} \tag{A.3}$$

这表明 J_h^N 的方差以 $1/N$ 的速度衰减，其中 N 是采样点数量。同时，有

$$I_h^2 = \Big(\int_{\Omega}h(x)dx\Big)^2 = \Big(\int_{\Omega}\frac{h(x)}{p(x)^{1/2}}p(x)^{1/2}\,dx\Big)^2 \le \int_{\Omega}\frac{h(x)^2}{p(x)}\,dx.$$

上式当且仅当 $p \propto h$ 时取等号，而且此时 (A.3) 中方差的常数项达到最小。

然而，一般情况下，我们很难找到 p 使得 $p \propto h$，因此重要性采样的思想是选择一个相对简单的概率分布（如高斯混合分布）作为 p，并使其尽可能接近 h。选择高斯混合分布的原因在于可以方便地从 p 中采样 $X^{(i)}$ 并计算 $h(X^{(i)})/p(X^{(i)})$。具体而言，可以将各高斯分布的均值设在 h 取得峰值的位置，标准差根据 h 的形状设定，并调整混合权重使 p 尽可能与 h 成比例。

总体而言，通过增加样本数量 N 并合理选择 p，可以减小 J_h^N 对 I_h 的近似误差。同时，收敛速率 $O(1/N)$ 不依赖于维度 d，因此在使用空间离散化的

情形下，Monte Carlo 方法可以有效地缓解维度灾难问题。

然而，在实际应用中，Monte Carlo 积分往往更加复杂。例如，我们常需用 Monte Carlo 积分来估计下式对 θ 的梯度：

$$I_h(\theta) := \frac{1}{2}\int_\Omega |h(x,\theta)|^2 dx,$$

其中 $h : \mathbb{R}^d \times \mathbb{R}^m \to \mathbb{R}^n$，$\theta \in \mathbb{R}^m$ 表示待优化的神经网络参数。由于大多数训练算法是基于一阶优化方法，我们需要近似

$$\nabla_\theta I_h(\theta) = \int_\Omega \nabla_\theta h(x,\theta) h(x,\theta)\, dx,$$

该量是一个 m 维向量。在使用 Monte Carlo 积分时，$\nabla_\theta I_h(\theta)$ 的不同分量可能具有不同方差，因此需要谨慎选择用于重要性采样的分布 p，尽量减小近似方差。正如第 二 章所示，这个随机梯度的方差对随机梯度方法（及类似算法）的收敛性具有重要影响。

A.2 Banach 空间与不动点理论

在本节中，我们对 Banach 空间与不动点理论进行简要回顾。本节中的结果是第 4.1 节中 Bellman 算子与 Markov 决策过程理论的基础。

我们首先给出 Banach 空间的定义，即完备赋范线性空间。我们先介绍相关概念，然后给出正式定义。随后介绍 Banach 不动点定理。最后，我们证明在第 4.1 节中考虑的 $B(\mathcal{X})$ 是一个 Banach 空间，从而可以应用该定理。

定义 A.2.1 (线性空间)**.** 设 X 为一个集合。如果在 X 上定义了加法和数乘运算（即对任意 $x, y \in X$ 和 $a \in \mathbb{R}$，有 $x + y \in X$ 且 $ax \in X$），并且对任意 $x, y, z \in X$ 和 $a, b \in \mathbb{R}$ 满足以下性质，则称 X 为 **线性空间**（亦称 **向量空间**）：

1. $x + y = y + x$；
2. $(x + y) + z = x + (y + z)$；
3. 存在 $0 \in X$ 使得对所有 $x \in X$，有 $x + 0 = 0 + x$；
4. 对任意 $x \in X$，存在 $x' \in X$（记为 $-x$）使得 $x + x' = 0$；
5. $a(bx) = (ab)x$；
6. $1 \cdot x = x$，其中 $1 \in \mathbb{R}$；

7. $(a+b)x = ax + bx$;

8. $a(x+y) = ax + ay$。

定义 A.2.2 (范数)**.** 函数 $\|\cdot\| : X \to \mathbb{R}$ 称为一个 **范数**（norm），如果对任意 $x, y, z \in X$ 和 $a \in \mathbb{R}$，以下性质成立：

1.（正定性）$\|x\| \geq 0$；且当且仅当 $x = 0$ 时 $\|x\| = 0$。

2.（齐次性）$\|ax\| = |a|\|x\|$。

3.（三角不等式）$\|x+y\| \leq \|x\| + \|y\|$。

如果线性空间 X 上定义了一个范数，则称 $(X, \|\cdot\|)$ 为一个 **赋范线性空间**。

定义 A.2.3 (距离)**.** 设 X 为一个集合。若函数 $d : X \times X \to \mathbb{R}$ 满足对任意 $x, y, z \in X$：

1. $d(x,y) \geq 0$；且当且仅当 $x = y$ 时 $d(x,y) = 0$;

2. $d(x,y) = d(y,x)$；以及

3. $d(x,z) \leq d(x,y) + d(y,z)$,

则称 d 为一个 **距离**（或 **度量**）。

如果 X 是一个带范数 $\|\cdot\|$ 的线性空间，则函数 $d : X \times X \to \mathbb{R}$ 定义为 $d(x,y) := \|x-y\|$ 是一个距离（这可以通过验证定义 A.2.3 中的条件容易得到）。在下文中，我们始终假设赋范线性空间 $(X, \|\cdot\|)$ 上的距离 d 由此方式定义。

定义 A.2.4 (Cauchy 序列)**.** 在赋范线性空间 $(X, \|\cdot\|)$ 中，一个序列 $\{x_k\}$ 称为 **Cauchy 序列**，如果对任意 $\epsilon > 0$，存在 $K \in \mathbb{N}$，使得对所有 $k, j \geq K$，都有 $\|x_k - x_j\| < \epsilon$。

定义 A.2.5 (Banach 空间)**.** 如果赋范线性空间 $(X, \|\cdot\|)$ 中的每个 Cauchy 序列都收敛到 X 中的某一点，则称其为 **完备的**。一个完备的赋范线性空间 $(X, \|\cdot\|)$ 称为 **Banach 空间**。

若上下文中范数 $\|\cdot\|$ 已明确，我们将 $(X, \|\cdot\|)$ 简记为 X。现在我们考虑 Banach 空间上的运算。

定义 A.2.6 (Lipschitz、压缩映射与不动点)**.** 设 X 为 Banach 空间。若映射（算子）$T : X \to X$ 满足存在常数 $\gamma > 0$ 使得

$$\|Tx - T\hat{x}\| \leq \gamma\|x - \hat{x}\|$$

对任意 $x, \hat{x} \in X$ 成立，则称 T 在 X 上是 **Lipschitz** 的，常数为 γ。当 $\gamma = 1$ 时，称 T 为 **非扩张**映射；当 $\gamma \in (0, 1)$ 时，称 T 为 **γ-压缩映射**（或称 T 是 **γ-压缩的**）。若存在 $x \in X$ 满足 $Tx = x$，则称 x 为 T 的一个 **不动点**。

定理 A.2.7 (Banach 不动点定理)**.** *设 X 为 Banach 空间，$T : X \to X$ 为一个 γ-压缩映射，其中 $\gamma \in (0, 1)$。则有：*

(i) T 在 X 中存在唯一的不动点 x^*。

*(ii) 对任意 $x \in X$，定义序列 $x_{k+1} = Tx_k$ 且 $x_0 = x$（该构造称为 **不动点迭代**）。则对任意 $k \in \mathbb{N}$，有*

$$\|x_{k+1} - x_k\| \leq \gamma\|x_k - x_{k-1}\|, \tag{A.4}$$

以及

$$\|x_k - x^*\| \leq \gamma^k\|x - x^*\|. \tag{A.5}$$

称 x_k 以速率 γ 线性收敛到 x^*。

证明. 我们同时证明上述两个结论。取任意 $x \in X$ 并令 $x_0 = x$。设 $\{x_k\}$ 为由 γ-压缩映射 T 生成的不动点迭代序列。则 (A.4) 由 T 的定义直接得到。进一步，对任意 $k \in \mathbb{N}$，有

$$\begin{aligned}
\|x_{k+1} - x_k\| &= \|Tx_k - Tx_{k-1}\| \\
&\leq \gamma\|x_k - x_{k-1}\| \\
&\leq \cdots \\
&\leq \gamma^k\|x_1 - x_0\|.
\end{aligned}$$

因此，对任意 $j \in \mathbb{N}$，

$$\begin{aligned}\|x_{k+j} - x_k\| &\le \|x_{k+j} - x_{k+j-1}\| + \cdots + \|x_{k+1} - x_k\| \\ &\le (\gamma^{k+j-1} + \cdots + \gamma^k)\|x_1 - x_0\| \\ &\le \frac{\gamma^k}{1-\gamma}\|x_1 - x_0\|.\end{aligned}$$

因此，当 k 足够大时，$\|x_{k+j} - x_k\|$ 可任意小（与 j 无关），从而 $\{x_k\}$ 为 X 中的 Cauchy 序列。由于 X 完备，x_k 收敛到某个 $x^* \in X$。此外，由 $\lim_{k\to\infty} \|Tx_k - Tx^*\| = \lim_{k\to\infty} \|x_{k+1} - x^*\| = 0$，可得 $\|Tx^* - x^*\| = 0$，即 $x^* = Tx^*$，因此 x^* 为不动点。

接下来，

$$\begin{aligned}\|x_k - x^*\| &= \|Tx_{k-1} - Tx^*\| \\ &\le \gamma\|x_{k-1} - x^*\| \\ &\le \cdots \\ &\le \gamma^k\|x_0 - x^*\| \\ &= \gamma^k\|x - x^*\|,\end{aligned}$$

这证明了 (A.5)。

若存在两个不同的不动点 x^* 和 $\hat{x}$，则 $\|x^* - \hat{x}\| > 0$，但

$$\|x^* - \hat{x}\| = \|Tx^* - T\hat{x}\| \le \gamma\|x^* - \hat{x}\| < \|x^* - \hat{x}\|,$$

产生矛盾。因此不动点唯一。 □

现在我们考虑第 4.1 节中的空间 $B(\mathcal{X})$。回顾 $\mathcal{X}$ 是一个集合，它可以是包含有限个或可数多个元素的集合，或者是某个欧几里得空间的子集。接下来我们定义

$$B(\mathcal{X}) := \Big\{v : \mathcal{X} \to \mathbb{R} : \sup_{x\in\mathcal{X}} |v(x)| < \infty\Big\}. \tag{A.6}$$

即，$B(\mathcal{X})$ 是定义在 $\mathcal{X}$ 上的所有一致有界实值函数所组成的集合。我们还在 $B(\mathcal{X})$ 上定义 ∞-范数：

$$\|v\|_\infty := \sup_{x\in\mathcal{X}} |v(x)|. \tag{A.7}$$

显然，$(B(\mathcal{X}), \|\cdot\|_\infty)$ 是一个赋范线性空间。接下来，我们想说明 $(B(\mathcal{X}), \|\cdot\|_\infty)$

是完备的。

为此，设 $\{v_k\}$ 是 $(B(\mathcal{X}), \|\cdot\|_\infty)$ 中的一个 Cauchy 序列。也就是说，对任意 $\epsilon > 0$，存在 $K \in \mathbb{N}$，使得当 $k, j \geq K$ 时，

$$\|v_k - v_j\|_\infty < \epsilon.$$

因此，对任意 $x \in \mathcal{X}$，有

$$|v_k(x) - v_j(x)| \leq \|v_k - v_j\|_\infty < \epsilon \tag{A.8}$$

对所有 $k, j \geq K$ 成立。这说明 $\{v_k(x)\}$ 是 $\mathbb{R}$ 中的一个 Cauchy 数列。因此，$\{v_k(x)\}$ 在 $\mathbb{R}$ 中收敛，并有极限，记为 $v(x)$。于是对每个 $x \in \mathcal{X}$，定义 $v : \mathcal{X} \to \mathbb{R}$ 为该极限。接着，对任意 $x \in \mathcal{X}$，令 (A.8) 中 $j \to \infty$，得到

$$|v_k(x) - v(x)| \leq \epsilon$$

只要 $k \geq K$ 就成立。由于 x 是任意的，因此我们得到

$$\|v_k - v\|_\infty \leq \epsilon \tag{A.9}$$

对所有 $k \geq K$ 成立。由于 ϵ 是任意的，可知当 $k \to \infty$ 时，$\|v_k - v\|_\infty \to 0$。此外，由 (A.9) 可见，对任意 $k \geq K$，有 $\|v\|_\infty \leq \epsilon + \|v_k\|_\infty < \infty$，这说明 $v \in B(\mathcal{X})$。综上，$\{v_k\}$ 在 ∞-范数下收敛，且其极限 v 仍属于 $B(\mathcal{X})$。因此，$B(\mathcal{X})$ 是完备的。由定义 A.2.5，$B(\mathcal{X})$ 是一个 Banach 空间。

A.3 信息论基础

在本节中，我们对信息论的基础知识及其与概率的关系作一个简要回顾。

定义 A.3.1 (熵)**.** 设 X 是一个服从 $\mathbb{R}^n$ 上概率分布 p 的随机变量。则 X 的**熵**（entropy）定义为

$$H(X) := \mathbb{E}_{X\sim p}[-\log p(X)] = -\int p(x)\log p(x)\,dx. \tag{A.10}$$

如果 X 是离散随机变量，即 X 的所有可能取值组成的集合 $\mathcal{X}$ 是有限集或可数无限集，则 (A.10) 中的积分应替换为对 $\mathcal{X}$ 的求和。除非某个公式仅适用于求和情形，否则本节统一使用积分记号。注意，我们总采用约定 $0\log 0 = 0$，

因为 $\lim_{z\to 0^+} z\log z = 0$。

随机变量的熵（等价地，其概率分布的熵）刻画了平均不确定性水平。熵对随机变量的均值（期望）是平移不变的，即随机变量的均值发生平移时，熵不变。方差和维数是决定熵值的主要因素，但对于一般随机变量而言，还有其他因素也会影响熵的大小。

如果 X 是离散随机变量，则 $H(X) \geq 0$。要说明这一点，只需注意对每个 $x \in \mathcal{X}$，由于 $p(x) \in [0,1]$，有 $p(x)\log p(x) \in [-e^{-1}, 0]$。

如果 X 是连续随机变量，则 $H(X)$ 可以为负。一个特殊且重要的例子是 d 维高斯随机变量 $X \sim \mathcal{N}(\mu, \Sigma)$ 的熵。可以验证，

$$H(X) = \frac{d}{2}(1 + \log 2\pi) + \frac{1}{2}\log\det(\Sigma).$$

由此可见，当 $\Sigma = \sigma^2 I_n$ 且 $\sigma \to 0$ 时，$H(X) \to -\infty$。

定义 A.3.2 (条件熵)**.** 设 X 和 Y 是两个随机变量，它们在 $\mathbb{R}^n \times \mathbb{R}^m$ 上具有联合概率分布 p。则在给定 X 的条件下，Y 的 **条件熵** 定义为

$$H(Y|X) := -\iint p(x,y)\log\frac{p(x,y)}{p_X(x)}\,dxdy, \tag{A.11}$$

其中 p_X 是 X 的边缘分布。

设 $H(X,Y)$ 表示 (X,Y) 的联合熵，即

$$H(X,Y) = -\iint p(x,y)\log p(x,y)dxdy.$$

则有

$$H(Y|X) = H(X,Y) - H(X).$$

容易验证，当且仅当 Y 完全由 X 决定（即 Y 是 X 的函数）时，$H(Y|X) = 0$。并且当且仅当 X 与 Y 相互独立时，$H(Y|X) = H(Y)$。

定义 A.3.3 (Bregman 散度)**.** 设 F 是一个凸泛函，其一阶变分为 δF。则与 F 相关的 **Bregman 散度** 定义为

$$D_F(g,f) := F(g) - F(f) - \int \delta F(f)\cdot(g-f),$$

其中 f 与 g 是任意两个函数。

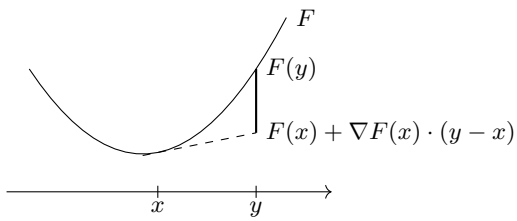


图 A.1: 与 F（凸曲线）相关的 Bregman 散度示意图。经过点 $(x, F(x))$ 且斜率为 $\nabla F(x)$ 的切线用虚线表示。在点 y 处，函数 F 与该切线的取值分别为 $F(y)$ 和 $F(x) + \nabla F(x) \cdot (y - x)$。它们的差（点 y 上方竖直实线段的长度）就是 Bregman 散度 $D_F(y, x) = F(y) - F(x) - \nabla F(x) \cdot (y - x)$。由于凸性，$F$ 在任意处都位于切线之上或与其重合，因此对所有 x, y 都有 $D_F(y, x) \geq 0$。

在实际中，Bregman 散度大多用于有限维情形。此时，$F : \Omega \to \mathbb{R}$ 是一个凸可微函数，且 $\Omega \subset \mathbb{R}^n$ 是一个凸集。于是，与 F 相关的 **Bregman 散度**定义为

$$D_F(y, x) := F(y) - F(x) - \nabla F(x) \cdot (y - x),$$

对任意 $x, y \in \Omega$ 成立。有限维情形下 Bregman 散度 $D_F(y, x)$ 的示意图见图 A.1。

当 F 不可微时，Bregman 散度仍可定义，此时 $\nabla F(x)$ 可以由 F 在 x 处某个指定的次梯度代替。不过该次梯度需要在定义中明确说明。这里我们不考虑这一推广。

Bregman 散度的一个简单例子是 $F(x) = \frac{1}{2}|x|^2$，其中 $x \in \mathbb{R}^n$。此时可验证

$$D_F(y, x) = \frac{1}{2}|y - x|^2.$$

容易验证，与 F 相关的 Bregman 散度具有如下性质，其中 F 可以是泛函或函数。以下统一使用 F 为函数时的记号。

1. 非负性：对所有 x, y，都有 $D_F(y, x) \geq 0$，因为 F 是凸的。若 F 严格凸，则当且仅当 $y = x$ 时，$D_F(y, x) = 0$。

2. 关于第一个变量的凸性：若 F 是（严格）凸的，则对任意 x，$D_F(\cdot, x)$ 也是（严格）凸的。

3. 关于第一个变量的凸性：对任意 x, y, z 和 $\lambda \in [0, 1]$，有

$$D_F((1 - \lambda)y + \lambda z, x) \leq (1 - \lambda)D_F(y, x) + \lambda D_F(z, x).$$

然而，Bregman 散度不是标准意义下的距离：它不具有对称性，也不满足三角不等式。

定义 A.3.4 (Kullback–Leibler 散度)**.** 两个概率分布 p 和 q 之间的 **Kullback–Leibler (KL) 散度** 定义为与熵泛函相关的 Bregman 散度，其中

$$F(\rho) := -\int \rho(x)\log\rho(x)dx$$

对任意 $\rho \in P := \{\rho : \mathbb{R}^n \to \mathbb{R} : \rho \geq 0, \int \rho = 1\}$ 成立。更具体地，

$$\mathrm{KL}(p,q) := D_F(p,q) = \int p(x)\log\frac{p(x)}{q(x)}\,dx$$

若 F 是定义在 $\mathcal{X}$ 上离散随机变量的熵，则

$$\mathrm{KL}(p,q) := D_F(p,q) = \sum_{x\in\mathcal{X}} p(x)\log\frac{p(x)}{q(x)}$$

对 $\mathcal{X}$ 上任意概率分布 p 和 q 成立。

由于 KL 散度是 Bregman 散度的一个特例，因此它具有后者的全部性质。特别地，我们应始终注意到 KL 散度是非负的，并且当且仅当处处有 $p = q$ 时，$\mathrm{KL}(p,q) = 0$（因为当 F 取为 p 的熵时，F 是严格凸的）。此外，KL 散度**不是**距离，因为它不对称，即便 $\mathrm{KL}(p,q)$ 和 $\mathrm{KL}(q,p)$ 都存在，它们一般也不相等；并且 KL 散度不满足三角不等式。接下来，我们进一步考察 KL 散度及其更多性质，因为在深度学习研究与应用中，它由于便于衡量两个概率分布之间的差异而被广泛使用。

假设 p 和 q 是定义在 $\mathbb{R}^n$ 上的两个概率分布。由于对数项的存在，它们之间的 KL 散度并不总是存在，尤其是在 p 和 q 的支撑集不相同时。

如果对某个 $x \in \mathbb{R}^n$ 有 $p(x) > 0$ 且 $q(x) = 0$，则

$$p(x)\log\frac{p(x)}{q(x)} = +\infty.$$

如果所有这类点组成的集合在 $\mathbb{R}^n$ 中具有正测度，则 $\mathrm{KL}(p,q)$ 无定义，因为按其定义相应积分不存在。在这种情况下，我们不能用 KL 散度来衡量 p 与 q 的差异，也无法对 $\mathrm{KL}(p,q)$ 进行诸如求梯度之类的操作。这发生在 $p(x) > 0$ 出现在 q 的支撑之外的情形。

如果对任意 $x \in \mathbb{R}^n$ 有 $p(x) = 0$ 且 $q(x) > 0$，则不会出现上述求值问题，

因为

$$p(x)\log\frac{p(x)}{q(x)}=0.$$

在这种情况下，KL 散度是有定义的，因为 KL 中的积分是有限的。

总而言之，$\mathrm{KL}(p,q)$ 的使用仅限于 p 的支撑包含于 q 的支撑这一情形。否则，我们可能需要使用 $\mathrm{KL}(q,p)$ 或其他距离、散度来衡量 p 与 q 的差异。

当我们在深度学习模型中使用 KL 散度时，尤其要对上述问题保持谨慎。更具体地，回忆第 五 章中的若干生成模型，我们使用 KL 散度来衡量给定数据分布 p_{data}（通常表现为数据集 $\mathcal{D}=\{x_i\in\mathbb{R}^d : i\in[M]\}$，其中 x_i 是独立地从 p_{data} 中抽取的样本）与生成分布或推前分布 p_θ 之间的差异，其中 θ 表示某个待训练深度神经网络的参数，p_θ 是由该网络诱导出的概率分布。在这种情况下，我们常常希望通过求解

$$\min_\theta\ \Big\{\mathrm{KL}(p_{\mathrm{data}},p_\theta)=-H(p_{\mathrm{data}})-\int p_{\mathrm{data}}(x)\log p_\theta(x)\,dx\Big\}.$$

来找到最优参数 θ。由于 p_{data} 是未知且固定的，$H(p_{\mathrm{data}})$ 与 θ 无关，因此上述问题可化为

$$\max_\theta\ \int p_{\mathrm{data}}(x)\log p_\theta(x)\,dx \quad\approx\quad \max_\theta\ \frac{1}{M}\sum_{i=1}^{M}\log p_\theta(x_i),$$

其中近似来自 Monte Carlo 积分，并利用了 $x_i\sim p_{\mathrm{data}}$。我们可以看出，这要求对所有 $x_i\in\mathcal{D}$，都有 $p_\theta(x_i)>0$。另一方面，由于使用神经网络，p_θ 往往具有某种连续性或光滑性。因此，在 p_{data} 的支撑之外的点 x 处，$p_\theta(x)>0$ 是很可能发生的。这虽然不会导致计算问题，但却可能推断或生成不属于 p_{data} 的样本。这可能导致对 θ 的估计不准确，甚至得到目标问题的错误解。在这种情况下，可能需要对方法作适当修正，或采用其他替代方法。

当所涉及的概率分布具有某些特殊形式时，KL 散度可以安全使用，并且常常有具体的表达式。下面给出两个例子。

例 A.3.5 (两个高斯分布之间的 KL 散度)**.** 设 p_1 和 p_2 是定义在 $\mathbb{R}^n$ 上的两个高斯概率分布。具体而言，

$$p_1=\mathcal{N}(\mu_1,\Sigma_1),\qquad p_2=\mathcal{N}(\mu_2,\Sigma_2),$$

其中 $\mu_1,\mu_2\in\mathbb{R}^n$，且 $\Sigma_1,\Sigma_2\in\mathbb{R}^{n\times n}$ 是对称正定矩阵。从 p_1 到 p_2 的 KL 散

度定义为

$$\mathrm{KL}(p_1, p_2) = \int p_1(x) \log \frac{p_1(x)}{p_2(x)}\, dx = \mathbb{E}_{X\sim p_1}\Big[\log \frac{p_1(X)}{p_2(X)}\Big]. \tag{A.12}$$

注意，高斯概率密度 $\mathcal{N}(x;\mu,\Sigma)$ 的对数为

$$\log \mathcal{N}(x;\mu,\Sigma) = -\frac{n}{2}\log(2\pi) - \frac{1}{2}\log\det(\Sigma) - \frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu).$$

将 p_1 与 p_2 的表达式代入 (A.12)，可得

$$\begin{aligned}\log \frac{p_1(X)}{p_2(X)} = \frac{1}{2}\log\frac{\det(\Sigma_2)}{\det(\Sigma_1)} \\ - \frac{1}{2}\Big[(X-\mu_1)^\top \Sigma_1^{-1}(X-\mu_1) - (X-\mu_2)^\top \Sigma_2^{-1}(X-\mu_2)\Big].\end{aligned} \tag{A.13}$$

对 (A.13) 两边关于 $X \sim p_1 = \mathcal{N}(\mu_1, \Sigma_1)$ 取期望，我们分别计算上式中的二次项。具体而言，

$$\begin{aligned}\mathbb{E}_{X\sim p_1}\big[(X-\mu_1)^\top \Sigma_1^{-1}(X-\mu_1)\big] &= \mathbb{E}_{X\sim p_1}\Big[\mathrm{tr}\Big((X-\mu_1)^\top \Sigma_1^{-1}(X-\mu_1)\Big)\Big] \\ &= \mathbb{E}_{X\sim p_1}\Big[\mathrm{tr}\Big(\Sigma_1^{-1}(X-\mu_1)(X-\mu_1)^\top\Big)\Big] \\ &= \mathrm{tr}\Big(\Sigma_1^{-1}\mathbb{E}_{X\sim p_1}\Big[(X-\mu_1)(X-\mu_1)^\top\Big]\Big) \\ &= \mathrm{tr}(\Sigma_1^{-1}\Sigma_1) \\ &= n.\end{aligned} \tag{A.14}$$

此外，有

$$X - \mu_2 = (X-\mu_1) + (\mu_1 - \mu_2),$$

因此

$$\mathbb{E}_{X\sim p_1}\big[(X-\mu_2)^\top \Sigma_2^{-1}(X-\mu_2)\big] = \mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (\mu_1-\mu_2)^\top \Sigma_2^{-1}(\mu_1-\mu_2), \tag{A.15}$$

其中交叉项由于 $\mathbb{E}_{X\sim p_1}[X-\mu_1] = 0$ 而消失。

结合 (A.12)、(A.13)、(A.14) 和 (A.15)，得到 KL 散度的闭式表达

$$\mathrm{KL}(p_1,p_2) = \frac{1}{2}\Big[\log\frac{\det(\Sigma_2)}{\det(\Sigma_1)} - n + \mathrm{tr}(\Sigma_2^{-1}\Sigma_1) + (\mu_2-\mu_1)^\top \Sigma_2^{-1}(\mu_2-\mu_1)\Big]. \tag{A.16}$$

如果 $\Sigma_1 = \Sigma_2 = \Sigma$，则 (A.16) 退化为

$$\mathrm{KL}(p_1, p_2) = \frac{1}{2}(\mu_2 - \mu_1)^\top \Sigma^{-1}(\mu_2 - \mu_1).$$

特别地，当且仅当 $\mu_1 = \mu_2$ 且 $\Sigma_1 = \Sigma_2$ 时，$\mathrm{KL}(p_1, p_2) = 0$。

例 A.3.6 (两个 Poisson 分布之间的 KL 散度)**.** 设 $p_1 = \mathrm{Poisson}(\lambda_1)$ 且 $p_2 = \mathrm{Poisson}(\lambda_2)$，其中 $\lambda_1, \lambda_2 > 0$。从 p_1 到 p_2 的 KL 散度定义为

$$\mathrm{KL}(p_1, p_2) = \sum_{x=0}^{\infty} p_1(x) \log \frac{p_1(x)}{p_2(x)}. \tag{A.17}$$

回忆参数为 λ 的 Poisson 分布的概率质量函数为

$$\Pr(X = x) = \frac{\lambda^x e^{-\lambda}}{x!}, \quad \text{其中} x = 0, 1, 2, \ldots.$$

将其代入 (A.17)，得到

$$\mathrm{KL}(p_1, p_2) = \sum_{x=0}^{\infty} p_1(x) \log \frac{p_1(x)}{p_2(x)} = \sum_{x=0}^{\infty} \frac{\lambda^x e^{-\lambda}}{x!} \Big(x \log \frac{\lambda_1}{\lambda_2} + \lambda_2 - \lambda_1 \Big).$$

注意到对于 $X \sim p_1$，有 $\mathbb{E}[X] = \sum_{x=0}^{\infty} x \cdot \frac{\lambda^x e^{-\lambda}}{x!} = \lambda_1$，从而得到闭式表达

$$\mathrm{KL}(p_1, p_2) = \lambda_1 \log \frac{\lambda_1}{\lambda_2} + \lambda_2 - \lambda_1.$$

容易验证，当且仅当 $\lambda_1 = \lambda_2$ 时，$\mathrm{KL}(p_1, p_2) = 0$。

KL 散度有若干变体，它们具有对称性，并且在 p 和 q 不共享相同支撑时仍然有定义。其中最著名的是 Jensen–Shannon 散度，其定义如下。

定义 A.3.7 (Jensen–Shannon 散度)**.** 两个概率分布 p 和 q 之间的 **Jensen–Shannon (JS) 散度** 定义为

$$\mathrm{JS}(p, q) := \frac{1}{2}\Big(\mathrm{KL}(p, r) + \mathrm{KL}(q, r) \Big) \tag{A.18}$$

其中 $r = (p + q)/2$ 是 p 与 q 的逐点平均。

注意到 r 本身也是一个概率分布，且其支撑包含了 p 和 q 的支撑。因此，$\mathrm{JS}(p, q)$ 总是良定义且取有限值。事实上，可以验证对任意 p 和 q，都有 $\mathrm{JS}(p, q) \in [0, \log 2]$。当 p 和 q 的支撑不相交时，取得最大值 $2 \log 2$。不过，需

要注意的是，这并不一定是优点，因为此时 $\mathrm{JS}(p,q)$ 关于 p 或 q 的广义梯度变为零，从而在实际中无法使用基于梯度的优化算法来更新它们（更准确地说，是无法更新生成它们或将其推前的神经网络参数；例如 p 可能是由某生成网络得到的概率分布，或者由某神经网络逼近的推前 push-forward 映射得到的）。

尽管如此，容易验证 $\mathrm{JS}(p,q) \geq 0$，且当且仅当 $p = q$ 时，$\mathrm{JS}(p,q) = 0$。此外，不同于 KL 散度，JS 散度是对称的，即对定义在 $\mathbb{R}^n$ 上的任意概率分布 p 和 q，都有 $\mathrm{JS}(p,q) = \mathrm{JS}(q,p)$。

尽管 JS 散度仅是 KL 散度的一个简单变形，并带来少量新性质，但它在使用上更困难，因为对许多典型概率分布而言，它没有具体的表达式。例如，两个高斯分布之间的 JS 散度不像 (A.16) 中的 KL 散度那样有具体的表达式，因为 (A.18) 中的 r 变成了两个高斯分布的混合分布，而从 p（或 q）到 r 的 KL 散度没有具体的表达式。

另外存在一种距离，它可以以更优美的数学方式度量两个概率分布之间的差异，并克服上述 KL 与 JS 散度的问题。这就是下面要定义的 Wasserstein 距离。

定义 A.3.8 (Wasserstein 距离)**.** 设

$$P := \{p : \mathbb{R}^n \to \mathbb{R} : p(x) \geq 0, \forall\, x \in \mathbb{R}^n, \int p(x)dx = 1\}$$

为定义在 $\mathbb{R}^n$ 上的概率分布空间。对 $s \in [1,\infty), p, q \in P$ 之间的 **Wasserstein-s 距离** 定义为

$$W_s(p,q) := \inf_{\pi \in \Pi(p,q)} \Big(\int |x-y|^s \pi(x,y)\, dx \Big)^{1/s}, \tag{A.19}$$

其中 $\Pi(p,q)$ 表示定义在 $\mathbb{R}^n \times \mathbb{R}^n$ 上、以 p 和 q 为边缘分布的联合概率分布全体，即

$$\Pi(p,q) := \Big\{ \pi : \mathbb{R}^d \times \mathbb{R}^d \to [0,\infty) : \begin{array}{l} p(x) = \int \pi(x,y)\, dy, \\ q(y) = \int \pi(x,y)\, dx, \end{array} \ \forall\, x, y \in \mathbb{R}^d \Big\}.$$

当 $s = \infty$ 时，$p, q \in P$ 之间的 Wasserstein-∞ 距离定义为

$$W_\infty(p,q) := \lim_{s \to \infty} W_s(p,q).$$

(A.19) 的最优解 π^* 称为从 p 到 q 的一个 **最优传输**。

作为一个例子，定义在 $\mathbb{R}^d$ 上的两个高斯分布 $p_1 = \mathcal{N}(\mu_1, \Sigma_1)$ 和 $p_2 = \mathcal{N}(\mu_2, \Sigma_2)$ 之间的 Wasserstein-2 距离可表示为

$$W_2(p_1, p_2) = \Big(|\mu_1 - \mu_2|^2 + \mathrm{tr}\Big(\Sigma_1 + \Sigma_2 - 2(\Sigma_1^{1/2}\Sigma_2\Sigma_1^{1/2})^{1/2}\Big)\Big)^{1/2}.$$

Wasserstein 距离具有许多重要的数学性质。其中一个基本性质是，W_s 在严格数学意义下确实是一个距离（见定义 A.2.3）。即，W_s 满足如下性质：

- 正定性：对所有 $p, q \in P$，有 $W_s(p, q) \geq 0$；并且当且仅当 $p = q$ 在 $\mathbb{R}^n$ 上几乎处处成立时，$W_s(p, q) = 0$。
- 对称性：对任意 $p, q \in P$，有 $W_s(p, q) = W_s(q, p)$。
- 三角不等式：对任意 $p, q, r \in P$，有 $W_s(p, q) \leq W_s(p, r) + W_s(r, q)$。

此外，即使 p 和 q 的支撑不同，$W_s(p, q)$ 仍然是良定义的。这相比 KL 散度是一个显著优势。进一步地，Wasserstein 距离具有许多有用的物理解释。例如，Wasserstein-1 距离也称为 **Earth mover distance, EMD**，它衡量将概率质量 p 运输为 q 的最优代价，其中将单位质量从位置 x 移动到位置 y 的代价与距离 $|x - y|$ 成正比。

Wasserstein 距离是最优传输领域中的一个基本概念，而最优传输理论在过去几十年中得到了广泛研究。最优传输理论与数学的许多分支相关联，包括几何分析、偏微分方程、概率论和最优控制。许多重要结果可见 [3,125,151]。

然而，Wasserstein 距离的主要问题在于它们在数值上难以计算。这是由于其定义中包含约束条件，以及其在实际中常用于高维空间。在简单情形下，当 p 和 q 是定义在一个较小有限集 $\mathcal{X}$ 上的概率分布时，它们实际上是归一化直方图，而 (A.19) 退化为一个线性规划问题，可以由现有优化算法求解。通过在 (A.19) 的目标函数中加入联合概率分布的熵正则项，可以显著加速计算，并且可通过求解该问题的对偶形式、使用称为 Sinkhorn 算法得到解。然而，这一技术并不适用于高维问题或定义在诸如 $\mathbb{R}^n$ 这类连续空间上的问题，原因是计算量过大。近年来，也发展出了一些基于深度学习的方法来计算 Wasserstein 距离。例如，可以将问题 (A.19) 转化为其对偶形式，然后将对偶变量参数化为深度神经网络，并通过训练来求解对偶问题，从而得到 (A.19) 的解。相关结果可见 [108]。但是，到目前为止，针对一般深度学习模型，仍缺乏一种在计算上高效且能准确逼近 Wasserstein 距离的方法。这阻碍了 Wasserstein 距离在深度学习中的广泛应用。

本节最后要介绍的概念是一个经典概念，称为互信息。与上面的各种散度

和距离不同，互信息利用随机变量之间的联合概率分布来衡量它们的依赖关系。互信息的正式定义如下。

定义 A.3.9 (互信息)**.** 设 X 和 Y 是两个具有联合概率分布 $p(x,y)$ 的随机变量。X 与 Y 之间的 **互信息**（mutual information）定义为

$$\mathrm{MI}(X,Y) = \mathrm{KL}(p, p_X p_Y) = \iint p(x,y)\frac{p(x,y)}{p_X(x)p_Y(y)}\,dxdy, \tag{A.20}$$

其中 p_X 和 p_Y 分别是 X 与 Y 的边缘概率分布。

注意，互信息是非负的，因为它被定义为从联合概率分布 p 到边缘概率分布乘积 $p_X p_Y$ 的 KL 散度。此外，我们可以看出，当且仅当 $p = p_X p_Y$ 时，$\mathrm{MI}(X,Y) = 0$，即 X 与 Y 相互独立。互信息也是对称的。不过，我们强调互信息在任何意义下都不是距离。尽管如此，较大的 $\mathrm{MI}(X,Y)$ 表明 X 与 Y 存在较强的依赖关系。事实上，两个随机变量的互信息与其（条件）熵 $H(X|Y)$ 之间有如下关系：

$$\begin{aligned}
\mathrm{MI}(X,Y) &= \int p(x,y)\log\frac{p(x,y)}{p_X(x)p_Y(y)}\,dxdy \\
&= \int p(x,y)\log\frac{p_{X|Y}(x|y)}{p_X(x)}\,dxdy \\
&= \int p(x,y)\log p_{X|Y}(x|y)\,dxdy - \int p_X(x)\log p_X(x)\,dx \\
&= H(X) - H(X|Y).
\end{aligned}$$

这再次说明，当 X 与 Y 独立时，$\mathrm{MI}(X,Y) = 0$，此时 $H(X|Y) = H(X)$。另一个极端情形是 X 完全由 Y 决定（即 X 是 Y 的函数），此时 $H(X|Y) = 0$ 且 $\mathrm{MI}(X,Y) = H(X)$。

A.4 随机微分方程基础

随机微分方程（stochastic differential equation，SDE）在现代深度学习研究与应用中的问题建模、模型构造和推导中变得愈发常见。例如，在第 5.3 节的扩散模型中，我们已经看到了 SDE 的大量应用。

在本节中，我们将简要回顾 SDE 的一些基础知识。具体而言，我们将介绍 Wiener 过程、随机积分与随机微分、Itô 公式以及 Fokker–Planck 方程的定义。

考虑一个粒子（作为随机游走者）在实直线 $\mathbb{R}$ 上运动。在时刻 $t=0$，它位于原点。此后，它在每个单位时间 Δt 内以步长 Δx 运动。也就是说，在每个 Δt 内，它以概率 $\frac{1}{2}$ 向左移动（位置变化为 $-\Delta x$），以概率 $\frac{1}{2}$ 向右移动（位置变化为 Δx）。

设 $X(t)$ 表示粒子在经过 n 步、时刻 $t=n\Delta t$ 时的位置。为了研究 $X(t)$ 的概率分布，我们考虑独立随机变量 $X_i \sim \mathrm{Bernoulli}(\frac{1}{2})$，其中 $i=1,2,\ldots$，因此 $X_i=1$ 或 0 的概率均为 $\frac{1}{2}$。显然，$\mathbb{E}[X_i]=\frac{1}{2}$ 且 $\mathrm{Var}[X_i]=\frac{1}{4}$。

令 $S_n := \sum_{i=1}^n X_i$，则 S_n 表示粒子在前 n 步中向右移动的次数，而 $n-S_n$ 表示向左移动的次数。于是有 $\mathbb{E}[S_n]=\frac{n}{2}$ 且 $\mathrm{Var}[S_n]=\frac{n}{4}$。进一步，

$$X(t)=S_n\Delta x-(n-S_n)\Delta x=(2S_n-n)\Delta x.$$

因此，$\mathbb{E}[X(t)]=0$，并且

$$\mathrm{Var}[X(t)]=(\Delta x)^2\,\mathrm{Var}[2S_n-n]=(\Delta x)^2 4\,\mathrm{Var}[S_n]=(\Delta x)^2 n=\frac{(\Delta x)^2}{\Delta t}t.$$

不失一般性，设 $\frac{(\Delta x)^2}{\Delta t}=1$，于是容易验证

$$X(t)=(2S_n-n)\Delta x=\frac{S_n-\frac{n}{2}}{\sqrt{\frac{n}{4}}}\sqrt{t}.$$

现在，对任意固定的 t，令 $n\to\infty$ 且保持 $\frac{(\Delta x)^2}{\Delta t}=1$，由 Laplace–De Moivre 定理可得

$$\begin{aligned}\lim_{n\to\infty}\Pr(a\le X(t)\le b)&=\lim_{n\to\infty}\Pr\Big(\frac{a}{\sqrt{t}}\le\frac{S_n-\frac{n}{2}}{\sqrt{\frac{n}{4}}}\le\frac{b}{\sqrt{t}}\Big)\\&=\frac{1}{\sqrt{2\pi}}\int_{a/\sqrt{t}}^{b/\sqrt{t}}e^{-\frac{x^2}{2}}\,dx\\&=\frac{1}{\sqrt{2\pi t}}\int_a^b e^{-\frac{x^2}{2t}}\,dx,\end{aligned}$$

这说明 $X(t)\sim\mathcal{N}(0,t)$。换言之，如果粒子按连续时间的意义运动，则其位置满足 $X(t)\sim\mathcal{N}(0,t)$。这导出了 **Wiener 过程**（也常称为 **Brownian motion**）的定义如下。

定义 A.4.1 (Wiener 过程)**.** 若实值随机过程 $W:[0,\infty)\to\mathbb{R}$ 满足：

1. 从原点出发：$W(0)=0$ 几乎处处成立；

2. 增量独立：对任意 $0 < t_1 < t_2 < \cdots < t_k$，随机变量

$$W(t_1),\ W(t_2) - W(t_1),\ \ldots,\ W(t_k) - W(t_{k-1})$$

相互独立；

3. 增量服从正态分布：对任意 $t > s \geq 0$，有 $W(t) - W(s) \sim \mathcal{N}(0, t-s)$；以及

4. 路径连续：映射 $t \mapsto W(t)$ 关于 t 连续。

则称 W 为一个 **Wiener 过程**。

注意，由于 $W(t) \sim \mathcal{N}(0,t)$，对任意 t 都有 $\mathbb{E}[W(t)] = 0$ 且 $\mathrm{Var}[W(t)] = \mathbb{E}[W(t)^2] = t$。

为了严格定义 SDE，我们需要一系列来自测度论的数学概念。下面我们只作简要介绍，更详细内容可参见 [38,93,106]。

设 (Ω, Γ, μ) 是一个概率空间，其中此处取 $\Omega = \mathbb{R}^d$，Γ 是 Ω 上的一个 σ-代数，而 μ 是一个概率测度，使得 Γ 中所有元素在 μ 下都是可测的。这里，Ω 的一个 σ-代数被定义为 Ω 的若干子集所构成的集合，它包含空集 $\emptyset$，并且对补集和可数并运算封闭。若 A 是 Ω 的某些子集组成的集合，而 $\Gamma(A)$ 是所有包含 A 的 σ-代数的交，则称 $\Gamma(A)$ 为由 A 生成的 σ-代数。可以证明，$\Gamma(A)$ 确实也是一个 σ-代数。由于随机变量可以视作 μ-可测函数，因此我们也可以将 $\Gamma(\{X_\alpha : \alpha \in \mathcal{A}\})$ 定义为由随机变量族 $\{X_\alpha : \alpha \in \mathcal{A}\}$ 在 Ω 上生成的 σ-代数，其中 $\mathcal{A}$ 是一个可以是无限的指标集，而这里的 X_α 指的是 $\mathbb{R}$ 中 Lebesgue 可测集合在映射 X_α 下所有原像组成的集合。

我们用 $\mathcal{W}(t) := \Gamma(\{W(s) : 0 \leq s \leq t\})$ 表示由 Wiener 过程截至时刻 t 生成的 σ-代数。我们也称 $\mathcal{W}(t)$ 为 Wiener 过程 W 在时刻 t 之前的历史。我们再用生成的 σ-代数 $\mathcal{W}^+(t) := \Gamma(\{W(s) - W(t) : s \geq t\})$ 表示 Wiener 过程 W 在时刻 t 之后的未来。若一族 σ-代数 $\{\mathcal{F}(t) \subset \Gamma : t \geq 0\}$ 相对于 W 满足：(i) 对所有 $0 \leq s < t$，有 $\mathcal{F}(s) \subset \mathcal{F}(t)$；(ii) 对所有 $t \geq 0$，有 $\mathcal{W}(t) \subset \mathcal{F}(t)$；且 (iii) 对所有 $t \geq 0$，$\mathcal{F}(t)$ 与 $\mathcal{W}^+(t)$ 相互独立，则称其相对于 W 是非预见的（也称适应的）。我们也称 $\mathcal{F}(t)$ 是截至时刻 t 的一个滤子（filter）。我们可以非正式地将 $\mathcal{F}(t)$ 理解为“包含截至时刻 t 的所有信息”的 σ-代数，也就是说，对任意 t，所有由 Wiener 过程截至时刻 t 所生成的随机变量，以及一些与 Wiener 过程独立的初始随机变量，都在 $\mathcal{F}(t)$ 下可测。

若实值随机过程 G 对任意 $t \geq 0$ 都满足 $G(t)$ 关于 $\mathcal{F}(t)$ 可测，则称 G 相对于 $\mathcal{F}$ 是非预见的。此外，用 $L^2(0,T)$ 表示所有满足 $\mathbb{E}[\int_0^T |G(t)|^2 dt] < \infty$

的实值非预见随机过程 G 所组成的空间。类似地，用 $L^1(0,T)$ 表示所有满足 $\mathbb{E}[\int_0^T |F(t)|dt] < \infty$ 的实值非预见随机过程 F 所组成的空间。$L^2(0,T)$ 中一类特殊的随机过程称为阶梯过程：设 $0 = t_0 < t_1 < \cdots < t_K = T$ 是区间 $[0,T]$ 的一个划分，且对 $t = 0,\ldots,K-1$，在每个区间 $[t_k, t_{k+1})$ 上有 $G(t) = G_k$，其中 G_k 是一个 $\mathcal{F}(t_k)$-可测随机变量，则称 G 为一个阶梯过程。对于这样一个属于 $L^2(0,T)$ 的阶梯过程 G，我们将其在 $(0,T)$ 上的 Itô 随机积分定义为

$$\int_0^T G(t)\,dW(t) := \sum_{k=0}^{K-1} G_k(W(t_{k+1}) - W(t_k)),$$

它是一个随机变量。可以证明，对任意 $G \in L^2(0,T)$，存在一列阶梯过程 $G_n \in L^2(0,T)$，使得

$$\mathbb{E}\Big[\int_0^T |G - G_n|^2\,dt\Big] \to 0$$

当 $n \to \infty$ 时成立。这里各 G_n 可能对应于 $[0,T]$ 上不同的划分，并且在不同子区间上取不同的随机变量。然后，Itô 随机积分的下列性质可以先对阶梯过程证明，再推广到 $L^2(0,T)$ 中所有随机过程：对任意 $a,b \in \mathbb{R}$ 和 $G,H \in L^2(0,T)$，

$$\int_0^T (aG(t) + bH(t))\,dW(t) = a\int_0^T G(t)\,dW(t) + b\int_0^T H(t)\,dW(t),$$

$$\mathbb{E}\Big[\int_0^T G(t)\,dW(t)\Big] = 0,$$

$$\mathbb{E}\Big[\int_0^T G(t)\,dW(t)\cdot\int_0^T H(t)\,dW(t)\Big] = \int_0^T G(t)\cdot H(t)\,dt.$$

基于上述 Itô 随机积分的定义，我们可以如下定义实值随机过程 X：

$$X(t) = X(0) + \int_0^t f(s,X(s))\,ds + \int_0^t g(s,X(s))\,dW(s)$$

对任意 $t \in [0,T]$，其中 $f,g : [0,T]\times\mathbb{R} \to \mathbb{R}$ 满足 $f(\cdot,X(\cdot)) \in L^1(0,T)$ 且 $g(\cdot,X(\cdot)) \in L^2(0,T)$。在这种情况下，我们也称 X 具有随机微分

$$dX(t) = f(t,X(t))\,dt + g(t,X(t))\,dW(t) \tag{A.21}$$

对所有 $t \geq 0$ 成立，初值为 $X(0)$。我们也称 X 服从 SDE (A.21)。SDE 可以推广到多维情形，其中 $X(t) \in \mathbb{R}^d$，$W(t) = (W_1(t),\ldots,W_m(t))^\top \in \mathbb{R}^m$，这里每个 $W_i(t)$ 都是一维 Wiener 过程，且 W 中这 m 个 Wiener 过程彼此

独立（我们称 W 为标准 m 维 Wiener 过程），同时 $f:[0,T]\times\mathbb{R}^d\to\mathbb{R}^d$，$g:[0,T]\times\mathbb{R}^d\to\mathbb{R}^{d\times m}$，并且仍满足相应的可积性条件。我们通常称 f 为漂移项，g 为扩散系数。

随机分析中最重要的性质之一称为 Itô 公式（也称 Itô 链式法则），其表述如下。证明可见例如 [38,106]。

引理 A.4.2 (Itô 公式)**.** *设 X 满足 SDE (A.21)，且 $u:[0,T]\times\mathbb{R}\to\mathbb{R}$ 连续（并且 $\partial_t u$、$\partial_x u$ 和 $\partial^2_{xx}u$ 都连续）。令 $Y(t):=u(t,X(t))$，则 Y 满足*

$$dY(t)=\Big(\partial_t u+\partial_x u\cdot f+\frac{1}{2}g^2\partial^2_{xx}u\Big)dt+(\partial_x u\cdot g)dW(t) \tag{A.22}$$

其中，为简化记号，下文统一记 $u=u(t,X(t))$、$f=f(t,X(t))$ 和 $g=g(t,X(t))$。

多维版本的公式 (A.22) 为

$$dY(t)=\Big(\partial_t u+\partial_x u\cdot f+\frac{1}{2}\langle\nabla^2u,gg^\top\rangle\Big)dt+\partial_x u\cdot g\cdot dW(t), \tag{A.23}$$

其中 $X(t)\in\mathbb{R}^d$，$W(t)\in\mathbb{R}^m$ 是标准 m 维 Wiener 过程，$f:[0,T]\times\mathbb{R}^d\to\mathbb{R}^d$，$g:[0,T]\times\mathbb{R}^d\to\mathbb{R}^{d\times m}$，$u:[0,T]\times\mathbb{R}^d\to\mathbb{R}$，$\nabla^2u$ 表示 u 的 Hessian 矩阵，且对矩阵 $A=[A_{ij}]_{i,j=1}^d$ 和 $B=[B_{ij}]_{i,j=1}^d$，定义 $\langle A,B\rangle:=\mathrm{tr}(A^\top B)=\sum_{i,j=1}^d A_{ij}B_{ij}$。

下面通过一个例子说明如何利用 Itô 公式求随机积分。不过需要注意的是，一般情况下随机积分通常难以解析求值。

例 A.4.3. 求

$$\int_0^T W(t)\,dW(t).$$

设 $X(t)=W(t)\in\mathbb{R}$，并令 $u(x):=x^2$。则 $\nabla u(x)=2x$，$\nabla^2u(x)=2$。由 Itô 公式，有

$$\begin{aligned}d(W(t)^2)&=d\,u(X(t))\\&=\Big(\nabla u(X(t))\cdot 0+\frac{1}{2}\nabla^2u(X(t))\cdot 1\Big)\,dt+\nabla u(X(t))\cdot 1\,dW(t)\\&=dt+2W(t)\,dW(t).\end{aligned}$$

对两边在 $(0,T)$ 上积分，得到

$$\int_0^T d(W(t)^2)=\int_0^T dt+2\int_0^T W(t)\,dW(t). \tag{A.24}$$

注意到 (A.24) 左边为

$$\int_0^T d(W(t)^2) = W(T)^2 - W(0)^2 = W(T)^2,$$

而右边为

$$T + 2\int_0^T W(t)\,dW(t).$$

因此，对 (A.24) 进行整理可得

$$\int_0^T W(t)\,dW(t) = \frac{W(T)^2}{2} - \frac{T}{2}.$$

现在我们从 $X(t)$ 的概率分布角度来考察该过程。设 SDE (A.21) 中的 $X(t)$ 在任意时刻 t 都服从概率分布 $\rho(\cdot,t)$。我们希望找出 $\rho(\cdot,t)$ 的性质。

为此，我们给出一个启发式但不够严格的推导。不过，所得出的 Fokker–Planck 方程结论是正确的。更严格的推导可参见 [118]。

设 $\phi:\mathbb{R}^d\to\mathbb{R}$ 是任意光滑测试函数。假设有 N 个相互独立的粒子 $X_i(t)$，它们在任意时刻 t 都服从分布 $\rho(\cdot,t)$。则通过 Monte Carlo 积分，我们有如下近似

$$\int \rho(x,t)\phi(x)\,dx \approx \frac{1}{N}\sum_{i=1}^N \phi(X_i(t)). \tag{A.25}$$

对两边关于时间求导，则 (A.25) 左边变为

$$\frac{d}{dt}\int \rho(x,t)\phi(x)\,dx = \int \partial_t\rho(x,t)\phi(x)\,dx. \tag{A.26}$$

由 Itô 公式，有

$$\begin{aligned} d\,\phi(X_i(t)) = &\Big(\nabla\phi(X_i(t))f_t(X_i(t)) + \frac{1}{2}\langle\nabla^2\phi(X_i(t)), g_t(X_i(t))g_t(X_i(t))^\top\rangle\Big)dt \\ &+ \nabla\phi(X_i(t))f_t(X_i(t))dW(t), \end{aligned} \tag{A.27}$$

其中为简便起见，我们将 $f(t,\cdot)$ 和 $g(t,\cdot)$ 分别记作 $f_t(\cdot)$ 和 $g_t(\cdot)$。下文也将 $\rho(\cdot,t)$ 记作 $\rho_t(\cdot)$。将 (A.27) 代入 (A.25) 右边的时间导数，并令 $N\to\infty$ 取连续极限，可得到

$$\int \rho_t(x)\Big(\nabla\phi(x)\cdot f_t(x) + \frac{1}{2}\langle\nabla^2\phi(x), g_t(x)g_t(x)^\top\rangle\Big)\,dx.$$

再对其做分部积分，得到

$$\int \Big(-\nabla \cdot (\rho_t(x) f_t(x)) + \frac{1}{2}\langle \nabla^2, g_t(x) g_t(x)^\top \rho(x,t)\rangle \Big) \phi(x)\, dx, \tag{A.28}$$

其中

$$\langle \nabla^2, A(x)\rangle := \sum_{i,j=1}^{d} \partial^2_{x_i x_j} A_{ij}(x)$$

对任意矩阵值函数 $A(x) = [A_{ij}(x)]_{i,j=1}^d$（$x \in \mathbb{R}^d$）成立。现在，由于 (A.26) 与 (A.28) 对任意 ϕ 都相等，从而推出

$$\partial_t \rho_t(x) = -\nabla \cdot (\rho_t(x) f_t(x)) + \frac{1}{2}\langle \nabla^2, g_t(x) g_t(x)^\top \rho_t(x)\rangle. \tag{A.29}$$

我们称 (A.29) 为 ρ 的 **Fokker–Planck 方程**。

注意，在许多人为设计的 SDE 中，例如第 5.3 节中的扩散模型，扩散系数常被设为与状态无关的函数 $g_t = \sigma_t I_d$，其中 $\sigma_t \in \mathbb{R}$ 可以依赖于时间 t，但不依赖于状态 x。此时，Fokker–Planck 方程可简化为

$$\partial_t \rho_t(x) = -\nabla \cdot (\rho_t(x) f_t(x)) + \frac{\sigma_t^2}{2} \Delta \rho_t(x).$$

在没有扩散项的情形，即 $g = 0$ 时，我们得到所谓的 **连续方程**：

$$\partial_t \rho_t(x) = -\nabla \cdot (\rho_t(x) f_t(x)), \tag{A.30}$$

相应的 SDE (A.21) 退化为 ODE

$$\dot{x}(t) = f(t, x(t)).$$

方程 (A.30) 在本书中多次出现，例如在第 3.3 节的 Neural ODE 方法和第 5.5 节的 flow matching 方法中。顺便提一句，连续方程通常是指遵循某种守恒律的一类方程，而 (A.30) 是一个保持概率守恒的特殊例子。

参考文献

[1] J. Adler and S. Lunz. Banach wasserstein gan. *Advances in neural information processing systems*, 31, 2018.

[2] M. S. Albergo and E. Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *International Conference on Learning Representations*, 2023.

[3] L. Ambrosio, E. Brué, D. Semola, et al. *Lectures on optimal transport*, volume 130. Springer, 2021.

[4] W. F. Ames. *Numerical methods for partial differential equations*. Academic press, 2014.

[5] M. Arjovsky, S. Chintala, and L. Bottou. Wasserstein generative adversarial networks. In *International Conference on Machine Learning*, pages 214–223, 2017.

[6] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath. Deep reinforcement learning: A brief survey. *IEEE signal processing magazine*, 34(6):26–38, 2017.

[7] M. Bardi, I. C. Dolcetta, et al. *Optimal control and viscosity solutions of Hamilton-Jacobi-Bellman equations*, volume 12. Springer, 1997.

[8] A. R. Barron. Universal approximation bounds for superpositions of a sigmoidal function. *IEEE Transactions on Information theory*, 39(3):930–945, 2002.

[9] S. Bartels et al. *Numerical methods for nonlinear partial differential equations*, volume 47. Springer, 2015.

[10] M. G. Bellemare, W. Dabney, and R. Munos. A distributional perspective on reinforcement learning. In *International conference on machine learning*, pages 449–458. PMLR, 2017.

[11] A. Ben-Tal and A. Nemirovski. *Lectures on modern convex optimization: analysis, algorithms, and engineering applications*, volume 2. Society for Industrial and Applied Mathematics, 1987.

[12] D. Bertsekas. *Dynamic programming and optimal control: Volume I*, volume 4. Athena scientific, 2012.

[13] D. Bertsekas. *Convex optimization algorithms*. Athena Scientific, 2015.

[14] D. Bertsekas. *Nonlinear programming*. Athena Scientific, 2016.

[15] D. Bertsekas. *Reinforcement learning and optimal control*, volume 1. Athena Scientific, 2019.

[16] D. P. Bertsekas, A. Nedić, and A. E. Ozdaglar. *Convex analysis and optimization*. Athena Scientific Belmont, 2003.

[17] D. P. Bertsekas and J. N. Tsitsiklis. Neuro-dynamic programming: an overview. In *Proceedings of 1995 34th IEEE conference on decision and control*, volume 1, pages 560–564. IEEE, 1995.

[18] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π: A vision-language-action flow model for general robot control. *CoRR*, 2024.

[19] J. M. Borwein and A. S. Lewis. *Convex analysis and nonlinear optimization: theory and examples*, volume 3. Springer, 2006.

[20] L. Bottou, F. E. Curtis, and J. Nocedal. Optimization methods for large-scale machine learning. *SIAM review*, 60(2):223–311, 2018.

[21] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[22] A. Bradley and P. Nakkiran. Classifier-free guidance is a predictor-corrector. In *NeurIPS 2024 Workshop on Mathematics of Modern Machine Learning*, 2024.

[23] A. E. Bryson. *Applied optimal control: optimization, estimation and control*. Routledge, 2018.

[24] R. L. Burden and J. D. Faires. *Numerical Analysis 9th edition*. Boston, MA: Brooks/Cole, 2011.

[25] A. Campbell, J. Yim, R. Barzilay, T. Rainforth, and T. Jaakkola. Generative flows on discrete state-spaces: Enabling multimodal flows with applications to protein co-design. In *International Conference on Machine Learning*, pages 5453–5512. PMLR, 2024.

[26] R. T. Chen and Y. Lipman. Flow matching on general geometries. In *International Conference on Learning Representations*, 2024.

[27] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud. Neural ordinary differential equations. *Advances in neural information processing systems*, 31, 2018.

[28] M. Chidambaram, K. Gatmiry, S. Chen, H. Lee, and J. Lu. What does guidance do? a fine-grained analysis in a simple setting. *Advances in Neural Information Processing Systems*, 37:84968–85005, 2024.

[29] E. K. Chong and S. H. Zak. *An introduction to optimization*, volume 76. John Wiley & Sons, 2013.

[30] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of control, signals and systems*, 2(4):303–314, 1989.

[31] W. Dabney, M. Rowland, M. Bellemare, and R. Munos. Distributional reinforcement learning with quantile regression. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[32] P. Dhariwal and A. Nichol. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34:8780–8794, 2021.

[33] J. Duchi, E. Hazan, and Y. Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(Jul):2121–2159, 2011.

[34] W. E and B. Yu. The deep ritz method: a deep learning-based numerical algorithm for solving variational problems. *Communications in Mathematics and Statistics*, 6(1):1–12, 2018.

[35] S. Ellacott. Aspects of the numerical analysis of neural networks. *Acta numerica*, 3:145–202, 1994.

[36] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024.

[37] L. C. Evans. *Partial differential equations*, volume 19 of *Graduate Studies in Mathematics*. American Mathematical Society, Providence, RI, 1998.

[38] L. C. Evans. *An introduction to stochastic differential equations*, volume 82. American Mathematical Soc., 2012.

[39] L. C. Evans. *An introduction to mathematical optimal control theory spring, 2024 version*. University of California, Berkeley, 2024.

[40] F. Farnia and A. Ozdaglar. Do gans always have nash equilibria? In *International Conference on Machine Learning*, pages 3029–3039. PMLR, 2020.

[41] A. Fawzi, M. Balog, A. Huang, T. Hubert, B. Romera-Paredes, M. Barekatain, A. Novikov, F. J. R. Ruiz, J. Schrittwieser, G. Swirszcz, et al. Discovering faster matrix multiplication algorithms with reinforcement learning. *Nature*, 610(7930):47–53, 2022.

[42] W. Fleming and R. Rishel. *Deterministic and Stochastic Optimal Control*. Springer, 1975.

[43] M. Fortunato, M. G. Azar, B. Piot, J. Menick, M. Hessel, I. Osband, A. Graves, V. Mnih, R. Munos, D. Hassabis, O. Pietquin, C. Blundell, and S. Legg. Noisy networks for exploration. In *International Conference on Learning Representations*, 2018.

[44] V. François-Lavet, P. Henderson, R. Islam, M. G. Bellemare, J. Pineau, et al. An introduction to deep reinforcement learning. *Foundations and Trends® in Machine Learning*, 11(3-4):219–354, 2018.

[45] N. Gaby and X. Ye. Approximation of solution operators for high-dimensional pdes. *Journal of Computational Physics*, 524:113709, 2025.

[46] N. Gaby and X. Ye. Hamiltonian theory and computation of optimal probability density control in high dimensions. *arXiv preprint arXiv:2505.18362*, 2025.

[47] N. Gaby, X. Ye, and H. Zhou. Neural control of parametric solutions for high-dimensional evolution pdes. *SIAM Journal on Scientific Computing*, 46(2):C155–C185, 2024.

[48] I. Gat, T. Remez, N. Shaul, F. Kreuk, R. T. Chen, G. Synnaeve, Y. Adi, and Y. Lipman. Discrete flow matching. *Advances in Neural Information Processing Systems*, 37:133345–133385, 2024.

[49] G. Golub and C. van Van Loan. *Matrix Computations*. The Johns Hopkins University Press, 3rd edition, 1996.

[50] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio. Generative adversarial nets. In *Advances in neural information processing systems*, pages 2672–2680, 2014.

[51] W. Grathwohl, R. T. Chen, J. Bettencourt, I. Sutskever, and D. Duvenaud. Ffjord: Free-form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2019.

[52] K. Gregor, F. Besse, D. Jimenez Rezende, I. Danihelka, and D. Wierstra. Towards conceptual compression. In *Advances in neural information processing systems*, volume 29, 2016.

[53] K. Gregor, G. Papamakarios, F. Besse, L. Buesing, and T. Weber. Temporal difference variational auto-encoder. In *International Conference on Learning Representations*, 2019.

[54] J. Gui, Z. Sun, Y. Wen, D. Tao, and J. Ye. A review on generative adversarial networks: Algorithms, theory, and applications. *IEEE transactions on knowledge and data engineering*, 35(4):3313–3332, 2021.

[55] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. C. Courville. Improved training of wasserstein gans. In *Advances in Neural Information Processing Systems*, pages 5767–5777, 2017.

[56] I. Gulrajani, K. Kumar, F. Ahmed, A. A. Taiga, F. Visin, D. Vazquez, and A. Courville. Pixelvae: A latent variable model for natural images. In *International Conference on Learning Representations*, 2017.

[57] Y. Guo, H. Yuan, Y. Yang, M. Chen, and M. Wang. Gradient guidance for diffusion models: An optimization perspective. *Advances in Neural Information Processing Systems*, 37:90736–90770, 2024.

[58] T. Haarnoja, H. Tang, P. Abbeel, and S. Levine. Reinforcement learning with deep energy-based policies. In *International conference on machine learning*, pages 1352–1361. PMLR, 2017.

[59] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *International conference on machine learning*, pages 1861–1870. Pmlr, 2018.

[60] E. Heitz, L. Belcour, and T. Chambon. Iterative α-(de) blending: A minimalist deterministic diffusion model. In *ACM SIGGRAPH 2023 Conference Proceedings*, pages 1–8, 2023.

[61] M. Hessel, J. Modayil, H. Van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[62] G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *science*, 313(5786):504–507, 2006.

[63] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[64] J. Ho and T. Salimans. Classifier-free diffusion guidance. In *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*, 2021.

[65] L. M. Hocking. *Optimal control: an introduction to the theory with applications*. Oxford University Press, 1991.

[66] P. Holderrieth, M. Havasi, J. Yim, N. Shaul, I. Gat, T. Jaakkola, B. Karrer, R. T. Chen, and Y. Lipman. Generator matching: Generative modeling with arbitrary markov processes. In *International Conference on Learning Representations*, 2025.

[67] K. Hornik. Approximation capabilities of multilayer feedforward networks. *Neural networks*, 4(2):251–257, 1991.

[68] K. Hornik. Some new results on neural network approximation. *Neural networks*, 6(8):1069–1072, 1993.

[69] K. Hornik, M. Stinchcombe, and H. White. Multilayer feedforward networks are universal approximators. *Neural networks*, 2(5):359–366, 1989.

[70] K. Hornik, M. Stinchcombe, H. White, and P. Auer. Degree of approximation results for feedforward networks approximating unknown mappings and their derivatives. *Neural computation*, 6(6):1262–1275, 1994.

[71] G. Huguet, J. Vuckovic, K. Fatras, E. Thibodeau-Laufer, P. Lemos, R. Islam, C. Liu, J. Rector-Brooks, T. Akhound-Sadegh, M. Bronstein, et al. Sequence-augmented se (3)-flow matching for conditional protein generation. *Advances in neural information processing systems*, 37:33007–33036, 2024.

[72] C. Jin, P. Netrapalli, and M. Jordan. What is local optimality in nonconvex-nonconcave minimax optimization? In *International conference on machine learning*, pages 4880–4889. PMLR, 2020.

[73] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: An optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024.

[74] H. J. Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of statistical mechanics: theory and experiment*, 2005(11):P11011, 2005.

[75] T. Karras, M. Aittala, T. Aila, and S. Laine. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35:26565–26577, 2022.

[76] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In Y. Bengio and Y. LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.

[77] D. P. Kingma, T. Salimans, R. Jozefowicz, X. Chen, I. Sutskever, and M. Welling. Improved variational inference with inverse autoregressive flow. In *Advances in Neural Information Processing Systems*, pages 4743–4751, 2016.

[78] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *International Conference on Learning Representations*, 2014.

[79] D. P. Kingma, M. Welling, et al. An introduction to variational autoencoders. *Foundations and Trends® in Machine Learning*, 12(4):307–392, 2019.

[80] V. Konda and J. Tsitsiklis. Actor-critic algorithms. *Advances in neural information processing systems*, 12, 1999.

[81] A. B. L. Larsen, S. K. Sønderby, H. Larochelle, and O. Winther. Autoencoding beyond pixels using a learned similarity metric. In *International conference on machine learning*, pages 1558–1566. PMLR, 2016.

[82] M. Le, A. Vyas, B. Shi, B. Karrer, L. Sari, R. Moritz, M. Williamson, V. Manohar, Y. Adi, J. Mahadeokar, et al. Voicebox: Text-guided multilingual universal speech generation at scale. *Advances in neural information processing systems*, 36:14005–14034, 2023.

[83] M. Leshno, V. Y. Lin, A. Pinkus, and S. Schocken. Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural networks*, 6(6):861–867, 1993.

[84] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[85] Y. Lipman, R. T. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

[86] Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Chen, D. Lopez-Paz, H. Ben-Hamu, and I. Gat. Flow matching guide and code. *arXiv preprint arXiv:2412.06264*, 2024.

[87] G.-H. Liu, A. Vahdat, D.-A. Huang, E. Theodorou, W. Nie, and A. Anandkumar. I2sb: Image-to-image schrödinger bridge. In *International Conference on Machine Learning*, pages 22042–22062. PMLR, 2023.

[88] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, et al. Muon is scalable for llm training. *arXiv preprint arXiv:2502.16982*, 2025.

[89] X. Liu, C. Gong, et al. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *International Conference on Learning Representations*, 2023.

[90] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[91] L. Maaløe, C. K. Sønderby, S. K. Sønderby, and O. Winther. Auxiliary deep generative models. In *International conference on machine learning*, pages 1445–1453. PMLR, 2016.

[92] J. Macki and A. Strauss. *Introduction to optimal control theory*. Springer Science & Business Media, 2012.

[93] X. Mao. *Stochastic differential equations and applications*. Elsevier, 2007.

[94] X. Mao, Q. Li, H. Xie, R. Y. Lau, Z. Wang, and S. Paul Smolley. Least squares generative adversarial networks. In *Proceedings of the IEEE international conference on computer vision*, pages 2794–2802, 2017.

[95] L. Metz, B. Poole, D. Pfau, and J. Sohl-Dickstein. Unrolled generative adversarial networks. In *International Conference on Learning Representations*, 2017.

[96] H. N. Mhaskar. Neural networks for optimal approximation of smooth and analytic functions. *Neural computation*, 8(1):164–177, 1996.

[97] H. N. Mhaskar and C. A. Micchelli. Dimension-independent bounds on the degree of approximation by neural networks. *IBM Journal of Research and Development*, 38(3):277–284, 1994.

[98] H. N. Mhaskar and C. A. Micchelli. Degree of approximation by neural and translation networks with a single hidden layer. *Advances in applied mathematics*, 16(2):151–183, 1995.

[99] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[100] H. Montanelli and Q. Du. New error bounds for deep relu networks using sparse grids. *SIAM Journal on Mathematics of Data Science*, 1(1):78–92, 2019.

[101] K. Neklyudov, R. Brekelmans, D. Severo, and A. Makhzani. Action matching: Learning stochastic dynamics from samples. In *International conference on machine learning*, pages 25858–25889. PMLR, 2023.

[102] Y. Nesterov. *Introductory Lectures on Convex Optimization: A Basic Course*. Springer Publishing Company, Incorporated, 1 edition, 2014.

[103] A. Q. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *International conference on machine learning*, pages 8162–8171. PMLR, 2021.

[104] A. Q. Nichol, P. Dhariwal, A. Ramesh, P. Shyam, P. Mishkin, B. Mcgrew, I. Sutskever, and M. Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. In *International Conference on Machine Learning*, pages 16784–16804. PMLR, 2022.

[105] J. Nocedal and S. Wright. *Numerical optimization*. Springer Science & Business Media, 2006.

[106] B. Øksendal. *Stochastic Differential Equations: An Introduction with Applications*. Springer Science & Business Media, 2010.

[107] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.

[108] G. Peyré and M. Cuturi. Computational optimal transport. *Foundations and Trends® in Machine Learning*, 11(5-6):355–607, 2019.

[109] A. Pinkus. Approximation theory of the mlp model in neural networks. *Acta numerica*, 8:143–195, 1999.

[110] T. Poggio, H. Mhaskar, L. Rosasco, B. Miranda, and Q. Liao. Why and when can deep-but not shallow-networks avoid the curse of dimensionality: a review. *International Journal of Automation and Computing*, 14(5):503–519, 2017.

[111] A. Polyak, A. Zohar, A. Brown, A. Tjandra, A. Sinha, A. Lee, A. Vyas, B. Shi, C.-Y. Ma, C.-Y. Chuang, et al. Movie gen: A cast of media foundation models. *CoRR*, 2024.

[112] L. S. Pontryagin. *Mathematical theory of optimal processes*. Routledge, 2018.

[113] G.-J. Qi. Loss-sensitive generative adversarial networks on lipschitz densities. *International Journal of Computer Vision*, 128(5):1118–1140, 2020.

[114] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[115] B. Recht. A tour of reinforcement learning: The view from continuous control. *Annual Review of Control, Robotics, and Autonomous Systems*, 2(1):253–279, 2019.

[116] S. J. Reddi, S. Kale, and S. Kumar. On the convergence of adam and beyond. In *International Conference on Learning Representations*, 2018.

[117] D. Rezende and S. Mohamed. Variational inference with normalizing flows. In *International conference on machine learning*, pages 1530–1538. PMLR, 2015.

[118] H. Risken. *The Fokker-Planck equation: methods of solution and applications*. Springer Verlag, 1985.

[119] H. Robbins and S. Monro. A stochastic approximation method. *The annals of mathematical statistics*, pages 400–407, 1951.

[120] R. Rockafellar. *Convex Analysis*. Princeton University Press, 1996.

[121] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[122] O. Ronneberger, P. Fischer, and T. Brox. U-net: Convolutional networks for biomedical image segmentation. *CoRR*, abs/1505.04597, 2015.

[123] D. Rumelhart, G. Hinton, and R. Williams. Learning internal representations by error propagation. In *Neurocomputing: foundations of research*, pages 673–695. 1988.

[124] G. A. Rummery and M. Niranjan. *On-line Q-learning using connectionist systems*, volume 37. University of Cambridge, Department of Engineering Cambridge, UK, 1994.

[125] F. Santambrogio. Optimal transport for applied mathematicians. *Birkäuser, NY*, 55:58–63, 2015.

[126] R. Sargent. Optimal control. *Journal of Computational and Applied Mathematics*, 124(1-2):361–371, 2000.

[127] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016.

[128] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015.

[129] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.

[130] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[131] Y. Shi, V. De Bortoli, A. Campbell, and A. Doucet. Diffusion schrödinger bridge matching. *Advances in Neural Information Processing Systems*, 36:62183–62223, 2023.

[132] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[133] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *International conference on machine learning*, pages 387–395. Pmlr, 2014.

[134] S. P. Singh and R. S. Sutton. Reinforcement learning with replacing eligibility traces. *Machine learning*, 22(1):123–158, 1996.

[135] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr, 2015.

[136] K. Sohn, H. Lee, and X. Yan. Learning structured output representation using deep conditional generative models. *Advances in neural information processing systems*, 28, 2015.

[137] C. K. Sønderby, T. Raiko, L. Maaløe, S. K. Sønderby, and O. Winther. Ladder variational autoencoders. *Advances in neural information processing systems*, 29, 2016.

[138] J. Song, C. Meng, and S. Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

[139] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, and B. Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021.

[140] R. S. Sutton and A. G. Barto. *Reinforcement learning: An introduction*. Bradford Books, 2018.

[141] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems*, 12, 1999.

[142] C. Szepesvári. *Algorithms for reinforcement learning*. Springer nature, 2022.

[143] J. W. Thomas. *Numerical partial differential equations: conservation laws and elliptic equations*, volume 33. Springer Science & Business Media, 2013.

[144] J. W. Thomas. *Numerical partial differential equations: finite difference methods*, volume 22. Springer Science & Business Media, 2013.

[145] T. Tieleman and G. Hinton. Rmsprop: Divide the gradient by a running average of its recent magnitude. coursera: Neural networks for machine learning. *COURSERA Neural Networks Mach. Learn*, 17:6, 2012.

[146] E. Todorov. Linearly-solvable markov decision problems. *Advances in neural information processing systems*, 19, 2006.

[147] A. Tong, K. Fatras, N. Malkin, G. Huguet, Y. Zhang, J. Rector-Brooks, G. Wolf, and Y. Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

[148] L. Trefethen and D. Bau. *Numerical Linear Algebra*. SIAM: Society for Industrial and Applied Mathematics, 1997.

[149] H. Van Hasselt, A. Guez, and D. Silver. Deep reinforcement learning with double q-learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 30, 2016.

[150] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[151] C. Villani. *Optimal transport: old and new*, volume 338. Springer Science & Business Media, 2008.

[152] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, and Q. Miao. Deep reinforcement learning: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 35(4):5064–5078, 2022.

[153] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas. Dueling network architectures for deep reinforcement learning. In *International conference on machine learning*, pages 1995–2003. PMLR, 2016.

[154] Z. Wang, Q. She, and T. E. Ward. Generative adversarial networks in computer vision: A survey and taxonomy. *ACM Computing Surveys (CSUR)*, 54(2):1–38, 2021.

[155] C. J. Watkins. *Learning from delayed rewards*. PhD thesis, University of Cambridge, 1989.

[156] C. J. Watkins and P. Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.

[157] J. Wu, Z. Huang, J. Thoma, D. Acharya, and L. Van Gool. Wasserstein divergence for gans. In *Proceedings of the European conference on computer vision (ECCV)*, pages 653–668, 2018.

[158] D. Yarotsky. Error bounds for approximations with deep relu networks. *Neural Networks*, 94:103–114, 2017.

[159] Y. Zang, G. Bao, X. Ye, and H. Zhou. Weak adversarial networks for high-dimensional partial differential equations. *Journal of Computational Physics*, page 109409, 2020.

[160] J. Zhao, M. Mathieu, and Y. LeCun. Energy-based generative adversarial networks. In *International Conference on Learning Representations*, 2017.

索引

Actor-Critic 方法, 216
AdaGrad, 111
Adam, 112
AdamW, 113
Armijo 条件, 74
Automatic differentiation
    reverse mode, 66

Banach 不动点定理, 273
Banach 空间, 272
Bellman 方程, 186
Bellman 算子, 186, 187
BFGS 方法, 83, 100
Bregman 散度, 276

Cauchy 序列, 272
Control Hamiltonian, 128

Double DQNs 算法, 212
DPG 方法, 220
DQN 算法, 211

Epoch, 103
Euler–Lagrange 方程, 121

Fokker–Planck 方程, 239, 250, 290

Hamilton–Jacobi–Bellman 方程, 146
Hamiltonian, 124
    动力学, 123
Hamiltonian 动力学, 124
Hessian, 50

Itô 公式, 288
Itô 链式法则, 288
Itô 随机积分, 287

Jacobian, 50
Jensen–Shannon 散度, 281

Karush–Kuhn–Tucker
    KKT 乘子, 58
    KKT 条件, 58
    KKT 点, 58
    定理, 56
Kullback–Leibler 散度, 278

Lagrange
    乘子, 55
    条件, 87, 94
    法, 90
Lagrangian
    函数, 89, 120

方法, 89, 90
点, 86
Lipschitz 连续, 105

Markov 决策过程, 182
Markov 奖励过程, 184
Markov 过程, 182
Monte Carlo 方法, 204
Monte Carlo 积分, 40, 269
Muon, 114

Neural ODE 方法, 164
Newton 方向, 78
Newton 法, 78
Newton–CG 法, 82

Ornstein–Uhlenbeck 过程, 238

Pontryagin 极大值原理, 128, 134, 144
PPO 方法, 219

Q 函数, 192
Q 学习算法, 210

RMSProp, 112

SAC 方法, 222
SARSA 方法, 208
Sherman–Morrison–Woodbury 公式, 83

Variational method, 120

Wasserstein 距离, 282
Weiner 过程, 285
Wolfe 条件, 75

下降方向, 73
不动点迭代, 90, 189
临界点, 6
二次罚函数, 84
二次罚函数法, 84
价值
迭代, 203
价值改进, 203
仿射函数, 1, 114
优势函数, 219
伴随方程, 133
似然函数, 2
对数, 3

值函数, 146
状态, 184
状态–动作, 192
偏微分方程, 38, 172
充分下降条件, 74
充分条件
二阶, 52, 60
全局极小值点, 48
共轭梯度法, 81
内点, 46
凸优化, 47, 49
凸函数, 47
强, 78
凸集, 47
凹函数, 47
切空间, 53
判别器, 233
利用, 209
割线方程, 83
动作, 181
动量, 111
协态, 133

反向传播, 62
变分, 120
变分自编码器, 226
    编码器, 226
    解码器, 226
变分锥, 139
可行方向, 51
可行集, 46
向量场
    条件生成, 254
    生成, 253
回溯线搜索, 74
基本随机梯度方法, 100
增广 Lagrange 函数, 93, 96
增广 Lagrange 方法, 92, 94
多层感知机, 8
多重变分, 138
奇异值, 115
奇异值分解, 114
奖励, 181
    终端, 128
    运行, 128
对角缩放, 111
小批量随机方法
    二阶, 104
小批量随机梯度方法, 103
局部极小值点, 48, 51
层
    输入, 7
    输出, 7
    隐藏, 7
布朗运动, 285
常微分方程, 123
广义动量, 123
异常问题, 144
强化学习
    基于模型, 198
得分函数, 242
    条件, 242, 243
得分匹配, 241
必要条件
    一阶, 51
    二阶, 52, 55, 58

扩散, 237
    系数, 237
扩散模型, 237
扰动函数, 132, 139
批量梯度方法, 100
拟 Newton 法, 82, 83
损失 hansh, 46
损失函数, 38, 45, 98, 104, 259
    期望, 99
    经验, 99
探索, 209
控制, 120
    容许集, 120
收敛
    局部, 79
收敛速率
    二次, 80
    次线性, 77, 103, 117
    线性, 92, 102, 117, 189, 273
    超线性, 84
无约束优化, 46, 73
时序差分方法, 206
普适逼近定理, 21
智能体, 181
曲率条件, 74

最优传输, 282, 283
最优控制, 127
    固定终止时刻且终点自由, 128
最速下降法, 74
有约束优化, 46, 84
本性上确界, 12
条件数, 81
极大似然估计, 3
极小值点
    全局, 85, 94, 244
    局部, 54–56, 58, 60, 61, 79
梯度, 49
梯度下降法, 73
正则化, 113
正规点
    等式和不等式约束, 56
    等式约束, 53
步长, 73
活跃集, 56
流匹配, 253
深度学习, 9
激活函数, 8
    CELU, 27
    ELU, 26
    GELU, 27
    ReLU, 10, 26
    sigmoid, 24
    SwiGLU, 27
    swish, 27
    tanh, 26

熵, 275
    条件, 276
状态, 181
环境, 181
生成
    引导, 263
生成器, 233
病态, 78
目标函数, 38, 45, 46, 71, 99
确定性优化, 71
神经网络
    LSTM, 33
    Wasserstein 生成, 236
    前馈, 11
    卷积, 29
    图, 31
    弱对抗, 41
    循环, 32
    残差, 28
    浅度, 8
    深度, 8
    物理信息, 39, 40
    生成对抗, 233
空间
    Sobolev, 12
策略
    ϵ-贪婪, 209
    平稳, 182
    改进, 199
    改进定理, 195
    最优, 193, 197
    评估, 199
    贪婪, 191, 197
    迭代, 199
    非平稳, 182
策略梯度方法, 212
简单变分, 131
约束集, 46

线性空间, 271
赋范, 272
终止判据, 75
网络架构, 10

聚点, 85
胡信息, 284
自动微分, 62
Hessian, 71
Hessian-vector, 70
Jacobian, 67
前向扫描, 64
前向模式, 64, 68
反向扫描, 66
反向模式, 71
梯度, 66
计算图, 62
自治, 166, 173
自适应方法, 111
范数, 10, 12, 272
Frobenius, 115
解算子, 174
计算单元, 11
证据, 229
证据下界 (ELBO), 230
超参数, 77
距离, 272
连续方程, 247, 250, 290

逻辑函数, 4
重要性采样, 269
链式法则, 62
随机优化, 98
随机微分方程, 237, 284
非活跃集, 56
鞍点问题, 234, 264